# BayesFilter

Bayesian Estimation for Structural State-Space Models

HMC-Safe Filtering, Analytic Likelihood Derivatives,
and Industrial-Scale Inference

Chak Wong

October 6, 2026

# Contents


**I Scope and Interfaces** **1**

**1 Introduction** **2**
1.1 Central Design Thesis . . . 2
1.2 Provenance and Claim Discipline . . . 3
1.3 Reader Map . . . 3

**2 State-Space Model Contracts** **4**
2.1 Degenerate Transitions and Lag Stacks . . . 5
2.2 Structural Nonlinear Transition Contract . . . 5
2.3 Masks and Mixed Frequency . . . 6
2.4 Backend Classes . . . 6

**3 Posterior Targets and HMC Requirements** **7**
3.1 Finite Failure Policy . . . 7
3.2 Compilation and Differentiability . . . 8

**4 BayesFilter API Design** **9**
4.1 Core Objects . . . 9
4.2 Structural Model Protocol . . . 10
4.3 Filter Run Metadata . . . 10
4.4 Structural Filter Implementation Requirements . . . 11
4.5 Return Contract . . . 11
4.6 Backend Promotion . . . 12
4.7 Client Adapter Boundary . . . 12
4.8 Fixed-Center Curvature API . . . 13
4.9 Posterior-Local Location And Scale Initializer . . . 13

**II Linear Gaussian Filtering** **15**

**5 Prediction-Error Decomposition** **16**
5.1 Linear Gaussian Specialization . . . 16
5.2 Kalman Recursion . . . 16
5.3 Log Likelihood . . . 17
5.4 Initialization . . . 17
5.5 Reference Role . . . 18

**6 Stable Linear Filtering** **19**
6.1 Backend Ladder . . . 19
6.2 Covariance Form as Reference Algebra . . . 19
6.3 Solve Form . . . 20
6.4 Square-Root and Spectral Backends . . . 20
6.5 Backend Agreement Tests . . . 20
6.6 Diagnostics . . . 21

**7 Missing Data and Mixed Frequency** **22**
7.1 Selection Form . . . 22
7.2 Mixed-Frequency State Augmentation . . . 22
7.3 Implementation Policy . . . 23
7.4 Derivative Deferral . . . 23

**8 Large-Scale Linear Gaussian State-Space Models** **24**
8.1 Scale Targets . . . 24
8.2 Backend Registry . . . 24
8.3 Validation Ladder . . . 25
8.4 Stress Scenarios . . . 25
8.5 Readiness Criteria . . . 26
8.6 Derivative Consolidation Hypotheses . . . 26

**III Analytic Derivatives** **28**

**9 Kalman Score Recursions** **29**
9.1 Prediction Derivatives . . . 29
9.2 Innovation Derivatives . . . 29
9.3 Gain and Update Derivatives . . . 30
9.4 Masks and Time-Varying Observation Blocks . . . 30
9.5 Evidence Status . . . 31

**10 Kalman Hessian and Observed Information** **32**
10.1 Second-Order Prediction . . . 32
10.2 Second-Order Innovation Objects . . . 33
10.3 Solve-Form Hessian . . . 33
10.4 Second-Order Gain and Update . . . 33
10.5 Information Objects and Sign Convention . . . 34
10.6 Hessian-Ready Backend Contract . . . 34
10.7 Evidence Status . . . 34

**11 Structural Derivatives** **36**
11.1 Provider Map . . . 36
11.2 Parameter Transforms . . . 37
11.3 Initial Conditions . . . 37
11.4 Sparsity, Masks, and Zero Blocks . . . 38
11.5 Metadata and Shape Contract . . . 38
11.6 Provider Validation . . . 39

**12 QR and Cholesky Factor Derivatives** **40**
12.1 Reconstruction Contract . . . 40
12.2 Cholesky Factor Derivatives . . . 41
12.3 QR Factor Derivatives . . . 41
12.4 Square-Root Kalman Stacks . . . 42
12.5 Spectral Factors . . . 43
12.6 Backend Parity Gates . . . 43
12.7 Fixed-Rank Rectangular QR Charts . . . 44
12.8 Affine-Support Gaussian Likelihood and Derivative . . . 44

**13 Custom-Gradient Wrappers for HMC** **45**
13.1 Same-Scalar Contract . . . 45
13.2 Gradient Callback Contract . . . 46
13.3 Hessian and Observed-Information Path . . . 46
13.4 Static-Shape and Compilation Contract . . . 47
13.5 HMC Safety Labels . . . 47

**14 Derivative Validation** **48**
14.1 Validation Ladder . . . 48
14.2 What Is Being Compared . . . 48
14.3 Recommended Rungs . . . 49
14.4 Finite-Difference Tolerances . . . 50
14.5 Backend-Specific Gates . . . 50
14.6 Validation Artifact . . . 50
14.7 Audit Record . . . 51
14.8 Singular-Support SR-UKF Validation Matrix . . . 51

**IV Nonlinear Filtering** **52**

**15 Extended Kalman Filtering** **53**
15.1 Local Linearization . . . 53
15.2 Likelihood Status . . . 53
15.3 HMC Contract . . . 54

**16 Sigma-Point Filters** **55**
16.1 Moment-Matching Interface . . . 55
16.2 Conjugate Unscented Transform Rules . . . 55
16.3 Approximation Boundary . . . 57
16.4 Derivative and Compilation Requirements . . . 57
16.5 Evidence Ladder . . . 57

**17 Square-Root Sigma-Point Filters** **59**
17.1 Pruned Second-Order Models: Executable API . . . 59
17.2 Backend Contract . . . 60
17.3 Factor-Propagating SR-UKF Score Contract . . . 60
17.3.1 Sigma-Point Placement From A Declared Factor . . . 61
17.3.2 Moment Factors And Signed Square-Root Updates . . . 61
17.3.3 Innovation Factor, Likelihood, And Score . . . 62

17.3.4 Filtered State And Propagated Factor Derivatives . . . 63
17.3.5 Admission Boundary . . . 64
17.4 HMC-Relevant Risks . . . 64
17.5 When to Prefer Square-Root Forms . . . 64
17.6 Implemented Direct-QR Recursions . . . 64
17.7 CUT With Square-Root Backends . . . 65

**18 SVD Sigma-Point Filters 66**
18.1 What the Source Audit Established . . . 66
18.2 Spectral Differentiation Risk . . . 66
18.3 Analytic Score and Hessian Recursion . . . 67
18.3.1 Map and Moment Derivatives . . . 68
18.3.2 UKF Update Objects . . . 68
18.3.3 Likelihood Score and Hessian . . . 70
18.4 Structural Fixed-Support Score . . . 70
18.5 Spectral Factor Derivative Contract . . . 72
18.6 Principal-Square-Root UKF Backend . . . 73
18.7 Validation Targets for SVD Score and Hessian . . . 75
18.8 SVD-CUT Score and Hessian . . . 75
18.8.1 SVD-CUT Moment Derivatives . . . 76
18.8.2 SVD-CUT Likelihood Gradient and Hessian . . . 77
18.8.3 Point Count, GPU, and XLA . . . 78
18.9 Status Labels for SVD-HMC . . . 79
18.10BayesFilter Policy . . . 79
18.11Current Role of SVD in the Direct-Factor Architecture . . . 80

**19 Structural DSGE Dynamics 81**
19.1 The Structural Split . . . 81
19.1.1 Literature Map for This Chapter . . . 82
19.2 Why Linear Kalman Filtering Can Hide the Problem . . . 83
19.3 Why Nonlinear Filters Must Respect the Structural Map . . . 84
19.4 Reusable BayesFilter Structural Filtering Step . . . 85
19.5 Degenerate Transitions . . . 86
19.6 Standard UKF, Structural UKF, and the Predictive Law . . . 86
19.6.1 The Original Unscented-Transform Pattern . . . 86
19.6.2 Standard Additive-Noise UKF Algorithm . . . 88
19.6.3 Structural UKF Algorithm . . . 89
19.6.4 UKF in Sequential-Monte-Carlo Kernel Notation . . . 90
19.6.5 Why These Algorithms Differ . . . 92
19.6.6 Why the Sigma-Point Variable Uses Pre-Transition Uncertainty . 93
19.7 Worked Example A: Nonlinear Structural Transition . . . 95
19.7.1 A Natural Objection: Does $\phi = 0$ Collapse the Example? . . . 98
19.7.2 Numerical Illustration . . . 98
19.8 Degenerate Linear Transition Example . . . 101
19.8.1 Worked structural derivation for the degenerate linear-transition case 102
19.8.2 Worked Degenerate Linear-Transition Example . . . 104
19.8.3 What Is Similar to the Structural-Deterministic Case? . . . 107
19.8.4 What Is Different from the Earlier Nonlinear-State Example? . . . 107

19.9 Structural Correctness Boundary . . . 107
19.10 Degeneracy and SVD . . . 108
19.11 Pruned DSGE and Adapter Implications . . . 115
19.12 Source-Project Lesson . . . 116
19.13 Validation Gates and Final Policy Rule . . . 116
19.13.1 Common Misunderstandings . . . 117

**V Particle Foundations and Proposal Transport 118**

**20 Particle-Filter Foundations 119**
20.1 Objects, assumptions, and claim-status vocabulary . . . 119
20.2 DPF-block notation and claim-status index . . . 120
20.3 Nonlinear state-space model and filtering recursion . . . 121
20.4 Empirical filtering measures . . . 122
20.5 Sequential importance sampling . . . 123
20.6 Sequential importance resampling and the bootstrap filter . . . 123
20.7 The bootstrap particle-filter likelihood estimator . . . 124
20.8 Differentiability boundary . . . 126
20.9 Degeneracy and effective sample size . . . 127
20.10 Baseline audit table . . . 128
20.11 What this chapter does and does not claim . . . 128

**21 Particle-Flow Foundations 129**
21.1 Objects, notation, and source roles . . . 129
21.2 From discrete reweighting to continuous transport . . . 130
21.3 Homotopy path between predictive and filtering laws . . . 131
21.4 Continuity equation and transport PDE . . . 132
21.5 EDH under Gaussian closure . . . 133
21.6 Linear-Gaussian recovery as the exact special case . . . 135
21.7 LEDH and local linearization . . . 135
21.8 Stiffness and discretization . . . 137
21.9 Exactness and approximation ledger . . . 138
21.10 What this chapter does and does not claim . . . 139

**22 Particle-Flow Particle Filters and Proposal Correction 140**
22.1 Objects, assumptions, and source roles . . . 140
22.2 Why proposal correction is needed . . . 141
22.3 Change of variables under the flow map . . . 141
22.4 Proposal-corrected PF-PF weights . . . 142
22.5 EDH/PF and LEDH/PF . . . 142
22.5.1 EDH/PF . . . 143
22.5.2 LEDH/PF . . . 143
22.6 Li–Coates Algorithm 1 as an implementation contract . . . 143
22.6.1 Why the OT block begins here . . . 144
22.6.2 What the later OT chapters inherit from PF-PF . . . 145
22.7 Marginalizing the ancestor . . . 145
22.8 The analytical total score . . . 147
22.9 The complete implemented recursion . . . 147

22.10Evidence from the three models . . . 148
22.11Jacobian determinant and log-determinant evolution . . . 150
22.12What the correction restores, and what it does not . . . 151
22.13Implementation audit obligations . . . 152
22.14Why this chapter matters for the later HMC discussion . . . 152
22.15What this chapter does and does not claim . . . 152

**VI Differentiable Resampling and Transport 153**

**23 Soft Differentiable Resampling 154**
23.1 What problem this chapter solves . . . 154
23.2 Running two-particle discontinuity cell . . . 154
23.3 Classical resampling bottleneck . . . 155
23.4 Soft resampling rule . . . 155
23.5 Worked two-particle calculation . . . 155
23.6 Algorithm: soft differentiable resampling . . . 156
23.7 Filtering-loop insertion point . . . 156
23.8 Verification contract . . . 157
23.9 What this chapter does and does not claim . . . 157

**24 Deterministic OT Equal-Weighting Baseline 158**
24.1 What problem this chapter solves . . . 158
24.2 Running three-particle cell: deterministic OT version . . . 159
24.3 Objects and notation . . . 160
24.4 Weighted cloud, equal-weight cloud, and the coupling set . . . 160
24.4.1 Why the filter needs a transport object, not just a scalar . . . 161
24.5 The unregularized deterministic OT baseline . . . 161
24.6 Barycentric projection: from coupling to equal-weight cloud . . . 162
24.7 Algorithm: deterministic OT equal-weighting . . . 162
24.8 Filtering-loop insertion point . . . 163
24.9 Verification and failure interpretation . . . 163
24.10What this chapter does and does not claim . . . 163

**25 Entropic OT, Sinkhorn, and Barycentric Projection 164**
25.1 What changes when entropy is added . . . 164
25.2 Running three-particle cell with entropic smoothing . . . 165
25.3 Entropic OT teacher object . . . 165
25.3.1 Exact teacher versus exact engineering route . . . 166
25.3.2 The current BayesFilter bottleneck . . . 166
25.4 Why the optimizer factorizes . . . 167
25.4.1 Why the balancing equations are Sinkhorn updates . . . 167
25.5 Algorithm: finite Sinkhorn equal-weighting . . . 168
25.5.1 Exact online and streaming Sinkhorn as the reference lane . . . 169
25.5.2 Sparse, screened, stabilized, and accelerated Sinkhorn variants . . . 169
25.6 Worked balancing pass on the running cell . . . 170
25.7 Barycentric projection as the output map . . . 171
25.8 Remedy options for barycentric covariance loss . . . 172
25.9 Why the current reset can accumulate bias . . . 182

25.10 A non-fused cubature residual Contract E candidate . . . 182
25.11 Generalized unscented transformation candidate . . . 184
25.12 Differentiation contracts . . . 188
25.13 Filtering-loop insertion point . . . 189
25.14 Verification contract . . . 189
25.15 What this chapter does and does not claim . . . 190
25.16 A complete higher-moment Contract E candidate . . . 190
25.16.1 Finite filtering, OT, and Contract E stages . . . 191
25.16.2 Finite higher-moment correction . . . 192
25.16.3 Why the residual design can obstruct kurtosis repair . . . 195
25.16.4 The pairwise skewness and kurtosis correction remains active . . . 196
25.16.5 Why a bounded coordinate map is not a stability proof . . . 197
25.16.6 A guarded finite fit with a protected fallback . . . 198
25.16.7 Complete algorithm and score statement . . . 200
25.16.8 What the actual likelihoods and scores show . . . 202
25.16.9 The first nonlinear comparisons . . . 204
25.17 A Zhao–Cui squared-TT moment teacher for Contract E . . . 207
25.17.1 Parallel TT filtering recursion . . . 207
25.17.2 Exact fixed-ALS fit and its forward derivative . . . 209
25.17.3 Exact observable contractions for a fixed squared TT . . . 210
25.17.4 Low-rank contractions for whitened higher moments . . . 211
25.17.5 Composition with Contract E . . . 212
25.17.6 Likelihood, score, and derivative statements . . . 212
25.17.7 Complete algorithm and complexity boundary . . . 213

**26 Custom Gradients for LEDH-PFPF-OT 216**
26.1 The scalar whose derivative is wanted . . . 216
26.2 Forward decomposition . . . 217
26.3 Canonical row quotient and Contract E composition . . . 218
26.4 Transport orientation . . . 219
26.5 VJP of the barycentric product . . . 219
26.6 A normalized transport matrix . . . 220
26.7 VJP of the quadratic cost . . . 221
26.8 Softmin and Sinkhorn adjoints . . . 221
26.9 Why a custom VJP is memory efficient . . . 223
26.10 Same-cloud geometry reuse in the finite JVP . . . 223
26.11 Embedding the OT VJP in the filtering loop . . . 225
26.12 A proposed score-aware teacher-projection extension . . . 225
26.12.1 The filtering derivative that must survive reset . . . 226
26.12.2 Parent-by-innovation teacher . . . 227
26.12.3 Positive compression as constrained feasibility . . . 228
26.12.4 What the projection proves about tangents and score . . . 229
26.12.5 Progressive score information and compact score-aware features . . . 230
26.12.6 Algorithmic form and complexity . . . 240
26.12.7 Concrete two-dimensional LGSSM witness . . . 241
26.12.8 Mathematical and engineering boundary . . . 243
26.13 Model-agnostic comparison of finite-filter scores . . . 243
26.13.1 Global score-vector norm diagnostics . . . 244

26.13.2 Average predictive-score OPG metric . . . 244
26.14 Implementation contract . . . 246
26.15 Relation to forward-mode JVP . . . 247
26.16 What this chapter does and does not claim . . . 247

**VII Learned Transport Extensions 249**

**27 Retained-Teacher Neural OT Warm Starts 250**
27.1 What problem this chapter solves . . . 250
27.2 Running warm-start cell . . . 251
27.3 What the teacher computes . . . 253
27.4 Algorithm: retained-teacher warm-started Sinkhorn . . . 254
27.5 Evidence contract for retained-teacher learning . . . 255
27.5.1 Permutation symmetry as a design check . . . 255
27.5.2 Residual hierarchy . . . 255
27.5.3 Where broader retained-teacher families fit . . . 257
27.5.4 Exact online and GPU Sinkhorn as the reference lane . . . 257
27.5.5 Nystrom kernel approximations . . . 258
27.5.6 Positive-feature Sinkhorn . . . 259
27.5.7 Direct low-rank coupling approximations . . . 260
27.5.8 Stochastic and minibatch cautions . . . 261
27.5.9 How the broader retained-teacher approximations fit here . . . 262
27.6 Filtering-loop insertion point . . . 262
27.7 Verification and non-claims . . . 263

**28 ICNN, Brenier Maps, and Monge-Gap Direct Map Learning 264**
28.1 What changes when the map itself is learned? . . . 264
28.2 Running direct-map cell . . . 265
28.3 Direct-map lane: object changed, evidence changed . . . 266
28.4 Brenier viewpoint . . . 267
28.5 ICNN/Brenier route . . . 268
28.6 Monge-gap route . . . 270
28.7 Structured variants and boundaries . . . 272
28.8 Why this is not the default first route . . . 273
28.8.1 Direct-map families as a different scalable-OT answer . . . 273
28.9 Verification and non-claims . . . 274

**29 Dynamic Geodesic and Operator Learners, and the Filtering Target Contract 275**
29.1 What changes when the learned object becomes richer? . . . 275
29.2 Running path-versus-map example . . . 276
29.3 Richer-object audit: endpoint, path, scalar, gradient . . . 278
29.4 Dynamic geodesic learners . . . 279
29.5 Operator learners . . . 280
29.6 Target-changing boundary families . . . 281
29.6.1 Sliced, subspace, and localization families as target-changing lanes 282
29.7 Same-scalar rule revisited . . . 283
29.8 How to audit a richer learned object . . . 283

29.9 Relation to the full HMC target chapter . . . 284
29.10 What this chapter does and does not claim . . . 285

**VIII HMC Target Interpretation, Verification, and Filter Choice 286**

**30 DPF HMC Target Correctness and Structural-Model Suitability 287**
30.1 Objects in the HMC target contract . . . 287
30.2 Target-status taxonomy . . . 289
30.3 Rung-by-rung target analysis . . . 290
30.3.1 EDH and LEDH flow proposals . . . 290
30.3.2 PF-PF proposal correction . . . 290
30.3.3 Soft differentiable resampling . . . 291
30.3.4 OT and entropic OT resampling . . . 291
30.3.5 Learned or amortized OT . . . 292
30.4 Structured target maps . . . 292
30.5 Pseudo-marginal and surrogate distinctions . . . 293
30.6 Why nonlinear DSGE models sharpen the issue . . . 294
30.7 Why MacroFinance models sharpen the issue . . . 294
30.8 Relation to surrogate-HMC acceleration . . . 295
30.9 Evidence ladder for stronger external language . . . 296
30.10 Recommendation . . . 296
30.11 What this chapter does and does not claim . . . 297

**31 DPF Literature-to-Debugging Verification Contract 298**
31.1 Diagnostic order . . . 298
31.2 Proposal log-probability autodiff topology . . . 299
31.3 Same-object deterministic tie-out contract . . . 301
31.4 Equation-to-test matrix . . . 302
31.5 Minimal controlled examples . . . 304
31.6 Diagnostic specification . . . 305
31.7 Source and implementation map . . . 306
31.8 Scalable OT reuse-risk and validation doctrine . . . 306
31.8.1 Code availability and reuse risk are prefilters, not verdicts . . . 307
31.8.2 Validation ladder before promotion . . . 308
31.8.3 Lane-specific narrow interpretations . . . 308
31.8.4 Recommended research order . . . 309
31.9 Promotion thresholds . . . 309
31.9.1 Closing scalable-OT synthesis . . . 310
31.10 Boundary of this contract . . . 310

**32 Filter Choice 311**
32.1 Decision Register . . . 311
32.2 Recommended Order . . . 312
32.3 Industrial Default . . . 312

**IX High-Dimensional Nonlinear Filtering 313**

**33 High-Dimensional Filtering Foundations 314**
33.1 The Filtering Problem In Its Least Forgiving Form . . . 314
33.2 Exact Discrete-Time Recursion And Likelihood . . . 315
33.2.1 Actual-SV Single-Target Derivation . . . 316
33.2.2 Actual-SV SR-UKF Augmented-Noise Adapter . . . 318
33.3 Analytical Likelihood-Gradient And Filtering Sensitivities . . . 321
33.4 Compact Sensitivity Score For Finite LEDH–PFPF–OT . . . 323
33.5 Zakai, Kushner–Stratonovich, And DMZ Equations . . . 326
33.6 Pathwise Robust DMZ And Memoryless Computation . . . 326
33.7 Approximate Targets And The HMC Boundary . . . 327
33.8 Why Dimension Breaks Filters . . . 328
33.9 Defects Passed To Synthesis . . . 328
33.10 Methodological Boundary . . . 329
33.11 Bibliographic Limits . . . 329

**34 Deterministic Gaussian Projection and Point-Rule Foundations 330**
34.1 Introduction And Scope . . . 330
34.1.1 Problem Setting And Deterministic Gaussian Lanes . . . 331
34.1.2 The Deterministic Gaussian Lane In One Page . . . 331
34.1.3 Approximation Hierarchy And Reading Guide . . . 331
34.2 Exact Filtering And Gaussian Projection . . . 332
34.2.1 Object Map And Deterministic Gaussian Conventions . . . 332
34.3 Deterministic Point-Rule Families . . . 334
34.4 Exports To Sparse-Grid Specialization And Later Chapters . . . 334

**35 Gaussian And High-Order Filters 336**
35.1 Purpose And Scope . . . 336
35.2 Low-Dimensional Construction Of FixedSGQF . . . 336
35.2.1 One nonlinear Gaussian moment as the basic quadrature problem 337
35.2.2 Tensor-product growth in two dimensions . . . 337
35.2.3 One-Dimensional Rules And Their Tensor Products . . . 337
35.2.4 Three-dimensional preview: why sparse grids help . . . 338
35.3 Sparse-Grid Rule Reconstructed In Source Order . . . 339
35.3.1 From Formula To Point Cloud . . . 339
35.3.2 Exactness, Point Count, And The UKF Relation . . . 341
35.4 What The Fixed Cloud Exports Forward . . . 341
35.5 Methodological Boundary And Non-Claims . . . 341

**36 Fixed-Cloud Filtering, Same-Branch SGQF, and Lane-Specific Validation 343**
36.1 Purpose And Scope . . . 343
36.2 Filtering Value Path On A Fixed Cloud . . . 343
36.2.1 Prediction Stage . . . 344
36.2.2 Observation Stage . . . 344
36.3 Worked One-Step Example With A Numeric Oracle . . . 345
36.4 General Analytical Derivative Framework For FixedSGQF . . . 348
36.4.1 Derivative Ledger . . . 349

36.4.2 Square-Root Branch . . . 349
36.4.3 Prediction Sensitivities . . . 349
36.4.4 Observation Sensitivities And Innovation Score . . . 350
36.5 Same-Branch Fixed-Cloud Scalar Contract . . . 350
36.6 Lane-Specific Validation Burden . . . 352
36.7 Implementation Contract For FixedSGQF . . . 353
36.7.1 Input, Output, Invariant, And Failure Contract . . . 354
36.7.2 One-Step Value-Path Contract . . . 355
36.7.3 Default Numerical Constants . . . 355
36.8 End-To-End Mathematical Algorithm . . . 356
36.9 Exports To Later Chapters . . . 356

**37 Low-Rank Density Filters, KR Maps, and Retained Objects 358**
37.1 Orientation And Scope . . . 358
37.2 Running Example And Notation . . . 358
37.3 Exact Filtering Target And Recursive Bottleneck . . . 359
37.4 Tensor-Train Approximation Toolkit . . . 359
37.4.1 Rank-Count Plausibility . . . 360
37.5 Coordinate Systems, Conditional Maps, And Retained Objects . . . 360
37.5.1 One Observation Step Through All Coordinate Systems . . . 361
37.6 TT Sequential Learning And Conditional KR Transports . . . 362
37.7 What This Chapter Exports Forward . . . 362
37.8 Tensor-Network Kalman And Square-Root Caution . . . 362
37.9 Complexity, Diagnostics, And Exports . . . 362
37.10 Methodological Boundary And Sources . . . 363

**38 Squared-TT Recursion and Fixed-Branch Likelihood Construction 364**
38.1 Purpose And Scope . . . 364
38.2 Squared-TT Filtering Object And Recursive Closure . . . 365
38.3 What Must Be Saved For The Next Time Step . . . 365
38.4 Fixed-Branch Likelihood Construction . . . 366
38.4.1 What A Core Object Looks Like . . . 366
38.5 Consolidated Fixed Least-Squares Formulation . . . 368
38.5.1 One Concrete Branch, Fully Instantiated . . . 369
38.5.2 Square-Root Fitting Object Versus Density Audit Object . . . 371
38.6 Initialization And Sweep Order . . . 372
38.6.1 Forward Core Update . . . 372
38.6.2 Derivative Core Update . . . 372
38.6.3 Alternating-Sweep Recomputations . . . 373
38.6.4 Stopping Rule And Failure Exits . . . 374
38.7 One Observation Step Through The Stored Objects . . . 374
38.8 Exports To The Same-Scalar Derivative Chapter . . . 375

**39 Fixed-Branch Approximate Likelihoods and Same-Scalar Gradients 376**
39.1 Why Fixed-Branch Discipline Is Shared Across The Two Lanes . . . 376
39.2 A Canonical Branch Record . . . 376
39.3 Fixed-Branch Approximate Likelihoods Across The Three Operational Layers . . . 377
39.4 Finite-Difference Contract And Branch-Validity Logic . . . 378

39.5 Value Path, Derivative Path, And What Is Actually Differentiated . . . . 379
39.6 Approximate Targets, Exact-Target Corrections, And HMC Admissibility 382
39.7 Exports To Validation And Promotion Chapters . . . . . 383
39.8 Implementation Boundary . . . . . 383

**40 Validation, Defect Calculus, and Method Promotion 384**
40.1 Validation Architecture And Benchmark Roles . . . . . 384
40.2 Finite-Difference Verification As The First Shared Gate . . . . . 385
40.3 Why The Synthesis Must Start With Defects . . . . . 385
40.4 Lane-Specific Validation Ladders Before Promotion . . . . . 386
40.4.1 FixedSGQF ladder . . . . . 387
40.4.2 Retained-object TT/KR ladder . . . . . 387
40.5 Mathematical Veto Ledger Before Interpretation . . . . . 388
40.6 Particle Collapse . . . . . 389
40.7 Row-Coverage Collapse . . . . . 390
40.7.1 The mechanism . . . . . 390
40.7.2 Why importance rows make it worse . . . . . 390
40.7.3 The repair: bridged coordinates, not better rows . . . . . 391
40.7.4 Ledger consequences . . . . . 391
40.8 Chained-Domain Truncation . . . . . 392
40.8.1 Why the domains must nest . . . . . 392
40.8.2 The truncated mass is geometric in dimension . . . . . 392
40.8.3 Truncation is a bias floor, not a resolution problem . . . . . 393
40.8.4 Ledger consequences . . . . . 394
40.9 Reference-Domain Selection . . . . . 394
40.9.1 Candidates and properties . . . . . 394
40.9.2 The reference algebra is closed-form in all three candidates . . . . 395
40.9.3 Domain validity eliminates the incumbent . . . . . 396
40.9.4 The defensive floor: bias cost and domination . . . . . 397
40.9.5 Approximation efficiency separates the survivors . . . . . 398
40.9.6 Selection, decisions, and what remains open . . . . . 399
40.10Validation Benchmarks And Promotion Logic . . . . . 400
40.11Defect Calculus And Promotion Rules . . . . . 401

**X HMC, Geometry, and Diagnostics 402**

**41 HMC for State-Space Likelihoods 403**
41.1 Target Contract . . . . . 403
41.2 Implementation Position . . . . . 403
41.3 Promoted Fixed-Trajectory Tuning Algorithm . . . . . 404
41.4 Public And Private Handoffs . . . . . 405
41.5 Diagnostic Use of NUTS . . . . . 405

**42 HMC Tuning Interfaces 406**
42.1 One candidate-set procedure . . . . . 406
42.2 Preparation and immutable geometry . . . . . 407
42.3 Broad search, repairs, and further evidence . . . . . 412
42.4 Acceptance, numerical health, and posterior diagnostics . . . . . 414

42.5 A frozen nonlinear transport . . . 415
42.6 Typed deterministic proposal-field mechanics . . . 417
42.7 Persistence, resource limits, and replay . . . 420
42.8 What the validation experiments establish . . . 427
42.9 Historical readers and complete interface inventory . . . 435

**43 Mass Matrices and Local Geometry 443**
43.1 Sources of Curvature . . . 443
43.2 Regularization Policy . . . 443
43.3 Geometry-Scaled Tuning Budgets . . . 443
43.4 Timing And Progress . . . 444
43.5 Fixed-Center Score Curvature . . . 445
43.6 One- and Two-Factor Covariance Geometry . . . 445
43.7 Cloud Size, Reuse, and Escalation . . . 446
43.8 Stability, Consensus, and Fallback . . . 446

**44 Boundary Handling and Safe Gradients 448**
44.1 Finite Invalid Returns . . . 448
44.2 Boundary Checklist . . . 448
44.3 Rank, Chart, and Affine-Support Boundaries . . . 449

**45 Occasionally Binding Constraints, Gradient Discontinuities, and a Failed Hypothesis 450**
45.1 Purpose and scope . . . 450
45.2 The failed hypothesis . . . 451
45.2.1 Narrative form . . . 451
45.2.2 Why it seemed plausible . . . 451
45.3 The rigorous rebuttal . . . 452
45.3.1 Softplus smoothness and curvature bound . . . 452
45.3.2 UKF composition: no gradient jump . . . 453
45.3.3 The variance collapse mechanism is backwards . . . 454
45.3.4 Innovation covariance bound: no unbounded inverse . . . 454
45.3.5 Sigma-point limit: convergence to linearization . . . 455
45.3.6 Leapfrog stability for smooth stiff potentials . . . 455
45.4 Specific errors in the failed hypothesis . . . 456
45.4.1 Error 1: Confusing stiffness with discontinuity . . . 456
45.4.2 Error 2: Inverting the variance collapse mechanism . . . 456
45.4.3 Error 3: Unbounded inverse claim without the $S \succeq R$ bound . . . 456
45.4.4 Error 4: Misunderstanding the sigma-point limit . . . 457
45.4.5 Error 5: Convention collision in scaling statements . . . 457
45.4.6 Error 6: Taxonomy confusion . . . 457
45.5 What the executed evidence actually shows . . . 458
45.6 The research plan that followed the wrong hypothesis . . . 458
45.6.1 Why the plan was abandoned . . . 459
45.7 Lessons for research discipline . . . 459
45.7.1 Lesson 1: Derive, don't narrate . . . 459
45.7.2 Lesson 2: Distinguish stiffness from discontinuity . . . 460
45.7.3 Lesson 3: Check mechanisms in both directions . . . 460
45.7.4 Lesson 4: Anchor claims to code, not to plans . . . 460

45.7.5 Lesson 5: Adversarial review catches what friendly review misses . 461
45.7.6 Lesson 6: "Not checked" is not "probably fine" . . . . . 461
45.8 What remains correct . . . . . 462
45.8.1 Stiffness is real . . . . . 462
45.8.2 Hard kinks are different . . . . . 462
45.8.3 Filter choice matters for HMC targets . . . . . 463
45.8.4 Funnel geometry is a real bottleneck . . . . . 463
45.9 Archival status and relation to other chapters . . . . . 464
45.9.1 This chapter in the BayesFilter monograph . . . . . 464
45.9.2 Relation to the retracted survey section . . . . . 464
45.9.3 Relation to the abandoned research plan . . . . . 464
45.9.4 Relation to executed work . . . . . 465
45.10Final summary . . . . . 465

**46 XLA and JIT Compilation** **467**
46.1 Full-Pipeline Requirement . . . . . 467
46.2 Device and Shape Constraints . . . . . 467
46.3 Explicit CPU Multiprocess Cloud Evaluation . . . . . 468

**47 Diagnostics and Failure Taxonomy** **469**
47.1 Filter and Derivative Diagnostics . . . . . 469
47.2 Sampler Diagnostics . . . . . 469
47.3 Warm-up and posterior precision . . . . . 469
47.4 Failure Taxonomy . . . . . 475
47.5 Curvature Diagnostic Roles . . . . . 475

**48 Transport Maps and Surrogate Geometry** **476**
48.1 Transported Targets . . . . . 476
48.2 NeuTra Position . . . . . 476
48.3 Surrogate Targets . . . . . 477
48.4 Diagnostics . . . . . 477
48.5 BayesFilter Policy . . . . . 478

**49 Neural Transport Hamiltonian Monte Carlo** **479**
49.1 Motivation: geometry is often the HMC bottleneck . . . . . 479
49.2 Hamiltonian Monte Carlo from first principles . . . . . 480
49.3 Exact neural change of variables . . . . . 481
49.4 The historical BayesFilter dense IAF . . . . . 481
49.5 Reverse-KL training and its gradient . . . . . 483
49.6 Weak nonlinear learning in q20 . . . . . 483
49.6.1 What failed, and what has already been checked . . . . . 484
49.6.2 Why reverse-KL stationarity need not remove the score residual . 485
49.6.3 Mechanism one: separate global scale from conditional deformation 485
49.6.4 Mechanism two: an exact alternative reverse-KL gradient . . . . . 486
49.6.5 Mechanism three: direct scalar deformation with an inexpensive log determinant . . . . . 487
49.6.6 How the three mechanisms will be distinguished . . . . . 488
49.6.7 A shared implementation of the proposed mechanisms . . . . . 489
49.7 Geometry induced in source space . . . . . 489

49.8 Complete BayesFilter algorithm . . . 490
49.9 Selecting among frozen transports . . . 491
49.10 Convergence and scientific diagnostics . . . 491
49.11 Filter-defined posterior targets used below . . . 493
49.12 Models in the common truth-tail experiment . . . 494
49.12.1 Four-dimensional exact LGSSM . . . 494
49.12.2 Predator–prey with UKF and SGQF likelihoods . . . 495
49.12.3 Parameterized spatial SIR with SGQF likelihood . . . 495
49.12.4 Structural deterministic dynamics with UKF likelihood . . . 496
49.13 Complete common experimental results . . . 496
49.14 Additional historical model tests . . . 496
49.15 Interpretation, failure modes, and nonclaims . . . 500

**50 Modern Importance Sampling** **502**
50.1 What importance weights measure . . . 502
50.2 Why dimension and missing modes matter . . . 503
50.3 Weighted kernels and adaptive mixtures . . . 504
50.4 Defensive and adaptive proposals . . . 505
50.5 Annealing and sequential importance sampling . . . 506
50.6 Annealed flow transport and continual training . . . 507
50.7 FAB: minimizing the importance-weight second moment . . . 508
50.7.1 Replay corrections and their limits . . . 509
50.7.2 Tail integrability is stronger than full support . . . 510
50.8 Combining FAB with IAF and NeuTra . . . 510
50.9 Stein methods and stochastic transport . . . 511
50.10 Evaluation and implementation . . . 513

**51 Neural-Force HMC** **514**
51.1 The computational question . . . 514
51.2 Phase-space Metropolis correction . . . 515
51.3 A sufficient learned-force construction . . . 516
51.4 Limits of the broader learned-Hamiltonian argument . . . 518
51.5 True-energy and delayed-acceptance kernels . . . 519
51.6 The filtering boundary . . . 521
51.6.1 Deterministic exact and approximate filters . . . 521
51.6.2 Fixed randomness and relaxed filters . . . 521
51.6.3 Pseudo-marginal likelihoods . . . 522
51.6.4 Filtering target taxonomy . . . 523
51.7 Training a useful position potential . . . 523
51.8 Algorithm and implementation contract . . . 524
51.9 Tests that answer the mathematical question . . . 525
51.10 Scientific evaluation ladder . . . 526
51.11 Historical evidence and its proper interpretation . . . 527
51.12 BayesFilter corrected-kernel experiments . . . 527
51.13 Conclusions and nonclaims . . . 529

**XI Industrial Applications and Case Studies 530**

**52 LGSSM Validation Ladder 531**
52.1 Evidence Already Available . . . 531
52.2 Validation Rungs . . . 531
52.3 Interpretation . . . 532

**53 Nonlinear SSM Validation Ladder 533**
53.1 Controlled Models . . . 533
53.1.1 Model A: Affine Gaussian Structural Oracle . . . 533
53.1.2 Model B: Nonlinear Accumulation . . . 533
53.1.3 Model C: Nonlinear Growth Benchmark . . . 534
53.1.4 Deferred Models . . . 534
53.2 Current V1 Filter Evidence . . . 535
53.3 Required Outputs . . . 536
53.4 Stop Rules . . . 536

**54 Neural State-Space Model Applications 537**
54.1 A Stochastic State-Space LSTM . . . 537
54.1.1 Parameter chart . . . 538
54.1.2 Structural SVD-UKF likelihood . . . 538
54.2 The Scalar Four-Parameter Application . . . 539
54.3 Why Parameter Agreement Is Not the Only Estimand . . . 539
54.4 Posterior-Predictive Law . . . 540
54.4.1 Horizon-specific moments . . . 540
54.5 MGFs and Stable Distribution Features . . . 541
54.6 What the Weights Mean . . . 541
54.7 Dependence-Aware Estimation of Predictive Features . . . 542
54.7.1 Why a fixed batch length is not enough . . . 543
54.8 A Proper-Score Definition of Practical Equivalence . . . 544
54.9 An Equivalence Gate, Not a Failure-to-Reject Gate . . . 546
54.10 Ordinary HMC and Exact-Corrected NeuTra-HMC . . . 547
54.11 What the NeuTra Experiments Established . . . 547
54.12 Three Evidence Ledgers . . . 550
54.13 Model Adequacy Is a Separate Question . . . 551
54.14 Claim Boundary . . . 551
54.15 Direct Visual Validation and the Complexity Ladder . . . 551
54.15.1 Successive state and memory complexity . . . 552
54.15.2 Matrix-free differentiation for nonlinear filters . . . 556
54.15.3 Execution and test evidence . . . 560

**55 NK SVD Case Study 562**
55.1 What Passed . . . 562
55.2 What Remains Blocked . . . 562
55.3 Lesson for BayesFilter . . . 563

**56 CIP/AFNS Case Study** **564**
56.1 Filtering Lessons . . . 564
56.2 BayesFilter Boundary . . . 564
56.3 Readiness Use . . . 564
56.4 DZ5 Fixed-Center Curvature Evidence . . . 565

**57 NAWM-Scale Design Target** **566**
57.1 Why NAWM Changes the Standard . . . 566
57.2 Design Requirements . . . 566
57.3 Hypothesis . . . 567

**58 Production Checklist** **568**
58.1 Target and Source . . . 568
58.2 Derivative and Filter . . . 568
58.3 Sampler and JIT . . . 569
58.4 Claim Discipline . . . 569
58.5 Fixed-Center Geometry Checklist . . . 569

**Appendices** **572**

**A Notation and Shape Conventions** **572**
A.1 Derivative Shapes . . . 572
A.2 Label Namespace . . . 573

**B Matrix Calculus Identities** **574**

**C Factor Derivative Proofs** **575**

**D MathDevMCP Workflows** **576**

**E ResearchAssistant Workflows** **577**
E.1 Workspace Policy . . . 577
E.2 Ingestion Workflow . . . 577
E.3 Claim Gate . . . 578
E.4 Remaining Source-Support Blockers . . . 578

**F Source Map** **579**

**G Experiment Templates** **580**
G.1 Filter Reference Template . . . 580
G.2 HMC Smoke Template . . . 580
G.3 Medium Recovery Template . . . 581
G.4 Strict Convergence Template . . . 581

# Part I

# Scope and Interfaces

# Chapter 1

# Introduction

BayesFilter is a monograph and software-design project for Bayesian estimation of structural state-space models. The word structural is important. The latent state is not merely an anonymous vector passed to a numerical routine: its coordinates may be stochastic shocks, deterministic lags, accounting identities, endogenous completions, auxiliary variables, or measurement-only summaries. A filtering backend that ignores those roles can evaluate the wrong likelihood, produce the wrong gradient, or make a sampler explore states the model never generates.

The project was extracted from DSGE, macro-finance, and analytic Kalman derivative work, but its purpose is broader and cleaner than any one source project. BayesFilter owns the reusable contracts for state-space likelihoods, derivatives, diagnostics, and compiled Bayesian sampling. The economics of NK, EZ, SGU, CIP, AFNS, and future NAWM-scale applications remain downstream case studies that stress those contracts.

The organizing premise is that filtering is not a small implementation detail inside Bayesian state-space inference. For Hamiltonian Monte Carlo, the filter is part of the posterior target: it supplies the log likelihood, the gradient path, the invalid-parameter behavior, and much of the geometry seen by the sampler. A filtering backend that is adequate for point likelihood evaluation may still be unsuitable for HMC if it has hidden non-finite branches, unstable spectral derivatives, dynamic shapes, structural state violations, or opaque compilation behavior.

This monograph separates four layers:

(i) the structural state-space model contract;

(ii) the likelihood and derivative backend;

(iii) the HMC target and diagnostic policy;

(iv) application case studies that stress the generic contracts.

The separation lets the linear Gaussian and analytic derivative spine be audited before nonlinear sigma-point filters, particle filters, transport maps, or DSGE-specific experiments are used as evidence.

## 1.1 Central Design Thesis

BayesFilter follows one design thesis:

> model projects own structural maps, and BayesFilter owns filters.

For an autoregression, the structural map may be a companion-form lag shift. For a macro-finance model, it may be an affine no-arbitrage state recursion with masked yields. For a DSGE model, it may be a perturbation solution that separates exogenous shocks from endogenous predetermined variables. These maps belong to the model project. The common filtering machinery, derivative validation, diagnostics, and HMC target discipline belong to BayesFilter.

This separation prevents each application project from rediscovering the same numerical failure modes. It also prevents a successful linear Gaussian implementation from being misused as a nonlinear structural filter simply because both expose a state vector and covariance matrix.

## 1.2 Provenance and Claim Discipline

The source documents for this project are read-only inputs. BayesFilter consolidates them into one notation system and one production-readiness policy, but it does not treat old prose as automatically authoritative. Every substantive claim should be traceable to one of four sources: a local source chapter or reset memo listed in `docs/source_map.yml`, a reviewed literature artifact, a code/test reference, or an explicitly recorded project decision such as the TensorFlow Probability NUTS benchmark.

The most important negative rule is equally simple: no backend is described as HMC-safe merely because it returns a finite likelihood on a smoke test. HMC safety requires a target contract, a derivative policy, finite failure handling, static-shape or compilation discipline where compilation is claimed, and diagnostics that can reveal when the sampler is exploring a numerically fragile region.

## 1.3 Reader Map

The book is written from the stable core outward. Part I fixes notation and interfaces, including structural state partitions. Part II develops the exact linear Gaussian likelihood because it is the regression oracle for more complicated filters. Part III records analytic derivative and custom-gradient policy before nonlinear filters are promoted into HMC experiments. Part IV discusses nonlinear filters, deterministic completions, and spectral risks with those derivative requirements already in place. Part V builds classical particle-filter foundations and the particle-flow proposal literature. Part VI develops differentiable resampling and entropic transport, and Part VII studies learned transport extensions. Part VIII interprets these filters as HMC targets, gives a debugging crosswalk, and records the filter-choice register. Part IX develops the high-dimensional nonlinear filtering lanes, Part X states the HMC, geometry, compilation, and diagnostic contracts, and Part XI closes with the industrial case studies so they can be judged against the generic BayesFilter checklist rather than driving the definitions.

# Chapter 2

# State-Space Model Contracts

BayesFilter treats a state-space model as a contract between a structural model and a filtering backend. The structural model may be a DSGE perturbation solver, an affine term-structure model, a macro-finance system, or a synthetic validation problem. The filter should not need to know that origin; it should receive typed maps with stable shapes and declared derivative policies.

**Definition 2.1** (BayesFilter state-space contract)**.** *For parameter vector $\theta \in \Theta \subseteq \mathbb{R}^{d_\theta}$, a BayesFilter state-space contract supplies latent states $x_t \in \mathbb{R}^{n_x}$, observations $y_t \in \mathbb{R}^{n_y}$, transition law*

$$p_\theta(x_{t+1} \mid x_t),$$

*observation law*

$$p_\theta(y_t \mid x_t),$$

*initial law $p_\theta(x_0)$, and an observation mask or selection rule when the available components of $y_t$ vary over time. A linear Gaussian specialization supplies matrices and vectors*

$$c_\theta, \quad F_\theta, \quad Q_\theta, \quad a_\theta, \quad H_\theta, \quad R_\theta,$$

*with dimensions fixed by the model instance.*

**Definition 2.2** (Structural state partition)**.** *A structural state partition is model metadata that decomposes the latent state coordinates into declared roles. The minimal BayesFilter partition records*

$$x_t = \mathrm{pack}(s_t, d_t, a_t),$$

*where $s_t$ is the stochastic block receiving current innovations directly or through a declared stochastic transition, $d_t$ is a deterministic-completion block, and $a_t$ is an optional auxiliary block used for lags, bookkeeping, or backend-specific summaries. The partition also records the innovation dimension and the pack/unpack order.*

The partition is part of the model contract. It should not be inferred only from a covariance matrix. A zero row of a shock-impact matrix is a useful diagnostic, but it is not a complete timing declaration. Conversely, a small positive nugget inserted for numerical regularization does not turn a deterministic identity into a structural source of randomness.

**Assumption 2.1** (Shape and value contract)**.** *For a fixed model instance, tensor ranks and leading dimensions are static throughout a compiled likelihood evaluation. Parameter-dependent covariance objects are symmetric positive semidefinite or positive definite as*

*required by the backend, and invalid parameter values are reported through an explicit finite-failure policy rather than accidental linear-algebra exceptions.*

The contract separates model maps from filter maps. A model map builds state-space objects from $\theta$. A filter map consumes those objects and the data. A derivative provider supplies first or second derivatives of the state-space objects, or declares that a backend relies on autodiff through primitive operations. This distinction is essential for large models: the derivative of a structural solver and the derivative of a Kalman or sigma-point recursion are different mathematical layers.

**Remark 2.1** (Structural deterministic dynamics are part of the contract)**.** *For DSGE models, the state-space contract must not hide the distinction between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states and current shocks. A collapsed linear transition may be sufficient for an exact Kalman filter, but nonlinear sigma-point and particle filters need the structural transition map. This is a release-gating issue; see Chapter 19.*

## 2.1 Degenerate Transitions and Lag Stacks

Structural deterministic coordinates are not special to DSGE models. An autoregression of order two already has the same issue. One companion-form representation is

$$m_t = \phi_1 m_{t-1} + \phi_2 \ell_{t-1} + \sigma \varepsilon_t, \tag{2.1}$$
$$\ell_t = m_{t-1}, \tag{2.2}$$
$$x_t = (m_t, \ell_t). \tag{2.3}$$

The lag coordinate $\ell_t$ is deterministic conditional on the previous state. In an exact linear Kalman backend, the corresponding transition covariance is singular:

$$Q = \begin{bmatrix} \sigma^2 & 0 \\ 0 & 0 \end{bmatrix}.$$

That singularity is structural information, not an implementation failure. For nonlinear sigma-point or particle filters, quadrature or simulation should be applied to the innovation or stochastic block and then lifted through the lag-shift map.

The same contract covers accumulated sums, stock-flow identities, mixed-frequency aggregation states, and deterministic no-arbitrage recursions. BayesFilter should therefore test degenerate transitions directly rather than only through full-rank synthetic examples.

## 2.2 Structural Nonlinear Transition Contract

For a nonlinear structural model, a backend should know whether it integrates over innovations, stochastic states, or the full latent state. A convenient contract is

$$s_t = T_s(s_{t-1}, \varepsilon_t; \theta), \tag{2.4}$$
$$d_t = T_d(d_{t-1}, s_{t-1}, s_t; \theta), \tag{2.5}$$
$$x_t = \mathrm{pack}(s_t, d_t, a_t), \tag{2.6}$$
$$y_t \sim p_\theta(y_t \mid x_t). \tag{2.7}$$

When a mixed structural model is evaluated by a full-state Gaussian approximation, the result must be labeled as an approximation. The default nonlinear filter for a mixed structural model should preserve deterministic identities pointwise for propagated sigma points or particles.

**Assumption 2.2** (Approximation labeling)**.** *If a backend integrates over a larger stochastic space than the declared model transition, or injects artificial noise into deterministic coordinates, the run metadata must record an approximation label. The label is part of the likelihood provenance and must be visible to HMC diagnostics and case-study reports.*

## 2.3 Masks and Mixed Frequency

A sparse panel should be represented by a deterministic mask or selection operator, not by changing the model definition at each time. At time $t$, the filter may work with the observed subvector $y_{t,\mathcal{O}_t}$ and the corresponding selected observation map. The public contract must state whether an all-missing time step performs only prediction, whether ragged panels are batched by pattern, and whether the implementation keeps static shapes for JIT compilation.

## 2.4 Backend Classes

BayesFilter distinguishes the following backend roles:

- exact linear Gaussian likelihood backends;
- square-root or solve-form variants of exact linear Gaussian backends;
- approximate nonlinear Gaussian filters such as EKF, UKF, CKF, and SVD-factored sigma-point filters;
- particle-filter or simulation-based likelihood estimators;
- diagnostic or surrogate backends that do not define the final HMC target without an explicit correction policy.

The same model can be evaluated by more than one backend, but the monograph must state which backend defines the posterior target and which backends are only diagnostic approximations.

# Chapter 3

# Posterior Targets and HMC Requirements

HMC samples an unconstrained or transformed parameter vector by integrating Hamiltonian dynamics for a log target. In BayesFilter, that log target usually contains a filtering likelihood. The target contract must therefore be stated before discussing sampler tuning.

**Definition 3.1** (Filtering posterior target)**.** *Let $u \in \mathbb{R}^{d_u}$ denote the coordinates used by the sampler and let $\theta = T(u)$ be the model parameter transform. A filtering posterior target has the form*

$$\log \pi(u \mid y_{1:T}) = \log p(T(u)) + \ell(T(u); y_{1:T}) + \log |\det DT(u)| + \textit{constant},$$

*where $\ell$ is the log likelihood returned by the chosen filtering backend.*

For HMC, the value alone is not enough. The backend and transform jointly define the gradient seen by the sampler. If a custom gradient is used, the exact HMC target remains the same only when the Metropolis correction evaluates that same scalar value. A deterministic nearby force field can still leave the invariant law exact under exact endpoint Metropolis correction, but then the approximation changes leapfrog efficiency rather than the target itself. If the acceptance step also uses a surrogate value, or if the dynamics are left unadjusted, the target changes unless the sampler is explicitly corrected for that surrogate. In particular, if the filtering backend exposes likelihood derivatives in model parameter coordinates $\theta$, the HMC wrapper must still construct the full sampler-coordinate target in $u$-space by adding the prior and transform Jacobian terms from Definition 3.1.

## 3.1 Finite Failure Policy

Invalid parameters are ordinary events during HMC warmup and exploration. A BayesFilter target must decide how to handle them. BayesFilter must state whether an invalid region is handled by the exact support-rejection contract or by a declared modified target. Under the exact contract, the value path returns $-\infty$ (equivalently $U_F = +\infty$) and the proposal is rejected. Under the modified-target contract, the value path returns a deterministic fallback scalar $c$ together with the exact derivative of that same returned scalar, so the sampled law is proportional to $\exp(\widetilde{\ell})$ where $\widetilde{\ell} = \ell$ on the valid region and $\widetilde{\ell} = c$ on the declared invalid region. If $c$ is constant on that region, the corresponding gradient must be zero. This finite fallback can be useful for compiled finite-valued

control flow, but it is not the same target as support rejection, and on an unbounded invalid region it can be improper. A linear-algebra failure that aborts the process is not an HMC-compatible failure mode.

The policy must be explicit for:

- parameter transforms and prior support;
- stationarity or determinacy restrictions;
- covariance positive-definiteness checks;
- failed factorizations or solve residuals;
- non-finite likelihood or gradient values.

## 3.2 Compilation and Differentiability

When a backend is advertised as JIT or XLA compatible, the claim refers to the complete value-and-gradient path needed by the sampler. Partial tracing of an outer function is not enough if the filter, derivative provider, or failure branch executes outside the compiled region.

The derivative policy should be one of:

- analytic score or Hessian recursion;
- custom gradient wrapping a certified analytic derivative;
- autodiff through smooth primitive operations;
- stopped-gradient approximation used only with an explicit correction or diagnostic label;
- not HMC-safe.

# Chapter 4

# BayesFilter API Design

The monograph is not an API reference, but its mathematical contracts should be precise enough that an implementation can be audited against them. The API shape below is therefore a documentation contract rather than a final Python interface.

## 4.1 Core Objects

A BayesFilter implementation should expose five separable objects:

- a parameter transform from sampler coordinates $u$ to model parameters $\theta$;
- a structural state partition describing stochastic, deterministic, and auxiliary coordinates;
- a model provider that builds state-space objects from $\theta$;
- a filter backend that returns log likelihood and diagnostics;
- a derivative provider or autodiff policy that returns the score and, when needed, curvature information.

For a linear Gaussian model, the model provider returns

$$(c, F, Q, a, H, R, m_0, P_0)$$

and optionally their first and second derivatives with respect to $\theta$. For a nonlinear model, it returns transition and observation maps together with the sigma-point, linearization, or simulation policy required by the selected backend.

## 4.2 Structural Model Protocol

The exact software interface may change, but implementations should be auditable against the following documentation contract:

| | |
|---|---|
| `partition` : | state names, role indices, innovation dimension, |
| `initial_law` : | $(m_0, P_0)$ or a sampling rule, |
| `innovation_law` : | $\Sigma_\varepsilon$ or an innovation sampler, |
| `unpack_state`, `pack_state` : | $x \leftrightarrow (s, d, a)$, |
| `propagate_stochastic` : | $(s_{t-1}, \varepsilon_t, \theta) \mapsto s_t$, |
| `complete_deterministic` : | $(d_{t-1}, s_{t-1}, s_t, \theta) \mapsto d_t$, |
| `observe` : | $x_t \mapsto p_\theta(y_t \mid x_t)$. |

Here $s_t$ denotes the declared stochastic block, $d_t$ the deterministic completion block, and $a_t$ any auxiliary or backend-specific block. A model with no deterministic block can set $d_t$ empty; a linear Gaussian companion form can still expose the partition even if the exact Kalman backend consumes only the collapsed matrices.

DSGE, MacroFinance, and other application projects should implement this protocol through adapters. BayesFilter should not require those projects to copy filter internals, and those projects should not define private near-duplicates of BayesFilter filters.

## 4.3 Filter Run Metadata

Every backend should return metadata that records what target was actually evaluated. At minimum, diagnostics should include:

- backend name and version;
- structural partition summary;
- integration space: innovation, stochastic-state, or full-state;
- deterministic-completion policy;
- approximation label, if any;
- derivative policy;
- compiled/eager execution status when relevant;
- finite-failure or warning codes.

This metadata prevents a collapsed approximation, a diagnostic surrogate, or a partially compiled run from being misreported as a structural HMC target.

For mixed structural models, the metadata must also be strong enough to audit deterministic completion. A sigma-point, cubature, or particle backend should therefore expose:

- whether deterministic identities were checked pointwise;
- the maximum deterministic identity residual when such a check is available;

- whether a full-state integration path was used for a mixed model;
- the approximation label attached to any enlarged stochastic space;
- whether artificial jitter was only a numerical linear-algebra device or part of the model transition being evaluated.

These fields are not cosmetic. They are the evidence that the backend followed the `structural_filter_step` contract from Chapter 19. If they are missing, the safest interpretation is that the result has not yet earned a structural filtering claim.

## 4.4 Structural Filter Implementation Requirements

A BayesFilter structural filter implementation should satisfy the following requirements before a client adapter uses it as an HMC likelihood:

(i) **Fail-closed partition validation:** mixed models must declare stochastic, deterministic-completion, and auxiliary coordinates before a non-full-state backend runs.

(ii) **Visible integration space:** the result must state whether the backend integrated over innovations, stochastic states, or the full state.

(iii) **Deterministic identity diagnostics:** test fixtures with known identities, such as AR(p) lag shifts or $k_t - \phi k_{t-1} - \gamma m_t^2 = 0$, must be checked pointwise for propagated points or particles.

(iv) **Approximation-label propagation:** if a mixed model is evaluated by a full-state Gaussian approximation, the label must survive into the likelihood result, HMC target wrapper, case-study tables, and reset memo.

(v) **Reference-oracle tests:** affine Gaussian cases must recover the exact Kalman value, and low-dimensional nonlinear cases should be compared with dense quadrature, high-particle simulation, or another explicit reference.

(vi) **Derivative and compilation gates:** value-side structural correctness is not a gradient guarantee. SVD/eigen derivatives, static shapes, eager/compiled parity, and HMC diagnostics need separate gates.

DSGE and MacroFinance adapters should supply model timing and economics. BayesFilter should supply this generic validation and filtering machinery. That boundary keeps the common structural-filtering rule in one place while still allowing client projects to own their own solution methods and priors.

## 4.5 Return Contract

The minimum likelihood return is

$$(\ell,\ d),$$

where $\ell$ is a scalar log likelihood and $d$ is a diagnostics record. A gradient-capable backend returns

$$(\ell,\ \nabla_\theta \ell,\ d),$$

and a curvature backend returns

$$(\ell,\ \nabla_\theta \ell,\ \nabla^2_{\theta\theta} \ell,\ d).$$

Diagnostics should include enough information to decide whether the result is a valid target evaluation, a finite invalid-region return, or a numerical warning that should influence sampler adaptation or backend selection.

## 4.6 Backend Promotion

BayesFilter should promote a backend through stages:

(i) value-only likelihood evaluation;

(ii) gradient validation on small problems;

(iii) compiled value-and-gradient execution;

(iv) stress tests under masks, boundaries, and near-singular covariances;

(v) HMC diagnostics on controlled targets;

(vi) application case studies.

No backend should skip directly from value-only likelihood tests to industrial HMC use.

## 4.7 Client Adapter Boundary

The adapter boundary is intentionally strict. A client project owns:

- structural equations and timing;
- parameter transformations and priors;
- model-specific observation equations;
- perturbation, no-arbitrage, or equilibrium solution objects;
- application-level HMC targets and case-study data.

BayesFilter owns:

- generic filter recursions;
- structural partition validation;
- square-root and SVD numerical backends;
- derivative validation ladders;
- finite-failure and diagnostic conventions;
- backend promotion gates.

The boundary is a maintenance device. It lets MacroFinance reuse analytic Kalman derivatives, DSGE reuse structural nonlinear filtering, and future large models reuse the same validation ladder without forking the numerical core.

## 4.8 Fixed-Center Curvature API

BayesFilter distinguishes two initialization contracts that must not be collapsed into one. The legacy `estimate_sequential_map_covariance` contract localizes a MAP candidate and therefore requires its standardized maximum absolute score to pass the configured terminal cap, a fresh terminal dense fit to be usable, and its projection burden to pass the configured cap. Those checks do not certify a global MAP, but they are part of that API's stated semantics.

The fixed-center contract instead accepts a reviewed center $c$ and disjoint exact-score clouds. It does not move $c$ or impose a stationarity gate. Its public result records the center and explanatory score; every dense or structured replicate; raw precision matrices, eigenvalues, inertia, projection burden, and selection-holdout error; pairwise stability metrics; and the final audit-cloud error.

Training and selection-holdout rows are separate for every replicate. The API rejects shared-memory aliases and exact copied offset rows across training, selection, and audit partitions. Disjoint generator seeds remain required lineage because this array-level check cannot establish stochastic independence. A third fresh audit cloud is evaluated only after family, target, and shrinkage weight selection; it can veto but cannot retune the result. The status `eligible_for_exact_hmc_canary` requires complete caller-reviewed stability caps and a passing audit cloud. Missing caps produce `diagnostic_only`; a failed audit produces `audit_holdout_rejected`; and no admissible branch produces `geometry_readiness_blocked`.

Only the eligible state can construct a `PrecomputedMassArtifact`. Its position role is `diagnostic_center`, never `map`. The implemented entry points are `fit_fixed_center_curvature`, `compare_precision_geometry`, and `consensus_shrunk_precision`. Exact HMC mechanics, retained-chain diagnostics, and posterior convergence remain separate gates.

## 4.9 Posterior-Local Location And Scale Initializer

The separately named `initialize_posterior_local_location_scale` API initializes a trainable transport without asserting that its center is a MAP. Starting from one truth-blind point and positive coordinate units $D$, its locator searches the smooth bounded chart

$$z_j = r\tanh(u_j/r), \qquad \theta = c_0 + Dz,$$

and applies the exact chain rule to the original target score. This is an optimizer chart, not a posterior change of variables, so it adds no density Jacobian.

Every exact finite eligible value can update the shared incumbent. At least two independently seeded constrained low-rank SPD fits are used only to nominate local movements. A winning cloud or surrogate row invalidates that fit for covariance handoff and forces a fresh centered fit. The final precision comes instead from two fixed-center training/selection replicates and a disjoint audit cloud. If the final cloud moves the incumbent after the bounded refit budget, the API returns location-only evidence rather than a covariance fitted at the wrong point.

For the terminal standardized precision $K$,

$$\Sigma_\theta = DK^{-1}D.$$

The returned center and $\frac{1}{2}\log\operatorname{diag}(\Sigma_\theta)$ may initialize trainable output shift and marginal log-scale parameters. The full covariance is retained as diagnostic geometry and is not installed as a fixed affine transport. Optional scalar and batch eligibility callbacks must agree with value/score finiteness; this fail-closed contract prevents a finite HMC-rejection sentinel with zero score from masquerading as a stationary initializer. The result remains single-start, posterior-local, and explicitly makes no MAP, mode-coverage, training, or HMC-readiness claim.

# Part II

# Linear Gaussian Filtering

# Chapter 5

# Prediction-Error Decomposition

The linear Gaussian model is the first BayesFilter reference target because its likelihood is exact and its derivative recursions can be audited against code and automatic-differentiation tests. This chapter records the value-only likelihood spine; score and Hessian recursions are deferred to Chapters 9–10.

## 5.1 Linear Gaussian Specialization

Under the contract in Definition 2.1, the linear Gaussian specialization is

$$x_{t+1} = c(\theta) + F(\theta)x_t + u_{t+1}, \qquad u_{t+1} \sim \mathcal{N}(0, Q(\theta)), \tag{5.1}$$

$$y_t = a(\theta) + H(\theta)x_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R(\theta)). \tag{5.2}$$

For the exact likelihood backend, $Q(\theta) \succeq 0$ and $R(\theta) \succ 0$ are required at every accepted target evaluation. The initial moments are denoted $m_{0|0}$ and $P_{0|0}$.

The semidefinite allowance for $Q$ is intentional. Companion-form lag stacks, mixed-frequency accumulators, and structural deterministic coordinates often produce process covariances with zero rows or deficient rank. This is not a reason to abandon the exact linear Gaussian likelihood. It is a requirement that the backend distinguish structural degeneracy from accidental numerical failure.

**Assumption 5.1** (Innovation regularity)**.** *For every time point contributing an observation, the selected innovation covariance $S_t$ defined below is positive definite on the observed coordinates. If the model or data mask makes $S_t$ singular, the likelihood backend must return a named finite-failure diagnostic or use a separately documented diffuse/singular-observation treatment.*

## 5.2 Kalman Recursion

Let

$$m_{t|s} = \mathbb{E}[x_t \mid y_{1:s}], \qquad P_{t|s} = \mathrm{Var}(x_t \mid y_{1:s}).$$

The prediction step is

$$m_{t|t-1} = c + F m_{t-1|t-1}, \tag{5.3}$$

$$P_{t|t-1} = F P_{t-1|t-1} F^\top + Q. \tag{5.4}$$

The innovation and innovation covariance are

$$v_t = y_t - a - Hm_{t|t-1}, \tag{5.5}$$
$$S_t = HP_{t|t-1}H^\top + R. \tag{5.6}$$

When $S_t \succ 0$, the Kalman gain and filtered moments are

$$K_t = P_{t|t-1}H^\top S_t^{-1}, \tag{5.7}$$
$$m_{t|t} = m_{t|t-1} + K_t v_t, \tag{5.8}$$
$$P_{t|t} = (I - K_t H)P_{t|t-1}. \tag{5.9}$$

The covariance update in (5.9) is the compact textbook form. Production implementations may use the Joseph form

$$P_{t|t} = (I - K_t H)P_{t|t-1}(I - K_t H)^\top + K_t R K_t^\top$$

or may update an algebraically equivalent factor rather than materializing a full covariance matrix. These are numerical representations of the same exact linear Gaussian likelihood, not different statistical models.

## 5.3 Log Likelihood

The prediction-error decomposition gives

$$\ell(\theta) = -\frac{1}{2}\sum_{t=1}^{T}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_{y,t}\log(2\pi)\right], \tag{5.10}$$

where $n_{y,t}$ is the number of observed components at time $t$. In a dense panel, $n_{y,t} = n_y$. In a masked panel, the same formula applies after selecting the observed components and the corresponding rows and columns of the observation objects.

If no components are observed at a time point, the measurement update and likelihood contribution are skipped and the filter carries the prediction forward. This convention treats missingness as part of the observation contract rather than as a change in the latent transition law.

Equation (5.10) is the exact likelihood for (5.1)–(5.2). It is also the reference value against which approximate nonlinear filters and alternative linear algebra backends should be tested whenever a controlled linear Gaussian case is available.

## 5.4 Initialization

Initial moments are part of the model contract. If $m_{0|0}$ and $P_{0|0}$ are fixed by design, derivative initial conditions in later chapters are zero. If $P_{0|0}$ is the stationary covariance, then its derivatives are governed by the derivative Lyapunov equations for the transition system. BayesFilter must record which initialization policy defines the target, because changing it changes the likelihood.

## 5.5 Reference Role

The prediction-error likelihood is the value oracle for the rest of the monograph. Later chapters may evaluate it through covariance, solve-form, Cholesky, QR, or SVD linear-algebra primitives, but they must agree on (5.10) within documented tolerances. Nonlinear filters should also recover this value when their model is specialized to a linear Gaussian case.

Derivative chapters differentiate this same scalar likelihood. A custom gradient, analytic score recursion, or Hessian recursion is valid for HMC only if it corresponds to the value defined here, plus the prior and parameter transform terms in Chapter 3.

# Chapter 6

# Stable Linear Filtering

The covariance-form equations in Chapter 5 are algebraically correct, but they are not always the best production primitive. Large panels, nearly collinear observations, low measurement noise, and repeated covariance updates can turn small roundoff differences into large differences in precision matrices and Hessian contractions.

## 6.1 Backend Ladder

BayesFilter uses a backend ladder rather than a single Kalman implementation:

(i) covariance-form recursion for clarity and small reference problems;

(ii) solve-form likelihood evaluation using factored innovation covariances;

(iii) Cholesky square-root propagation where positive pivots are reliable;

(iv) QR square-root propagation when reconstruction and solve diagnostics are needed;

(v) SVD or spectral fallbacks only with explicit diagnostic labels.

The ladder is a production policy, not a mathematical change to the likelihood. All exact linear Gaussian backends must agree with (5.10) within documented tolerances.

## 6.2 Covariance Form as Reference Algebra

The covariance-form recursion is the clearest way to state the likelihood, but it is not automatically the safest way to compute it. In finite precision, the compact covariance update can lose symmetry or positive semidefiniteness after many time steps. Implementations should therefore symmetrize only as a documented numerical projection, not as a way to hide model invalidity.

For small reference tests, covariance form is useful because it exposes the same moments that appear in the mathematical recursion. For production likelihoods, BayesFilter should prefer solve and factor forms that avoid explicit inverses and provide residual diagnostics.

## 6.3 Solve Form

Let $S_t = L_t L_t^\top$ and define $z_t = L_t^{-1} v_t$. Then

$$\log\det S_t = 2\sum_k \log L_{t,kk}, \qquad v_t^\top S_t^{-1} v_t = z_t^\top z_t.$$

The likelihood can therefore be evaluated with triangular solves and log diagonal pivots rather than by explicitly forming $S_t^{-1}$. This is the preferred reference form for implementation because the same solves appear in the analytic score and Hessian recursions.

The solve-form contract is:

- factor the selected innovation covariance;
- solve for the whitened innovation;
- accumulate log determinant and quadratic form from the factor and solve result;
- record factor pivots and solve residuals.

The backend should not form an explicit inverse except in small diagnostic tests where the numerical error is itself being measured.

## 6.4 Square-Root and Spectral Backends

Cholesky square-root filters are efficient when positive pivots are reliable. QR square-root filters are useful when the implementation needs stable stack updates, reconstruction checks, and a clear path to derivative-factor validation. SVD or eigenspectral factorizations are useful for rank-deficient or nearly rank-deficient linear Gaussian problems, but they should be reported as spectral backend choices with their own diagnostics.

This distinction matters because the word SVD appears in two different parts of the monograph. An SVD-factored exact linear Gaussian backend is still evaluating (5.10). A nonlinear SVD sigma-point filter is an approximate nonlinear Gaussian filter whose value and gradient policy require separate tests. Success of the former does not certify the latter.

## 6.5 Backend Agreement Tests

Every exact linear Gaussian backend should be tested against at least one reference backend on controlled models. A minimal agreement test records:

- log-likelihood difference;
- maximum filtered-mean difference;
- maximum filtered-covariance or reconstructed-factor difference;
- minimum pivot or singular value;
- solve residual norm;
- whether the process covariance is full rank or structurally singular.

For singular $Q$ examples, the test should assert that the likelihood is finite and that the structural degeneracy is reported as model structure or backend telemetry, not as an unexplained numerical exception.

## 6.6 Diagnostics

Stable filtering backends should return diagnostics, not only a scalar likelihood. The MacroFinance large-scale backend ladder tests track:

- minimum factor pivots;
- reconstruction errors for prediction, innovation, and filtered factors;
- solve residual norms;
- finite log likelihood, gradient, and Hessian values;
- backend deltas against the solve-form reference.

Those tests classify SVD fallback triggers as no trigger, candidate trigger, or confirmed trigger. BayesFilter should preserve that caution: a spectral fallback is a diagnostic event, not evidence that the HMC gradient path is automatically safe.

The diagnostic record should travel with the likelihood into HMC reports. A finite log likelihood with repeated tiny pivots or frequent fallback triggers is not the same engineering result as a finite log likelihood with comfortable factor margins. Both may be mathematically valid evaluations, but they imply different sampler risk.

# Chapter 7

# Missing Data and Mixed Frequency

Real macro-finance panels are rarely dense. Missing observations, mixed-frequency releases, publication lags, and ragged starts must be handled as part of the filtering contract rather than as ad hoc data cleaning outside the likelihood.

## 7.1 Selection Form

Let $M_t$ be a deterministic selection matrix that extracts the observed components of $y_t$. The dense observation equation

$$y_t = a + Hx_t + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R)$$

becomes

$$M_t y_t = M_t a + M_t H x_t + M_t \varepsilon_t, \qquad M_t \varepsilon_t \sim \mathcal{N}(0, M_t R M_t^\top).$$

The prediction-error likelihood in (5.10) then uses the selected innovation and selected innovation covariance. If no components are observed at time $t$, the filter performs prediction without a measurement update.

Equivalently, write $\mathcal{O}_t$ for the observed row set. The per-period objects are

$$v_{t,\mathcal{O}_t} = y_{t,\mathcal{O}_t} - a_{\mathcal{O}_t} - H_{\mathcal{O}_t} m_{t|t-1}, \qquad S_{t,\mathcal{O}_t} = H_{\mathcal{O}_t} P_{t|t-1} H_{\mathcal{O}_t}^\top + R_{\mathcal{O}_t,\mathcal{O}_t}.$$

The likelihood contribution uses the dimension $|\mathcal{O}_t|$. If $\mathcal{O}_t$ is empty, the contribution is zero and the filtered moments equal the predicted moments. This convention is important for ragged panels because it makes missingness explicit in the likelihood rather than hidden in preprocessed data.

## 7.2 Mixed-Frequency State Augmentation

Mixed-frequency models often require deterministic auxiliary states. Examples include cumulators for quarterly growth, publication-lag states, and running averages used to align daily, monthly, and quarterly releases. These coordinates are part of the structural state contract from Definition 2.2. Their process covariance may be singular because the auxiliary rows are deterministic functions of lagged states and current high-frequency states.

BayesFilter should therefore distinguish two operations:

(i) augmenting the state so the observation equation is linear or conditionally linear at the target sampling frequency;

(ii) selecting the observed rows at each time point.

Both operations preserve the prediction-error likelihood when the augmented transition and observation maps are the declared model. A small ridge added only for numerical conditioning must be recorded as numerical regularization, not as a new source of structural shock variation.

## 7.3 Implementation Policy

The public backend must state whether it supports sparse panels. A dense-only backend should fail before likelihood evaluation if the observations contain NaNs, infinities, or masks that imply unsupported missing-data behavior. The MacroFinance missing-data policy tests explicitly check that unsupported sparse observation masks raise a named pending-support error rather than leaking NaNs into the Kalman recursion.

For compiled likelihoods, there are two viable policies:

- group time steps by mask pattern and keep each compiled path static;
- keep a fixed dense shape and use selection or masking operations whose shape is known at trace time.

Either policy is acceptable if the likelihood value and derivative contract are unchanged and the diagnostics identify which observations contributed to each time step.

The diagnostic record should include at least:

- number of observed rows at each time or by mask pattern;
- count of all-missing prediction-only steps;
- mask-pattern identifiers used for compiled grouping;
- whether missingness was handled by row selection, dense masking, or an unsupported fallback;
- any regularization applied to augmented deterministic rows.

## 7.4 Derivative Deferral

The value contract above is independent of the derivative implementation. Later derivative chapters must state whether mask patterns are differentiated as fixed data, whether all-missing steps contribute zero derivative to the measurement likelihood, and whether mixed-frequency augmentation parameters enter the transition, the observation map, or both. This chapter fixes the likelihood object; it does not yet certify any derivative backend.

# Chapter 8

# Large-Scale Linear Gaussian State-Space Models

Large linear Gaussian models are the bridge between textbook Kalman filtering and industrial Bayesian state-space inference. They are still exact, but their dimension exposes the numerical and compilation issues that small examples can hide.

## 8.1 Scale Targets

A BayesFilter scale target should record:

- state dimension $n_x$;
- observation dimension $n_y$;
- time length $T$;
- parameter dimension $d_\theta$;
- dense or sparse observation policy;
- backend ladder and fallback rules;
- memory layout and compilation policy.

The MacroFinance large-scale LGSSM tests include scenarios such as `baseline_10x3x5`, low-measurement-noise collinearity, ill-conditioned process noise, and near-rank-loss candidates. BayesFilter uses these as infrastructure evidence, not as a claim that every NAWM-sized model is already solved.

## 8.2 Backend Registry

Large-scale runs should be described by a backend registry rather than by a single method name. A registry entry should identify:

- the value backend, such as covariance form, solve form, Cholesky square root, QR square root, or spectral fallback;

- the derivative backend, such as tape autodiff, analytic score, analytic Hessian, custom gradient, or finite-difference diagnostic;
- the initialization policy;
- the missing-data policy;
- static-shape and compilation assumptions;
- finite-failure diagnostics and fallback labels.

This registry is part of the scientific result. A reported HMC run without backend metadata is not reproducible enough to diagnose whether a divergence, small effective sample size, or slow compilation came from the posterior geometry, the filtering algebra, the derivative path, or the execution mode.

## 8.3 Validation Ladder

The validation ladder should move from exact values to sampler behavior:

(i) value parity against the prediction-error decomposition in Chapter 5;

(ii) filtered-moment and backend-delta parity across covariance, solve, and square-root implementations on controlled small cases;

(iii) stress cases with low measurement noise, nearly collinear observations, singular or ill-conditioned process covariance, and long panels;

(iv) derivative parity on small cases before the derivative backend is used by HMC;

(v) compiled-path checks for static shape, finite diagnostics, and stable recompilation behavior;

(vi) short HMC smoke runs that exercise the target without being used as convergence evidence;

(vii) medium and strict HMC recovery only after the value and derivative gates have passed.

The same ladder should be reused for MacroFinance adapters, DSGE adapters, and future BayesFilter-native examples. Passing it for one model family is useful evidence about the backend, but it does not automatically transfer to another state partition, observation mask policy, or parameterization.

## 8.4 Stress Scenarios

The stress suite should include both numerical and modeling edge cases:

- structurally singular $Q$ from AR($p$) companion states, mixed-frequency accumulators, or deterministic structural coordinates;

- nearly singular innovation covariance from collinear measurements or very small measurement noise;
- long panels where repeated updates test covariance symmetry and factor reconstruction;
- ragged panels and all-missing time steps;
- parameter draws near admissibility boundaries;
- backend fallback triggers and spectral-gap telemetry.

Each scenario should define the expected diagnostic outcome before it is used as an HMC target. Some cases should pass with comfortable numerical margins; others should return named finite failures. Both outcomes are useful if they are intentional and reproducible.

## 8.5 Readiness Criteria

A large-scale LGSSM backend is ready for HMC experiments only after:

(i) the value backend matches a solve-form or covariance-form oracle;

(ii) gradient and Hessian backends match finite differences or autodiff on small controlled cases;

(iii) square-root or QR backends match the solve-form backend within stated tolerances on dimension-scaled cases;

(iv) diagnostics remain finite under stress scenarios;

(v) shape and compilation behavior are documented.

The current evidence supports this ladder as a validation design. It does not remove the need to rerun the ladder for each new model family, parameterization, or missing-data policy.

## 8.6 Derivative Consolidation Hypotheses

The next analytic-derivative pass should test the following hypotheses against the MacroFinance derivation and implementation:

(i) the Chapter 5 value contract is identical to the likelihood differentiated in the MacroFinance analytic Kalman note;

(ii) analytic score recursions match autodiff or finite differences on small dense and singular-$Q$ cases;

(iii) Hessian recursions match finite-difference-on-gradient diagnostics on small cases before they are used for mass-matrix or curvature proposals;

(iv) solve-form derivatives avoid explicit innovation-covariance inverses in production code;

(v) derivative diagnostics identify the first time step and matrix block responsible for nonfinite values or large parity errors.

These are hypotheses, not completed certification. They should be promoted to claims only after source derivations, code, and numerical tests agree.

# Part III

# Analytic Derivatives

# Chapter 9

# Kalman Score Recursions

The score recursion differentiates the prediction-error decomposition from Chapter 5. In BayesFilter this is the first derivative contract for exact linear Gaussian HMC targets.

For parameter component $\theta_i$, write

$$\dot{z}^{(i)} = \frac{\partial z}{\partial \theta_i}.$$

All objects in the filtering recursion may depend on $\theta$:

$$c_t, \quad F_t, \quad Q_t, \quad a_t, \quad H_t, \quad R_t.$$

Time subscripts are kept in this chapter because missing-data masks and mixed-frequency observation systems can make the effective observation equation time varying even when the underlying structural model is time homogeneous.

## 9.1 Prediction Derivatives

The differentiated prediction step is

$$\dot{m}^{(i)}_{t|t-1} = \dot{c}^{(i)}_t + \dot{F}^{(i)}_t m_{t-1|t-1} + F_t \dot{m}^{(i)}_{t-1|t-1}, \tag{9.1}$$

$$\dot{P}^{(i)}_{t|t-1} = \dot{F}^{(i)}_t P_{t-1|t-1} F_t^\top + F_t \dot{P}^{(i)}_{t-1|t-1} F_t^\top + F_t P_{t-1|t-1} \dot{F}^{(i)\top}_t + \dot{Q}^{(i)}_t. \tag{9.2}$$

These equations also define the initial-condition contract: if the implementation fixes $(m_{0|0}, P_{0|0})$ as data, the initial derivatives are zero; if it uses a stationary initial covariance, the derivatives must come from the corresponding derivative Lyapunov equations and be reported as part of the structural provider.

## 9.2 Innovation Derivatives

The source note derives the innovation derivatives

$$\dot{v}^{(i)}_t = -\dot{a}^{(i)}_t - \dot{H}^{(i)}_t m_{t|t-1} - H_t \dot{m}^{(i)}_{t|t-1}, \tag{9.3}$$

$$\dot{S}^{(i)}_t = \dot{H}^{(i)}_t P_{t|t-1} H_t^\top + H_t \dot{P}^{(i)}_{t|t-1} H_t^\top + H_t P_{t|t-1} \dot{H}^{(i)\top}_t + \dot{R}^{(i)}_t. \tag{9.4}$$

Using $\partial_i S^{-1} = -S^{-1} \dot{S}^{(i)} S^{-1}$, the per-time score contribution is

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}^{(i)}_t) + 2\dot{v}^{(i)\top}_t S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}^{(i)}_t S_t^{-1} v_t\right]. \tag{9.5}$$

For production code, BayesFilter prefers the solve form. Let $S_t w_t = v_t$. Then the same score can be written as

$$\frac{\partial \ell_t}{\partial \theta_i} = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right]. \tag{9.6}$$

The trace term may be computed by explicit materialization in small tests, but production code should prefer factor solves or trace estimators appropriate to the dimension and backend.

## 9.3 Gain and Update Derivatives

The score contribution at time $t$ can be computed as soon as (9.3)–(9.4) are available. The recursion must still propagate derivatives to the next time step. For the covariance-form Kalman update,

$$K_t = P_{t|t-1} H_t^\top S_t^{-1},$$

the gain derivative is

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1}^{(i)} H_t^\top S_t^{-1} + P_{t|t-1}\dot{H}_t^{(i)\top} S_t^{-1} - P_{t|t-1} H_t^\top S_t^{-1}\dot{S}_t^{(i)} S_t^{-1}. \tag{9.7}$$

Equivalently, if the implementation treats the gain as the solved matrix $S_t K_t^\top = H_t P_{t|t-1}$, then

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t}\left(\dot{H}_t^{(i)} P_{t|t-1} + H_t \dot{P}_{t|t-1}^{(i)} - \dot{S}_t^{(i)} K_t^\top\right). \tag{9.8}$$

The filtered mean and covariance derivatives are

$$\dot{m}_{t|t}^{(i)} = \dot{m}_{t|t-1}^{(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \tag{9.9}$$

$$\dot{P}_{t|t}^{(i)} = -\dot{K}_t^{(i)} H_t P_{t|t-1} - K_t \dot{H}_t^{(i)} P_{t|t-1} + (I - K_t H_t)\dot{P}_{t|t-1}^{(i)}. \tag{9.10}$$

Equation (9.10) differentiates the simplified covariance update $P_{t|t} = (I - K_t H_t) P_{t|t-1}$. A Joseph-form or square-root backend may use a different algebraic update for numerical reasons, but it must produce derivatives of the same filtered covariance it uses in the next prediction step.

## 9.4 Masks and Time-Varying Observation Blocks

For missing data, let $M_t$ select the observed rows at time $t$. The effective observation objects are

$$y_t^o = M_t y_t, \qquad a_t^o = M_t a_t, \qquad H_t^o = M_t H_t, \qquad R_t^o = M_t R_t M_t^\top.$$

The score recursion above applies after replacing $(y_t, a_t, H_t, R_t)$ by $(y_t^o, a_t^o, H_t^o, R_t^o)$. The mask is data, not a differentiated parameter, so $\dot{M}_t = 0$. If no rows are observed, the likelihood and score contribution for that time are zero and the update is the identity:

$$m_{t|t} = m_{t|t-1}, \qquad P_{t|t} = P_{t|t-1},$$

with the same equalities for first derivatives. This convention ensures that the derivative path corresponds to the same masked likelihood value used by the filter.

## 9.5 Evidence Status

The current MathDevMCP AST audit identifies Cholesky/solve, prediction, update, quadratic-form, and derivative-recursion structure in that file, but it does not by itself certify the algebra. The stronger evidence is the MacroFinance test suite comparing solve, QR square-root, autodiff, and finite difference results on controlled LGSSMs.

# Chapter 10

# Kalman Hessian and Observed Information

The Hessian layer is necessary for observed-information diagnostics, local identification, and mass-matrix construction, but it is also the easiest place to overstate the state of the mathematics. BayesFilter therefore records the Hessian as a source-backed contract and validation target, not as a finished proof for every backend.

For second derivatives, write

$$\ddot{z}^{(ij)} = \frac{\partial^2 z}{\partial\theta_i \partial\theta_j}.$$

The MacroFinance note expands the second-order prediction, innovation, and gain recursions by product rule. In solve form, the source introduces

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t}\left(\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t\right), \tag{10.1}$$

where $\mathcal{S}_{S_t}(b)$ denotes solving $S_t x = b$. It then decomposes the per-time Hessian contribution into log-determinant, innovation-linear, and quadratic terms:

$$\frac{\partial^2 \ell_t}{\partial\theta_i \partial\theta_j} = -\frac{1}{2}\left[T_{ij}^{\log\det} + T_{ij}^{v} - T_{ij}^{S}\right]. \tag{10.2}$$

## 10.1 Second-Order Prediction

The second-order prediction step is the product-rule derivative of (9.1)–(9.2). For the predicted mean,

$$\ddot{m}_{t|t-1}^{(ij)} = \ddot{c}_t^{(ij)} + \ddot{F}_t^{(ij)} m_{t-1|t-1} + \dot{F}_t^{(i)} \dot{m}_{t-1|t-1}^{(j)} + \dot{F}_t^{(j)} \dot{m}_{t-1|t-1}^{(i)} + F_t \ddot{m}_{t-1|t-1}^{(ij)}. \tag{10.3}$$

For the predicted covariance,

$$\begin{aligned}
\ddot{P}_{t|t-1}^{(ij)} = {} & \ddot{F}_t^{(ij)} P_{t-1|t-1} F_t^\top + \dot{F}_t^{(i)} \dot{P}_{t-1|t-1}^{(j)} F_t^\top + \dot{F}_t^{(i)} P_{t-1|t-1} \dot{F}_t^{(j)\top} \\
& + \dot{F}_t^{(j)} \dot{P}_{t-1|t-1}^{(i)} F_t^\top + F_t \ddot{P}_{t-1|t-1}^{(ij)} F_t^\top + F_t \dot{P}_{t-1|t-1}^{(i)} \dot{F}_t^{(j)\top} \\
& + \dot{F}_t^{(j)} P_{t-1|t-1} \dot{F}_t^{(i)\top} + F_t \dot{P}_{t-1|t-1}^{(j)} \dot{F}_t^{(i)\top} + F_t P_{t-1|t-1} \ddot{F}_t^{(ij)\top} + \ddot{Q}_t^{(ij)}.
\end{aligned} \tag{10.4}$$

These equations are the second-order source recursions in the MacroFinance analytic derivative note.

## 10.2 Second-Order Innovation Objects

The innovation derivatives are

$$\ddot{v}_t^{(ij)} = -\ddot{a}_t^{(ij)} - \ddot{H}_t^{(ij)} m_{t|t-1} - \dot{H}_t^{(i)} \dot{m}_{t|t-1}^{(j)} - \dot{H}_t^{(j)} \dot{m}_{t|t-1}^{(i)} - H_t \ddot{m}_{t|t-1}^{(ij)}, \tag{10.5}$$

$$\begin{aligned} \ddot{S}_t^{(ij)} = {} & \ddot{H}_t^{(ij)} P_{t|t-1} H_t^\top + \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} H_t^\top + \dot{H}_t^{(i)} P_{t|t-1} \dot{H}_t^{(j)\top} \\ & + \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} H_t^\top + \dot{H}_t^{(j)} P_{t|t-1} \dot{H}_t^{(i)\top} + H_t \ddot{P}_{t|t-1}^{(ij)} H_t^\top \\ & + H_t \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} + H_t \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} + H_t P_{t|t-1} \ddot{H}_t^{(ij)\top} + \ddot{R}_t^{(ij)}. \end{aligned} \tag{10.6}$$

Together with the first-order innovation objects from Chapter 9, these are the only model-specific inputs needed by the solve-form likelihood Hessian.

## 10.3 Solve-Form Hessian

The compact statement above is useful only if the three terms are kept explicit. Let $S_t w_t = v_t$ and define $\dot{w}_t^{(j)}$ by (10.1). The second derivative of the solved vector is

$$\ddot{w}_t^{(ij)} = \mathcal{S}_{S_t} \left( \ddot{v}_t^{(ij)} - \ddot{S}_t^{(ij)} w_t - \dot{S}_t^{(i)} \dot{w}_t^{(j)} - \dot{S}_t^{(j)} \dot{w}_t^{(i)} \right). \tag{10.7}$$

The log-determinant term is

$$T_{ij}^{\log\det} = \operatorname{tr}\left( S_t^{-1} \ddot{S}_t^{(ij)} - S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} \right). \tag{10.8}$$

The innovation-linear term is

$$T_{ij}^{v} = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top} \dot{w}_t^{(j)}. \tag{10.9}$$

The quadratic term is

$$T_{ij}^{S} = 2\dot{w}_t^{(j)\top} \dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t. \tag{10.10}$$

Substituting (10.8)–(10.10) into (10.2) gives the observed Hessian contribution. The formula requires only the innovation objects

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}.$$

It is therefore backend-neutral. A linear Kalman filter, a square-root Kalman filter, or a sigma-point filter may all use the same likelihood Hessian layer once their own recursions supply these six objects.

## 10.4 Second-Order Gain and Update

The covariance-form backend also propagates the Hessian of the filtered state objects. Define

$$\dot{S}_t^{-1(i)} = -S_t^{-1} \dot{S}_t^{(i)} S_t^{-1}, \tag{10.11}$$

$$\ddot{S}_t^{-1(ij)} = S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} + S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} \dot{S}_t^{(j)} S_t^{-1} - S_t^{-1} \ddot{S}_t^{(ij)} S_t^{-1}. \tag{10.12}$$

Then

$$\dot{K}_t^{(i)} = \dot{P}_{t|t-1}^{(i)} H_t^\top S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} H_t^\top \dot{S}_t^{-1(i)}, \tag{10.13}$$

$$\begin{aligned}\ddot{K}_t^{(ij)} = {} & \ddot{P}_{t|t-1}^{(ij)} H_t^\top S_t^{-1} + \dot{P}_{t|t-1}^{(i)} \dot{H}_t^{(j)\top} S_t^{-1} + \dot{P}_{t|t-1}^{(i)} H_t^\top \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} \dot{H}_t^{(i)\top} S_t^{-1} + P_{t|t-1} \ddot{H}_t^{(ij)\top} S_t^{-1} + P_{t|t-1} \dot{H}_t^{(i)\top} \dot{S}_t^{-1(j)} \\ & + \dot{P}_{t|t-1}^{(j)} H_t^\top \dot{S}_t^{-1(i)} + P_{t|t-1} \dot{H}_t^{(j)\top} \dot{S}_t^{-1(i)} + P_{t|t-1} H_t^\top \ddot{S}_t^{-1(ij)}.\end{aligned} \tag{10.14}$$

The filtered mean Hessian is

$$\ddot{m}_{t|t}^{(ij)} = \ddot{m}_{t|t-1}^{(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \tag{10.15}$$

For the simplified covariance update,

$$\begin{aligned}\ddot{P}_{t|t}^{(ij)} = {} & -\ddot{K}_t^{(ij)} H_t P_{t|t-1} - \dot{K}_t^{(i)} \dot{H}_t^{(j)} P_{t|t-1} - \dot{K}_t^{(i)} H_t \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} \dot{H}_t^{(i)} P_{t|t-1} - K_t \ddot{H}_t^{(ij)} P_{t|t-1} - K_t \dot{H}_t^{(i)} \dot{P}_{t|t-1}^{(j)} \\ & - \dot{K}_t^{(j)} H_t \dot{P}_{t|t-1}^{(i)} - K_t \dot{H}_t^{(j)} \dot{P}_{t|t-1}^{(i)} + (I - K_t H_t) \ddot{P}_{t|t-1}^{(ij)}.\end{aligned} \tag{10.16}$$

The gain and covariance formulas are useful as a covariance-form audit oracle. Solve-form or square-root implementations may avoid explicit (10.11)–(10.12) internally, but they must agree with these recursions on controlled well-conditioned systems.

## 10.5 Information Objects and Sign Convention

This chapter computes Hessians of the log likelihood contribution $\ell_t(\theta)$. The observed information contribution is

$$\mathcal{J}_t(\theta) = -\nabla_\theta^2 \ell_t(\theta),$$

and posterior precision at a mode additionally includes the negative Hessian of the log prior. BayesFilter should not call a matrix a mass matrix, local covariance, or identification object unless the sign convention and prior contribution have been stated explicitly.

## 10.6 Hessian-Ready Backend Contract

A backend is Hessian-ready when it can return, for every time step and active parameter pair,

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)}$$

for the same likelihood value it evaluates. To propagate the scan, it must also return either the covariance-form update derivatives (10.15)–(10.16) or an equivalent factor/backend-specific derivative update with reconstruction diagnostics. This is the bridge used later by the SVD sigma-point chapter: the nonlinear backend supplies approximate innovation derivatives, and the solve-form Hessian layer remains the same.

## 10.7 Evidence Status

The source derivation is detailed, but BayesFilter should keep three evidence levels distinct:

- formula provenance from the MacroFinance analytic derivative note;
- structural code evidence from MathDevMCP AST audits;
- numerical parity evidence from tests against autodiff and finite differences.

The current MathDevMCP audit of the MacroFinance solve differentiated Kalman backend found the Kalman operation graph and solve structure but reported a mismatch for a strict `logdet/trace` operation query and missing explicit shape and covariance guards. This should be treated as a production-readiness prompt, not as a proof failure. The generic LGSSM autodiff-validation tests are the stronger mathematical check for the current implementation because they compare likelihood, gradient, and Hessian against solve backends, TensorFlow autodiff, and finite differences on controlled small-dimensional models.

# Chapter 11

# Structural Derivatives

Kalman derivative recursions assume that the derivatives of the state-space objects are already available. Structural derivatives are the upstream layer: they map model parameters into

$$c_\theta,\ F_\theta,\ Q_\theta,\ a_\theta,\ H_\theta,\ R_\theta$$

and their first and second derivatives.

## 11.1 Provider Map

Let $\psi \in \mathbb{R}^p$ denote the parameter vector used by the numerical provider. In HMC this is usually an unconstrained or transformed parameter vector, not necessarily the raw economic vector. The structural provider declares the map

$$g(\psi) = \mathrm{vec}(c(\psi), F(\psi), Q(\psi), a(\psi), H(\psi), R(\psi), m_0(\psi), P_0(\psi))\,. \tag{11.1}$$

The structural map contract uses the reduced-form objects $(c, F, Q, a, H, R, m_0, P_0)$ because the filtering scan cannot be differentiated without an explicit initial-condition policy.

For every active coordinate $\psi_i$, the first-order provider contract is

$$\dot{G}^{(i)} = \left(\dot{c}^{(i)}, \dot{F}^{(i)}, \dot{Q}^{(i)}, \dot{a}^{(i)}, \dot{H}^{(i)}, \dot{R}^{(i)}, \dot{m}_0^{(i)}, \dot{P}_0^{(i)}\right). \tag{11.2}$$

For every active pair $(\psi_i, \psi_j)$, the second-order contract is

$$\ddot{G}^{(ij)} = \left(\ddot{c}^{(ij)}, \ddot{F}^{(ij)}, \ddot{Q}^{(ij)}, \ddot{a}^{(ij)}, \ddot{H}^{(ij)}, \ddot{R}^{(ij)}, \ddot{m}_0^{(ij)}, \ddot{P}_0^{(ij)}\right). \tag{11.3}$$

Equations (11.2)–(11.3) are the objects consumed by Chapters 9 and 10. A backend may store them as dense tensors, sparse blocks, named arrays, or lazy linear operators, but the contract is mathematical: each block must mean a derivative of the same state-space law used to evaluate the likelihood.

For BayesFilter, a derivative provider must declare:

- parameter order and units;
- shape of every first and second derivative tensor;
- whether derivatives are analytic, autodiff, finite-difference, or unavailable;
- which parameters affect which reduced-form objects;
- whether cross-derivatives are implemented or intentionally zero.

## 11.2 Parameter Transforms

Let $\phi = T(\psi)$ be the raw economic parameter vector and let $M(\phi)$ be any reduced-form block among $c, F, Q, a, H, R, m_0, P_0$. Write

$$M_\alpha = \frac{\partial M}{\partial \phi_\alpha}, \qquad M_{\alpha\beta} = \frac{\partial^2 M}{\partial \phi_\alpha \partial \phi_\beta},$$

and

$$T_{\alpha,i} = \frac{\partial \phi_\alpha}{\partial \psi_i}, \qquad T_{\alpha,ij} = \frac{\partial^2 \phi_\alpha}{\partial \psi_i \partial \psi_j}.$$

Then

$$\dot{M}^{(i)} = \sum_\alpha M_\alpha T_{\alpha,i}, \tag{11.4}$$

$$\ddot{M}^{(ij)} = \sum_{\alpha,\beta} M_{\alpha\beta} T_{\alpha,i} T_{\beta,j} + \sum_\alpha M_\alpha T_{\alpha,ij}. \tag{11.5}$$

These are the BayesFilter versions of the MacroFinance transformed-parameter chain-rule equations. They matter because HMC often runs on $\psi = \log \sigma$, $\psi = \log \kappa$, Cholesky coordinates, Fisher transforms, or stationarity transforms. A correct derivative with respect to a raw standard deviation is wrong for HMC if it is passed off as the derivative with respect to the log standard deviation.

## 11.3 Initial Conditions

The initial moments in (11.1) are part of the model law. There are three common cases:

(i) fixed diffuse or fixed proper initialization: $\dot{m}_0 = \dot{P}_0 = \ddot{m}_0 = \ddot{P}_0 = 0$ because the initial moments are data of the filtering problem;

(ii) stationary initialization: $m_0$ and $P_0$ solve model equations and must be differentiated together with those equations;

(iii) estimated or hierarchical initialization: $m_0$ and $P_0$ have their own parameter blocks and cross derivatives.

For a stationary linear system with $m_0 = c + Fm_0$, the first derivative satisfies

$$(I - F)\dot{m}_0^{(i)} = \dot{c}^{(i)} + \dot{F}^{(i)} m_0. \tag{11.6}$$

For $P_0 = FP_0F^\top + Q$, the first derivative satisfies the Lyapunov derivative equation

$$\dot{P}_0^{(i)} = F\dot{P}_0^{(i)}F^\top + \dot{F}^{(i)}P_0F^\top + FP_0\dot{F}^{(i)\top} + \dot{Q}^{(i)}. \tag{11.7}$$

Second derivatives follow by differentiating (11.6) and (11.7). BayesFilter should record whether these equations were solved analytically, by a differentiable Lyapunov solver, by autodiff through the stationary solver, or intentionally not used.

## 11.4 Sparsity, Masks, and Zero Blocks

A derivative tensor entry equal to zero is different from a derivative entry that is unavailable. The provider must distinguish:

$$\ddot{M}^{(ij)} = 0 \quad \text{declared structural zero,} \qquad \ddot{M}^{(ij)} \text{ not implemented.}$$

Structural zeros are useful for large models because they allow the score and Hessian scan to skip blocks without changing the mathematical target. Missing entries are not a mathematical shortcut; they mean the backend cannot certify the requested derivative object.

Observation masks are data-side selections. If $J_t$ selects observed coordinates at time $t$, the provider may expose either the full $(a_t, H_t, R_t)$ together with $J_t$ or the already-masked blocks

$$a_t^o = J_t a_t, \qquad H_t^o = J_t H_t, \qquad R_t^o = J_t R_t J_t^\top.$$

The same selection applies to derivatives. Since $J_t$ is data, not a parameter, no derivative of the mask is taken. This convention is the structural upstream counterpart of the masking rule in Section 9.4.

## 11.5 Metadata and Shape Contract

The provider must make the following metadata inspectable before likelihood evaluation:

- names and order of $\psi_i$;
- transform type and raw unit for each parameter;
- state and observation coordinate names;
- dimensions of each reduced-form block;
- tensor layout conventions, including whether Hessian tensors store all pairs or only the symmetric upper triangle;
- symmetry and positive-definiteness policies for $Q$, $R$, and $P_0$;
- deterministic or algebraic state-coordinate roles when the provider is used by a structural nonlinear filter.

These declarations are not decorative. Without them, a finite likelihood can be computed from a different parameter ordering, unit system, or deterministic state convention than the derivative tensors assume.

The MacroFinance large-scale derivative-provider tests check derivative shapes, finite-difference parity by object type, reduced-form rank diagnostics, and parameter-unit rescaling. BayesFilter should preserve these as public contract tests before a structural model is allowed to feed an HMC target.

## 11.6 Provider Validation

The minimum validation ladder for a structural provider is:

(i) finite value check: all blocks in (11.1) are finite, conformable, symmetric where required, and admissible before filtering;

(ii) shape check: (11.2)–(11.3) have the declared dimensions for every active parameter and pair;

(iii) transform check: (11.4)–(11.5) agree with finite differences in transformed coordinates;

(iv) block finite-difference check: each reduced-form object is checked before it is inserted into a long filtering scan;

(v) end-to-end check: the resulting likelihood score and Hessian agree with finite differences or autodiff on small systems where that comparison is reliable;

(vi) diagnostic check: structural zeros, unavailable derivatives, regularization, rejected parameter points, and mask policies are reported.

Only after these checks pass should the structural provider feed an HMC target or an observed-information diagnostic. This is the upstream analogue of the factor parity gates in Chapter 12.

# Chapter 12

# QR and Cholesky Factor Derivatives

Square-root filters propagate factors rather than covariance matrices. Their derivative contract is therefore not only a Kalman contract; it is also a factor calculus contract. The likelihood score and Hessian in Chapters 9–10 are expressed in terms of $v_t$, $S_t$, and their derivatives. A factor backend is acceptable only if the factors it propagates reproduce those covariance objects and the same derivative objects.

BayesFilter uses three levels of factor evidence:

(i) Cholesky or QR identities stated in the source derivation;

(ii) reconstruction diagnostics proving that differentiated factors reconstruct the differentiated covariance objects;

(iii) backend parity against solve-form likelihood, score, and Hessian.

The QR square-root tests in MacroFinance require prediction, innovation, and filtered factor derivative reconstruction errors below tight tolerances on small-dimensional parameter grids, and they compare QR likelihood, gradient, and Hessian against the solve-form backend. Those tests are currently the right standard for BayesFilter factor-derivative chapters: a factor formula is not production-ready until both reconstruction and end-to-end likelihood parity pass.

## 12.1 Reconstruction Contract

Let $A(\theta)$ be any covariance-like symmetric positive semidefinite object used by the filter, and let an implemented factor satisfy

$$A_\star(\theta) = C(\theta)C(\theta)^\top.$$

The subscript $\star$ is deliberate. For a Cholesky backend, $A_\star = A$ on a positive-definite branch. For a spectral backend, $A_\star$ may be a rank-truncated, floored, or symmetrized version of $A$. The first derivative reconstructed by the factor is

$$\dot{A}_\star^{(i)} = \dot{C}^{(i)}C^\top + C\dot{C}^{(i)\top}. \tag{12.1}$$

The second derivative reconstructed by the factor is

$$\ddot{A}_\star^{(ij)} = \ddot{C}^{(ij)}C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \tag{12.2}$$

Equations (12.1)–(12.2) are the universal audit equations for every factor backend. The derivative may be obtained by analytic formulas, custom gradients, or autodiff, but the resulting $C$, $\dot{C}$, and $\ddot{C}$ must reconstruct the implemented covariance branch that produced the likelihood value.

## 12.2 Cholesky Factor Derivatives

Suppose $A = LL^\top$, where $A$ is positive definite and $L$ is lower triangular with positive diagonal. Define the lower-triangular half-diagonal operator

$$\Phi(X)_{ab} = \begin{cases} X_{ab}, & a > b, \\ X_{aa}/2, & a = b, \\ 0, & a < b. \end{cases}$$

For a first derivative, form

$$B^{(i)} = L^{-1}\dot{A}^{(i)}L^{-\top}, \qquad G^{(i)} = \Phi\left(B^{(i)}\right).$$

Then the Cholesky factor derivative is

$$\dot{L}^{(i)} = LG^{(i)}. \tag{12.3}$$

For a second derivative, remove the known first-order products before applying the same triangular split:

$$C^{(ij)} = L^{-1}\left(\ddot{A}^{(ij)} - \dot{L}^{(i)}\dot{L}^{(j)\top} - \dot{L}^{(j)}\dot{L}^{(i)\top}\right)L^{-\top}, \qquad H^{(ij)} = \Phi\left(C^{(ij)}\right).$$

Then

$$\ddot{L}^{(ij)} = LH^{(ij)}. \tag{12.4}$$

The formulas are local smooth-branch formulas: they require a positive diagonal and become numerically fragile near a Cholesky pivot boundary. A backend that adds jitter before Cholesky must report whether its derivatives target the original $A$ or the implemented $A + \delta I$.

## 12.3 QR Factor Derivatives

QR square-root filters often factor a stack rather than a covariance matrix. Let $M(\theta) \in \mathbb{R}^{m\times n}$ have full column rank, $m \geq n$, and use the thin convention

$$M = QR, \qquad Q^\top Q = I_n, \qquad R \text{ upper triangular}, \qquad R_{aa} > 0.$$

The positive diagonal fixes the local sign convention. Define

$$A^{(i)} = Q^\top \dot{M}^{(i)} R^{-1}.$$

Let $\Omega^{(i)}$ be the skew-symmetric matrix identified from

$$\Omega^{(i)}_{ab} = \begin{cases} A^{(i)}_{ab}, & a > b, \\ 0, & a = b, \\ -A^{(i)}_{ba}, & a < b, \end{cases} \qquad T^{(i)} = A^{(i)} - \Omega^{(i)}.$$

Then

$$\dot{R}^{(i)} = T^{(i)}R, \tag{12.5}$$
$$\dot{Q}^{(i)} = Q\Omega^{(i)} + (I - QQ^\top)\dot{M}^{(i)}R^{-1}. \tag{12.6}$$

For second derivatives, define the effective second-order stack derivative

$$E^{(ij)} = \ddot{M}^{(ij)} - \dot{Q}^{(i)}\dot{R}^{(j)} - \dot{Q}^{(j)}\dot{R}^{(i)}$$

and

$$B^{(ij)} = Q^\top E^{(ij)}R^{-1}, \qquad C_Q^{(ij)} = -\dot{Q}^{(i)\top}\dot{Q}^{(j)} - \dot{Q}^{(j)\top}\dot{Q}^{(i)}.$$

The matrix $\Gamma^{(ij)} = Q^\top\ddot{Q}^{(ij)}$ is determined by

$$\Gamma_{ab}^{(ij)} = \begin{cases} B_{ab}^{(ij)}, & a > b, \\ C_{Q,aa}^{(ij)}/2, & a = b, \\ C_{Q,ab}^{(ij)} - B_{ba}^{(ij)}, & a < b, \end{cases} \qquad U^{(ij)} = B^{(ij)} - \Gamma^{(ij)}.$$

Then

$$\ddot{R}^{(ij)} = U^{(ij)}R, \tag{12.7}$$
$$\ddot{Q}^{(ij)} = Q\Gamma^{(ij)} + (I - QQ^\top)E^{(ij)}R^{-1}. \tag{12.8}$$

The QR formulas are local to a fixed rank and sign branch. Pivoting, column-rank loss, or diagonal sign corrections are branch events and must be reported as derivative-policy events, not hidden inside a smooth Hessian claim.

## 12.4 Square-Root Kalman Stacks

The generic formulas above enter the filter through stack factorizations. If $C_{t-1|t-1}C_{t-1|t-1}^\top = P_{t-1|t-1}$ and $G_tG_t^\top = Q_t$, the prediction covariance can be declared by the stack

$$M_t^P = \begin{bmatrix} F_tC_{t-1|t-1} & G_t \end{bmatrix}, \qquad P_{t|t-1} = M_t^P M_t^{P\top}.$$

Equivalently, a thin QR factorization of $M_t^{P\top} = Q_t^P R_t^P$ gives the lower factor $C_{t|t-1} = R_t^{P\top}$. The first and second derivatives of $M_t^P$ are obtained by product rule from the derivatives of $F_t$, $C_{t-1|t-1}$, and $G_t$, and (12.5)–(12.7) then give the derivatives of $C_{t|t-1}$.

Likewise, if $E_tE_t^\top = R_t$,

$$M_t^S = \begin{bmatrix} H_tC_{t|t-1} & E_t \end{bmatrix}, \qquad S_t = M_t^S M_t^{S\top},$$

and QR of $M_t^{S\top}$ gives the innovation factor used by the solve-form likelihood. The filtered covariance can be audited by the Joseph stack

$$M_t^U = \begin{bmatrix} (I - K_tH_t)C_{t|t-1} & K_tE_t \end{bmatrix}, \qquad P_{t|t} = M_t^U M_t^{U\top}.$$

Derivatives of $K_t$ are those in (9.7) and (10.14). Therefore a square-root implementation has two equivalent audit paths: reconstruct (12.1)–(12.2) from differentiated factors, and compare the resulting score/Hessian objects to the covariance-form recursions in Chapters 9–10.

## 12.5 Spectral Factors

SVD sigma-point filters use a spectral factor, not a triangular factor, to generate point displacements. If

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2},$$

then the factor actually reconstructs

$$P_+ = CC^\top = U\Lambda_+ U^\top,$$

where $\Lambda_+$ includes the rank, floor, or smoothing policy used by the implementation. Therefore spectral factor validation must distinguish two questions:

(i) Do $C$ and its derivatives reconstruct $P_+$ and its derivatives?

(ii) If $P_+ \neq P$, is the difference a declared model approximation, a finite-arithmetic regularization, or an invalid fallback?

This distinction is essential for HMC. A hard eigenvalue floor can produce a finite likelihood while changing the derivative target at the floor boundary. If the floor is hard, derivatives are branch derivatives of $P_+$, not derivatives of the pre-regularized covariance $P$. If the floor is smooth, the floor derivative is part of the model actually differentiated. Either way, the factor reconstruction equations in (12.1)–(12.2) must be checked against $P_+$.

Chapter 18 therefore treats spectral derivatives as a separate contract with gap telemetry, active-floor reporting, and branch diagnostics. Its simple-spectrum SVD/eigen formulas define one smooth-branch policy; they do not remove the need for finite-difference checks across spectral gap, rank, and floor regimes.

## 12.6 Backend Parity Gates

A factor backend is ready to support analytic gradients and Hessians only after the following checks pass on controlled systems:

(i) value reconstruction: $\|A_\star - CC^\top\|$ is within tolerance for prediction, innovation, and filtered covariance objects;

(ii) first- and second-derivative reconstruction: (12.1)–(12.2) match the declared $\dot{A}_\star$ and $\ddot{A}_\star$;

(iii) solve parity: the factor solve gives the same $w_t$, score, and Hessian contribution as the covariance solve on well-conditioned systems;

(iv) branch reporting: jitter, rank truncation, pivoting, sign flips, active floors, and fallback paths are emitted as diagnostics;

(v) backend parity: Cholesky-refactorized, QR square-root, and SVD/spectral backends agree where their mathematical branches represent the same covariance law.

These gates prevent a common mistake: a numerically stable factor can make the likelihood finite while silently changing the derivative target. The factor chapter therefore supplies the algebraic contracts that the validation chapter turns into executable tests.

## 12.7 Fixed-Rank Rectangular QR Charts

The default full-rank recursion uses a square lower factor, but exact singular laws require a rectangular factor. Let $A \in \mathbb{R}^{n\times k}$ have constant rank $r \le \min(n,k)$, set $B = A^\top$, and let a rank-discovery preflight freeze a row permutation $\Pi$ such that

$$B\Pi = [B_1 \; B_2], \qquad B_1 \in \mathbb{R}^{k\times r}$$

has full column rank. On this fixed chart, use the positive-diagonal thin QR and projection

$$B_1 = QR_{11}, \qquad R_{12} = Q^\top B_2, \qquad G = \Pi \begin{bmatrix} R_{11} \\ R_{12} \end{bmatrix}^\top. \tag{12.9}$$

The chart residual is $E_2 = (I - QQ^\top)B_2$. When $E_2 = 0$, $A = GQ^\top$ and hence $AA^\top = GG^\top$, without forming either covariance. In finite arithmetic the implementation requires

$$\|E_2\|_F \le \tau_{\text{chart}} \max(1, \|A\|_F), \qquad \min_j |(R_{11})_{jj}| > \tau_{\text{pivot}} \max(\|A\|_F, 10^{-300}).$$

The repository float64 defaults are $\tau_{\text{chart}} = 10^{-10}$ and $\tau_{\text{pivot}} = 10^{-12}$. They are numerical admission thresholds, not changes to the statistical model.

For a parameter derivative, differentiate the same positive-diagonal QR branch. If $C = Q^\top \dot{B}_1 R_{11}^{-1}$, choose the skew matrix $\Omega$ from the strictly lower part of $C$ so that

$$\dot{Q} = (I - QQ^\top)\dot{B}_1 R_{11}^{-1} + Q\Omega, \qquad \dot{R}_{11} = (C - \Omega)R_{11}, \tag{12.10}$$

and then

$$\dot{R}_{12} = \dot{Q}^\top B_2 + Q^\top \dot{B}_2.$$

These equations define a smooth local chart; they do not define a derivative through a rank or pivot change.

## 12.8 Affine-Support Gaussian Likelihood and Derivative

Let $G \in \mathbb{R}^{n\times r}$ have full column rank and let $G = UR$ be its positive-diagonal thin QR. For an innovation $e$ in $\mathcal{R}(G)$, define $z = R^{-1}U^\top e$. The density with respect to the $r$-dimensional affine-support measure is

$$\ell_{\text{supp}} = -\frac{1}{2}\left[ r\log(2\pi) + 2\sum_{j=1}^{r} \log R_{jj} + z^\top z \right]. \tag{12.11}$$

It is $-\infty$ off support. On a fixed support and QR branch,

$$\dot{\ell}_{\text{supp}} = -\sum_j \frac{\dot{R}_{jj}}{R_{jj}} - z^\top \dot{z}, \qquad \dot{z} = R^{-1}(\dot{U}^\top e + U^\top \dot{e} - \dot{R}z). \tag{12.12}$$

The derivative includes motion of the support basis and the innovation. A log-determinant-only derivative is incomplete.

# Chapter 13

# Custom-Gradient Wrappers for HMC

A custom gradient is the exact HMC force only when it returns the derivative of the same scalar log target used in the Metropolis correction. A deterministic nearby force field can still leave the invariant law exact when the acceptance step evaluates the exact target value, but then the approximation changes leapfrog efficiency rather than the target. Bias arises when the value path is also changed, when the chain is left unadjusted, or when the force construction breaks the reversible volume-preserving integrator used by the correction step.

## 13.1 Same-Scalar Contract

Let the production value path return the full sampler-coordinate posterior target

$$\mathcal{L}(u) = \ell(T(u); y_{1:T}) + \log p(T(u)) + \log \left|\det DT(u)\right|, \tag{13.1}$$

where the likelihood term uses the same structural provider, masks, factor branch, regularization policy, and parameter transforms declared in Chapters 11–12. When a chapter later writes a likelihood-only derivative such as $\nabla_\theta \ell$, that object should be read as one component of the full sampler-coordinate target gradient, not as the entire HMC target by itself. The custom-gradient path is valid only if it returns

$$g_i(u) = \frac{\partial \mathcal{L}(u)}{\partial u_i}. \tag{13.2}$$

If the value path evaluates a floored spectral covariance $P_+$, the gradient must be the derivative of that implemented value path. If the value path uses a jittered innovation covariance $S_t + \delta I$, the gradient must include the same jitter policy or clearly report that the parameter point is outside the certified smooth branch. The wrapper must not evaluate one scalar and return the gradient of another.

BayesFilter's custom-gradient wrapper policy is:

(i) compute the primal log likelihood with the production backend;

(ii) return an analytic score that has been validated against the same primal value on controlled cases;

(iii) expose diagnostics for finite failures and backend fallbacks;

(iv) keep tensor shapes static inside compiled sampler paths;

(v) use Hessians for mass matrices or diagnostics, not inside every HMC leapfrog step unless explicitly justified.

This policy reflects the MacroFinance motivation: nested tape Hessians through full Kalman scans are useful as small-model validation oracles, but they are not the intended large-model production path.

## 13.2 Gradient Callback Contract

A custom-gradient wrapper should expose a contract equivalent to the following mathematical tuple:

$$\text{value_and_score}(u) = (\mathcal{L}(u), \nabla_u \mathcal{L}(u), d(u)) ,$$

where $d(u)$ is a diagnostic record. The diagnostic record must include:

- the structural provider and parameter transform identifiers;
- the active observation-mask policy;
- the covariance/factor backend used by the value path;
- any jitter, nugget, PSD projection, rank truncation, active spectral floor, pivoting, sign correction, or fallback branch;
- the minimum relevant Cholesky pivot, QR diagonal, or spectral gap;
- finite-value and finite-gradient status.

The gradient callback should return a failure status rather than silently replacing the derivative with zeros, prior gradients, or a different backend. Emergency fallbacks can be useful for optimization diagnostics, but HMC must know whether it is using the certified derivative of the returned scalar target. If the wrapper deliberately returns a finite invalid-region fallback scalar, then the only HMC-safe fallback gradient is the exact derivative of that fallback scalar. In particular, a constant low-density fallback must have zero gradient; a nonzero surrogate direction defines a different target unless accompanied by an explicit correction scheme.

## 13.3 Hessian and Observed-Information Path

The HMC hot path needs values and gradients. Hessians are usually used outside the leapfrog loop for mass-matrix initialization, local identification, and curvature diagnostics. If a wrapper exposes a Hessian, it should target

$$H_{ij}(\psi) = \frac{\partial^2 \mathcal{L}(\psi)}{\partial \psi_i \, \partial \psi_j}$$

for the same scalar in (13.1). The posterior-curvature matrix is $-H(\psi)$. If the chapter or implementation also reports likelihood-only observed information, it must state that

object separately as the negative Hessian of the likelihood term alone, together with the sign convention and any prior contribution. A Hessian path that omits prior curvature, transformed-parameter curvature, or jitter/floor branch terms must label itself as a partial diagnostic, not as posterior curvature.

The recommended production pattern is:

(i) value-plus-score path for MAP and HMC;

(ii) analytic or custom Hessian path for mode diagnostics and mass-matrix construction;

(iii) small-model autodiff Hessian path as an oracle, not as the large-model default.

## 13.4 Static-Shape and Compilation Contract

Compiled HMC paths require stable shapes. Parameter dimension, state dimension, observation dimension, maximum active observation size, and the layout of diagnostic records should be static across leapfrog calls. Missing data should be represented by fixed-shape masks or predeclared selection patterns rather than by dynamically changing tensor ranks. Fixed sigma-point rules must keep the number and ordering of points stable. Factor backends must report branch events without changing the differentiable return signature.

This static-shape rule is an implementation constraint, but it has mathematical content: changing the set of coordinates, points, or factors during a compiled scan can turn a smooth derivative contract into a piecewise branch derivative without making the branch visible to the sampler.

## 13.5 HMC Safety Labels

BayesFilter should distinguish the following labels:

- **value-correct**: the scalar likelihood or posterior value matches a reference value path;
- **gradient-correct**: the returned score matches finite differences or autodiff of that same scalar on certified cases;
- **curvature-correct**: the Hessian or observed information matches the same scalar and sign convention;
- **sampler-usable**: short HMC runs have finite log probabilities, finite gradients, and acceptable diagnostic telemetry;
- **production-gated**: compiled/eager parity, branch diagnostics, mask tests, stress tests, and representative workload tests all pass.

A finite HMC run is not by itself a gradient proof. Conversely, a correct gradient on small smooth systems does not certify spectral floors, rank changes, or DSGE deterministic-completion branches unless those regimes have been validated explicitly.

# Chapter 14

# Derivative Validation

Derivative validation is a ladder. Passing one rung is useful evidence, but it does not certify the whole industrial target.

## 14.1 Validation Ladder

BayesFilter derivative providers should pass:

(i) shape and finite-value checks;

(ii) finite-difference checks on scalar or tiny LGSSMs;

(iii) autodiff parity on small models where tape Hessians are feasible;

(iv) backend parity among covariance, solve, Cholesky, and QR variants;

(v) factor reconstruction and solve-residual diagnostics;

(vi) compiled/eager parity for TensorFlow backends;

(vii) stress tests for low noise, collinearity, near rank loss, and masks.

MacroFinance currently supplies examples of these tests, including QR square-root parity against autodiff and finite differences, solve backend agreement with the existing differentiated backend, TensorFlow eager/compiled parity, and large-scale backend ladder diagnostics. BayesFilter should import the testing discipline before importing any claim of production readiness.

## 14.2 What Is Being Compared

Every derivative test must name the scalar or tensor being compared. For a sampler-coordinate posterior target,

$$\mathcal{L}(u) = \ell(T(u); y_{1:T}) + \log p(T(u)) + \log \left|\det DT(u)\right|,$$

the score test compares

$$\hat{g}_i(u) \quad \text{to} \quad \frac{\mathcal{L}(u + he_i) - \mathcal{L}(u - he_i)}{2h}$$

or to an autodiff score of that same scalar. The Hessian test compares

$$\hat{H}_{ij}(u) \quad \text{to} \quad \frac{\hat{g}_i(u + he_j) - \hat{g}_i(u - he_j)}{2h}$$

or to a tape Hessian of the same scalar. If the reported analytic derivative is likelihood-only but the finite difference includes the prior or transform Jacobian, the test is invalid. Conversely, if a chapter validates $\nabla_\theta \ell$ in model-parameter coordinates, that result should be read as a component-level backend check rather than as the complete HMC target gradient unless the prior and Jacobian terms are added consistently. If the value path contains jitter, floors, masks, transformed parameters, or invalid- region fallback values, the comparison value must contain the same choices.

## 14.3 Recommended Rungs

The derivative validation ladder should be run in this order:

(i) **Finite values and shapes.** Check that value, score, Hessian, state moments, factors, provider tensors, and diagnostics are finite and have declared dimensions.

(ii) **Symmetry and admissibility.** Check that covariance-like objects are symmetric, positive semidefinite/definite under their declared policy, and that Hessians are symmetric within tolerance.

(iii) **Block finite differences.** Validate structural derivatives of $c, F, Q, a, H, R, m_0, P_0$ before running a long filter scan.

(iv) **Likelihood finite differences.** Compare score and Hessian of the complete scalar target on tiny systems.

(v) **Autodiff parity.** Use taped gradients and Hessians as small-system oracles where nested tapes are feasible.

(vi) **Backend parity.** Compare covariance, solve, Cholesky, QR, and SVD/spectral backends on systems where they represent the same law.

(vii) **Factor reconstruction.** Check (12.1)–(12.2) for prediction, innovation, and filtered covariance factors.

(viii) **Mask tests.** Test dense all-observed masks, sparse deterministic masks, and all-missing periods using the convention in Section 9.4.

(ix) **Spectral and branch stress tests.** Vary spectral gaps, Cholesky pivots, QR diagonals, rank thresholds, floors, jitter, and fallback branches while checking diagnostics.

(x) **Eager/compiled parity.** Compare values and derivatives under eager execution, compiled execution, and XLA/JIT paths when those paths are intended for production.

(xi) **Sampler smoke tests.** Run short HMC/NUTS checks only after deterministic value-and-gradient tests pass, and interpret them as sampler usability evidence, not proof of the derivative formulas.

## 14.4 Finite-Difference Tolerances

Finite differences are not exact tests; they are numerical experiments. The step size should be selected relative to the transformed parameter scale, for example

$$h_i = \eta(1 + |\psi_i|),$$

with several $\eta$ values checked when possible. A central difference should show a stable error plateau before the result is accepted. Parameters near constraints, branch boundaries, or spectral floors should be tested separately from smooth interior points, because the expected derivative may be a branch derivative rather than a smooth derivative of the unregularized model.

## 14.5 Backend-Specific Gates

Each backend has a distinct failure mode:

- covariance form can lose symmetry or positive semidefiniteness under finite arithmetic;
- solve form can hide ill conditioning unless solve residuals and log-determinant policies are reported;
- Cholesky form depends on positive pivots and any jitter policy;
- QR form depends on full column rank, diagonal sign convention, and pivot policy;
- SVD/spectral form depends on gap, rank, floor, sign/order, and fallback policy;
- sigma-point form additionally depends on fixed point rules, weights, nonlinear map derivatives, and moment reconstruction.

Backend parity is meaningful only when the compared backends target the same law. A covariance backend using $P$ and an SVD backend using a floored $P_+$ are not required to have identical derivatives unless $P_+ = P$ on that test case.

## 14.6 Validation Artifact

Every promoted derivative backend should leave an artifact containing:

- model name, parameter vector, transform policy, and reference point;
- backend name and value branch;
- tests run, tolerances, and maximum absolute/relative errors;
- finite-value, finite-gradient, and finite-Hessian status;
- branch diagnostics: jitter, PSD projection, spectral floors, pivots, rank truncation, sign/order corrections, and fallbacks;
- interpretation: which claim is supported and which regimes remain untested.

This artifact is the bridge between a mathematical derivation and an HMC claim. Without it, phrases such as “analytic Hessian”, “SVD sigma-point gradient”, or “HMC-ready backend” are too ambiguous to audit.

## 14.7 Audit Record

Each derivative chapter should record the mathematical assumptions, implementation branch conditions, and tests that support it. If a symbolic or automated audit is inconclusive, the monograph should say so. An inconclusive audit can still be valuable: it identifies which guards, shapes, or noncommutative matrix steps need manual proof before a formula is promoted to peer-review-ready status.

## 14.8 Singular-Support SR-UKF Validation Matrix

The fixed-rank rectangular route requires tests that distinguish small scale from poor conditioning. Uniformly scaling every factor tests homogeneity, but does not change the condition number. The validation matrix therefore also uses anisotropic families with retained singular scales ranging from 1 to $10^{-14}$, and it requires deterministic rejection once the fixed chart pivot is below the declared relative threshold.

On accepted branches, centered finite differences cover the rectangular factor, affine-support log likelihood, score, conditional gain, posterior factor, filtered mean, and multi-period accumulated score. The same fixture is run eagerly and with XLA. Separate branch tests deliberately move an observation off support or cross a rank, pivot, or chart boundary; the value is then the declared support rejection and the analytical score is invalid rather than silently continued.

For the nugget experiment $P_\varepsilon = GG^\top + \varepsilon I_n$, tests must not compare the raw ambient likelihood with a finite support likelihood. For $e \in \mathcal{R}(G)$, rank $r$, the valid numerical limit is

$$\ell_{\text{supp}} = \lim_{\varepsilon \downarrow 0} \left\{ \ell_\varepsilon + \frac{n-r}{2} \log(2\pi\varepsilon) \right\}. \tag{14.1}$$

Off support, the null-space quadratic sends the raw and renormalized ambient values to $-\infty$. Tests cover both cases and use multiple values of $\varepsilon$ to verify convergence.

For the analytic-derivative chapter set, the current audit record is:

- the score, Hessian, structural-provider, Cholesky, and QR chapters are treated as one connected derivative contract rather than isolated formulas;
- local symbolic tools are useful for context lookup and algebraic smoke tests, but they do not replace manual proof of noncommutative matrix calculus steps;
- automated literature helpers may fail to summarize the exact SVD/eigen derivative-validation cluster relevant to a given implementation branch;
- readiness for downstream use remains a test-supported claim, not a prose claim.

# Part IV

# Nonlinear Filtering

# Chapter 15

# Extended Kalman Filtering

The extended Kalman filter is the most direct nonlinear extension of the linear Gaussian spine in Chapter 5. It replaces nonlinear transition and observation maps by local linear approximations and then applies the same prediction-error likelihood form to the resulting Gaussian approximation. In BayesFilter this is an approximate likelihood backend, not an exact replacement for the Kalman filter except in the affine-Gaussian special case.

## 15.1 Local Linearization

Consider the nonlinear contract

$$x_{t+1} = f_\theta(x_t) + u_{t+1}, \qquad u_{t+1} \sim \mathcal{N}(0, Q_\theta), \tag{15.1}$$

$$y_t = h_\theta(x_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta). \tag{15.2}$$

Let

$$F_t = \left.\frac{\partial f_\theta(x)}{\partial x^\top}\right|_{x=m_{t-1|t-1}}, \qquad H_t = \left.\frac{\partial h_\theta(x)}{\partial x^\top}\right|_{x=m_{t|t-1}}.$$

The EKF prediction and update are

$$m_{t|t-1} = f_\theta(m_{t-1|t-1}), \tag{15.3}$$

$$P_{t|t-1} = F_t P_{t-1|t-1} F_t^\top + Q_\theta, \tag{15.4}$$

$$v_t = y_t - h_\theta(m_{t|t-1}), \tag{15.5}$$

$$S_t = H_t P_{t|t-1} H_t^\top + R_\theta, \tag{15.6}$$

$$K_t = P_{t|t-1} H_t^\top S_t^{-1}, \tag{15.7}$$

$$m_{t|t} = m_{t|t-1} + K_t v_t. \tag{15.8}$$

The covariance may be updated in simplified form or in a Joseph-style linearized form. The backend must state which form defines the numerical recursion, because this choice affects finite-arithmetic behavior even when the first-order algebra is equivalent.

## 15.2 Likelihood Status

The EKF log likelihood uses the same innovation expression as (5.10), but the innovations and innovation covariances come from local linearization. Therefore the EKF target is

an approximate target unless $f_\theta$ and $h_\theta$ are affine on the region reached by the filter. An HMC chain using the EKF samples from the posterior induced by that approximate likelihood and prior, not from the exact nonlinear state-space posterior.

For BayesFilter documentation and tests, the EKF should be used in three roles:

(i) a baseline nonlinear Gaussian approximation;

(ii) a regression oracle in cases where the nonlinear maps collapse to affine maps and the exact Kalman likelihood is available;

(iii) a diagnostic comparison for sigma-point and particle backends.

## 15.3 HMC Contract

An EKF backend is HMC-eligible only after it declares:

- how the Jacobians $F_t$ and $H_t$ are computed;
- whether gradients with respect to $\theta$ are tape-based, analytic, or custom;
- what happens when $S_t$ is singular, non-finite, or numerically indefinite;
- whether all loop lengths, masks, and state dimensions remain static under compilation;
- which parity tests compare the EKF to exact linear Gaussian cases.

The older nonlinear-filtering chapters in the source projects motivate this contract, but BayesFilter tightens it for HMC: differentiability and finite failure behavior are part of the target definition, not implementation afterthoughts.

# Chapter 16

# Sigma-Point Filters

Sigma-point filters replace local linearization with deterministic quadrature. They propagate a finite set of points through nonlinear maps and reconstruct approximate Gaussian moments from weighted averages. This makes them more accurate than the EKF for some nonlinear transformations, while preserving the Gaussian filtering closure used by the prediction-error likelihood. The price is that the likelihood remains approximate and the derivative path becomes more complicated.

## 16.1 Moment-Matching Interface

At each time step, a sigma-point backend starts from a Gaussian approximation

$$x_t \mid y_{1:t-1} \approx \mathcal{N}(m_{t|t-1}, P_{t|t-1}).$$

A deterministic rule constructs points $\chi_{t|t-1}^{(j)}$ and weights $w_j^{(m)}, w_j^{(c)}$. Passing the points through the observation map gives

$$z_t^{(j)} = h_\theta(\chi_{t|t-1}^{(j)}).$$

The backend then forms approximate predicted observations, innovation covariances, and cross covariances:

$$\bar{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{16.1}$$

$$P_{zz,t} = \sum_j w_j^{(c)} (z_t^{(j)} - \bar{z}_t)(z_t^{(j)} - \bar{z}_t)^\top + R_\theta, \tag{16.2}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (\chi_{t|t-1}^{(j)} - m_{t|t-1})(z_t^{(j)} - \bar{z}_t)^\top. \tag{16.3}$$

The gain is $K_t = P_{xz,t} P_{zz,t}^{-1}$ and the innovation is $v_t = y_t - \bar{z}_t$. The same Gaussian innovation likelihood formula is then used, with $P_{zz,t}$ replacing $S_t$.

## 16.2 Conjugate Unscented Transform Rules

The ordinary unscented transform is not the only deterministic quadrature rule that can be used in the interface above. The conjugate unscented transform (CUT) of Adurthi

et al. [2012a,b] constructs higher-order Gaussian point sets by combining symmetric conjugate point families. The experimental file

`/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py`

implements a deprecated TensorFlow square-root CUT4 demonstration for a coordinated-turn tracking model. It is not wired into the current BayesFilter filter hierarchy, but it is a useful design reference because it separates the sigma-point rule from the square-root covariance backend.

For a standard normal variable $Z \in \mathbb{R}^n$, the CUT4-G rule used there has

$$N_{\mathrm{CUT4}} = 2n + 2^n$$

points: $2n$ axis points and $2^n$ hypercube-corner points. In the convention used by the demo, for $n \geq 3$,

$$r_a^2 = \frac{n+2}{2}, \qquad r_c^2 = \frac{n+2}{n-2},$$

with weights

$$w_a = \frac{4}{(n+2)^2}, \qquad w_c = \frac{(n-2)^2}{2^n(n+2)^2}.$$

More explicitly, define

$$\mathcal{A} = \{\pm r_a e_i : i = 1, \ldots, n\}, \qquad \mathcal{C} = \{r_c s : s \in \{-1, +1\}^n\}.$$

The CUT4-G quadrature operator is

$$\mathcal{Q}_{\mathrm{CUT4}}[\varphi] = w_a \sum_{\xi \in \mathcal{A}} \varphi(\xi) + w_c \sum_{\xi \in \mathcal{C}} \varphi(\xi). \tag{16.4}$$

**Proposition 16.1** (CUT4-G Gaussian moment exactness)**.** *For $n \geq 3$, the point set (16.4) integrates every standard-normal monomial in $Z \in \mathbb{R}^n$ of total degree at most five exactly. Equivalently, it is a degree-five Gaussian rule.*

*Proof.* Both $\mathcal{A}$ and $\mathcal{C}$ are centrally symmetric and sign-symmetric in every coordinate. Hence every monomial with at least one odd coordinate exponent has zero quadrature value, matching the standard normal. This covers all odd total degrees through five and also the mixed even-odd monomials.

It remains to check total degrees 0, 2, and 4. The total mass is

$$2nw_a + 2^n w_c = 1.$$

For a coordinate $i$,

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^2] = 2w_a r_a^2 + 2^n w_c r_c^2 = 1 = \mathbb{E}[Z_i^2],$$

and, for $i \neq j$, $\mathcal{Q}_{\mathrm{CUT4}}[z_i z_j] = 0 = \mathbb{E}[Z_i Z_j]$ by sign symmetry. The fourth marginal moment is

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^4] = 2w_a r_a^4 + 2^n w_c r_c^4 = 3 = \mathbb{E}[Z_i^4],$$

while the mixed fourth moment is

$$\mathcal{Q}_{\mathrm{CUT4}}[z_i^2 z_j^2] = 2^n w_c r_c^4 = 1 = \mathbb{E}[Z_i^2 Z_j^2], \qquad i \neq j.$$

All remaining fourth-degree monomials contain an odd coordinate exponent and therefore vanish on both sides. These identities exhaust the Gaussian monomials of total degree at most five, so the rule has degree-five exactness. □

Thus the rule is symmetric, has positive weights, and integrates Gaussian polynomials through degree five. By contrast, the usual $2n+1$ unscented rule is commonly described as third-degree accurate for Gaussian moment propagation. The extra corner points are what allow CUT4-G to match cross moments such as $\mathbb{E}[Z_i^2 Z_j^2]$ that axis-only rules do not represent in the same way.

The price is exponential growth in the corner set. CUT4-G uses 42 points at $n = 5$, 1044 points at $n = 10$, and becomes unattractive in large structural states unless the quadrature is applied to a low-dimensional stochastic block. This reinforces the structural lesson of Chapter 19: higher-order quadrature should be spent on the declared stochastic integration coordinates, not on deterministic-completion directions that add no stochastic law.

## 16.3 Approximation Boundary

BayesFilter should label sigma-point filters as approximate likelihood backends unless a special model structure supplies an exactness argument. The source SR-UKF material contains model-specific claims about quadratic observation equations and pruned DSGE structure. Those claims are useful case-study material, but they should not be promoted to a general BayesFilter theorem without a separate derivation in BayesFilter notation and a regression test against an exact reference.

For HMC, this distinction matters. If the sigma-point likelihood is used in the target, the chain targets the sigma-point posterior. If the sigma-point filter is only a surrogate for an exact nonlinear likelihood, then a correction or delayed-acceptance policy is needed before exact-posterior claims are made.

## 16.4 Derivative and Compilation Requirements

Sigma-point HMC requires differentiating through:

- the covariance factor used to generate points;
- the nonlinear transition and observation maps;
- the weighted covariance assembly;
- linear solves or factorizations for $P_{zz,t}$;
- any PSD projection, jitter, clipping, or fallback branch.

The derivative provider must return the derivative of the same approximate log likelihood that supplies the Metropolis value. Static shapes are also non-negotiable: the number of sigma points, state dimension, observed-data mask shape, and scan length must not vary inside the compiled transition.

## 16.5 Evidence Ladder

The minimum evidence ladder for a sigma-point backend is:

(i) exact recovery on affine Gaussian systems against the Kalman backend;

(ii) finite value and finite gradient on controlled nonlinear systems;

(iii) eager/compiled parity where a compiled sampler is claimed;

(iv) comparison to a denser quadrature, long simulation, or trusted reference on low-dimensional nonlinear examples;

(v) HMC diagnostics that remain explicitly separated from convergence proof.

This ladder is the consolidation of the source-project SVD/HMC validation plans, not a completed BayesFilter certification result.

CUT backends should pass the same ladder plus one rule-specific check: for standard-normal polynomial test functions up to the declared degree, the implemented points and weights should reproduce the analytic Gaussian moments before the points are used in a filter. This catches mistakes in point enumeration, weights, dimension restrictions, and transformed-coordinate scaling before nonlinear filtering errors obscure the quadrature issue.

# Chapter 17

# Square-Root Sigma-Point Filters

Square-root sigma-point filters propagate a factor of the covariance rather than the covariance matrix itself. Their purpose is numerical: avoid avoidable loss of symmetry and positive semi-definiteness, reduce solve instability, and make the filtering recursion easier to monitor. In BayesFilter, square-root bookkeeping is a backend discipline. It does not by itself make an approximate likelihood exact or an autodiff gradient safe for HMC.

## 17.1 Pruned Second-Order Models: Executable API

For a model with separate first- and second-order state components, use `TFPrunedSRUKFModel` and `tf_pruned_srukf_filter` from `bayesfilter.nonlinear`. The parameterized entry point `make_pruned_srukf_value_and_score` compiles the likelihood and its total parameter derivative. The field shapes, initialization conventions, and usage example are in `docs/reference/pruned-direct-factor-srukf.md`.

Let $f_t$ and $s_t$ denote first- and second-order deviations, and let $e_t$ be the innovation. With caller-supplied coefficients the recursion is

$$f_t = Af_{t-1} + Be_t, \qquad s_t = As_{t-1} + c + \tfrac{1}{2}H[(f_{t-1}, e_t), (f_{t-1}, e_t)].$$

Both state components remain random and are conditioned jointly on the observations. The quadratic form uses only first-order deviations and innovations. Its full Hessian includes state–innovation cross terms, with the usual one-half convention; the caller supplies the second-order forcing $c$. The observation equation is

$$y_t = d + C(f_t + s_t) + \tfrac{1}{2}J[f_t, f_t] + v_t.$$

This is a coefficient contract, not a solver for the coefficients. In particular, Taylor-expanding a learned transition defines a different model from an independently derived equilibrium perturbation solution.

The initial joint factor may have zero rows for a deterministic second-order component. Scaled Householder QR of the joint observation/state residual stack propagates those factors without adding noise to fill the null space. If the lower factor is

$$L = \begin{bmatrix} L_y & 0 \\ L_{xy} & L_f \end{bmatrix},$$

then $L_f L_f^\top$ is the conditional state covariance and the mean correction is $L_{xy}L_y^{-1}(y_t - \bar{y}_t)$. Only $L_y$ must be nonsingular. The filter returns numerical validity flags even inside XLA;

singular innovation laws and nonfinite values cannot be accepted silently. Derivatives apply on a fixed structural-rank branch, not at arbitrary rank changes.

The positive-weight unscented rule uses $\alpha = 1$, $\beta = 2$, $\kappa = 0$ on the fixed augmented dimension. It does not integrate all fourth moments exactly. The API therefore computes an approximate Gaussian likelihood; passing factor and gradient tests does not establish posterior convergence.

## 17.2 Backend Contract

A square-root backend stores a factor $C_t$ such that $P_t = C_t C_t^\top$ or an equivalent factorization. The factor may be Cholesky, QR-derived, SVD/eigen-derived, or another auditable representation. The backend contract must state:

- the factor orientation and reconstruction identity;
- the predict update and measurement update used for the factor;
- whether covariance matrices are ever reconstructed;
- how rank loss, jitter, and PSD projection are handled;
- which operations define the differentiable target path.

The source SR-UKF material is useful here precisely because it exposed a maintenance risk: a method can be called square-root while still reconstructing full covariances and applying simplified covariance updates internally. The BayesFilter monograph should therefore describe the actual implemented recursion, not only the label attached to the filter.

## 17.3 Factor-Propagating SR-UKF Score Contract

This section defines the generic square-root UKF score object used by BayesFilter when the phrase "factor-propagating SR-UKF analytical score" is used. It is not the historical SVD sigma-point score of Chapter 18. It is also not the strict-SPD principal matrix-square-root derivative branch in Section 18.6. The backend below propagates a declared factor and differentiates the declared factor branch. It must not differentiate an eigendecomposition or a principal square-root map as a hidden substitute for the factor recursion.

**Audit assumptions.** All derivatives in this section are local derivatives on one fixed branch of the declared factor recursion. The branch fixes the sigma-point rule, weights, offsets, factor ranks, column order, QR pivot policy, sign convention, and the ordered update/downdate sequence. If any of those choices changes, the derivative below is not reused. For a step with state dimension $d_x$, observation dimension $d_y$, sigma variable dimension $n_a$, and factor rank $r_a$, the objects have shapes

$$\mu_a \in \mathbb{R}^{n_a}, \quad C_a \in \mathbb{R}^{n_a \times r_a}, \quad x^{(j)} \in \mathbb{R}^{d_x}, \quad z^{(j)} \in \mathbb{R}^{d_y},$$

$$P_{xx,\star} \in \mathbb{R}^{d_x \times d_x}, \quad S_\star \in \mathbb{R}^{d_y \times d_y}, \quad P_{xz,\star} \in \mathbb{R}^{d_x \times d_y}, \quad K \in \mathbb{R}^{d_x \times d_y}, \quad v, w, q \in \mathbb{R}^{d_y}.$$

For the ordinary Gaussian innovation likelihood, the audited branch requires $S_\star = C_S C_S^\top$ to be symmetric positive definite on the observed coordinates. The implementation may evaluate $S_\star^{-1} b$ only through declared solves and must report solve residual and conditioning diagnostics. The trace term in the score is square because $\dot{S}_\star^{(i)} \in \mathbb{R}^{d_y \times d_y}$.

### 17.3.1 Sigma-Point Placement From A Declared Factor

Let the sigma-point variable for one filtering step be

$$a_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_a, C_a C_a^\top), \qquad C_a \in \mathbb{R}^{n_a \times r_a}.$$

The factor may be rectangular. Rank deficiency of the represented law is therefore allowed on a fixed-rank branch; no inverse of $C_a$, no Cholesky factor of $C_a C_a^\top$, and no principal square root of $C_a C_a^\top$ is part of the placement contract. For fixed standardized offsets $\xi^{(j)} \in \mathbb{R}^{r_a}$ and weights $(w_j^{(m)}, w_j^{(c)})$, the SR-UKF placement is

$$a_t^{(j)} = \mu_a + C_a \xi^{(j)}. \tag{17.1}$$

The offsets and weights define the UKF rule. The factor $C_a$ defines the gauge in which that rule is placed.

For a parameter component $\theta_i$, the first derivative of a placed point on a fixed factor branch is

$$\dot{a}_t^{(j,i)} = \dot{\mu}_a^{(i)} + \dot{C}_a^{(i)} \xi^{(j)}. \tag{17.2}$$

The offsets are not differentiated. A branch that changes offsets, rank, pivoting, jitter, sign convention, or active support is a new branch and must fail closed for analytical score admission unless that branch has its own reviewed derivative contract.

### 17.3.2 Moment Factors And Signed Square-Root Updates

Let $g_\theta$ be any smooth propagation map applied to the placed points, and write $u^{(j)} = g_\theta(a_t^{(j)})$. The weighted mean and centered deviations are

$$\bar{u} = \sum_j w_j^{(m)} u^{(j)}, \qquad e^{(j)} = u^{(j)} - \bar{u}.$$

For a covariance object that may also include an additive factor $GG^\top$, define positive and negative weighted stacks

$$E_+ = \left[ \{ \sqrt{w_j^{(c)}}\, e^{(j)} : w_j^{(c)} > 0 \} \quad G \right], \qquad E_- = \left[ \{ \sqrt{-w_j^{(c)}}\, e^{(j)} : w_j^{(c)} < 0 \} \right].$$

A factor-propagating SR-UKF backend chooses a declared branch $\mathcal{B}$ consisting of a positive-stack QR factorization followed by ordered signed update/downdate primitives for the columns of $E_-$. The implemented factor is

$$C_u = \mathrm{SR}_{\mathcal{B}}(E_+, E_-), \qquad C_u C_u^\top = A_{u,\star} = E_+ E_+^\top - E_- E_-^\top. \tag{17.3}$$

The subscript $\star$ records the implemented branch. If a downdate fails, a pivot changes, a sign convention changes, or a stabilizing floor is activated, then the branch has changed. Such an event is a diagnostic stop for the analytical score branch unless a separate reviewed branch states the new implemented covariance $A_{u,\star}$ and its derivative.

The first derivative of the propagated point is

$$\dot{u}^{(j,i)} = D_a g_\theta(a_t^{(j)})\, \dot{a}_t^{(j,i)} + \partial_i g_\theta(a_t^{(j)}), \tag{17.4}$$

and the mean and covariance-object derivatives are

$$\dot{\bar{u}}^{(i)} = \sum_j w_j^{(m)} \dot{u}^{(j,i)}, \tag{17.5}$$

$$\dot{A}_{u,\star}^{(i)} = \sum_j w_j^{(c)} \left( \dot{e}^{(j,i)} e^{(j)\top} + e^{(j)} \dot{e}^{(j,i)\top} \right) + \dot{G}^{(i)} G^\top + G \dot{G}^{(i)\top}, \tag{17.6}$$

where $\dot{e}^{(j,i)} = \dot{u}^{(j,i)} - \dot{\bar{u}}^{(i)}$. The factor derivative is the derivative of the same declared QR/update branch and must reconstruct the implemented covariance derivative:

$$\dot{A}_{u,\star}^{(i)} = \dot{C}_u^{(i)} C_u^\top + C_u \dot{C}_u^{(i)\top}. \tag{17.7}$$

Equation (17.7) is the local audit equation for the SR-UKF factor branch. Chapter 12 supplies the QR and ordinary Cholesky smooth-branch identities used by value and reconstruction audits. The direct block-QR and rectangular QR derivatives are derived below for the nonnegative-weight DZ5 route. The monograph does *not* derive a complete ordered signed rank-one update/downdate derivative for arbitrary negative-weight rules. Such signed branches remain outside the admitted default score contract unless a separate reviewed primitive and reconstruction test are supplied.

### 17.3.3 Innovation Factor, Likelihood, And Score

Apply the moment construction to the predicted state points $x^{(j)}$ and observation points $z^{(j)}$. This produces

$$\bar{x}, \qquad \bar{z}, \qquad P_{xx,\star}, \qquad S_\star, \qquad P_{xz,\star}.$$

The weighted state covariance object itself may be written in the same signed stack notation as

$$P_{xx,\star} = E_{x,+} E_{x,+}^\top - E_{x,-} E_{x,-}^\top, \tag{17.8}$$

where $E_{x,+}$ collects the positive covariance-weight state deviations and $E_{x,-}$ collects the negative covariance-weight state deviations. The innovation factor is the declared SR factor of the observation covariance:

$$C_S C_S^\top = S_\star. \tag{17.9}$$

On the same fixed factor branch, its first derivative reconstructs the observation covariance derivative:

$$\dot{S}_\star^{(i)} = \dot{C}_S^{(i)} C_S^\top + C_S \dot{C}_S^{(i)\top}. \tag{17.10}$$

For the ordinary Gaussian innovation likelihood, $S_\star$ must be full rank on the observed coordinates. If it is not, the backend must either use a separately reviewed singular-density branch or fail closed; it must not add an unreported nugget and call the result the same likelihood.

Let $v = y_t - \bar{z}$. The solve-form likelihood uses triangular or factor solves from the declared innovation branch:

$$C_S q = v, \qquad C_S^\top w = q, \qquad \ell_t = -\frac{1}{2}\left[ d_y \log(2\pi) + 2\log|\det C_S| + q^\top q \right]. \tag{17.11}$$

Equivalently, $S_\star w = v$. The first derivative of the innovation is $\dot{v}^{(i)} = -\dot{\tilde{z}}^{(i)}$. Differentiating

$$-\frac{1}{2}\{\log\det S_\star + v^\top S_\star^{-1} v\}$$

on the same SPD solve branch uses $\partial_i \log\det S_\star = \mathrm{tr}(S_\star^{-1}\dot{S}_\star^{(i)})$ and $\partial_i S_\star^{-1} = -S_\star^{-1}\dot{S}_\star^{(i)}S_\star^{-1}$. With $w = S_\star^{-1}v$, the analytical score contribution is

$$\dot{\ell}_t^{(i)} = -\frac{1}{2}\left[\mathrm{tr}(S_\star^{-1}\dot{S}_\star^{(i)}) + 2\dot{v}^{(i)\top}w - w^\top \dot{S}_\star^{(i)} w\right]. \tag{17.12}$$

The trace term may be evaluated through the innovation factor branch, but the object being differentiated is $S_\star = C_S C_S^\top$ from (17.9). A derivative of a principal matrix-square-root map is not the generic SR-UKF score route defined here.

### 17.3.4 Filtered State And Propagated Factor Derivatives

The UKF gain and filtered mean are

$$K = P_{xz,\star}S_\star^{-1}, \qquad \mu_{t|t} = \bar{x} + Kv.$$

Equivalently, $KS_\star = P_{xz,\star}$. Differentiating this solve equation on the same invertible innovation branch gives the gain derivative in solve form:

$$\dot{K}^{(i)}S_\star = \dot{P}_{xz,\star}^{(i)} - K\dot{S}_\star^{(i)}, \tag{17.13}$$

$$\dot{\mu}_{t|t}^{(i)} = \dot{\bar{x}}^{(i)} + \dot{K}^{(i)}v + K\dot{v}^{(i)}. \tag{17.14}$$

The filtered covariance object is

$$P_{t|t,\star} = P_{xx,\star} - KS_\star K^\top.$$

The filtered square-root factor $C_{t|t}$ is produced by the same declared signed factor branch. Let the predicted state covariance object be written in signed-stack form

$$P_{xx,\star} = E_{x,+}E_{x,+}^\top - E_{x,-}E_{x,-}^\top,$$

where $E_{x,+}$ collects the positive covariance-weight state deviations and $E_{x,-}$ collects the negative covariance-weight state deviations. Let $C_{KS}$ denote a factor whose covariance is $KS_\star K^\top$. Then the filtered branch must use the complete positive and negative stacks:

$$C_{t|t} = \mathrm{SR}_{\mathcal{B}_t}\left(E_{x,+},\, [E_{x,-} \;\; C_{KS}]\right), \qquad C_{t|t}C_{t|t}^\top = P_{t|t,\star}, \tag{17.15}$$

so that the signed branch reconstructs

$$E_{x,+}E_{x,+}^\top - E_{x,-}E_{x,-}^\top - C_{KS}C_{KS}^\top = P_{xx,\star} - KS_\star K^\top.$$

When all covariance weights are nonnegative, $E_{x,-}$ is empty and this reduces to the usual positive-stack factor update. On a negative-weight branch, omitting $E_{x,-}$ changes the covariance target and is therefore not the same filtered object. The first derivative must reconstruct the implemented branch:

$$\dot{P}_{t|t,\star}^{(i)} = \dot{C}_{t|t}^{(i)}C_{t|t}^\top + C_{t|t}\dot{C}_{t|t}^{(i)\top}. \tag{17.16}$$

Equations (17.14) and (17.16) are the state handoff contract for a multi-step analytical score recursion.

### 17.3.5 Admission Boundary

An implementation may be called an admitted factor-propagating SR-UKF analytical score only if its diagnostics identify the factor branch, update/downdate events, reconstruction residuals in (17.7) and (17.16), innovation solve residuals, and score provenance. The admitted provenance must not contain an autodiff tape fallback, the historical SVD/eigenderivative score route, or the strict-SPD principal-square-root derivative route. Those routes can remain diagnostic comparators, but they do not satisfy the generic SR-UKF score contract in this section.

## 17.4 HMC-Relevant Risks

For HMC, the relevant question is not only whether the factor recursion keeps $P_t$ positive semi-definite in floating point. The sampler also needs a stable derivative of the scalar log target. Cholesky, QR, and SVD factors have different derivative behavior near rank loss or repeated singular values. A backend that is robust for value evaluation can still be fragile as a raw tape-gradient path.

BayesFilter should therefore classify square-root sigma-point evidence in two columns:

(i) value-recursion evidence, such as finite likelihoods, factor reconstruction residuals, and affine-Gaussian parity;

(ii) derivative evidence, such as finite gradients, AD/custom-gradient parity, spectral-gap telemetry, and HMC leapfrog stability.

Only the second column is directly relevant to HMC production readiness.

## 17.5 When to Prefer Square-Root Forms

Square-root forms are natural when observation noise is small, latent covariances are nearly singular, or long scans accumulate asymmetry. They are also useful for large LGSSMs where factor diagnostics expose numerical problems earlier than a final log likelihood does. They should not be treated as a universal answer to nonlinear filtering: if the moment closure is poor, the filter is still approximating the wrong target; if the derivative of the factorization is unstable, HMC can still fail.

## 17.6 Implemented Direct-QR Recursions

BayesFilter's default full-rank route is `direct_qr_block_conditional`. For a predicted-state residual stack $X$ and an observation residual/noise stack $Y$, form only the joint direct stack

$$A = \begin{bmatrix} Y \\ X \end{bmatrix}, \qquad A^\top = Q \begin{bmatrix} R_{yy} & R_{yx} \\ 0 & R_{xx} \end{bmatrix}.$$

With positive QR diagonals,

$$L_y = R_{yy}^\top, \qquad K = R_{yx}^\top R_{yy}^{-\top}, \qquad L_f = R_{xx}^\top. \tag{17.17}$$

Thus $L_y L_y^\top$ is the innovation covariance and $L_f L_f^\top$ is the conditional state covariance. The runtime forms neither covariance, no Schur complement, and performs no SVD or

eigendecomposition. The same QR derivative produces derivatives of all three objects. A scale-aware relative pivot floor of $10^{-12}$ rejects a numerically unresolved full-rank chart.

Exact singular branches use the rectangular construction in Section 12.7. Rank discovery may use SVD on the direct stack before the differentiated run. The score run freezes rank, row-pivot permutations, positive QR signs, support identity, and tolerances, then uses QR only. Sigma points are generated in the retained latent factor width, not padded ambient zero directions. The update uses

$$Y = URV^\top, \qquad K = XVR^{-1}U^\top, \qquad X_f = X(I - VV^\top),$$

where these quantities are obtained from fixed QR charts. A second fixed rectangular QR chart factors $X_f$. The likelihood is the affine-support density (12.11).

The score is local. If rank, pivot permutation, sign, chart residual, observation support, or retained support changes, diagnostics set `score_valid=false` and return a nonfinite score sentinel. The value-only SVD discovery route is never a hidden score fallback. The current DZ5 rule has nonnegative covariance weights; this fixed-support admission does not extend to arbitrary signed-weight rules or across their PSD boundaries.

## 17.7 CUT With Square-Root Backends

The conjugate unscented transform in Section 16.2 is a point-and-weight rule. It can be combined with any square-root factor backend that maps standardized points $\xi^{(j)}$ into state points

$$\chi^{(j)} = m + C\xi^{(j)}, \qquad CC^\top = P_\star.$$

The experimental `/home/chakwong/python/src/dsge_hmc/filters/CUTSRUKF.py` module uses positive CUT4-G weights together with QR square-root covariance updates. It also keeps rank-deficient process noise as a rectangular factor $G$ with $Q = GG^\top$, avoiding an invalid Cholesky factorization of a singular $Q$.

An SVD/eigen factor can be used in the same location:

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2}.$$

This may make value evaluation more robust when $P$ is singular or nearly singular. It does not change CUT's quadrature degree; it changes the numerical factor used to place the CUT points. Therefore a stable CUT-SVD filter must report the same factor diagnostics as an SVD sigma-point filter: the covariance actually reconstructed, active floors or rank truncations, spectral gaps, sign or order choices, and fallback branches. In short, CUT can be combined with SVD, but the result is a higher-order quadrature rule on top of an SVD factor policy, not a new theorem that removes spectral derivative risk.

# Chapter 18

# SVD Sigma-Point Filters

The SVD sigma-point filter is the most important nonlinear-filtering lesson from the recent source-project debugging work. It improved value-side robustness by using spectral covariance factors and explicit finite-arithmetic guards, but it also exposed why raw autodiff through spectral decompositions is not a comfortable industrial HMC default. BayesFilter should preserve both lessons.

## 18.1 What the Source Audit Established

The math/code audit recorded in the source-project SVD filter audit result from 2026-04-25 established the following bounded facts:

- the linear SVD Kalman path uses a Joseph-style covariance update;
- the nonlinear sigma-point path uses the simplified update

$$P^+ = P^- - K P_{zz} K^\top;$$

- PSD projection occurs after the candidate covariance is formed, so it cannot prevent overflow or non-finite values during the formation of $P^+$;
- the conditioning guard with threshold `_SVD_CONDITIONING_MAX_ABS = 1e100` is an engineering finite-arithmetic guard, not a mathematically derived posterior threshold;
- prior short HMC diagnostics were numerical-path regressions, not convergence evidence.

These are implementation facts, not general theorems about SVD filtering. BayesFilter should cite them as project evidence and should avoid converting them into unsupported production-readiness claims.

## 18.2 Spectral Differentiation Risk

The Phase 2B literature gate accepted the Matrix Backprop source for the bounded claim that gradients through SVD or eigendecomposition contain terms with denominators involving singular-value or eigenvalue gaps. When two singular values or eigenvalues

are equal, the decomposition is not uniquely differentiable; when they are merely close, derivative terms can become large and numerically unstable.

This is directly relevant to HMC. A large DSGE or NAWM-sized model can visit parameter regions where covariance factors are ill conditioned, nearly rank deficient, or spectrally clustered. A value-side SVD factor may still return a finite covariance approximation, while the tape gradient through the spectral basis becomes noisy, discontinuous, or non-finite. Larger dimensions make close spectral gaps more likely in practice, so BayesFilter should require gap telemetry and derivative validation before using SVD sigma-point gradients inside production HMC.

## 18.3 Analytic Score and Hessian Recursion

The SVD sigma-point derivative problem has two layers. The likelihood layer is not new: after the sigma-point update has produced

$$v_t, \quad S_t, \quad \dot{v}_t^{(i)}, \quad \dot{S}_t^{(i)}, \quad \ddot{v}_t^{(ij)}, \quad \ddot{S}_t^{(ij)},$$

the score and Hessian are exactly the solve-form expressions in Chapter 10. The new work is the sigma-point layer: differentiate the point cloud, the nonlinear maps, and the weighted moment assemblies that generate these innovation objects.

Thus this chapter plugs into the analytic-derivative stack as follows. Chapter 11 supplies derivatives of structural maps and parameter transforms; Chapter 12 defines what differentiated SVD factors must reconstruct; Chapter 13 requires the SVD score to differentiate the same implemented scalar value; and Chapter 14 supplies the finite-difference, backend-parity, branch, and HMC-gating tests.

Let a Gaussian input at some predict or update substep be

$$X \sim \mathcal{N}(m(\theta), P(\theta)).$$

Let an SVD/eigendecomposition factor be written

$$P = U\Lambda U^\top, \qquad C = U\Lambda_+^{1/2},$$

where $\Lambda_+$ denotes the eigenvalue policy actually used for point generation: active positive eigenvalues, a smooth floor, or a hard floor. A fixed sigma-point rule can be written as

$$\chi^{(r)} = m + C\xi^{(r)}, \qquad r = 0, \ldots, N-1, \tag{18.1}$$

where the standardized offsets $\xi^{(r)}$ and weights $w_r^{(m)}, w_r^{(c)}$ are fixed by the UKF, CKF, or another deterministic quadrature rule. For the UKF, for example, the nonzero offsets are $\pm\gamma e_a$. If tuning parameters such as $\alpha$ or $\kappa$ are estimated, the same formulas hold with additional $\dot{\xi}^{(r)}$, $\ddot{\xi}^{(r)}$, $\dot{w}_r$, and $\ddot{w}_r$ terms. BayesFilter's default derivative contract treats the rule as fixed.

With fixed quadrature offsets, the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \tag{18.2}$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \tag{18.3}$$

Thus all spectral difficulty enters through $\dot{C}$ and $\ddot{C}$; the remaining recursion is ordinary chain rule and product rule.

### 18.3.1 Map and Moment Derivatives

Let $g_\theta$ denote either the transition map in the predict step or the observation map in the update step, and define

$$\eta^{(r)} = g_\theta(\chi^{(r)}).$$

Using $D_x g_\theta$ for the Jacobian in the state argument, and $D^2_{xx} g_\theta[a, b]$ for the second directional derivative in directions $a, b$, the propagated point derivatives are

$$\dot\eta^{(r,i)} = D_x g_\theta(\chi^{(r)})\dot\chi^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \tag{18.4}$$

$$\begin{aligned}\ddot\eta^{(r,ij)} &= D_x g_\theta(\chi^{(r)})\ddot\chi^{(r,ij)} + D^2_{xx} g_\theta(\chi^{(r)})[\dot\chi^{(r,i)}, \dot\chi^{(r,j)}] \\ &\quad + D^2_{x\theta_i} g_\theta(\chi^{(r)})\dot\chi^{(r,j)} + D^2_{x\theta_j} g_\theta(\chi^{(r)})\dot\chi^{(r,i)} + \partial^2_{ij} g_\theta(\chi^{(r)}).\end{aligned} \tag{18.5}$$

These equations are the exact place where the structural model derivatives enter the SVD sigma-point score and Hessian.

The sigma-point mean and covariance of the transformed cloud are

$$\bar\eta = \sum_{r=0}^{N-1} w_r^{(m)} \eta^{(r)}, \tag{18.6}$$

$$M_\eta = \sum_{r=0}^{N-1} w_r^{(c)} \Delta_\eta^{(r)} \Delta_\eta^{(r)\top} + \Omega_\theta, \qquad \Delta_\eta^{(r)} = \eta^{(r)} - \bar\eta, \tag{18.7}$$

where $\Omega_\theta$ is the process-noise covariance in a predict step or the measurement-noise covariance in an update step. Their derivatives are

$$\dot{\bar\eta}^{(i)} = \sum_r w_r^{(m)} \dot\eta^{(r,i)}, \tag{18.8}$$

$$\ddot{\bar\eta}^{(ij)} = \sum_r w_r^{(m)} \ddot\eta^{(r,ij)}, \tag{18.9}$$

and, with $\dot\Delta_\eta^{(r,i)} = \dot\eta^{(r,i)} - \dot{\bar\eta}^{(i)}$,

$$\dot M_\eta^{(i)} = \sum_r w_r^{(c)} \left[\dot\Delta_\eta^{(r,i)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \dot\Delta_\eta^{(r,i)\top}\right] + \dot\Omega_\theta^{(i)}, \tag{18.10}$$

$$\ddot M_\eta^{(ij)} = \sum_r w_r^{(c)} \left[\ddot\Delta_\eta^{(r,ij)} \Delta_\eta^{(r)\top} + \Delta_\eta^{(r)} \ddot\Delta_\eta^{(r,ij)\top} + \dot\Delta_\eta^{(r,i)} \dot\Delta_\eta^{(r,j)\top} + \dot\Delta_\eta^{(r,j)} \dot\Delta_\eta^{(r,i)\top}\right] + \ddot\Omega_\theta^{(ij)}. \tag{18.11}$$

Equations (18.2)–(18.11) are the analytic derivative recursion for any sigma-point predict moment.

### 18.3.2 UKF Update Objects

At the observation update, take the input mean and covariance to be $m_t^- = m_{t|t-1}$ and $P_t^- = P_{t|t-1}$, construct points $\chi_t^{(r)}$ by (18.1), and propagate

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}).$$

Equations (18.8)–(18.11) give

$$\bar{z}_t, \quad S_t = \sum_r w_r^{(c)} (z_t^{(r)} - \bar{z}_t)(z_t^{(r)} - \bar{z}_t)^\top + R_\theta,$$

and their first and second derivatives. The innovation is

$$v_t = y_t - \bar{z}_t, \qquad \dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \qquad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}, \tag{18.12}$$

unless the data transformation itself depends on parameters.

The cross-covariance used in the Kalman gain is

$$P_{xz,t} = \sum_r w_r^{(c)} \Delta_x^{(r)} \Delta_z^{(r)\top}, \qquad \Delta_x^{(r)} = \chi_t^{(r)} - m_t^-, \qquad \Delta_z^{(r)} = z_t^{(r)} - \bar{z}_t. \tag{18.13}$$

Its first and second derivatives are

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[ \dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \tag{18.14}$$

$$\ddot{P}_{xz,t}^{(ij)} = \sum_r w_r^{(c)} \left[ \ddot{\Delta}_x^{(r,ij)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \ddot{\Delta}_z^{(r,ij)\top} + \dot{\Delta}_x^{(r,i)} \dot{\Delta}_z^{(r,j)\top} + \dot{\Delta}_x^{(r,j)} \dot{\Delta}_z^{(r,i)\top} \right]. \tag{18.15}$$

The gain is best differentiated as a solved matrix equation. Since $K_t = P_{xz,t} S_t^{-1}$, equivalently

$$S_t K_t^\top = P_{xz,t}^\top.$$

Therefore

$$\dot{K}_t^{(i)\top} = \mathcal{S}_{S_t} \left( \dot{P}_{xz,t}^{(i)\top} - \dot{S}_t^{(i)} K_t^\top \right), \tag{18.16}$$

$$\ddot{K}_t^{(ij)\top} = \mathcal{S}_{S_t} \left( \ddot{P}_{xz,t}^{(ij)\top} - \ddot{S}_t^{(ij)} K_t^\top - \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} - \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} \right). \tag{18.17}$$

Finally, the usual sigma-point update

$$m_t^+ = m_t^- + K_t v_t, \tag{18.18}$$

$$P_t^+ = P_t^- - K_t S_t K_t^\top \tag{18.19}$$

has derivatives

$$\dot{m}_t^{+(i)} = \dot{m}_t^{-(i)} + \dot{K}_t^{(i)} v_t + K_t \dot{v}_t^{(i)}, \tag{18.20}$$

$$\ddot{m}_t^{+(ij)} = \ddot{m}_t^{-(ij)} + \ddot{K}_t^{(ij)} v_t + \dot{K}_t^{(i)} \dot{v}_t^{(j)} + \dot{K}_t^{(j)} \dot{v}_t^{(i)} + K_t \ddot{v}_t^{(ij)}. \tag{18.21}$$

For the covariance, define

$$\dot{P}_t^{+(i)} = \dot{P}_t^{-(i)} - \dot{K}_t^{(i)} S_t K_t^\top - K_t \dot{S}_t^{(i)} K_t^\top - K_t S_t \dot{K}_t^{(i)\top}, \tag{18.22}$$

and

$$\begin{aligned} \ddot{P}_t^{+(ij)} = \ddot{P}_t^{-(ij)} &- \ddot{K}_t^{(ij)} S_t K_t^\top - \dot{K}_t^{(i)} \dot{S}_t^{(j)} K_t^\top - \dot{K}_t^{(i)} S_t \dot{K}_t^{(j)\top} \\ &- \dot{K}_t^{(j)} \dot{S}_t^{(i)} K_t^\top - K_t \ddot{S}_t^{(ij)} K_t^\top - K_t \dot{S}_t^{(i)} \dot{K}_t^{(j)\top} \\ &- \dot{K}_t^{(j)} S_t \dot{K}_t^{(i)\top} - K_t \dot{S}_t^{(j)} \dot{K}_t^{(i)\top} - K_t S_t \ddot{K}_t^{(ij)\top}. \end{aligned} \tag{18.23}$$

The next time step then refactors $P_t^+$ and differentiates the new SVD factor. This is where the spectral contract below enters the recursion again.

### 18.3.3 Likelihood Score and Hessian

The SVD sigma-point likelihood contribution is the Gaussian innovation likelihood

$$\ell_t = -\frac{1}{2}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right],$$

where $S_t$ and $v_t$ are the sigma-point objects above. Let $S_t w_t = v_t$. The score is

$$\partial_i \ell_t = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right], \tag{18.24}$$

and the Hessian is obtained by substituting the sigma-point $\dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ into (10.1)–(10.10). Thus the UKF/SVD Hessian is not a separate likelihood theorem. It is the Kalman solve-form Hessian applied to approximate moments generated by (18.1)–(18.23).

## 18.4 Structural Fixed-Support Score

Chapter 19 explains why a structural sigma-point filter places points on the pre-transition uncertainty variable

$$A_t = (x_{t-1}, \varepsilon_t), \qquad A_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_{a,t}, P_{a,t}), \tag{18.25}$$

and then completes the state pointwise:

$$A_t^{(r)} = \mu_{a,t} + C_{a,t}\xi^{(r)}, \qquad X_t^{(r)} = F_\theta(A_t^{(r)}), \qquad Z_t^{(r)} = h_\theta(X_t^{(r)}). \tag{18.26}$$

Equations (18.25)–(18.26) are the derivative version of Proposition 19.1. The filtered state is still $x_t$; the sigma-point variable is the pre-transition uncertainty whose push-forward generates the predictive law. This is the object that must be differentiated for deterministic-completion examples such as Model C in Chapter 53.

The subtle case is a fixed structural zero direction. Write

$$P_{a,t} = U_t \Lambda_t U_t^\top, \qquad \Lambda_t = \operatorname{diag}(\lambda_{1t}, \ldots, \lambda_{dt}),$$

and split the spectral indices into

$$\mathcal{I}_t^+ = \{a : \lambda_{at} > \tau\}, \qquad \mathcal{I}_t^0 = \{a : \lambda_{at} \le \tau\}.$$

The structural fixed-support branch assumes that the zero-support block is part of the model law, not a numerical floor:

$$u_b^\top \dot{P}_{a,t}^{(i)} u_c = 0 \qquad \text{whenever } b \in \mathcal{I}_t^0 \text{ or } c \in \mathcal{I}_t^0, \tag{18.27}$$

$$|\lambda_{at} - \lambda_{bt}| > g_{\min} \qquad \text{for every differentiated active pair.} \tag{18.28}$$

Here "differentiated active pair" means any pair that enters the derivative of an active factor column: active-active pairs and active-null pairs. Gaps between two null directions do not matter because their factor columns are identically zero on this branch.

The implemented factor is

$$C_{a,t} = \left[\sqrt{\lambda_{1t}}u_{1t} \quad \cdots \quad \sqrt{\lambda_{dt}}u_{dt}\right], \qquad \sqrt{\lambda_{at}}u_{at} = 0 \quad \text{for } a \in \mathcal{I}_t^0. \tag{18.29}$$

For an active column $a \in \mathcal{I}_t^+$,

$$\dot{u}_{at}^{(i)} = \sum_{b \neq a} u_{bt} \frac{u_{bt}^\top \dot{P}_{a,t}^{(i)} u_{at}}{\lambda_{at} - \lambda_{bt}}, \tag{18.30}$$

$$\dot{c}_{at}^{(i)} = \sqrt{\lambda_{at}}\, \dot{u}_{at}^{(i)} + \frac{u_{at}^\top \dot{P}_{a,t}^{(i)} u_{at}}{2\sqrt{\lambda_{at}}}\, u_{at}. \tag{18.31}$$

For a null column $a \in \mathcal{I}_t^0$,

$$c_{at} = 0, \qquad \dot{c}_{at}^{(i)} = 0. \tag{18.32}$$

Under (18.27), these column rules reconstruct the derivative of the same pre-transition covariance:

$$\dot{P}_{a,t}^{(i)} = \dot{C}_{a,t}^{(i)} C_{a,t}^\top + C_{a,t} \dot{C}_{a,t}^{(i)\top}. \tag{18.33}$$

The reconstruction target is not a nuggeted covariance. A positive floor that turns a structural zero direction into positive variance changes the law and must be reported as a different regularized branch.

With this fixed-support factor, the score recursion is the ordinary sigma-point chain rule. For a scalar parameter component $\theta_i$,

$$\dot{A}_t^{(r,i)} = \dot{\mu}_{a,t}^{(i)} + \dot{C}_{a,t}^{(i)} \xi^{(r)}, \tag{18.34}$$

$$\dot{X}_t^{(r,i)} = D_a F_\theta(A_t^{(r)}) \dot{A}_t^{(r,i)} + \partial_i F_\theta(A_t^{(r)}), \tag{18.35}$$

$$\dot{Z}_t^{(r,i)} = D_x h_\theta(X_t^{(r)}) \dot{X}_t^{(r,i)} + \partial_i h_\theta(X_t^{(r)}). \tag{18.36}$$

The moment derivatives are

$$\dot{\hat{x}}_t^{(i)} = \sum_r w_r^{(m)} \dot{X}_t^{(r,i)}, \qquad \dot{\hat{z}}_t^{(i)} = \sum_r w_r^{(m)} \dot{Z}_t^{(r,i)}, \tag{18.37}$$

$$\dot{S}_t^{(i)} = \sum_r w_r^{(c)} \left[ \dot{\Delta}_z^{(r,i)} \Delta_z^{(r)\top} + \Delta_z^{(r)} \dot{\Delta}_z^{(r,i)\top} \right] + \dot{R}_t^{(i)}, \tag{18.38}$$

$$\dot{P}_{xz,t}^{(i)} = \sum_r w_r^{(c)} \left[ \dot{\Delta}_x^{(r,i)} \Delta_z^{(r)\top} + \Delta_x^{(r)} \dot{\Delta}_z^{(r,i)\top} \right], \tag{18.39}$$

where

$$\Delta_x^{(r)} = X_t^{(r)} - \hat{x}_t, \qquad \Delta_z^{(r)} = Z_t^{(r)} - \hat{z}_t,$$

and dotted centered quantities subtract the corresponding dotted means. The innovation derivative remains

$$v_t = y_t - \hat{z}_t, \qquad \dot{v}_t^{(i)} = -\dot{\hat{z}}_t^{(i)}.$$

Therefore the per-period score is

$$\partial_i \ell_t = -\frac{1}{2} \left[ \operatorname{tr}(S_t^{-1} \dot{S}_t^{(i)}) + 2 \dot{v}_t^{(i)\top} S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t^{(i)} S_t^{-1} v_t \right]. \tag{18.40}$$

This sign convention is the direct derivative of the likelihood in (19.107); the term $\dot{v}_t = -\dot{\hat{z}}_t$ is where the observation mean derivative enters.

The diagnostics for this branch must report at least the fixed null count, the maximum fixed-null derivative residual in (18.27), the active spectral gap in (18.28), the factor reconstruction residual in (18.33), and the deterministic-completion residual of the propagated points. Passing these checks means that the score differentiates the implemented structural sigma-point likelihood. It does not certify a nonlinear Hessian, HMC target, or GPU/XLA scaling claim.

## 18.5 Spectral Factor Derivative Contract

The preceding section assumes $\dot{C}$ and $\ddot{C}$ are available. This subsection states what those derivatives mean for an SVD/eigen factor. Suppose first that $P(\theta)$ has a smooth simple spectrum and no active hard floor:

$$P = \sum_{a=1}^{n} \lambda_a u_a u_a^\top, \qquad \lambda_a > 0, \qquad \lambda_a \neq \lambda_b \ (a \neq b).$$

For a parameter component $\theta_i$,

$$\dot{\lambda}_a^{(i)} = u_a^\top \dot{P}^{(i)} u_a, \tag{18.41}$$

$$\dot{u}_a^{(i)} = \sum_{b \neq a} u_b \frac{u_b^\top \dot{P}^{(i)} u_a}{\lambda_a - \lambda_b}. \tag{18.42}$$

For the column factor $c_a = \sqrt{\lambda_a}\, u_a$,

$$\dot{c}_a^{(i)} = \sqrt{\lambda_a}\, \dot{u}_a^{(i)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} u_a. \tag{18.43}$$

Second derivatives can be written in the same eigenbasis. Define $A_a^{(i)} = \dot{P}^{(i)} - \dot{\lambda}_a^{(i)} I$ and $A_a^{(ij)} = \ddot{P}^{(ij)} - \ddot{\lambda}_a^{(ij)} I$. Then

$$\ddot{\lambda}_a^{(ij)} = u_a^\top \ddot{P}^{(ij)} u_a + 2 \sum_{b \neq a} \frac{(u_b^\top \dot{P}^{(i)} u_a)(u_b^\top \dot{P}^{(j)} u_a)}{\lambda_a - \lambda_b}. \tag{18.44}$$

The second derivative of the eigenvector is determined by differentiating the eigenequation and the normalization condition:

$$\ddot{u}_a^{(ij)} = -u_a\, \dot{u}_a^{(i)\top} \dot{u}_a^{(j)} + \sum_{b \neq a} u_b \frac{u_b^\top \left( A_a^{(ij)} u_a + A_a^{(i)} \dot{u}_a^{(j)} + A_a^{(j)} \dot{u}_a^{(i)} \right)}{\lambda_a - \lambda_b}. \tag{18.45}$$

Consequently

$$\begin{aligned} \ddot{c}_a^{(ij)} = {} & \sqrt{\lambda_a}\, \ddot{u}_a^{(ij)} + \frac{\dot{\lambda}_a^{(i)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(j)} + \frac{\dot{\lambda}_a^{(j)}}{2\sqrt{\lambda_a}} \dot{u}_a^{(i)} \\ & + \left( \frac{\ddot{\lambda}_a^{(ij)}}{2\sqrt{\lambda_a}} - \frac{\dot{\lambda}_a^{(i)} \dot{\lambda}_a^{(j)}}{4\lambda_a^{3/2}} \right) u_a. \end{aligned} \tag{18.46}$$

Equations (18.41)–(18.46) are the branch-local eigenderivative formulas for an eigenfactor covariance square root on a smooth simple-spectrum branch, before any active hard-floor or ordering/sign fallback is crossed.

The formulas also show why the derivative path is fragile. Eigenvector derivatives contain denominators $\lambda_a - \lambda_b$; the Hessian contains the same gaps again through $\dot{u}_a$ and through differentiation of the gap. This is the spectral-gap risk identified by matrix-backpropagation calculus [Ionescu et al., 2015]. If eigenvalues are repeated, individual eigenvectors are not uniquely differentiable. If a hard floor $\lambda_a^+ = \max(\lambda_a, \epsilon)$ is used, the

derivative is branch dependent and undefined at $\lambda_a = \epsilon$. A smooth floor $\lambda_a^+ = \phi_\epsilon(\lambda_a)$ replaces $\dot{\lambda}_a$ and $\ddot{\lambda}_a$ by

$$\dot{\lambda}_a^+ = \phi_\epsilon'(\lambda_a)\dot{\lambda}_a, \qquad \ddot{\lambda}_a^+ = \phi_\epsilon'(\lambda_a)\ddot{\lambda}_a + \phi_\epsilon''(\lambda_a)\dot{\lambda}_a^{(i)}\dot{\lambda}_a^{(j)},$$

but it does not remove the eigenvector gap denominators.

For this reason BayesFilter should validate SVD factor derivatives through reconstruction identities, not merely by checking that autodiff returns finite numbers. The reconstruction target is the covariance represented by the implemented factor,

$$P_+ = CC^\top = U\Lambda_+ U^\top,$$

not necessarily the pre-floor covariance $P$:

$$\dot{P}_+^{(i)} \stackrel{?}{=} \dot{C}^{(i)}C^\top + C\dot{C}^{(i)\top}, \tag{18.47}$$

$$\ddot{P}_+^{(ij)} \stackrel{?}{=} \ddot{C}^{(ij)}C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}. \tag{18.48}$$

The residuals in (18.47)–(18.48), the minimum spectral gap, the active-floor set, and any fallback branch are part of the derivative output contract.

## 18.6 Principal-Square-Root UKF Backend

The preceding eigenderivative formulas define the historical `tf_svd_ukf` branch. BayesFilter also has a distinct promoted strict-SPD backend, `tf_principal_sqrt_ukf`, whose factor is the principal matrix square root rather than a signed or ordered eigenvector column factor. The historical eigenderivative branch is retained for diagnostic comparison and bounded regression use, not as the HMC-facing production route on the promoted strict-SPD scope. The two names are backend contracts, not aliases.

On a strict-SPD branch, let

$$P \in \mathbb{S}^n_{++}, \qquad S = P^{1/2} = S^\top \succ 0, \qquad S^2 = P.$$

For a covariance perturbation $dP$, the Frechet derivative of the principal square root is the unique solution $dS$ of the Sylvester equation

$$S\,dS + dS\,S = dP. \tag{18.49}$$

Equivalently, for a parameter component $\theta_i$,

$$S\,\dot{S}^{(i)} + \dot{S}^{(i)}S = \dot{P}^{(i)}. \tag{18.50}$$

Because the eigenvalues of strict-SPD $S$ are positive, the linear Sylvester operator $X \mapsto SX + XS$ is invertible on all matrices. This is the reason the principal-square-root derivative does not contain the active repeated-positive-eigenvalue denominators that appear in (18.42). This statement is only a strict-SPD square-root derivative statement. It is not a rank-loss theorem, not a structural-null completion rule, and not an HMC convergence result.

For second derivatives on the same strict-SPD branch, differentiate (18.50) once more:

$$S\,\ddot{S}^{(ij)} + \ddot{S}^{(ij)}S = \ddot{P}^{(ij)} - \dot{S}^{(i)}\dot{S}^{(j)} - \dot{S}^{(j)}\dot{S}^{(i)}. \tag{18.51}$$

Thus the same Sylvester primitive supplies the Hessian-level square-root factor derivative once $\dot{S}^{(i)}$, $\dot{S}^{(j)}$, and $\ddot{P}^{(ij)}$ are available.

The same strict-SPD rule applies at the update stage to the innovation covariance. Let

$$S_t \in \mathbb{S}^{n_y}_{++}, \qquad R_t = S_t^{1/2} = R_t^\top \succ 0,$$

where $S_t$ is the innovation covariance assembled from the sigma-point moment recursion in (18.7)–(18.15). Then for each parameter component $\theta_i$ the promoted principal-square-root UKF contract uses

$$R_t \dot{R}_t^{(i)} + \dot{R}_t^{(i)} R_t = \dot{S}_t^{(i)} \tag{18.52}$$

rather than differentiating an eigendecomposition of $S_t$. Likewise the second-order update branch, when needed, is defined by

$$R_t \ddot{R}_t^{(ij)} + \ddot{R}_t^{(ij)} R_t = \ddot{S}_t^{(ij)} - \dot{R}_t^{(i)} \dot{R}_t^{(j)} - \dot{R}_t^{(j)} \dot{R}_t^{(i)}. \tag{18.53}$$

On this branch the repeated-positive-eigenvalue hazard is handled by the same strict-SPD Sylvester operator as in the placement stage; the backend must not reintroduce eigen-gap denominators by differentiating innovation eigenvectors or innovation eigenvalues under a separate SVD branch.

This update-side contract also keeps the Gaussian score and gain derivatives in solve form. Let $S_t w_t = v_t$ as in (18.24). Then

$$S_t \dot{w}_t^{(i)} = \dot{v}_t^{(i)} - \dot{S}_t^{(i)} w_t, \tag{18.54}$$

and

$$\partial_i \log\det S_t = 2\,\mathrm{tr}\left(R_t^{-1}\dot{R}_t^{(i)}\right) = \mathrm{tr}\left(S_t^{-1}\dot{S}_t^{(i)}\right). \tag{18.55}$$

Therefore the promoted principal-square-root branch may use a symmetric square root of $S_t$ for value-side solves and diagnostics, but its derivative contract is the solve/Sylvester form in (18.52)– (18.55), not an innovation-eigendecomposition derivative.

The sigma-point rule has square-root gauge freedom. If $CC^\top = P$, then the standardized point cloud $\chi^{(r)} = m + C\xi^{(r)}$ matches the same first and second Gaussian moments whenever the chosen rule's offsets and weights have the required standardized moment identities. Replacing an eigenvector factor $C = U\Lambda^{1/2}$ by the principal factor $S = P^{1/2}$ is therefore a change of square-root gauge for moment placement. It does not imply that nonlinear propagated values may differ only by roundoff: after a nonlinear map $g$, the clouds $\{g(m+C\xi^{(r)})\}$ and $\{g(m+S\xi^{(r)})\}$ may differ. BayesFilter must therefore validate the implemented likelihood and score, not claim nonlinear value identity across gauges.

The current principal-square-root backend is strict-SPD only. It must keep fixed-null support disabled unless a separate reviewed branch specifies how structural nulls, active floors, and rank-deficient covariance laws are represented and differentiated. A promoted principal-square-root UKF backend must fail closed whenever the strict-SPD assumptions for either placement or innovation covariance are violated. An active floor, a semidefinite innovation law, or a structural-null completion rule is a different backend branch, not a silent reinterpretation of (18.50) or (18.52). Required diagnostics for this backend include the Sylvester residuals in (18.50) and (18.52), the reconstruction residuals $\dot{P}^{(i)} - (\dot{S}^{(i)}S + S\dot{S}^{(i)})$ and $\dot{S}_t^{(i)} - (\dot{R}_t^{(i)}R_t + R_t\dot{R}_t^{(i)})$, the minimum eigenvalues of $P$ and $S_t$, any jitter or floor policy, and eager/compiled parity for the complete value-and-score path.

The Phase 2026-06 strict-SPD implementation evidence is bounded: repeated positive-spectrum fixtures, reconstruction checks, compiled value/score smoke, and the two-candidate NK replay are engineering artifacts. They do not establish posterior correctness, HMC convergence, runtime superiority, GPU readiness, default readiness, or full baseline launch readiness.

## 18.7 Validation Targets for SVD Score and Hessian

The analytical derivation above leads to concrete tests:

(i) On affine Gaussian systems, the SVD sigma-point score and Hessian must match the exact Kalman score and Hessian from Chapters 9–10.

(ii) On small nonlinear systems, the analytical score and Hessian must match finite differences of the same SVD sigma-point likelihood.

(iii) The factor residuals (18.47)–(18.48) must remain small on well-separated spectra.

(iv) Tests must deliberately include clustered eigenvalues, zero eigenvalues, floor crossings, and sign/order changes, and report when the derivative is a branch derivative rather than the derivative of a smooth mathematical SVD.

(v) HMC diagnostics must record whether gradients came from the structural analytic recursion, a custom spectral derivative, raw autodiff, or a regularized fallback.

These tests turn the phrase "SVD sigma-point Hessian" into an auditable object: a sigma-point moment derivative recursion, a solve-form likelihood Hessian, and a declared spectral-factor derivative policy.

## 18.8 SVD-CUT Score and Hessian

The conjugate unscented transform can be inserted into the same spectral sigma-point filter. The precise object is an SVD-placed CUT rule: use an SVD/eigen factor to place the CUT offsets, then use the ordinary sigma-point moment and likelihood recursions. This is sometimes called CUT-SVD and sometimes SVD-CUT. The name is less important than the contract

$$P_\star = CC^\top,$$

where $P_\star$ is the covariance actually represented by the implemented factor. If eigenvalues have been floored, truncated, or otherwise regularized, $P_\star$ is the floored or truncated covariance, not necessarily the pre-regularized covariance submitted to the factor routine.

Let the SVD branch return a factor $C(\theta) \in \mathbb{R}^{n_x \times q}$ and let $Z \sim \mathcal{N}(0, I_q)$. A CUT rule in the factor coordinates supplies standardized offsets and weights

$$\{(\xi^{(r)}, w_r) : r = 0, \ldots, N_{\mathrm{CUT}} - 1\}, \qquad \xi^{(r)} \in \mathbb{R}^q.$$

For CUT4-G, when $q \geq 3$,

$$N_{\mathrm{CUT4}} = 2q + 2^q.$$

The SVD-CUT points are

$$\chi^{(r)} = m + C\xi^{(r)}, \qquad r = 0, \ldots, N_{\mathrm{CUT}} - 1. \tag{18.56}$$

The quadrature approximation to an expectation under the implemented Gaussian law $X_\star = m + CZ \sim \mathcal{N}(m, P_\star)$ is

$$\mathcal{Q}_{\text{SVD-CUT}}[\varphi] = \sum_{r=0}^{N_{\text{CUT}}-1} w_r \varphi(m + C\xi^{(r)}). \tag{18.57}$$

**Proposition 18.1** (Polynomial exactness under affine SVD placement)**.** *Suppose the standardized CUT rule is exact for standard-normal polynomials on $\mathbb{R}^q$ through degree $d$. If $CC^\top = P_\star$, then (18.57) integrates every polynomial $p : \mathbb{R}^{n_x} \to \mathbb{R}$ of total degree at most $d$ exactly under $X_\star \sim \mathcal{N}(m, P_\star)$. The statement remains true when $P_\star$ is singular.*

*Proof.* For any polynomial $p$ of degree at most $d$, define

$$\psi(z) = p(m + Cz).$$

The affine composition cannot increase total degree, so $\psi$ is a polynomial on $\mathbb{R}^q$ of degree at most $d$. By standardized CUT exactness,

$$\sum_r w_r \psi(\xi^{(r)}) = \mathbb{E}[\psi(Z)].$$

Substituting the definition of $\psi$ gives

$$\sum_r w_r p(m + C\xi^{(r)}) = \mathbb{E}[p(m + CZ)] = \mathbb{E}[p(X_\star)].$$

No inverse of $C$ is used. Hence rank deficiency of $P_\star = CC^\top$ does not affect the formal quadrature exactness statement; it only changes the affine support on which the degenerate Gaussian law lives. □

This proposition is the clean mathematical answer to the accuracy question: abstracting from finite arithmetic, SVD placement does not by itself destroy CUT accuracy. It is not a recommendation to hide a structural state split inside a large singular covariance. A full zero-padded factor may reproduce the same law, but it also creates duplicate or nearly duplicate points in null directions and pushes the derivative path through the larger spectral branch.

### 18.8.1 SVD-CUT Moment Derivatives

The analytic gradient and Hessian follow by differentiating (18.56)–(18.57). Assume first that the CUT offsets and weights are fixed. If CUT tuning parameters are estimated, the same derivation applies after adding the explicit $\dot{\xi}^{(r)}$, $\ddot{\xi}^{(r)}$, $\dot{w}_r$, and $\ddot{w}_r$ terms.

For a parameter component $\theta_i$, the point derivatives are

$$\dot{\chi}^{(r,i)} = \dot{m}^{(i)} + \dot{C}^{(i)}\xi^{(r)}, \tag{18.58}$$

$$\ddot{\chi}^{(r,ij)} = \ddot{m}^{(ij)} + \ddot{C}^{(ij)}\xi^{(r)}. \tag{18.59}$$

Let $g_\theta$ be the nonlinear map being integrated. In a predict step this is the transition map; in an update step it is the observation map. Write

$$y^{(r)} = g_\theta(\chi^{(r)}), \qquad \bar{y} = \sum_r w_r y^{(r)}, \qquad d^{(r)} = y^{(r)} - \bar{y}.$$

The propagated point derivatives are

$$\dot{y}^{(r,i)} = D_x g_\theta(\chi^{(r)})\dot{\chi}^{(r,i)} + \partial_i g_\theta(\chi^{(r)}), \tag{18.60}$$

$$\begin{aligned}\ddot{y}^{(r,ij)} = {} & D_x g_\theta(\chi^{(r)})\ddot{\chi}^{(r,ij)} + D^2_{xx} g_\theta(\chi^{(r)})[\dot{\chi}^{(r,i)}, \dot{\chi}^{(r,j)}] \\ & + D^2_{x\theta_i} g_\theta(\chi^{(r)})\dot{\chi}^{(r,j)} + D^2_{x\theta_j} g_\theta(\chi^{(r)})\dot{\chi}^{(r,i)} + \partial^2_{ij} g_\theta(\chi^{(r)}).\end{aligned} \tag{18.61}$$

Therefore

$$\dot{\bar{y}}^{(i)} = \sum_r w_r \dot{y}^{(r,i)}, \tag{18.62}$$

$$\ddot{\bar{y}}^{(ij)} = \sum_r w_r \ddot{y}^{(r,ij)}. \tag{18.63}$$

Let $\Omega_\theta$ denote the additive covariance in the moment being formed: $Q_\theta$ in a predict step or $R_\theta$ in an observation update. The SVD-CUT covariance moment is

$$M = \sum_r w_r d^{(r)} d^{(r)\top} + \Omega_\theta. \tag{18.64}$$

With

$$\dot{d}^{(r,i)} = \dot{y}^{(r,i)} - \dot{\bar{y}}^{(i)}, \qquad \ddot{d}^{(r,ij)} = \ddot{y}^{(r,ij)} - \ddot{\bar{y}}^{(ij)},$$

its derivatives are

$$\dot{M}^{(i)} = \sum_r w_r \left[\dot{d}^{(r,i)} d^{(r)\top} + d^{(r)} \dot{d}^{(r,i)\top}\right] + \dot{\Omega}^{(i)}_\theta, \tag{18.65}$$

$$\ddot{M}^{(ij)} = \sum_r w_r \left[\ddot{d}^{(r,ij)} d^{(r)\top} + d^{(r)} \ddot{d}^{(r,ij)\top} + \dot{d}^{(r,i)} \dot{d}^{(r,j)\top} + \dot{d}^{(r,j)} \dot{d}^{(r,i)\top}\right] + \ddot{\Omega}^{(ij)}_\theta. \tag{18.66}$$

*Derivation.* Equations (18.58) and (18.59) are the first and second derivatives of the affine point map $\chi^{(r)} = m + C\xi^{(r)}$, using fixed $\xi^{(r)}$. Equations (18.60) and (18.61) are the first and second multivariate chain rules for $g_\theta(\chi^{(r)}(\theta))$. Equations (18.62) and (18.63) follow from linearity of the weighted sum. Finally, differentiating $d^{(r)} d^{(r)\top}$ once and twice gives

$$\partial_i (dd^\top) = \dot{d}_i d^\top + d\dot{d}_i^\top$$

and

$$\partial^2_{ij} (dd^\top) = \ddot{d}_{ij} d^\top + d\ddot{d}_{ij}^\top + \dot{d}_i \dot{d}_j^\top + \dot{d}_j \dot{d}_i^\top,$$

which proves (18.65) and (18.66) after summing over the fixed weights and adding the derivatives of $\Omega_\theta$. □

### 18.8.2 SVD-CUT Likelihood Gradient and Hessian

For an observation update, set $g_\theta = h_\theta$ and identify

$$\bar{z}_t = \bar{y}, \qquad S_t = M, \qquad v_t = y_t^{\text{obs}} - \bar{z}_t.$$

If the recorded observation does not depend on $\theta$, then

$$\dot{v}_t^{(i)} = -\dot{\bar{z}}_t^{(i)}, \qquad \ddot{v}_t^{(ij)} = -\ddot{\bar{z}}_t^{(ij)}. \tag{18.67}$$

Let $S_t w_t = v_t$. The per-time SVD-CUT log likelihood is

$$\ell_t = -\frac{1}{2}\left[\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right]. \tag{18.68}$$

Its score is

$$\partial_i \ell_t = -\frac{1}{2}\left[\operatorname{tr}(S_t^{-1}\dot{S}_t^{(i)}) + 2\dot{v}_t^{(i)\top} w_t - w_t^\top \dot{S}_t^{(i)} w_t\right]. \tag{18.69}$$

For the Hessian, define

$$\dot{w}_t^{(j)} = \mathcal{S}_{S_t}\left(\dot{v}_t^{(j)} - \dot{S}_t^{(j)} w_t\right). \tag{18.70}$$

Then

$$T_{ij}^{\log\det} = \operatorname{tr}\left(S_t^{-1}\ddot{S}_t^{(ij)} - S_t^{-1}\dot{S}_t^{(j)} S_t^{-1}\dot{S}_t^{(i)}\right), \tag{18.71}$$

$$T_{ij}^{v} = 2\ddot{v}_t^{(ij)\top} w_t + 2\dot{v}_t^{(i)\top}\dot{w}_t^{(j)}, \tag{18.72}$$

$$T_{ij}^{S} = 2\dot{w}_t^{(j)\top}\dot{S}_t^{(i)} w_t + w_t^\top \ddot{S}_t^{(ij)} w_t, \tag{18.73}$$

and

$$\partial_{ij}^2 \ell_t = -\frac{1}{2}\left[T_{ij}^{\log\det} + T_{ij}^{v} - T_{ij}^{S}\right]. \tag{18.74}$$

*Derivation.* Equations (18.65) and (18.66), with $\Omega_\theta = R_\theta$, supply $\dot{S}_t$ and $\ddot{S}_t$. Equation (18.67) supplies $\dot{v}_t$ and $\ddot{v}_t$. Differentiating the solve $S_t w_t = v_t$ gives

$$S_t \dot{w}_t^{(j)} + \dot{S}_t^{(j)} w_t = \dot{v}_t^{(j)},$$

hence (18.70). The differential identities $\partial_i \log\det S = \operatorname{tr}(S^{-1}\dot{S}_i)$ and $\partial_i S^{-1} = -S^{-1}\dot{S}_i S^{-1}$ give (18.69). Differentiating the three pieces of (18.68) a second time gives (18.71) and (18.74). This is the same solve-form likelihood Hessian as Chapter 10; the SVD-CUT-specific work is the construction of $v_t, S_t, \dot{v}_t, \dot{S}_t, \ddot{v}_t, \ddot{S}_t$ from the CUT point cloud. □

All dependence on the SVD backend enters through $C, \dot{C}, \ddot{C}$. Consequently the derivative must reconstruct the same implemented covariance branch:

$$\dot{P}_\star^{(i)} = \dot{C}^{(i)} C^\top + C\dot{C}^{(i)\top},$$

and

$$\ddot{P}_\star^{(ij)} = \ddot{C}^{(ij)} C^\top + C\ddot{C}^{(ij)\top} + \dot{C}^{(i)}\dot{C}^{(j)\top} + \dot{C}^{(j)}\dot{C}^{(i)\top}.$$

These are exactly the reconstruction identities (18.47)–(18.48). If a hard eigenvalue floor is active, the equations describe the branch derivative of the floored covariance $P_\star$, not the derivative of the pre-floor covariance. If eigenvalues are clustered, repeated, reordered, or sign-flipped, the smooth-branch formulas in Section 18.5 may fail or become ill-conditioned.

### 18.8.3 Point Count, GPU, and XLA

Modern GPUs and XLA make large sigma-point clouds much less painful than scalar Python loops, but they do not make the point count disappear. The CUT map evaluations are embarrassingly parallel over points and chains, and the point dimension is static when $q$ is fixed. This fits the XLA discipline in Chapter 46: construct a fixed tensor of offsets,

evaluate the transition or observation map in batch, and reduce over the point axis inside the compiled value-and-gradient path.

The remaining costs still scale with

$$N_{\mathrm{CUT4}} = 2q + 2^q.$$

For one observation update with output dimension $n_y$, the value-side moment assembly contains

$$O(N_{\mathrm{CUT}}\, c_h) \quad \text{map work}, \qquad O(N_{\mathrm{CUT}}\, n_y^2) \quad \text{innovation-covariance reductions},$$

plus $O(N_{\mathrm{CUT}}\, n_x n_y)$ if the cross covariance and Kalman gain are formed. With $p$ parameters, a direct analytic score stores or recomputes $O(N_{\mathrm{CUT}}\, p\, n_y)$ propagated first derivatives, and a direct Hessian stores or recomputes $O(N_{\mathrm{CUT}}\, p^2\, n_y)$ propagated second derivatives, in addition to the SVD-factor derivative work. XLA can fuse many elementwise operations and run the point axis in parallel, but memory traffic, reductions, spectral factorization, compile time, and the sequential time scan remain real constraints.

Thus the correct engineering conclusion is conditional. A large number of CUT points may be acceptable on modern GPU/XLA hardware when the stochastic factor dimension $q$ is modest, the maps are sufficiently expensive to amortize compile and launch overhead, tensor shapes are static, and chains or particles provide enough batch parallelism. It is not generally acceptable to collapse a large DSGE state into a full-state CUT rule and rely on the GPU to absorb the $2^q$ corner set. The preferred design remains structural SVD-CUT: apply higher-order CUT quadrature only to the declared stochastic integration block, then complete deterministic coordinates through the structural transition map and use SVD regularization only where the covariance factor actually needs it.

## 18.9 Status Labels for SVD-HMC

The SVD/HMC gap-closure plan introduced four labels that BayesFilter should reuse:

**filter-correct** the likelihood and gradients agree with a reference behavior on controlled cases;

**sampler-usable** short or medium HMC runs produce finite chains and useful diagnostics;

**converged** strict multi-chain posterior diagnostics pass on the intended target;

**production-XLA-gated** fixed-solution, full-solve, boundary rejection, gradient, and tiny HMC paths compile and run under the production compilation policy.

These labels prevent a recurring mistake: a clean smoke run is not a convergence result, and a finite likelihood is not a validated HMC gradient.

## 18.10 BayesFilter Policy

BayesFilter may use SVD sigma-point filters as robust approximate likelihood infrastructure and as a diagnostic backend. To use them as HMC production targets, the project must additionally provide:

(i) affine-Gaussian recovery against the exact Kalman backend;

(ii) nonlinear controlled recovery against dense quadrature or a trusted reference;

(iii) finite value and finite gradient stress tests across invalid and near-boundary parameter proposals;

(iv) eigenvalue or singular-value gap telemetry during HMC;

(v) eager/compiled parity for the complete value-and-gradient path;

(vi) a custom-gradient or analytic-gradient policy if raw spectral autodiff fails the preceding checks.

For industrial-size models, prevention remains preferable to debugging: use the direct-factor QR route on admitted full-rank or fixed-rank charts, and keep the principal-square-root and historical eigenderivative SVD-UKF branches as diagnostic comparators rather than default production routes.

## 18.11 Current Role of SVD in the Direct-Factor Architecture

The preceding spectral recursion is retained as historical and diagnostic material. It is not the default SR-UKF implementation. The current default propagates factors and applies direct QR to residual stacks as described in Section 17.6; it does not reconstruct a covariance and spectrally decompose it at each time step.

SVD still has a useful bounded role for an unknown singular direct stack: it can discover numerical rank and nominate a fixed QR chart for a subsequent value/score run. Discovery is value-only because singular-vector gauges are nonunique at repeated singular values and derivatives contain inverse spectral gaps. Once rank and chart are frozen, the admitted derivative is the rectangular QR derivative, not an SVD derivative. This separation preserves SVD's rank-revealing robustness without importing equal-eigenvalue gauge singularities into every temporal score evaluation.

For $P_\varepsilon = GG^\top + \varepsilon I_n$, SVD also explains the limit but does not make the raw ambient likelihood finite as $\varepsilon \to 0$. With rank $r < n$, the raw likelihood tends to $+\infty$ on support because of the null-space normalization and to $-\infty$ off support because of the null-space quadratic. Only the renormalized on-support limit

$$\lim_{\varepsilon \downarrow 0} \left[ \ell_\varepsilon + \frac{n-r}{2} \log(2\pi\varepsilon) \right]$$

equals the finite affine-support likelihood. It is a density with respect to a different measure, so using it is an explicit statistical contract rather than a numerical trick.

# Chapter 19

# Structural Deterministic Dynamics in DSGE Filtering

**Production warning.** Do not implement DSGE nonlinear filtering by silently treating every declared state coordinate as an exchangeable noisy Gaussian coordinate. Many DSGE models contain a structural split between shock-driven exogenous states and endogenous or accounting states that are deterministic conditional on lagged states, controls, and current shocks. This chapter proves the law-level reason: for mixed structural models, the nonlinear prediction target is the pushforward of the lagged filtering law and the current innovation law through the structural transition. A full-state additive-noise approximation is therefore valid only when it preserves that law or is explicitly labeled and tested as an approximation.

## 19.1 The Structural Split

The state vector in a DSGE likelihood is not merely a numerical vector. It usually carries economic timing. In the language of the structural state partition from Chapter 2, a common DSGE representation separates a declared stochastic block from a deterministic-completion block.

**Definition 19.1** (Structural transition notation)**.** *Throughout this chapter, write the full structural state as $x_t = (m_t, k_t)$, where $m_t$ is the declared stochastic or exogenous block and $k_t$ is the deterministic-completion block. A mixed structural transition is*

$$m_t = T_m(m_{t-1}, \varepsilon_t; \theta), \tag{19.1}$$
$$k_t = T_k(k_{t-1}, m_{t-1}, m_t; \theta), \tag{19.2}$$
$$x_t = (m_t, k_t). \tag{19.3}$$

*Equivalently, define the structural map*

$$F_\theta(x_{t-1}, \varepsilon_t) = \left(T_m(m_{t-1}, \varepsilon_t; \theta), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t; \theta); \theta)\right). \tag{19.4}$$

*Let $\pi^x_{t-1|t-1}$ denote the filtering law of $x_{t-1}$ and let $\nu^\varepsilon_\theta$ denote the law of $\varepsilon_t$. The pre-transition law is*

$$\pi^a_t = \pi^x_{t-1|t-1} \otimes \nu^\varepsilon_\theta \tag{19.5}$$

*when the current innovation is conditionally independent of the lagged filtering law. The predictive law of $x_t$ is*

$$\pi^x_{t|t-1}(B) = \Pr_\theta(x_t \in B \mid y_{1:t-1})$$

*for measurable state sets $B$. The pushforward of $\pi^a_t$ by $F_\theta$ is the measure*

$$(F_\theta)_\# \pi^a_t(B) = \pi^a_t\big(F_\theta^{-1}(B)\big).$$

*To place sigma points on a random variable means to choose weighted points $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ whose weighted mean and covariance match the Gaussian approximation to that variable, then propagate $x_t^{(j)} = F_\theta(a_t^{(j)})$ point by point.*

Thus, for mixed structural models,

$$\pi^x_{t|t-1} = (F_\theta)_\# \pi^a_t,$$

or, in density notation when the densities exist,

$$p_\theta(x_t \mid y_{1:t-1}) = \iint \delta(x_t - F_\theta(x_{t-1}, \varepsilon_t))\, p(\varepsilon_t)\, p_\theta(x_{t-1} \mid y_{1:t-1})\, d\varepsilon_t\, dx_{t-1}.$$

In a perturbation solution, the same economics may also be represented by a compact law of motion

$$x_t = h_x x_{t-1} + \eta \varepsilon_t$$

at first order, with zero or rank-deficient innovation rows for the deterministic-completion block. The compact representation is useful, but it must not erase the timing contract or the structural metadata described in Chapter 4. In nonlinear filtering, the correct random input is therefore the innovation or declared stochastic block. The endogenous block is then generated by the model-implied deterministic-completion map. Here "deterministic" means no independent innovation in that coordinate once $(x_{t-1}, \varepsilon_t)$ is fixed; it does not mean zero predictive variance conditional only on the lagged filtering state.

This is the distinction emphasized in the DSGE filtering literature. It is also an instance of the generic BayesFilter structural-transition contract from Chapter 2: the backend integrates over the declared stochastic block and completes the deterministic block pointwise. Standard Bayesian DSGE expositions write the likelihood in state-space form after the model solution has supplied a transition law for predetermined variables and an observation equation for measured macroeconomic series [Herbst and Schorfheide, 2015, An and Schorfheide, 2007]. Pruned perturbation methods make the same point in a different language: the first-order component is propagated stochastically, while the higher-order correction is propagated by a separate deterministic recursion driven by the lower-order state [Kim et al., 2008, Andreasen et al., 2018]. The filtering backend must honor the state-space law supplied by the structural solution, not merely the dimension of the state vector.

### 19.1.1 Literature Map for This Chapter

The chapter uses four strands of literature, and each supports a different claim.

(i) **State-space likelihoods and DSGE solutions.** Standard state-space and Bayesian DSGE references provide the filtering likelihood setup and the distinction between a structural model solution and the numerical recursion used to evaluate it [Durbin and Koopman, 2012, Herbst and Schorfheide, 2015, An and Schorfheide, 2007].

(ii) **Pruned nonlinear DSGE state spaces.** Pruning papers show that nonlinear DSGE approximations naturally separate lower-order stochastic propagation from higher-order deterministic correction recursions [Kim et al., 2008, Andreasen et al., 2018]. That literature motivates the structural partition used here, but it does not by itself choose a sigma-point implementation.

(iii) **Unscented and sigma-point filtering.** Julier and Uhlmann justify deterministic sigma points as a moment-matching approximation for nonlinear transformations, while van der Merwe's sigma-point Kalman-filter treatment supplies the dynamic state-space notation and augmented-noise filtering pattern used below [Julier and Uhlmann, 1996, 1997, van der Merwe, 2004].

(iv) **Particle and factor backends.** Particle-filter references support the same law-level distinction in a simulation backend: sample the declared transition and weight by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Square-root and SVD chapters in this monograph address numerical factorization and derivative risk; they do not replace the structural-law requirement.

The contribution of this chapter is to connect these strands: in a mixed DSGE transition, sigma-point, SVD sigma-point, cubature, and particle methods should approximate the pushforward law generated by the structural map $F_\theta(x_{t-1}, \varepsilon_t)$.

## 19.2 Why Linear Kalman Filtering Can Hide the Problem

For a linear Gaussian state-space model,

$$x_t = c + Fx_{t-1} + u_t, \qquad u_t \sim \mathcal{N}(0, Q), \tag{19.6}$$

$$y_t = a + Hx_t + e_t, \qquad e_t \sim \mathcal{N}(0, R), \tag{19.7}$$

the Kalman filter only needs the conditional mean and covariance of $x_t \mid x_{t-1}$. If a DSGE solution implies a singular or nearly singular $Q$, the exact linear Gaussian likelihood remains well defined when the collapsed representation preserves the same conditional law induced by the structural transition. Numerically stable implementations may then handle that degenerate conditional Gaussian law through diffuse, square-root, solve-form, or carefully regularized recursions as appropriate [Durbin and Koopman, 2012].

This is why the following collapsed first-order representation can be acceptable in an exact Kalman path:

$$Q = \eta\eta^\top, \qquad P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q.$$

The key point is not that structural determinism disappears. The key point is that the collapsed linear representation still encodes the same one-step conditional Gaussian law. Deterministic-completion coordinates are not wrong merely because they are included in $x_t$. They obtain randomness through their dependence on lagged stochastic coordinates, and their direct innovation variance may be zero. The exact linear Kalman recursion uses only the correct first two conditional moments.

The danger is that this success can train the implementation culture into a bad habit: it makes the state vector look like a homogeneous Gaussian vector. That habit becomes

wrong when the backend changes from exact linear filtering to nonlinear sigma-point or particle filtering. The relevant design decision is then the integration space recorded in the filter metadata, not merely the numerical dimension of $x_t$.

A related misunderstanding should be removed explicitly. A deterministic- completion coordinate need not have zero one-step predictive variance given the lagged state. It can inherit randomness through the current stochastic block. In the Chapter 2 AR(2) lag-stack example, the lag coordinate has no new innovation of its own but still remains random under the lagged filtering distribution. Likewise, in DSGE models with (19.1)–(19.2), $k_t$ may satisfy

$$\mathrm{Var}(k_t \mid x_{t-1}) > 0 \tag{19.8}$$

even though $k_t$ has no independent innovation once $(x_{t-1}, \varepsilon_t)$ is fixed.

## 19.3 Why Nonlinear Filters Must Respect the Structural Map

Sigma-point and particle filters approximate a transition law, not just a matrix covariance update. In a generic nonlinear state-space model,

$$x_t = f_\theta(x_{t-1}, \varepsilon_t), \qquad y_t = g_\theta(x_t, e_t),$$

the filter has to decide which variables are integrated over and which are deterministic transforms. In BayesFilter language, this is the choice of integration space recorded in the filter metadata. For DSGE models with (19.1)–(19.2), the natural nonlinear prediction step is:

(i) represent the filtering distribution of the lagged structural state;

(ii) draw, integrate, or choose quadrature points for the exogenous innovation $\varepsilon_t$ or current exogenous state $m_t$;

(iii) compute each corresponding endogenous state $k_t$ using $T_k(k_{t-1}, m_{t-1}, m_t; \theta)$;

(iv) evaluate the observation map on the resulting structurally valid points.

The endogenous coordinates should not receive arbitrary independent "sigma-point noise" merely because they are entries of $x_t$. If a sigma point moves an accounting identity or predetermined endogenous state outside the model-implied support of the structural transition, the filter is no longer approximating the intended DSGE state-space model. It is approximating a different artificial model relative to the declared structural law. This is exactly the approximation boundary from Chapter 16: quadrature may approximate the target law, but injecting additional stochastic coordinates changes the target unless that change is declared and justified.

Particle-filter literature makes the same distinction operationally. A particle filter propagates particles through the state transition and weights them by the observation density [Gordon et al., 1993, Doucet et al., 2001, Andrieu et al., 2010]. Written in the structural notation, a basic propagation and weighting step is

$$x_t^{(i)} = F_\theta\left(x_{t-1}^{(a_i)}, \varepsilon_t^{(i)}\right), \qquad w_t^{(i)} \propto p_\theta\left(y_t \mid x_t^{(i)}\right). \tag{19.9}$$

When part of the transition is deterministic, particles should be propagated by sampling the declared stochastic block and then deterministically completing the remaining coordinates. Adding artificial noise to deterministic coordinates can be used only as a declared proposal or regularization device, with the proposal/target correction and approximation status stated explicitly. It is not the default filtering law.

## 19.4 Reusable BayesFilter Structural Filtering Step

The doctrine above can be stated as a reusable backend contract. At every time step, the filter should treat the structural transition as the object being approximated:

$$x_t = F_\theta(x_{t-1}, \varepsilon_t), \qquad y_t \sim p_\theta(y_t \mid x_t).$$

The filtered state is still the full state $x_t$. The integration variable is the pre-transition uncertainty that generates the predictive law.

**Algorithm:** `structural_filter_step`.

(1) Start from the filtered approximation to $p_\theta(x_{t-1} \mid y_{1:t-1})$ and the declared innovation law $p_\theta(\varepsilon_t)$.

(2) Form sigma points, quadrature points, particles, or linearization points in the declared integration space: $(x_{t-1}, \varepsilon_t)$, $(s_{t-1}, d_{t-1}, \varepsilon_t)$, or an equivalent stochastic-state parameterization.

(3) For each point, call the structural transition map:

$$s_t^{(j)} = T_s(s_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta), \qquad d_t^{(j)} = T_d(d_{t-1}^{(j)}, s_{t-1}^{(j)}, s_t^{(j)}; \theta). \tag{19.10}$$

Deterministic-completion coordinates are outputs of this map, not independent new stochastic coordinates.

(4) Pack $x_t^{(j)} = \text{pack}(s_t^{(j)}, d_t^{(j)}, a_t^{(j)})$ and evaluate the observation map or density on these structurally valid points.

(5) Apply the selected update: exact Kalman, Gaussian sigma-point closure, particle weighting/resampling, or another declared backend.

(6) Return the likelihood contribution, filtered approximation, and metadata recording integration space, deterministic-completion policy, approximation label, derivative policy, and finite-failure diagnostics.

This algorithm is intentionally backend-neutral. For a linear Gaussian Kalman backend, the structural transition may collapse to matrices $(F, Q)$ as long as the same conditional Gaussian law is preserved. For UKF, CKF, SVD sigma-point, and particle backends, the pointwise structural transition is the default. A mixed-model full-state Gaussian approximation may still be offered as a diagnostic or legacy path, but it must carry an approximation label and must be tested against structural references.

## 19.5 Degenerate Transitions Are Structural

Degenerate transition covariance is not an exceptional corner case in DSGE models. It is often the mathematical expression of timing, identities, and endogenous accumulation. Capital, debt, lagged output growth, risk-premium terms, and other predetermined variables may be deterministic conditional on the previous state and current shocks. A full covariance matrix with small positive diagonal padding can keep linear algebra routines alive, but it does not by itself define the intended economic transition. BayesFilter therefore separates three ideas that are often confused in practice:

(i) an exact degenerate transition law that already matches the structural model;

(ii) numerical regularization used to evaluate that law stably; and

(iii) an altered transition law that injects new stochastic degrees of freedom into deterministic-completion coordinates.

Only the first two preserve the original structural model automatically.

The implementation policy is therefore:

- distinguish structural determinism from numerical regularization;
- never let a nugget or PSD projection change the declared stochastic dimension without a written approximation label;
- expose deterministic blocks in diagnostics, rather than burying them under a full-rank covariance;
- test degenerate and rank-deficient transition cases directly.

This point is separate from the SVD spectral-gradient issue in Chapter 18. SVD factors may improve value-side robustness for nearly singular covariances, and an SVD sigma-point filter can be the right numerical backend for a structurally declared sigma-point rule. But SVD is a factorization choice, not a state-law declaration. It repairs the reviewer's degeneracy concern only if the sigma-point backend factors the covariance of the correct integration variable, such as $(x_{t-1}, \varepsilon_t)$, and then completes deterministic coordinates pointwise. If the backend first invents a full-rank post-transition covariance for deterministic-completion coordinates, an SVD factor merely represents that altered law more stably.

## 19.6 Standard UKF, Structural UKF, and the Predictive Law

### 19.6.1 The Original Unscented-Transform Pattern

Julier and Uhlmann's original unscented-transform construction starts from a random variable $X \in \mathbb{R}^L$ with mean $\bar{x}$ and covariance $P_x$ and approximates the moments of $Y = g(X)$ by propagating deterministic sigma points through $g$ [Julier and Uhlmann, 1996, 1997]. The $(\alpha, \beta, \kappa)$ scaled rule used for the formulas below belongs to the later

sigma-point Kalman filter convention [van der Merwe, 2004]. In the notation used here, one convenient scaled UKF rule forms

$$\chi_0 = \bar{x}, \tag{19.11}$$

$$\chi_i = \bar{x} + \left[\sqrt{(L+\lambda)P_x}\right]_i, \qquad i = 1, \ldots, L, \tag{19.12}$$

$$\chi_{i+L} = \bar{x} - \left[\sqrt{(L+\lambda)P_x}\right]_i, \qquad i = 1, \ldots, L, \tag{19.13}$$

where $[\sqrt{(L+\lambda)P_x}]_i$ denotes the $i$th column or row of the chosen matrix square root. With

$$\lambda = \alpha^2(L+\kappa) - L,$$

the usual scaled weights are

$$w_0^{(m)} = \frac{\lambda}{L+\lambda}, \qquad w_0^{(c)} = \frac{\lambda}{L+\lambda} + 1 - \alpha^2 + \beta, \tag{19.14}$$

$$w_i^{(m)} = w_i^{(c)} = \frac{1}{2(L+\lambda)}, \qquad i = 1, \ldots, 2L. \tag{19.15}$$

The propagated points are $\upsilon_i = g(\chi_i)$, and the transformed moments are approximated by

$$\bar{y} \approx \sum_{i=0}^{2L} w_i^{(m)} \upsilon_i, \tag{19.16}$$

$$P_y \approx \sum_{i=0}^{2L} w_i^{(c)} (\upsilon_i - \bar{y})(\upsilon_i - \bar{y})^\top. \tag{19.17}$$

The early unscented-transform report analyzes this construction by Taylor expansion: the method matches the input mean and covariance and pushes the resulting deterministic ensemble through the nonlinear map, yielding lower-order moment accuracy that improves on first-order linearization under the smoothness conditions stated there [Julier and Uhlmann, 1996]. Van der Merwe's sigma-point Kalman filter treatment uses the same family logic: sigma points are selected to capture the important first and second moments of the prior Gaussian random variable and then weighted after nonlinear propagation [van der Merwe, 2004].

This chapter uses that scaled UKF convention for exposition. Later UKF, CDKF, and square-root variants differ in point placement, weights, and factorization details, but the structural question is invariant: which random variable is the sigma-point rule approximating? If the wrong variable is chosen, the transform may approximate the wrong law accurately.

In the additive-noise filtering setting, the unscented-transform idea is usually applied to an augmented Gaussian variable containing the prior state and current additive-noise variables. The sigma points therefore approximate the variables that generate the next predicted state and predicted observation. For mixed structural DSGE transitions, that same principle is exactly why the sigma-point variable is $(x_{t-1}, \varepsilon_t)$ or an equivalent pre-transition parameterization, not an artificially perturbed post-transition full state.

### 19.6.2 Standard Additive-Noise UKF Algorithm

To avoid confusion, write the additive-noise UKF pattern explicitly. Suppose the model is declared as

$$x_t = f_\theta(x_{t-1}, u_t), \qquad y_t = h_\theta(x_t, e_t), \tag{19.18}$$

with independent Gaussian state innovation $u_t \sim \mathcal{N}(0, Q_t)$ and measurement innovation $e_t \sim \mathcal{N}(0, R_t)$. Given a previous Gaussian filtering approximation, one additive-noise UKF step is:

**U1. Inputs.** Start from

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(m_{t-1|t-1}, P_{t-1|t-1}).$$

The implementation receives $f_\theta$, $h_\theta$, $Q_t$, $R_t$, the observed value $y_t$, sigma-point parameters $(\alpha, \beta, \kappa)$, and a declared covariance-factorization policy.

**U2. Augmented variable.** Form

$$b_t = (x_{t-1}, u_t, e_t), \qquad b_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_b, P_b),$$

with

$$\mu_b = (m_{t-1|t-1}, 0, 0), \qquad P_b = \mathrm{blockdiag}(P_{t-1|t-1}, Q_t, R_t).$$

If a variant handles measurement noise by adding $R_t$ after observation propagation, omit $e_t$ from $b_t$ and add $R_t$ in step U6.

**U3. Sigma points.** Generate $\{b_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_b}$ from $(\mu_b, P_b)$ using (19.13)–(19.15). Write $b_t^{(j)} = (x_{t-1}^{(j)}, u_t^{(j)}, e_t^{(j)})$.

**U4. State propagation.** Propagate

$$x_t^{(j)} = f_\theta(x_{t-1}^{(j)}, u_t^{(j)}).$$

Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \qquad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

**U5. Observation propagation.** Form

$$z_t^{(j)} = h_\theta(x_t^{(j)}, e_t^{(j)})$$

or $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ with $R_t$ added below, depending on the chosen variant.

**U6. Innovation moments.** Compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.19}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t^{\mathrm{add}}, \tag{19.20}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top, \tag{19.21}$$

where $R_t^{\mathrm{add}} = 0$ if measurement noise was included in $b_t$ and $R_t^{\mathrm{add}} = R_t$ otherwise.

**U7. Gaussian update and likelihood.** With $v_t = y_t - \hat{z}_t$,

$$K_t = P_{xz,t} S_t^{-1}, \tag{19.22}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \tag{19.23}$$
$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.24}$$
$$\ell_t = -\frac{1}{2}\left[d_y \log(2\pi) + \log|S_t| + v_t^\top S_t^{-1} v_t\right]. \tag{19.25}$$

**U8. Return metadata.** Return $(\ell_t, \hat{x}_{t|t}, P_{t|t})$ with metadata recording the augmented variable $b_t$, the factorization method, sigma-point parameters, whether measurement noise was augmented or added, and any PSD/jitter/finiteness interventions.

This is the textbook pattern many readers have in mind. It is perfectly natural when the declared state transition itself is an additive-noise model on the full state.

### 19.6.3 Structural UKF Algorithm

For a mixed structural model, the update algebra remains the same, but the sigma-point variable is chosen differently because the predictive law is generated differently. For the structural transition (19.1)–(19.2), a structural UKF step is:

**S1. Inputs.** Start from the filtering approximation

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}).$$

The implementation receives $T_m$, $T_k$, the observation map $h_\theta$, the innovation law $\varepsilon_t \sim \mathcal{N}(0, \Sigma_{\varepsilon,t})$, observation covariance $R_t$, the observed value $y_t$, sigma-point parameters, and a factorization policy.

**S2. Pre-transition variable.** Form

$$a_t = (x_{t-1}, \varepsilon_t), \qquad a_t \mid y_{1:t-1} \approx \mathcal{N}(\mu_a, P_a),$$

with

$$\mu_a = (\mu_{t-1}, 0), \qquad P_a = \mathrm{blockdiag}(P_{t-1}, \Sigma_{\varepsilon,t}).$$

This is the sigma-point variable because $\pi^x_{t|t-1} = (F_\theta)_\# \pi^a_t$.

**S3. Sigma points.** Generate $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2L_a}$ from $(\mu_a, P_a)$. Write $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$.

**S4. Stochastic-block propagation.** For each point compute

$$m_t^{(j)} = T_m(m_{t-1}^{(j)}, \varepsilon_t^{(j)}; \theta).$$

**S5. Deterministic completion.** For the same point compute

$$k_t^{(j)} = T_k(k_{t-1}^{(j)}, m_{t-1}^{(j)}, m_t^{(j)}; \theta), \qquad x_t^{(j)} = (m_t^{(j)}, k_t^{(j)}).$$

There is no independent sigma-point coordinate for $k_t$ unless the model declares one.

**S6. State moments.** Compute

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \qquad P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(x_t^{(j)} - \hat{x}_{t|t-1})^\top.$$

**S7. Observation points and moments.** Evaluate $z_t^{(j)} = h_\theta(x_t^{(j)}, 0)$ or the appropriate noise-augmented observation variant, then compute

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.26}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \tag{19.27}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_{t|t-1})(z_t^{(j)} - \hat{z}_t)^\top. \tag{19.28}$$

**S8. Gaussian update and likelihood.** Apply exactly the update from U7:

$$v_t = y_t - \hat{z}_t, \qquad K_t = P_{xz,t} S_t^{-1}, \qquad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t,$$

with $P_{t|t} = P_{xx,t} - K_t S_t K_t^\top$ and the same Gaussian innovation log-likelihood formula.

**S9. Return metadata.** Return the likelihood contribution and filtered Gaussian approximation with metadata recording `integration_space` equal to $(x_{t-1}, \varepsilon_t)$, `deterministic_completion` equal to `pointwise`, structural identity diagnostics, sigma-point parameters, factorization method, and any approximation label.

Structural UKF is not a different Kalman-gain formula. It is the same Gaussian update algebra applied to a predictive sigma-point cloud that respects the structural transition law.

### 19.6.4 UKF in Sequential-Monte-Carlo Kernel Notation

The same point can be expressed in the probability-kernel notation used in Chopin and Papaspiliopoulos [2020]. A state-space model is specified by an initial law $P_{\theta,0}(dx_0)$, transition kernels $P_{\theta,t}(x_{t-1}, dx_t)$, and observation densities $f_{\theta,t}(y_t \mid x_t)$. In the bootstrap Feynman–Kac formalism, the transition kernel is

$$M_{\theta,t}(x_{t-1}, dx_t) = P_{\theta,t}(x_{t-1}, dx_t),$$

and the potential is

$$G_{\theta,t}(x_t) = f_{\theta,t}(y_t \mid x_t). \tag{19.29}$$

If $\eta_{t-1}$ denotes the filtering law of $x_{t-1}$ given $y_{1:t-1}$, the exact one-step recursion is

$$\xi_t(dx_t) = \int \eta_{t-1}(dx_{t-1}) P_{\theta,t}(x_{t-1}, dx_t), \tag{19.30}$$

$$\lambda_t = \xi_t(G_{\theta,t}) = \int G_{\theta,t}(x_t)\, \xi_t(dx_t), \tag{19.31}$$

$$\eta_t(dx_t) = \frac{G_{\theta,t}(x_t)\xi_t(dx_t)}{\xi_t(G_{\theta,t})}. \tag{19.32}$$

Equations (19.30)–(19.32) are the filtering target. A particle filter approximates the measures in these equations by random weighted particles. A UKF instead approximates the same prediction/update recursion by deterministic quadrature and Gaussian projection.

For the mixed structural transition in (19.4), the transition kernel is the pushforward kernel

$$P^{\mathrm{str}}_{\theta,t}(x,B) = \int \mathbf{1}\{F_\theta(x,\varepsilon)\in B\}\,\nu^{\varepsilon}_{\theta,t}(d\varepsilon) = \int \delta_{F_\theta(x,\varepsilon)}(B)\nu^{\varepsilon}_{\theta,t}(d\varepsilon), \tag{19.33}$$

for measurable state sets $B$. This is a perfectly valid probability kernel even when it is singular with respect to Lebesgue measure. Kernel notation is therefore useful here: it does not force a degenerate structural transition to pretend that it has a full-dimensional transition density. The predictive integral in (19.30) becomes, for any test function $\psi$,

$$\xi^{\mathrm{str}}_t(\psi) = \iint \psi(F_\theta(x_{t-1},\varepsilon_t))\,\eta_{t-1}(dx_{t-1})\nu^{\varepsilon}_{\theta,t}(d\varepsilon_t). \tag{19.34}$$

Thus the kernel version of the structural statement is:

$$P_{\theta,t} = P^{\mathrm{str}}_{\theta,t}, \qquad M_{\theta,t} = P^{\mathrm{str}}_{\theta,t}, \qquad G_{\theta,t}(x) = f_{\theta,t}(y_t \mid x).$$

Now suppose the incoming filter is approximated by a Gaussian $\widehat{\eta}_{t-1} = \mathcal{N}(\widehat{m}_{t-1}, \widehat{P}_{t-1})$. Let

$$\widehat{\alpha}_t(da) = \widehat{\eta}_{t-1}(dx_{t-1})\nu^{\varepsilon}_{\theta,t}(d\varepsilon_t), \qquad a = (x_{t-1},\varepsilon_t). \tag{19.35}$$

The structural UKF replaces integrals against $\widehat{\alpha}_t$ by the deterministic sigma-point rule

$$\mathcal{U}^{(r)}_t[\psi] = \sum_j w^{(r)}_j \psi(a^{(j)}_t), \qquad r\in\{m,c\}.$$

With

$$x^{(j)}_t = F_\theta(a^{(j)}_t), \qquad z^{(j)}_t = h_\theta(x^{(j)}_t),$$

the UKF predictive and observation moments are

$$\widehat{m}^-_t = \sum_j w^{(m)}_j x^{(j)}_t, \tag{19.36}$$

$$\widehat{P}^-_t = \sum_j w^{(c)}_j (x^{(j)}_t - \widehat{m}^-_t)(x^{(j)}_t - \widehat{m}^-_t)^\top, \tag{19.37}$$

$$\widehat{z}_t = \sum_j w^{(m)}_j z^{(j)}_t, \tag{19.38}$$

$$\widehat{S}_t = \sum_j w^{(c)}_j (z^{(j)}_t - \widehat{z}_t)(z^{(j)}_t - \widehat{z}_t)^\top + R_t, \tag{19.39}$$

$$\widehat{P}_{xz,t} = \sum_j w^{(c)}_j (x^{(j)}_t - \widehat{m}^-_t)(z^{(j)}_t - \widehat{z}_t)^\top. \tag{19.40}$$

Equivalently, UKF builds the Gaussian joint approximation

$$\widehat{J}_t(dx_t,dy_t) = \mathcal{N}\left(\begin{bmatrix}x_t\\ y_t\end{bmatrix}; \begin{bmatrix}\widehat{m}^-_t\\ \widehat{z}_t\end{bmatrix}, \begin{bmatrix}\widehat{P}^-_t & \widehat{P}_{xz,t}\\ \widehat{P}^\top_{xz,t} & \widehat{S}_t\end{bmatrix}\right) dx_t\,dy_t. \tag{19.41}$$

The UKF likelihood factor and update are then the marginal and conditional Gaussian objects induced by (19.41):

$$\widehat{\lambda}_t^{\mathrm{UKF}} = \varphi_{d_y}(y_t; \widehat{z}_t, \widehat{S}_t), \tag{19.42}$$

$$\begin{aligned}\widehat{\eta}_t^{\mathrm{UKF}}(dx_t) &= \widehat{J}_t(dx_t \mid y_t) \\ &= \mathcal{N}\Big(dx_t;\, \widehat{m}_t^- + \widehat{P}_{xz,t}\widehat{S}_t^{-1}(y_t - \widehat{z}_t),\, \widehat{P}_t^- - \widehat{P}_{xz,t}\widehat{S}_t^{-1}\widehat{P}_{xz,t}^\top\Big).\end{aligned} \tag{19.43}$$

The corresponding contribution to the log likelihood is $\log \widehat{\lambda}_t^{\mathrm{UKF}}$. For actual stochastic-volatility rows, augmentation is admissible only when marginalizing the augmented variable reproduces the same observation density intended by the inference problem. In the transformed actual-SV setting that means preserving the exact law $z_t = \log(y_t^2)$, $z_t - \log(\beta^2) - h_t \sim \log(\chi_1^2)$. The previous actual-SV Gaussian-closure framing failed this condition: it used augmentation to construct a different innovation scalar rather than the intended normalizer. That construction may still define a Gaussian-projection filter, but it does not define the correct actual-SV likelihood target. If measurement noise is augmented rather than added, replace $a = (x_{t-1}, \varepsilon_t)$ by $a = (x_{t-1}, \varepsilon_t, e_t)$, set $z_t^{(j)} = h_\theta(F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)}), e_t^{(j)})$, and omit the extra $R_t$ term in (19.40).

This notation also states the equivalence condition for collapsed SVD constructions. If a full-state degenerate SVD implementation uses another kernel

$$\widetilde{P}_{\theta,t}(x, B) = \int \delta_{\widetilde{F}_\theta(x,\eta)}(B)\widetilde{\nu}_{\theta,t}(d\eta),$$

then it is a law-equivalent implementation of the structural model only when

$$\widetilde{P}_{\theta,t}(x, B) = P_{\theta,t}^{\mathrm{str}}(x, B) \qquad \text{for the relevant } x \text{ and measurable } B, \tag{19.44}$$

or at least when the two kernels induce the same predictive measure from the current filtering approximation. Equality of a covariance matrix, or existence of an SVD factor for a singular covariance, is not by itself equality of probability kernels. This is the kernel-notation version of the structural warning in this chapter.

### 19.6.5 Why These Algorithms Differ

| Object | Standard additive-noise UKF | Structural UKF |
|---|---|---|
| Filtered object | Full state $x_t$ | Full state $x_t$ |
| Sigma-point variable | Declared augmented additive-noise variable $(x_{t-1}, u_t, e_t)$ | Pre-transition structural variable $(x_{t-1}, \varepsilon_t)$ |
| Direct process noise | Whatever the additive model declares | Only the declared stochastic block |
| Deterministic coordinates | None unless constraints are declared | Completed by $T_k$ pointwise |
| Target law | Additive-noise predictive law | Structural pushforward law |
| Failure when applied to this chapter's mixed structural model | Model-law error plus moment approximation error | Moment approximation error only |
| SVD/square-root role | Covariance factorization | Covariance factorization, not structural repair |

The update equations are shared. The exact difference is upstream: which random variable receives sigma points and which map generates the predictive state cloud.

### 19.6.6 Why the Sigma-Point Variable Uses Pre-Transition Uncertainty

**Proposition 19.1** (Predictive pushforward and sigma-point space)**.** *Under the structural transition*

$$m_t = T_m(m_{t-1}, \varepsilon_t), \tag{19.45}$$
$$k_t = T_k(k_{t-1}, m_{t-1}, m_t), \tag{19.46}$$
$$x_t = (m_t, k_t), \tag{19.47}$$

*with innovation law $p(\varepsilon_t)$ independent of $x_{t-1} \mid y_{1:t-1}$, the following hold:*

*(i) the one-step predictive law $p(x_t \mid y_{1:t-1})$ is the pushforward of the joint law of $(x_{t-1}, \varepsilon_t)$ under the structural map;*

*(ii) a UKF that approximates this predictive law should place sigma points on $(x_{t-1}, \varepsilon_t)$, or on an equivalent parameterization of the same pre-transition uncertainty;*

*(iii) a naive full-state additive-noise UKF targets the same predictive law only if its induced predictive law coincides with that pushforward law; if it adds a direct perturbation to $k_t$, it targets a different law.*

*Proof.* Define the structural map

$$F(x_{t-1}, \varepsilon_t) = \big(T_m(m_{t-1}, \varepsilon_t), T_k(k_{t-1}, m_{t-1}, T_m(m_{t-1}, \varepsilon_t))\big). \tag{19.48}$$

Because the current innovation is independent of the lagged filtering law, the joint pre-transition uncertainty factorizes as

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1})p(\varepsilon_t). \tag{19.49}$$

Let $\varphi$ be any bounded test function on the state space of $x_t$. Then

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1})p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.50}$$

Equation (19.50) is exactly the expectation of $\varphi$ under the pushforward of the joint law of $(x_{t-1}, \varepsilon_t)$ through $F$, which proves part (i).

Moreover, under the structural transition,

$$x_t = F(x_{t-1}, \varepsilon_t) \qquad \text{almost surely.} \tag{19.51}$$

So once $(x_{t-1}, \varepsilon_t)$ is fixed, both $m_t$ and $k_t$ are fixed. There is no independent new perturbation attached to $k_t$ under the intended model. Therefore any sigma-point rule intended to approximate $p(x_t \mid y_{1:t-1})$ should be applied to the pre-transition uncertainty variables that generate that law, namely $(x_{t-1}, \varepsilon_t)$ or an exactly equivalent parameterization. This proves part (ii).

Finally, if a full-state additive-noise UKF omits $\varepsilon_t$, then it misses genuine current-shock uncertainty. If it instead adds a direct perturbation to $k_t$, then it enlarges the stochastic space and defines a different predictive law unless that perturbation is degenerate in exactly the way required to reproduce the same pushforward distribution in (19.50). This proves part (iii). □

The proposition does not say that the filter stops targeting $x_t$. The filtered state remains $x_t$. The point is that the sigma-point variable is the pre-transition uncertainty whose pushforward generates the predictive law of $x_t$. This is analogous to process-noise augmentation in a standard additive- noise UKF, but in the mixed structural setting the distinction is sharper: $\varepsilon_t$ is a genuine new source of uncertainty, while $k_t$ is a structural completion of that uncertainty rather than an independently shocked coordinate.

**Proposition 19.2** (Local structural UKF Taylor accuracy)**.** *Let $A_t = (x_{t-1}, \varepsilon_t)$ have a Gaussian approximation with mean $\mu_a$ and covariance $P_a$, and let $X_t = F_\theta(A_t)$ where $F_\theta$ is the structural map in (19.4). Write $P_a = \eta^2 \Sigma_a$, with $\Sigma_a$ fixed and $\eta$ measuring the local scale of the input uncertainty. Suppose the sigma-point deviations $\xi_j = a_t^{(j)} - \mu_a$ satisfy*

$$\sum_j w_j^{(m)} \xi_j = 0, \qquad \sum_j w_j^{(m)} \xi_j \xi_j^\top = P_a, \qquad \sum_j w_j^{(c)} \xi_j \xi_j^\top = P_a, \tag{19.52}$$

*and suppose $F_\theta$ is three-times continuously differentiable on a neighborhood containing the relevant input mass and sigma points. Let $J$ be the Jacobian of $F_\theta$ at $\mu_a$, and let $H_r$ be the Hessian of the $r$th component of $F_\theta$ at $\mu_a$. Then the structural UKF prediction matches the transformed mean through the quadratic Taylor term:*

$$\mathbb{E}[X_{t,r} \mid y_{1:t-1}] = F_{\theta,r}(\mu_a) + \frac{1}{2} \operatorname{tr}(H_r P_a) + O(\eta^3), \tag{19.53}$$

$$\hat{x}^{\mathrm{UKF}}_{t|t-1,r} = F_{\theta,r}(\mu_a) + \frac{1}{2} \operatorname{tr}(H_r P_a) + O(\eta^3). \tag{19.54}$$

*It also matches the leading second-order covariance term:*

$$\operatorname{Cov}(X_t \mid y_{1:t-1}) = J P_a J^\top + O(\eta^3), \tag{19.55}$$

$$P^{\mathrm{UKF}}_{xx,t} = J P_a J^\top + O(\eta^3). \tag{19.56}$$

*Thus, under the local small-uncertainty Taylor expansion used here, the structural UKF matches the transformed mean through the quadratic term and matches the leading covariance term for the Gaussian approximation on the correct input law $(x_{t-1}, \varepsilon_t)$. This is a local expansion statement for the selected structural input law, not a blanket exactness theorem for nonlinear structural transitions. It does not apply to a naive post-transition full-state sigma-point cloud unless that cloud induces the same law as $(F_\theta)_\# \pi_t^a$.*

*Proof.* By Proposition 19.1, the structural predictive law of $x_t$ is the law of $F_\theta(A_t)$. The structural UKF therefore applies the unscented-transform rule to the transformation $g = F_\theta$ and the input variable $A_t$.

For a component $r$, Taylor expansion around $\mu_a$ gives

$$F_{\theta,r}(\mu_a + \delta) = F_{\theta,r}(\mu_a) + J_r \delta + \frac{1}{2} \delta^\top H_r \delta + O(\|\delta\|^3).$$

Taking expectation under the Gaussian approximation for $A_t - \mu_a$ yields the first line of (19.54), because $\mathbb{E}[A_t - \mu_a] = 0$ and $\mathbb{E}[(A_t - \mu_a)(A_t - \mu_a)^\top] = P_a$. Taking the weighted sum over sigma points gives the second line of (19.54) by the same first- and second-moment identities for $\xi_j$.

For covariance, subtract the corresponding mean expansion. The leading centered term is $J(A_t - \mu_a)$ for the true law and $J\xi_j$ for the sigma points. Their second moments

are both $JP_aJ^\top$ by moment matching. The quadratic Taylor terms have size $O(\eta^2)$, so their covariance contribution with the leading linear term is $O(\eta^3)$ and their self-covariance contribution is $O(\eta^4)$. This proves (19.56). These are the local unscented-transform moment calculations for the selected sigma-point rule [Julier and Uhlmann, 1996, van der Merwe, 2004], applied to the structural input variable. If a naive full-state cloud is constructed from a different input law, the same UT calculation applies only to that different law; it gives no accuracy guarantee for the structural pushforward target. □

## 19.7 Worked Example A: Nonlinear Structural Transition

Let

$$m_t = \rho m_{t-1} + \sigma\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \tag{19.57}$$

$$k_t = \phi k_{t-1} + \gamma m_t^2, \tag{19.58}$$

$$y_t = m_t + k_t + e_t, \qquad e_t \sim \mathcal{N}(0,R). \tag{19.59}$$

The state is $x_t = (m_t, k_t)$, but only the innovation $\varepsilon_t$ is new stochastic input at time $t$. Conditional on $(m_{t-1}, k_{t-1}, \varepsilon_t)$, the coordinate $k_t$ is deterministic.

Suppose the previous filtering distribution is Gaussian:

$$x_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(\mu_{t-1}, P_{t-1}), \qquad \mu_{t-1} = \begin{bmatrix} \bar{m} \\ \bar{k} \end{bmatrix}. \tag{19.60}$$

The structural UKF should augment the previous state with the innovation. This step needs to be motivated carefully, because it is exactly where the structural UKF departs from the naive full-state construction. The point is not to pad the state arbitrarily. The point is to place sigma points on the variables that actually generate the one-step uncertainty in the structural transition. In this example, the predictive law is generated by the joint uncertainty in the lagged state $(m_{t-1}, k_{t-1})$ and the current shock $\varepsilon_t$. Once those three quantities are fixed, both $m_t$ and $k_t$ are fixed by the structural map.

That is why the augmented variable is

$$a_t = (x_{t-1}, \varepsilon_t), \tag{19.61}$$

rather than an artificially padded post-transition full state. Standard additive-noise UKF presentations often augment with process noise for the same reason: sigma points should live on the pre-transition uncertainty variables. For a mixed structural model, however, that principle has a sharper consequence. The shock $\varepsilon_t$ is a genuine new source of uncertainty, while $k_t$ is not. If the augmentation omitted $\varepsilon_t$, the propagated sigma points would miss current shock uncertainty. If the augmentation instead added a direct perturbation for $k_t$, the sigma points would describe a different transition law from the one declared by the model.

So the structural UKF should augment the previous state with the innovation. This places sigma points in the same pre-transition uncertainty variables that define the predictive law, rather than in an artificially padded post-transition full state:

$$a_t = \begin{bmatrix} m_{t-1} \\ k_{t-1} \\ \varepsilon_t \end{bmatrix}, \qquad a_t \mid y_{1:t-1} \approx \mathcal{N}\left( \begin{bmatrix} \bar{m} \\ \bar{k} \\ 0 \end{bmatrix}, \begin{bmatrix} P_{t-1} & 0 \\ 0 & 1 \end{bmatrix} \right). \tag{19.62}$$

Let $\{a_t^{(j)}, w_j^{(m)}, w_j^{(c)}\}_{j=0}^{2n_a}$ be the usual unscented points and weights for this $n_a = 3$ dimensional augmented Gaussian [Julier and Uhlmann, 1997]. For every point $a_t^{(j)} = (m_{t-1}^{(j)}, k_{t-1}^{(j)}, \varepsilon_t^{(j)})$, the structural transition is evaluated as

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \tag{19.63}$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma \left(m_t^{(j)}\right)^2, \tag{19.64}$$

$$x_t^{(j)} = \begin{bmatrix} m_t^{(j)} \\ k_t^{(j)} \end{bmatrix}. \tag{19.65}$$

This is the crucial step: the filter never creates an independent sigma-point perturbation for $k_t$. It creates sigma points for the previous state and the current innovation, then computes $k_t$ by the structural map (19.58).

The structural and naive full-state UKF constructions now differ at the level that matters for the update. The structural UKF forms sigma points for $(x_{t-1}, \varepsilon_t)$, propagates them through the structural transition, and therefore computes moments of the target law declared by the model. A naive full-state UKF instead behaves as if there were an additive-noise model of the form

$$x_t = \tilde{f}(x_{t-1}) + \tilde{u}_t, \qquad \tilde{u}_t \sim \mathcal{N}(0, \tilde{Q}),$$

with a full-state covariance $\tilde{Q}$ that assigns direct new variance to all or most coordinates of $x_t$. The three failure claims can now be derived in the notation of this toy model.

**Prediction-law failure.** Under the structural model,

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = \phi k_{t-1} + \gamma m_t^2. \tag{19.66}$$

So conditional on $(m_{t-1}, k_{t-1})$, the one-step predictive law is induced by a single scalar shock $\varepsilon_t$. Eliminating the shock-driven update for $m_t$ gives the structural identity

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0. \tag{19.67}$$

Every propagated sigma point in the structural UKF satisfies this identity, so its sigma-point cloud lies on the support of the structural predictive law. A naive full-state UKF instead propagates points

$$\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$$

from an additive full-state covariance model. In general those points satisfy

$$\tilde{k}_t^{(j)} - \phi k_{t-1}^{(j)} - \gamma\big(\tilde{m}_t^{(j)}\big)^2 \neq 0, \tag{19.68}$$

so the naive sigma-point cloud approximates moments of a different one-step law. That is the prediction-law failure.

**Cross-covariance failure.** For the observation equation $z_t = m_t + k_t$, the structural UKF computes

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} \big(x_t^{(j)} - \hat{x}_{t|t-1}\big)\big(z_t^{(j)} - \hat{z}_t\big),$$

with

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}, \qquad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma\big(m_t^{(j)}\big)^2.$$

Hence all variation in the second state coordinate enters through lagged-state variation and the common shock channel already represented in $m_t^{(j)}$. Under the naive full-state UKF,

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} \big(\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}\big)\big(\tilde{z}_t^{(j)} - \tilde{z}_t\big),$$

with

$$\tilde{z}_t^{(j)} = \tilde{m}_t^{(j)} + \tilde{k}_t^{(j)}.$$

Because $\tilde{k}_t^{(j)}$ now contains a direct artificial perturbation channel, the second component of $P_{xz,t}^{\text{naive}}$ contains covariance terms between the observation and that artificial variation. Those terms do not exist in the structural model, where $k_t$ is completed deterministically from the same shock path that drives $m_t$. That is the cross-covariance failure.

**Update-law failure.** The familiar UKF update equations,

$$K_t = P_{xz,t} S_t^{-1}, \qquad \hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \qquad P_{t|t} = P_{t|t-1} - K_t S_t K_t^\top,$$

do not fail algebraically. They fail semantically when $(\hat{x}_{t|t-1}, S_t, P_{xz,t})$ are computed from the wrong predictive law. Once the naive full-state approximation changes either $P_{xz,t}$ or $S_t$, it changes the gain itself:

$$K_t^{\text{naive}} = P_{xz,t}^{\text{naive}} \left(S_t^{\text{naive}}\right)^{-1} \neq P_{xz,t}^{\text{struct}} \left(S_t^{\text{struct}}\right)^{-1} = K_t^{\text{struct}}$$

in general. The existing numerical comparison later in this example shows this explicitly: adding artificial variance to $k_t$ changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$, which then changes the approximate log-likelihood contribution. The same change propagates into the Kalman gain and posterior update. So the update law is not wrong as linear algebra; it is correctly updating the wrong approximate model.

The predicted state moments are

$$\hat{x}_{t|t-1} = \sum_j w_j^{(m)} x_t^{(j)}, \tag{19.69}$$

$$P_{xx,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1}\right)\left(x_t^{(j)} - \hat{x}_{t|t-1}\right)^\top. \tag{19.70}$$

For the observation equation (19.59), define

$$z_t^{(j)} = m_t^{(j)} + k_t^{(j)}.$$

Then the predicted observation, innovation variance, and cross covariance are

$$\hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.71}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)^2 + R, \tag{19.72}$$

$$P_{xz,t} = \sum_j w_j^{(c)} \left(x_t^{(j)} - \hat{x}_{t|t-1}\right)(z_t^{(j)} - \hat{z}_t). \tag{19.73}$$

The Gaussian measurement update is

$$v_t = y_t - \hat{z}_t, \tag{19.74}$$
$$K_t = P_{xz,t} S_t^{-1}, \tag{19.75}$$
$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t v_t, \tag{19.76}$$
$$P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.77}$$

and the one-step approximate log likelihood contribution is

$$\ell_t = -\frac{1}{2}\left[\log(2\pi S_t) + \frac{v_t^2}{S_t}\right].$$

### 19.7.1 A Natural Objection: Does $\phi = 0$ Collapse the Example?

The model does not collapse just because $\phi = 0$. In that case

$$k_t = \gamma m_t^2, \qquad m_t = \rho m_{t-1} + \sigma \varepsilon_t,$$

so $k_t$ is still random whenever $\gamma \neq 0$ and the conditional law of $m_t$ is nondegenerate. What disappears is the inherited lagged-$k$ channel, not the current-shock channel. The support becomes the lower-dimensional manifold

$$\{(m, k) : k = \gamma m^2\}, \tag{19.78}$$

conditional on the parameters and the lagged state distribution. It collapses to a point only in degenerate cases, for example if $\gamma = 0$ or if $m_t$ is itself degenerate. With the numerical calibration below but setting $\phi = 0$, the recomputed structural UKF moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.04247 \end{bmatrix},$$

and the observation moments are

$$\hat{z}_t = 0.11024, \qquad S_t = 0.56807, \qquad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.04247 \end{bmatrix}.$$

The $k$ variance is smaller because lagged $k_{t-1}$ no longer feeds into $k_t$, but it is not zero: it is induced by the quadratic transformation of the shock-driven $m_t$.

### 19.7.2 Numerical Illustration

Use the parameters

$$\rho = 0.8, \quad \sigma = 0.5, \quad \phi = 0.7, \quad \gamma = 0.4, \quad R = 0.25,$$

and previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \qquad P_{t-1} = \mathrm{diag}(0.04, 0.09). \tag{19.79}$$

Use the scaled unscented convention $\alpha = 1$, $\beta = 2$, and $\kappa = 0$. Since $n_a = 3$, this gives $\lambda = \alpha^2(n_a + \kappa) - n_a = 0$, central mean weight $w_0^{(m)} = 0$, central covariance weight $w_0^{(c)} = 2$, and six off-center mean/covariance weights equal to $1/6$. The augmented covariance is

$$\mathrm{diag}(0.04, 0.09, 1),$$

so $\sqrt{(n_a+\lambda)P_a}$ has diagonal entries 0.34641, 0.51962, and 1.73205. The seven sigma points and their deterministic structural completion are:

| $j$ | $m_{t-1}^{(j)}$ | $k_{t-1}^{(j)}$ | $\varepsilon_t^{(j)}$ | $m_t^{(j)}$ | $k_t^{(j)}$ | $z_t^{(j)}$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0.34641 | 0 | 0 | 0.27713 | 0.03072 | 0.30785 |
| 2 | 0 | 0.51962 | 0 | 0 | 0.36373 | 0.36373 |
| 3 | 0 | 0 | 1.73205 | 0.86603 | 0.30000 | 1.16603 |
| 4 | −0.34641 | 0 | 0 | −0.27713 | 0.03072 | −0.24641 |
| 5 | 0 | −0.51962 | 0 | 0 | −0.36373 | −0.36373 |
| 6 | 0 | 0 | −1.73205 | −0.86603 | 0.30000 | −0.56603 |

For example, the third positive innovation point gives $m_t = 0.5(1.73205) = 0.86603$ and $k_t = 0.4(0.86603)^2 = 0.30000$. No row contains an independently sampled "new" shock for $k_t$.

Using the stated covariance weights, the structural propagation gives

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0.11024 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.08657 \end{bmatrix}.$$

The predicted observation moments are

$$\hat{z}_t = 0.11024, \qquad S_t = 0.61217, \qquad P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix}.$$

For an observation $y_t = 0.30$,

$$v_t = 0.18976, \qquad K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix},$$

and therefore

$$\hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix}, \qquad P_{t|t} = \begin{bmatrix} 0.15152 & -0.03897 \\ -0.03897 & 0.07433 \end{bmatrix}.$$

The one-step log likelihood contribution is approximately

$$\ell_t = -0.70297.$$

These numbers are illustrative calculations under the stated sigma-point convention and should be read as a worked example rather than general performance evidence.

**Side-by-side prediction/update contrast.** For this toy model, the structural UKF treats

$$a_t = (x_{t-1}, \varepsilon_t)$$

as the sigma-point object and computes propagated points by

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \qquad k_t^{(j)} = \phi k_{t-1}^{(j)} + \gamma\big(m_t^{(j)}\big)^2.$$

It therefore forms

$$P_{xz,t}^{\text{struct}} = \sum_j w_j^{(c)} \big(x_t^{(j)} - \hat{x}_{t|t-1}\big)\big(z_t^{(j)} - \hat{z}_t\big),$$

where every $x_t^{(j)}$ lies on the support generated by the structural map. A naive full-state UKF instead produces an artificial cloud $\tilde{x}_t^{(j)} = (\tilde{m}_t^{(j)}, \tilde{k}_t^{(j)})$ with

$$\tilde{k}_t^{(j)} \neq \phi k_{t-1}^{(j)} + \gamma\big(\tilde{m}_t^{(j)}\big)^2 \text{ in general,}$$

and therefore forms

$$P_{xz,t}^{\text{naive}} = \sum_j w_j^{(c)} \big(\tilde{x}_t^{(j)} - \tilde{x}_{t|t-1}\big)\big(\tilde{z}_t^{(j)} - \tilde{z}_t\big),$$

which mixes genuine shock-driven variation with artificial perturbations of the deterministic-completion coordinate.

**Structural identity test.** A simple way to see the difference is to check whether propagated points satisfy

$$k_t - \phi k_{t-1} - \gamma m_t^2 = 0.$$

The structural UKF satisfies this identity point by point for every propagated sigma point. A naive full-state UKF with direct perturbations to $k_t$ does not. That is the diagnostic place where the structural target and this naive full-state approximation part company.

Now compare this with an off-manifold full-state approximation that adds an independent artificial innovation variance 0.04 directly to $k_t$ after the structural map. Nothing in (19.57)–(19.59) justifies that extra shock. It changes the innovation variance from $S_t = 0.61217$ to $S_t = 0.65217$ and changes the same observation's log likelihood contribution from $-0.70297$ to $-0.73282$. The numerical difference is small in this toy calibration, but the conceptual difference is large: the latter filter is approximating a different approximate state-space model relative to the declared structural law.

To connect these numbers back to the three failures above, write the naive full-state approximation as using

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0 \\ 0 & 0.12657 \end{bmatrix},$$

which differs from the structural covariance only by the artificial extra variance in the $k_t$ coordinate. Under the same observation equation $z_t = m_t + k_t$, this gives

$$\tilde{S}_t = 0.65217, \qquad \tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix},$$

and therefore

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19408 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix},$$

for the same observation $y_t = 0.30$.

These values make the three failures concrete:

(i) **Prediction-law failure:** the structural update uses $P_{xx,t} = \text{diag}(0.27560, 0.08657)$, while the naive full-state approximation uses $\tilde{P}_{xx,t} = \text{diag}(0.27560, 0.12657)$ because it has inserted artificial uncertainty directly into $k_t$.

(ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.08657 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.27560 \\ 0.12657 \end{bmatrix}.$$

The second component is larger only because the naive construction lets the observation load directly on artificial variation in $k_t$.

(iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.45020 \\ 0.14141 \end{bmatrix}, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.08543 \\ 0.13707 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.42259 \\ 0.19407 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.08019 \\ 0.14707 \end{bmatrix}.$$

The update formula has not changed, but the gain and posterior correction do because they are now computed from the wrong predictive moments.

So the numerical illustration does not merely show a slightly different likelihood value. It shows that once the naive approximation changes the predictive covariance and cross covariance, the entire measurement update shifts with it.

The lesson is not that UKF is wrong. The lesson is that the UKF must be applied to the correct structural transition. Sigma points belong in the declared stochastic integration space. Endogenous deterministic coordinates are completed by the model point by point. In BayesFilter terms, this is a design rule for mixed structural models and an approximation-label boundary for any backend that instead enlarges the stochastic space.

## 19.8 Worked Example B: Degenerate Linear Transition with Nonlinear Measurement

The most practically relevant counterpart to the previous example is not a fully stochastic linear-Gaussian state transition. It is a degenerate linear transition together with a nonlinear measurement equation. That case is closer to many DSGE and affine term-structure applications than a fully nonlinear latent transition. The key point is that the latent transition may still have a structural split even when it is linear. A nonlinear measurement equation does not remove that split.

Consider the linear structural transition

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \tag{19.80}$$

$$k_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t, \tag{19.81}$$

$$x_t = (m_t, k_t), \tag{19.82}$$

and the nonlinear exponential-affine measurement equation

$$y_t = \exp(A^\top x_t + \mu) + e_t, \qquad e_t \sim \mathcal{N}(0, R). \tag{19.83}$$

The latent transition is linear, but it is still structurally degenerate: conditional on $(x_{t-1}, \varepsilon_t)$, the endogenous coordinate $k_t$ is completed deterministically from the same shock path that drives $m_t$. So the structural issue from the previous section does not disappear. What changes is where the nonlinearity appears. The latent transition is linear, while the measurement map is nonlinear.

This means there are now two layers to keep separate:

(i) the latent predictive law is still generated by the pre-transition uncertainty variables $(x_{t-1}, \varepsilon_t)$, not by direct full-state perturbations of $x_t$;

(ii) even after the latent cloud is propagated correctly, the UKF must still approximate nonlinear observation moments and cross covariances for the map $x_t \mapsto \exp(A^\top x_t + \mu)$.

The first issue is structural-modeling correctness. The second is observation- side quadrature quality.

### 19.8.1 Worked structural derivation for the degenerate linear-transition case

**Accepted assumptions.** For this section, treat the following as explicit modeling assumptions:

(A1) the innovation is conditionally independent of the lagged filtering law, so that

$$p(x_{t-1}, \varepsilon_t \mid y_{1:t-1}) = p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t); \tag{19.84}$$

(A2) the latent map, observation map, and composed map defined below are measurable and have the finite moments needed for the stated expectations;

(A3) whenever we refer to the observation-side-only error regime, we assume that the latent predictive law from (19.87) is preserved exactly;

(A4) no invertibility of $T_k$ is required: it is enough that $k_t$ is a well-defined measurable deterministic completion of $(k_{t-1}, m_{t-1}, m_t)$;

(A5) if one wants to replace the innovation variable $\varepsilon_t$ by the current exogenous state $m_t$ as the sigma-point variable, then for fixed $m_{t-1}$ the map

$$\varepsilon_t \mapsto T_m(m_{t-1}, \varepsilon_t) \tag{19.85}$$

should be injective, or invertible onto its image with measurable inverse. This extra assumption is only needed for that reparameterization step, not for the pushforward identities proved below.

**Latent predictive pushforward.** Define the degenerate linear structural map

$$F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t) = \Big(\rho m_{t-1} + \sigma\varepsilon_t,\, \phi k_{t-1} + \psi m_{t-1} + \chi(\rho m_{t-1} + \sigma\varepsilon_t)\Big). \tag{19.86}$$

Then for any bounded test function $\varphi$ on the latent-state space,

$$\mathbb{E}[\varphi(x_t) \mid y_{1:t-1}] = \iint \varphi(F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.87}$$

Hence the latent predictive law is the pushforward of the joint law of $(x_{t-1}, \varepsilon_t)$ through $F_{\mathrm{lin}}$.

*Derivation.* Under the state transition (19.80)–(19.82) and the factorization assumption (19.84), the conditional expectation of any bounded test function $\varphi(x_t)$ is obtained by substituting $x_t = F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t)$ and integrating over the joint law of $(x_{t-1}, \varepsilon_t)$. This gives (19.87), which is exactly the pushforward identity.

**Deterministic-completion identity.** Under (19.80)–(19.81), all admissible latent states satisfy

$$k_t - \phi k_{t-1} - \psi m_{t-1} - \chi m_t = 0. \tag{19.88}$$

Therefore, once $(x_{t-1}, \varepsilon_t)$ is fixed, both $m_t$ and $k_t$ are fixed; there is no independent new perturbation attached to $k_t$ in the latent transition.

*Derivation.* Substitute (19.80) into (19.81) and rearrange terms. The resulting identity is exactly (19.88).

**Observation predictive pushforward.** Define the nonlinear noise-free observation map

$$h(x_t) = \exp(A^\top x_t + \mu). \tag{19.89}$$

and the composed map

$$G(x_{t-1}, \varepsilon_t) = h(F_{\mathrm{lin}}(x_{t-1}, \varepsilon_t))\,. \tag{19.90}$$

Then for any bounded test function $\psi_0$ on the noise-free observation space,

$$\mathbb{E}\big[\psi_0\big(h(x_t)\big) \mid y_{1:t-1}\big] = \iint \psi_0(G(x_{t-1}, \varepsilon_t))\, p(x_{t-1} \mid y_{1:t-1}) p(\varepsilon_t)\, d\varepsilon_t\, dx_{t-1}. \tag{19.91}$$

Hence the noise-free observation predictive law is the pushforward of the same pre-transition uncertainty through the composed map.

*Derivation.* Compose the latent pushforward identity (19.87) with the measurable observation map (19.89). This yields (19.91).

**Naive full-state perturbation changes the latent law.** If a naive full-state UKF introduces a direct perturbation to the endogenous coordinate $k_t$, then its propagated latent sigma-point cloud does not, in general, satisfy (19.88) pointwise. Therefore it does not target the latent predictive law characterized by (19.87).

*Diagnostic check.* By the deterministic-completion identity above, every admissible latent state under the structural model satisfies (19.88). A naive full-state UKF that adds an independent perturbation to $k_t$ creates propagated points whose $k_t$ values are not determined solely by $(x_{t-1}, \varepsilon_t)$, so the identity fails in general. Hence the resulting sigma-point cloud cannot represent the same latent predictive law.

**Failure propagation and observation-side-only error.** Under the preceding worked identities:

(i) if a naive full-state UKF directly perturbs $k_t$, then prediction-law failure, cross-covariance failure, and update-law failure all still apply;

(ii) if the latent predictive law from (19.87) is preserved exactly and only the transformed observation moments from (19.91) are approximated by quadrature, then the error is not structural-model failure; it is only observation-side quadrature error.

*Reason.* Part (i) follows because the naive construction changes the latent predictive law, and the nonlinear observation law is the pushforward of that same latent law. Once the latent law changes, the observation moments, state-observation cross covariance, and therefore the UKF update objects also change. Part (ii) follows from Assumption (A3): if the latent law is preserved and only the nonlinear observation moments are approximated, then the remaining error arises only in the observation-side quadrature.

The formal statements above separate three layers that are easy to blur in practice: accepted modeling assumptions, exact pushforward identities, and the interpretation of failure modes. The filtered state is still $x_t$, but the latent predictive law is generated by the pre-transition uncertainty variables $(x_{t-1}, \varepsilon_t)$. The nonlinear measurement equation adds another approximation layer, but it does not remove the need to respect the degenerate latent transition. Assumption (A5) is stronger than the pushforward arguments need: it is included only to justify replacing $\varepsilon_t$ by $m_t$ as an equivalent sigma-point variable when that reparameterization is desired.

### 19.8.2 Worked Degenerate Linear-Transition Example

Use

$$\rho = 0.8, \qquad \sigma = 0.5, \qquad \phi = 0.7, \qquad \psi = 0.3, \qquad \chi = 0.4,$$

with previous filtering moments

$$\mu_{t-1} = (0, 0)^\top, \qquad P_{t-1} = \operatorname{diag}(0.04, 0.09), \tag{19.92}$$

and measurement parameters

$$A = \begin{bmatrix} 1.1 \\ 0.6 \end{bmatrix}, \qquad \mu = 0.10, \qquad R = 0.04.$$

Because the latent transition remains structurally degenerate, the correct sigma-point object is still

$$a_t = (x_{t-1}, \varepsilon_t),$$

not a naively perturbed full post-transition state. Using the same scaled unscented construction as in the previous example, the propagated latent points are obtained from

$$m_t^{(j)} = \rho m_{t-1}^{(j)} + \sigma \varepsilon_t^{(j)}, \tag{19.93}$$

$$k_t^{(j)} = \phi k_{t-1}^{(j)} + \psi m_{t-1}^{(j)} + \chi m_t^{(j)}. \tag{19.94}$$

For each propagated point, define the nonlinear noise-free observation

$$z_t^{(j)} = \exp\left(1.1 m_t^{(j)} + 0.6 k_t^{(j)} + 0.10\right).$$

The same seven sigma points used earlier now produce the following latent and measurement values:

| $j$ | $m_{t-1}^{(j)}$ | $k_{t-1}^{(j)}$ | $\varepsilon_t^{(j)}$ | $m_t^{(j)}$ | $k_t^{(j)}$ | $z_t^{(j)}$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 1.10517 |
| 1 | 0.34641 | 0 | 0 | 0.27713 | 0.21477 | 1.70524 |
| 2 | 0 | 0.51962 | 0 | 0 | 0.36373 | 1.37470 |
| 3 | 0 | 0 | 1.73205 | 0.86603 | 0.34641 | 3.52709 |
| 4 | −0.34641 | 0 | 0 | −0.27713 | −0.21477 | 0.71626 |
| 5 | 0 | −0.51962 | 0 | 0 | −0.36373 | 0.88848 |
| 6 | 0 | 0 | −1.73205 | −0.86603 | −0.34641 | 0.34629 |

The predicted latent moments are

$$\hat{x}_{t|t-1} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix}.$$

The nonlinear predicted observation moments are

$$\hat{z}_t = 1.42635, \qquad S_t = 1.32191, \qquad P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix}.$$

So the structural UKF gain is

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}.$$

If the realized observation is $y_t = 1.50$, then

$$v_t = 0.07365, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix}.$$

As in the previous example, these numbers are illustrative calculations under the stated sigma-point convention. They demonstrate the law and moment objects being compared; they are not standalone performance evidence for a production filter.

Now compare this with a naive full-state approximation that adds an artificial latent perturbation variance 0.04 directly to the deterministic-completion coordinate $k_t$. To keep the comparison reproducible, take the naive rule to form a four-dimensional sigma-point variable

$$\tilde{a}_t = (x_{t-1}, \varepsilon_t, \delta_t), \qquad \delta_t \sim \mathcal{N}(0, 0.04),$$

and to propagate

$$\tilde{k}_t = \phi k_{t-1} + \psi m_{t-1} + \chi m_t + \delta_t.$$

That artificial perturbation changes the latent predictive covariance to

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

while leaving the mean unchanged. Under the same nonlinear measurement map, the corresponding naive moment approximation is

$$\tilde{z}_t = 1.44483, \qquad \tilde{S}_t = 1.57838, \qquad \tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

Hence the naive full-state gain becomes

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix},$$

and for the same observation $y_t = 1.50$ the naive posterior mean update is

$$\tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

These numbers make the three failures concrete in this linear-transition / nonlinear-measurement setting:

(i) **Prediction-law failure:** the structural latent law uses

$$P_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.09948 \end{bmatrix},$$

whereas the naive full-state approximation uses

$$\tilde{P}_{xx,t} = \begin{bmatrix} 0.27560 & 0.11984 \\ 0.11984 & 0.13948 \end{bmatrix},$$

because it has inserted artificial uncertainty directly into $k_t$.

(ii) **Cross-covariance failure:** the structural update uses

$$P_{xz,t} = \begin{bmatrix} 0.50479 \\ 0.24852 \end{bmatrix},$$

whereas the naive approximation uses

$$\tilde{P}_{xz,t} = \begin{bmatrix} 0.53757 \\ 0.28867 \end{bmatrix}.$$

The extra direct perturbation of $k_t$ changes how the nonlinear observation loads on the second state coordinate.

(iii) **Update-law failure:** the structural gain and posterior mean are

$$K_t = \begin{bmatrix} 0.38186 \\ 0.18800 \end{bmatrix}, \qquad \hat{x}_{t|t} = \begin{bmatrix} 0.02813 \\ 0.01385 \end{bmatrix},$$

whereas the naive full-state approximation gives

$$\tilde{K}_t = \begin{bmatrix} 0.34058 \\ 0.18289 \end{bmatrix}, \qquad \tilde{x}_{t|t} = \begin{bmatrix} 0.01879 \\ 0.01009 \end{bmatrix}.$$

The update formula itself is unchanged, but the predicted moments and cross covariance are not, so the gain and posterior correction are computed for a different approximate model.

### 19.8.3 What Is Similar to the Structural-Deterministic Case?

The similarity is that the structural latent transition is still generated by $(x_{t-1}, \varepsilon_t)$, not by independent perturbations of every coordinate of $x_t$. If a naive full-state UKF directly perturbs $k_t$, it again changes the latent predictive law before the observation map is even evaluated. So the structural warning remains active even though the latent transition is linear.

The nonlinear observation adds a second approximation layer. Once the latent cloud is propagated correctly, the UKF still has to approximate

$$\hat{y}_t, \qquad S_t, \qquad P_{xz,t}$$

for the map $x_t \mapsto \exp(A^\top x_t + \mu)$. Poor quadrature here still changes the gain and posterior update, just as in other nonlinear Gaussian filters.

### 19.8.4 What Is Different from the Earlier Nonlinear-State Example?

In the earlier toy example, the latent state equation itself was nonlinear, so the UKF had to approximate both the latent pushforward through the nonlinear state map and the resulting observation moments. Here the latent transition is linear, but structurally degenerate. That removes one source of approximation error but not the structural state-partition issue. The UKF still needs the correct pre-transition uncertainty variable because $k_t$ remains a deterministic-completion coordinate.

So this example isolates a different practical regime:

(i) the latent transition is linear but not full-state stochastic;

(ii) the nonlinear burden sits in the measurement equation;

(iii) the UKF can still fail in two distinct ways:

- structurally, if it perturbs $k_t$ directly and therefore changes the latent predictive law;
- numerically, if it uses the correct latent construction but poorly approximates the nonlinear observation moments.

This is exactly why the structural deterministic-dynamics chapter should not be read as applying only to nonlinear state transitions. The same latent state-partition discipline remains necessary when the state equation is linear but degenerate and the observation map is nonlinear.

## 19.9 What Structural Correctness Does and Does Not Guarantee

Structural correctness guarantees that the filter is approximating the intended latent predictive law rather than an altered full-state approximation. It does not, by itself, guarantee that the resulting Gaussian closure is exact, that nonlinear observation moments are approximated with negligible error, or that the derivative path is stable enough

for HMC. In BayesFilter terms, structural correctness is a law-specification requirement; it is not a blanket certificate of moment accuracy, derivative accuracy, or production readiness.

In particular, structural correctness does not solve four other problems that must be evaluated separately:

(i) it does not make the Gaussian sigma-point closure exact for a genuinely nonlinear transformation;

(ii) it does not remove observation-side quadrature error in models such as (19.83);

(iii) it does not resolve numerical covariance-factor issues when square roots, SVD factors, or PSD projections are used;

(iv) it does not guarantee a stable value-gradient path for HMC, especially when spectral decompositions or fallback branches are differentiated.

Thus the right mental model is layered: first make sure the latent law is correct, then measure the quality of the nonlinear approximation, then audit the numerical and derivative path.

## 19.10 Structural Degeneracy, Numerical Degeneracy, and SVD

The literature and the source-project audit motivate a distinction that should be made explicit:

| Issue | What it means | What fixes it |
|---|---|---|
| Structural degeneracy | Some latent coordinates are deterministic-completion variables under the model law | Respect the declared stochastic integration space and complete the deterministic block pointwise |
| Numerical degeneracy | A covariance or innovation matrix is singular or ill-conditioned in computation | Use a stable solve, square-root, or spectral factorization appropriate to the declared law |
| SVD/square-root representation | A numerical backend chooses a robust factorization of a covariance object | This may improve value-side robustness, but it does not by itself correct a structurally wrong latent law |
| Derivative/spectral risk | Raw autodiff through spectral or fallback branches may be unstable | Requires derivative audits, stress tests, and possibly custom gradients |

This is why an SVD sigma-point filter does not automatically solve the problem discussed in this chapter. It may solve a factorization problem while leaving the latent-law specification problem untouched. Conversely, a structural SVD sigma-point filter is a sensible backend when it factors the covariance of the declared pre-transition variable,

propagates those points through $F_\theta$, and records the usual value and derivative diagnostics from Chapters 17 and 18. The question is therefore not "SVD or structural UKF?" The correct question is:

$$\text{Which law is being factored and propagated?} \tag{19.95}$$

**Proposition 19.3** (Degenerate covariance and formal sigma-point accuracy)**.** *Let $A_\delta = \mu + \delta Z$ be a possibly degenerate Gaussian random variable in $\mathbb{R}^L$, with $\mathbb{E}[Z] = 0$ and $\mathrm{Cov}(Z) = \Sigma \succeq 0$. Thus $\mathrm{Cov}(A_\delta) = P_\delta = \delta^2\Sigma$ may be singular. Let $\{a_\delta^{(j)}, w_j^{(m)}, w_j^{(c)}\}$ be any sigma-point rule whose deviations $\xi_j = a_\delta^{(j)} - \mu = \delta\zeta_j$ have fixed standardized points $\zeta_j$ and satisfy*

$$\sum_j w_j^{(m)}\xi_j = 0, \qquad \sum_j w_j^{(m)}\xi_j\xi_j^\top = P_\delta, \qquad \sum_j w_j^{(c)}\xi_j\xi_j^\top = P_\delta.$$

*If $G : \mathbb{R}^L \to \mathbb{R}^d$ is three-times continuously differentiable and its third-order Taylor remainders around $\mu$ have expected size $O(\delta^3)$ under the Gaussian law and weighted size $O(\delta^3)$ under the sigma-point cloud, then the sigma-point transformed mean and covariance have the same local order as in the nonsingular case:*

$$\mathbb{E}[G_r(A_\delta)] = G_r(\mu) + \frac{1}{2}\,\mathrm{tr}(H_r P_\delta) + O(\delta^3), \tag{19.96}$$

$$\hat{g}_r^{\mathrm{SP}} = G_r(\mu) + \frac{1}{2}\,\mathrm{tr}(H_r P_\delta) + O(\delta^3), \tag{19.97}$$

$$\mathrm{Cov}(G(A_\delta)) = JP_\delta J^\top + O(\delta^3), \tag{19.98}$$

$$P_G^{\mathrm{SP}} = JP_\delta J^\top + O(\delta^3), \tag{19.99}$$

*where $J$ is the Jacobian of $G$ at $\mu$ and $H_r$ is the Hessian of the $r$th component. Positive definiteness of $P_\delta$ is not used. Consequently, abstracting from finite arithmetic and factor-construction errors, rank deficiency of the state-noise covariance does not by itself lower the formal local sigma-point moment accuracy for the law being propagated.*

*Proof.* The usual Taylor proof only uses the first two moments of the input variable and the matching first two weighted moments of the sigma-point cloud. For a component $r$,

$$G_r(\mu + \Delta) = G_r(\mu) + J_r\Delta + \frac{1}{2}\Delta^\top H_r\Delta + O(\|\Delta\|^3). \tag{19.100}$$

Taking expectations with $\Delta = A_\delta - \mu$ gives the first expansion because $\mathbb{E}[\Delta] = 0$ and $\mathbb{E}[\Delta\Delta^\top] = P_\delta$. Taking the weighted sum with $\Delta = \xi_j$ gives the sigma-point mean expansion by the same identities. For covariance, the leading centered term is $J\Delta$, whose second moment is $JP_\delta J^\top$ both for the true Gaussian law and for the sigma-point cloud. The quadratic remainder terms enter at order $O(\delta^3)$ or smaller. No inverse, Cholesky pivot, or full-rank support argument appears in this proof, so the conclusion is unchanged when $P_\delta$ is only positive semidefinite. □

Proposition 19.3 verifies the reviewer's claim in its precise value-side form. If a model is written as a full-state transition with a degenerate Gaussian innovation, and if that transition induces exactly the same conditional law as the structural pushforward in Proposition 19.1, then an exact SVD sigma-point rule for that degenerate law has the same formal local moment order as the corresponding nondegenerate rule. Degeneracy

is not, by itself, a mathematical objection to sigma-point filtering. This is a formal value-side statement abstracting from finite arithmetic, factor-construction choices, and derivative-branch effects. Those implementation issues still require the diagnostics below.

That statement is an equivalence result, not a recommendation to hide the structural split inside a large singular covariance matrix. To make the comparison precise, let the structural route use

$$A_t = (x_{t-1}, \varepsilon_t), \qquad x_t = F_\theta(A_t), \qquad \pi_t^x = (F_\theta)_\# \pi_t^a,$$

where $\pi_t^a = \pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon$. Let a collapsed full-state SVD route instead use an augmented variable

$$\widetilde{A}_t = (x_{t-1}, \widetilde{u}_t), \qquad \widetilde{u}_t \sim \mathcal{N}(0, \widetilde{Q}_t), \qquad \widetilde{x}_t = \widetilde{F}_\theta(\widetilde{A}_t),$$

with $\widetilde{Q}_t = \widetilde{C}_t \widetilde{C}_t^\top$ obtained by an SVD or spectral rule. The collapsed route is acceptable only when the following mathematical diagnostics are satisfied.

(i) **Law declaration is pushforward equality.** The two declarations are equivalent only if

$$(\widetilde{F}_\theta)_\# \left( \pi_{t-1|t-1}^x \otimes \mathcal{N}(0, \widetilde{Q}_t) \right) = (F_\theta)_\# \left( \pi_{t-1|t-1}^x \otimes \nu_\theta^\varepsilon \right),$$

or, equivalently, if $\mathbb{E}[\varphi(\widetilde{x}_t) \mid y_{1:t-1}] = \mathbb{E}[\varphi(x_t) \mid y_{1:t-1}]$ for every bounded test function $\varphi$. Matching a covariance matrix is only a necessary moment check, not this equality of laws.

*Example.* If $m_t = \varepsilon_t$ and $k_t = \varepsilon_t^2$, then the structural conditional law is supported on the parabola $\{(m, k) : k = m^2\}$. A collapsed additive Gaussian transition $\widetilde{x}_t = \mu + \widetilde{u}_t$ with any covariance $\widetilde{Q}_t$ has affine Gaussian support. It cannot equal the parabolic pushforward unless $\varepsilon_t$ is degenerate. A singular covariance matrix by itself therefore does not encode the nonlinear timing contract.

(ii) **Sigma-point support is a constraint residual.** Let $c_\theta(a, x) = 0$ collect the deterministic identities implied by the structural transition, for example $c_\theta(A_t, x_t) = x_t - F_\theta(A_t)$. A structural sigma-point cloud satisfies

$$\Delta_{\mathrm{id}}^{\mathrm{str}} = \max_j \| c_\theta(a_t^{(j)}, F_\theta(a_t^{(j)})) \| = 0.$$

The collapsed cloud must satisfy the corresponding check

$$\Delta_{\mathrm{id}}^{\mathrm{col}} = \max_j \| c_\theta(\widetilde{a}_t^{(j)}, \widetilde{F}_\theta(\widetilde{a}_t^{(j)})) \| = 0,$$

after translating its variables into the structural coordinates. If this residual is not defined because the collapsed adapter no longer exposes the relevant shock or timing variables, the adapter cannot demonstrate structural equivalence.

*Example.* For the linear deterministic identity $x_t = (m_t, k_t) = (\varepsilon_t, \varepsilon_t)$, the support condition is $c(m, k) = k - m = 0$. The exact rank-one covariance $\widetilde{Q} = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ has an SVD factor whose points lie on $k = m$. After a nugget, $\widetilde{Q}_\eta = \widetilde{Q} + \eta^2 I$, sigma points have normal coordinate displacement $\sqrt{L + \lambda}\, \eta$ in the direction $(1, -1)/\sqrt{2}$, so $\Delta_{\mathrm{id}}^{\mathrm{col}} > 0$.

(iii) **Dimension and cost are point-count identities.** For a symmetric $2L+1$ rule, the number of model evaluations is determined by the dimension $L$ of the factored variable. If the previous state has dimension $n$ and the declared shock has dimension $q$, the structural augmented rule uses

$$N_{\mathrm{str}} = 2(n+q)+1.$$

A full-state additive-noise route that augments by an $n$-dimensional degenerate noise vector uses

$$N_{\mathrm{col}} = 2(n+n)+1, \qquad N_{\mathrm{col}} - N_{\mathrm{str}} = 2(n-q).$$

This overhead is pure bookkeeping if the extra $n-q$ directions are zero-noise directions.

*Example.* With $n = 100$ state coordinates and $q = 10$ genuine shocks, the structural augmented rule uses 221 points, while the full-state degenerate-noise augmentation uses 401 points. In the two-coordinate toy covariance $\mathrm{diag}(1,0)$ treated as a two-dimensional factor, the zero column produces two sigma points equal to the mean; they add evaluations and alter the weight bookkeeping without adding stochastic information.

(iv) **Finite arithmetic is normal-support leakage.** Let $M_t$ be a matrix whose columns span the exact linear support of a degenerate Gaussian factor, and let $N_t$ span the normal space $(\mathrm{col}\, M_t)^{\perp}$. Exact support preservation requires

$$N_t^{\top} \widetilde{C}_t = 0 \quad \Longleftrightarrow \quad N_t^{\top} \widetilde{Q}_t N_t = 0.$$

A nugget, PSD projection, rank threshold, or asymmetric reconstruction should therefore be reported through

$$\ell_{\perp,t} = \|N_t^{\top} \widetilde{Q}_t N_t\|_{\mathrm{op}} \quad \text{or} \quad \max_j \|N_t^{\top}(\widetilde{x}_t^{(j)} - \bar{x}_t)\|.$$

*Example.* In the identity example $k = m$, the normal vector is $N = (1,-1)^{\top}/\sqrt{2}$. For $\widetilde{Q}_\eta = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right) + \eta^2 I$,

$$N^{\top} \widetilde{Q}_\eta N = \eta^2.$$

Hence the nominally deterministic relation has positive artificial variance $\eta^2$ in finite arithmetic. This is not a harmless representation of the same law; it is a regularized law unless explicitly removed before propagation.

(v) **Gradient risk is controlled by spectral gaps.** If $\widetilde{C}_t(\theta)$ is produced from an eigendecomposition or SVD, derivatives of the spectral vectors contain gap denominators. For a symmetric eigendecomposition, schematically,

$$du_i = \sum_{j \neq i} u_j \frac{u_j^{\top}(d\widetilde{Q}_t)u_i}{\lambda_i - \lambda_j} + \text{terms along } u_i.$$

Therefore the gradient audit must track

$$g_t^{\min} = \min_{i \neq j} |\lambda_i(\widetilde{Q}_t) - \lambda_j(\widetilde{Q}_t)|, \qquad \|\partial_\theta \widetilde{C}_t\| \lesssim \frac{\|\partial_\theta \widetilde{Q}_t\|}{g_t^{\min}}$$

away from rank and ordering branches. Structural completion does not remove factor-gradient issues, but it avoids adding spectral choices for artificial zero-noise coordinates.

*Example.* Let $\widetilde{Q}(\theta) = \left(\begin{smallmatrix} 1 & \theta \\ \theta & 1 \end{smallmatrix}\right)$. The eigenvalues are $1+\theta$ and $1-\theta$, with gap $2|\theta|$. At $\theta = 0$ the covariance is perfectly smooth but the ordered eigenspace is not unique; raw autodiff through the spectral factor can be discontinuous or unstable even though the value-side covariance is benign.

(vi) **Diagnostics must separate model law from regularization.** A collapsed route should report both the identity residual ((ii)) and the normal leakage ((iv)). Otherwise a finite likelihood can be produced by a different state-space model. For a scalar observation that loads on a deterministic coordinate, this difference is explicit: if the structural law is

$$k_t = 0, \qquad y_t = k_t + e_t, \qquad e_t \sim \mathcal{N}(0, r),$$

then $y_t \sim \mathcal{N}(0, r)$. If a collapsed covariance injects artificial variance $\eta^2$ into $k_t$, then the implemented likelihood uses $y_t \sim \mathcal{N}(0, r+\eta^2)$, and the one-period log-likelihood change is

$$\Delta\ell_t = -\frac{1}{2}\left[\log\frac{r+\eta^2}{r} + y_t^2\left(\frac{1}{r+\eta^2} - \frac{1}{r}\right)\right].$$

A finite value of this likelihood is therefore not evidence that the structural transition was respected; it may only show that the nugget made the altered model numerically evaluable.

These diagnostics matter for the UKF because the UKF update is only as correct as the sigma-point cloud used to build its predictive moments. For a noise-free observation map $h_\theta$ and observation noise covariance $R_t$, the structural UKF forms points

$$x_t^{(j)} = F_\theta(a_t^{(j)}), \qquad z_t^{(j)} = h_\theta(x_t^{(j)}),$$

and then computes

$$\hat{x}_t = \sum_j w_j^{(m)} x_t^{(j)}, \qquad \hat{z}_t = \sum_j w_j^{(m)} z_t^{(j)}, \tag{19.101}$$

$$P_{xx,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(x_t^{(j)} - \hat{x}_t)^\top, \tag{19.102}$$

$$S_t = \sum_j w_j^{(c)} (z_t^{(j)} - \hat{z}_t)(z_t^{(j)} - \hat{z}_t)^\top + R_t, \tag{19.103}$$

$$P_{xz,t} = \sum_j w_j^{(c)} (x_t^{(j)} - \hat{x}_t)(z_t^{(j)} - \hat{z}_t)^\top. \tag{19.104}$$

The Gaussian UKF update then uses

$$v_t = y_t - \hat{z}_t, \qquad K_t = P_{xz,t} S_t^{-1}, \tag{19.105}$$

$$\hat{x}_{t|t} = \hat{x}_t + K_t v_t, \qquad P_{t|t} = P_{xx,t} - K_t S_t K_t^\top, \tag{19.106}$$

with log-likelihood contribution

$$\ell_t = -\frac{1}{2}\left[d_y \log(2\pi) + \log\det S_t + v_t^\top S_t^{-1} v_t\right]. \tag{19.107}$$

Thus a collapsed SVD route is not merely changing an implementation detail. It replaces the point cloud $\{F_\theta(a_t^{(j)})\}$ in (19.104) by $\{\widetilde{F}_\theta(\widetilde{a}_t^{(j)})\}$. The error enters through

$$\Delta\hat{z}_t = \widetilde{\hat{z}}_t - \hat{z}_t, \tag{19.108}$$

$$\Delta S_t = \widetilde{S}_t - S_t, \tag{19.109}$$

$$\Delta P_{xz,t} = \widetilde{P}_{xz,t} - P_{xz,t}, \tag{19.110}$$

$$\Delta K_t = \widetilde{P}_{xz,t}\widetilde{S}_t^{-1} - P_{xz,t}S_t^{-1}. \tag{19.111}$$

Even when the formal sigma-point rule is accurate for its own input law, (19.111) is controlled by whether the collapsed cloud matches the structural cloud on the particular functions $h_\theta$, $x\,h_\theta(x)^\top$, and $h_\theta(x)h_\theta(x)^\top$ used in the UKF update. Equality of predictive laws is the robust sufficient condition; accidental equality of a few low-order moments is not a model-contract proof. This is the precise UKF sense in which the law-level diagnostics above are not optional metadata.

The six diagnostics above correspond to the UKF recursion as follows.

(i) **Pushforward equality** is the condition that the two point clouds approximate the same expectations in (19.104). If ((i)) fails, then $\widetilde{\hat{x}}_t$, $\widetilde{\hat{z}}_t$, $\widetilde{S}_t$, and $\widetilde{P}_{xz,t}$ are moments of a different predictive law.

(ii) **Identity residuals** test whether every propagated point used in (19.104) is an admissible structural state. If $\Delta_{\mathrm{id}}^{\mathrm{col}} > 0$, then the UKF averages include states that the DSGE transition assigns probability zero.

(iii) **Point-count overhead** affects the number of evaluations of $F_\theta$ and $h_\theta$ required to build the same objects $\hat{x}_t, \hat{z}_t, S_t, P_{xz,t}$. Extra zero-noise directions do not add information; they add factorization and map-evaluation work before the update in (19.106).

(iv) **Normal-support leakage** enters the UKF through $S_t$ and $P_{xz,t}$. Artificial variance in a deterministic-completion direction changes the innovation covariance and the state-observation cross covariance, so it changes both the likelihood (19.107) and the gain $K_t = P_{xz,t}S_t^{-1}$.

(v) **Spectral-gap gradient risk** is the derivative of the same UKF likelihood and update with respect to parameters. If $\partial_\theta \widetilde{C}_t$ is unstable, then the gradients of $\widetilde{S}_t$, $\widetilde{P}_{xz,t}$, $\widetilde{K}_t$, and $\widetilde{\ell}_t$ inherit that instability.

(vi) **Diagnostic separation** tells the reader whether (19.106) and (19.107) were evaluated under the structural law or under a regularized law. Without the identity and leakage diagnostics, a finite UKF likelihood cannot distinguish those two cases.

A one-dimensional observation example shows the mechanism without requiring filtering expertise. Let

$$m_t = \varepsilon_t, \qquad k_t = m_t, \qquad y_t = k_t + e_t, \qquad \varepsilon_t \sim \mathcal{N}(0,q), \quad e_t \sim \mathcal{N}(0,r). \tag{19.112}$$

The structural predictive state has

$$\mathrm{Var}(m_t) = q, \qquad \mathrm{Var}(k_t) = q, \qquad \mathrm{Cov}(m_t,k_t) = q, \qquad S_t^{\mathrm{str}} = q + r.$$

The Kalman gain components implied by the UKF moment objects are therefore

$$K_t^{\mathrm{str}} = \frac{\mathrm{Cov}((m_t, k_t), y_t)}{\mathrm{Var}(y_t)} = \frac{1}{q+r}\begin{pmatrix} q \\ q \end{pmatrix}.$$

If the collapsed SVD route adds a small independent numerical variance $\eta^2$ to the deterministic-completion coordinate $k_t$, then

$$\widetilde{k}_t = m_t + \eta\xi_t, \qquad \xi_t \sim \mathcal{N}(0,1), \tag{19.113}$$

and the UKF objects become

$$\widetilde{S}_t = q + \eta^2 + r, \qquad \widetilde{K}_t = \frac{1}{q+\eta^2+r}\begin{pmatrix} q \\ q+\eta^2 \end{pmatrix}.$$

The likelihood and update have changed:

$$\widetilde{\ell}_t - \ell_t = -\frac{1}{2}\left[\log\frac{q+\eta^2+r}{q+r} + y_t^2\left(\frac{1}{q+\eta^2+r} - \frac{1}{q+r}\right)\right], \tag{19.114}$$

$$\widetilde{K}_{m,t} - K_{m,t} = -\frac{q\eta^2}{(q+r)(q+\eta^2+r)}, \tag{19.115}$$

$$\widetilde{K}_{k,t} - K_{k,t} = \frac{\eta^2 r}{(q+r)(q+\eta^2+r)}. \tag{19.116}$$

In this example, the six diagnostics above appear in concrete form:

(i) **Pushforward equality** is the difference between the structural law $k_t = m_t$ and the collapsed law $\widetilde{k}_t = m_t + \eta\xi_t$. If $\eta > 0$, the predictive law is no longer supported on $k = m$, and the UKF moments in (19.10) are not the moments in (19.10).

(ii) **Identity residuals** are measured by $c(m,k) = k - m$. The structural points have $c = 0$ pointwise, while the collapsed points have $c(\widetilde{m}_t, \widetilde{k}_t) = \eta\xi_t$ and therefore positive residual whenever the artificial direction is sampled.

(iii) **Point-count overhead** is hidden in the construction of $\widetilde{k}_t$. The structural rule needs points for the one genuine shock $\varepsilon_t$; the collapsed route adds a second numerical direction $\xi_t$ before computing the same scalar observation $y_t$.

(iv) **Normal-support leakage** is the extra term $\eta^2$ in $\widetilde{S}_t = q + \eta^2 + r$ in (19.10). That term is exactly the artificial variance normal to the structural line $k = m$, and it changes the likelihood and gain in (19.116).

(v) **Spectral-gradient risk** enters if $\eta^2$ is produced by an SVD threshold, nugget branch, or spectral factor of a nearly singular covariance. Then derivatives of $\widetilde{S}_t$, $\widetilde{K}_t$, and $\widetilde{\ell}_t$ inherit the derivative behavior of that spectral branch.

(vi) **Diagnostic separation** is the need to report whether the likelihood is based on $S_t^{\mathrm{str}} = q + r$ or on $\widetilde{S}_t = q + \eta^2 + r$. Both are finite UKF likelihoods, but (19.116) shows that they are not the same likelihood unless $\eta = 0$.

So the nugget changes both the likelihood weight assigned to the observation and the way the observation is fed back into $m_t$ and $k_t$. This is the UKF problem in concrete terms: the filter can remain numerically stable and still perform the prediction, likelihood evaluation, and update for a different state-space model.

The practical conclusion is therefore conservative. SVD is a good numerical backend for singular or nearly singular covariance factors, and it can be used inside a structurally correct sigma-point filter. But using SVD on a collapsed full-state covariance is usually a harder implementation contract: it has more linear algebra to audit, more fragile derivatives for HMC, and more ways for a regularization choice to become an unrecorded model change. The structural adapter is preferred because it states the economic law directly and leaves SVD to do the narrower job of factoring the covariance that actually belongs to that law.

For HMC promotion, one more distinction matters. Matrix backpropagation formulas for spectral decompositions contain denominators involving singular-value or eigenvalue gaps, so repeated or nearly repeated spectral values are derivative-risk regions [Ionescu et al., 2015]. That risk is orthogonal to the structural-law issue. A structurally correct SVD sigma-point value path still needs spectral-gap telemetry, finite-gradient stress tests, and compiled parity before it can be treated as a production HMC target.

## 19.11 Pruned DSGE and Adapter Implications

For pruned second-order DSGE approximations, the source DSGE project already uses the standard first-order/second-order decomposition:

$$x_t^f = h_x x_{t-1}^f + \eta \varepsilon_t, \tag{19.117}$$

$$x_t^s = h_x x_{t-1}^s + \frac{1}{2} H_{xx}(x_{t-1}^f \otimes x_{t-1}^f) + \frac{1}{2} h_{ss}, \tag{19.118}$$

$$y_t = g_x(x_t^f + x_t^s) + \frac{1}{2} G_{xx}(x_t^f \otimes x_t^f) + \frac{1}{2} g_{ss} + e_t. \tag{19.119}$$

That decomposition is necessary, but it is not sufficient for large DSGE filtering. It separates perturbation order; it does not automatically separate exogenous shock states from endogenous deterministic states inside $x_t^f$. A robust BayesFilter DSGE adapter should therefore carry both bookkeeping axes,

$$\text{perturbation order: } (x^f, x^s), \qquad \text{structural timing: } (m, k). \tag{19.120}$$

The adapter-facing implication is simple: the model must expose the exogenous transition, the deterministic endogenous transition, the observation map, and a declaration of which variables define the stochastic integration space. For a sigma-point implementation, that adapter contract should compile to the following concrete objects:

(i) dimensions and names for the filtered state, the declared innovation, and deterministic-completion coordinates;

(ii) a function that maps each point $(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$ to $x_t^{(j)} = F_\theta(x_{t-1}^{(j)}, \varepsilon_t^{(j)})$;

(iii) a deterministic-identity residual, evaluated on every propagated point;

(iv) an observation map evaluated only after structural completion;

(v) an approximation label if any legacy full-state or regularized covariance path is used instead.

The pruned perturbation decomposition tells the adapter what equations exist; the structural metadata tells the filter which variables receive quadrature.

## 19.12 Source-Project Lesson

The recent source-project audit found a concrete instance of the structural failure discussed in this chapter. The default DSGE SVD sigma-point adapter uses a collapsed transition of the form

$$Q_w = \eta\eta^\top + \epsilon I, \qquad x^f_{t|t-1} = h_x x^f_{t-1|t-1},$$

and propagates the covariance as

$$P_{t|t-1} = h_x P_{t-1|t-1} h_x^\top + Q_w.$$

It tracks the pruned $x^s$ correction separately, but it does not expose a partition between exogenous and endogenous coordinates inside $x^f$. For the smallest NK example, where the states are only the exogenous demand and monetary-policy shocks, this is not a serious modeling error. For Rotemberg NK, SGU, EZ, and NAWM-scale DSGE models, however, the state vector contains endogenous/predetermined components, so treating every coordinate as an exchangeable noisy Gaussian coordinate becomes a structural filtering bug rather than a mere numerical tuning issue.

This does not mean the generic SVD sigma-point machinery is unusable. The source audit found a more precise split: a generic sigma-point path that accepts user-supplied transition points or a structural transition map can be reused as an approximate backend, while the default mixed-DSGE adapter must be gated unless it exposes the exogenous/endogenous partition and deterministic completion. In other words, reuse the factorization and diagnostics; do not reuse an adapter that silently changes the stochastic integration space.

## 19.13 Validation Gates and Final Policy Rule

No future BayesFilter report should claim that a nonlinear DSGE filter is value-valid, gradient-valid, sampler-usable, converged, or production-ready in the sense of Chapter 58 unless the following structural tests are passed or explicitly waived with written justification:

1. **Metadata test:** the model exposes exogenous, endogenous, and deterministic state blocks, and their dimensions match the shock-impact matrix and transition maps.

2. **Constraint-support test:** propagated sigma points or particles satisfy deterministic identities to numerical tolerance.

3. **Linear recovery test:** when the structural transition is linear, the structural nonlinear filter recovers the exact Kalman likelihood up to the declared approximation tolerance.

4. **Degenerate-transition test:** zero-innovation endogenous rows remain deterministic except for explicitly declared numerical regularization.

5. **SVD/factorization test:** if a square-root or SVD sigma-point backend is used, diagnostics state whether the factor belongs to the structural integration variable or to a legacy full-state approximation; the latter must carry an approximation label.

6. **Toy DSGE test:** a controlled model with

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = f(k_{t-1}, m_{t-1}, m_t),$$

compares structural propagation against a dense quadrature or high-particle reference.

7. **Case-study test:** Rotemberg NK, SGU, and any EZ/NAWM target must document which state coordinates are shock-driven, which are deterministic conditional on the shock path, and how the filter honors that timing.

8. **Derivative-promotion test:** any HMC claim through SVD or eigenspectral factors must pass spectral-gap, finite-gradient, and eager/compiled parity checks.

### 19.13.1 Common Misunderstandings

The most common reader errors in this area are:

(i) a deterministic endogenous state need not have zero one-step predictive variance; it can inherit randomness through the declared stochastic block;

(ii) a singular or rank-deficient transition covariance is not, by itself, a modeling error in the exact linear-Gaussian case;

(iii) numerical regularization is not the same thing as declaring a new stochastic state coordinate;

(iv) a full-state nonlinear approximation is not automatically forbidden, but it must be labeled as an approximation if it enlarges the stochastic space beyond the structural transition contract.

> For nonlinear DSGE filtering, the stochastic integration variables are not automatically the entries of the state vector. They are the variables declared stochastic by the structural transition. Deterministic endogenous states must be computed, not noised.

If an implementation violates this rule, it must be labeled as an artificial approximation and justified against a reference. Otherwise, the filter is solving a different approximate state-space model relative to the declared structural law. Failing one of the validation gates above does not automatically make a backend useless, but it does mean the strongest claim labels from Chapter 58 are not yet justified.

# Part V

# Particle Foundations and Proposal Transport

# Chapter 20

# Particle-Filter Foundations

This chapter develops the particle-filter baseline for the differentiable particle-filter program. The goal is not yet to make the filter differentiable. The goal is to establish, in BayesFilter notation, the nonlinear filtering recursion, the empirical particle approximation, the bootstrap particle-filter likelihood estimator, and the exact point at which the later differentiable program departs from the classical construction.

Particle methods matter because the nonlinear filtering problem is generally not closed under finite-dimensional moments. When the predictive law or filtering law is materially non-Gaussian, a Gaussian closure such as the EKF or UKF may be a useful approximation but no longer defines the exact filtering object. The particle filter instead approximates the filtering law directly as a weighted random measure. This makes it the natural baseline for the particle-flow, proposal-correction, resampling-relaxed, and differentiable transport staircase that follows.

## 20.1 Objects, assumptions, and claim-status vocabulary

The later DPF chapters rely on this chapter as the reference object. The following notation is therefore fixed before introducing any differentiable relaxation.

| Object | Meaning in this chapter |
|---|---|
| $x_t \in \mathbb{R}^{n_x}$ | latent state at time $t$ |
| $y_t \in \mathbb{R}^{n_y}$ | observation at time $t$ |
| $\theta \in \Theta$ | fixed model parameter while the filtering recursion is defined |
| $p_\theta(x_t \mid x_{t-1})$ | transition density or Markov kernel density |
| $p_\theta(y_t \mid x_t)$ | observation density, evaluated at the realized observation $y_t$ |
| $p_\theta(x_t \mid y_{1:t-1})$ | predictive law before assimilating $y_t$ |
| $p_\theta(x_t \mid y_{1:t})$ | filtering law after assimilating $y_t$ |
| $p_\theta(y_t \mid y_{1:t-1})$ | one-step likelihood factor |
| $q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$ | sequential proposal density |
| $\tilde{w}_t^{(i)}, w_t^{(i)}$ | unnormalized and normalized particle weights |
| $a_{t-1}^{(i)}$ | ancestor index selected during resampling |
| $\hat{\pi}_t^N$ | weighted empirical approximation to the filtering law |
| $Z_t^N$ | average incremental bootstrap weight at time $t$ |
| $\hat{p}_\theta^N(y_{1:T})$ | bootstrap particle-filter estimator of the marginal likelihood |

The standing assumptions for the formulas below are the standard ones needed to make the displayed densities and expectations well defined: the transition and observation kernels are dominated by reference measures; the proposal has support wherever the trajectory target has support; the relevant integrals are finite; and resampling schemes are conditionally unbiased in the sense that the expected empirical measure after resampling equals the normalized empirical measure before resampling. These assumptions are not cosmetic. They mark the difference between an exact model identity, a finite-particle Monte Carlo estimator statement, and an implementation diagnostic.

The chapter uses the following claim-status vocabulary. The filtering recursion and likelihood factorization are exact model identities. Weighted particles are finite-$N$ Monte Carlo approximations. The bootstrap likelihood estimator is an unbiased estimator of the likelihood, not of the log likelihood and not of the score. ESS and ancestor-count summaries are diagnostics, not proofs of posterior correctness or failure.

## 20.2 DPF-block notation and claim-status index

This chapter begins the DPF block, so the following index fixes the notation used across the later particle-flow, PF-PF, resampling/OT, learned-OT, HMC, and debugging chapters. A symbol may acquire a method-specific subscript in a later chapter, but its mathematical role should not change silently.

Table 20.1: DPF-block notation index.

| Category | Symbols | Fixed role across the DPF block |
|---|---|---|
| State and observations | $x_t$, $x_{0:t}$, $y_t$, $y_{1:t}$ | Latent state, latent trajectory, current observation, and observation history for the state-space model. |
| Parameters | $\theta$, $u$ | Structural/model parameter and, in HMC chapters, unconstrained sampler coordinate after any parameter transform. |
| Filtering laws | $p_\theta(x_t \mid y_{1:t-1})$, $p_\theta(x_t \mid y_{1:t})$ | Predictive and filtering laws; these are the reference statistical objects. |
| Particles and ancestors | $x_t^{(i)}$, $\tilde{x}_t^{(j)}$, $a_{t-1}^{(i)}$, $a_j$ | Pre- or post-update particles, equal-weight output particles, and discrete ancestor indices. |
| Weights and empirical measures | $\tilde{w}_t^{(i)}$, $w_t^{(i)}$, $\hat{\pi}_t^N$, $\hat{\pi}_{t,\mathrm{eq}}^N$ | Unnormalized/normalized weights and weighted/equal-weight empirical measures. |
| Likelihood estimates | $Z_t^N$, $\hat{p}_\theta^N(y_{1:T})$, $\widehat{L}(y \mid \theta, \omega)$ | Average incremental particle weight, bootstrap likelihood estimator, and generic randomized likelihood estimator used in pseudo-marginal discussion. |
| Proposals | $q_\theta$, $q_{t,0}$, $q_{t,1}$, $\gamma_t$ | Sequential proposal, pre-flow proposal, post-flow proposal, and unnormalized one-step target density. |
| Flow maps and Jacobians | $\pi_{t,\lambda}$, $v_{t,\lambda}$, $\Phi_t$, $J_t$, $D\Phi_t$ | Homotopy density, pseudo-time velocity field, flow map, and forward Jacobian of the flow. |
| Transport couplings | $\Pi$, $\Pi_0^\star$, $\Pi_\varepsilon^\star$, $\Pi_{\varepsilon,K}$ | Unregularized, entropy-regularized, and finite-Sinkhorn transport couplings. |
| Sinkhorn scalings | $C$, $K_\varepsilon$, $a$, $b$, $\varepsilon$, $K$ | Cost matrix, Gibbs kernel, row/column scalings, regularization level, and finite iteration count. |
| Learned maps | $T_\psi$, $Y_\tau$, $R_{\psi,\tau}$, $\mathcal{D}$ | Learned student map, teacher output, student-teacher residual, and training distribution. |
| HMC scalar and gradient | $\ell_\star(u)$, $U_\star(u)$, $g(u)$, $H_\star(u,p)$ | Named log target, potential, gradient used by the integrator, and Hamiltonian. The value and gradient must refer to the same scalar. |

Table 20.2: DPF-block claim-status vocabulary.

| Status phrase | Use only when | Disallowed shortcut |
|---|---|---|
| Exact model identity | The statement follows from the specified state-space model, Bayes' rule, change of variables, or a named exact special case. | Calling a closure, relaxation, solver, or learned map exact because it is written in equations. |
| Unbiased particle estimator | The expectation is taken over the stated particle randomness and the target object is the likelihood or normalizing constant. | Inferring unbiased log likelihood, score, pathwise gradient, or HMC validity. |
| Consistent approximation | A convergence regime and target object are stated. | Treating increasing $N$ or decreasing tolerance as a finite-sample proof. |
| Approximate closure | A Gaussian, local-linear, finite-particle, or numerical closure replaces the reference filtering object. | Delaying the approximation label until a later chapter. |
| Relaxed target | A smooth resampling or transport object replaces the classical categorical operation and the scalar target is the relaxed object. | Implying the original posterior is preserved without a correction. |
| Learned surrogate | A trained map approximates a named teacher under a declared training distribution. | Using runtime, smoothness, or training loss as posterior validity evidence. |
| Engineering hypothesis | The claim is a bounded recommendation or diagnostic rule, with required tests stated. | Presenting implementation preference as mathematical validation. |
| Unsupported claim | The statement lacks a local derivation, reviewed source support, or documented test evidence. | Leaving the statement in the main argument without weakening or removing it. |

## 20.3 Nonlinear state-space model and filtering recursion

Let $x_t \in \mathbb{R}^{n_x}$ denote the latent state and $y_t \in \mathbb{R}^{n_y}$ the observation at time $t$. Fix a parameter vector $\theta \in \Theta \subset \mathbb{R}^{d_\theta}$. The nonlinear state-space model is specified by a transition density and an observation density,

$$x_t \mid x_{t-1} \sim p_\theta(x_t \mid x_{t-1}), \tag{20.1}$$

$$y_t \mid x_t \sim p_\theta(y_t \mid x_t). \tag{20.2}$$

Write $y_{1:t} = (y_1, \dots, y_t)$ for the observation history. The predictive law and the filtering law satisfy the standard Bayesian recursion

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}, \tag{20.3}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}, \tag{20.4}$$

where the one-step predictive density of the observation is

$$p_\theta(y_t \mid y_{1:t-1}) = \int p_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{20.5}$$

The prediction equation is the Chapman–Kolmogorov identity applied to the conditional law of $x_{t-1}$ given the observations already seen. For any measurable set $B$,

$$\mathbb{P}_\theta(x_t \in B \mid y_{1:t-1}) = \int \mathbb{P}_\theta(x_t \in B \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}.$$

When the transition kernel has density $p_\theta(x_t \mid x_{t-1})$, this set identity gives (20.3). The update equation is Bayes' rule with the predictive law as prior and the observation density as likelihood:

$$p_\theta(x_t \mid y_{1:t}) = \frac{p_\theta(y_t \mid x_t, y_{1:t-1}) p_\theta(x_t \mid y_{1:t-1})}{p_\theta(y_t \mid y_{1:t-1})}.$$

The conditional-independence convention of the state-space model reduces $p_\theta(y_t \mid x_t, y_{1:t-1})$ to $p_\theta(y_t \mid x_t)$, giving (20.4). Finally, repeated use of the chain rule gives the marginal likelihood factorization

$$p_\theta(y_{1:T}) = p_\theta(y_1) \prod_{t=2}^{T} p_\theta(y_t \mid y_{1:t-1}),$$

with the same notation covering $t = 1$ by interpreting $y_{1:0}$ as the empty history. The marginal likelihood then factorizes as

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{1:t-1}). \tag{20.6}$$

Equations (20.3)–(20.6) are exact. The difficulty is computational rather than conceptual: outside special classes such as the linear-Gaussian case, the integrals are rarely available in closed form. Particle filters replace these exact distributions by random empirical approximations whose quality improves with particle count under appropriate assumptions.

## 20.4 Empirical filtering measures

A particle filter represents the filtering law by a weighted empirical measure. At time $t$, let $\{(x_t^{(i)}, w_t^{(i)})\}_{i=1}^N$ be particles and normalized weights, with $w_t^{(i)} \geq 0$ and $\sum_{i=1}^N w_t^{(i)} = 1$. The empirical filtering measure is

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^{N} w_t^{(i)} \, \delta_{x_t^{(i)}}(dx), \tag{20.7}$$

where $\delta_z$ is the Dirac point mass at $z$. The approximation of a test function $\varphi$ under the filtering law is then

$$\int \varphi(x) \, \hat{\pi}_t^N(dx) = \sum_{i=1}^{N} w_t^{(i)} \varphi(x_t^{(i)}). \tag{20.8}$$

Thus the filter approximates the entire posterior law by a finite atomic measure, not only its first two moments.

This is the main structural distinction between particle methods and the Gaussian-approximation filters of the previous chapters. The particle filter is not built by closing the recursion in a finite-dimensional moment family. Instead, it propagates a random discrete approximation to the law itself.

## 20.5 Sequential importance sampling

To derive the basic particle recursion, introduce a proposal density $q_\theta(x_t \mid x_{0:t-1}, y_{1:t})$ from which particles can be sampled. Assume that the proposal is positive whenever the trajectory target below is positive. Without this support condition the importance ratio is not a valid correction; particles may miss regions that carry target probability. Suppose a full trajectory proposal factors as

$$q_\theta(x_{0:t} \mid y_{1:t}) = q_\theta(x_0 \mid y_1) \prod_{s=1}^{t} q_\theta(x_s \mid x_{0:s-1}, y_{1:s}). \tag{20.9}$$

The target filtering distribution over trajectories is proportional to

$$p_\theta(x_{0:t}, y_{1:t}) = p_\theta(x_0) \prod_{s=1}^{t} p_\theta(x_s \mid x_{s-1}) p_\theta(y_s \mid x_s). \tag{20.10}$$

The unnormalized importance weight is therefore

$$\tilde{w}_t(x_{0:t}) = \frac{p_\theta(x_{0:t}, y_{1:t})}{q_\theta(x_{0:t} \mid y_{1:t})}. \tag{20.11}$$

This ratio is a trajectory-level object. It is not yet a resampling rule and not yet a differentiable likelihood map. It is the Radon–Nikodym correction that converts expectations under the proposal path law into expectations under the unnormalized trajectory target, provided the support and integrability conditions above hold. Using the factorization in (20.9)–(20.10), one obtains the recursive update

$$\tilde{w}_t^{(i)} = \tilde{w}_{t-1}^{(i)} \frac{p_\theta(y_t \mid x_t^{(i)}) p_\theta(x_t^{(i)} \mid x_{t-1}^{(i)})}{q_\theta(x_t^{(i)} \mid x_{0:t-1}^{(i)}, y_{1:t})}. \tag{20.12}$$

Normalizing gives

$$w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{j=1}^{N} \tilde{w}_t^{(j)}}. \tag{20.13}$$

Equation (20.12) is the generic sequential importance sampling (SIS) update. It already shows the two main mathematical stresses of particle filtering:

(i) the proposal must be chosen carefully to avoid explosive weight variability;

(ii) even when the estimator is well defined, the normalized weights tend to concentrate over time, which leads to degeneracy.

## 20.6 Sequential importance resampling and the bootstrap filter

Sequential importance sampling alone is typically unstable over long horizons, because the weights become increasingly uneven. Sequential importance resampling (SIR) introduces a resampling step to refresh the cloud when the weight distribution becomes too concentrated.

The most classical special case is the bootstrap particle filter, which uses the transition prior itself as the proposal:

$$q_\theta(x_t \mid x_{0:t-1}, y_{1:t}) = p_\theta(x_t \mid x_{t-1}). \tag{20.14}$$

Substituting (20.14) into (20.12) gives the simplified incremental weight

$$\tilde{w}_t^{(i)} \propto w_{t-1}^{(i)} p_\theta(y_t \mid x_t^{(i)}). \tag{20.15}$$

The bootstrap recursion at time $t$ can therefore be written as:

**Step 1: Select ancestors:** if resampling is performed before propagation, draw $a_{t-1}^{(i)}$ from the current normalized weights; otherwise keep $a_{t-1}^{(i)} = i$.

**Step 2: Propagate:** sample $x_t^{(i)} \sim p_\theta(x_t \mid x_{t-1}^{(a_{t-1}^{(i)})})$.

**Step 3: Weight:** let $w_{t-1}^{\star(i)} = 1/N$ if resampling was just performed and $w_{t-1}^{\star(i)} = w_{t-1}^{(i)}$ otherwise. Compute $\tilde{w}_t^{(i)} = N\, w_{t-1}^{\star(i)}\, p_\theta(y_t \mid x_t^{(i)})$ and normalize the weights. On the resample-every-step branch this reduces to the familiar $\tilde{w}_t^{(i)} = p_\theta(y_t \mid x_t^{(i)})$.

**Step 4: Resample:** after recording the likelihood factor, either resample using the normalized weights and reset weights to $1/N$, or carry the weighted cloud forward, together with its normalized weights, until the next resampling decision.

This yields the standard bootstrap/SIR particle filter of Gordon et al. [1993], elaborated in the SMC text of Doucet et al. [2001]. The particle-MCMC literature [Andrieu and Roberts, 2009, Andrieu et al., 2010] later uses the likelihood estimator generated by this construction, but that pseudo-marginal role is a statement about an unbiased value-side estimator inside a Metropolis–Hastings construction. It is not a statement that the realized resampling path is smooth in $\theta$.

## 20.7 The bootstrap particle-filter likelihood estimator

The bootstrap particle filter supplies a natural estimator of the factors in the marginal-likelihood decomposition (20.6). Define the average incremental weight at time $t$ by

$$Z_t^N = \frac{1}{N} \sum_{i=1}^{N} \tilde{w}_t^{(i)}. \tag{20.16}$$

The corresponding likelihood estimator is

$$\hat{p}_\theta^N(y_{1:T}) = \prod_{t=1}^{T} Z_t^N. \tag{20.17}$$

The estimator-status result BayesFilter needs at this stage is the following.

**Proposition 20.1** (Bootstrap particle-filter likelihood-estimator status)**.** *Fix the observations $y_{1:T}$ and the parameter $\theta$. Assume the model densities are dominated and integrable, the bootstrap propagation step samples from $p_\theta(x_t \mid x_{t-1})$, and the Feynman–Kac quantities used below are finite: for every bounded measurable $\varphi$ used in the induction, the operator $Q_t\varphi$ is well defined, the incremental weights have finite conditional expectation, and the displayed conditional expectations are finite. Assume also that every resampling step used by the algorithm is conditionally unbiased for the current normalized empirical measure. Here conditional unbiasedness means that, for every bounded measurable test function $\psi$ in the same finite-expectation class, the expected post-resampling empirical average equals the pre-resampling weighted empirical average, conditional on the current weighted cloud. Biased or deterministic approximation schemes that do not satisfy this identity are not covered by the proposition. Then the estimator (20.17) is an unbiased estimator of the marginal likelihood:*

$$\mathbb{E}\big[\hat{p}_\theta^N(y_{1:T})\big] = p_\theta(y_{1:T}).$$

*This claim is about the likelihood itself. It does not claim that $\log \hat{p}_\theta^N(y_{1:T})$ is an unbiased estimator of $\log p_\theta(y_{1:T})$, and it does not claim that differentiating a realized particle system gives an unbiased score. Indeed, by Jensen's inequality, $\mathbb{E}[\log \hat{p}_\theta^N(y_{1:T})] \le \log p_\theta(y_{1:T})$, with strict inequality unless the estimator is almost surely constant, so the log of the estimator is biased downward.*

*Proof.* For compactness, define the bootstrap Feynman–Kac operator

$$(Q_t\varphi)(x_{t-1}) = \int \varphi(x_t) p_\theta(y_t \mid x_t) p_\theta(x_t \mid x_{t-1})\, dx_t$$

for bounded measurable test functions $\varphi$. Also define the unnormalized filtering functional

$$\gamma_t(\varphi) = p_\theta(y_{1:t}) \int \varphi(x_t) p_\theta(x_t \mid y_{1:t})\, dx_t.$$

The prediction/update recursion implies $\gamma_t(\varphi) = \gamma_{t-1}(Q_t\varphi)$ and $\gamma_t(1) = p_\theta(y_{1:t})$.

The filtration convention is as follows. At the end of time $t-1$ the algorithm has a weighted empirical measure $\hat{\pi}_{t-1}^N$ and a normalizing-constant estimate $\prod_{s=1}^{t-1} Z_s^N$. Any resampling randomness used to choose the particles propagated at time $t$ is then drawn, and the transition propagation at time $t$ is conditionally independent across particles given that post-resampling cloud. All test functions in the induction are bounded measurable, or otherwise integrable enough that the conditional expectations below are finite.

Let $\mathcal{F}_{t-1}^N$ denote the sigma-field generated by the particle system just before propagation at time $t$, after any resampling decision at time $t-1$. Let $\bar{\pi}_{t-1}^N$ denote the *post-decision measure* from which the transition propagation is drawn. If resampling is used, $\bar{\pi}_{t-1}^N$ is the unweighted empirical measure of the resampled cloud and conditional unbiasedness gives

$$\mathbb{E}[\bar{\pi}_{t-1}^N(\psi) \mid \hat{\pi}_{t-1}^N] = \hat{\pi}_{t-1}^N(\psi)$$

for bounded $\psi$. If resampling is not used, the post-decision measure is instead the weighted carried measure

$$\bar{\pi}_{t-1}^N(\psi) = \sum_{i=1}^N w_{t-1}^{\star(i)} \psi(x_{t-1}^{(i)}),$$

so the same identity holds trivially because $\bar{\pi}_{t-1}^N = \hat{\pi}_{t-1}^N$ as weighted measures on that branch.

After bootstrap propagation and weighting with the carried post-decision weights,

$$Z_t^N \hat{\pi}_t^N(\varphi) = \frac{1}{N} \sum_{i=1}^{N} N\, w_{t-1}^{\star(i)}\, p_\theta(y_t \mid x_t^{(i)}) \varphi(x_t^{(i)}).$$

Conditional on $\mathcal{F}_{t-1}^N$, the propagated particles are drawn from the transition kernel, so

$$\mathbb{E}\big[Z_t^N \hat{\pi}_t^N(\varphi) \mid \mathcal{F}_{t-1}^N\big] = \bar{\pi}_{t-1}^N(Q_t\varphi).$$

Taking the conditional expectation over any resampling randomness replaces $\bar{\pi}_{t-1}^N$ by $\hat{\pi}_{t-1}^N$ in expectation. Multiplying by the previous normalizing-constant estimate, which is measurable with respect to the previous particle history, gives

$$\mathbb{E}\left[\left(\prod_{s=1}^{t} Z_s^N\right) \hat{\pi}_t^N(\varphi)\right] = \mathbb{E}\left[\left(\prod_{s=1}^{t-1} Z_s^N\right) \hat{\pi}_{t-1}^N(Q_t\varphi)\right].$$

Iterating this identity and using $\gamma_t(\varphi) = \gamma_{t-1}(Q_t\varphi)$ gives

$$\mathbb{E}\left[\left(\prod_{s=1}^{t} Z_s^N\right) \hat{\pi}_t^N(\varphi)\right] = \gamma_t(\varphi).$$

Taking $\varphi \equiv 1$ yields

$$\mathbb{E}\left[\prod_{s=1}^{t} Z_s^N\right] = \gamma_t(1) = p_\theta(y_{1:t}).$$

Setting $t = T$ proves the displayed likelihood-unbiasedness identity. This is the local BayesFilter version of the standard SMC normalizing-constant unbiasedness argument; see Doucet et al. [2001] and Andrieu et al. [2010] for the full particle-system treatment. □

This proposition is mathematically central for the later DPF program. It tells us that the classical bootstrap particle filter supplies an unbiased estimator of the marginal likelihood, even though it does not supply a pathwise differentiable likelihood map. Later differentiable constructions must therefore be compared against this baseline carefully: a differentiable surrogate may be smoother to optimize or differentiate, but it need not preserve this exact estimator status.

## 20.8 Differentiability boundary

The unbiasedness result above is a value-side Monte Carlo statement. It should not be read as a pathwise differentiability statement. For a fixed random seed, the propagation step may be reparameterizable in some models, but the resampling step is a categorical map from weights to ancestor indices. Small changes in $\theta$ can leave the selected ancestor unchanged over an interval and then switch it discontinuously when a cumulative weight boundary is crossed. The realized resampled cloud is therefore not a smooth function of $\theta$.

This distinction creates three separate objects that later chapters must not blur:

(i) the true likelihood $p_\theta(y_{1:T})$;

(ii) an unbiased randomized estimator $\hat{p}_\theta^N(y_{1:T})$ of that likelihood;

(iii) a pathwise differentiable scalar produced by a relaxed, transported, or learned particle computation.

The first object is the statistical target, the second can support pseudo-marginal value-side constructions under their own assumptions, and the third may be useful for optimization or HMC only after its target status is stated. In finite precision the estimator should be accumulated in log domain, using a per-time log-sum-exp over incremental log weights, because the raw product form underflows quickly for realistic horizons. A gradient through the third object is not automatically a gradient of the first or of the expectation of the second.

## 20.9 Degeneracy and effective sample size

The main numerical weakness of the particle filter is weight degeneracy. As the observation sequence grows, the normalized weights frequently concentrate on a small subset of particles. A standard summary is the effective sample size (ESS),

$$\mathrm{ESS}_t = \frac{1}{\sum_{i=1}^N (w_t^{(i)})^2}. \tag{20.18}$$

The extreme cases are informative:

- if all weights are equal, then $\mathrm{ESS}_t = N$;
- if one particle carries almost all the mass, then $\mathrm{ESS}_t \approx 1$.

When ESS collapses, the atomic approximation (20.7) still exists, but its practical variance may become extremely large. Resampling alleviates this by discarding low-weight particles and duplicating high-weight particles, but resampling does not remove the deeper dimensionality problem. The classical literature shows that in many high-dimensional settings the number of particles needed to stabilize the weight distribution can grow very rapidly with the effective dimension of the observation problem [Bengtsson et al., 2008, Snyder et al., 2008].

This is the exact point where the later chapters enter. Particle-flow methods, PF-PF corrections, differentiable resampling, OT relaxations, and learned OT operators are all responses to some combination of:

(i) degeneracy under standard resampling;

(ii) lack of differentiability of the resampling step;

(iii) computational burden of high-quality resampling or transport maps;

(iv) mismatch between a useful learning objective and a valid HMC target.

A mathematically clean DPF monograph must therefore begin here: by making the classical particle-filter object precise before modifying it.

## 20.10 Baseline audit table

Table 20.3: Classical particle-filter baseline audit.

| Object or claim | Status | Assumptions | Implementation diagnostic |
|---|---|---|---|
| Filtering recursion and likelihood factorization | Exact model identity | Dominated transition and observation kernels; fixed $\theta$; finite normalizers. | Compare predictive and filtering factors against a Kalman reference in a linear-Gaussian model. |
| Trajectory importance ratio | Exact algebraic identity for the specified target/proposal pair | Proposal support covers target support; finite weights. | Check target and proposal log densities separately on a small trajectory. |
| Weighted empirical measure | Finite-$N$ Monte Carlo approximation | Particle system generated by the declared proposal and weighting rule. | Compare bounded test-function expectations across increasing $N$. |
| Bootstrap likelihood estimator | Unbiased estimator of likelihood, not log likelihood or score | Bootstrap proposal; conditionally unbiased resampling; integrable likelihood factors. | In a controlled model, average repeated estimates and compare to the analytic likelihood. |
| ESS and ancestor diagnostics | Engineering diagnostic, not posterior-validity proof | Weights normalized and finite; diagnostic interpreted with model dimension and observation informativeness. | Track ESS, log-weight variance, and unique ancestor count by time and parameter region. |
| Pathwise gradient through resampling | Not supplied by the classical bootstrap filter | Categorical ancestor map is used without relaxation. | Finite-difference a fixed-seed resampled path and identify discontinuities or zero-almost-everywhere regions. |

## 20.11 What this chapter does and does not claim

This chapter establishes the exact nonlinear filtering recursion and the classical bootstrap particle-filter baseline. It does *not* yet claim any of the following:

- a pathwise differentiable likelihood map;
- a particle-flow approximation;
- a proposal-corrected flow filter;
- a differentiable resampling rule;
- an HMC-ready particle backend.

Those belong to the later chapters. The role of the present chapter is more basic and more important: it fixes the reference object against which every later DPF construction must be interpreted.

# Chapter 21

# Particle-Flow Foundations

This chapter develops the mathematical foundations of particle-flow methods. Where the previous chapter fixed the classical particle-filter baseline, the present chapter studies a different idea: instead of relying on repeated reweighting and resampling to move from the predictive law to the filtering law, one may attempt to transport particles continuously along a pseudo-time path.

This transport point of view is attractive because it addresses a real weakness of classical particle filters: in sharply informative or moderately high-dimensional problems, repeated weight concentration can make the empirical measure numerically poor long before the state-space model itself ceases to be well defined. Particle-flow methods attempt to reduce that instability by constructing a velocity field that pushes the predictive cloud toward the posterior cloud. The mathematical difficulty is that the transport law is not free: one must define the interpolating densities, the continuity equation, the velocity field, and the assumptions under which the resulting flow is exact or only approximate.

This chapter therefore treats particle flow first as a transport problem, not as a final likelihood engine. In particular, the present chapter does *not* yet claim that a raw particle flow defines the final HMC target. It first develops the homotopy and flow mathematics, identifies the Gaussian- closure approximation behind the classical EDH construction, and states the exact special case in which the transport recovers the Kalman update. The subsequent PF-PF chapter then asks how to restore a clearer proposal/target relationship by importance correction.

## 21.1 Objects, notation, and source roles

The particle-flow literature is easy to misread because the same word "flow" can refer to several distinct objects: a density path, a velocity field, a particle ODE, a numerical map produced by integrating that ODE, or a proposal mechanism later corrected by an importance weight. This chapter uses the following objects.

| Symbol or phrase | Meaning in this chapter |
|---|---|
| $\pi_{t\|t-1}$ | predictive law of $x_t$ given $y_{1:t-1}$ before assimilating $y_t$ |
| $\pi_{t\|t}$ | filtering law of $x_t$ given $y_{1:t}$ after assimilating $y_t$ |
| $\pi_{t,\lambda}$ | normalized homotopy density at pseudo-time $\lambda \in [0,1]$ |
| $Z_t(\lambda)$ | normalizing constant of the homotopy path |
| $v_{t,\lambda}$ | velocity field whose flow is intended to transport $\pi_{t,\lambda}$ |
| $\Phi_{t,\lambda}$ | flow map generated by the particle ODE from pseudo-time 0 to $\lambda$ |
| $A_t(\lambda), b_t(\lambda)$ | affine EDH velocity coefficients under a Gaussian closure |
| $P_{t,\lambda}, m_{t,\lambda}$ | covariance and mean of the Gaussian homotopy closure |
| $G_t^{(i)}$ | particle-local observation Jacobian in LEDH |
| $\Lambda_t, \Lambda_t^{(i)}$ | global and particle-local likelihood precision contributions |
| $\eta_t^{(i)}$ | particle-local Gaussian information vector in LEDH |

The source role is also deliberately limited. Daum–Huang particle flow [Daum and Huang, 2008] motivates the EDH construction. Invertible particle-flow particle filters [Li and Coates, 2017] explain why later chapters must treat a flow as a proposal map with a density correction rather than as an automatically correct posterior sampler. Particle-flow applications such as Hu and van Leeuwen [2021] motivate numerical and high-dimensional caution. These sources locate the method family; they do not remove the need for the local derivations below, and they do not establish a final HMC target for BayesFilter.

## 21.2 From discrete reweighting to continuous transport

Let the predictive law at time $t$ be

$$\pi_{t|t-1}(dx) = p_\theta(x_t \in dx \mid y_{1:t-1}),$$

and let the filtering law be

$$\pi_{t|t}(dx) = p_\theta(x_t \in dx \mid y_{1:t}).$$

The bootstrap particle filter approximates the transition from $\pi_{t|t-1}$ to $\pi_{t|t}$ by assigning observation-driven weights and then resampling the cloud. Particle-flow methods replace this discrete update by a continuous pseudo-time transport from a prior-like distribution to a posterior-like distribution. The intent is to move particles toward regions of high posterior density before resampling is needed, thereby reducing weight collapse.

The conceptual shift is therefore from

(i) a *weighted empirical update* followed by discrete resampling,

to

(ii) a *continuous transport path* whose endpoint approximates the filtering law.

The advantage of the second view is geometric: the cloud is moved by a velocity field rather than by repeated duplication and deletion of particles. The cost is mathematical and numerical: the transport field must be derived, integrated, and interpreted correctly.

## 21.3 Homotopy path between predictive and filtering laws

Particle-flow methods introduce a pseudo-time parameter $\lambda \in [0,1]$ and construct an interpolating family of densities $\pi_{t,\lambda}$. Let

$$\bar{\pi}_{t,\lambda}(x) = p_\theta(x_t = x \mid y_{1:t-1}) p_\theta(y_t \mid x)^\lambda$$

be the unnormalized path. The normalized path is

$$\pi_{t,\lambda}(x) = \frac{\bar{\pi}_{t,\lambda}(x)}{Z_t(\lambda)}.$$

Equivalently, in the classical Daum–Huang construction one defines the log-homotopy

$$\log \pi_{t,\lambda}(x) = \log p_\theta(x_t = x \mid y_{1:t-1}) + \lambda \log p_\theta(y_t \mid x) - \log Z_t(\lambda), \tag{21.1}$$

where

$$Z_t(\lambda) = \int p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \, dx_t \tag{21.2}$$

is the normalizing constant.

This definition deserves more emphasis than a short formula often receives. First, the endpoint interpretation is exact:

$$\pi_{t,0}(x) = p_\theta(x_t = x \mid y_{1:t-1}), \qquad \pi_{t,1}(x) = p_\theta(x_t = x \mid y_{1:t}).$$

Thus the family joins the predictive law to the filtering law without changing what the endpoints mean. Second, the normalization is not merely a technical constant. It is the term that keeps the intermediate object a probability density rather than an unnormalized score. Third, the homotopy does not yet choose a transport field; it only chooses the density path the field is supposed to realize.

Differentiating (21.1) with respect to $\lambda$ gives

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t \mid x) - \partial_\lambda \log Z_t(\lambda). \tag{21.3}$$

The term involving $Z_t(\lambda)$ is crucial: it keeps the intermediate density normalized. Omitting it may produce a convenient algebraic simplification, but it changes the law being transported. This is one of the first places where a future implementation can quietly go wrong: if the code is written only in terms of an unnormalized path, the resulting transported object may no longer match the intended predictive-to-filtering homotopy.

One can make the normalizer term explicit. Differentiating (21.2) gives

$$\frac{dZ_t(\lambda)}{d\lambda} = \int p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \log p_\theta(y_t \mid x_t) \, dx_t.$$

This differentiation under the integral sign is not automatic. A sufficient local condition is that, on the pseudo-time interval being used, the likelihood is positive where the predictive density has support and there is an integrable envelope dominating

$$p_\theta(x_t \mid y_{1:t-1}) p_\theta(y_t \mid x_t)^\lambda \left|\log p_\theta(y_t \mid x_t)\right|.$$

Equivalently, the predictive law must give finite expectation to the log-likelihood factor along the homotopy and permit dominated differentiation of $Z_t(\lambda)$. If this condition fails, the centered identity below should be treated as a formal diagnostic rather than as a justified derivative calculation. Therefore

$$\partial_\lambda \log Z_t(\lambda) = \int \log p_\theta(y_t \mid x)\, \pi_{t,\lambda}(x)\, dx.$$

The derivative in (21.3) is thus the centered log-likelihood under the intermediate law:

$$\partial_\lambda \log \pi_{t,\lambda}(x) = \log p_\theta(y_t \mid x) - \mathbb{E}_{\pi_{t,\lambda}}[\log p_\theta(y_t \mid X_t)].$$

This centered form is a useful audit identity. It shows that the density path preserves total probability, since integrating $\partial_\lambda \pi_{t,\lambda}$ over $x$ gives zero.

For debugging purposes, the main lesson of this section is simple: if a claimed particle-flow implementation does not recover the correct endpoint target even in a simple controlled case, one of the first questions to ask is whether the normalized homotopy was implemented or whether an unnormalized shortcut was silently substituted.

## 21.4 Continuity equation and transport PDE

Suppose the particle cloud is transported by a velocity field $v_{t,\lambda}(x)$ through the pseudo-time ODE

$$\frac{dx(\lambda)}{d\lambda} = v_{t,\lambda}(x(\lambda)). \tag{21.4}$$

The following derivation is a classical strong-form calculation. It assumes that the density is differentiable in $\lambda$, continuously differentiable in $x$ on the region of interest, that $\pi_{t,\lambda} v_{t,\lambda}$ is integrable, and that the boundary term in the integration-by-parts step vanishes. This is satisfied, for example, for smooth rapidly decaying densities on $\mathbb{R}^{n_x}$ with controlled velocity, for compactly supported smooth test functions in the weak form, or for bounded domains with an explicitly imposed no-flux or otherwise stated boundary condition. Without one of these admissible settings, the PDE should be read as a formal transport equation rather than a fully justified global statement. For reviewer-grade use, the weak-form identity with compactly supported test functions is the safer default; the displayed strong-form PDE is used only when the stated differentiability, integrability, and boundary-flux hypotheses hold. Mass conservation over any regular test region then implies that the evolving density must satisfy the continuity equation

$$\partial_\lambda \pi_{t,\lambda}(x) + \nabla \cdot \big(\pi_{t,\lambda}(x) v_{t,\lambda}(x)\big) = 0. \tag{21.5}$$

Equivalently, for any smooth compactly supported test function $\varphi$,

$$\frac{d}{d\lambda} \int \varphi(x) \pi_{t,\lambda}(x)\, dx = \int \nabla \varphi(x)^\top v_{t,\lambda}(x) \pi_{t,\lambda}(x)\, dx,$$

and integration by parts gives (21.5). Dividing by $\pi_{t,\lambda}(x)$ where the density is positive yields

$$\partial_\lambda \log \pi_{t,\lambda}(x) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \tag{21.6}$$

Combining (21.3) and (21.6) gives the basic particle-flow PDE:

$$\log p_\theta(y_t \mid x) - \partial_\lambda \log Z_t(\lambda) = -\nabla \cdot v_{t,\lambda}(x) - \nabla \log \pi_{t,\lambda}(x)^\top v_{t,\lambda}(x). \tag{21.7}$$

This equation is exact as a conservation statement. What is not yet fixed is a unique velocity field. The PDE constrains the field but does not determine it without additional structure. In particular, if a vector field has zero weighted divergence with respect to $\pi_{t,\lambda}$, adding it to one solution can leave the same density evolution. EDH and LEDH should therefore be read as particular closure choices, not as the only possible solution of the transport PDE.

## 21.5 EDH under Gaussian closure

The classical exact Daum–Huang (EDH) construction becomes explicit after a Gaussian closure. The word "exact" in the name is historical and must be read with the regime stated here: the derivation is exact for the Gaussian family it postulates, and it is an exact filtering transport only when that Gaussian family is the true homotopy family.

Assume that the predictive law is approximated by

$$p_\theta(x_t = x \mid y_{1:t-1}) \approx \mathcal{N}(x; m_{t|t-1}, P_{t|t-1})$$

and that the observation density is Gaussian with linear observation map $H_t$ and covariance $R_t$:

$$p_\theta(y_t \mid x) \propto \exp\left\{-\frac{1}{2}(y_t - H_t x)^\top R_t^{-1}(y_t - H_t x)\right\}.$$

Here $P_{t|t-1}$ and $R_t$ are assumed symmetric positive definite, and the matrix products have the conformable dimensions $H_t \in \mathbb{R}^{n_y \times n_x}$, $P_{t|t-1} \in \mathbb{R}^{n_x \times n_x}$, and $R_t \in \mathbb{R}^{n_y \times n_y}$. These assumptions are what make the inverse and solve notation below meaningful; an implementation should use linear solves and record conditioning diagnostics rather than treating the displayed inverses as a numerical instruction. At pseudo-time $\lambda$ the interpolating density is then approximated by

$$\pi_{t,\lambda}(x) \approx \mathcal{N}(x; m_{t,\lambda}, P_{t,\lambda}).$$

The likelihood precision is

$$\Lambda_t = H_t^\top R_t^{-1} H_t. \tag{21.8}$$

Collecting the quadratic terms in the log-homotopy gives

$$P_{t,\lambda}^{-1} = P_{t|t-1}^{-1} + \lambda \Lambda_t,$$

so the homotopy covariance satisfies

$$P_{t,\lambda} = \left(P_{t|t-1}^{-1} + \lambda \Lambda_t\right)^{-1}, \tag{21.9}$$

and the matrix inverse derivative gives the covariance evolution. To see the step explicitly, set $M_{t,\lambda} = P_{t|t-1}^{-1} + \lambda \Lambda_t$, so that $M_{t,\lambda} P_{t,\lambda} = I$. Differentiating this identity yields $\Lambda_t P_{t,\lambda} + M_{t,\lambda}(dP_{t,\lambda}/d\lambda) = 0$ and hence

$$\frac{dP_{t,\lambda}}{d\lambda} = -P_{t,\lambda} \Lambda_t P_{t,\lambda}. \tag{21.10}$$

The corresponding information vector is

$$h_{t,\lambda} = P_{t|t-1}^{-1} m_{t|t-1} + \lambda H_t^\top R_t^{-1} y_t, \qquad m_{t,\lambda} = P_{t,\lambda} h_{t,\lambda}.$$

Differentiating $m_{t,\lambda}$ gives

$$\frac{dm_{t,\lambda}}{d\lambda} = -P_{t,\lambda}\Lambda_t m_{t,\lambda} + P_{t,\lambda}H_t^\top R_t^{-1} y_t.$$

The EDH ansatz now chooses an affine velocity field

$$v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda), \tag{21.11}$$

whose mean and covariance evolution match the Gaussian homotopy. If $X_\lambda \sim \mathcal{N}(m_{t,\lambda}, P_{t,\lambda})$ evolves under this affine field, then

$$\frac{dm_{t,\lambda}}{d\lambda} = A_t(\lambda)m_{t,\lambda} + b_t(\lambda), \qquad \frac{dP_{t,\lambda}}{d\lambda} = A_t(\lambda)P_{t,\lambda} + P_{t,\lambda}A_t(\lambda)^\top.$$

Choosing $A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t$ matches (21.10). The offset $b_t(\lambda)$ is then determined by the mean equation:

$$b_t(\lambda) = \frac{dm_{t,\lambda}}{d\lambda} - A_t(\lambda)m_{t,\lambda}.$$

Using

$$m_{t,\lambda} = m_{t|t-1} + \lambda P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1})$$

rewrites this offset in the standard BayesFilter EDH convention. The coefficients are

$$A_t(\lambda) = -\frac{1}{2}P_{t,\lambda}\Lambda_t, \tag{21.12}$$

$$b_t(\lambda) = -A_t(\lambda)m_{t|t-1} + \big(I + \lambda A_t(\lambda)\big)P_{t,\lambda}H_t^\top R_t^{-1}(y_t - H_t m_{t|t-1}). \tag{21.13}$$

Thus the EDH pseudo-time ODE is

$$\frac{dx}{d\lambda} = A_t(\lambda)x + b_t(\lambda). \tag{21.14}$$

The Gaussian closure should be read with care. The continuity equation and the homotopy construction are exact at their own level. The approximation enters when one assumes that the intermediate family remains Gaussian and then chooses an affine velocity field that is consistent with that Gaussian family. In other words, EDH is exact as a transport formula only when the closure assumption is exactly true; otherwise it is a mathematically structured approximation.

A future implementation should therefore separate four tasks explicitly:

(i) evaluate the predictive Gaussian inputs $(m_{t|t-1}, P_{t|t-1})$;

(ii) build the precision contribution $\Lambda_t$;

(iii) build the information contribution $H_t^\top R_t^{-1} y_t$ or its locally shifted analogue;

(iv) integrate the affine pseudo-time dynamics (21.14) using a numerically stable scheme.

If the transported cloud behaves unexpectedly, the first debugging question is not merely whether the ODE solver was stable, but also whether the Gaussian closure itself was plausible for the state-space regime under study.

## 21.6 Linear-Gaussian recovery as the exact special case

The most important exactness statement for EDH is the linear-Gaussian recovery result. Suppose the predictive law is Gaussian and the observation model is truly linear-Gaussian. Then the homotopy family remains Gaussian for all $\lambda$. The covariance endpoint at $\lambda = 1$ is

$$P_{t,1} = \left(P_{t|t-1}^{-1} + H_t^\top R_t^{-1} H_t\right)^{-1}, \tag{21.15}$$

which is exactly the Kalman posterior covariance in information form. The mean endpoint follows from the endpoint information vector:

$$m_{t,1} = P_{t,1}\left(P_{t|t-1}^{-1} m_{t|t-1} + H_t^\top R_t^{-1} y_t\right). \tag{21.16}$$

Using the Kalman gain

$$K_t = P_{t|t-1} H_t^\top (H_t P_{t|t-1} H_t^\top + R_t)^{-1},$$

the same mean can be written as

$$m_{t,1} = m_{t|t-1} + K_t(y_t - H_t m_{t|t-1}).$$

Thus, in the linear-Gaussian case, EDH is not merely an approximation: it transports the predictive Gaussian law to the exact Kalman update, provided the affine ODE is integrated exactly or to controlled numerical tolerance.

This statement is crucial for BayesFilter because it supplies the exact special case against which any EDH implementation should be tested. It is also the boundary line for the rest of the chapter: outside this special case, the particle-flow path is no longer guaranteed exact merely by writing down the homotopy.

## 21.7 LEDH and local linearization

The local EDH (LEDH) construction replaces the single global observation linearization by a particle-specific local linearization. This is not merely an implementation refinement. It changes the closure from one Gaussian homotopy shared by the cloud to a family of particle-local Gaussian approximations.

For particle $x^{(i)}$, let

$$G_t^{(i)} = \left.\frac{\partial g_\theta(x)}{\partial x}\right|_{x=x^{(i)}} \tag{21.17}$$

and write the local linear approximation of the observation map as

$$g_\theta(x) \approx g_\theta(x^{(i)}) + G_t^{(i)}(x - x^{(i)}). \tag{21.18}$$

Equivalently, the local observation equation is approximated by

$$y_t \approx G_t^{(i)} x + \left(g_\theta(x^{(i)}) - G_t^{(i)} x^{(i)}\right) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_t).$$

Collecting quadratic and linear terms in this local Gaussian likelihood gives the local precision

$$\Lambda_t^{(i)} = (G_t^{(i)})^\top R_t^{-1} G_t^{(i)} \tag{21.19}$$

and the local Gaussian information vector

$$\eta_t^{(i)} = (G_t^{(i)})^\top R_t^{-1}\big(y_t - g_\theta(x^{(i)}) + G_t^{(i)} x^{(i)}\big), \tag{21.20}$$

which is the particle-wise analogue of the global Gaussian information term $H_t^\top R_t^{-1} y_t$ in EDH. The corresponding local homotopy covariance is

$$P_{t,\lambda}^{(i)} = \big(P_{t|t-1}^{-1} + \lambda \Lambda_t^{(i)}\big)^{-1}. \tag{21.21}$$

The associated local mean is

$$m_{t,\lambda}^{(i)} = P_{t,\lambda}^{(i)} \left(P_{t|t-1}^{-1} m_{t|t-1} + \lambda \eta_t^{(i)}\right).$$

The local linearization of the observation map near particle $x^{(i)}$ therefore produces particle-specific flow coefficients

$$A_t^{(i)}(\lambda) = -\frac{1}{2} P_{t,\lambda}^{(i)} \Lambda_t^{(i)}, \tag{21.22}$$

$$b_t^{(i)}(\lambda) = -A_t^{(i)}(\lambda) m_{t|t-1} + \big(I + \lambda A_t^{(i)}(\lambda)\big) P_{t,\lambda}^{(i)} \big(\eta_t^{(i)} - \Lambda_t^{(i)} m_{t|t-1}\big), \tag{21.23}$$

with pseudo-time evolution. The centered information term $\eta_t^{(i)} - \Lambda_t^{(i)} m_{t|t-1}$ is required for consistency with the local mean $m_{t,\lambda}^{(i)}$ defined above: differentiating that mean in $\lambda$ and matching $b_t^{(i)} = dm_{t,\lambda}^{(i)}/d\lambda - A_t^{(i)} m_{t,\lambda}^{(i)}$ produces exactly this centered form, and in the globally linear case it reduces to the EDH offset displayed earlier. The Li–Coates source-form coefficients quoted below are the same flow written directly in source notation.

$$\frac{dx^{(i)}}{d\lambda} = A_t^{(i)}(\lambda) x^{(i)} + b_t^{(i)}(\lambda). \tag{21.24}$$

For the Li–Coates PF-PF(LEDH) construction used in the next chapter, there is an additional implementation-facing distinction that must not be collapsed. The local linearization is not a single deterministic affine map shared by all particles, and it is not computed from a cloud-level average. For particle $i$, Algorithm 1 of Li and Coates [2017] first builds a particle-specific predicted covariance $P^i$ and a zero-noise transition anchor

$$\bar{\eta}_0^i = g_k(x_{k-1}^i, 0).$$

It also samples an actual pre-flow proposal particle

$$\eta_0^i = g_k(x_{k-1}^i, v_k).$$

During pseudo-time integration, the Jacobian used in the LEDH coefficients is evaluated at the moving auxiliary state $\bar{\eta}_\lambda^i$, not at a fixed global mean and not at another particle. In the notation of Li and Coates [2017], the local observation linearization is

$$H^i(\lambda) = \left.\frac{\partial h(\eta, 0)}{\partial \eta}\right|_{\eta = \bar{\eta}_\lambda^i}, \qquad e^i(\lambda) = h(\bar{\eta}_\lambda^i, 0) - H^i(\lambda)\bar{\eta}_\lambda^i. \tag{21.25}$$

With the particle-specific prediction covariance $P^i$ held as the covariance input for that particle's flow step, the source-form LEDH coefficients are

$$A^i(\lambda) = -\frac{1}{2} P^i H^i(\lambda)^\top \big(\lambda H^i(\lambda) P^i H^i(\lambda)^\top + R\big)^{-1} H^i(\lambda), \tag{21.26}$$

$$b^i(\lambda) = \big(I + 2\lambda A^i(\lambda)\big)\left[\big(I + \lambda A^i(\lambda)\big) P^i H^i(\lambda)^\top R^{-1}\{z_k - e^i(\lambda)\} + A^i(\lambda)\bar{\eta}_0^i\right]. \tag{21.27}$$

The auxiliary anchor and the actual proposal particle are then migrated by the same particle-local affine update at each pseudo-time step:

$$\bar{\eta}^i_{\lambda_j} = \bar{\eta}^i_{\lambda_{j-1}} + \epsilon_j \left[ A^i_j(\lambda_j)\bar{\eta}^i_{\lambda_{j-1}} + b^i_j(\lambda_j) \right], \tag{21.28}$$

$$\eta^i_{\lambda_j} = \eta^i_{\lambda_{j-1}} + \epsilon_j \left[ A^i_j(\lambda_j)\eta^i_{\lambda_{j-1}} + b^i_j(\lambda_j) \right]. \tag{21.29}$$

This auxiliary/actual split is the key source-faithfulness point for LEDH in PF-PF. The auxiliary path determines the local linearization, while the actual path is the transported proposal particle whose density must later be corrected. Using a single global affine map, or linearizing every particle at a fixed cloud-level point, is therefore not Li–Coates Algorithm 1 LEDH.

The same source-faithfulness point applies to the per-particle covariance objects. The covariance $P^i$ is part of the Li–Coates Algorithm 1 state used to construct the particle-local LEDH coefficients; BayesFilter does not treat this covariance state as a new filter invention. When a later differentiable OT or Sinkhorn resampling layer is attached to this Algorithm 1 route, the additional BayesFilter obligation is bookkeeping: specify how the covariance auxiliary state is carried through the chosen transport map, and test that this carry rule preserves the intended route. The transport layer supplies the resampling relaxation; it does not replace the source algorithm's covariance lifecycle or by itself prove HMC suitability.

This construction explains both the attraction and the danger of LEDH. The attraction is that each particle uses a local approximation intended to track nonlinear observation geometry more closely than a single global linearization. The danger is that the transported law now depends on a family of particle-specific linearizations, each of which may behave poorly if the Jacobian is unstable, ill-conditioned, or evaluated in a region where the local approximation is misleading. Hence LEDH is potentially more adaptive than EDH, but it is also more fragile numerically and interpretively. LEDH should not be read as guaranteed improvement without model-specific evidence.

From an implementation and debugging standpoint, LEDH adds at least three new failure routes relative to EDH:

(i) inaccurate or unstable local Jacobians;

(ii) poor local-information-vector construction;

(iii) path-dependent variation in stiffness across particles.

These are not secondary engineering details. They are direct consequences of how the local approximation enters the mathematics.

## 21.8 Stiffness and discretization

Even when the flow equations are well defined, their numerical integration may be difficult. The main difficulty is stiffness. When the observation noise is small or the effective likelihood precision is large, the eigenvalues of the drift matrix in the EDH or LEDH ODE can span several orders of magnitude. In that regime, explicit solvers require very small pseudo-time steps to avoid large truncation error or instability.

Formally, the stiffness is tied to the conditioning of the homotopy covariance or precision family. In the local case, for example, large eigenvalue spreads in $\Lambda_t^{(i)}$ may force

extremely fine pseudo-time resolution near the posterior end of the path. This matters for implementation because the flow construction is only useful if it can be integrated stably and repeatedly inside a filtering recursion.

The practical burden can be stated more concretely. If the pseudo-time ODE is integrated with an explicit method such as RK4, then the local truncation error is governed by powers of the step size, but the stability envelope is controlled by the operator norm and spectral spread of the drift matrix. In sharp- likelihood regimes, decreasing the observation noise or increasing the local precision can force the stable step size down dramatically. For LEDH this may happen non-uniformly across the cloud, because different particles see different local Jacobians and therefore different local precision matrices.

This is exactly why stiffness deserves more than a short warning. It creates a direct literature-to-debugging bridge:

- if a flow implementation behaves erratically only when observation noise becomes small, stiffness is a prime mathematical suspect;
- if certain particles show unstable trajectories while others remain well behaved, the local LEDH precision field is a prime suspect;
- if a nominally correct flow map fails under a coarse pseudo-time grid, the first question should be whether the flow is underresolved rather than whether the underlying theory is wrong.

For BayesFilter, the main consequence is therefore both conceptual and operational. Stiffness is not merely a performance issue. It is one of the main ways in which a formally attractive flow construction can become numerically unreliable enough to alter the practical behavior of the filter. That is why later implementation-facing chapters must record discretization policy, stiffness diagnostics, and stress tests explicitly, and why stiffness diagnostics should be treated as debugging tools rather than as optional background reading.

## 21.9 Exactness and approximation ledger

The particle-flow chapter has several layers that must not be collapsed.

| Layer | Status | Main implementation diagnostic |
|---|---|---|
| Normalized homotopy endpoints | exact model identity when the predictive density and likelihood are the intended model objects | recover predictive law at $\lambda = 0$ and filtering law at $\lambda = 1$ in analytic test cases |
| Continuity equation | exact conservation statement under regularity and boundary assumptions | verify transported test-function moments against ODE integration in controlled problems |
| Velocity field choice | not unique; EDH/LEDH are closure choices | record the chosen field and compare alternatives on benchmark models |
| EDH Gaussian closure | exact for the Gaussian family; exact filtering only in linear-Gaussian recovery | test covariance and mean endpoints against the Kalman update |
| LEDH local linearization | approximate closure using particle-local Jacobians and information vectors | monitor Jacobian conditioning, local precision spectra, and particle-wise endpoint dispersion |
| Pseudo-time integration | numerical approximation to the chosen ODE | run step size, tolerance, stiffness, and log-det sensitivity checks |

This ledger is deliberately conservative. It permits a strong statement about linear-Gaussian recovery, but it blocks any inference that a raw EDH or LEDH cloud is already a corrected nonlinear filter, a pseudo-marginal likelihood estimator, or an HMC-ready target.

## 21.10 What this chapter does and does not claim

This chapter develops the particle-flow transport mathematics itself. It does *not* yet claim:

- that the transported cloud alone defines the final corrected filtering law in the nonlinear case;
- that raw EDH or LEDH transport is already the final HMC target;
- that the Jacobian/proposal correction issue has been resolved;
- that differentiable resampling or OT relaxations have been introduced.

Those belong to the later PF-PF and resampling chapters. The role of the present chapter is narrower and mathematically prior: it explains how the transport path is constructed, where the Gaussian-closure or local-linearization approximation enters, and why the linear-Gaussian case supplies the exact special benchmark.

# Chapter 22

# Particle-Flow Particle Filters and Proposal Correction

The previous chapter developed particle flow as a transport construction. The main unresolved question left there was whether the transported cloud itself may already be treated as the corrected filtering object. In general, the answer is no. Outside exact special cases, the flow should be interpreted first as a proposal mechanism. The role of the present chapter is to make that proposal interpretation explicit and to derive the corresponding importance correction.

The starting question is concrete: once a particle has been moved by a deterministic flow, what density does that particle have, and what weight corrects it to the filtering target? We first derive the ancestor-conditioned answer used by Li–Coates. We then derive BayesFilter's optional marginal-mixture answer and its analytical total derivative. The final recursion includes the UKF covariance lifecycle, Contract E–Chol transport reset, and dual-cap GenUT correction; the later transport chapters develop those reset operations in detail. The implementation and numerical evidence described here are current through 6 October 2026.

## 22.1 Objects, assumptions, and source roles

The PF-PF argument depends on naming the target and proposal objects before writing any weight. This chapter uses:

| Object | Meaning |
|---|---|
| $x_{t-1}$ | ancestor state after the previous filtering step |
| $x_{t,0}$ | pre-flow particle sampled from a one-step proposal |
| $x_{t,1}$ | post-flow particle after applying the flow map |
| $q_{t,0}$ | pre-flow proposal density for $x_{t,0}$ conditional on $(x_{t-1}, y_t)$ |
| $\Phi_t$ | differentiable flow map from pre-flow to post-flow coordinates |
| $\Phi_t^{-1}$ | inverse map used to express the pre-image of a post-flow particle |
| $J_t = D\Phi_t$ | forward Jacobian matrix of the flow map |
| $q_{t,1}$ | induced post-flow proposal density for $x_{t,1}$ |
| $\gamma_t$ | unnormalized one-step filtering target density |
| $\tilde{w}_t$ | unnormalized importance weight correcting $q_{t,1}$ to $\gamma_t$ |

The source role is narrow. General SMC references [Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020] supply the proposal and importance-weight framework. Invertible

particle-flow particle filtering [Li and Coates, 2017] motivates treating particle flow as a proposal transformation with a Jacobian correction. These sources do not remove the need to state the target/proposal ratio locally, and they do not imply that finite-particle, numerical, or HMC target issues have disappeared.

## 22.2 Why proposal correction is needed

In the linear-Gaussian special case studied in the previous chapter, EDH can recover the exact Kalman update. Outside that case, however, the transport map is typically built under Gaussian closure or local linearization. Even if the transport moves particles toward high posterior density, the moved cloud is not automatically distributed according to the exact filtering law.

The right interpretation is therefore proposal-theoretic. Let $x_{t,0}$ denote a pre-flow particle sampled from a proposal law, and let the flow map transport it to

$$x_{t,1} = \Phi_t(x_{t,0}). \tag{22.1}$$

The question is then not merely where the particle moved, but which density on $x_{t,1}$ is induced by the transformation and how that induced proposal density must be corrected to target the filtering posterior.

This is the basic PF-PF point emphasized in the invertible particle-flow literature: the flow is best viewed as a proposal-improving device until its proposal-density effect has been accounted for. The exact change-of-variables identity is not itself the approximation; the approximation enters through the choice of flow family, the numerical integration of the flow, and the finite particle system.

## 22.3 Change of variables under the flow map

Assume that for each fixed time step and observation, the flow map $\Phi_t : \mathbb{R}^{n_x} \to \mathbb{R}^{n_x}$ is a differentiable bijection on the region of interest and that its forward Jacobian $D\Phi_t(x_{t,0})$ is an $n_x \times n_x$ nonsingular matrix along the proposal paths being weighted. Local nonsingularity alone is insufficient if several pre-images map to the same endpoint: their density contributions would have to be summed. The affine maps used below are globally one-to-one whenever their matrices are nonsingular. Let the pre-flow proposal density be

$$q_{t,0}(x_{t,0} \mid x_{t-1}, y_t). \tag{22.2}$$

Then the transported particle density is given by the standard change-of-variables formula,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(x_{t,0} \mid x_{t-1}, y_t) \left|\det D\Phi_t(x_{t,0})\right|^{-1}, \tag{22.3}$$

where $x_{t,0} = \Phi_t^{-1}(x_{t,1})$. The determinant in (22.3) is the determinant of the *forward* map from $x_{t,0}$ to $x_{t,1}$. Equivalently,

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = q_{t,0}(\Phi_t^{-1}(x_{t,1}) \mid x_{t-1}, y_t) \left|\det D\Phi_t^{-1}(x_{t,1})\right|.$$

These two forms are identical because $|\det D\Phi_t^{-1}(x_{t,1})| = |\det D\Phi_t(x_{t,0})|^{-1}$. The chapter uses the forward-map convention because the flow ODE naturally evolves $x_{t,0}$ to $x_{t,1}$ and the log-determinant ODE below accumulates $\log|\det D\Phi_t(x_{t,0})|$.

Thus the Jacobian determinant is not an optional numerical decoration. It is part of the proposal density itself. Any flow-based importance scheme that forgets this factor changes the proposal law and therefore changes the targeting relationship of the resulting importance weights.

## 22.4 Proposal-corrected PF-PF weights

Let the intended one-step filtering target be proportional to

$$\gamma_t(x_t) \propto p_\theta(y_t \mid x_t) p_\theta(x_t \mid x_{t-1}). \tag{22.4}$$

This target is conditional on the ancestor state. In a full particle filter, the ancestor itself is random and selected from the previous weighted cloud. The present derivation is therefore a one-step conditional density-ratio calculation; the full likelihood estimator also depends on ancestor selection, resampling policy, particle count, and time recursion.

For a generic pre-flow proposal $q_{t,0}$, the corrected importance weight is

$$\tilde{w}_t \propto \frac{\gamma_t(x_{t,1})}{q_{t,1}(x_{t,1} \mid x_{t-1}, y_t)} = \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|}{q_{t,0}(x_{t,0} \mid x_{t-1}, y_t)}. \tag{22.5}$$

The bootstrap/prior-proposal formula below is the special case $q_{t,0} = p_\theta(\cdot \mid x_{t-1})$.

If the pre-flow particle is sampled from the transition prior,

$$q_{t,0}(x_{t,0} \mid x_{t-1}, y_t) = p_\theta(x_{t,0} \mid x_{t-1}), \tag{22.6}$$

and then transported to $x_{t,1} = \Phi_t(x_{t,0})$, the post-flow proposal density is

$$q_{t,1}(x_{t,1} \mid x_{t-1}, y_t) = p_\theta(x_{t,0} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|^{-1}. \tag{22.7}$$

The corresponding unnormalized importance weight is therefore

$$\tilde{w}_t \propto \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1})}{q_{t,1}(x_{t,1} \mid x_{t-1}, y_t)} = \frac{p_\theta(y_t \mid x_{t,1}) p_\theta(x_{t,1} \mid x_{t-1}) \left|\det D\Phi_t(x_{t,0})\right|}{p_\theta(x_{t,0} \mid x_{t-1})}. \tag{22.8}$$

Equation (22.8) is the central correction formula for the PF-PF viewpoint. It shows exactly what proposal correction restores: once the transformation of proposal density has been accounted for, the flow-transported particle may be reweighted against the desired posterior factor in the usual importance-sampling way.

The formula also shows exactly what is *not* optional in code. A PF-PF implementation must evaluate, on the same path, the observation density at the post-flow particle, the transition density at the post-flow particle, the pre-flow proposal density at the pre-image, and the forward log determinant. Changing any one of these objects changes the proposal-to-target ratio being computed.

## 22.5 EDH/PF and LEDH/PF

The correction formula is structurally the same whether the flow map comes from EDH or LEDH. The difference lies in how the map itself is built.

### 22.5.1 EDH/PF

For EDH/PF, the transport map is generated by the global affine ODE from the previous chapter. The resulting proposal is easier to analyze because the flow field is common across the cloud. The main approximation point is the Gaussian closure used to derive the affine coefficients. If the affine ODE is integrated with a common map $\Phi_t$, the same forward log determinant applies to every particle up to its path through the time-varying affine field.

### 22.5.2 LEDH/PF

For LEDH/PF, the map is particle-specific: each particle evolves under a local linearization and therefore under its own drift coefficients. This can improve local adaptation to nonlinear observation structure, but it also makes the proposal analysis more delicate. The local map remains proposal-correctable in principle through the same change-of-variables logic, but the numerical burden of evaluating particle-specific Jacobians or log-determinants becomes more significant. A code path that reuses an EDH log determinant for LEDH has changed the proposal density unless the local maps are actually identical.

The important mathematical point is that EDH/PF and LEDH/PF are not merely flow algorithms. They are proposal-corrected particle methods. That is why they occupy a different target-status category from raw flow transport.

## 22.6 Li–Coates Algorithm 1 as an implementation contract

Algorithm 1 of Li and Coates [2017] is more specific than the generic proposal-correction derivation above. It is a recursive particle filter in which each particle carries both a state and a covariance object. For BayesFilter, this algorithm must be read as an implementation contract, not as a vague instruction to apply a deterministic affine map.

At the start of time step $k$, particle $i$ has an ancestor $x^i_{k-1}$, an importance weight $w^i_{k-1}$, and a posterior covariance $P^i_{k-1}$ inherited from the previous time step. The covariance lifecycle in Algorithm 1 is

$$(x^i_{k-1}, P^i_{k-1}) \xrightarrow{\text{EKF/UKF prediction}} (m^i_{k|k-1}, P^i) \xrightarrow{\text{EKF/UKF update}} (m^i_{k|k}, P^i_k). \tag{22.9}$$

The source states that EKF or UKF covariance prediction equations may be used when the dynamic model is nonlinear. The source does not make optimal transport resampling part of Algorithm 1, and it should not be cited as evidence for any BayesFilter OT relaxation. In the BayesFilter rebuild, using UKF for the prediction and update arrows in (22.9) is a reviewed BayesFilter implementation choice; it is not a claim that the paper's reported experiments used UKF in every scenario.

After the prediction arrow, Algorithm 1 defines two distinct state objects:

$$\bar{\eta}^i_0 = g_k(x^i_{k-1}, 0), \tag{22.10}$$

$$\eta^i_0 = g_k(x^i_{k-1}, v_k). \tag{22.11}$$

The first is the zero-noise transition anchor used to drive the local linearization. The second is the actual pre-flow proposal particle drawn through the dynamic model. The

implementation must keep both objects. If the code transports only the actual proposal and never maintains the auxiliary zero-noise path, then the local Jacobian field is not the one specified by Algorithm 1.

For pseudo-time grid $0 = \lambda_0 < \lambda_1 < \cdots < \lambda_{N_\lambda} = 1$ with $\epsilon_j = \lambda_j - \lambda_{j-1}$, Algorithm 1 repeats the following particle-specific update. Set $P = P^i$ and compute $A_j^i(\lambda_j)$ and $b_j^i(\lambda_j)$ from the LEDH formulas using the linearization at the current auxiliary state $\bar{\eta}^i$. Then move the auxiliary and actual states by the same local affine step:

$$\bar{\eta}^i \leftarrow \bar{\eta}^i + \epsilon_j\{A_j^i(\lambda_j)\bar{\eta}^i + b_j^i(\lambda_j)\}, \tag{22.12}$$

$$\eta_1^i \leftarrow \eta_1^i + \epsilon_j\{A_j^i(\lambda_j)\eta_1^i + b_j^i(\lambda_j)\}. \tag{22.13}$$

At the same pseudo-time step, the forward determinant multiplier is accumulated as

$$\theta^i \leftarrow \theta^i \left|\det\{I + \epsilon_j A_j^i(\lambda_j)\}\right|, \qquad \theta_0^i = 1. \tag{22.14}$$

Thus, at the end of the pseudo-time integration,

$$\theta^i = \prod_{j=1}^{N_\lambda} \left|\det\{I + \epsilon_j A_j^i(\lambda_j)\}\right|. \tag{22.15}$$

This is the discrete Algorithm 1 version of the forward determinant in (22.8). It is particle-specific because the LEDH coefficients are particle-specific.

After the flow, Algorithm 1 sets $x_k^i = \eta_1^i$ and applies the PF-PF importance update

$$w_k^i \propto \frac{p(x_k^i \mid x_{k-1}^i)p(z_k \mid x_k^i)\theta^i}{p(\eta_0^i \mid x_{k-1}^i)} w_{k-1}^i. \tag{22.16}$$

The denominator is the density of the pre-flow transition sample, while the transition density in the numerator is evaluated at the post-flow particle $x_k^i$. Replacing both densities by the same value, dropping $\theta^i$, or using a determinant from a different map changes the proposal-to-target ratio.

Finally, the weights are normalized, the EKF/UKF update produces $P_k^i$, and any resampling step resamples the triple

$$\{x_k^i, P_k^i, w_k^i\}_{i=1}^{N_p}. \tag{22.17}$$

The covariance component is part of the particle state for the next recursion. Resampling only the states while leaving covariances in their old order breaks the Algorithm 1 lifecycle. Likewise, introducing differentiable or optimal transport resampling may be a BayesFilter extension, but it must be labelled as an extension and audited separately from the Li–Coates source algorithm.

### 22.6.1 Why the OT block begins here

After importance weighting, the filter has a weighted post-flow ensemble and particle-local covariances. To continue with equally weighted particles, the implemented algorithm applies entropic transport, the Contract E–Chol reset and the higher-moment correction. The transport chapters derive these operations; Section 22.9 places them in the complete recursion together with either importance-weight policy.

### 22.6.2 What the later OT chapters inherit from PF-PF

The input to resampling is the weighted ensemble together with its particle-local state. In Li–Coates Algorithm 1 this is the triple in (22.17). A transport layer must produce an operation that acts on that ensemble, such as

$$\begin{gathered}\text{a coupling matrix, barycentric transform, or factorized operator,}\\ \text{or a directly transported equally weighted cloud.}\end{gathered} \tag{22.18}$$

A value of $\mathrm{OT}(\mu,\nu)$ alone cannot continue the filtering recursion; the next particle locations and their covariances are required.

Write the PF-PF output cloud schematically as

$$\mu_t^N = \sum_{i=1}^{N_p} w_k^i\, \delta_{x_k^i}, \tag{22.19}$$

with carried covariance or particle-local state attached to each support point. A deterministic OT equal-weighting layer or entropic OT/Sinkhorn layer, in the spirit of ensemble-transform and differentiable OT resampling [Reich, 2013, Corenflos et al., 2021], then asks for a map from $\mu_t^N$ to an equally weighted ensemble. The following transport chapters derive couplings and barycentric projections. In the implemented filter, the Contract E–Chol reset additionally restores specified moments, and GenUT controls higher-moment discrepancies. These steps operate on the weights produced by the chosen importance correction.

These details give a sharper audit checklist than the generic PF-PF equations:

(i) each particle has its own $P_{k-1}^i$, predicted $P^i$, and updated $P_k^i$;

(ii) the local LEDH Jacobian is evaluated along the particle's auxiliary zero-noise path;

(iii) the actual proposal particle and the auxiliary path are both migrated;

(iv) the determinant product in (22.15) uses the same local matrices that migrated the actual proposal particle;

(v) the weight uses the post-flow transition density, the observation density, the determinant product, the pre-flow proposal density, and the previous particle weight;

(vi) resampling, if used, resamples covariances together with states and weights.

## 22.7 Marginalizing the ancestor

The ancestor-conditioned identity is correct under its stated assumptions. Nevertheless, a child moved near a different ancestor's predictive mass can receive an extreme weight when its correction uses only its own ancestor. In the range–bearing example, narrow transition distributions separate nearby predictive components while nonlinear observations move children across them. Marginalization evaluates how much predictive and proposal density all ancestors assign to the same child. It changes the finite estimator; it does not repair an algebraic error in the original identity.

Condition on the incoming cloud, observation, and offline controls. Let $\omega_j$ be normalized weights and $m_j$ zero-innovation transition means. The implemented extension requires additive full-rank Gaussian transitions:

$$f_j(x) = \mathcal{N}(x; m_j, Q_\theta), \qquad p_N(x) = \sum_j \omega_j f_j(x), \qquad Z_N = \int g_\theta(y_t \mid x) p_N(x)\, dx. \tag{22.20}$$

Here $Q_\theta$ is the transition-noise covariance, not the UKF predicted covariance $P^j$ used to construct the flow. Substituting one for the other would change the proposal density.

The auxiliary path depends on the ancestor and observation but not on the child's current innovation. Consequently its finite Euler steps define an affine map $F_j(u) = B_j u + c_j$. For $M_{jr} = I + \epsilon_r A_{jr}$, accumulate

$$B_{j,0} = I,\ c_{j,0} = 0, \qquad B_{j,r+1} = M_{jr} B_{j,r}, \quad c_{j,r+1} = M_{jr} c_{j,r} + \epsilon_r b_{jr}. \tag{22.21}$$

This is the same ordered sequence that moves the actual particle. The auxiliary endpoint is $a_j = F_j(m_j)$, so

$$q_j(x) = \mathcal{N}(x; a_j, S_j), \quad S_j = B_j Q_\theta B_j^\top, \qquad q_N(x) = \sum_j \omega_j q_j(x). \tag{22.22}$$

The determinant correction is already present through $S_j$; adding a separate determinant would count it twice. Every component covariance must be positive definite. A singular transition, including an AR(2) state representation with deterministic coordinates, requires a derivation in the innovation measure and is unsupported by this full-state Gaussian formula.

For one child $X_i \sim q_i$ per ancestor, compare

$$\begin{aligned} V_i^{\text{anc}} &= \omega_i g_\theta(y_t \mid X_i) f_i(X_i)/q_i(X_i), \\ V_i^{\text{mar}} &= \omega_i g_\theta(y_t \mid X_i) p_N(X_i)/q_N(X_i). \end{aligned} \tag{22.23}$$

The outer $\omega_i$ is necessary because every component supplies one child. If instead $M$ labels were sampled independently from $\omega$, the estimator would average $M$ mixture ratios with coefficient $1/M$.

Assume $g_\theta p_N$ is integrable and $q_N > 0$ wherever $g_\theta p_N > 0$. For random children having the stated conditional marginal densities,

$$\begin{aligned} \mathbb{E}\left[\sum_i V_i^{\text{mar}} \mid \text{incoming cloud}\right] &= \sum_i \omega_i \int q_i(x) g_\theta(y_t \mid x) \frac{p_N(x)}{q_N(x)}\, dx \\ &= \int g_\theta(y_t \mid x) p_N(x)\, dx = Z_N. \end{aligned} \tag{22.24}$$

Independence between children is unnecessary for this expectation calculation. Cancellation of $q_i$ gives the same expectation for the old weights. Both therefore target the incoming cloud's predictive mixture; their distribution of finite-sample error differs.

This applies the marginal-particle-filter construction [Klaas et al., 2005, Section 3] to the actual affine LEDH proposals. For a random label $J \sim \omega$ and $X \mid J \sim q_J$, conditioning $g(X) f_J(X)/q_J(X)$ on $X$ gives $g(X) p_N(X)/q_N(X)$. The law of total variance gives a Rao–Blackwell variance inequality if second moments exist. That argument concerns a random

label, and does not establish smaller variance for one fixed child per ancestor, smaller log-likelihood error, or a more accurate score. Fixing the random design and composing deterministic resets over time also removes any justification for reading (22.24) as an unbiasedness theorem for that finite log likelihood. For an identity flow, $q_j = f_j$ and both weight laws reduce to $\omega_i g(X_i)$, providing a useful algebraic check.

## 22.8 The analytical total score

Fix base innovations, reset designs, and offline controls, and write the finite log likelihood as $\widehat{\ell}_N(\theta)$. The analytical recursion computes its total derivative. It does not hold particle locations fixed or substitute the derivative of the exact model likelihood. Let $D$ denote a parameter-direction derivative on a neighborhood where the program is valid and differentiable; it includes previous particles, weights, UKF covariances, both flow paths, and the current evaluation point.

For $u = C^{-1}(x-\mu)$, direct differentiation of a Gaussian log density gives

$$D\log\mathcal{N}(x;\mu,C) = u^\top(D\mu - Dx) + \tfrac{1}{2}u^\top(DC)u - \tfrac{1}{2}\operatorname{tr}(C^{-1}DC). \tag{22.25}$$

The first term includes the moving child; the others differentiate the quadratic form and covariance determinant. In a proposal component,

$$DS_j = (DB_j)QB_j^\top + B_j(DQ)B_j^\top + B_jQ(DB_j)^\top, \qquad DB_{j,r+1} = (DM_{jr})B_{j,r} + M_{jr}DB_{j,r}. \tag{22.26}$$

The code obtains $Da_j$ from the existing auxiliary-path tangent, not from a separately constructed flow.

For either mixture $h_N = \sum_j \omega_j h_j$, define responsibilities $r_j^h = \omega_j h_j(X_i)/h_N(X_i)$. Differentiating the sum then dividing gives

$$D\log h_N(X_i) = \sum_j r_j^h\{D\log\omega_j + D\log h_j(X_i)\}. \tag{22.27}$$

These responsibilities are coefficients of the first derivative. Differentiating them again here would incorrectly add second-derivative terms. Consequently,

$$\begin{aligned} D\log V_i^{\mathrm{mar}} &= D\log\omega_i + D\log g_\theta(y_t \mid X_i) + D\log p_N(X_i) - D\log q_N(X_i), \\ \widehat{\ell}_N &= \sum_t \log\sum_i V_{ti}, \qquad D\widehat{\ell}_N = \sum_t\sum_i w_{ti}D\log V_{ti}, \\ Dw_{ti} &= w_{ti}\left(D\log V_{ti} - \sum_j w_{tj}D\log V_{tj}\right), \qquad w_{ti} = V_{ti}/\sum_j V_{tj}. \end{aligned} \tag{22.28}$$

These finite-program identities determine the quantity the implementation must compute. Accuracy relative to the model score is a separate question. Angle-branch boundaries and failed factorizations remain explicit validity conditions.

## 22.9 The complete implemented recursion

The executable filter performs the following sequence at each observation. Its score uses analytical recursive tangents; automatic differentiation and finite differences are independent diagnostic checks.

1. Carry incoming particles, weights, particle-local UKF covariances and their parameter tangents. The initial distribution includes its parameter dependence.

2. Predict each UKF covariance. Form its zero-innovation anchor and actual transition draw. Evolve both through the same local affine LEDH steps, carrying their tangents and the discrete determinant.

3. Use (22.16) for the default ancestor policy. For the optional marginal policy, also carry $B_j, DB_j$ in that loop and use the complete mixtures and tangents in (22.23)–(22.28). No component truncation or nearest-ancestor approximation is used.

4. Accumulate the log normalizer and score, normalize the weights, and perform the UKF observation covariance update.

5. Apply entropic transport and the existing Contract E–Chol reset, carrying covariances through the same transport. The total derivative includes direct source-moment/weight dependence and dependence through transport. A transported-cloud-only derivative would be wrong.

6. Apply the existing GenUT higher-moment correction, including diagonal and pairwise controls, trust-region constraints and coordinate caps, with its analytical derivative and validity checks. Carry the equally weighted outgoing cloud and reset covariances into the next observation.

The statistical change is step 3. Its changed weights also alter subsequent moments, transport, correction, and future flow paths. Those algorithms are unchanged, but their inputs and outputs generally differ.

The consumer `canonical_batch_fused_value_score` forwards `importance_weight_policy` to the shared executor `ledh_canonical_score_tf.py`. That executor calls `marginal_prior_ratio_tangent` in `ledh_marginal_weights_tf.py`. The default is `ancestor`; the extension requires `marginal_mixture`. Unsupported transition callbacks, innovation geometries, and incompatible modes fail explicitly. Marginal weighting, OT reset, and GenUT correction are BayesFilter extensions; they are not claimed to be unchanged Li–Coates Algorithm 1.

Both mixtures sum every component using log-sum-exp and exact equal row/column tiles. The extent is $K = N$ for $N \leq 3000$; otherwise it is the largest divisor of $N$ at most 3000. Thus $N = 4096$ uses $K = 2048$. Tiling bounds intermediate storage but retains quadratic pair count, so runtime must be compared alongside error.

## 22.10 Evidence from the three models

The completed 5 October experiment tested scalar LGSSM at $T = 2$, three-state LGSSM at $T = 2, 10$, and scalar KSC stochastic volatility at $T = 10, 20$. KSC retained the seven-component observation mixture and corrected scale derivative. Each scope had three parameter points, one untouched dataset and 32 paired designs at $N = 256$. References were exact Kalman values and scores for LGSSM and refined one-dimensional integration for KSC. All 15 comparisons had simultaneous paired upper bounds below the declared deterioration margin 0.05 for absolute log-likelihood error and $0.1\|s - s_{\text{ref}}\|_2/\sqrt{p}$. This supports material non-inferiority conditional on those datasets, not improvement in every model or replication across datasets.

The four-state, two-observation range–bearing experiment also used $N = 4096$. The table gives mean absolute log-likelihood errors over 32 paired designs; the difficult regime reduces process-noise scale. Quadrature provided the $T = 1$ reference and a larger replicated PF with Fisher scores the $T = 20$ reference.

| Horizon | Regime | Ancestor | Marginal | Bootstrap PF |
|---|---|---|---|---|
| 1 | Nominal | 1.083 | 0.125 | 0.059 |
| 1 | Low process noise | 2.428 | 0.497 | 0.045 |
| 20 | Nominal | 1.271 | 0.143 | 0.122 |
| 20 | Low process noise | 21.294 | 0.770 | 0.489 |

Paired intervals support smaller likelihood and scaled-score errors than the ancestor policy in all four cells. The difficult $T = 20$ mean scaled-score error decreased from 6.325 to 0.351. Bootstrap PF nevertheless had smaller observed likelihood error in every row. This is a partial repair; the simple-comparator check prevents promotion. Reference Monte Carlo error and finite-particle bias remain relevant at $T = 20$.

Fourteen full-filter FP64 finite-difference cases covered every physical parameter direction of the seven scopes and both policies. Saved default regressions were unchanged, an $N = 4096$ mixture primitive passed an exact-tile comparison, and 23 focused tests passed. The first full-filter derivative checks used batch size one. The 6 October checks subsequently compared distinct parameter rows in one batch with separate consumer calls: all tested values and scores agreed exactly, with largest relative finite-difference discrepancy $1.78 \times 10^{-5}$. Fresh $T = 1$ range–bearing quadrature and the larger replicated PF reference agreed within recorded uncertainty. At $N = 4096$, three repeated evaluations of each policy at two M13 parameter points were finite and exactly repeatable. The largest FP32/FP64 difference was 0.00530 in log likelihood and 0.00340 in scaled score, both below the declared 0.05 materiality margin. These precision checks support retaining FP32 for the campaign; they do not compare either filter with the model truth.

Fresh-data replication has now finished for the scalar LGSSM at $T = 10, 20$ and the three-state LGSSM at $T = 10$, all with $N = 1024$. Each scope used 16 independently generated datasets and 32 paired particle designs per dataset, evaluated at three parameter points. Dataset-level paired error differences, rather than individual designs, determined the uncertainty intervals. All nine parameter cells met the predeclared 0.05 non-inferiority margin for both likelihood and scaled-score error. The simultaneously adjusted approximate Student intervals supported smaller errors only in the cells counted below; the other comparisons did not establish a ranking.

| Scope | Non-inferior cells | Smaller value error | Smaller score error |
|---|---|---|---|
| LGSSM $d = 1,\ T = 10$ | 3/3 | 0/3 | 2/3 |
| LGSSM $d = 1,\ T = 20$ | 3/3 | 1/3 | 2/3 |
| LGSSM $d = 3,\ T = 10$ | 3/3 | 1/3 | 1/3 |

These conclusions concern the tested regimes and the declared materiality margin. With only 16 dataset clusters, interval coverage remains approximate. The exact Kalman filter remains the accuracy and cost reference for LGSSM.

At $N = 1024$, neither tested numerical-control setting produced valid results throughout calibration and validation for $d = 10, T = 10$, $d = 20, T = 10$, or $d = 20, T = 240$. Both weight policies encountered invalid filter states. Those scopes therefore supply no

accuracy comparison. The cause has not been localized, and the larger-particle stages remain part of the experiment.

The running 6 October campaign continues across horizons $10, 20, 50, 120, 240$, higher-dimensional LGSSMs, the native KSC and range–bearing models, reference refinement, particle-count ladders and equal-runtime comparators. The computational comparisons use FP32/TF32 for both LEDH and the inexpensive filters; FP64 is reserved for independent accuracy references. Bootstrap filtering supplies a likelihood alone, whereas the measured LEDH call also supplies its score. Thus this comparison measures the cost of obtaining a likelihood from the current implementations; it cannot establish relative efficiency for HMC forces. Uncompleted cells supply no evidence.

The scientific goal remains testing surrogate HMC. A fixed finite likelihood defines a numerical target; using a learned surrogate force is another choice. When that force constructs a reversible, volume-preserving proposal and a Metropolis test evaluates the chosen endpoint target, the target in the test determines the invariant law. Force accuracy alone does not establish agreement of that law with the statistical posterior. The experiments above test the likelihood/score component and supply no new posterior draws, acceptance, effective sample sizes or R-hat evidence.

## 22.11 Jacobian determinant and log-determinant evolution

Directly forming a full Jacobian matrix for every particle and time step may be numerically and computationally burdensome. A more useful identity is obtained by differentiating the Jacobian matrix along the flow.

Let

$$J_t(\lambda) = \frac{\partial x_t(\lambda)}{\partial x_t(0)}. \tag{22.29}$$

The terminal forward determinant in (22.3) is $\det J_t(1)$. The inverse-density form would use $-\log|\det J_t(1)|$ instead. This sign distinction is one of the most common PF-PF implementation errors.

If the flow satisfies

$$\frac{dx_t(\lambda)}{d\lambda} = v_{t,\lambda}(x_t(\lambda)), \tag{22.30}$$

then differentiation with respect to the initial point gives

$$\frac{dJ_t(\lambda)}{d\lambda} = \nabla v_{t,\lambda}(x_t(\lambda))J_t(\lambda), \qquad J_t(0) = I. \tag{22.31}$$

The derivation is direct: differentiate the flow equation with respect to the initial coordinate $x_t(0)$ and apply the chain rule. Applying Jacobi's formula $d\log|\det J|/d\lambda = \operatorname{tr}(J^{-1}dJ/d\lambda)$ then yields

$$\frac{d}{d\lambda}\log|\det J_t(\lambda)| = \operatorname{tr}\big(\nabla v_{t,\lambda}(x_t(\lambda))\big). \tag{22.32}$$

because $J_t^{-1}(\nabla v_{t,\lambda})J_t$ has the same trace as $\nabla v_{t,\lambda}$. Thus

$$\log|\det D\Phi_t(x_{t,0})| = \int_0^1 \operatorname{tr}\big(\nabla v_{t,\lambda}(x_t(\lambda))\big)\, d\lambda.$$

For an affine flow $v_{t,\lambda}(x) = A_t(\lambda)x + b_t(\lambda)$, this simplifies to

$$\frac{d}{d\lambda} \log |\det J_t(\lambda)| = \operatorname{tr}(A_t(\lambda)). \tag{22.33}$$

The practical consequence is clear: one may integrate a scalar log-determinant alongside the particle-flow ODE rather than carrying a full dense Jacobian for every particle in the affine case. In more general settings, the same identity still shows that the determinant is part of the dynamical system, not an external afterthought. This also gives a direct debugging criterion: if the corrected weights behave inconsistently with the transported cloud, the first mathematical suspects are the Jacobian/log-determinant path, the numerical flow integration, and the finite-particle approximation rather than the formal change-of-variables identity itself.

## 22.12 What the correction restores, and what it does not

The proposal correction in (22.8) restores a clearer importance-sampling interpretation. More precisely, it restores the correct proposal-to-target density ratio for the transported proposal generated by the flow map. This is mathematically important: it distinguishes PF-PF from a pure transport approximation that simply declares the endpoint cloud to be posterior- like without density correction.

What the correction does *not* automatically restore is an exact analytic filter in the nonlinear case. Several approximation layers may still remain:

- the flow may itself be derived under Gaussian closure or local linearization;
- the ODE is integrated numerically rather than in closed form;
- the particle system still uses a finite number of particles;
- Jacobian or log-determinant quantities may be computed with numerical approximation.

Therefore PF-PF is mathematically stronger than raw flow transport, but it is not automatically equivalent to an exact nonlinear filtering likelihood. The exact identity in this chapter is the change-of-variables correction for the proposal density; what remains approximate are the flow construction itself when closure or local linearization is used, the numerical integration of that flow, and the finite-particle system used to estimate the target. Its status is better described as a proposal-corrected particle approximation whose value-side interpretation is clearer than that of uncorrected flow transport.

At trajectory level, this one-step correction is multiplied through the filtering recursion in the same spirit as ordinary sequential importance sampling. For a particle path and ancestor history, the product of incremental corrections defines the path proposal weight. That statement is still conditional on using the stated proposal, the stated flow map, and the stated determinant convention at each time step. It is not a separate theorem that the finite-particle likelihood estimate is exact in the nonlinear case.

## 22.13 Implementation audit obligations

The formulas above imply concrete checks before PF-PF can be used as a value-side candidate in later HMC experiments.

(i) **Affine density check**: choose a low-dimensional affine map with known determinant and compare (22.3) against a direct transformed-density calculation.

(ii) **Forward/inverse sign check**: verify that the proposal density uses $-\log|\det D\Phi_t|$ while the numerator form in (22.8) contributes $+\log|\det D\Phi_t|$.

(iii) **Jacobian ODE check**: compare the integrated trace in (22.32) with an automatic-differentiation or finite-difference determinant in small dimension.

(iv) **Corrected versus uncorrected weights**: run the same transported cloud with and without the determinant/proposal-density term to confirm that the diagnostic detects the expected discrepancy.

(v) **Path consistency**: on fixed random seeds, verify that the scalar value and analytical total score use the same proposal density, flow map, and log-determinant path; autodiff is a diagnostic comparator only.

These are not tuning diagnostics. They are algebraic and numerical checks of the target/proposal object itself.

## 22.14 Why this chapter matters for the later HMC discussion

This chapter is the first point in the rebuilt DPF block where a sharper target question becomes possible. Raw flow transport gives a geometrically appealing approximation, but PF-PF introduces an explicit proposal density and an explicit correction factor. That is why later HMC-suitability analysis will treat proposal-corrected EDH/PF or LEDH/PF as the first rungs in this ladder with an explicit proposal-corrected value-side interpretation. That is a narrower statement than saying they are HMC-ready.

The key lesson is not that PF-PF solves all target issues. The key lesson is that it restores the proposal-correction logic that a valid particle method requires. This is exactly the right place in the monograph to separate flow-as-transport from flow-as-proposal.

## 22.15 What this chapter does and does not claim

This chapter derives both ancestor-conditioned and marginal-mixture importance corrections, the analytical derivative of the implemented finite filter, and how the weighting step composes with the UKF, OT reset, and GenUT correction. The October experiments support a conditional partial repair. The expanded fresh-data campaign and the later HMC assessment must still establish how the component behaves across datasets, larger scopes, and posterior inference.

# Part VI

# Differentiable Resampling and Transport

# Chapter 23

# Soft Differentiable Resampling

The previous chapters established the classical particle-filter setting in which a weighted cloud of particles is turned back into an equally weighted cloud by a resampling step. The mathematical difficulty is that the standard resampling map is discrete: it duplicates some particles, deletes others, and changes abruptly when weights cross threshold values. That makes naive pathwise differentiation break down.

This chapter studies the simplest smooth replacement of that discrete step. It opens the part on differentiable resampling and transport by showing, in the smallest possible setting, how differentiability can be bought by changing the resampling map itself before moving to other differentiable and transport-based alternatives. The neighboring literature includes truncated-gradient differentiable particle filters [Jonschkowski et al., 2018], learned differentiable resampling [Zhu et al., 2020], resampling-mixture particle-filter networks [Karkus et al., 2018], and entropy-regularized transport resampling [Corenflos et al., 2021].

## 23.1 What problem this chapter solves

Suppose a weighted particle cloud is

$$\hat{\pi}_t^N(dx) = \sum_{i=1}^N w_t^{(i)} \, \delta_{x_t^{(i)}}(dx), \qquad w_t^{(i)} \geq 0, \quad \sum_{i=1}^N w_t^{(i)} = 1. \tag{23.1}$$

Classical multinomial resampling tries to convert this into an equally weighted cloud by drawing ancestor indices from the categorical law defined by the weights [Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020]. This is fine as a simulation rule. It is not fine if one wants the realized output cloud to depend smoothly on the weights.

The chapter therefore asks one very specific question:

> How can we make the resampled cloud depend more smoothly on the weights, and what bias is introduced when we do so?

## 23.2 Running two-particle discontinuity cell

The smallest useful example has two particles. Let

$$x_1 = -1, \qquad x_2 = 2, \qquad w = (p, 1-p), \qquad p \in (0,1),$$

and let one fixed common random number $U \in (0,1)$ be used to define the first resampled ancestor. Standard categorical resampling chooses

$$a(U,p) = \begin{cases} 1, & U \le p, \\ 2, & U > p. \end{cases} \tag{23.2}$$

For a fixed $U$, this is a step function of $p$: small changes in the weight usually do nothing, and then a threshold crossing changes the realized ancestor abruptly.

**Plain-language takeaway.** If the algorithm literally copies an ancestor chosen by a discrete threshold, the output particle usually stays exactly the same when the weights move a little. A pathwise derivative of that realized output is therefore typically zero until a jump occurs.

## 23.3 Classical resampling bottleneck

For standard multinomial resampling, the equal-weight cloud is obtained by sampling ancestor indices

$$a_j \sim \text{Categorical}(w_t^{(1:N)})$$

and setting

$$\tilde{x}_t^{(j)} = x_t^{(a_j)}. \tag{23.3}$$

This map is exact for the classical resampling law, but it is not pathwise differentiable with respect to the weights because the realized ancestor path is piecewise constant in those weights [Zhu et al., 2020, Jonschkowski et al., 2018].

## 23.4 Soft resampling rule

A simple smooth surrogate interpolates between the sampled ancestor and the weighted mean of the cloud. This chapter uses "soft resampling" in this narrow local sense; it is a deliberately minimal teaching rule, and it is not the same algorithm as the resampling-mixture rule of Karkus et al. [2018]. Define

$$\bar{x}_t = \sum_{i=1}^{N} w_t^{(i)} x_t^{(i)}, \tag{23.4}$$

and then set

$$\tilde{x}_t^{(j)} = \alpha x_t^{(a_j)} + (1-\alpha)\bar{x}_t, \qquad \alpha \in [0,1]. \tag{23.5}$$

When $\alpha = 1$, the rule reduces to hard categorical resampling. When $\alpha < 1$, the output depends smoothly on the weighted mean, so the realized output cloud becomes less discontinuous as a function of the weights.

## 23.5 Worked two-particle calculation

In the two-particle cell,

$$\bar{x} = p(-1) + (1-p)2 = 2 - 3p.$$

The soft-resampling output for one realized ancestor is therefore

$$\tilde{x}(U,p) = \alpha x_{a(U,p)} + (1-\alpha)(2-3p). \tag{23.6}$$

Two immediate facts are now visible.

First, when the ancestor is held fixed, the dependence on $p$ is smooth through the weighted mean term. Second, the object being computed has changed: the surrogate particle is no longer equal to the chosen ancestor itself unless $\alpha = 1$.

The weighted mean is preserved in conditional expectation because

$$\mathbb{E}[\tilde{x}_t^{(j)} \mid X, w] = \alpha \sum_i w_i x_i + (1-\alpha)\bar{x}_t = \bar{x}_t. \tag{23.7}$$

So the surrogate keeps the first-moment center of mass, but not necessarily other nonlinear summaries of the cloud.

## 23.6 Algorithm: soft differentiable resampling

**Algorithm 1** `soft_resampling_step`

**Require:** weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, interpolation parameter $\alpha \in [0,1]$, realized ancestor indices $a_j$ if hard ancestors are retained, or a separately declared softened ancestor rule if they are not

**Ensure:** equal-weight surrogate cloud $\widetilde{X} = (\tilde{x}^{(1)}, \ldots, \tilde{x}^{(N)})$ and the declared ancestor policy

1: Compute the weighted mean $\bar{x} = \sum_{i=1}^N w_i x_i$
2: **for** each output index $j = 1, \ldots, N$ **do**
3: Form $\tilde{x}^{(j)} = \alpha x_{a_j} + (1-\alpha)\bar{x}$
4: **end for**
5: Return the equal-weight cloud $\widetilde{X}$, the chosen $\alpha$, and the declared ancestor policy used to define $a_j$

**Approximation enters here.** This algorithm is not another notation for categorical resampling. It is a new surrogate object that trades exactness for smoother dependence on the weights.

## 23.7 Filtering-loop insertion point

Inside the filtering loop, the soft surrogate sits exactly where hard resampling would have sat:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{soft resampling}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \tag{23.8}$$

It changes only the equal-weighting rule. It does not, by itself, change the preceding weighting rule or add an OT correction.

## 23.8 Verification contract

A minimal verification pass should check:

(1) the weighted mean is preserved as in (23.7),

(2) the surrogate becomes the hard categorical rule when $\alpha = 1$,

(3) nonlinear test-function bias is visible when expected,

(4) the declared ancestor policy is recorded explicitly,

(5) any downstream gradient claim states whether ancestors remain discrete or are themselves relaxed.

A failed mean-preservation check is evidence that the surrogate has been implemented incorrectly. A successful mean-preservation check is not evidence that the classical categorical target has been preserved.

## 23.9 What this chapter does and does not claim

This chapter teaches the simplest smooth resampling surrogate and its bias. It does *not* claim:

- that the surrogate preserves the exact categorical resampling law,
- that preserving the weighted mean preserves the full distribution,
- that the resulting scalar is automatically the same target later used by HMC or training.

Its role is narrower: it shows, in the smallest possible setting, how resampling can be made smoother and where the resulting approximation enters. The next chapter keeps the equal-weighting goal but replaces the surrogate rule by a declared transport object.

# Chapter 24

# Deterministic OT Equal-Weighting Baseline

The previous chapter introduced the resampling bottleneck and explained why one might replace discrete ancestor selection by a transport object. The present chapter slows that idea down. Its purpose is to teach the deterministic optimal- transport equal-weighting baseline itself before entropy, Sinkhorn iteration, or learned acceleration are added.

The key conceptual move is simple. A weighted particle cloud and an equal-weight particle cloud are not the same object. Deterministic OT equal-weighting asks for a mass-redistribution rule that sends the weighted source cloud to an equal-weight target cloud as cheaply as possible under a declared cost [Villani, 2003, Peyré and Cuturi, 2019, Reich, 2013]. This already changes the resampling object relative to categorical ancestor sampling, but it does so in a clean, finite-dimensional, deterministic way.

## 24.1 What problem this chapter solves

Suppose a particle filter has produced support points $x_1, \dots, x_N \in \mathbb{R}^{d_x}$ with normalized weights $w_1, \dots, w_N$. A later stage of the filter may want an equally weighted cloud so that no single particle dominates future propagation or summary statistics. Classical resampling achieves this by random duplication and elimination [Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020]. The deterministic OT baseline instead asks:

> How should mass be reassigned from the weighted cloud to an equal-weight output cloud if we want to move it as little as possible under a declared transport cost?

This chapter answers that question in three layers:

(i) define the weighted and equal-weight empirical objects;

(ii) define the coupling matrix that records how source mass is reassigned;

(iii) convert that coupling into the equal-weight output cloud by barycentric projection.

These are the same basic objects that underlie deterministic ensemble-transform and OT-based equal-weighting formulations [Reich, 2013, Peyré and Cuturi, 2019].

| Stage | Source object | Output object | What later chapters change |
|---|---|---|---|
| This chapter | weighted empirical cloud | deterministic coupling and barycentric equal-weight cloud | nothing yet; fixes the baseline object |
| Next chapter | same weighted cloud | entropically regularized coupling and barycentric cloud | replaces the deterministic LP by an entropic teacher and finite Sinkhorn route |
| Later OT chapters | same weighted cloud | accelerated, learned, or target-changing transports | may preserve the teacher, compress it, or replace it entirely |

## 24.2 Running three-particle cell: deterministic OT version

Reuse the running one-dimensional cell from the previous chapter:

$$x_1 = -2, \qquad x_2 = 0, \qquad x_3 = 3, \qquad w = \left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right), \qquad u = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right).$$

Using the source support itself as a simple square target support, the quadratic cost matrix is

$$C = \begin{pmatrix} 0 & 4 & 25 \\ 4 & 0 & 9 \\ 25 & 9 & 0 \end{pmatrix}. \tag{24.1}$$

A feasible coupling for this problem is

$$\Pi^{\mathrm{cell}} = \begin{pmatrix} \frac{1}{3} & \frac{1}{6} & 0 \\ 0 & \frac{1}{6} & \frac{1}{6} \\ 0 & 0 & \frac{1}{6} \end{pmatrix}. \tag{24.2}$$

The row sums are $(1/2, 1/3, 1/6)$, matching the source weights, and the column sums are $(1/3, 1/3, 1/3)$, matching the equal target marginal. The barycentric output cloud is therefore

$$\tilde{x}^{(1)} = 3\left(\frac{1}{3}(-2)\right) = -2, \tag{24.3}$$

$$\tilde{x}^{(2)} = 3\left(\frac{1}{6}(-2) + \frac{1}{6}(0)\right) = -1, \tag{24.4}$$

$$\tilde{x}^{(3)} = 3\left(\frac{1}{6}(0) + \frac{1}{6}(3)\right) = 1.5. \tag{24.5}$$

So one deterministic equal-weight cloud associated with this feasible transport is

$$\{-2, -1, 1.5\} \quad \text{with equal weights } \frac{1}{3}.$$

This single worked cell is enough to make three key points visible:

(i) the coupling is a mass-routing matrix, not yet the output cloud;

(ii) the output particles need not be duplicates of the source support;

(iii) deterministic equal-weighting already differs from categorical resampling even before entropy or differentiability are mentioned.

## 24.3 Objects and notation

The chapter uses the following core objects.

| Object | Meaning in this chapter |
|---|---|
| $\hat{\pi}^N$ | weighted empirical source measure |
| $\hat{\pi}^N_{\mathrm{eq}}$ | equal-weight empirical output measure |
| $w$ | source weight vector |
| $u$ | target equal-weight vector, usually $(1/N, \dots, 1/N)$ |
| $C$ | declared transport cost matrix |
| $\Pi$ | coupling matrix sending source mass to target mass |
| $\mathcal{U}(w, u)$ | feasible coupling set with row sums $w$ and column sums $u$ |
| $\Pi_0^\star$ | optimal coupling for the declared unregularized OT problem |
| $B(\Pi)$ | barycentric equal-weight cloud induced by coupling $\Pi$ |

## 24.4 Weighted cloud, equal-weight cloud, and the coupling set

The weighted source cloud is

$$\hat{\pi}^N(dx) = \sum_{i=1}^N w_i \, \delta_{x_i}(dx), \qquad w_i \geq 0, \quad \sum_{i=1}^N w_i = 1. \tag{24.6}$$

The equal-weight target cloud is

$$\hat{\pi}^N_{\mathrm{eq}}(dx) = \frac{1}{N} \sum_{j=1}^N \delta_{\tilde{x}^{(j)}}(dx). \tag{24.7}$$

A coupling is a nonnegative matrix telling us how much mass from source particle $i$ contributes to output slot $j$. The feasible set is

$$\mathcal{U}(w, u) = \left\{ \Pi \in \mathbb{R}_+^{N \times N} : \Pi \mathbf{1} = w, \ \Pi' \mathbf{1} = u \right\}. \tag{24.8}$$

The row constraints say each source particle gives away exactly its original weight. The column constraints say each output slot receives exactly one equal share of total mass.

**Reader checkpoint.** At this stage the coupling $\Pi$ is only a bookkeeping object. It is not yet a random ancestor rule and not yet the final equal-weight cloud. The next step is to choose which feasible coupling is preferred.

### 24.4.1 Why the filter needs a transport object, not just a scalar

The scalable-OT survey sharpens the main practical boundary here. For the BayesFilter filtering recursion, a fast scalar optimal-transport value is not by itself a usable resampling output. The filter needs an object that actually acts on the weighted particle cloud. In the deterministic equal-weighting baseline, that usable object is the coupling $\Pi$ together with its barycentric action, in the spirit of ensemble-transform and differentiable OT-resampling work [Reich, 2013, Corenflos et al., 2021]. Schematically, the required outputs are of the form

coupling matrix, barycentric projection, factorized transport operator, or directly transported equal-w (24.9)

A reported scalar transport cost or value certificate may help compare candidate transports, but it does not yet produce the next particle representation used by the filter.

This is why the present chapter slows down on couplings and barycentric maps. Given the weighted source measure

$$\hat{\pi}^N(dx) = \sum_{i=1}^{N} w_i \, \delta_{x_i}(dx) \tag{24.10}$$

and equal-weight target marginal $u$, the mathematical output needed by later code is not merely the scalar

$$\min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle, \tag{24.11}$$

but the minimizing coupling itself together with the map that turns that coupling into an equal-weight cloud. The later entropic and Sinkhorn chapter will replace this exact linear-program teacher by an entropically regularized teacher, but it inherits the same computational doctrine: a usable transport object must be produced, not only a scalar score.

## 24.5 The unregularized deterministic OT baseline

Given a declared cost matrix $C$, the deterministic transport baseline chooses a feasible coupling with minimal cost:

$$\Pi_0^\star \in \arg\min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle. \tag{24.12}$$

This is a linear program. It is exact for the chosen deterministic OT object, not for classical categorical resampling. The distinction matters. A reader who keeps only one sentence in mind should keep this one:

> The deterministic OT baseline solves a transport projection problem exactly for that declared transport problem, but that problem is already different from random ancestor sampling.

## 24.6 Barycentric projection: from coupling to equal-weight cloud

Once a coupling has been chosen, the final equal-weight output cloud is obtained by barycentric projection. For each output slot $j$, define

$$B(\Pi)_j = \frac{1}{u_j} \sum_{i=1}^{N} \Pi_{ij} x_i. \tag{24.13}$$

In the equal-weight case $u_j = 1/N$, this becomes

$$B(\Pi)_j = N \sum_{i=1}^{N} \Pi_{ij} x_i. \tag{24.14}$$

Operationally, each column of the coupling is turned into an output particle by forming the weighted average of the source particles that contribute mass to that column.

This is the key step that makes the OT baseline implementation-facing. The coupling is a matrix of mass transfers; the barycentric map is the actual equal- weight cloud later code would use.

## 24.7 Algorithm: deterministic OT equal-weighting

**Algorithm 2** `deterministic_ot_equalweight_step`

**Require:** weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, equal target marginal $u$, declared target support rule $z_j$, and cost matrix $C_{ij} = c(x_i, z_j)$

**Ensure:** optimal coupling $\Pi_0^\star$, barycentric equal-weight cloud $\widetilde{X}$, and declared transport metadata

1: Form the feasible coupling set $\mathcal{U}(w, u)$

2: Solve the linear program $\Pi_0^\star \in \arg\min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle$

3: **for** each output slot $j = 1, \ldots, N$ **do**

4: Compute the barycentric output particle

$$\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^{N} \Pi_{ij}^\star x_i$$

5: **end for**

6: Return the equal-weight cloud $\widetilde{X} = (\tilde{x}^{(1)}, \ldots, \tilde{x}^{(N)})$, the coupling $\Pi_0^\star$, the declared target support rule, and any transport diagnostics

**Approximation enters here.** This algorithm is exact for the unregularized deterministic OT object in (24.12). It is not exact for categorical resampling, and it is not yet the entropically smoothed object used later for differentiable computation.

## 24.8 Filtering-loop insertion point

Inside a filtering loop, this deterministic OT step sits after the weighted cloud has been constructed and before the equal-weight cloud is passed to the next propagation or summary stage. In symbols:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{deterministic OT equal-weighting}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \tag{24.15}$$

If auxiliary particle-local state is carried elsewhere in the filter, this chapter does not yet decide how that state is transported. That is a separate implementation contract. The present chapter fixes only the equal-weighting rule for the particle locations and masses themselves.

## 24.9 Verification and failure interpretation

A minimal verification contract should check:

(1) row sums equal the source weights exactly,

(2) column sums equal the equal target marginal exactly,

(3) the barycentric output has the declared shape $N \times d_x$,

(4) the target support rule is recorded explicitly,

(5) any claim about downstream filtering is kept separate from this local OT equality check.

A failed marginal check is evidence that the coupling solver or bookkeeping is wrong. It is not evidence that OT is conceptually invalid. A good marginal check is evidence only that the deterministic OT object has been implemented correctly, not that the output cloud preserves the same law as categorical resampling.

## 24.10 What this chapter does and does not claim

This chapter teaches the clean deterministic transport baseline for equal- weighting. It does *not* claim:

- that deterministic OT equal-weighting is the same object as classical categorical resampling,
- that it is automatically differentiable in the sense required later,
- that barycentric projection preserves the original particle-filter target,
- that the output cloud is already the final HMC or training target.

Its role is narrower: it fixes the exact deterministic OT baseline so that the next chapter can explain what changes when entropy, Sinkhorn iteration, and solver differentiation are added. More precisely, the next chapter will preserve the need for a usable transport object and the same barycentric output map while changing the teacher coupling from the unregularized linear-program object to an entropically regularized one.

# Chapter 25

# Entropic OT, Sinkhorn, and Barycentric Projection

The previous chapter fixed the deterministic OT equal-weighting baseline. This chapter explains the main differentiable computational route used in the OT lane: entropic regularization, finite Sinkhorn iteration, barycentric projection, and the distinction between the mathematical teacher object and the numerical object actually differentiated in code.

This chapter has one dominant goal: a reader should be able to understand what object finite Sinkhorn computes, how barycentric output is produced from it, where approximation enters, and what diagnostics must be checked before the result is used in a downstream filtering scalar.

## 25.1 What changes when entropy is added

The previous chapter fixed the unregularized deterministic baseline by choosing a feasible coupling with minimal declared transport cost,

$$\Pi_0^\star \in \arg\min_{\Pi \in \mathcal{U}(w,u)} \langle C, \Pi \rangle. \tag{25.1}$$

Entropic OT changes that teacher object before any numerical approximation enters. For a regularization level $\varepsilon > 0$, it replaces the linear program by an objective that also penalizes overly concentrated couplings through the entropy term. In plain language, the plan is now rewarded for spreading mass more smoothly instead of concentrating all mass as sharply as possible.

This change matters mathematically as well as numerically. The transport object is no longer the unregularized deterministic optimizer from (25.1). It is a different teacher problem whose solution is smoother and whose first-order conditions can be rewritten as a matrix-scaling problem. That rewrite is what later makes Sinkhorn iteration possible. Thus the chapter's logic is:

(i) change the deterministic teacher by adding entropy,

(ii) derive the factorized form of the new optimizer,

(iii) derive the balancing equations for its scaling vectors,

(iv) and only then discuss the finite numerical route used in code.

The gain is numerical smoothness and a tractable scaling-based solver. The cost is that the transport object is no longer the unregularized deterministic OT solution. This is the chapter's first approximation layer, and later finite iteration will add another one on top of it.

## 25.2 Running three-particle cell with entropic smoothing

Keep the same running cell as before:

$$x_1 = -2, \qquad x_2 = 0, \qquad x_3 = 3, \qquad w = \left(\frac{1}{2}, \frac{1}{3}, \frac{1}{6}\right), \qquad u = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right),$$

with quadratic cost matrix

$$C = \begin{pmatrix} 0 & 4 & 25 \\ 4 & 0 & 9 \\ 25 & 9 & 0 \end{pmatrix}. \tag{25.2}$$

For $\varepsilon = 1$, the Gibbs kernel is

$$K_1 = \begin{pmatrix} 1 & e^{-4} & e^{-25} \\ e^{-4} & 1 & e^{-9} \\ e^{-25} & e^{-9} & 1 \end{pmatrix}. \tag{25.3}$$

The entries are heavily concentrated near the diagonal, but not exactly zero away from it. That is a simple visual sign of what entropy has changed: the plan is no longer forced into the sharpest possible assignment.

## 25.3 Entropic OT teacher object

Let $\mathcal{U}(w, u)$ be the feasible coupling set from the previous chapter. The entropically regularized OT problem is [Cuturi, 2013, Peyré and Cuturi, 2019]

$$\Pi_\varepsilon^\star = \arg\min_{\Pi \in \mathcal{U}(w,u)} \left\{ \langle \Pi, C \rangle + \varepsilon \sum_{i,j} \Pi_{ij}(\log \Pi_{ij} - 1) \right\}. \tag{25.4}$$

This chapter uses the same entropy convention as the earlier OT material, so the minimizer is the exact teacher for the chosen regularized object, not for the unregularized linear program and not for categorical resampling.

**Reader checkpoint.** At this stage, the chapter has changed the mathematical object once: from the unregularized deterministic OT coupling to the entropically smoothed OT coupling. Nothing numerical has happened yet. The next step still stays at the exact-object level: first identify the factorized teacher form, and only after that turn to the finite executable route.

### 25.3.1 Exact teacher versus exact engineering route

a useful extra distinction before moving on to finite Sinkhorn iterations. There are two different senses in which a route may still be called "exact" for the entropic OT object.

First, there is the mathematical teacher object $\Pi_\varepsilon^\star$ from (25.4). That is the exact minimizer of the entropically regularized problem. Second, there is an engineering route that preserves that same entropic object while changing how the matrix-scaling updates are executed in memory or on hardware. Exact online, streaming, and GPU Sinkhorn methods belong to this second category: they do not change the entropic teacher problem, but they avoid materializing the full cost, kernel, or coupling whenever possible [Feydy et al., 2019, Charlier et al., 2021, Ye et al., 2026, Xiao, 2026].

This distinction matters for the BayesFilter transport lane. A method that recomputes the same entropic scaling updates through blocked reductions or streamed kernel application is still on the semantics-preserving reference lane. By contrast, a method that replaces the kernel, truncates the coupling family, or changes the transport objective has moved to a different approximation class even if it is motivated by scalability. The reader should therefore keep separate:

(i) the exact entropic teacher object,

(ii) the finite numerical object produced by a declared Sinkhorn execution path,

(iii) the engineering strategy used to realize that numerical path.

The next sections fix the second object explicitly and later staged chapters will discuss scalability routes that either preserve or alter it.

### 25.3.2 The current BayesFilter bottleneck

The need for scalable OT in BayesFilter is not only a generic literature concern. The current TensorFlow route already distinguishes dense and streamed execution. Dense mode materializes the effective transport matrix and applies it to the particle cloud directly. Streamed mode avoids storing that full matrix, but it still loops over row and column blocks and therefore still computes the same underlying pairwise transport interactions.

For equal-weight barycentric output, the streamed route still forms costs of the schematic form

$$C_{ij} = \frac{1}{2}\|\tilde{x}_i - \tilde{x}_j\|_2^2, \tag{25.5}$$

then log-transport weights of the schematic form

$$\log T_{ij} = \frac{f_i + g_j - C_{ij}}{\varepsilon} - \ell_j + \log N + \log w_j, \tag{25.6}$$

and finally blockwise transport application by accumulation,

$$z_i = \sum_j T_{ij} x_j. \tag{25.7}$$

The gain is real: memory pressure is reduced because the full transport matrix need not be kept in RAM or device memory. But the all-pairs arithmetic burden remains. Streaming therefore solves part of the storage problem without yet solving the full large-$N$, large-$d_x$ interaction problem. This is exactly why the derivation below matters: once the teacher object has been rewritten as a scaling problem, one can see clearly which later routes merely change execution strategy and which ones change the transport object itself.

## 25.4 Why the optimizer factorizes

The next step is to derive the multiplicative structure of the entropic optimizer rather than only state it. Introduce Lagrange multipliers $\alpha \in \mathbb{R}^N$ for the row constraints and $\beta \in \mathbb{R}^N$ for the column constraints. With the entropy convention from (25.4), the Lagrangian is

$$\mathcal{L}(\Pi, \alpha, \beta) = \sum_{i,j} [C_{ij}\Pi_{ij} + \varepsilon \Pi_{ij}(\log \Pi_{ij} - 1)] + \sum_i \alpha_i \left( \sum_j \Pi_{ij} - w_i \right) \tag{25.8}$$

$$+ \sum_j \beta_j \left( \sum_i \Pi_{ij} - u_j \right). \tag{25.9}$$

Stationarity with respect to each entry $\Pi_{ij}$ gives

$$0 = \frac{\partial \mathcal{L}}{\partial \Pi_{ij}} = C_{ij} + \varepsilon \log \Pi_{ij} + \alpha_i + \beta_j, \tag{25.10}$$

so

$$\log \Pi_{ij} = -\frac{C_{ij}}{\varepsilon} - \frac{\alpha_i}{\varepsilon} - \frac{\beta_j}{\varepsilon}. \tag{25.11}$$

Exponentiating entrywise yields

$$\Pi_{ij} = \exp\left(-\frac{\alpha_i}{\varepsilon}\right) \exp\left(-\frac{C_{ij}}{\varepsilon}\right) \exp\left(-\frac{\beta_j}{\varepsilon}\right). \tag{25.12}$$

Define the positive scalings

$$a_i = \exp(-\alpha_i/\varepsilon), \qquad b_j = \exp(-\beta_j/\varepsilon), \qquad (K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon). \tag{25.13}$$

Then the optimizer must have the Gibbs-scaled form

$$\Pi^\star_\varepsilon = \operatorname{diag}(a) K_\varepsilon \operatorname{diag}(b). \tag{25.14}$$

This is the key structural step. The entropic teacher problem has been converted from a constrained optimization over all couplings into the search for two positive scaling vectors that make a known kernel satisfy the marginal constraints.

### 25.4.1 Why the balancing equations are Sinkhorn updates

Once (25.14) is available, the marginal constraints from $\mathcal{U}(w, u)$ become explicit equations for the scaling vectors. The row constraints require

$$\Pi^\star_\varepsilon \mathbf{1} = \operatorname{diag}(a) K_\varepsilon \operatorname{diag}(b) \mathbf{1} = a \odot (K_\varepsilon b) = w, \tag{25.15}$$

where $\odot$ denotes entrywise multiplication. Therefore

$$a = w \oslash (K_\varepsilon b). \tag{25.16}$$

Likewise the column constraints require

$$(\Pi^\star_\varepsilon)^\top \mathbf{1} = \operatorname{diag}(b) K_\varepsilon^\top \operatorname{diag}(a) \mathbf{1} = b \odot (K_\varepsilon^\top a) = u, \tag{25.17}$$

so

$$b = u \oslash (K_\varepsilon^\top a). \tag{25.18}$$

These are exactly the balancing equations behind Sinkhorn iteration. The algorithm is not a mysterious OT oracle added from outside the derivation. It simply alternates the two marginal repairs implied by the factorized optimizer.

**Reader checkpoint.** The chapter has now introduced three different objects that should not be confused:

(i) the exact entropic teacher problem (25.4),

(ii) the factorized teacher optimizer (25.14),

(iii) and the balancing equations (25.16)–(25.18) that a finite numerical routine will try to satisfy.

Finite Sinkhorn iteration begins only after these exact relations have been fixed.

## 25.5 Algorithm: finite Sinkhorn equal-weighting

**Algorithm 3** `sinkhorn_equalweight_step`

**Require:** weighted particle cloud $\{x_i, w_i\}_{i=1}^N$, equal target marginal $u$, cost matrix $C$, regularization $\varepsilon > 0$, initialization $(a^{(0)}, b^{(0)})$, iteration budget $K$, and stabilization policy

**Ensure:** finite coupling $\Pi_{\varepsilon,K}$, barycentric equal-weight cloud $\widetilde{X}$, row/column residuals, and declared numerical metadata

Form the Gibbs kernel $(K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon)$
**for** $k = 0, \dots, K-1$ **do**
    Update $a^{(k+1)} = w \oslash (K_\varepsilon b^{(k)})$
    Update $b^{(k+1)} = u \oslash (K_\varepsilon' a^{(k+1)})$
**end for**
Form the finite coupling $\Pi_{\varepsilon,K} = \operatorname{diag}(a^{(K)}) K_\varepsilon \operatorname{diag}(b^{(K)})$
Compute residuals

$$r_{\text{row}} = \|\Pi_{\varepsilon,K}\mathbf{1} - w\|_1, \qquad r_{\text{col}} = \|\Pi_{\varepsilon,K}'\mathbf{1} - u\|_1$$

**for** each output slot $j = 1, \dots, N$ **do**
    Compute $\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij} x_i$
**end for**
Return the coupling, equal-weight cloud, residuals, $\varepsilon$, $K$, initialization policy, and stabilization metadata

The algorithm returns a *finite* numerical object, not the exact teacher optimizer from (25.4). After $K$ balancing passes, the executed coupling is

$$\Pi_{\varepsilon,K} = \operatorname{diag}(a^{(K)}) K_\varepsilon \operatorname{diag}(b^{(K)}), \tag{25.19}$$

with residuals that measure how closely the declared row and column constraints have been met. This distinction is the second approximation layer of the chapter: first the deterministic teacher was replaced by an entropic teacher, and then that entropic teacher was approximated by a finite numerical route. The next custom- gradient chapter will differentiate this declared finite route, not an abstract teacher floating above the executed computation.

**Reader checkpoint.** It is useful to keep the chain of objects explicit:

(i) deterministic OT teacher $\Pi_0^\star$,

(ii) exact entropic teacher $\Pi_\varepsilon^\star$,

(iii) finite executed coupling $\Pi_{\varepsilon,K}$,

(iv) barycentric equal-weight cloud built from $\Pi_{\varepsilon,K}$.

Later differentiation claims must attach to the actually executed object in this chain, not jump back silently to one of the earlier teachers.

### 25.5.1 Exact online and streaming Sinkhorn as the reference lane

The scalable-OT survey also clarifies how BayesFilter should talk about scaling without silently changing the transport object. A finite Sinkhorn route can be made substantially more practical by changing the execution strategy rather than the underlying entropic teacher problem. Streaming, online, or GPU-specialized implementations still belong to the reference lane when they preserve the same entropic scaling equations and the same barycentric output rule while avoiding full materialization of the cost or kernel matrix [Feydy et al., 2019, Charlier et al., 2021, Ye et al., 2026, Xiao, 2026].

In that reference-lane setting, the implementation may compute the needed row-wise or column-wise reductions in blocks, keep only partial transport state in memory, or fuse barycentric application with solver passes. What must remain fixed is the declared numerical object: the finite coupling $\Pi_{\varepsilon,K}$, its residual checks, and the barycentric output derived from that same finite coupling. Thus a streamed or GPU-tiled Sinkhorn routine is not automatically a new approximation class. It becomes a new approximation class only when the scaling equations, kernel, feasible family, or output object itself are changed.

This exact-reference reading is important because it separates engineering success from scientific overclaim. Exact-online or GPU-specialized Sinkhorn may reduce memory traffic, improve device utilization, and make larger particle clouds practical while still computing the same finite entropic object. But these routes normally leave the dominant all-pairs interaction structure in place: every full row or column update still depends on interactions with the whole opposing cloud. So they are the right first scaling lane when the goal is to preserve semantics, but they are not by themselves the final answer to truly large-$N$, large-$d_x$ transport.

This chapter therefore treats exact-online and streaming implementations as engineering variants of the same entropic reference lane. Later staged chapters on retained-teacher approximations will take up the different case where the transport object itself is compressed, factorized, or otherwise altered for scalability.

This is the chapter's main callable procedure. A reader who implements only one OT route from the monograph should be able to implement this one.

### 25.5.2 Sparse, screened, stabilized, and accelerated Sinkhorn variants

The scalable-OT survey also suggests a useful boundary inside the broader Sinkhorn family. Some variants still belong naturally to the exact-reference lane because they mainly

stabilize or schedule the same scaling equations more carefully. Others move toward approximation because they restrict which entries, rows, or columns are treated as active, or because they replace dense kernel use by a screened or sparse surrogate [Schmitzer, 2016, Alaya et al., 2019, Schmitzer, 2019, Altschuler et al., 2017, Dvurechensky et al., 2018, Abid and Gower, 2018, Lin et al., 2019, Xu and Chen, 2026, Genevay et al., 2016].

At the most conservative end, stabilization policies remain part of the same finite-Sinkhorn object. Log-domain updates, shifting, epsilon scheduling, or other numerical defenses are execution details attached to the same declared coupling $\Pi_{\varepsilon,K}$ and should therefore be judged first by the usual marginal residual and barycentric parity checks.

Sparse or screened routes require a sharper audit. If they only prune obviously negligible work while preserving the same effective scaling equations and output object to numerical tolerance, they may still be read as near-reference engineering variants. But once the active support, feasible family, or effective kernel semantics are altered in a way that changes the finite transport object, the method is no longer only a better implementation of the same reference lane. It has crossed into retained-teacher or structure-changing territory and must be judged accordingly.

A similar caution applies to accelerated, greedy, or stochastic update schedules. These methods may reduce iteration count, repair the most violated row or column first, or otherwise improve convergence behavior of the balancing routine. That is valuable, but it does not by itself remove dense-kernel access. If each major update still needs interactions against the whole opposing cloud, the dominant large-$N$, large-$d_x$ arithmetic bottleneck remains.

This is why the present chapter keeps these variants conceptually downstream from the finite teacher object rather than promoting them to new transport doctrines. They refine how the reference route is executed. They do not automatically change what object the filter is claiming to compute.

## 25.6 Worked balancing pass on the running cell

Take the simplest initialization $b^{(0)} = (1, 1, 1)'$. Then the first row-balancing step computes

$$a^{(1)} = w \oslash (K_1 b^{(0)}). \tag{25.20}$$

For the running cell,

$$K_1 b^{(0)} \approx (1.0183,\ 1.0184,\ 1.0001),$$

so

$$a^{(1)} \approx (0.491,\ 0.327,\ 0.167).$$

The next column-balancing step then uses these row-repaired values to push the column sums toward $(1/3, 1/3, 1/3)$. The exact arithmetic is less important than the interpretation: finite Sinkhorn repeatedly repairs whichever marginal is currently wrong.

**Plain-language takeaway.** Sinkhorn is not a mysterious OT oracle. It is an alternating balancing procedure for a smoothed transport matrix. That is why later learned warm-start methods try to predict better initial balancing coordinates rather than replacing the solver outright.

## 25.7 Barycentric projection as the output map

The solver output is still only a coupling matrix. The equal-weight particle cloud that later code consumes is obtained by barycentric projection:

$$\tilde{x}^{(j)} = \frac{1}{u_j} \sum_{i=1}^{N} \Pi_{ij} x_i. \tag{25.21}$$

In the equal-weight case, this is

$$\tilde{x}^{(j)} = N \sum_{i=1}^{N} \Pi_{ij} x_i. \tag{25.22}$$

This step is where the numerical coupling becomes the actual equal-weight output cloud. It is therefore the place where an implementation must be especially clear about shapes, normalization, and any auxiliary state that is or is not carried alongside the particle locations.

**Proposition 25.1** (Barycentric equal-weighting contracts covariance)**.** *Let $X$ be a random variable that takes values $x_1, \dots, x_N \in \mathbb{R}^d$ with probabilities $w_1, \dots, w_N$. Let $J$ be an equal-weight output index, $\mathbb{P}(J = j) = 1/N$, and suppose a coupling between $X$ and $J$ has the same source and target marginals as the OT coupling used in (25.22). The barycentric output associated with index $J$ is*

$$\widetilde{X} = \mathbb{E}[X \mid J].$$

*Then*

$$\mathbb{E}[\widetilde{X}] = \mathbb{E}[X], \tag{25.23}$$

*but*

$$\mathrm{Cov}(\widetilde{X}) = \mathrm{Cov}(X) - \mathbb{E}[\mathrm{Cov}(X \mid J)] \preceq \mathrm{Cov}(X). \tag{25.24}$$

*Equality holds only when the conditional covariance $\mathrm{Cov}(X \mid J)$ is zero almost surely.*

*Proof.* The mean identity follows from the tower property:

$$\mathbb{E}[\widetilde{X}] = \mathbb{E}[\mathbb{E}[X \mid J]] = \mathbb{E}[X].$$

For the covariance, apply the law of total covariance:

$$\mathrm{Cov}(X) = \mathrm{Cov}(\mathbb{E}[X \mid J]) + \mathbb{E}[\mathrm{Cov}(X \mid J)]\,.$$

Since $\widetilde{X} = \mathbb{E}[X \mid J]$, rearranging gives (25.24). The second term is positive semidefinite, so the barycentric covariance is no larger in Loewner order. Equality requires the conditional covariance term to vanish almost surely. □

**Implication for filtering values.** This proposition is the finite-$N$ reason that a barycentric equal-weight cloud should not be treated as a categorical-resampling cloud. Categorical resampling draws particles from the weighted empirical law and is unbiased for test functions conditional on the weighted cloud. Barycentric projection instead replaces each conditional cloud by its conditional mean. It preserves affine test functions, including the first moment, but it generally changes quadratic and nonlinear test functions.

In a linear-Gaussian state-space model, the next predictive likelihood increment depends on the predictive covariance through $S_t = HP_{t|t-1}H' + R$. A covariance-contracted reset can therefore change the finite-$N$ scalar being estimated even when Sinkhorn marginal residuals are small, except in special cases where the lost covariance lies in observation-null directions or otherwise cancels from the downstream likelihood increment. The exact Kalman likelihood remains a valid comparator for a no-OT weighted particle approximation, but it is not a direct oracle for the barycentric-reset scalar unless the reset route also preserves the relevant moments or the comparison is explicitly redefined.

**Implication for SV, SIR, and nonlinear models.** The same issue is not special to LGSSM. In stochastic-volatility models, future observation densities are nonlinear functions of latent volatility, so the finite-$N$ likelihood scalar can change when barycentric projection compresses the spread of the volatility particles. In SIR and other nonlinear compartmental models, the transition and observation maps are nonlinear in the state, so preserving only the particle mean is not enough to preserve infection tails, state constraints, threshold behavior, or future likelihood curvature. A value or gradient computed through the barycentric-reset filter can still be a correct gradient of the executed barycentric scalar; it is not automatically the gradient of the categorical-resampling particle-filter likelihood, the exact filtering likelihood, or a moment-preserving approximation. Consequently, SIR or SV gradient checks must be same-scalar checks unless an additional evidence gate shows that barycentric bias is negligible for the declared model, particle count, regularization, and horizon.

The distinction matters for the later custom-gradient chapter (Chapter 26). That chapter does not differentiate an abstract teacher floating outside the executed computation. It must differentiate the declared finite Sinkhorn route together with the barycentric map that actually produces the equal-weight cloud used by the filtering scalar.

## 25.8 Remedy options for barycentric covariance loss

The remedy has to be stated at the level of the reset map, because the covariance defect is not a vague numerical pathology of Sinkhorn. It is a deterministic consequence of replacing each column of a transport plan by its conditional mean. This subsection makes that statement as a finite-dimensional matrix calculation.

For the algebra in this subsection, assume first that the Sinkhorn marginal constraints have been solved exactly. Finite iteration and floating-point residuals add normalization errors that should be checked separately, but they are not the mechanism discussed here. Let

$$X = [x_1, \dots, x_N] \in \mathbb{R}^{d_x \times N}, \qquad w \in \mathbb{R}^N_+, \qquad \mathbf{1}^\top w = 1, \qquad W = \operatorname{diag}(w).$$

The weighted empirical measure before reset is

$$\nu_w = \sum_{i=1}^N w_i \delta_{x_i}.$$

Any deterministic equal-weight reset whose output particles are linear combinations of the input particles can be written as

$$Y = XD, \qquad y_j = \sum_{i=1}^N x_i D_{ij}, \tag{25.25}$$

for some $N \times N$ transform matrix $D$. The barycentric Sinkhorn reset is the special case

$$D_B = N\Pi_{\varepsilon,K}, \tag{25.26}$$

because the target marginal is $u_j = 1/N$. The row and column marginal constraints on $\Pi_{\varepsilon,K}$ become the first-order transform constraints

$$D^\top \mathbf{1} = \mathbf{1}, \qquad D\mathbf{1} = Nw. \tag{25.27}$$

The first condition says that every output particle is an affine combination of the input particles. The second says that the equal-weight output cloud has the same mean as the weighted input cloud:

$$\bar{y}_D := \frac{1}{N} Y\mathbf{1} = \frac{1}{N} XD\mathbf{1} = Xw =: \mu_w. \tag{25.28}$$

The second moment is where the problem lives. Define the weighted input covariance and the equal-weight output covariance by

$$\Sigma_w := \sum_{i=1}^{N} w_i (x_i - \mu_w)(x_i - \mu_w)^\top = X(W - ww^\top)X^\top, \tag{25.29}$$

$$\Sigma_D := \frac{1}{N} \sum_{j=1}^{N} (y_j - \mu_w)(y_j - \mu_w)^\top = \frac{1}{N} X(D - w\mathbf{1}^\top)(D - w\mathbf{1}^\top)^\top X^\top. \tag{25.30}$$

A deterministic reset is therefore *second-order exact for the current cloud* when it satisfies (25.27) and

$$\Sigma_D = \Sigma_w. \tag{25.31}$$

A stronger, cloud-independent sufficient condition is

$$(D - w\mathbf{1}^\top)(D - w\mathbf{1}^\top)^\top = N(W - ww^\top). \tag{25.32}$$

Equations (25.27)–(25.32) are the finite-dimensional version of the second-order accurate ensemble-transform condition used by Acevedo, de Wiljes, and Reich: the output ensemble matches both the importance-weighted mean and covariance; see their Definition 3.1 and Sections 5–6 for the Sinkhorn-based second-order ETPF construction [Acevedo et al., 2017]. The factor $N$ in (25.32) is not a convention-free decoration: it follows from using the biased equal-weight output covariance $N^{-1}\sum_j (y_j - \mu_w)(y_j - \mu_w)^\top$ together with a transform matrix satisfying $D\mathbf{1} = Nw$.

**Proposition 25.2** (Why the barycentric Sinkhorn reset is not second-order exact)**.** *Assume $D_B = N\Pi_{\varepsilon,K}$ is nonnegative and satisfies (25.27). Interpret $J$ as a uniform output index and define a source index $I$ by*

$$\mathbb{P}(I = i \mid J = j) = D_{B,ij}.$$

*Then $X_I$ has the weighted empirical law $w$, the barycentric output is $Y_J = \mathbb{E}[X_I \mid J]$, and*

$$\Sigma_w - \Sigma_{D_B} = \mathbb{E}[\mathrm{Cov}(X_I \mid J)] \succeq 0. \tag{25.33}$$

*Consequently $D_B$ satisfies the second-order cloud contract (25.31) only if the conditional covariance inside every output column vanishes as a state-space covariance. Equivalently, for each output column $j$, all source particles with $D_{B,ij} > 0$ must have the same state value, up to duplicate particle locations. In generic diffuse entropic couplings this condition fails.*

*Proof.* The column sums in (25.27) make $D_{B,\cdot j}$ a probability vector. Since $(1/N)D_B\mathbf{1} = w$, the unconditional law of $I$ is $w$. The output particle in column $j$ is exactly

$$y_j = \sum_i x_i D_{B,ij} = \mathbb{E}[X_I \mid J = j].$$

Applying the law of total covariance gives

$$\mathrm{Cov}(X_I) = \mathrm{Cov}(\mathbb{E}[X_I \mid J]) + \mathbb{E}[\mathrm{Cov}(X_I \mid J)].$$

Because $J$ is uniform,

$$\mathrm{Cov}(\mathbb{E}[X_I \mid J]) = \frac{1}{N}\sum_{j=1}^{N}(y_j - \mu_w)(y_j - \mu_w)^\top = \Sigma_{D_B}.$$

The left side is $\Sigma_w$, which proves (25.33). □

This is why the bug is not solved by asking Sinkhorn to converge more tightly. A smaller marginal residual makes $D_B$ a better approximation to the declared entropic barycentric transform. It does not change the identity (25.33), because that identity is about the output map $D_B \mapsto XD_B$, not about whether the coupling has been balanced accurately. A real remedy must therefore change, or explicitly keep, the mathematical object being estimated. The following contracts are mutually different finite-$N$ estimators.

**Contract A: keep the same barycentric scalar.** Set $D = D_B$ and accept that $\Sigma_D \preceq \Sigma_w$. The reported value and gradient are then only claims about the finite barycentric-reset scalar. The correct derivative check is a same-scalar finite-difference check of that exact executed computation. This contract repairs value-gradient naming, not the covariance loss. In notation, if the filter returns $L_B(\theta)$, where every reset uses $Y_B(\theta) = X(\theta)D_B(\theta)$, then the admissible gradient claim is

$$\nabla_\theta L_B(\theta),$$

not the gradient of the categorical particle-filter likelihood and not the gradient of an exact Kalman or nonlinear filtering likelihood.

**Contract B: stochastic categorical output.** Given the same nonnegative $D_B$, draw $I_j$ independently or by a chosen resampling scheme from $\mathbb{P}(I_j = i \mid J = j) = D_{B,ij}$ and output $z_j = x_{I_j}$. Then for any test function $\varphi$ on the particle support,

$$\mathbb{E}\left[\frac{1}{N}\sum_{j=1}^{N}\varphi(z_j)\,\middle|\, D_B, X\right] = \sum_{i=1}^{N} w_i\varphi(x_i). \tag{25.34}$$

This restores the unbiased empirical-measure property of categorical resampling in expectation [Gordon et al., 1993, Doucet et al., 2001, Chopin and Papaspiliopoulos, 2020], but the realized reset is random and discontinuous as a function of weights and particles. It is therefore a different estimator and not a pathwise differentiable replacement for $D_B$.

**Contract C: second-order deterministic transform.** Replace $D_B$ by

$$D^\sharp = D_B + \Delta, \qquad \Delta^\top \mathbf{1} = 0, \qquad \Delta \mathbf{1} = 0, \tag{25.35}$$

and choose $\Delta$ so that

$$\frac{1}{N} X(D^\sharp - w\mathbf{1}^\top)(D^\sharp - w\mathbf{1}^\top)^\top X^\top = X(W - ww^\top)X^\top. \tag{25.36}$$

The stronger matrix-level target is

$$(D^\sharp - w\mathbf{1}^\top)(D^\sharp - w\mathbf{1}^\top)^\top = N(W - ww^\top). \tag{25.37}$$

One concrete reference object satisfying the matrix target is the NETF-type square-root transform

$$D_{\mathrm{NETF}}(Q) = w\mathbf{1}^\top + \sqrt{N}\,(W - ww^\top)^{1/2} Q, \qquad Q \in \mathbb{R}^{N\times N}, \qquad Q^\top Q = QQ^\top = I_N, \qquad Q\mathbf{1} = \mathbf{1}. \tag{25.38}$$

Here $(W - ww^\top)^{1/2}$ denotes the principal symmetric square root. Indeed, $(W - ww^\top)\mathbf{1} = 0$, so the square root also annihilates $\mathbf{1}$, and $D_{\mathrm{NETF}}(Q)^\top \mathbf{1} = \mathbf{1}$, $D_{\mathrm{NETF}}(Q)\mathbf{1} = Nw$, and (25.32) holds:

$$(D_{\mathrm{NETF}} - w\mathbf{1}^\top)(D_{\mathrm{NETF}} - w\mathbf{1}^\top)^\top = N(W - ww^\top)^{1/2} QQ^\top (W - ww^\top)^{1/2} = N(W - ww^\top).$$

Thus Contract C is mathematically explicit: the filter is no longer using the barycentric transform $D_B$; it is using a new transform $D^\sharp$ whose mean and covariance are the declared finite-$N$ moments. Inside this contract, the NETF-type transform above and a second-order corrected ETPF are distinct finite-$N$ choices. The NETF-type transform is a reference moment-preserving object, not a promise of closeness to $D_B$. A second-order corrected ETPF instead tries to stay close to the OT transform while enforcing the same moment equations [Reich, 2013, Acevedo et al., 2017]. The unavoidable cost is that $D^\sharp$ need not be nonnegative. Hence the corrected particles need not lie in the convex hull of the pre-reset particles, and bounded, positive, simplex-valued, or otherwise constrained state spaces need an explicit constraint policy.

**Contract D: affine state-space moment restoration.** Instead of correcting the $N \times N$ transform matrix, one may correct the already formed barycentric cloud. Let $Y_B = XD_B$ and $\Sigma_B = \Sigma_{D_B}$. If there exists a matrix $A_m \in \mathbb{R}^{d_x \times d_x}$ such that

$$A_m \Sigma_B A_m^\top = \Sigma_w, \tag{25.39}$$

then

$$y_j^\sharp = \mu_w + A_m(y_j - \mu_w), \qquad j = 1, \ldots, N, \tag{25.40}$$

has equal-weight mean $\mu_w$ and covariance $\Sigma_w$. A full-rank example is $A_m = \Sigma_w^{1/2}\Sigma_B^{-1/2}$. If $\Sigma_B$ is singular, exact affine restoration can exist only if $\mathrm{rank}(\Sigma_w) \le \mathrm{rank}(\Sigma_B)$; jitter, truncation, or pseudoinverses define an additional approximation policy, not an identity. This contract is simple to test in an LGSSM, but it is a new finite target and requires its own value gate, same-scalar gradient gate, and state-constraint audit.

**Contract E: positive spread, residual noise, and affine restoration.** The preceding contracts leave a gap between two desirable but incompatible properties. Contract C gives an exact second-order deterministic transform, but the transform can be signed. Contract D gives a simple state-space covariance repair, but it can be ill-conditioned when the barycentric cloud has already collapsed in one or more directions. A natural intermediate construction is to split the job into three explicit stages:

$$D_B \longrightarrow D^+ \longrightarrow \widetilde{Y} \longrightarrow Y^\star.$$

Here $D^+$ is still a nonnegative first-order transform, $\widetilde{Y}$ adds a small reparameterized residual in the missing covariance directions, and $Y^\star$ applies the final affine moment restoration. This is not a drop-in form of the second-order ETPF of Acevedo et al. [2017], nor is it the nonlinear transport-map framework of Spantini et al. [2022]. It is a new finite reset contract assembled from familiar components: positive ensemble transforms [Reich, 2013, Corenflos et al., 2021], and nearby classes of second-order, stochastic, conditional, and learned ensemble transforms. The exact three-stage composition below was not found in the inspected sources, so the surrounding design-space remarks are intentionally narrower than theorem- or algorithm-level support. The citations in this paragraph are therefore used only where their source identity is already stable in the rewrite tree; unresolved or provisional neighboring records are not needed for the mathematical contract stated below and are omitted here. The citations support ingredients and nearby design space; they do not prove that this contract is stable, unbiased, or preferable for LEDH-PFPF-OT. Those are separate value, gradient, conditioning, and constraint gates.

The construction is easiest to state on the linear support of the weighted cloud. Write

$$\mathcal{S}_w := \operatorname{range}(\Sigma_w), \qquad P_w := \text{the orthogonal projector onto } \mathcal{S}_w.$$

Let $r_w := \operatorname{rank}(\Sigma_w)$. Since $\Sigma_w$ is the covariance of $N$ centered weighted particles, $r_w \le N-1$; if a modified construction asks for a rank larger than $N-1$, no equal-weight cloud of $N$ particles can realize it. All square roots below are principal square roots on their stated supports. If the rank of $\Sigma_w$, $G_+$, or $\widetilde{\Sigma}$ changes along a parameter path, the corresponding square-root, pseudoinverse, or support projector derivative may fail to be classical. A differentiable implementation therefore needs either a fixed-rank contract or an explicit spectral floor and smoothed spectral projector policy. This is a numerical assumption, not a consequence of optimal transport.

The first stage chooses

$$D^+ \in \mathcal{D}_1^+ := \{D \in \mathbb{R}_+^{N\times N} : D^\top \mathbf{1} = \mathbf{1},\ D\mathbf{1} = Nw\}. \tag{25.41}$$

The notation $D^+$ does not mean that the matrix is invertible. The relevant rank condition is a state-space condition on the cloud

$$Y^+ := XD^+, \qquad \Sigma_+ := \frac{1}{N}(Y^+ - \mu_w\mathbf{1}^\top)(Y^+ - \mu_w\mathbf{1}^\top)^\top.$$

The selection rule for $D^+$ may be a sharper Sinkhorn plan, a spread-penalized positive transport solve, or a positive projection of a partial second-order correction. The mathematical contract is weaker and more auditable than any particular optimizer: $D^+$ must remain in $\mathcal{D}_1^+$, must be close enough to the declared transport target for the application, and must pass a spectral gate on $\Sigma_+$. If the gate fails, this contract has not produced a usable reset.

**Proposition 25.3** (The residual covariance to be repaired is positive)**.** *Let* $D^+ \in \mathcal{D}_1^+$. *Define*

$$G_+ := \Sigma_w - \Sigma_+.$$

*Then*

$$G_+ = \frac{1}{N}\sum_{j=1}^{N}\sum_{i=1}^{N} D_{ij}^+ (x_i - y_j^+)(x_i - y_j^+)^\top \succeq 0. \tag{25.42}$$

*Equality holds if and only if each output column of* $D^+$ *has zero conditional state covariance, in the same sense as Proposition 25.2. Moreover* $\operatorname{range}(G_+) \subseteq \operatorname{range}(\Sigma_w)$.

*Proof.* The proof is the same total-covariance argument as Proposition 25.2, with $D_B$ replaced by $D^+$. Nonnegativity and the column-sum constraint make each column $D_{\cdot j}^+$ a conditional probability vector. The row-sum constraint gives the unconditional law $w$. Therefore the covariance of the weighted source draw decomposes into the covariance of the column barycentres plus the mean conditional covariance inside the columns. The first term is $\Sigma_+$, and the second is exactly the right-hand side of (25.42). This proves positive semidefiniteness. Since $0 \preceq G_+ \preceq \Sigma_w$, every vector in $\ker(\Sigma_w)$ is also in $\ker(G_+)$; for symmetric positive semidefinite matrices this is equivalent to $\operatorname{range}(G_+) \subseteq \operatorname{range}(\Sigma_w)$. □

The second stage adds a differentiable residual in the directions represented by $G_+$. Let $\Xi \in \mathbb{R}^{d_x \times N}$ be an auxiliary base-noise matrix, independent of the model parameters, satisfying

$$\mathbb{E}[\Xi] = 0, \qquad \Xi \mathbf{1} = 0, \qquad \mathbb{E}\left[\frac{1}{N}\Xi\Xi^\top\right] = I_{d_x}.$$

One concrete reparameterized choice is

$$H_N := I_N - \frac{1}{N}\mathbf{1}\mathbf{1}^\top, \qquad \Xi := \sqrt{\frac{N}{N-1}}\, Z H_N, \qquad Z_{ij} \overset{\text{iid}}{\sim} N(0,1), \qquad N \geq 2.$$

The matrix $Z$ is drawn independently of $X$ and $D^+$. Then $\Xi\mathbf{1} = 0$, and $\mathbb{E}[N^{-1}\Xi\Xi^\top] = I_{d_x}$ because $H_N$ is an idempotent projector with trace $N-1$. A fixed quasi-random or orthogonal design may be used instead, but then its empirical covariance is part of the finite contract and must be reported. For $0 \leq \rho \leq 1$ and $\tau \geq 0$, define

$$\widetilde{Y} := Y^+ + B_{\rho,\tau}\Xi, \qquad B_{\rho,\tau} := \sqrt{\rho}\,(G_+ + \tau P_w)^{1/2}. \tag{25.43}$$

The parameter $\rho$ controls how much of the gap is injected before the affine map. The parameter $\tau$ is not a proof device; it is a numerical floor on $\mathcal{S}_w$. Since $\operatorname{range}(G_+) \subseteq \mathcal{S}_w$, the residual covariance $G_+ + \tau P_w$ is positive semidefinite and has no component orthogonal to the weighted cloud support. If $\tau$ is set too small, the affine whitening in the next stage can amplify the auxiliary noise violently. Therefore $\tau$ and $\rho$ must be chosen by conditioning gates, not by the desire to make the perturbation invisible.

**Proposition 25.4** (Expected effect of the residual stage)**.** *Assume* $D^+ \in \mathcal{D}_1^+$, $\mathbb{E}[\Xi \mid X, D^+] = 0$, $\Xi\mathbf{1} = 0$, *and* $\mathbb{E}[N^{-1}\Xi\Xi^\top \mid X, D^+] = I_{d_x}$. *Conditional on* $X$ *and* $D^+$, *the residual cloud* $\widetilde{Y}$ *in* (25.43) *has mean* $\mu_w$, *and its covariance around* $\mu_w$ *satisfies*

$$\mathbb{E}\left[\frac{1}{N}(\widetilde{Y} - \mu_w\mathbf{1}^\top)(\widetilde{Y} - \mu_w\mathbf{1}^\top)^\top \,\middle|\, X, D^+\right] = \Sigma_+ + \rho(G_+ + \tau P_w). \tag{25.44}$$

*In particular, when* $\rho = 1$ *and* $\tau = 0$, *the conditional expected covariance is* $\Sigma_w$.

*Proof.* Because $D^+\mathbf{1} = Nw$, the equal-weight mean of $Y^+$ is $\mu_w$. Because $\Xi\mathbf{1} = 0$, the equal-weight mean of $B_{\rho,\tau}\Xi$ is zero. Expanding the covariance of $\widetilde{Y}$ around $\mu_w$ gives the covariance of $Y^+$, two cross terms, and the residual covariance. The cross terms have conditional expectation zero because the auxiliary noise has conditional mean zero given $X, D^+$. The residual covariance has conditional expectation

$$B_{\rho,\tau}\mathbb{E}\left[\frac{1}{N}\Xi\Xi^\top \,\middle|\, X, D^+\right] B_{\rho,\tau}^\top = B_{\rho,\tau}B_{\rho,\tau}^\top = \rho(G_+ + \tau P_w),$$

by the definition of the principal square root. This proves (25.44). □

The proposition is deliberately an expectation statement. For a realized finite noise matrix $\Xi$, the covariance of $\widetilde{Y}$ need not equal $\Sigma_w$ even when $\rho = 1$ and $\tau = 0$. This is not a defect in the algebra; it is the price of using a reparameterized stochastic residual rather than a discontinuous categorical resampling step. The next affine stage turns the realized cloud into an exact finite second-moment object only when the support condition in Proposition 25.6 holds and the exact Moore–Penrose inverse square root is used.

**Proposition 25.5** (A small residual can repair support, but not sample-rank limits)**.** *Assume $r_w \le N-1$, $0 < \rho \le 1$, and $\tau > 0$. Draw $Z$ with entries having a joint density, for example independent standard Gaussian entries, and set $\Xi = \sqrt{N/(N-1)}\, ZH_N$. Conditional on $X$ and $D^+$, the residual matrix $B_{\rho,\tau}\Xi$ has column span equal to $\mathcal{S}_w$ with probability one. Consequently*

$$\operatorname{range}(\Sigma_w) \subseteq \operatorname{range}(\widetilde{\Sigma})$$

*with probability one. If $r_w > N-1$, no choice of $N$ equal-weight output particles can realize covariance $\Sigma_w$, because every centered $d_x \times N$ cloud has rank at most $N-1$.*

*Proof.* Since $\tau > 0$,

$$G_+ + \tau P_w$$

is strictly positive definite on $\mathcal{S}_w$ and zero on $\mathcal{S}_w^\perp$. Hence $B_{\rho,\tau}$ has range $\mathcal{S}_w$ and rank $r_w$. Let $U \in \mathbb{R}^{d_x \times r_w}$ have orthonormal columns spanning $\mathcal{S}_w$. The centering matrix $H_N$ has rank $N-1$, and $U^\top ZH_N$ has rank $r_w$ almost surely when $r_w \le N-1$, because its $r_w \times (N-1)$ coordinate representation on $\operatorname{range}(H_N)$ has a joint density and at least one $r_w \times r_w$ minor is a nonzero polynomial in those variables. Multiplication by the nonsingular restriction of $B_{\rho,\tau}$ on $\mathcal{S}_w$ therefore produces a residual cloud whose span is exactly $\mathcal{S}_w$ almost surely.

The deterministic centered cloud $Y^+ - \mu_w\mathbf{1}^\top$ also lies in $\mathcal{S}_w$. The event that adding this fixed matrix to the full-rank random residual reduces the rank on $\mathcal{S}_w$ is again a polynomial zero event. To see that the relevant polynomial is not identically zero, choose any centered matrix $R_0$ with column span $\mathcal{S}_w$; then the same $r_w \times r_w$ minor of $(Y^+ - \mu_w\mathbf{1}^\top) + tR_0$ is nonzero for all sufficiently large $|t|$. Hence the zero set has probability zero under the joint-density assumption. Thus the realized centered cloud $\widetilde{Y} - \mu_w\mathbf{1}^\top$ has rank $r_w$ on $\mathcal{S}_w$ almost surely, which is the displayed support inclusion. The final statement follows because any centered cloud $Y - \bar{y}\mathbf{1}^\top$ has columns summing to zero and therefore rank at most $N-1$. □

The third stage removes the remaining finite-sample mismatch by affine restoration. Define the realized residual covariance

$$\widetilde{\Sigma} := \frac{1}{N}(\widetilde{Y} - \mu_w\mathbf{1}^\top)(\widetilde{Y} - \mu_w\mathbf{1}^\top)^\top.$$

If $\widetilde{\Sigma}$ is nonsingular on the support of $\Sigma_w$, set

$$A_{\rho,\tau} := \Sigma_w^{1/2}\,\widetilde{\Sigma}^{\dagger,-1/2}, \qquad Y^\star := \mu_w \mathbf{1}^\top + A_{\rho,\tau}(\widetilde{Y} - \mu_w \mathbf{1}^\top), \tag{25.45}$$

where $\widetilde{\Sigma}^{\dagger,-1/2}$ denotes the Moore–Penrose inverse of the principal square root.

**Proposition 25.6** (Affine restoration after the residual stage)**.** *Assume* $D^+ \in \mathcal{D}_1^+$, $\Xi\mathbf{1} = 0$, *and*

$$\operatorname{range}(\Sigma_w) \subseteq \operatorname{range}(\widetilde{\Sigma}).$$

*Then the output* $Y^\star$ *in* (25.45) *has equal-weight mean* $\mu_w$ *and equal-weight covariance* $\Sigma_w$.

*Proof.* The mean statement follows from $D^+\mathbf{1} = Nw$ and $\Xi\mathbf{1} = 0$. For the covariance, let $P_{\widetilde{\Sigma}}$ be the orthogonal projector onto $\operatorname{range}(\widetilde{\Sigma})$. By the definition of the Moore–Penrose inverse square root,

$$\widetilde{\Sigma}^{\dagger,-1/2}\widetilde{\Sigma}\widetilde{\Sigma}^{\dagger,-1/2} = P_{\widetilde{\Sigma}}.$$

The range assumption implies that this projector acts as the identity on $\operatorname{range}(\Sigma_w)$. Since the range of $\Sigma_w^{1/2}$ is $\operatorname{range}(\Sigma_w)$, this also implies $P_{\widetilde{\Sigma}}\Sigma_w^{1/2} = \Sigma_w^{1/2}$. By symmetry of $P_{\widetilde{\Sigma}}$ and $\Sigma_w^{1/2}$, it also implies $\Sigma_w^{1/2}P_{\widetilde{\Sigma}} = \Sigma_w^{1/2}$. Hence

$$\frac{1}{N}(Y^\star - \mu_w\mathbf{1}^\top)(Y^\star - \mu_w\mathbf{1}^\top)^\top = \Sigma_w^{1/2}P_{\widetilde{\Sigma}}\Sigma_w^{1/2} = \Sigma_w.$$

□

An implementation may replace $\widetilde{\Sigma}^{\dagger,-1/2}$ by a floored or otherwise regularized inverse square root for conditioning. That defines an approximate restoration policy, not Proposition 25.6. The approximation must report its spectral floor, covariance residual, and conditioning diagnostics separately from the exact algebra above.

Contract E is attractive because it does not ask the nonnegative transform to do the impossible. Positivity is used only to build a noncollapsed first-order cloud; residual noise supplies controlled missing directions; the affine map enforces the declared second moment. It is also dangerous for exactly the same reason. The final particles need not remain in the convex hull of the source particles after the residual and affine stages, and a small residual can be made large by $\widetilde{\Sigma}^{-1/2}$. The contract is therefore admissible only with explicit gates:

$$\operatorname{rank}_{\tau_{\mathrm{eig}}}(\widetilde{\Sigma}) = r_w \quad \text{or} \quad \|(I - P_{\widetilde{\Sigma},\tau_{\mathrm{eig}}})P_w\| \le \epsilon_{\mathrm{supp}}, \qquad \lambda_{\min}^{\tau}(\widetilde{\Sigma}) > \tau_{\mathrm{eig}}, \qquad \kappa^{\tau}(\widetilde{\Sigma}) < \kappa_{\max}, \qquad \frac{\|\Sigma_w - \Sigma_+\|}{\|\Sigma_w\|}$$

where $P_{\widetilde{\Sigma},\tau_{\mathrm{eig}}}$ denotes the numerical support projector obtained from the same eigenvalue threshold used by the rank gate. The spectral quantities $\lambda_{\min}^{\tau}(\widetilde{\Sigma})$ and $\kappa^{\tau}(\widetilde{\Sigma})$ are computed only on this retained $\tau_{\mathrm{eig}}$-support. Eigenvalues below the threshold are treated as discarded numerical null directions and must be charged to the support-projector gate above; they must not simultaneously veto the fallback through a full positive-spectrum condition. If the implementation claims the exact Moore–Penrose restoration rather than a floored numerical restoration, then the full positive spectrum must be used and the support-projector fallback is no longer sufficient. It must also record the auxiliary-noise law or fixed design, random seeds, Monte Carlo standard errors, same-scalar finite-difference checks, and constraint violations after the affine map. In SIR, SV, and other constrained models, the construction should normally be tested first in validated unconstrained or transformed coordinates.

**Contract E–Chol: ridged square-root gauge for differentiability.** The support-exact form above is mathematically natural but a poor default coordinate system for a manual gradient implementation. It asks the code to differentiate eigenvectors, support projectors, principal square roots, and Moore–Penrose inverse square roots. When eigenvalues repeat, eigenvectors are not unique. When an eigenvalue crosses the numerical floor, the retained rank changes. Thus an eigendecomposition implementation can be perfectly reasonable for a forward diagnostic while still being a bad coordinate chart for a gradient-bearing reset. This is the same engineering lesson as the square-root sigma-point discussion in Chapter 18: the covariance object may be smooth even when a particular spectral coordinate representation is not. Classical numerical linear algebra also treats Cholesky factorization as the stable coordinate chart for strictly positive definite covariances; see, for example, Higham [2002].

A practical differentiable variant is therefore to replace the support-exact eigensystem route by a strictly positive definite, ridged Cholesky gauge. This is a different numerical contract, not a proof that the eigensystem contract was unnecessary. For the canonical BayesFilter route, the realized scalar ridge $\lambda > 0$ is selected outside the candidate-dependent differentiated callable, stored with the prepared inputs, and bound into route identity. The same value is used for the value, gradient, finite-difference center, and every finite-difference endpoint. Candidate-dependent ridge escalation followed by a stopped derivative computes a different gradient and is wrong relative to the canonical total-gradient claim. A failed Cholesky factor or nonpositive factor diagonal is an invalid-chart veto, not a trigger for an undisclosed fallback or escalation.

The canonical route fixes $\rho = 1$. In row-vector storage, let the prepared, parameter-independent residual design be $\Xi \in \mathbb{R}^{N \times d_x}$, with $\mathbf{1}^\top \Xi = 0$. Its realized entries, not merely a seed distribution, are part of the finite target. For source rows $X_i$, normalized probabilities $w_i$, and row-quotient rows $Y_i^+$, define the population moments

$$\mu_w := \sum_i w_i X_i, \qquad \Sigma_w := \sum_i w_i (X_i - \mu_w)(X_i - \mu_w)^\top, \tag{25.46}$$

$$\bar{Y}^+ := N^{-1} \sum_i Y_i^+, \qquad \Sigma_+ := N^{-1} \sum_i (Y_i^+ - \bar{Y}^+)(Y_i^+ - \bar{Y}^+)^\top, \tag{25.47}$$

and $G_+ := \mathrm{sym}(\Sigma_w - \Sigma_+)$. Then define

$$B_\lambda := \mathrm{chol}(G_+ + \lambda I), \qquad \widetilde{Y} := Y^+ + \Xi B_\lambda^\top. \tag{25.48}$$

All source and equal-weight covariances in this route are population empirical covariances with denominator $N$, not $N - 1$. The factor $\sqrt{N/(N-1)}$, when used to construct $\Xi$ from centered Gaussian noise, standardizes that fixed design in expectation and does not change the covariance convention. Then form

$$\widetilde{\Sigma} := \frac{1}{N} (\widetilde{Y} - \mathbf{1}\bar{\widetilde{y}}^\top)^\top (\widetilde{Y} - \mathbf{1}\bar{\widetilde{y}}^\top), \qquad L_w := \mathrm{chol}(\Sigma_w + \lambda I), \qquad L_{\widetilde{\Sigma}} := \mathrm{chol}(\widetilde{\Sigma} + \lambda I).$$

The affine operator is not formed with an inverse explicitly. It is the triangular-solve operator

$$A_\lambda := L_w L_{\widetilde{\Sigma}}^{-1}, \qquad Y_\lambda^\star := \mathbf{1}\mu_w^\top + (\widetilde{Y} - \mathbf{1}\bar{\widetilde{y}}^\top) A_\lambda^\top. \tag{25.49}$$

The recentering in (25.49) is essential: it is what preserves the target mean independently of the ridge.

**Proposition 25.7** (Ridged Cholesky restoration)**.** *Assume the Cholesky factors in* (25.49) *exist. Then*

$$A_\lambda(\widetilde{\Sigma} + \lambda I)A_\lambda^\top = \Sigma_w + \lambda I,$$

*and $Y_\lambda^\star$ has equal-weight mean $\mu_w$. The covariance of $Y_\lambda^\star$ around $\mu_w$ satisfies*

$$\frac{1}{N}(Y_\lambda^\star - \mathbf{1}\mu_w^\top)^\top(Y_\lambda^\star - \mathbf{1}\mu_w^\top) - \Sigma_w = \lambda(I - A_\lambda A_\lambda^\top).$$

*Proof.* The mean statement follows immediately from the centering in (25.49). Since $\widetilde{\Sigma} + \lambda I = L\!\not{L}^\top$,

$$A_\lambda(\widetilde{\Sigma} + \lambda I)A_\lambda^\top = L_w L^{-1} L\!\not{L}^\top \not{L}^{-\top} L_w^\top = L_w L_w^\top = \Sigma_w + \lambda I.$$

Subtracting $A_\lambda(\lambda I)A_\lambda^\top$ from both sides gives the displayed unridged covariance residual. □

Contract E–Chol is the only reset semantics eligible for canonical BayesFilter value, total-gradient, score-admission, leaderboard, default, or HMC-facing status. This selection does not establish that the route has passed those scientific gates. Its advantage is that the derivative can be written through Cholesky factors and triangular solves, without eigenvector gauges or rank-switching projectors. Its cost is the explicit $\lambda$-bias above. Artifacts must report the prepared realized ridge, the exact ridged-identity residual, and the separate raw-covariance residual. The proposition proves the ridged identity; calling the raw covariance exactly restored when $\lambda > 0$ is wrong.

The positive finite transport feeding this reset must also be interpreted literally. If the fixed-iteration streaming solver emits a numerator and row mass

$$Q_i = \sum_j P_{ij} X_j, \qquad M_i = \sum_j P_{ij},$$

then the transported cloud is

$$Y_i^+ = Q_i / M_i. \tag{25.50}$$

There is no row-mass floor in the canonical target. Each $M_i$ must be finite and positive. Treating $Q_i$ as the cloud merely because the finite solver intends unit row mass changes both the value and its derivative.

At each filtering time the declared transition and finite LEDH flow are applied first, corrected log weights and the current observed-data likelihood increment are then computed, and only afterward is an active reset applied. The reset affects later increments through the carried cloud; it does not modify the already-computed current increment. Active branches carry $Y_\lambda^\star$ with uniform log weights. Inactive branches carry the flowed cloud with its normalized log weights. The reset mask, prepared residual design, ridge, Sinkhorn schedule, and chunk sizes are fixed parts of the finite program.

**Contract F: no reset or delayed reset.** If the effective sample size is acceptable, carry the weighted cloud forward and do not apply $D_B$. This avoids the deterministic covariance loss at that step. It does not solve weight degeneracy in long horizons or high dimensions, but it is a clean diagnostic: if a no-reset weighted route agrees with an exact LGSSM comparator while the barycentric route fails, the reset contract rather than the transition or likelihood code is implicated.

## 25.9 Why the current reset can accumulate bias

Contract E–Chol is a declared finite filtering program, not an identity for the exact filtering distribution. At time $t$, the flowed particles $x_{t,i}$ and previous weights $w_{t-1,i}$ first form

$$r_{t,i} = w_{t-1,i}\,\frac{p_\theta(x_{t,i} \mid x_{t-1,i})g_\theta(y_t \mid x_{t,i})}{q_\theta(x_{t,i} \mid x_{t-1,i})}, \qquad \widehat{\ell}_t^N = \log \sum_i r_{t,i}, \tag{25.51}$$

where the implemented flow Jacobian is included in the ratio. Only after this increment has been accumulated does an active reset replace the cloud. The reset therefore cannot change the already computed increment, but it can change all later increments through the carried particles.

There are four separate approximation layers: finite-particle log normalization; finite entropic Sinkhorn iteration; barycentric projection; and the residual/ affine reset. Marginal residuals certify only the finite coupling calculation. They do not remove the barycentric covariance contraction, the finite log-sum bias, or the loss of higher-order shape information. Repeated resets can feed these local errors into later nonlinear transition and observation functionals. The LGSSM is therefore a controlled diagnostic for this mechanism, not the scientific endpoint for which the method is intended.

## 25.10 A non-fused cubature residual Contract E candidate

This candidate retains the original staged route. It does not solve a joint OT–moment optimization. The coupling, barycentric quotient, residual stage, and Cholesky restoration remain separate finite maps; only the prepared residual design is changed.

Let $d, M \in \mathbb{N}$, let $N = 2dM$, and use the population-covariance convention (denominator $N$) throughout this section. The reset is defined on a fixed finite-arithmetic branch: the row ordering, the branch of every Cholesky factor, the ridge $\lambda > 0$, and all Sinkhorn controls are fixed and independent of the model parameter for the derivative statement below. The particle-count condition is therefore

$$N = 2dM. \tag{25.52}$$

Define the standardized spherical–radial cubature directions

$$\xi_{a,+} = \sqrt{d}\,e_a, \qquad \xi_{a,-} = -\sqrt{d}\,e_a, \qquad a = 1, \dots, d, \tag{25.53}$$

and $D_c = \sqrt{d}\begin{bmatrix} I_d \\ -I_d \end{bmatrix}$. Let $\Xi_c \in \mathbb{R}^{N\times d}$ repeat every row of $D_c$ exactly $M$ times. The row order and any fixed independent permutation are part of the finite route identity.

**Proposition 25.8** (Cubature design moments)**.** *For $\mathbf{1}_N \in \mathbb{R}^N$, the replicated design $\Xi_c \in \mathbb{R}^{N\times d}$ satisfies*

$$\frac{1}{N}\mathbf{1}^\top \Xi_c = 0, \qquad \frac{1}{N}\Xi_c^\top \Xi_c = I_d. \tag{25.54}$$

*Proof.* Opposite directions cancel in every coordinate, so the row sum is zero. Also $D_c^\top D_c = d(I_d + I_d) = 2dI_d$. Repeating the block $M$ times and using $N = 2dM$ gives

$$N^{-1}\Xi_c^\top \Xi_c = (2dM)^{-1} M(2dI_d) = I_d.$$

□

**Proposition 25.9** (Affine cubature moment preservation)**.** *Let $a_r = 1/N$ and let $\xi_r$ be the rows of $\Xi_c$. For any $m \in \mathbb{R}^d$, $L \in \mathbb{R}^{d\times d}$, $F \in \mathbb{R}^{p\times d}$, and $b \in \mathbb{R}^p$, the cubature rule obeys*

$$\sum_{r=1}^{N} a_r\{F(m + L\xi_r) + b\} = Fm + b, \tag{25.55}$$

$$\sum_{r=1}^{N} a_r(FL\xi_r)(FL\xi_r)^\top = FL\,L^\top F^\top. \tag{25.56}$$

*Proof.* The first identity is $Fm + b + FL(N^{-1}\Xi_c^\top \mathbf{1}_N)$, which equals $Fm + b$ by Proposition 25.8. For the second identity, factor out $FL$ and $L^\top F^\top$, then use $N^{-1}\Xi_c^\top \Xi_c = I_d$. Thus the rule is exact for the first two moments of every affine image of a standardized Gaussian moment pair. This standalone statement does not imply that the particle log-sum likelihood is unbiased. □

The OT stage is unchanged. For flowed source particles $x_j$ and normalized probabilities $\alpha_j$, write the equal-target coupling as $\Pi$, with source marginal $\alpha$ and target marginal $u_i = 1/N$. The implementation uses $\Gamma = N\Pi$, so

$$\Gamma\mathbf{1} = \mathbf{1}, \qquad \Gamma^\top\mathbf{1} = N\alpha, \tag{25.57}$$

up to recorded finite-Sinkhorn residuals, and the barycentric output is

$$Y_i^+ = \sum_j \Gamma_{ij} x_j. \tag{25.58}$$

Cubature is not an OT target in this candidate; it enters only after (25.58).

Define

$$\mu_w = \sum_j \alpha_j x_j, \qquad \Sigma_w = \sum_j \alpha_j (x_j - \mu_w)(x_j - \mu_w)^\top, \tag{25.59}$$

$$\bar{Y}^+ = N^{-1}\sum_i Y_i^+, \qquad \Sigma_+ = N^{-1}\sum_i (Y_i^+ - \bar{Y}^+)(Y_i^+ - \bar{Y}^+)^\top. \tag{25.60}$$

Set $G = \mathrm{sym}(\Sigma_w - \Sigma_+)$ and $B = \mathrm{chol}(G + \lambda I)$, $\lambda > 0$, then inject

$$\widetilde{Y} = Y^+ + \Xi_c B^\top. \tag{25.61}$$

Let $\widetilde{\Sigma}$ be its population covariance and define

$$L_w = \mathrm{chol}(\Sigma_w + \lambda I), \quad L_{\widetilde{Y}} = \mathrm{chol}(\widetilde{\Sigma} + \lambda I), \quad A_c = L_w L_{\widetilde{Y}}^{-1}, \tag{25.62}$$

where the inverse denotes a triangular solve. The reset is

$$X_i^{E,c} = \mu_w + (\widetilde{Y}_i - \bar{\widetilde{Y}})A_c^\top. \tag{25.63}$$

**Proposition 25.10** (Mean and ridged covariance)**.** *Assume $G + \lambda I_d$, $\Sigma_w + \lambda I_d$, and $\widetilde{\Sigma} + \lambda I_d$ are strictly positive definite, so that all three displayed Cholesky factors exist. The equal-weight output $X^{E,c} \in \mathbb{R}^{N\times d}$ has mean $\mu_w$ and satisfies*

$$A_c(\widetilde{\Sigma} + \lambda I)A_c^\top = \Sigma_w + \lambda I, \qquad \Sigma_{E,c} - \Sigma_w = \lambda(I - A_c A_c^\top). \tag{25.64}$$

*Proof.* Because $\bar{\widetilde{Y}} = N^{-1}\sum_i \widetilde{Y}_i$, the centered output rows sum to zero, and its population covariance is $\Sigma_{E,c} = A_c\widetilde{\Sigma}A_c^\top$. With $L_w L_w^\top = \Sigma_w + \lambda I_d$ and $L_{\widetilde{Y}}L_{\widetilde{Y}}^\top = \widetilde{\Sigma} + \lambda I_d$,

$$\begin{aligned} A_c(\widetilde{\Sigma} + \lambda I_d)A_c^\top &= L_w L_{\widetilde{Y}}^{-1} L_{\widetilde{Y}} L_{\widetilde{Y}}^\top L_{\widetilde{Y}}^{-\top} L_w^\top \\ &= L_w L_w^\top = \Sigma_w + \lambda I_d. \end{aligned}$$

Subtracting $\lambda A_c A_c^\top$ from both sides gives $\Sigma_{E,c} - \Sigma_w = \lambda(I_d - A_c A_c^\top)$. □

The proposition concerns the finite weighted empirical target $\Sigma_w$, not the exact filtering covariance. To make the role of the second stage explicit, write $Y_c^+ = Y^+ - \bar{Y}^+$ and $C_{+c} = N^{-1}(Y_c^+)^\top \Xi_c$. Proposition 25.8 gives

$$\widetilde{\Sigma} = \Sigma_+ + C_{+c}B^\top + BC_{+c}^\top + BB^\top. \tag{25.65}$$

Thus $B = \mathrm{chol}(G + \lambda I_d)$ is not by itself a covariance correction unless $C_{+c} = 0$. The subsequent $A_c$ uses the realized $\widetilde{\Sigma}$, including both cross terms, and is the operation that establishes the ridged identity.

**Proposition 25.11** (Current likelihood and total-score boundary)**.** *If the cubature residual reset is applied after the current increment* (25.51)*, that increment is exactly unchanged relative to canonical Contract E. On a fixed finite-arithmetic branch, with parameter-independent $\lambda$ and $\Xi_c$, differentiating every dependency through the staged map gives the total derivative of the new cubature-residual finite likelihood scalar.*

*Proof.* The current increment depends only on pre-reset particles, weights, densities, and the flow Jacobian. The full scalar is a composition through $(x, \alpha, \Pi, Y^+, \mu_w, \Sigma_w, \widetilde{\Sigma}, A_c, X^{E,c})$. Since $d\Xi_c = d\lambda = 0$, the ordinary chain rule through the fixed SPD Cholesky branches gives the total derivative of every later increment. This proves a same-scalar derivative statement for the new finite program, not an exact-posterior score statement. □

For affine maps $f(x) = Fx+b$ and $h(x) = Hx+c$, the preceding affine-moment proposition gives exact transformed means and covariances for the standalone cubature rule. A cubature Gaussian filter therefore agrees with the corresponding Kalman moment recursion up to arithmetic error when its input is represented by that rule. The staged particle route still has finite-particle, finite-Sinkhorn, barycentric, and reset approximations; this does not make its particle log-sum estimator unbiased for the Kalman likelihood.

## 25.11 Generalized unscented transformation candidate

Ebeigbe et al. Ebeigbe et al. [2025] propose a $2d+1$ point generalized unscented transform (GenUT). In the source paper, the sigma points match the mean and covariance exactly and then use the remaining axis freedom to match the *physical-coordinate diagonal* third and fourth central moments, constructed through elementwise powers of a chosen covariance square root. The construction used below keeps the same per-axis algebra but applies it to whitened moments of $Z = C^{-1}(X-m)$. For correlated states those whitened diagonal moments are generally different targets from the paper's physical diagonal moments, so this section should be read as a GenUT-motivated local variant rather than as a restatement of the paper's multivariate construction.

Let $m \in \mathbb{R}^d$, $C \in \mathbb{R}^{d\times d}$ be nonsingular with $CC^\top = P \succ 0$, and let $Z = C^{-1}(X - m)$. Define the coordinate moments

$$s_a = \mathbb{E}[Z_a^3], \qquad k_a = \mathbb{E}[Z_a^4]. \tag{25.66}$$

For each axis use standardized points $0, -u_a e_a, +v_a e_a$, with weights $w_0, b_a, c_a$. The axis constraints are

$$-b_a u_a + c_a v_a = 0, \qquad b_a u_a^2 + c_a v_a^2 = 1, \tag{25.67}$$
$$-b_a u_a^3 + c_a v_a^3 = s_a, \qquad b_a u_a^4 + c_a v_a^4 = k_a. \tag{25.68}$$

**Proposition 25.12** (GenUT axis construction)**.** *Fix an axis $a$. If $k_a > s_a^2$, set*

$$u_a = \frac{-s_a + \sqrt{4k_a - 3s_a^2}}{2}, \qquad v_a = u_a + s_a, \qquad c_a = \frac{1}{v_a(u_a + v_a)}, \qquad b_a = \frac{1}{u_a(u_a + v_a)}. \tag{25.69}$$

*Then the four axis constraints hold. Setting $w_0 = 1 - \sum_a (b_a + c_a)$ normalizes the rule, but $w_0$ is not guaranteed to be nonnegative.*

*Proof.* The first constraint gives $b_a = c_a v_a / u_a$. Substitution into the second gives $c_a v_a(u_a + v_a) = 1$. The third therefore reduces to $v_a - u_a = s_a$. Substitution into the fourth yields $u_a^2 + s_a u_a + s_a^2 - k_a = 0$, whose positive root is the displayed $u_a$. The expression for $v_a$ and the weights then follow from the first two constraints. Since $\sqrt{4k_a - 3s_a^2} > |s_a|$, both $u_a$ and $v_a$ are strictly positive, and therefore $b_a, c_a > 0$. The central weight supplies the remaining normalization. The condition $k_a > s_a^2$ ensures a real, positive axis construction, but it says nothing about $w_0$. □

**Proposition 25.13** (GenUT moment identities)**.** *Define $2d + 1$ points $z_0 = 0$, $z_{a,-} = -u_a e_a$, and $z_{a,+} = v_a e_a$, with weights $w_0, b_a, c_a$, and assume $w_0 = 1 - \sum_a (b_a + c_a)$ is finite. Then the GenUT rule has zero mean, identity covariance, diagonal third moments $s_a$, and diagonal fourth moments $k_a$ in whitened coordinates. The physical points $x_r = m + Cz_r$ therefore have mean $m$ and covariance $P$.*

*Proof.* Normalization is the definition of $w_0$. For the mean, each axis contributes $(-b_a u_a + c_a v_a)e_a = 0$. For the covariance, off-diagonal entries vanish because every noncentral point has only one nonzero coordinate, while the $a$-th diagonal entry is $b_a u_a^2 + c_a v_a^2 = 1$. The diagonal third and fourth entries are exactly the last two axis equations; mixed third/fourth entries involving two distinct coordinates vanish for this rule. Finally, $\sum_r w_r(m + Cz_r) = m + C\sum_r w_r z_r = m$, and $\sum_r w_r(x_r - m)(x_r - m)^\top = C(\sum_r w_r z_r z_r^\top)C^\top = CC^\top = P$. □

**Proposition 25.14** (GenUT positivity and representation boundary)**.** *Under $k_a > s_a^2$, the noncentral weights $b_a, c_a$ are positive. The full GenUT rule is a positive probability rule if and only if $w_0 \geq 0$. It can be represented exactly as an equal-weight $N$-row design by replication only when all its positive weights are rational with a common denominator dividing $N$ (or when an explicitly declared positive approximation is used). A negative $w_0$ is a signed quadrature rule and cannot be used as a positive entropic-OT marginal.*

*Proof.* The positivity of $b_a, c_a$ follows from the strict positivity of $u_a, v_a, u_a + v_a$. Since the weights sum to one, nonnegativity of all weights is equivalent to $w_0 \geq 0$. Exact equal-weight replication assigns each row mass $1/N$, so a weight $q$ is representable by rows precisely when $Nq$ is a nonnegative integer; a common denominator gives the stated condition. Signed masses violate the positive-marginal assumptions of entropic OT. □

For a feasible positive GenUT residual design $\Xi_g$, obtained by mapping the whitened points $z_r$ through the declared square root and then using an exact positive equal-weight replication when the condition in Proposition 25.14 holds, define

$$\widetilde{Y}_g = Y^+ + \Xi_g B_g^\top, \qquad B_g = \mathrm{chol}(G_g + \lambda I), \tag{25.70}$$

where the displayed Cholesky argument is assumed strictly positive definite. Apply the same Cholesky restoration as in (25.62). This is the GenUT residual candidate. Its current likelihood increment is unchanged because the reset is post-increment. The variance statement is the same ridged identity as in Proposition 25.10, with $c$ replaced by $g$, provided the realized $\widetilde{\Sigma}_g$ is used. Its total score is the total derivative of the new finite route when the empirical third/fourth moments, square-root gauge, point coordinates, weights, and restoration are all differentiated. Neither future likelihood equality nor exact posterior variance/score equality follows.

**Proposition 25.15** (Value, variance, and score for both staged residuals)**.** *Let $q \in \{c, g\}$ denote either the cubature residual or a feasible positive GenUT residual, and let $R_q$ be its post-increment reset. Assume all Cholesky arguments are strictly positive definite and the parameter is inside a fixed differentiable branch. Then:*

*(i) the current finite likelihood increment $\widehat{\ell}_t^N$ in (25.51) is identical before and after replacing the canonical reset by $R_q$;*

*(ii) the reset mean is $\mu_w$, and its population covariance obeys*

$$\Sigma_{R_q} + \lambda A_q A_q^\top = \Sigma_w + \lambda I_d,$$

*so the un-ridged covariance equals $\Sigma_w$ only in the special case $A_q A_q^\top = I_d$ (or in the zero-ridge limit when that limit exists);*

*(iii) differentiating the complete staged computation gives the total derivative of the complete finite likelihood scalar generated by $R_q$, including the derivatives of the OT map, moment maps, residual design, Cholesky factors, and all later recursion.*

*The proposition does not assert equality with the exact filtering likelihood, exact filtering variance, or exact filtering score. Except in a special insensitivity case, the full future likelihood scalar for $R_c$ or $R_g$ is different from canonical Contract E because the carried cloud differs.*

*Proof.* The reset is evaluated after $\widehat{\ell}_t^N$, so item (i) is a direct statement about program order. Item (ii) is the proof of Proposition 25.10; the same algebra applies to any $q$ whose realized centered design and affine restoration are used. For item (iii), on a fixed branch the finite filter is a composition of differentiable maps. The chain rule therefore gives the derivative of the same scalar. A later increment is a function of the carried reset cloud, so there is no reason for it to agree with the canonical value unless the later functional is insensitive to the changed directions. This is a finite-program statement, not an exact filtering theorem. □

| Property | Cubature residual | Feasible positive GenUT residual | Exactness boundary |
|---|---|---|---|
| Current likelihood increment | unchanged | unchanged | reset is post-increment; this is a program-order identity |
| Reset mean | $\mu_w$ | $\mu_w$ | finite empirical target only |
| Reset variance | same ridged identity | same ridged identity | exact $\Sigma_w$ only without the ridge residual |
| Higher moments | symmetric second-order design | selected whitened diagonal $s_a, k_a$ when feasible | neither matches the full correlated filtering law |
| Full future likelihood | generally changed | generally changed | carried cloud changes downstream increments |
| Score | total derivative of cubature finite scalar | total derivative of GenUT finite scalar | not an exact-posterior score claim |

The constrained GenUT variant in the paper moves points toward support bounds. That operation preserves second-order constraints only under its stated re-solving rules and generally sacrifices exact fourth-moment matching. A parameter-dependent constraint slack is therefore a new route control and must be included in the route identity and derivative graph.

| Remedy | What it fixes | Main cost | Required audit |
|---|---|---|---|
| Same-scalar barycentric contract | claim mismatch for the executed scalar | does not restore covariance | same-scalar finite differences and moment-drift report |
| Stochastic resampling | unbiased empirical-measure semantics in expectation | discontinuous random branch | estimator contract, seed/randomness policy, MCSE |
| Second-order transform | mean and covariance matching | possible negative transform entries and nonphysical states | moment gate, constraint gate, custom VJP gate |
| Affine moment restoration | local covariance restoration diagnostic | new finite target and rank/jitter choices | LGSSM value/gradient gate and state-constraint audit |
| Positive residual-affine hybrid | covariance restoration with positive first stage | novel randomized finite target and conditioning sensitivity | spectrum gate, MCSE, same-scalar gradient gate, constraint audit |
| No-reset threshold | avoids unnecessary deterministic collapse | leaves weight degeneracy unresolved | ESS policy and weighted-cloud comparator |

**Implication for SIR, SV, and learned teachers.** Yes, this issue can affect SIR, stochastic-volatility, and other nonlinear-model gradients. The effect is harder to expose than in an LGSSM because there is no closed-form Kalman likelihood oracle, but the mechanism is the same: a mean-preserving reset can still change nonlinear transition, observation, and future-likelihood functionals. Formally, let $\Phi_t$ denote the rest of the filtering computation after a reset at time $t$. The barycentric route and a moment-preserving route evaluate different functions,

$$L_B(\theta) = \Phi_t(X_t(\theta)D_B(\theta);\theta)\,, \qquad L_\sharp(\theta) = \Phi_t\big(X_t(\theta)D^\sharp(\theta);\theta\big)\,,$$

unless $\Phi_t$ is insensitive to the covariance directions removed by (25.33). Therefore equality of autodiff and finite differences for $L_B$ certifies only the executed barycentric route. It does not certify $L_\sharp$, a categorical-resampling route, or an exact filtering likelihood. A learned teacher trained to imitate the current barycentric teacher $D_B$ will inherit this moment contract rather than repair it. A learned or retained teacher can be a remedy only if its target is changed to one of the declared objects above, for example $D^\sharp$, $D_{\mathrm{NETF}}$, the stochastic estimator, or the affine-corrected cloud $Y^\star$, and the downstream filtering diagnostics are rerun under that new finite target.

## 25.12 Differentiation contracts

The final major distinction of the chapter is between three different derivative objects:

(i) derivative of the exact regularized OT optimizer,

(ii) derivative of the finite unrolled Sinkhorn computation,

(iii) derivative of some scalar built from that finite computation.

In code, the second and third are the most common. A gradient is only the correct gradient of the exact executed scalar if the computational graph used in autodiff matches the scalar value actually reported by the code. The next chapter specializes this point to the declared LEDH-PFPF-OT scalar and its custom VJP.

**Proposition 25.16** (Stopped transport normalization gives a partial derivative)**.** *Let a finite Sinkhorn transport step use normalized particles*

$$u_i(x) = \frac{x_i - \bar{x}}{s(x)}, \qquad \bar{x} = \frac{1}{N}\sum_{k=1}^{N} x_k,$$

*where $s(x)$ is the scale used to build the transport cost. Let*

$$L(x) = \Psi\big(x, u(x)\big)$$

*be any downstream scalar depending on both the carried particles $x$ and the transport-normalized particles $u(x)$. If a reverse pass treats $\bar{x}$ and $s(x)$ as constants, then it computes the partial derivative of $\Psi(x, u)$ with respect to $x$ under the artificial map*

$$u_i = \frac{x_i - c}{s},$$

*not the total derivative of $L(x)$. Therefore, unless the omitted terms are zero or a different objective is explicitly declared, this stopped derivative is wrong relative to the claim that it is the score of the executed finite scalar.*

*Proof.* Write $a_i = \partial L/\partial u_i$. The true differential through

$$u_i(x) = \frac{x_i - \bar{x}}{s(x)}$$

is

$$dL_u = \sum_i a_i^\top \frac{dx_i - d\bar{x}}{s(x)} - \left(\sum_i a_i^\top (x_i - \bar{x})\right) \frac{ds(x)}{s(x)^2}.$$

Since $d\bar{x} = N^{-1}\sum_k dx_k$, the contribution to the adjoint of $x_i$ is

$$\frac{a_i}{s(x)} - \frac{1}{N}\sum_k \frac{a_k}{s(x)} - \frac{1}{s(x)^2}\left(\sum_k a_k^\top (x_k - \bar{x})\right) \nabla_{x_i} s(x).$$

The stopped rule keeps only $a_i/s(x)$. It drops the mean term and the scale term. Hence it is the derivative of an artificial partial-derivative problem with fixed $c$ and fixed $s$, not the total derivative of $L(x)$. □

The same warning applies to Sinkhorn cost keys, adaptive starting $\varepsilon_0$, finite-iteration potentials, and any other quantity computed from the current particles. If the scalar value recomputes such a quantity when $\theta$ changes, then its score includes the corresponding chain-rule term. A hand-written VJP may use internal stops only to avoid double counting, but it must add back every blocked derivative. Otherwise the result is a partial derivative and must be named as such.

| Derivative object | What is being differentiated? | What can go wrong? |
|---|---|---|
| Exact regularized optimizer | the ideal entropic OT solution | not what finite code usually executes directly |
| Finite unrolled Sinkhorn route | the declared numerical balancing procedure | residuals or stabilization may change the executed object |
| Downstream scalar built from finite Sinkhorn | the actual reported value used in filtering or HMC-adjacent code | value-gradient mismatch if autodiff follows a different graph |

## 25.13 Filtering-loop insertion point

Inside a filter, the finite EOT/Sinkhorn equal-weighting step sits after the weighted particle cloud has been formed and before an equally weighted cloud is passed onward:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{EOT/Sinkhorn}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \tag{25.71}$$

If the broader filtering algorithm carries auxiliary particle-local state, this chapter does not yet assert how that state is transported. Its contract is narrower: it specifies the equal-weighting rule for the particle locations and the mass-routing matrix from which that equal-weight cloud is derived.

## 25.14 Verification contract

A reader implementing this route should verify at least:

(1) row and column residuals are reported and decrease with additional iterations or stronger stabilization,

(2) the initialization, $\varepsilon$, iteration budget, and stabilization policy are recorded,

(3) the barycentric cloud has the declared shape and normalization,

(4) any reported gradient is tied to the exact computational graph that was actually executed,

(5) any stronger filtering or HMC claim is kept separate from these local numerical checks.

A failed residual check is evidence about the finite solver or stabilization path, not about the abstract EOT object. A successful residual check is evidence that the numerical route is behaving as declared, not that it preserves the original categorical target.

## 25.15 What this chapter does and does not claim

This chapter teaches the main differentiable OT computational route in a near- implementation style. The derivation above fixes a specific ladder:

(i) deterministic OT chooses an exact coupling for the unregularized baseline,

(ii) entropic OT changes that teacher object,

(iii) Sinkhorn balancing computes a finite numerical approximation to the entropic teacher,

(iv) barycentric projection turns that finite coupling into the equal-weight cloud consumed by later filtering code.

Within that ladder, exact-online and streamed execution strategies remain on the reference lane when they preserve the same scaling equations and the same barycentric output contract. Later retained-teacher chapters ask whether the same teacher can be compressed or approximated while still judged against this reference object. Still later changed-object chapters ask a different question again: what happens when the transport object itself is replaced by a learned map, altered support, or new target contract.

It does *not* claim:

- that entropic OT is the same object as unregularized OT,
- that finite Sinkhorn is the exact regularized optimizer,
- that a barycentric equal-weight cloud is the same object as categorical ancestor sampling,
- that autodiff through the executed graph is automatically the gradient of the original filtering likelihood.

Its role is to fix the entropic teacher object, the finite numerical route, the output cloud construction, and the differentiation contract that the recovered custom-gradient bridge and later learned-transport chapters build on.

## 25.16 A complete higher-moment Contract E candidate

This section defines the complete opt-in algorithm that is implemented and tested below. It is important to name the object correctly. A classical unscented transform replaces the distribution of an input $x$ by sigma points and evaluates the nonlinear map $f$ at those sigma points. The algorithm here instead propagates the particle rows through $f$, evaluates their likelihoods, applies entropic OT and Contract E, and then applies a finite higher-moment correction to the carried cloud. GenUT supplies the motivation and the axis algebra, but this is not the classical GenUT quadrature algorithm, and the whitened diagonal targets used here are not the same multivariate moment targets stated in the source paper. The existing Contract E route remains unchanged when correction is disabled. The optional protected no-fit comparator below additionally applies its declared cap and affine restoration.

### 25.16.1 Finite filtering, OT, and Contract E stages

Let $x^+_{t-1,i} \in \mathbb{R}^d$, $i = 1, \dots, N$, be the equally weighted cloud carried from time $t-1$. The innovation rows $e_{t,i}$, their ordering, and all numerical controls are fixed as part of the finite route. The transition and likelihood stages are

$$x^-_{t,i} = f_\theta(x^+_{t-1,i}, e_{t,i}), \tag{25.72}$$

$$\ell_{t,i} = \log g_\theta(y_t \mid x^-_{t,i}), \qquad \widehat{\ell}^N_t = \log\left(\frac{1}{N}\sum_{i=1}^N e^{\ell_{t,i}}\right), \tag{25.73}$$

$$\alpha_{t,i} = \frac{e^{\ell_{t,i}}}{\sum_{j=1}^N e^{\ell_{t,j}}}. \tag{25.74}$$

The weighted empirical target, its mean, and its population covariance are

$$\widehat{\pi}^w_t = \sum_{i=1}^N \alpha_{t,i}\delta_{x^-_{t,i}}, \qquad \mu_w = \sum_{i=1}^N \alpha_{t,i}x^-_{t,i}, \tag{25.75}$$

$$P_w = \sum_{i=1}^N \alpha_{t,i}(x^-_{t,i} - \mu_w)(x^-_{t,i} - \mu_w)^\top. \tag{25.76}$$

For completeness, the implemented finite Sinkhorn stage uses the cost $C_{ij} = \|x^-_{t,i} - x^-_{t,j}\|_2^2$, a positive kernel $K_{ij} = \exp\{-C_{ij}/(\epsilon\bar{C})\}$, where $\bar{C} = \max(N^{-2}\sum_{ij} C_{ij}, 10^{-3})$, and fixed iteration counts. Writing $u_i = 1/N$, each iteration applies

$$a \leftarrow u \oslash (Kb + \tau\mathbf{1}), \qquad b \leftarrow \alpha \oslash (K^\top a + \tau\mathbf{1}), \qquad \tau = 10^{-7}, \tag{25.77}$$

and returns the finite nonnegative coupling $\Pi = \operatorname{diag}(a)K\operatorname{diag}(b)$. Because a finite iteration need not satisfy either marginal exactly, the carried barycenter is the row quotient

$$m_i = \sum_j \Pi_{ij}, \qquad y^+_i = \frac{\sum_j \Pi_{ij}x^-_{t,j}}{m_i}. \tag{25.78}$$

The route fails closed unless every $m_i > \tau$ and the source marginal of the quotient coupling $\overline{\Pi}_{ij} = u_i\Pi_{ij}/m_i$ satisfies its declared tolerance. Replacing the quotient by $N\sum_j \Pi_{ij}x_j$ is valid only when the finite row marginal is already exactly $u$.

Let $\bar{y} = N^{-1}\sum_i y^+_i$ and $P_+ = N^{-1}\sum_i (y^+_i - \bar{y})(y^+_i - \bar{y})^\top$. Contract E uses a fixed zero-mean residual design $\Xi \in \mathbb{R}^{N\times d}$ and

$$G = \operatorname{sym}(P_w - P_+), \qquad B = \operatorname{chol}(G + \lambda I_d), \tag{25.79}$$

$$\widetilde{y}_i = y^+_i + \Xi_i B^\top, \qquad \overline{\widetilde{y}} = N^{-1}\sum_i \widetilde{y}_i. \tag{25.80}$$

If $P_\Xi = N^{-1}\Xi^\top\Xi$ and $C_{y\Xi} = N^{-1}\sum_i (y^+_i - \bar{y})\Xi_i^\top$, then its realized population covariance is

$$\widetilde{P} = P_+ + C_{y\Xi}B^\top + BC_{y\Xi}^\top + BP_\Xi B^\top. \tag{25.81}$$

Thus $BB^\top$ alone is not the injected covariance when the cross term is nonzero. The second Contract E stage uses the realized $\widetilde{P}$:

$$L_wL_w^\top = P_w + \lambda I_d, \qquad L_EL_E^\top = \widetilde{P} + \lambda I_d, \tag{25.82}$$

$$A = L_wL_E^{-1}, \qquad x^E_i = \mu_w + (\widetilde{y}_i - \overline{\widetilde{y}})A^\top. \tag{25.83}$$

Every inverse in this section denotes a triangular solve.

**Proposition 25.17** (Contract E mean and ridged covariance)**.** *Suppose $G+\lambda I_d$, $P_w+\lambda I_d$, and $\widetilde{P}+\lambda I_d$ are strictly positive definite. Under the population-covariance convention, the cloud in (25.83) has mean $\mu_w$ and covariance $P_E$ satisfying*

$$P_E + \lambda AA^\top = P_w + \lambda I_d, \qquad P_E - P_w = \lambda(I_d - AA^\top). \tag{25.84}$$

*Proof.* The centered rows in (25.83) sum to zero, so their affine image has mean $\mu_w$. Its covariance is $P_E = A\widetilde{P}A^\top$. The Cholesky definitions give

$$A(\widetilde{P}+\lambda I_d)A^\top = L_w L_E^{-1} L_E L_E^\top L_E^{-\top} L_w^\top = L_w L_w^\top = P_w + \lambda I_d.$$

Subtraction of $\lambda AA^\top$ proves both identities. In particular, the first stage alone does not prove covariance restoration, and positive ridge does not generally give the un-ridged equality $P_E = P_w$. □

**Proposition 25.18** (Current-increment value boundary)**.** *Any reset applied after (25.73) leaves the current increment $\widehat{\ell}_t^N$ exactly unchanged. It generally changes later increments and therefore generally changes the complete finite filtering value.*

*Proof.* The current increment is a function only of the pre-reset rows $x_{t,i}^-$ and their likelihoods. A reset changes the rows supplied to the next transition. Unless all later transition and likelihood functionals are insensitive to that change, a later increment changes. This is a program-order identity for the finite estimator, not equality with the exact filtering likelihood. □

### 25.16.2 Finite higher-moment correction

The construction below is deliberately narrower than two established higher-order sigma-point rules. Ebeigbe et al.'s GenUT uses $2d+1$ weighted points to match the mean, covariance, and selected diagonal third and fourth moments when its moment and positivity conditions hold [Ebeigbe et al., 2025, Secs. III–V, Algorithms 1–2]. Under state constraints, their Algorithm 2 preserves the lower-order construction but explicitly accepts that exact diagonal kurtosis matching can be lost. The higher-order unscented transform (HOUT) of Easley and Berry [2021, Definition 4.1, Theorem 4.2, Algorithm 4.1] instead decomposes the full third- and fourth-order tensors into rank-one terms and builds a variable-size signed-weight quadrature rule. It has controlled moment error, but Remark 4.4 shows that its absolute quadrature condition number can grow like the reciprocal requested tolerance. Neither method is a drop-in positive equal-weight map of the existing $N$-particle Contract E cloud. We therefore retain the weighted source moments as a least-squares target and borrow the established principles that constraints can preclude exact higher-moment matching and that the numerical correction must be damped. The exact finite composition below is a BayesFilter candidate, not a theorem claimed by either source.

Assume now that the un-ridged weighted covariance is strictly positive definite and write $C_w C_w^\top = P_w$. With row-vector notation, define

$$z_i = (x_{t,i}^- - \mu_w)C_w^{-\top}, \qquad s_a = \sum_i \alpha_{t,i} z_{ia}^3, \qquad k_a = \sum_i \alpha_{t,i} z_{ia}^4. \tag{25.85}$$

These are the selected standardized diagonal moments motivated by the GenUT axis rule of Proposition 25.12. Full third- and fourth-order tensors are not formed. Selected mixed entries are retained by the pairwise correction in Section 25.16.4.

Before attempting a correction, the implementation records necessary one-dimensional realizability diagnostics. For any standardized scalar random variable, Pearson's moment inequality gives

$$k_a \geq s_a^2 + 1. \tag{25.86}$$

For an equal-weight standardized cloud of exactly $N > 1$ rows, a convenient self-contained necessary upper bound is

$$k_a \leq N - 1. \tag{25.87}$$

Bounds on the corresponding sample skewness and kurtosis are classical [Johnson and Lowe, 1979]; that source gives sharper joint bounds, but the closed primary text was not available for technical inspection in this audit, so the implementation uses only the following directly proved inequality. A short derivation of (25.87) uses the zero-sum constraint. Fix $u_1 = x$. Among the remaining $N-1$ rows with sum $-x$, Cauchy–Schwarz implies $\sum_{i=2}^{N} u_i^2 \geq x^2/(N-1)$. Since $\sum_i u_i^2 = N$, this gives $x^2 \leq N - 1$. The same argument applies to every row, hence $u_i^2 \leq N - 1$ and

$$\frac{1}{N}\sum_i u_i^4 \leq \frac{1}{N}(N-1)\sum_i u_i^2 = N - 1.$$

This bound is conservative rather than sharp, which is appropriate for a necessary-feasibility diagnostic.

Equations (25.86) and (25.87) are necessary marginal conditions, not sufficient conditions for a jointly realizable $d$-dimensional cloud. The repair does not clip $s_a$ or $k_a$, because doing so would silently change the empirical moment teacher and hence the finite value program. An infeasible target remains the declared least-squares target, but exact matching is forbidden as a claim and every finite displacement is bounded below.

At one correction iteration, standardize the current equal-weight rows to $u_{ia}$ so that $N^{-1}\sum_i u_i = 0$ and $N^{-1}\sum_i u_i u_i^\top = I_d$. Let

$$m_{r,a} = \frac{1}{N}\sum_i u_{ia}^r, \qquad d_a = \begin{bmatrix} s_a - m_{3,a} \\ k_a - m_{4,a} \end{bmatrix}. \tag{25.88}$$

For each coordinate define two projected moment directions

$$q_{3,ia} = u_{ia}^2 - 1 - m_{3,a}u_{ia}, \tag{25.89}$$
$$q_{4,ia} = u_{ia}^3 - m_{3,a} - m_{4,a}u_{ia}. \tag{25.90}$$

The $2 \times 2$ local moment Jacobian is

$$J_a = \begin{bmatrix} 3N^{-1}\sum_i u_{ia}^2 q_{3,ia} & 3N^{-1}\sum_i u_{ia}^2 q_{4,ia} \\ 4N^{-1}\sum_i u_{ia}^3 q_{3,ia} & 4N^{-1}\sum_i u_{ia}^3 q_{4,ia} \end{bmatrix}. \tag{25.91}$$

The historical update used unscaled normal equations with an absolute ridge. That is unsafe when one column of $J_a$ is many orders of magnitude larger than another: forming $J_a^\top J_a$ squares the scale disparity and a fixed ridge can become numerically irrelevant. Define instead

$$h_{a,j} = \sqrt{\|J_{a,:,j}\|_2^2 + \varepsilon_s^2}, \qquad H_a = \operatorname{diag}(h_{a,1}, h_{a,2}), \tag{25.92}$$
$$A_a = J_a H_a^{-1}, \qquad M_a = A_a^\top A_a + \lambda I_2, \tag{25.93}$$

where $\varepsilon_s > 0$ is a column-scale floor and $\lambda > 0$ is a relative Levenberg–Marquardt damping parameter. The scaled finite coefficient is

$$y_a = M_a^{-1} A_a^\top d_a, \qquad c_a = \rho H_a^{-1} y_a, \tag{25.94}$$

$$\Delta_{ia} = c_{a,1} q_{3,ia} + c_{a,2} q_{4,ia}. \tag{25.95}$$

This is a column-scaled Levenberg–Marquardt step for the local nonlinear least-squares problem [Marquardt, 1963]. Column scaling is not an ad hoc addition: Osborne [1976, Sec. 3, steps (iii) and Note (ii)] recommends scaling the Jacobian columns to unit length so that the terms added to the normal matrix are comparable to its original elements. If $\varepsilon_s = 0$ and both columns are nonzero, the columns of $A_a$ have unit norm. Therefore $\operatorname{tr}(A_a^\top A_a) = 2$, its eigenvalues lie in $[0, 2]$, and

$$\kappa_2(M_a) \le \frac{2 + \lambda}{\lambda}. \tag{25.96}$$

With a positive scale floor the column norms are at most one, so the same upper bound remains valid. This bound concerns the scaled two-by-two solve; it does not prove the nonlinear moment problem is feasible.

To bound the actual cloud move, let the RMS standardized displacement of row $i$ be

$$r_i = \sqrt{d^{-1} \sum_a \Delta_{ia}^2}, \qquad g_i = \left(1 + \frac{r_i^2}{\tau^2}\right)^{-1/2}, \qquad v_{ia} = u_{ia} + g_i \Delta_{ia}, \tag{25.97}$$

for trust radius $\tau > 0$. The map is smooth for finite inputs and its post-cap RMS is $r_i / \sqrt{1 + r_i^2/\tau^2} < \tau$. Thus even an infeasible residual or a large local coefficient cannot move any row farther than the declared radius before rewhitening. This is not a bound on the complete iteration after its covariance rescaling. The corrected $v_i$ are re-centered and re-whitened before the next finite iteration. After the declared number $S$ of iterations, the standardized rows $u_i^H$ are mapped back by

$$x_i^H = \mu_w + u_i^H C_w^\top. \tag{25.98}$$

**Proposition 25.19** (Projected directions and local moment Jacobian)**.** *For each coordinate, both directions in* (25.90) *have zero empirical mean and zero empirical inner product with $u_a$. Moreover, $J_a$ in* (25.91) *is the derivative at $c_a = 0$ of the map from the two correction coefficients to the third and fourth empirical moments before rewhitening.*

*Proof.* Because $m_{1,a} = 0$ and $m_{2,a} = 1$,

$$N^{-1} \sum_i q_{3,ia} = m_{2,a} - 1 - m_{3,a} m_{1,a} = 0, \quad N^{-1} \sum_i u_{ia} q_{3,ia} = m_{3,a} - m_{1,a} - m_{3,a} m_{2,a} = 0.$$

The corresponding expressions for $q_4$ are $m_{3,a} - m_{3,a} - m_{4,a} m_{1,a} = 0$ and $m_{4,a} - m_{3,a} m_{1,a} - m_{4,a} m_{2,a} = 0$. For $v = u + c_1 q_3 + c_2 q_4$, differentiation of $N^{-1} \sum_i v_i^3$ and $N^{-1} \sum_i v_i^4$ at $c = 0$ gives respectively $3N^{-1} \sum_i u_i^2 q_j$ and $4N^{-1} \sum_i u_i^3 q_j$, which are exactly the rows of $J_a$. Thus each direction preserves its coordinate mean and variance to first order; cross-coordinate covariances need not be preserved by the marginal proposal. Un-ridged full-rank rewhitening restores the full covariance exactly. The numerical relative-ridge convention is distinguished below. □

**Proposition 25.20** (Exact first and second moments after rewhitening)**.** *Let $v_i \in \mathbb{R}^d$ have population mean $\bar{v}$, strictly positive definite population covariance $Q$, and Cholesky factor $RR^\top = Q$. Set $u_i = (v_i - \bar{v})R^{-\top}$. Then*

$$\frac{1}{N}\sum_i u_i = 0, \qquad \frac{1}{N}\sum_i u_i u_i^\top = I_d. \tag{25.99}$$

*Consequently* (25.98) *has mean $\mu_w$ and exactly the un-ridged population covariance $P_w$.*

*Proof.* Centering proves the first identity. Row-vector multiplication gives

$$\frac{1}{N}\sum_i u_i^\top u_i = R^{-1}QR^{-\top} = I_d.$$

The affine map in (25.98) therefore has covariance $C_w I_d C_w^\top = P_w$, and its centered rows sum to zero. □

The update (25.94) is a scaled damped local Gauss–Newton/Levenberg–Marquardt step, not an exact projection. The smooth cap (25.97) bounds the executed move but can reduce moment progress. Rewhitening can change the third and fourth moments, and the nonlinear moment-feasibility problem can be infeasible or have multiple solutions. The implementation therefore reports the necessary-feasibility diagnostics, pre/post-cap displacement, and post-rewhitening residuals, and makes no exact higher-moment matching claim.

### 25.16.3 Why the residual design can obstruct kurtosis repair

A correction cannot change a moment unless its particle configuration supplies a direction in which that moment can move. This is a concrete obstruction for the scalar replicated-axis residual design. When the OT barycenters have nearly collapsed, residual injection dominates, and standardization of repeated $-1, +1$ residuals gives a cloud close to two symmetric points. For the exact binary cloud,

$$\overline{u} = 0, \qquad \overline{u^2} = 1, \qquad m_3 = 0, \qquad m_4 = 1.$$

Substitution in (25.90) gives $q_3 = u^2 - 1 = 0$ and $q_4 = u^3 - u = 0$ at every particle. Consequently $J = 0$, and a damped solve returns a zero displacement even when the target kurtosis is near three. A larger correction strength cannot repair this exact degeneracy. The obstruction concerns this design and these correction directions; it does not say that every cloud produced by binary residual injection must itself be exactly binary.

For any zero-mean, unit-variance scalar cloud there is also the useful identity

$$m_4 - 1 = \mathrm{Var}_N(u^2). \tag{25.100}$$

Indeed, $\mathrm{Var}_N(u^2) = \overline{u^4} - (\overline{u^2})^2$. Kurtosis measures variation in squared distance from the mean. Binary points all have the same distance and therefore attain the lower value one. To represent larger kurtosis at the same variance, the cloud needs different radii: many smaller distances and some larger ones. Replacing the binary residual design supplies those radii before the moment fit begins.

The richer option uses a *fixed finite normal-quantile design*, rather than sampling a new continuous random variable at each reset. In one dimension, let $\Phi$ be the standard normal distribution function and set

$$b_i = \Phi^{-1}\left(\frac{i+1/2}{N}\right), \quad i = 0, \dots, N-1, \qquad \Xi_i = \frac{b_i - \bar{b}}{\sqrt{N^{-1}\sum_j (b_j - \bar{b})^2}}. \tag{25.101}$$

The implementation arranges opposite-sign pairs. In several dimensions it uses fixed permutations of the quantile coordinates, then centers and whitens the entire design so that $N^{-1}\Xi^\top\Xi = I_d$, provided its covariance has full rank. Both the ordering and the design are frozen for a finite value/score evaluation, so $\mathrm{d}\Xi/\mathrm{d}\theta = 0$. The implemented construction retains the divisibility requirement $2d \mid N$. It is still an empirical distribution with $N$ atoms; "richer" describes its range of coordinate magnitudes. In particular, finite quantiles do not have exactly Gaussian fourth moments, and subsequent residual injection need not preserve their moments because the barycenters and cross covariance in (25.81) also contribute.

A September 2026 scalar KSC diagnostic separated this design change from the coordinate-cap change described below. The weighted input kurtosis at the examined reset was 2.916221. The same reset, under four choices, gave:

| Residual design and coordinate cap | Raw reset | After fit | Final reset |
|---|---|---|---|
| Repeated axes, original cap | 1.000473 | 1.577655 | 1.328293 |
| Normal quantiles, original cap | 2.972274 | 2.966277 | 1.586420 |
| Repeated axes, identity-core cap | 1.000473 | 1.577655 | 1.577655 |
| Normal quantiles, identity-core cap | 2.972274 | 2.966277 | 2.966277 |

These are standardized fourth moments at one saved reset, not estimates of a population-wide advantage. They expose two different losses of shape: the binary residual design starts near kurtosis one, while the original cap can erase the spread supplied by the richer design. The saved reset values are recorded in the September 29 KSC repair note.[1] The $T = 120$ comparison gives a more informative conclusion than its binary accuracy screen: the revised inverse-CDF route reduced score error on both examined datasets, while likelihood error fell on one and rose on the other. Section 25.16.8 gives the actual values and their uncertainty. A repaired reset moment and a repaired filtering likelihood remain different empirical questions.

### 25.16.4 The pairwise skewness and kurtosis correction remains active

Marginal moments cannot describe all dependence between state coordinates. For the whitened weighted source $z_i$ in (25.85), the pairwise correction retains the targets

$$S^\star_{ab} = \sum_i \alpha_i z_{ia}^2 z_{ib}, \qquad K^\star_{ab} = \sum_i \alpha_i z_{ia}^2 z_{ib}^2, \qquad a \neq b. \tag{25.102}$$

Co-skewness $S^\star_{ab}$ is ordered: generally $S^\star_{ab} \neq S^\star_{ba}$. Co-kurtosis $K^\star$ is symmetric. These selected mixed moments supplement the marginal moments; they are not the complete

[1] `docs/benchmarks/sqmc-ksc-reset-repair-results-20260929.md`

third- and fourth-order tensors. For example, $\mathbb{E}[z_a^3 z_b]$ and entries involving three or four distinct coordinates are outside this pairwise fit. An explicitly supplied moment teacher may replace the weighted empirical targets, but that changes what is being fitted and needs its own accuracy evidence.

Let $U$ be the current $N \times d$ standardized cloud, let $U^{\circ 2}$ denote entrywise squares, and let $M_3, M_4$ be fixed selection masks, with zero diagonal. The all-pairs option sets every off-diagonal entry to one. Define

$$S(U) = N^{-1}(U^{\circ 2})^\top U, \qquad K(U) = N^{-1}(U^{\circ 2})^\top U^{\circ 2}, \tag{25.103}$$

$$R_3 = M_3 \odot (S^\star - S(U)), \qquad R_4 = M_4 \odot (K^\star - K(U)). \tag{25.104}$$

The implemented direction before projection is

$$G = \frac{2U \odot (UR_3^\top) + (U^{\circ 2})R_3 + 2U \odot ((U^{\circ 2})R_4^\top)}{\max(d-1, 1)}. \tag{25.105}$$

For binary masks and symmetric $M_4$, this equals $-N/\max(d-1,1)$ times the gradient with respect to $U$ of $\frac{1}{2}\|R_3\|_F^2 + \frac{1}{4}\|R_4\|_F^2$. The factor one quarter accounts for the two occurrences of a symmetric co-kurtosis entry. To check the factors directly, $\partial S_{ab}/\partial u_{ic} = N^{-1}(2u_{ia}u_{ib}\mathbf{1}_{c=a} + u_{ia}^2\mathbf{1}_{c=b})$; co-kurtosis has the corresponding two squared-coordinate terms. Summing these derivatives gives (25.105).

The next projection removes the mean and the first-order covariance change:

$$G_c = G - \mathbf{1}\bar{G}, \qquad A = \operatorname{sym}(N^{-1}U^\top G_c), \qquad H = G_c - UA. \tag{25.106}$$

If $\bar{U} = 0$ and $N^{-1}U^\top U = I$, then $\bar{H} = 0$ and $N^{-1}(U^\top H + H^\top U) = 0$: substituting $H$ cancels $2A$. The projection therefore preserves mean and covariance to first order, although a finite step still needs rewhitening. With a positive floor $\varepsilon_p$, normalize and cap the direction as

$$V = \frac{H}{\sqrt{(Nd)^{-1}\|H\|_F^2 + \varepsilon_p}}, \quad r_i^2 = d^{-1}\|V_i\|_2^2, \quad D_i = \eta_p \frac{V_i}{\sqrt{1 + r_i^2/c_p^2}}. \tag{25.107}$$

Each proposed displacement has RMS less than $\eta_p c_p$. The candidate $U + D$ is then whitened. Thus the existing pairwise cap remains a *displacement* cap; changing the final coordinate cap does not remove it. For $d = 1$, there are no off-diagonal pairs and the pairwise loop is skipped. A scalar KSC result consequently provides no test of pairwise repair.

### 25.16.5 Why a bounded coordinate map is not a stability proof

The original standardized coordinate cap is, for even $p$,

$$f_{\text{old}}(u) = \frac{u}{[1 + (u/c)^p]^{1/p}}, \qquad c > 0. \tag{25.108}$$

It bounds each mapped coordinate by $c$, but compresses magnitudes throughout the cloud. With $c = 0.98$ and $p = 8$, moderate and large standardized coordinates are pushed toward similar magnitudes. After variance restoration, that compression can leave too little variation in $u^2$, and hence too little kurtosis by (25.100).

The revised optional map is exactly the identity inside radius $r$:

$$f_{r,c}(u) = \begin{cases} u, & |u| \le r, \\ \operatorname{sign}(u)\{r + w\tanh[(|u| - r)/w]\}, & |u| > r, \end{cases} \qquad w = rc. \tag{25.109}$$

At the join, the outside value is $\operatorname{sign}(u)r$, its first derivative is one and its second derivative is zero. It therefore joins the identity with two continuous derivatives. Outside, its derivative is $\operatorname{sech}^2[(|u| - r)/w]$, between zero and one, and $|f_{r,c}(u)| < r(1 + c)$. The September candidate uses $r = 8$ and $c = 0.98$, giving a pre-restoration bound of 15.84. These are recorded candidate settings, not universal settings for another model or tuning scope. The map is followed by centering, covariance restoration and mapping to $(\mu_w, P_w)$; those operations are part of the reset.

There are three reasons why this bound alone cannot prevent a higher-moment fit from going wrong. First, the cap is applied *after* the fitting iterations, so it cannot repair an earlier invalid solve or factorization. Second, a bounded displacement need not improve a nonlinear moment residual, especially after rewhitening. Third, whitening can amplify a cloud whose smallest covariance eigenvalue has collapsed. A bound before whitening is not a bound after whitening. Even well inside radius eight, a single standardized coordinate of magnitude eight contributes $8^4/1008 \simeq 4.06$ to a 1008-particle fourth moment. Finiteness alone is therefore a weak test of higher-moment stability.

### 25.16.6 A guarded finite fit with a protected fallback

The safety repair retains the scaled LM solve, both displacement caps and the final coordinate cap. It adds a test of each proposed step *after* rewhitening and a separate test of the complete capped reset. The objective must be specified because marginal and mixed moments can compete. For a selected target entry $t_j$, let $m_j(U)$ denote its empirical moment and let $a_j = \max(1, |t_j|)$. Stack the marginal third and fourth moments and, when pairwise correction is configured, the selected entries of (25.102). The checked discrepancy is

$$\mathcal{L}(U) = \sum_{j \in \mathcal{J}} \left( \frac{m_j(U) - t_j}{a_j} \right)^2 . \tag{25.110}$$

This scale gives absolute error near a zero target and relative error for large targets. Symmetric co-kurtosis entries occur twice in the all-pairs sum. The normalization is an explicit safety objective, not a proof that every moment has equal scientific importance. In particular, a decrease in $\mathcal{L}$ can accompany an increase in one individual kurtosis residual; the individual residual families must remain visible. The marginal LM and pairwise proposals need not be descent directions for this combined objective. The acceptance test, rather than that assumption, decides whether to use them.

For a current standardized cloud $U$, define its empirical covariance $C$ and a capped proposal displacement $D$. Try the fixed sequence $\alpha \in \{1, 1/2, \ldots, 1/128\}$, stopping at the first accepted trial. For $V_\alpha = U + \alpha D$, let $C_\alpha$ be its centered empirical covariance. Before factoring it, require finite values and tangents, a valid input covariance, and

$$\tfrac{1}{4}C \preceq C_\alpha \preceq \tfrac{9}{4}C. \tag{25.111}$$

The positive-semidefinite inequalities are checked by symmetric eigenvalues. An inadmissible trial does not enter the Cholesky branch. A valid trial is centered and whitened,

using the same normalization convention as its original marginal or pairwise update, to obtain $\mathcal{W}(V_\alpha)$. Accept it only if the resulting value, analytical tangent and loss are finite and

$$\mathcal{L}(\mathcal{W}(V_\alpha)) \le \mathcal{L}(U) + \tau_{\mathrm{fp}}(\mathcal{L}(U)), \qquad \tau_{\mathrm{fp}}(l) = 32\epsilon_{\mathrm{mach}}(1 + |l|). \tag{25.112}$$

If all eight trials fail, retain $U$ and its incoming tangent. This is a bounded search, with no assertion of eventual acceptance or convergence. Repeated rejection is reported, rather than concealed by a zero-loss claim.

To interpret the covariance envelope, put $B = C^{-1/2}C_\alpha C^{-1/2}$. Equation (25.111) implies $\lambda(B) \in [1/4, 9/4]$. Consequently $\|B^{-1/2}\|_2 \le 2$ and $\|B^{1/2}\|_2 \le 3/2$. These bounds concern relative whitening in the current covariance metric; they do not bound the source covariance, all accumulated derivatives, or the final physical-state coordinates. The implementation uses a roundoff margin $32\epsilon_{\mathrm{mach}}\,\mathrm{tr}(C)/d$ in the matrix tests and requires the input minimum eigenvalue to exceed that margin. It also requires the trial minimum eigenvalue to exceed $32\epsilon_{\mathrm{mach}}\,\mathrm{tr}(C_\alpha)/d$: otherwise a Loewner roundoff allowance could admit a singular trial from an input already near that allowance. The displayed exact-arithmetic bounds should therefore be read with that numerical allowance.

The last check includes operations outside the fitting loop. Let $\mathcal{F}(U)$ mean applying the chosen coordinate cap, recentering, restoring covariance and mapping to the weighted source scale. Compute both $X_{\mathrm{fit}} = \mathcal{F}(U_{\mathrm{fit}})$ and $X_0 = \mathcal{F}(U_0)$, where $U_0$ is the initial standardized raw Contract E cloud and has undergone no moment fitting. Evaluate their moments in the same whitening convention used for reporting. Return the fitted cloud only if it is valid and

$$\mathcal{L}(\mathcal{W}(X_{\mathrm{fit}})) \le \mathcal{L}(\mathcal{W}(X_0)) + \tau_{\mathrm{fp}}(\mathcal{L}(\mathcal{W}(X_0))). \tag{25.113}$$

Otherwise return $X_0$, together with its correctly recomputed total tangent. The baseline must itself be valid; an invalid baseline makes the route invalid. Both alternatives use the *same* coordinate cap and restoration. Thus rejecting a fit does not bypass the tail protection. In this optional safety mode, a zero-iteration call deliberately evaluates that protected baseline; the historical zero-iteration, safety-disabled call remains a no-op.

The exact covariance proposition above assumes un-ridged whitening. The numerical implementation also uses, where declared, the relative ridge $Q_\delta = \mathrm{sym}(Q) + \delta\,\mathrm{tr}(Q)I/d$, with $\delta = 10^{-12}$. If $RR^\top = Q_\delta$, then the empirical covariance after whitening is

$$R^{-1}QR^{-\top} = I - \delta\,\mathrm{tr}(Q)R^{-1}R^{-\top}/d.$$

It is not exactly the identity. The ridge scale and its tangent belong to the finite numerical program, and covariance residuals are checked numerically. A small coefficient alone does not imply a small effect when $Q$ is badly conditioned. The step guard checks the fit; it does not rescue invalid source moments or an invalid upstream Contract E factorization.

The implementation records the trial count, rejected-step count, smallest accepted step, input-validity flag, largest accepted loss increase, protected baseline loss, final loss and final-fallback flag. It is optional through `moment_safety=True` in the shared marginal/pairwise routine and the canonical filtering consumer. Full projected-cumulant correction is a distinct path and is explicitly rejected with this option, rather than silently omitted from its safety objective. The richer residual design and identity-core cap also remain explicit settings; none has been promoted to a universal default.

**Bounded numerical evidence.** The October 2026 safety comparison used fresh fixed fixtures in dimensions 1, 2, 3 and 10, with healthy, skewed, mixture, outlier, concentrated-weight, nearly collinear and repeated-axis cases. All 112 GPU/XLA cases passed the finite-value and declared loss checks; a 14-case CPU reference also passed. The guard rejected at least one complete proposed step in 23 GPU cases and used a reduced step without a complete rejection in another 33 cases. These counts describe the evaluated fixtures, not probabilities of failure. The guard was less accurate in the declared combined moment loss than at least one simple comparator in 26 of the 112 GPU cases, including mixture, concentrated-weight and repeated-axis regimes. This vetoes an accuracy-based promotion. It does not contradict the narrower safety comparison with the identically protected no-fit cloud.

A subsequent edge-case audit added the explicit positive trial-eigenvalue check described above. Its focused CPU regression passed, and eight further GPU/XLA checks of the final implementation covered active tail capping, healthy-update agreement and analytical derivatives. The largest FP64 finite-difference tangent discrepancy in those checks was $1.18 \times 10^{-10}$. Healthy GPU particle values were unchanged; their largest tangent difference was $4.34 \times 10^{-19}$, so bitwise equality of all GPU tangents is not asserted. The broad comparison's largest relative covariance residual was $1.10 \times 10^{-3}$ in FP32/TF32 and $1.32 \times 10^{-15}$ in FP64. These measured distinctions are why the exact covariance proposition must not be presented as an unconditional floating-point identity. The complete comparisons, source versions, failures, commands and limitations are recorded in `docs/benchmarks/ledh-moment-safety-results-20261001.md`. The original historical explosive input was not recovered and replayed; fresh stress fixtures cannot establish that every such failure is resolved.

### 25.16.7 Complete algorithm and score statement

For the LEDH–PFPF–OT filter, the following sequence specifies one complete time step. It is essential to distinguish the proposal stage from the reset: the bootstrap weights in (25.73)– (25.74) illustrate the reset's input, whereas an LEDH proposal requires the importance correction in Chapter 22. Using observation likelihoods alone after a general particle-flow proposal would compute different weights.

(1) Carry each particle, its UKF posterior covariance, its weight and their analytical parameter tangents from the preceding step. Predict its local Gaussian, generate its fixed innovation, and execute the declared finite LEDH flow using the particle's own covariance lifecycle. Apply the proposal-density and flow-Jacobian importance correction; accumulate the log normalizer and normalize the resulting weights. Update the local UKF covariance and its total tangent. These local covariances are distinct from the global empirical reset covariance.

(2) Regard the corrected weighted cloud as $\{x_{t,i}^{-}, \alpha_{t,i}\}_{i=1}^{N}$. Compute its mean and covariance, diagonal shape targets, and any selected pairwise targets. Keep the source and weight derivatives in all these quantities.

(3) Run the fixed finite Sinkhorn and balancing schedule, form the row-normalized barycenters in (25.78), and check the declared marginal and validity diagnostics. The streaming implementation uses the repository's exact-divisor chunk policy.

(4) Inject the frozen residual design through (25.80) and apply the realized-covariance Contract E restoration (25.83). The revised option uses (25.101); it does not change the OT weights or claim to reproduce the full source law. Standardize this raw reset to obtain $U_0$.

(5) For each of the declared $S$ marginal iterations, compute the moment directions, scaled LM coefficients and capped displacement in (25.90)–(25.97). With safety enabled, apply the covariance and post-whitening tests (25.111)– (25.112); reduce or reject the displacement as specified. With safety disabled, execute the original capped update and whitening.

(6) For each of the declared $P$ pairwise iterations, form (25.104), project and cap its direction by (25.106)–(25.107), and apply the same optional guard and rewhitening. Skip this loop when $d = 1$ or $P = 0$.

(7) Apply the declared coordinate map, recenter, restore covariance and map back to the source mean and scale. With safety enabled, compare this complete result with the identically capped and restored no-fit cloud using (25.113). Select both value and total tangent from the accepted alternative and retain the diagnostics. Fail closed if the protected baseline is invalid.

(8) Carry the selected particles with equal weights, together with the reset route's per-particle covariance carry and its analytical derivative, to the next observation. Continue the score recursion through the same branch as the value recursion.

All iteration counts, masks, caps, damping controls, residual permutations and halving schedules are fixed within an evaluation. Scope-specific offline tuning, when needed for an accuracy claim, precedes the untouched claim run; this safety mechanism is not permission to retune controls inside HMC. The particle count satisfies the chosen residual-design rule: the implemented replicated-axis and normal-quantile designs require $2d \mid N$, while an exactly replicated positive GenUT design requires representable masses. The correction retains all $N$ rows. Marginal and pairwise value calculations use $O(Nd + d^2)$ additional storage beyond the OT workspace, with additional storage for the requested tangent directions. Pairwise products and covariance checks add $O(Nd^2 + d^3)$ work per trial; eight trials bound the work, not the moment error.

The analytical tangent must follow all dependencies of this finite algorithm. For example, writing a dot for a parameter-direction derivative,

$$\dot{s}_a = \sum_i \dot{\alpha}_i z_{ia}^3 + 3 \sum_i \alpha_i z_{ia}^2 \dot{z}_{ia}, \tag{25.114}$$

$$\dot{K}^\star_{ab} = \sum_i \dot{\alpha}_i z_{ia}^2 z_{ib}^2 + 2 \sum_i \alpha_i (z_{ia} \dot{z}_{ia} z_{ib}^2 + z_{ia}^2 z_{ib} \dot{z}_{ib}). \tag{25.115}$$

The weight terms cannot be dropped. If $CC^\top = P$, put $E = C^{-1} \dot{P} C^{-\top}$ and let $\mathcal{T}(E)$ retain its strict lower triangle and half its diagonal. Then

$$\dot{C} = C\mathcal{T}(E), \qquad \dot{u} = (\dot{x} - \dot{\mu})C^{-\top} - u\dot{C}^\top C^{-\top}. \tag{25.116}$$

These formulas also differentiate a declared relative ridge through its trace scale. Differentiate the LM linear system by solving $M\dot{y} = \dot{b} - \dot{M}y$, where $b = A^\top d$; differentiate the

pairwise matrix products and smooth caps by the ordinary chain rule. The identity-core cap multiplies a tangent by one in its core and by the $\operatorname{sech}^2$ factor in its tail.

On a locally unchanged acceptance branch, the selected $\alpha$ is a fixed member of the halving schedule, so $\dot{V}_\alpha = \dot{U} + \alpha\dot{D}$. A rejected step carries $\dot{U}$, and a final fallback carries the total derivative of $X_0$, rather than a zero tangent. The eigenvalue comparisons and acceptance decisions select a branch; they are not differentiated as if they were smooth functions. Consequently branch boundaries may have a discontinuous derivative or value. A finite-difference check must verify that its two perturbations select the same branches, and local agreement is not a global smoothness or HMC theorem.

**Proposition 25.21** (Finite-route total-score correctness)**.** *Fix the innovation rows and ordering, residual design, finite Sinkhorn and balance counts, $\epsilon$, OT ridge, $\rho$, LM damping $\lambda$, column-scale floor $\varepsilon_s$, trust radius $\tau$, marginal and pairwise counts, masks, coordinate cap, and safety schedule. Assume every required covariance is strictly positive definite, every row mass is positive, the maximum branch in the cost scale and all safety-acceptance and final-fallback branches are fixed locally, and every validity condition holds. If the recursive tangent includes every operation listed above, then its accumulated score is the total derivative with respect to $\theta$ of the same finite scalar $\widehat{L}^N(\theta) = \sum_t \widehat{\ell}_t^N(\theta)$ executed by the value program.*

*Proof.* On the stated branch, the transition and likelihood adapters, normalization, finite Sinkhorn recurrences, row quotient, moment maps, matrix products, Cholesky factors, triangular solves, column scales, $2 \times 2$ damped solves, smooth trust and coordinate caps, selected pairwise operations, and finite rewhitening maps are differentiable on this branch. The ordinary chain rule therefore applies to their finite composition. The implemented tangent includes the dependence of the normalized weights, OT kernel and scalings, target moments, directions $q_3, q_4$, Jacobians $J_a$, column scales $H_a$, scaled systems $M_a$, damped coefficients $c_a$, row cap factors $g_i$, and both standardization maps. Induction over time then shows that the carried particle tangent is the derivative of the carried particle value and that the sum of increment tangents is $d\widehat{L}^N/d\theta$.

This proves correctness for the executed finite approximation. It does not prove that $\widehat{L}^N$ equals the exact filtering log likelihood or that its derivative equals the exact filtering score. Those are model-specific statistical approximation questions tested against available references. □

**Correctness boundary.** The propositions establish the OT quotient semantics, the Contract E ridged identity, exact post-correction empirical mean and covariance under the un-ridged full-rank assumptions, and the local total derivative of one fixed finite program. Declared ridges have the covariance discrepancy derived above; safety selection gives its stated loss bound, not global differentiability. They do not establish global higher-moment feasibility, exact mixed moments, unbiased likelihood, exact posterior score, superiority over Contract E, HMC readiness, or a default change. The model campaign below can reject this candidate or keep it viable; it cannot by itself prove broad nonlinear-model validity.

### 25.16.8 What the actual likelihoods and scores show

The downstream question is whether preserving the weighted cloud's shape brings the accumulated likelihood and score closer to the filtering reference. For the scalar KSC

example, the full seven-mixture likelihood and its two parameter derivatives can be evaluated by a separately refined quadrature. The September comparison used this reference at $\theta = (\gamma, \log \beta) = (1.5, 0)$, two untouched datasets of length 120, and eight particle designs per method. It ran in FP64 with GPU/XLA and TF32 disabled. Tables 25.1 and 25.2 report the actual quantities, including the Gaussian Kalman approximation as a simple comparator. The revised arm uses the richer residual, identity-core cap and controls selected for that scope; the comparison does not isolate the causal effect of each change. The full-mixture reference is an oracle for the declared seven-mixture target, rather than for a different stochastic-volatility observation law. The numerical values, refinement checks and component uncertainties are preserved in the comparison record.[2]

| Method | Log likelihood | $\gamma$ score | $\log \beta$ score |
|---|---|---|---|
| Seven-mixture reference | -270.702896202 | -0.508659556 | -1.291005193 |
| Gaussian Kalman heuristic | -275.542306325 | -0.813750759 | -0.892859070 |
| IID / original | -271.316778239 | -0.432267129 | -1.085171066 |
| IID / revised | -270.879543409 | -0.633906248 | -1.217855750 |
| Inverse CDF / original | -271.239752570 | -0.375158888 | -1.089143021 |
| Inverse CDF / revised | -270.821584886 | -0.580225487 | -1.269873809 |
| Permutation / original | -271.238543975 | -0.371483139 | -1.087564459 |
| Permutation / revised | -270.823316362 | -0.577390069 | -1.269882173 |
| Ablation / original | -271.239262624 | -0.368794285 | -1.087245064 |
| Ablation / revised | -270.823316362 | -0.577390069 | -1.269882173 |

Table 25.1: KSC at $T = 120$, dataset A (seed 243001). Particle entries are means of eight independent designs with $N = 1008$. The revised arm uses the September design/cap repair and its selected controls; it does not include the October safety guard.

| Method | Log likelihood | $\gamma$ score | $\log \beta$ score |
|---|---|---|---|
| Seven-mixture reference | -284.105359942 | 2.358935308 | 0.299061235 |
| Gaussian Kalman heuristic | -289.119307428 | 2.448686517 | 0.655680321 |
| IID / original | -284.231222413 | 2.856512226 | 0.299130287 |
| IID / revised | -284.459443680 | 2.338869846 | 0.347294048 |
| Inverse CDF / original | -284.160036239 | 3.035272629 | 0.331255873 |
| Inverse CDF / revised | -284.225769251 | 2.368748070 | 0.350155526 |
| Permutation / original | -284.159942453 | 3.026332424 | 0.330404077 |
| Permutation / revised | -284.227196844 | 2.366669388 | 0.350044821 |
| Ablation / original | -284.158477299 | 3.046262376 | 0.333190588 |
| Ablation / revised | -284.227196844 | 2.366669388 | 0.350044821 |

Table 25.2: KSC at $T = 120$, dataset B (seed 243002). Particle entries are means of eight independent designs with $N = 1008$. The revised arm uses the September design/cap repair and its selected controls; it does not include the October safety guard.

The inverse-CDF route illustrates the distinction between likelihood and score accuracy. On dataset A its mean absolute log-likelihood error across designs fell from 0.536856 to 0.163187; on dataset B it rose from 0.125318 to 0.213643. Its mean score-vector error

[2] `docs/plans/artifacts/sqmc-ksc-reset-repair-20260929/attempt-01/analysis-01/comparisons.md`

norm fell from 0.329639 to 0.144456 on A and from 0.681811 to 0.146093 on B. These are means of per-design errors, which differ from the error of a mean reported in the tables. The paired 95% bootstrap intervals for revised minus original score-error norm were $[-0.291356, -0.084593]$ and $[-0.654304, -0.399304]$, respectively. They support a conditional reduction on these two datasets; they are exploratory intervals across several examined cells, without a multiplicity adjustment.

The uncertainty in each score coordinate also matters. For the revised inverse-CDF arm, the standard errors of the mean $\gamma$ and $\log \beta$ scores were $(0.053617, 0.012488)$ on A and $(0.055220, 0.026190)$ on B. Thus a visible difference in the last digits is not itself evidence of a persistent bias. Conversely, the near equality of the original IID $\log \beta$ score and reference on B does not establish that every coordinate improved after repair. The aggregate score norm and the individual coordinates answer different questions. These results make the revised mechanism worth testing further, while its failed strict accuracy screen and the dataset-B likelihood deterioration prevent a general accuracy or default-readiness conclusion.

To see why a likelihood offset and a score discrepancy need not move together, write the error of a fixed-design finite filter as

$$e_N(\theta) = \widehat{L}^N(\theta) - L(\theta). \tag{25.117}$$

Here $L$ is the exact log likelihood for the declared model. At a parameter where both functions are differentiable and the numerical selection branches are locally fixed, subtracting their derivatives gives

$$\widehat{s}_N(\theta) - s(\theta) = \nabla_\theta e_N(\theta). \tag{25.118}$$

A constant likelihood offset can therefore coexist with an accurate score; a small likelihood error at one parameter can coexist with a steep error gradient. Likewise the error in a log-likelihood ratio between two parameters is $e_N(\theta') - e_N(\theta)$, not either offset separately. Assessing a parameter neighborhood, both score coordinates and branch changes is necessary before using these observations to argue for gradient-based inference. The tables at one parameter do not establish HMC readiness.

There are also two distinct fourth-moment questions. Fitting can bring a reset closer to the weighted particle cloud's fourth moment. That target may itself differ from the filtering distribution's fourth moment. Let $m_4^R$, $m_4^W$ and $m_4^\star$ denote the reset, weighted-cloud and reference fourth moments, all expressed in one common centering and scaling convention. Then

$$m_4^R - m_4^\star = (m_4^R - m_4^W) + (m_4^W - m_4^\star).$$

The proposed correction addresses the first term. The second requires an independent filtering reference or a better source cloud. Comparing likelihoods and scores with their oracle directly tests the downstream approximation, but does not by itself identify which fourth-moment term caused an error.

### 25.16.9 The first nonlinear comparisons

Predator–prey and spatial SIR test different weaknesses of the repair. Predator–prey has two state coordinates and nonlinear coupled dynamics, so pairwise moments already have a substantive role. Its six score coordinates are the physical parameters $r, K, a, s, u, v$ used by the declared transition. The spatial SIR case has 18 state coordinates, nine observed

infectious coordinates, and three parameters: the log infection-rate scale, log removal-rate scale and log observation-noise scale. Its larger collection of mixed moments tests whether fitting dependence remains numerically useful as dimension grows.

A master tester now sends both models through the same finite LEDH executor, per-particle UKF covariance recursion, Contract E reset and analytical total score. It compares covariance-only, original capped, richer marginal-only, richer marginal-and-pairwise, and guarded marginal-and-pairwise configurations. The last pair isolates the optional guard; the marginal-only comparison isolates the additional mixed-moment correction. Likelihoods, every score coordinate, replication uncertainty, invalid evaluations and reset diagnostics are retained separately for each model, dataset, parameter point, particle count and ancestry rule.[3]

Fresh synthetic observations follow the declared initial Gaussian law and a transition before each observation. This timing matters: a bootstrap filter that observes the initial state before its first transition estimates a different likelihood for the same numerical observations. The independent reference used here starts from the same initial law and saved observations as LEDH, then transitions before every observation. It also retains the same parameter values, including their original floating-point representation.

The reference is a Gaussian bootstrap particle filter with an analytical Fisher-identity score. Write $x_{0:T}$ for a complete latent trajectory, $f_\theta$ for its transition density and $g_\theta$ for the observation density. Because the initial law in these examples is parameter independent, regular differentiation under the integral gives

$$\nabla_\theta \log p_\theta(y_{1:T}) = \mathbb{E}_\theta\left[\sum_{t=1}^{T}\{\nabla_\theta \log f_\theta(x_t \mid x_{t-1}) + \nabla_\theta \log g_\theta(y_t \mid x_t)\} \,\middle|\, y_{1:T}\right].$$

Each particle carries this accumulated complete-data score through resampling; the final weighted average estimates the conditional expectation. State values are held fixed when differentiating the complete-data densities, and their Gaussian covariance derivatives are included. This is an approximation to the observed-data score, whereas LEDH differentiates its own finite numerical likelihood. Their finite-particle scores need not coincide.

The reference passed fixed-state finite-difference checks for both nonlinear models and a separate linear-Gaussian comparison with an exact Kalman likelihood and score. Four independent replications were then evaluated at 8,192, 32,768, 131,072 and 524,288 particles. The reported reference likelihood is the logarithm of the mean likelihood estimate; its score averages the Fisher estimates with likelihood weights. Delete-one-replication jackknife standard errors describe Monte Carlo variation. They do not certify that finite-particle bias has vanished, so this reference is not called an exact nonlinear oracle.

The first comparison used $T = 20$ and $N = 1008$ for LEDH, with inherited, untuned numerical controls. The two predator–prey datasets used seeds 260401 and 260402; the SIR dataset used 260401. LEDH ran on the GPU with XLA, FP32 and TF32. Predator–prey returned finite likelihoods and all six score coordinates in 70 evaluations across the five correction choices and the tested ancestry rules. Sixty-nine also passed the separate trace-score check. Table 25.3 gives the IID means, which make the difference between the two models visible.

For predator–prey dataset 260401, the guarded score mean in the order $(r, K, a, s, u, v)$ was

$$(-33.52772,\ -0.54696,\ 0.01964,\ 4.56826,\ -8.54557,\ 10.63757),$$

[3] `docs/benchmarks/ledh-nonlinear-master.md`

| Model / dataset | Covariance only | Guarded pairwise | Reference (MCSE) |
|---|---|---|---|
| Predator–prey, 260401 | −97.43870 | −97.35035 | −97.34297 (0.00657) |
| Predator–prey, 260402 | −99.29759 | −99.28713 | −99.31152 (0.00366) |
| SIR, 260401, FP64 | −975.57354 | −1041.35394 | −678.07747 (0.01486) |

Table 25.3: Actual log likelihoods at $T = 20$. LEDH means use two IID designs on dataset 260401 and four on predator–prey dataset 260402. The SIR row is a same-data FP64 precision diagnostic; all ten initial FP32 SIR evaluations failed. The SIR guarded mean includes one finite evaluation whose separate trace-score check failed. References use four replications at 524,288 particles. These small-sample means do not establish a ranking.

while the reference was

$$(-32.90524,\ -0.54394,\ 0.02053,\ 4.48734,\ -8.73240,\ 10.86545).$$

The reference coordinate MCSEs were

$$(0.07550, 0.00339, 0.0000942, 0.00827, 0.02200, 0.02770).$$

For dataset 260402, the corresponding guarded and reference score means were

$$(-21.21510,\ 0.03513,\ 0.01890,\ 1.52016,\ -6.60116,\ 8.16064),$$
$$(-21.14020,\ 0.03931,\ 0.01868,\ 1.50221,\ -6.53110,\ 8.07440).$$

The reference coordinate MCSEs there were

$$(0.09711, 0.00155, 0.00000666, 0.00506, 0.00271, 0.00270).$$

The full per-design scores, LEDH replication standard errors and conditional comparisons with covariance-only, original and marginal-only correction are retained in the result record.[4] The values support further predator–prey validation, but two or four design replications at a single parameter point do not establish that the repair improves the observed-data score.

SIR exposes two separate problems. In the localized FP32 run, the first failure occurred at observation 4 in the Contract E reset. Ancestry, UKF prediction, UKF update and model-callback validity checks still passed; flow states and derivatives and likelihood weights were finite. Covariance-only reset states remained finite but their derivatives did not. The guarded configuration lost finite reset states and derivatives, so the higher-moment guard no longer had a valid input to protect. These observations locate the failure in the reset but do not yet identify its exact factorization or prove a remedy.

Replaying the exact FP32 observations and random inputs in FP64 gave finite SIR likelihoods and scores. Accuracy still failed conspicuously relative to the independent reference. The two guarded likelihoods were −1041.82982 and −1040.87806, with score vectors

$$(-165.24294,\ -186.13243,\ 963.39536),$$
$$(3230.85495,\ -1581.97484,\ 936.41325).$$

[4] `docs/benchmarks/ledh-nonlinear-execution-results-20261002.md`

In the log-scale parameter order given above, the reference score was

$$(106.31321, -65.96031, 5.58635),$$

with coordinate MCSEs $(5.53617, 2.42038, 0.03538)$. Thus restoring finite arithmetic did not restore filtering accuracy. The hundreds of log-likelihood units separating SIR's LEDH and reference estimates are far larger than the measured reference replication noise.

Two finite evaluations also retained trace-score consistency vetoes: original FP32 predator–prey differed by $6.47 \times 10^{-4}$ in its first score coordinate, and one guarded FP64 SIR evaluation differed by $1.71 \times 10^{-7}$. The recorded tolerances were not relaxed. Together with the absence of scope-specific calibration and untouched validation, these findings prevent admission or a default-readiness conclusion. The next SIR step is to diagnose the failing reset factorization and its analytical derivative, then evaluate any protection for non-harm before calibrating the approximation. It is not evidence that fitting fourth moments should be abandoned: the failure occurs in a common reset stage, and the independent likelihood and score comparison remains necessary even when that stage is numerically healthy.

## 25.17 A Zhao–Cui squared-TT moment teacher for Contract E

The empirical targets in (25.85) preserve higher moments already present in the weighted particle cloud. They cannot correct Monte Carlo error in those same empirical moments. This section defines a different, opt-in construction: run a deterministic squared tensor-train (TT) density recursion in parallel with the particle filter, use the TT density only to compute standardized shape moments, and give those moments to the higher-moment Contract E correction. The construction avoids a dense tensor grid.

The source-grounded operations are the adjacent-density recursion, squared-TT non-negative representation, and paired-core marginal contractions in Algorithms 1–2 and Proposition 2 of Zhao and Cui [2024]. The author implementation constructs the adjacent approximation in `third_party/audit/zhao_cui_tensor_ssm_p10/source/models/full_sol.m`, lines 21–136, and performs squared-TT marginal recursions in `third_party/audit/zhao_cui_tensor_ss` lines 25–85. Using the resulting moments as Contract E targets is not an operation claimed by Zhao and Cui. The assembled route is therefore `extension_or_invention`; it must not be called a source-faithful Zhao–Cui filter.

### 25.17.1 Parallel TT filtering recursion

The teacher is evaluated conditionally at the current HMC parameter $\theta$. The parameter is not an additional random coordinate and is never integrated out. Only latent/static variables explicitly belonging to the filtering state enter the TT chart. In the state-only case the adjacent order is $(x_t, x_{t-1})$. If $\vartheta$ denotes a genuine carried latent variable, its coordinate order and marginalization must be declared by the model. Given the previous conditional approximation $\widehat{\pi}^{\mathrm{TT}}_{t-1}(x_{t-1}, \vartheta; \theta)$, define

$$q_t(x_t, \vartheta, x_{t-1}) = \widehat{\pi}^{\mathrm{TT}}_{t-1}(x_{t-1}, \vartheta) p_\theta(x_t \mid x_{t-1}, \vartheta) p_\theta(y_t \mid x_t, \vartheta). \tag{25.119}$$

Here $\vartheta$ must not be identified with the HMC parameter $\theta$. Every factor in $q_t$ is evaluated at the fixed $\theta$ supplied by the outer likelihood program. On a declared product-measure coordinate chart $r = (r_1, \dots, r_m)$, first pull the target back to the reference measure. If $x = \Psi_t(r)$ is the complete chart, define

$$\bar{q}_t(r) = \frac{q_t(\Psi_t(r))\,|\det D\Psi_t(r)|}{\omega_\nu(r)}, \qquad \omega_\nu(r) = \frac{d\nu}{dr}. \tag{25.120}$$

The Jacobian factor and, when $\nu$ is not Lebesgue measure, the inverse reference density belong to the Radon–Nikodym target and its tangent; omitting either computes moments under the wrong measure. Define

$$c_t = \max_{\text{fit rows}} \log \bar{q}_t(r), \qquad s_t(r) = \exp\!\left\{\tfrac{1}{2}[\log \bar{q}_t(r) - c_t]\right\}. \tag{25.121}$$

The fixed-branch fitter minimizes its declared weighted ridge objective against $s_t$, not against $\bar{q}_t$. If $h_t$ denotes the fitted square-root TT, form

$$\lambda_t \ge 0, \qquad \tau_t = e^{-c_t}\lambda_t, \qquad \widetilde{q}_t(r) = h_t(r)^2 + \tau_t q_{0,t}(r), \qquad Z_t^{\mathrm{TT}} = \int \widetilde{q}_t(r)\,d\nu(r), \qquad \nu = \bigotimes_{k=1}^{m} \nu_k. \tag{25.122}$$

Here $q_{0,t} \ge 0$, $\lambda_t$ is the defensive weight in the unscaled pulled-back target, and $\tau_t$ is its coefficient in scaled TT units. Thus an exact fit gives $\widetilde{q}_t = e^{-c_t}(\bar{q}_t + \lambda_t q_{0,t})$. If instead a route declares $\tau_t$ directly and keeps it fixed, it defines a different scale-shift-dependent defensive mixture and must carry that semantics in its route identity. The normalized teacher density is $\widehat{q}_t^{\mathrm{TT}} = \widetilde{q}_t / Z_t^{\mathrm{TT}}$. Marginalizing the coordinates for $\vartheta$ and $x_{t-1}$ gives

$$\widehat{\pi}_t^{\mathrm{TT}}(x_t) = \int \widehat{q}_t^{\mathrm{TT}}(x_t, \vartheta, x_{t-1})\,d\vartheta\,dx_{t-1}. \tag{25.123}$$

The recursion also carries the $(x_t, \vartheta)$ marginal needed at the next time. It is therefore a parallel approximate density filter, not a one-time fit to current particle moments. Fitting it to empirical particle moments would destroy the intended independence of the teacher. Fitting $h_t$ to $\bar{q}_t$, rather than to (25.121), would approximately represent $\bar{q}_t^2$ and is wrong relative to this density target.

All ranks, bases, coordinate order, fitting designs, sweep counts, ridge and floor controls, and discrete solver branches are fixed for one score-bearing program. "Fixed branch" means these choices do not change with $\theta$; it does not mean that the fitted cores are numerically constant in $\theta$.

**Proposition 25.22** (Square-root fit target and conditional parameter semantics)**.** *Assume $\bar{q}_t(r) > 0$ on the declared fit rows and let $c_t$ be independent of the fit-row index. If $h_t(r) = s_t(r)$ exactly, then $h_t(r)^2 = \exp(-c_t)\bar{q}_t(r)$. Thus, with $\tau_t = 0$, normalization of $h_t^2$ gives the normalized conditional target. Treating $\theta$ as a chart coordinate and integrating it instead changes the probability measure and defines a different algorithm.*

*Proof.* Squaring (25.121) gives $s_t(r)^2 = \exp(-c_t)\bar{q}_t(r)$. The positive scalar $\exp(-c_t)$ cancels from the normalizing quotient. The conditional statement follows because $\theta$ is fixed in every factor of $q_t$; integrating a variable requires specifying a measure for it and therefore defines a different joint target. For the unscaled defensive weight in (25.122), both components have the common factor $e^{-c_t}$, which cancels. A coefficient $\tau_t$ held fixed while $c_t$ changes does not share this factor and therefore changes the normalized density. □

### 25.17.2 Exact fixed-ALS fit and its forward derivative

Let the frozen fit rows be $r^{(n)}$, with positive weights $w_n$, and write $s_n = s_t(r^{(n)})$. A realized sweep schedule is a fixed sequence of active axes $k_1, \dots, k_U$. Immediately before update $u$, contract all non-active cores at the fit rows to obtain the design $A_u \in \mathbb{R}^{M \times p_u}$. Vectorizing the active core as $c_u$, the declared ridge update is

$$H_u = A_u^\top W A_u + \rho I, \qquad b_u = A_u^\top W s, \qquad H_u c_u = b_u, \tag{25.124}$$

where $W = \mathrm{diag}(w_1, \dots, w_M)$. The implementation may use objective-preserving column scaling and an augmented least-squares solve, but the returned unscaled coefficient vector must solve (25.124). Any condition-number or floor branch is part of the frozen finite route.

For one parameter direction, let dots denote total directional derivatives of the state immediately before update $u$. Because $A_u$ depends on every non-active core, its derivative must be built from their current tangents. The exact update JVP is

$$\dot{H}_u = \dot{A}_u^\top W A_u + A_u^\top W \dot{A}_u + A_u^\top \dot{W} A_u + \dot{\rho} I, \tag{25.125}$$

$$\dot{b}_u = \dot{A}_u^\top W s + A_u^\top \dot{W} s + A_u^\top W \dot{s}, \tag{25.126}$$

$$H_u \dot{c}_u = \dot{b}_u - \dot{H}_u c_u. \tag{25.127}$$

After each value update, both $c_u$ and $\dot{c}_u$ replace the active core and its tangent before constructing update $u+1$. Recomputing every $\dot{c}_u$ from only the terminal fitted cores, or setting $\dot{A}_u = 0$ after another core has acquired a tangent, is a partial derivative and is wrong relative to the fixed-ALS program.

On a locally fixed maximizing row $n_t^*$, the scale and target tangents are

$$\dot{c}_t = \dot{\log \bar{q}}_t(r^{(n_t^*)}), \qquad \dot{s}_n = \tfrac{1}{2} s_n [\dot{\log \bar{q}}_t(r^{(n)}) - \dot{c}_t]. \tag{25.128}$$

The derivative of $\log \bar{q}_t$ includes the analytical model score, any moving-chart/Jacobian term, and, for $t > 0$, the tangent of the normalized previous TT marginal. Fixed reference rows and weights have $\dot{W} = 0$; a parameter-dependent design or ridge must instead include the corresponding terms above. The current reference replay fixes the reference rows, basis, weights, and ridge, hence uses $\dot{W} = 0$ and $\dot{\rho} = 0$, while retaining $\dot{A}_u$ through all non-active core tangents. It cannot be reused for a moving-basis, moving-row, or parameter-dependent-ridge route without adding those missing derivatives. For the scale-consistent defensive convention in (25.122),

$$\dot{\tau}_t = e^{-c_t}(\dot{\lambda}_t - \lambda_t \dot{c}_t) \tag{25.129}$$

including the zero-weight case. Omitting the $-\tau_t \dot{c}_t$ contribution changes the derivative of the represented defensive mixture.

The scale shift is a numerical conditioning device, not a likelihood increment in the hybrid particle route. In particular, its tangent must enter the square-root fit target through (25.128), but neither $c_t$ nor $\log Z_t^{\mathrm{TT}}$ is added to the particle likelihood. The latter is the normalizer of an auxiliary approximate density, whereas the particle increment is defined by the normalized particle weights before the reset.

**Proposition 25.23** (Sequential fixed-ALS JVP)**.** *Assume the update schedule and all discrete branches are locally fixed and every $H_u$ is nonsingular. Starting from the derivative of the declared initial cores, sequential application of (25.127) returns the total directional derivative of every terminal fitted core produced by (25.124).*

*Proof.* At the first update, differentiating $H_1 c_1 = b_1$ gives $H_1 \dot{c}_1 = \dot{b}_1 - \dot{H}_1 c_1$, hence the stated formula. Suppose all core values and tangents before update $u$ equal those of the finite value program. Product-rule contraction of the non-active cores gives the total $\dot{A}_u$; differentiating $H_u c_u = b_u$ gives the stated $\dot{c}_u$ by a linear solve with the same $H_u$. Replacing the active core and tangent establishes the induction hypothesis for update $u+1$. Induction over the finite schedule proves the claim. □

### 25.17.3 Exact observable contractions for a fixed squared TT

Write the square-root TT in a basis $\phi_{k\ell}$ as

$$h(r) = G_1(r_1) \cdots G_m(r_m), \qquad G_k(r_k) = \sum_{\ell=1}^{n_k} C_{k\ell} \phi_{k\ell}(r_k), \tag{25.130}$$

where $C_{k\ell} \in \mathbb{R}^{R_{k-1} \times R_k}$ and $R_0 = R_m = 1$. For a separable observable $g(r) = \prod_k g_k(r_k)$, define

$$M_k[g_k]_{\ell s} = \int \phi_{k\ell}(u) \phi_{ks}(u) g_k(u) \, d\nu_k(u) \tag{25.131}$$

and the paired-core transfer

$$A_k[g_k]_{(a,A),(b,B)} = \sum_{\ell,s} C_k[a,\ell,b] C_k[A,s,B] M_k[g_k]_{\ell s}. \tag{25.132}$$

**Proposition 25.24** (Squared-TT observable contraction)**.** *For* (25.130) *and any integrable separable* $g$*,*

$$\int g(r) h(r)^2 \, d\nu(r) = e_1^\top A_1[g_1] \cdots A_m[g_m] e_1. \tag{25.133}$$

*Consequently*

$$\mathbb{E}_{\widehat{q}^{\mathrm{TT}}}[g] = \frac{e_1^\top A_1[g_1] \cdots A_m[g_m] e_1 + \tau \int g q_0 \, d\nu}{e_1^\top A_1[1] \cdots A_m[1] e_1 + \tau \int q_0 \, d\nu}. \tag{25.134}$$

*Proof.* Expand both copies of $h$ in their basis and TT-rank indices. Product measure and separability allow Fubini's theorem to split every basis-pair integral by coordinate. The integral at coordinate $k$ is exactly $M_k[g_k]$; summing the two TT rank paths gives (25.132). Left-to-right contraction proves (25.133). Linearity adds the defensive term, and division by the same expression with $g = 1$ proves (25.134). □

It is wrong to contract unsquared TT cores with integral vectors: moments of $h^2$ require paired cores and (25.131). It is also wrong to omit the $\tau q_0$ numerator term when $\tau > 0$.

For coordinate monomials choose $g_k(u) = u^{\alpha_k}$. For an affine current-state chart

$$x_t = a_t + B_t r_t, \tag{25.135}$$

the contracted reference mean $m_r$ and covariance $P_r$ give

$$\mu_t^{\mathrm{TT}} = a_t + B_t m_r, \qquad P_t^{\mathrm{TT}} = B_t P_r B_t^\top. \tag{25.136}$$

**Proposition 25.25** (Mean and covariance of the TT teacher)**.** *If* $Z_t^{\mathrm{TT}} > 0$*, the monomial contractions and* (25.136) *equal the mean and covariance of the normalized finite density* $\widehat{q}_t^{\mathrm{TT}}$ *under the declared affine chart.*

*Proof.* Apply Proposition 25.24 with $g = r_i$ and $g = r_i r_j$. These give $m_r$ and $P_r = \mathbb{E}[r r^\top] - m_r m_r^\top$. Standard affine moment identities then give (25.136). □

### 25.17.4 Low-rank contractions for whitened higher moments

Let $L_t^{\mathrm{TT}}(L_t^{\mathrm{TT}})^\top = P_t^{\mathrm{TT}}$ and define

$$z = (L_t^{\mathrm{TT}})^{-1}(x_t - \mu_t^{\mathrm{TT}}). \tag{25.137}$$

Each component is an affine form in the reference coordinates, $z_i = c_i + \sum_k a_{ik} r_k$. Directly forming every raw third- and fourth-order tensor would require $O(d^3)$ and $O(d^4)$ outputs. That is not the proposed high-dimensional algorithm.

For nonnegative integers $p, q$, maintain a bivariate polynomial state $v_{ab} = S^a T^b$, $0 \le a \le p$, $0 \le b \le q$. When coordinate $r_k$ adds $\alpha_k r_k$ to $S$ and $\beta_k r_k$ to $T$, the exact local update is

$$(S + \alpha_k r_k)^a (T + \beta_k r_k)^b = \sum_{u=0}^{a} \sum_{v=0}^{b} \binom{a}{u} \binom{b}{v} S^u T^v \alpha_k^{a-u} \beta_k^{b-v} r_k^{a-u+b-v}. \tag{25.138}$$

Initialize with $S = c_i, T = c_j$, apply (25.138) over the coordinates, and select state $(p, q)$. Replacing each $r_k^s$ by the operator matrix $M_k[r_k^s]$ couples this fixed-size automaton to the paired-core contraction. Its auxiliary dimension is at most $(p+1)(q+1)$, hence at most 9 for the pairwise moments below.

**Proposition 25.26** (Affine-form mixed moments without a dense moment tensor)**.** *Assume $M_k[r_k^s]$ is exact for $0 \le s \le p+q$. The coupled paired-core/polynomial-automaton contraction returns*

$$\int (c_i + a_i^\top r)^p (c_j + a_j^\top r)^q h(r)^2 \, d\nu(r) \tag{25.139}$$

*exactly for the represented TT $h$, without constructing a dense third- or fourth-order moment tensor.*

*Proof.* Equation (25.138) is the binomial theorem in the two accumulated affine forms. Induction over $k$ shows that the selected final state is exactly the product in (25.139). Expanding that automaton path and the two copies of the TT, then applying Fubini coordinate by coordinate, replaces each local monomial by its exact operator matrix. The remaining rank contractions are precisely the stated integral. □

The teacher supplies

$$\gamma_i^{\mathrm{TT}} = \mathbb{E}[z_i^3], \quad \kappa_i^{\mathrm{TT}} = \mathbb{E}[z_i^4], \quad C_{ij}^{(3),\mathrm{TT}} = \mathbb{E}[z_i^2 z_j], \quad C_{ij}^{(4),\mathrm{TT}} = \mathbb{E}[z_i^2 z_j^2]. \tag{25.140}$$

The two pairwise quantities have different index semantics. The co-skew term $C_{ij}^{(3),\mathrm{TT}} = \mathbb{E}[z_i^2 z_j]$ is ordered: requesting $(i, j)$ does not request $(j, i)$, which would instead be $\mathbb{E}[z_j^2 z_i]$. The co-kurtosis term is symmetric, so one requested unordered pair $\{i, j\}$ supplies both matrix entries $C_{ij}^{(4),\mathrm{TT}} = C_{ji}^{(4),\mathrm{TT}}$. The implementation therefore maintains separate structural masks for ordered co-skew and symmetric co-kurtosis entries; uncomputed entries are never silently interpreted as zero targets. Only declared pairs $(i, j) \in \mathcal{E}$ need be contracted. All pairs give $O(d^2)$ outputs and at least $O(d^2)$ work even though no dense fourth-order tensor is formed. For a sparse interaction graph or other predeclared structural set with $|\mathcal{E}| = O(d)$, the number of requested shape moments remains linear in $d$. Pair selection is fixed outside HMC and cannot be tuned on the claim path.

### 25.17.5 Composition with Contract E

The default composition is deliberately hybrid. Let $\mu_t^w, P_t^w$ be the likelihood-weighted particle moments from (25.85). Contract E and the final affine map keep $\mu_t^w, P_t^w$ as their first-two-moment targets. Only the standardized shape targets in (25.140) replace their empirical counterparts in the bounded correction (25.94)–(25.98). This keeps the current Contract E mean/covariance claim intact while giving the shape correction information not computed from the same particle cloud.

Using $\mu_t^{\mathrm{TT}}, P_t^{\mathrm{TT}}$ themselves as reset targets is a different algorithm. It may be tested as an ablation, but it does not preserve the particle weighted covariance and must not be silently substituted for the hybrid route.

**Proposition 25.27** (Hybrid first-two-moment preservation)**.** *Assume $P_t^w$ and every correction covariance are positive definite. If the bounded correction is rewhitened after each iteration and mapped back with the Cholesky factor of $P_t^w$, the final equally weighted cloud has mean $\mu_t^w$ and covariance $P_t^w$, irrespective of the TT shape targets.*

*Proof.* The TT targets affect only the finite correction directions and coefficients. After the final correction, Proposition 25.20 gives zero mean and identity covariance in standardized coordinates. Mapping back by $\mu_t^w$ and the Cholesky factor of $P_t^w$ proves the result. □

The higher moments are not guaranteed to match (25.140): the correction uses damped finite steps, moment constraints can conflict, and rewhitening changes higher moments. Post-correction residuals are mandatory diagnostics.

### 25.17.6 Likelihood, score, and derivative statements

The particle likelihood increment $\widehat{\ell}_t^N$ is computed before the TT-driven reset. Adding the teacher after that increment leaves the already computed scalar at time $t$ unchanged. The reset changes the cloud carried to $t+1$, so it can and generally will change future approximate increments. There is no theorem that it preserves the full filtering distribution, the complete finite-horizon particle likelihood, or the exact model likelihood.

For a core direction $\dot{C}_k$, operator direction $\dot{M}_k$, and transfer $A_k = A_k(C_k, M_k)$, the product rule gives

$$\dot{A}_k = A_k(\dot{C}_k, C_k, M_k) + A_k(C_k, \dot{C}_k, M_k) + A_k(C_k, C_k, \dot{M}_k), \tag{25.141}$$

and, for $H_g = e_1^\top \prod_k A_k e_1$,

$$\dot{H}_g = \sum_{j=1}^{m} e_1^\top A_1 \cdots A_{j-1} \dot{A}_j A_{j+1} \cdots A_m e_1. \tag{25.142}$$

If $N_g = H_g + \tau Q_{0,g}$, $Z = N_1$, then

$$\frac{d}{d\theta}\mathbb{E}_{\widehat{q}^{\mathrm{TT}}}[g] = \frac{\dot{N}_g Z - N_g \dot{Z}}{Z^2}, \qquad \dot{N}_g = \dot{H}_g + \dot{\tau} Q_{0,g} + \tau \dot{Q}_{0,g}. \tag{25.143}$$

The same product rule applies to polynomial-automaton transfers, including chart Jacobian, Cholesky, and affine-form coefficient derivatives. For a non-affine chart, a whitened

state component is not generally a polynomial in the reference coordinates. The polynomial automaton is then valid only when the transformed observable is represented in a declared finite basis; otherwise fixed quadrature must construct the operator matrices, and its error is an additional approximation that must be measured.

**Proposition 25.28** (Total score of the fixed TT-teacher finite program)**.** *Fix the TT bases, ranks, fitting designs, sweep counts, regularization, coordinate ordering, pair set $\mathcal{E}$, particle innovations, OT controls, and all discrete branches. Assume every finite fit solve and Cholesky factor is nonsingular near $\theta$. If the recursive tangent includes (i) every fitted-core tangent, including dependence through the previous TT marginal; (ii) every observable, normalizer, chart, whitening, and defensive-density term in (25.141)–(25.143); and (iii) the complete particle, OT, Contract E, and correction tangent, then the accumulated score equals the total derivative of the same executed finite likelihood program.*

*Proof.* On the fixed branch, each TT fit is a finite composition of smooth target evaluations and nonsingular fixed-design solves. Equations (25.141)–(25.143) are the product and quotient rules for the exact finite contractions. Cholesky and triangular-solve JVPs differentiate whitening, and Proposition 25.21 differentiates the particle/OT/reset composition. Induction over time includes the dependence of the next TT fit on the previous TT marginal and of the next particle increment on the reset cloud. The chain rule gives the derivative of the same finite scalar returned by the value program. □

A tangent through only the final TT contraction, treating fitted cores as constants, is a partial derivative and is wrong relative to this total-score claim. A pre-fitted teacher frozen across $\theta$ defines a different finite approximation; a zero target tangent can be correct for that explicitly frozen approximation but is not the score of the recursively refitted teacher. Runtime rank selection, stochastic fitting choices, finite-difference production scores, and an autodiff-only score route are ineligible for the intended HMC path.

### 25.17.7 Complete algorithm and complexity boundary

Before the first observation, freeze the basis family, ranks, fit and holdout rows, fit weights, coordinate charts, ALS sweep schedule, ridge and conditioning policy, defensive density, ordered co-skew mask, symmetric co-kurtosis mask, particle innovations, OT controls, Contract E controls, and bounded-correction controls. These are part of the finite-program identity; none may be reselected as $\theta$ changes. Initialize the TT teacher from the declared initial-state density and initialize the particle cloud by the ordinary fixed particle route.

For each observation time $t$, execute the following ordered program:

(1) Hold the outer HMC parameter $\theta$ fixed. Evaluate the conditional adjacent log target from (25.119). At $t = 0$, replace the previous TT marginal by the declared initial density. At $t > 0$, evaluate the normalized carried TT marginal and its quotient-rule tangent. Never add $\theta$ as a TT integration coordinate.

(2) Pull the target back through the complete chart using (25.120). Include the chart Jacobian, inverse reference density, and all their tangents. Select the fixed local maximizing row and construct $s_t, \dot{s}_t$ using (25.121) and (25.128).

(3) Starting from the declared value and tangent cores, replay every ALS update in its frozen order. At update $u$, rebuild $A_u$ and $\dot{A}_u$ from the current cores, solve (25.124), solve (25.127), then immediately replace both the active core and its tangent. Veto a non-finite value, branch change, excessive condition number, or failed primal/JVP algebraic residual.

(4) Form $\widetilde{q}_t = h_t^2 + \tau_t q_{0,t}$, its normalizer, and their tangents. Contract the previous-state coordinates and any genuine carried latent coordinates, retaining the normalized $(x_t, \vartheta)$ TT marginal and its tangent for time $t+1$.

(5) By paired-core contractions, compute the teacher mean and covariance. Differentiate the covariance, Cholesky factor, and triangular solves to obtain the whitening coefficients and their tangents. Contract diagonal skew/kurtosis and only the declared ordered co-skew and symmetric co-kurtosis entries, including the defensive numerator, normalizer, and quotient tangents.

(6) Independently propagate the particle rows with the frozen innovations, evaluate their observation weights, and record $\widehat{\ell}_t^N$, the normalized weights, and their analytical tangents. This particle scalar, not $c_t + \log Z_t^{\mathrm{TT}}$, is the hybrid likelihood increment.

(7) Execute the declared OT quotient and canonical Contract E–Chol reset. This produces an equally weighted cloud with the particle weighted mean and covariance; propagate the total particle/weight/transport tangent.

(8) Apply the fixed number of bounded higher-moment correction steps using the TT shape values and tangents. Rewhiten after every step, map back with the particle weighted mean and covariance, and record every post-correction target residual. The residuals are diagnostics; the method does not claim exact higher-moment matching.

(9) Carry the corrected equal-weight particle cloud and tangent, together with the normalized TT $(x_t, \vartheta)$ marginal and tangent, to time $t+1$. After the final observation return $\sum_t \widehat{\ell}_t^N$ and the sum of its analytically propagated tangents.

This order makes the two probability calculations explicit. The particle lane owns the finite likelihood value and score. The TT lane owns only an auxiliary normalized filtering approximation used to define shape targets. The two lanes meet at the reset after the current particle increment has been recorded; therefore the TT normalizer cannot be substituted for the particle increment without changing the algorithm.

For maximum basis size $n$, maximum TT rank $R$, and observable-automaton dimension $\rho \le 9$, the density stores $O(mnR^2)$ coefficients and a contraction has working state $O(\rho R^2)$. A conservative dense local contraction costs up to $O(m\rho^2 n^2 R^4)$, with lower cost available from contraction ordering and orthogonal bases. Mean/covariance output costs $O(d^2)$. Selected moments cost at least $O(|\mathcal{E}|)$ outputs. These costs do not depend on particle count $N$, but the per-step TT fit can dominate them and TT rank can grow sharply with nonlinear coupling. No dimension-independent rank bound is claimed for the models of interest.

**Correctness and promotion boundary.** The propositions prove exact moments and their JVPs for a fixed represented squared-TT density, and exact first-two particle moments after the hybrid reset. They do not prove that the TT density approximates the true filter, that its rank stays bounded, that the correction reaches its TT targets, that likelihood or score bias decreases, or that one TT fit per step is affordable inside a high-dimensional nonlinear HMC evaluation. Dense-quadrature parity, fixed-branch derivative parity, LGSSM/Kalman checks, nonlinear value/score no-regression, and GPU/XLA memory/runtime evidence remain separate gates.

The current variable-rank TT containers and the reference fixed-ALS replay in `bayesfilter/highdi` store cores as a Python tuple, so a straight implementation of the products above uses a setup-static Python loop that TensorFlow traces into an unrolled graph. That is sufficient for an independent TensorFlow mechanics reference, but it is ineligible for the BayesFilter XLA runtime under the no-Python-loop policy. Candidate runtime promotion additionally requires a padded/masked or otherwise batch-native core representation whose contraction loop is a TensorFlow control-flow operation. The reference modules implement the conditional square-root target, sequential fixed-ALS value/JVP equations, normalized carried marginal, and TT shape-target JVP in a setup-static recursion. A separate setup-static composition accepts the particle likelihood increment and tangent, runs the existing OT and Contract E–Chol JVP, and applies the TT shape targets while returning the particle scalar unchanged. These references are not a model-scale filter and do not yet implement the graph-native form.

For the padded candidate, let every core be embedded in $\widetilde{G}_k \in \mathbb{R}^{R\times B\times R}$, and let a fixed mask $P_k$ select the entries of the variable-rank core. All inactive entries of $\widetilde{G}_k$ and $\dot{\widetilde{G}}_k$ are zero. At an update of axis $k$, the padded design $\widetilde{A}_k$ has zero columns outside $P_k$. More precisely, for $M$ prepared rows and $p = R^2B$, let $\widetilde{A}_k \in \mathbb{R}^{M\times p}$, $W = \operatorname{diag}(w_1,\dots,w_M) \in \mathbb{R}^{M\times M}$, $s \in \mathbb{R}^M$, and let $P_k \in \mathbb{R}^{p\times p}$ be the diagonal zero–one projector onto the active core entries. Then $\widetilde{A}_k = \widetilde{A}_k P_k$. For a fixed $\rho > 0$, the padded ridge equations split as

$$\left(\widetilde{A}_k^\top W \widetilde{A}_k + \rho I\right)\widetilde{g}_k = \widetilde{A}_k^\top W s, \qquad \widetilde{g}_k = P_k \widetilde{g}_k \in \mathbb{R}^p, \tag{25.144}$$

because every inactive equation is $\rho\widetilde{g}_{k,j} = 0$, while the active block is exactly equation (25.124). Thus padding does not alter the represented update when the ridge is strictly positive. The zero-ridge case is not part of this candidate: choosing a pseudoinverse convention for the inactive block would define a different finite numerical program.

Let $S_k$ be the fixed-branch positive diagonal column scale computed from the primal padded design. The candidate solves

$$\left[S_k^{-1}\left(\widetilde{A}_k^\top W \widetilde{A}_k + \rho I\right) S_k^{-1}\right] z_k = S_k^{-1}\widetilde{A}_k^\top W s, \qquad \widetilde{g}_k = S_k^{-1} z_k. \tag{25.145}$$

This is an invertible change of coordinates, not a changed objective. The JVP is still equation (25.127) embedded in the padded space; differentiating the auxiliary scale is unnecessary because the claimed coefficient is characterized by the unscaled normal equations. The graph propagates the left and right environments and their tangents with TensorFlow control flow and consumes a fixed update schedule. It separately gates the condition of the scaled primal system and the unscaled derivative system, and records both algebraic residuals. Masks, ranks, bases, rows, schedule, ridge, and fit count are fixed before tracing. Hence this construction removes the Python core/sweep loops without introducing parameter-dependent branch or rank selection; it does not by itself establish that the fitted TT is an accurate filtering approximation.

# Chapter 26

# Custom Gradients for LEDH-PFPF-OT

The preceding chapters introduced three ingredients separately: local exact Daum–Huang particle flow, proposal-corrected PF-PF weighting, and entropy-regularized OT resampling. In the OT sequence immediately before this chapter, the deterministic baseline fixed the transport-object and barycentric-output contract, while the entropic chapter fixed the exact teacher object, the finite Sinkhorn route that approximates it, and the rule that later differentiation claims must attach to that executed finite route. This chapter explains how those pieces can be differentiated without asking an automatic-differentiation system to retain the entire filtering graph in memory. The key object is a custom vector-Jacobian product, or VJP, for the executed LEDH-PFPF-OT scalar.

The chapter is intentionally narrow. It does not claim that the relaxed OT-resampled particle filter is the same object as categorical resampling. It does not claim that finite Sinkhorn iteration is the exact unregularized OT optimizer. It does not claim HMC correctness for a nonlinear model. Its claim is more basic and more useful: if the value path is a declared finite LEDH-PFPF-OT computation, then the gradient path must be the derivative of that same computation, and the expensive OT block has a structured adjoint.

## 26.1 The scalar whose derivative is wanted

Let $\theta$ denote the model parameter and let all random numbers used by the diagnostic or sampler target be fixed as part of the deterministic value path. For one filtering pass, write the executed scalar as

$$\mathcal{L}_K(\theta) = \mathrm{Reduce}\left(\text{LEDH-PFPF-OT}_K(\theta; y_{1:T}, \xi)\right), \tag{26.1}$$

where $\xi$ denotes the fixed particle and process-noise streams, $K$ denotes the finite Sinkhorn or annealed-transport iteration budget, and Reduce is the declared scalar reduction, such as a mean over fixed seeds. The derivative needed by HMC or optimization is

$$\nabla_\theta \mathcal{L}_K(\theta),$$

not the derivative of a different resampling rule, a different Sinkhorn stopping policy, or a different random seed construction.

**Definition 26.1** (Same-scalar custom-gradient contract)**.** *A custom-gradient implementation for LEDH-PFPF-OT is same-scalar if its primal return is exactly $\mathcal{L}_K(\theta)$ and its VJP returns*

$$v^\top D_\theta \mathcal{L}_K(\theta)$$

*for every upstream scalar multiplier $v$. For a scalar log target the usual case is $v = 1$.*

This contract is the same HMC principle as Chapter 13, but the present chapter specializes it to the LEDH-PFPF-OT computation. The particle-flow part follows the Li–Coates invertible-particle-flow proposal logic [Li and Coates, 2017]; the OT relaxation follows the entropy-regularized resampling route of Corenflos et al. [2021], with the Sinkhorn background of Cuturi [2013] and Peyré and Cuturi [2019]. The derivative identities below are project derivations for the declared BayesFilter finite program; the citations explain the particle-flow, entropy-regularized OT, and Sinkhorn objects being differentiated.

## 26.2 Forward decomposition

At one observation time, the canonical value path is transition-first and can be written as a composition

$$\theta \longmapsto (X^-, \log w^-, A_{\text{flow}}) \longmapsto (X, \log w) \longmapsto (\Delta_t, X^\star, \log w^\star) \longmapsto \mathcal{L}_t. \tag{26.2}$$

Here $X^- \in \mathbb{R}^{N\times d}$ is the pre-observation proposal cloud, $\log w^-$ are proposal-correction log weights, and $A_{\text{flow}}$ denotes the LEDH auxiliary quantities needed for the proposal density and log-determinant correction. The local LEDH construction uses particle-local linearizations and particle-local covariance state as in Li–Coates PF-PF. Corrected logits first produce the current likelihood increment $\Delta_t$ and normalized weights. Only after that increment has entered the scalar does an active reset map the weighted cloud to an equal-weight cloud

$$(X, \log w) \longmapsto X^\star.$$

This chapter focuses on the last map because it is the part that can dominate memory. Naive reverse-mode autodiff through all pairwise costs, Sinkhorn updates, and barycentric products may retain objects of size $O(KN^2)$. A custom VJP instead treats the reset block as a finite differentiable program and applies its adjoint by recomputation, streaming reductions, or checkpointing.

The canonical reset in this chapter is Contract E–Chol. In the current repository policy, the canonical reset identifier is `contract_e_chol_v1`, and the canonical total-derivative composition identifier is `contract_e_chol_total_direct_moments_weights_plus_streaming`. A raw barycentric cloud, compact forward sensitivity of that raw scalar, or full-history/manual reverse derivative of that raw scalar is historical diagnostic evidence only; the repository route label for those historical value- correct but noncanonical reset semantics is `historical_raw_barycentric_diagnostic_only`. Even an algebraically correct derivative of the raw-barycentric scalar is wrong relative to the canonical Contract E total-gradient claim. There is no canonical-to-raw fallback.

## 26.3 Canonical row quotient and Contract E composition

The executed finite streaming transport does not get to assume its intended row marginal is exact. It emits

$$M_i = \sum_j P_{ij}, \qquad Q_i = \sum_j P_{ij} X_j, \qquad Y_i^+ = Q_i / M_i. \tag{26.3}$$

The canonical route has no denominator floor. For a tangent and an upstream cotangent,

$$\dot{Y}_i^+ = \frac{\dot{Q}_i - Y_i^+ \dot{M}_i}{M_i}, \qquad G_i^Q = G_i^{Y^+} / M_i, \qquad G_i^M = -\langle G_i^{Y^+}, Y_i^+ \rangle / M_i. \tag{26.4}$$

Both $G^Q$ and $G^M$ must enter the finite Sinkhorn pullback.

Contract E consumes the same source cloud $X$, normalized probabilities $w$, row-quotient cloud $Y^+$, fixed residual design $\Xi$, and prepared ridge $\lambda$. It depends directly on the weighted source moments as well as indirectly through transport. In row-vector storage its local moments and reset are

$$\mu_w = \sum_i w_i X_i, \qquad \Sigma_w = \sum_i w_i (X_i - \mu_w)(X_i - \mu_w)^\top, \tag{26.5}$$

$$\bar{Y}^+ = N^{-1} \sum_i Y_i^+, \qquad \Sigma_+ = N^{-1} \sum_i (Y_i^+ - \bar{Y}^+)(Y_i^+ - \bar{Y}^+)^\top, \tag{26.6}$$

$$G_+ = \operatorname{sym}(\Sigma_w - \Sigma_+), \quad B_\lambda = \operatorname{chol}(G_+ + \lambda I), \quad \widetilde{Y} = Y^+ + \Xi B_\lambda^\top, \tag{26.7}$$

$$\widetilde{\Sigma} = \frac{1}{N} \sum_i (\widetilde{Y}_i - \bar{\widetilde{Y}})(\widetilde{Y}_i - \bar{\widetilde{Y}})^\top, \quad A_\lambda = \operatorname{chol}(\Sigma_w + \lambda I) \operatorname{chol}(\widetilde{\Sigma} + \lambda I)^{-1}, \tag{26.8}$$

$$X_i^\star = \mu_w + (\widetilde{Y}_i - \bar{\widetilde{Y}}) A_\lambda^\top. \tag{26.9}$$

The implementation evaluates the last operator by a triangular solve rather than forming an explicit inverse. Its cloud VJP first returns

$$G_X^{\mathrm{mom}}, \quad G_w^{\mathrm{mom}}, \quad G_{Y^+}, \quad G_\Xi, \quad g^\lambda.$$

Composing $G_{Y^+}$ through (26.4) and the finite Sinkhorn program returns transport terms. In common coordinates the canonical totals are

$$G_X = G_X^{\mathrm{mom}} + G_X^{\mathrm{transport}}, \qquad G_w = G_w^{\mathrm{mom}} + G_w^{\mathrm{transport}}. \tag{26.10}$$

The implementation naturally receives a normalized-log-weight cotangent from transport. It therefore forms

$$G_{\log w} = G_{\log w}^{\mathrm{transport}} + w \odot G_w^{\mathrm{mom}} \tag{26.11}$$

before applying the log-normalization pullback exactly once. A probability-coordinate moment adjoint must not be added directly to a log-weight adjoint, and normalization must not be applied twice. The residual and ridge adjoints are returned for parity, but the canonical route fixes their prepared values, so $\dot{\Xi} = 0$ and $\dot{\lambda} = 0$. A future parameter-dependent design or ridge would be a different route and would have to differentiate that dependence fully.

## 26.4 Transport orientation

The OT chapters used a coupling $\Pi$ with source index first and output slot second. The TensorFlow LEDH-PFPF-OT route is more naturally described by the row-stochastic transport matrix

$$T = N\Pi^\top, \qquad T\mathbf{1} = \mathbf{1}, \qquad T^\top \mathbf{1} = Nw, \tag{26.12}$$

where $w = \mathrm{softmax}(\log w)$. The transported equal-weight cloud is

$$\widetilde{X} = TX, \qquad \widetilde{x}_j = \sum_{i=1}^{N} T_{ji} x_i. \tag{26.13}$$

Thus row $j$ of $T$ tells how output slot $j$ averages the source particles. The row sums make each output particle a barycenter. The column sums say that source particle $i$ contributes total mass proportional to its weight.

**Remark 26.1** (Why orientation matters)**.** *If a derivation uses* $\Pi$ *but the code applies* $TX$*, transpose errors are easy to introduce. The VJP formulas below use* $T$*, not* $\Pi$*. A reader who wants the coupling form can substitute* $T = N\Pi^\top$*.*

## 26.5 VJP of the barycentric product

The simplest and most important adjoint identity is the VJP of the barycentric matrix product.

**Proposition 26.1** (Barycentric VJP)**.** *Let* $\widetilde{X} = TX$*, with* $T \in \mathbb{R}^{N\times N}$ *and* $X \in \mathbb{R}^{N\times d}$*. Use the Frobenius inner product for matrix adjoints, and let* $G_{\widetilde{X}} \in \mathbb{R}^{N\times d}$ *be the upstream adjoint for* $\widetilde{X}$*. Then the direct adjoints are*

$$G_X^{\mathrm{direct}} = T^\top G_{\widetilde{X}}, \qquad G_T = G_{\widetilde{X}} X^\top. \tag{26.14}$$

*Proof.* Taking a differential gives

$$d\widetilde{X} = dT\, X + T\, dX.$$

Pairing with $G_{\widetilde{X}}$ under the Frobenius inner product,

$$\begin{aligned} \langle G_{\widetilde{X}}, d\widetilde{X}\rangle &= \langle G_{\widetilde{X}}, dT\, X\rangle + \langle G_{\widetilde{X}}, T\, dX\rangle \\ &= \langle G_{\widetilde{X}} X^\top, dT\rangle + \langle T^\top G_{\widetilde{X}}, dX\rangle. \end{aligned}$$

The coefficients of $dT$ and $dX$ are exactly (26.14). □

Equation (26.14) is only the direct part of the particle adjoint. Because $T$ itself depends on $X$ and $\log w$, the complete VJP also pulls $G_T$ backward through the Sinkhorn or annealed-transport construction.

## 26.6 A normalized transport matrix

For fixed final potentials $f, g \in \mathbb{R}^N$, fixed normalized log weights $\log w_i$, and cost $C_{ji} = c(z_j, z_i)$, the executed row-stochastic transport can be written in column-normalized form

$$T_{ji} = N w_i \frac{\exp\{(f_j + g_i - C_{ji})/\varepsilon\}}{\sum_{r=1}^{N} \exp\{(f_r + g_i - C_{ri})/\varepsilon\}}. \tag{26.15}$$

This expression enforces $T^\top \mathbf{1} = Nw$ exactly for the displayed normalization. The row condition $T\mathbf{1} \approx \mathbf{1}$ is enforced by the potentials produced by finite Sinkhorn or annealed Sinkhorn iteration. In the TensorFlow implementation this is the convention used by the final transport application: $f$ indexes output rows, $g$ indexes source columns, the normalization is a column log-sum-exp over output rows, and the transported particles are obtained by the matrix product $TX$.

Define

$$p_{ji} = \frac{T_{ji}}{N w_i}, \qquad \sum_{j=1}^{N} p_{ji} = 1. \tag{26.16}$$

The vector $p_{\cdot i}$ is the normalized column distribution associated with source particle $i$.

**Proposition 26.2** (VJP of the normalized transport output)**.** *At fixed finite values of $f, g \in \mathbb{R}^N$, $C \in \mathbb{R}^{N\times N}$, $\varepsilon > 0$, and normalized weights $w_i > 0$, $\sum_i w_i = 1$, let $T$ be defined by (26.15). Let $G_T \in \mathbb{R}^{N\times N}$ be the upstream adjoint for $T$. For each source column $i$, set*

$$\bar{G}_i = \sum_{r=1}^{N} p_{ri} (G_T)_{ri}, \qquad S_{ji} = T_{ji}\big((G_T)_{ji} - \bar{G}_i\big). \tag{26.17}$$

*Then the VJP of (26.15) satisfies*

$$(G_{\log w})_i \mathrel{+}= \sum_{j=1}^{N} (G_T)_{ji} T_{ji}, \tag{26.18}$$

$$(G_f)_j \mathrel{+}= \frac{1}{\varepsilon} \sum_{i=1}^{N} S_{ji}, \tag{26.19}$$

$$(G_g)_i \mathrel{+}= \frac{1}{\varepsilon} \sum_{j=1}^{N} S_{ji}, \tag{26.20}$$

$$(G_C)_{ji} \mathrel{+}= -\frac{1}{\varepsilon} S_{ji}. \tag{26.21}$$

*If $\log w$ is itself produced by a* logsoftmax*, then (26.18) must be composed with the* logsoftmax *VJP.*

*Proof.* Let

$$a_{ji} = \frac{f_j + g_i - C_{ji}}{\varepsilon}, \qquad p_{ji} = \frac{\exp(a_{ji})}{\sum_r \exp(a_{ri})}, \qquad T_{ji} = N w_i p_{ji}.$$

For a perturbation of one column $i$,

$$dT_{ji} = T_{ji} \left[ d\log w_i + da_{ji} - \sum_{r=1}^{N} p_{ri} da_{ri} \right].$$

Pairing this expression with $(G_T)_{ji}$ and collecting the coefficient of $da_{ji}$ gives $S_{ji} = T_{ji}((G_T)_{ji} - \bar{G}_i)$. Since

$$da_{ji} = \frac{1}{\varepsilon}(df_j + dg_i - dC_{ji}),$$

the adjoints for $f$, $g$, and $C$ are (26.19)–(26.21). The coefficient of $d \log w_i$ is $\sum_j (G_T)_{ji} T_{ji}$, giving (26.18). □

This proposition is the core OT VJP product. It says that the upstream matrix $G_T$ should first be column-centered under the transport-induced probability $p_{\cdot i}$. Only after that centering should the adjoint be pushed into the potentials and costs. In the displayed column-normalized map, $\sum_j S_{ji} = 0$, so the final-potential adjoint $G_g$ is identically zero at this last normalization stage. This is not a bug: adding the same $g_i$ to every logit in one normalized column does not change that column's probabilities. Earlier Sinkhorn-loop adjoints may still involve the variables that produced $g$.

## 26.7 VJP of the quadratic cost

For the common squared-distance cost

$$C_{ji} = \frac{1}{2}\|z_j - z_i\|^2,$$

where the same scaled particle cloud $Z$ supplies the row and column points, the cost adjoint has another simple form.

**Proposition 26.3** (Quadratic-cost VJP)**.** *Let $G_C$ be the upstream adjoint for $C_{ji} = \frac{1}{2}\|z_j - z_i\|^2$, where $Z = (z_1, \dots, z_N)^\top \in \mathbb{R}^{N \times d}$. The contribution to the adjoint of $z_k \in \mathbb{R}^d$ is*

$$(G_Z)_k \mathrel{+}= \sum_{i=1}^{N} (G_C)_{ki}(z_k - z_i) - \sum_{j=1}^{N} (G_C)_{jk}(z_j - z_k). \tag{26.22}$$

*Proof.* The differential of one cost entry is

$$dC_{ji} = (z_j - z_i)^\top (dz_j - dz_i).$$

Multiplying by $(G_C)_{ji}$, summing over $j, i$, and collecting the coefficient of $dz_k$ gives the two sums in (26.22): the first from terms where $k$ is the row index, and the second from terms where $k$ is the column index. □

If centering, scaling, or key construction is stopped by design, the VJP stops there too. If the selected mode differentiates through centering or scaling, then (26.22) must be further composed with the VJP of those preprocessing maps.

## 26.8 Softmin and Sinkhorn adjoints

The finite Sinkhorn or annealed-transport step is a loop of softmin operations. The exact schedule can differ between a fixed-target Sinkhorn route and the FilterFlow-style annealed route, but the local adjoint primitive is the same.

**Definition 26.2** (Softmin block)**.** *For a query index $j$, costs $C_{jr}$, values $v_r$, and temperature $\varepsilon > 0$, define*

$$m_j = -\varepsilon \log \sum_{r=1}^{N} \exp\{(v_r - C_{jr})/\varepsilon\}. \tag{26.23}$$

**Proposition 26.4** (Softmin VJP for fixed temperature)**.** *Assume $\varepsilon > 0$ is held fixed and all displayed exponentials are finite. Let $G_m \in \mathbb{R}^N$ be the upstream adjoint for $m$, and define*

$$q_{jr} = \frac{\exp\{(v_r - C_{jr})/\varepsilon\}}{\sum_s \exp\{(v_s - C_{js})/\varepsilon\}}.$$

*For fixed $\varepsilon$,*

$$(G_v)_r \mathrel{+}= -\sum_{j=1}^{N} (G_m)_j q_{jr}, \tag{26.24}$$

$$(G_C)_{jr} \mathrel{+}= (G_m)_j q_{jr}. \tag{26.25}$$

*Proof.* Differentiating (26.23) gives

$$dm_j = -\sum_r q_{jr}(dv_r - dC_{jr}) = \sum_r q_{jr} dC_{jr} - \sum_r q_{jr} dv_r.$$

Pairing with $G_m$ and collecting coefficients yields (26.24) and (26.25). □

The sign and scale in Proposition 26.4 are for a potential-scale value $v$. Some implementations, including the TensorFlow helper used in the current BayesFilter route, call the same operation with a log-scale input $h_r$:

$$m_j = -\varepsilon \log \sum_r \exp\{h_r - C_{jr}/\varepsilon\}.$$

This is the same formula with $v_r = \varepsilon h_r$. Therefore the local VJP with respect to $h_r$ is $(G_h)_r \mathrel{+}= -\varepsilon \sum_j (G_m)_j q_{jr}$, while the VJP with respect to a potential-scale component $v_r$ remains (26.24). If $h_r = \log \alpha_r + b_r/\varepsilon$, then the adjoint to $b_r$ again has the potential-scale coefficient $-\sum_j (G_m)_j q_{jr}$.

For an annealed schedule, the loop state contains several potentials, for example $(a_y, b_x, a_x, b_y)$. Abstractly write the finite loop as

$$s_{k+1} = F_k(s_k; Z, \log w, \varepsilon_k), \qquad k = 0, \ldots, K-1. \tag{26.26}$$

The reverse loop is then

$$(G_{s_k}, G_Z, G_{\log w}) \mathrel{+}= DF_k(s_k; Z, \log w, \varepsilon_k)^\top G_{s_{k+1}}, \qquad k = K-1, \ldots, 0. \tag{26.27}$$

Each multiplication by $DF_k^\top$ is implemented by applying the softmin VJP above, the averaging VJP for damped updates, and the cost VJP for the pairwise squared distances. This is ordinary reverse-mode differentiation of a finite program, but expressed as an explicit adjoint scan rather than as a raw tape over the whole program.

**Proposition 26.5** (Finite-Sinkhorn VJP by reverse scan)**.** *Assume the finite transport block is the deterministic program*

$$(T, s_K) = \mathcal{S}_K(Z, \log w)$$

*obtained by a fixed number of differentiable softmin, averaging, cost, and normalization operations with fixed branch decisions. Then a custom VJP for $\mathcal{S}_K$ is obtained by:*

*(i) applying Proposition 26.2 to move $G_T$ into final potentials, costs, and log weights;*

*(ii) applying Proposition 26.3 to move final-cost adjoints into $Z$;*

*(iii) scanning (26.27) backward through the finite potential updates;*

*(iv) composing any resulting $G_Z$ with the chosen centering, scaling, and stop-gradient policy.*

*The result equals the VJP of the executed finite program.*

*Proof.* The finite transport block is a composition of differentiable maps under the fixed branch and fixed iteration assumptions. Propositions 26.2, 26.3, and 26.4 give the VJP for the nontrivial primitive maps. The VJP of a composition is obtained by applying the transposed local Jacobians in reverse order. That reverse composition is exactly the scan described above. □

## 26.9 Why a custom VJP is memory efficient

The naive reverse graph for the OT block stores pairwise costs, logits, normalizers, potentials, and intermediate loop states for every time step. With $N$ particles and $K$ Sinkhorn updates, this can scale like $O(KN^2)$ in stored tensors before the rest of the filtering graph is considered.

The custom VJP changes the storage problem. The primal path must retain or reconstruct only enough information to replay the finite program:

- the input particles and log weights entering the OT block;
- the branch choices, masks, iteration count, temperature schedule, and stabilization policy;
- final potentials, and optionally sparse checkpoints of intermediate potentials;
- any stopped quantities needed to reproduce the executed scalar.

During the backward pass, pairwise blocks can be recomputed and reduced immediately. This trades additional arithmetic for lower peak memory. The trade is natural in HMC, because one scalar log target needs one reverse VJP to obtain all parameter-score components, while a forward-mode JVP would need one pass per requested direction.

## 26.10 Same-cloud geometry reuse in the finite JVP

The canonical finite forward-JVP route repeatedly evaluates softmin operations on the same scaled particle cloud during one OT solve. Let $Z = (z_1, \dots, z_N)$, let the parameter tangent axis have arbitrary size $P$, and define

$$C_{ij} = \frac{1}{2}\|z_i - z_j\|^2, \tag{26.28}$$

$$\dot{C}_{ij}^{(p)} = (z_i - z_j)^\top \big(\dot{z}_i^{(p)} - \dot{z}_j^{(p)}\big), \qquad p = 1, \dots, P. \tag{26.29}$$

The Sinkhorn potentials change from iteration to iteration, but $Z$ and its tangent do not change within that finite solve. Therefore the same $C$ and $\dot{C}$ appear in every annealed softmin, every terminal-balancing softmin, and the final transport assembly.

**Proposition 26.6** (Same-cloud cache preserves the finite JVP)**.** *Fix the input cloud $Z$, its parameter tangents $\dot{Z}$, the finite temperature schedule, the potential initialization, the iteration counts, and all branch decisions for one OT solve. Replacing each recomputation of $C(Z)$ and $DC(Z)[\dot{Z}]$ by the cached tensors in (26.28)–(26.29) leaves the mathematical softmin values and JVPs unchanged. Consequently it leaves the declared finite Sinkhorn, terminal-balance, row-quotient, and Contract E total JVP unchanged in exact arithmetic.*

*Proof.* For log-scale inputs $h_j$, one softmin and its JVP are

$$m_i = -\varepsilon \log \sum_j \exp\{h_j - C_{ij}/\varepsilon\},$$
$$\dot{m}_i = -\dot{\varepsilon} \log \sum_j e^{\ell_{ij}} - \varepsilon \sum_j q_{ij}\dot{\ell}_{ij},$$
$$\dot{\ell}_{ij} = \dot{h}_j - \frac{\dot{C}_{ij}}{\varepsilon} + \frac{C_{ij}\dot{\varepsilon}}{\varepsilon^2}, \qquad q_{ij} = \frac{e^{\ell_{ij}}}{\sum_r e^{\ell_{ir}}}.$$

Within the fixed solve, every occurrence of $C_{ij}$ is the deterministic function in (26.28), and every occurrence of $\dot{C}_{ij}$ is its directional derivative in (26.29). Substituting stored copies of those same tensors changes neither displayed expression. The finite solver is a composition of these softmins, affine potential updates, reductions, and the final transport and Contract E maps. Induction over the fixed iteration sequence therefore gives the same value and tangent at every subsequent state. □

This optimization addresses a concrete generic bottleneck. With $K$ annealed Sinkhorn updates and $L$ terminal-balance updates, the current same-cloud JVP evaluates four softmins before the loop, four per Sinkhorn update, two after it, and two per balance update. Thus it otherwise constructs or differentiates pairwise geometry

$$4K + 2L + 6$$

times per OT solve, before counting the final transport assembly. For the measured $K = 20, L = 8$ witness this is 102 repeated large geometry/JVP traversals per filtering time step. Caching reduces those geometry constructions to one $C$ and one $\dot{C}$ per solve; it does not reduce the required softmax reductions or change either iteration count.

The storage tradeoff is explicit. For batch size $B$, particle count $N$, parameter count $P$, and scalar width $s$ bytes, the two cached tensors require approximately

$$sBN^2(1 + P) \tag{26.30}$$

bytes before compiler liveness and other filter state are counted. The formula supports any positive $P$, but does not imply that dense caching is feasible for every $(B, N, P)$. The cache is therefore a scoped optimization: it may be enabled only when the route uses one exact same-cloud tile and a scope-specific GPU memory/performance check passes. Otherwise the streamed recomputation path remains the correct fallback. No cache may cross a time step, parameter evaluation, changed cloud, changed tangent, or changed dtype.

On the July 2026 TF32/XLA diagnostic witness with $T = 50$, $N = 1024$, $K = 20$, and $L = 8$, five synchronized warm calls at $B = 1$ had median times 3.013 seconds without the cache and 0.302 seconds with it, a descriptive 9.97-fold speedup. At $B = 16$, three warm calls had medians 25.922 and 2.591 seconds, respectively, a descriptive 10.00-fold speedup.

Both arms used one XLA `StatelessWhile`, the repository $K = N$ chunk policy, the same prepared inputs, and the same 8 GiB device cap. The relative score-norm differences were $3.24 \times 10^{-5}$ and $1.78 \times 10^{-5}$, respectively; all finite, replay, chart, marginal, and reset gates passed. These differences are floating-point evaluation-order effects, not a change in the exact-arithmetic finite target. LGSSM is only the measured performance witness: the implementation and parity tests carry an arbitrary final parameter axis and do not use Kalman or LGSSM formulas.

An exact parameter-axis batching candidate for the Contract E Cholesky reset was also tested. Although it matched the per-direction algebra on $P \in \{1, 3, 5, 18\}$, its measured $B = 1$ median rose from 0.302 to 0.413 seconds. It is therefore rejected as an optimization and the per-direction reset JVP remains the active route. Correct vectorization is not evidence of a performance gain.

## 26.11 Embedding the OT VJP in the filtering loop

The full LEDH-PFPF-OT gradient is a reverse scan through time. At time $t$, the adjoint entering the OT block contains two sources: downstream dependence of future particles, and direct dependence of the current scalar contribution. The OT VJP returns adjoints for the pre-transport particles and log weights:

$$(G_{X_t}, G_{\log w_t}) = \mathrm{VJP}_{\mathrm{OT},t}(G_{\widetilde{X}_t}).$$

The log-weight adjoint then flows through the PF-PF proposal-correction terms: transition density, observation density, proposal density, and flow log-determinant. The particle adjoint flows through the LEDH migration and the model transition and observation callbacks. Repeating this from $t = T$ down to $t = 1$ yields the parameter score.

**Proposition 26.7** (Filtering-loop custom VJP)**.** *Suppose each time-step map in the executed LEDH-PFPF-OT value path is differentiable under fixed branch decisions, fixed random streams, fixed masks, and fixed iteration budgets. If each time-step custom VJP is the VJP of that same executed time-step map, then the reverse time scan returns the derivative of the executed scalar $\mathcal{L}_K(\theta)$.*

*Proof.* The full filter value is a finite composition of the time-step maps and the final scalar reduction. By the chain rule, the VJP of the full composition is the product of the time-step transposed Jacobians in reverse time order. The assumption says that each custom time-step VJP equals the corresponding transposed Jacobian multiplication for the executed map. Therefore their reverse-time composition equals the VJP of the executed scalar. □

## 26.12 A proposed score-aware teacher-projection extension

Contract E–Chol restores a selected value-side object: the weighted source mean and a ridged covariance identity. Its fixed residual design and prepared ridge are not independent controls for likelihood accuracy, score accuracy, and structural support. In particular, agreement of first two moments does not imply agreement of the tangent

filtering measure. This section therefore studies a major algorithmic extension, not a tuning change to `contract_e_chol_v1`.

The provisional name is *Contract E–teacher projection*, abbreviated Contract E–TP. It has no canonical, leaderboard, default, or HMC-facing status. It changes three parts of the reset contract:

(1) it constructs a larger finite teacher by pairing parent particles with fixed innovation points;

(2) it compresses selected teacher feature expectations into a smaller positive weighted cloud; and

(3) it carries those nonuniform weights into the next LEDH step instead of resetting the weights to $1/N$.

The current Contract E transport geometry may supply a reference cloud, preferred anchors, or a displacement objective. It cannot guarantee score-feature feasibility by itself. The existence result below instead uses anchors selected from the teacher support. For a structurally degenerate model this is important: every teacher point is generated by the declared structural map, so compression never invents an off-support full-state barycenter.

**Teacher-projection status note.** The name *Contract E–teacher projection* is a convenience label for this chapter's extension route, not a canonical, leaderboard, default, or HMC-facing identity. Any release wording must preserve that distinction.

### 26.12.1 The filtering derivative that must survive reset

Let $\eta_{t,\theta}$ be a normalized filtering measure. For a fixed test function $h$, its tangent measure $\zeta_{t,\theta}$ is defined weakly by

$$\zeta_{t,\theta}(h) = D_\theta\{\eta_{t,\theta}(h)\}. \tag{26.31}$$

The conditional path-score representation and the corresponding backward particle recursion are given in the inspected arXiv/technical-report version of Del Moral et al. [2015, Section 2, Eq. (2.5), and Algorithm 1]; that algorithm evaluates the backward statistic with pairwise parent–child work. The project construction below is different: it compresses a finite teacher measure while preserving selected expectations and their derivatives. The literature supplies the filtering-derivative target; it does not prove the new compression rule.

For a parameter-dependent feature $h_\theta$, the total derivative is

$$D_\theta\{\eta_{t,\theta}(h_\theta)\} = \zeta_{t,\theta}(h_\theta) + \eta_{t,\theta}(D_\theta h_\theta). \tag{26.32}$$

Here $\zeta_{t,\theta}(h_\theta)$ means that the tangent measure at the fixed parameter is applied to the frozen function $x \mapsto h_\theta(x)$; the second term accounts for the feature's explicit parameter dependence. This distinction matters below. Preserving a parameter-dependent predictive potential preserves the displayed total derivative. It does not separately identify the two terms on the right unless the feature system also identifies them.

### 26.12.2 Parent-by-innovation teacher

Suppose the incoming finite cloud has parent points and positive normalized weights

$$\{(x_i, a_i)\}_{i=1}^{N_p}, \qquad \sum_i a_i = 1,$$

and let a fixed innovation rule have points and positive weights

$$\{(\varepsilon_j, b_j)\}_{j=1}^{N_\varepsilon}, \qquad \sum_j b_j = 1.$$

For a structural transition $F_\theta$, form

$$z_{ij}(\theta) = F_\theta(x_i, \varepsilon_j). \tag{26.33}$$

Thus a nominal $N$-parent by $N$-innovation construction has $N^2$ candidates. If $c_{t,ij}(\theta) > 0$ denotes the executed current-observation and proposal-correction factor, define

$$u_{ij}(\theta) = a_i b_j c_{t,ij}(\theta), \qquad \pi_{ij}(\theta) = \frac{u_{ij}(\theta)}{\sum_{r,s} u_{rs}(\theta)}. \tag{26.34}$$

All fixed LEDH noises, flow steps, proposal corrections, and branch choices that belong to the finite target must be included in $c_{t,ij}$. Omitting them would construct a teacher for a different scalar.

After flattening $(i, j)$ to $a = 1, \ldots, N_T$, the teacher is

$$\eta_{t,\theta}^T = \sum_{a=1}^{N_T} \pi_a(\theta) \delta_{z_a(\theta)}. \tag{26.35}$$

This is a filtering teacher, not a smoother: it uses the current filtered parents and current innovations. An $O(N^2)$ backward tangent statistic may also be evaluated on this finite system, but the score-preservation theorem below does not inject that statistic as an externally supplied gradient. It differentiates the same finite primal constraints.

Choose $K$ finite differentiable features and include mass explicitly:

$$\psi_\theta(x) = \big(1, x^\top, \mathrm{vech}(xx^\top)^\top, r_{t+1,\theta,\xi}(x), \varphi_{1,\theta}(x), \ldots\big)^\top \in \mathbb{R}^K. \tag{26.36}$$

Here $r_{t+1,\theta,\xi}(x)$ is the next-step finite predictive contribution conditional on parent $x$, with the same fixed finite noise stream $\xi$ and the same transition, LEDH proposal, importance correction, and observation program that the next value evaluation will execute. For a model transition density it has the idealized form

$$r_{t+1,\theta}(x) = \int f_\theta(x' \mid x) g_\theta(y_{t+1} \mid x')\, dx', \tag{26.37}$$

or the exact fixed-rule finite sum corresponding to that integral. Define the teacher target

$$b_\theta = \eta_{t,\theta}^T(\psi_\theta) = \sum_{a=1}^{N_T} \pi_a(\theta) \psi_\theta(z_a(\theta)). \tag{26.38}$$

### 26.12.3 Positive compression as constrained feasibility

Let $s_\ell(\theta)$, $\ell = 1, \dots, m$, be selected teacher candidates and write

$$\Phi_\theta = \begin{bmatrix} \psi_\theta(s_1(\theta)) & \cdots & \psi_\theta(s_m(\theta)) \end{bmatrix}. \tag{26.39}$$

The student weights are required to satisfy

$$\Phi_\theta q_\theta = b_\theta, \qquad q_\theta \geq 0. \tag{26.40}$$

Because the first feature is one, the first equation is $\mathbf{1}^\top q_\theta = 1$. Structural support, mass, moments, and predictive features are constraints, not terms in an arbitrarily weighted multi-objective loss. A displacement or Contract E proximity objective may be minimized only inside this feasible set.

**Proposition 26.8** (Finite positive teacher coreset)**.** *For any finite teacher in* (26.35) *and any finite $K$-component feature vector whose first component is one, there exists a nonnegative student supported on at most $K$ teacher candidates that satisfies* (26.40) *at the fixed parameter value.*

*Proof.* The full teacher weights are already a feasible nonnegative solution because $b_\theta = \Phi_\theta^T \pi_\theta$. If a feasible solution has more than $K$ positive entries, the corresponding columns in $\mathbb{R}^K$ are linearly dependent. Hence there is a nonzero vector $c$ supported on those entries with $\Phi_\theta c = 0$. The first row of $\Phi_\theta$ is all ones, so $\mathbf{1}^\top c = 0$; therefore $c$ has both positive and negative entries. Set

$$\alpha = \min_{c_\ell > 0} \frac{q_\ell}{c_\ell}, \qquad q' = q - \alpha c.$$

Then $q' \geq 0$, at least one formerly positive component is zero, and $\Phi_\theta q' = \Phi_\theta q = b_\theta$. Repeating the elimination produces a feasible solution with at most $K$ positive entries. □

Proposition 26.8 is a pointwise existence result. It does not say that the active support varies smoothly with $\theta$. HMC cannot repeatedly choose a different linear-program basis and then ignore those switches. A differentiable finite target needs a fixed local chart or a globally smooth positive projection.

**Proposition 26.9** (Fixed active-set differentiable chart)**.** *Let $A$ select exactly $K$ teacher candidates. Suppose, at a center $\theta_0$, that the square matrix $\Phi_{A,\theta_0}$ is nonsingular and*

$$q_{\theta_0} = \Phi_{A,\theta_0}^{-1} b_{\theta_0} > 0$$

*componentwise. If $\Phi_{A,\theta}$ and $b_\theta$ are continuously differentiable, then in a neighborhood of $\theta_0$ the fixed-support rule*

$$q_\theta = \Phi_{A,\theta}^{-1} b_\theta \tag{26.41}$$

*is positive and continuously differentiable. For any parameter direction its tangent is the linear solve*

$$\dot{q} = \Phi_A^{-1} \left( \dot{b} - \dot{\Phi}_A q \right). \tag{26.42}$$

*Proof.* Matrix inversion is continuously differentiable on the open set of nonsingular matrices. Thus $q_\theta$ is continuously differentiable near $\theta_0$. Strict positivity is an open condition, so it persists after reducing the neighborhood if necessary. Differentiating $\Phi_A q = b$ yields $\dot{\Phi}_A q + \Phi_A \dot{q} = \dot{b}$, and solving for $\dot{q}$ gives (26.42). □

When more than $K$ fixed anchors are desired, a semi-analytical Euclidean projection is

$$\min_q \ \frac{1}{2}(q-q_0)^\top R(q-q_0) \quad \text{subject to} \quad \Phi q = b, \tag{26.43}$$

where $R$ is positive definite and $q_0$ may encode Contract E transport preferences. If $\Phi$ has full row rank, its equality-constrained solution is

$$q = q_0 + R^{-1}\Phi^\top(\Phi R^{-1}\Phi^\top)^{-1}(b - \Phi q_0). \tag{26.44}$$

This requires a $K \times K$ solve rather than a per-step high-dimensional optimizer. Positivity is not implied by (26.44); a negative component is a failed chart, not something that may be clipped without changing the constraints.

### 26.12.4 What the projection proves about tangents and score

**Proposition 26.10** (Selected-feature tangent preservation)**.** *Under Proposition 26.9, define*

$$\eta^S_{t,\theta} = \sum_{\ell \in A} q_{\ell,\theta}\delta_{s_\ell(\theta)}.$$

*Then throughout the fixed-chart neighborhood,*

$$\eta^S_{t,\theta}(\psi_\theta) = \eta^T_{t,\theta}(\psi_\theta), \tag{26.45}$$

*and, in every parameter direction,*

$$D_\theta\{\eta^S_{t,\theta}(\psi_\theta)\} = D_\theta\{\eta^T_{t,\theta}(\psi_\theta)\}. \tag{26.46}$$

*For a parameter-independent retained feature $h$, this implies $\zeta^S_{t,\theta}(h) = \zeta^T_{t,\theta}(h)$.*

*Proof.* Equation (26.45) is exactly the constraint $\Phi_A q = b$. Differentiating its left side and substituting (26.42) gives

$$\dot\Phi_A q + \Phi_A \dot q = \dot\Phi_A q + \dot b - \dot\Phi_A q = \dot b,$$

which is the derivative of the teacher target. When $h$ is parameter independent, definition (26.31) gives the last statement. □

The proposition does not train a gradient to look like a teacher score. The same differentiable primal solve produces both the student value and its tangent. Consequently there is no omitted optimizer hypergradient and no externally overwritten score.

**Proposition 26.11** (One-step value and score preservation)**.** *Assume the current likelihood increment $\Delta_t(\theta)$ is computed before reset, as required by the canonical time order. Suppose the positive next-step finite predictive contribution $r_{t+1,\theta,\xi}$ is a retained feature and is exactly the contribution executed at the next step. Define*

$$Z^T_{t+1}(\theta) = \eta^T_{t,\theta}(r_{t+1,\theta,\xi}), \qquad Z^S_{t+1}(\theta) = \eta^S_{t,\theta}(r_{t+1,\theta,\xi}).$$

*Then, throughout the fixed-chart neighborhood,*

$$Z^S_{t+1}(\theta) = Z^T_{t+1}(\theta), \tag{26.47}$$

$$\log Z^S_{t+1}(\theta) = \log Z^T_{t+1}(\theta), \tag{26.48}$$

$$D_\theta \log Z^S_{t+1}(\theta) = D_\theta \log Z^T_{t+1}(\theta). \tag{26.49}$$

*Therefore the two-step partial scalars $\Delta_t + \log Z^S_{t+1}$ and $\Delta_t + \log Z^T_{t+1}$, and their scores, are identical.*

*Proof.* The first equality is the retained predictive-feature row of (26.45). Positivity permits taking logarithms. The log values are equal as functions throughout the fixed-chart neighborhood, so their total derivatives are equal. The current increment is shared because it entered the scalar before either reset. □

This result is exact relative to the declared finite teacher and selected one-step feature. It is not a proof of equality to the exact nonlinear filter, nor does it determine all future likelihood functionals. A full recursive method must construct and validate a new teacher and feature chart at every reset while retaining a globally differentiable finite target. Whether a single fixed chart remains positive over an HMC-relevant parameter region is an open admission question.

### 26.12.5 Progressive score information and compact score-aware features

The one-step proposition above explains both the success of a two-observation test and its limitation. The feature $r_{t+1,\theta}$ identifies one future normalizing constant, but it does not identify how score information accumulated before time $t$ is distributed over the current state. A scalar cumulative score cannot repair this loss: it is constant across particles and therefore adds no constraint beyond mass. The state-dependent object is the conditional additive score statistic.

For clarity, first consider a hidden Markov model with initial density $\mu_\theta$, transition density $f_\theta$, and observation density $g_\theta$. With observations through time $t$, define the complete-data additive score

$$H_{t,\theta}(x_{0:t}) = \nabla_\theta \log \mu_\theta(x_0) + \sum_{s=1}^{t} \nabla_\theta \log f_\theta(x_s \mid x_{s-1}) + \sum_{s=0}^{t} \nabla_\theta \log g_\theta(y_s \mid x_s), \tag{26.50}$$

and its state-conditioned version

$$\tau_{t,\theta}(x) = \mathbb{E}_\theta[H_{t,\theta}(X_{0:t}) \mid X_t = x, y_{0:t}]\,. \tag{26.51}$$

This is a $p$-vector when $\theta \in \mathbb{R}^p$. Its filtering mean is the observed-data score $S_{t,\theta} = \eta_{t,\theta}(\tau_{t,\theta})$. The centered progressive score field is

$$c_{t,\theta}(x) = \tau_{t,\theta}(x) - S_{t,\theta}, \qquad \eta_{t,\theta}(c_{t,\theta}) = 0. \tag{26.52}$$

This conditional expectation is the finite-dimensional statistic used in the backward particle interpretation of the filter derivative [Del Moral et al., 2015, Section 2, Eq. (2.5), Algorithm 1]. Computing it does not require sampling complete trajectories or storing their histories, but the exact finite-particle update uses pairwise parent–child work.

**Proposition 26.12** (Tangent measure represented by the centered score field)**.** *Assume differentiation may be interchanged with the state integrals and all displayed expectations are finite. For every parameter-independent integrable test function $h$,*

$$\zeta_{t,\theta}(h) = \nabla_\theta\{\eta_{t,\theta}(h)\} = \eta_{t,\theta}\big(hc_{t,\theta}^\top\big)\,. \tag{26.53}$$

*For a differentiable parameter-dependent $h_\theta$,*

$$\nabla_\theta\{\eta_{t,\theta}(h_\theta)\} = \eta_{t,\theta}(\nabla_\theta h_\theta) + \eta_{t,\theta}\big(h_\theta c_{t,\theta}^\top\big)\,. \tag{26.54}$$

*Proof.* Write the normalized filtering density as $\eta_{t,\theta}(x_t) = \gamma_{t,\theta}(x_t)/Z_{t,\theta}$. Differentiating the unnormalized path integral for $\gamma_{t,\theta}$ multiplies its integrand by the complete-data score (26.50). Conditioning on $X_t = x_t$ therefore gives $\nabla_\theta \log \gamma_{t,\theta}(x_t) = \tau_{t,\theta}(x_t)$. Integrating this identity gives $\nabla_\theta \log Z_{t,\theta} = S_{t,\theta}$, hence $\nabla_\theta \eta_{t,\theta}(x) = \eta_{t,\theta}(x) c_{t,\theta}(x)$. Integration against $h$ proves (26.53). The product rule adds $\eta_{t,\theta}(\nabla_\theta h_\theta)$ when the feature depends explicitly on the parameter, proving (26.54). □

The discrete $O(N^2)$ recursion is explicit for an ordinary particle approximation of the model filter. Given parents $(x_i, a_i, \tau_i)_{i=1}^{N_p}$ and common child locations $z_k$, define the normalized backward weights

$$B_{ki} = \frac{a_i f_\theta(z_k \mid x_i)}{\sum_{j=1}^{N_p} a_j f_\theta(z_k \mid x_j)}. \tag{26.55}$$

Then the child mark is

$$\tau'_k = \nabla_\theta \log g_\theta(y_{t+1} \mid z_k) + \sum_{i=1}^{N_p} B_{ki} \left\{ \tau_i + \nabla_\theta \log f_\theta(z_k \mid x_i) \right\}. \tag{26.56}$$

Initial-law score terms enter the initial marks.

Equations (26.55)–(26.56) must not be copied unchanged into the corrected LEDH finite program. Its teacher candidates are parent-specific deterministic quadrature/proposal images, so their discrete backward kernel is not automatically the continuous model kernel $f_\theta$. A finite-target mark must be derived from the exact executed proposal measure, including transition, observation, proposal correction, Jacobian, quadrature mass, and support-motion terms. Using only $f_\theta$, or treating the derivative of a moving atomic measure as an ordinary density score, computes a different object. Until that derivation is supplied, model-filter score fields may be used only as labeled diagnostic features.

The full tensor-product proposal $\{\phi_j(x)c_k(x) : j = 1, \dots, K_0,\ k = 1, \dots, p\}$ uses $K_0 p$ additional constraints and is generally too large. A compact diagnostic candidate retains

$$\Psi_{t,\theta}(x) = \left(\phi_{t,\theta}(x)^\top, c_{t,\theta}(x)^\top, \{r_{t+1,\theta}(x) c_{t,\theta}(x)\}^\top\right)^\top. \tag{26.57}$$

The displayed form adds $2p$, rather than $K_0 p$, rows. There is a smaller gauge-fixed implementation. Replacing every parent mark by $\tau_i + a$, for a common vector $a$, replaces every child mark in (26.56) by $\tau'_k + a$, because each backward row sums to one. Subsequent centering removes $a$ exactly. Hence an implementation that carries centered marks and explicitly re-centers them after selection need not impose the $p$ zero-mean rows through the projection solve; it verifies zero mean as an invariant instead. The nonredundant compact chart then appends only $r_{t+1,\theta} c_{t,\theta}$, using $K_0 + p$ rows. This asks the coreset to retain the score-bearing interaction with the next potential. It is a feature-sufficiency hypothesis, not yet an exact finite-LEDH tangent theorem.

At the first LGSSM reset this reduction is also forced by rank: the observation scale component of $c_{t,\theta}$ is a quadratic function of the current state and lies in the mass/first/second-moment span. Retaining all $2p$ rows makes that feature matrix singular. Dropping the zero-mean rows by the gauge argument avoids data-dependent feature deletion and leaves a fixed 16-row LGSSM chart.

**Proposition 26.13** (One-step historical-score identity for a dominated filter)**.** *Let teacher and student be two dominated filtering measures whose tangent measures admit Proposition 26.12 with the same declared score-coordinate convention. Suppose*

$$\eta_t^S(r_{t+1,\theta}) = \eta_t^T(r_{t+1,\theta}), \tag{26.58}$$
$$\eta_t^S(r_{t+1,\theta}c_S^\top) = \eta_t^T(r_{t+1,\theta}c_T^\top), \tag{26.59}$$
$$\eta_t^S(\nabla_\theta r_{t+1,\theta}) = \eta_t^T(\nabla_\theta r_{t+1,\theta}). \tag{26.60}$$

*Then the next normalizing constant and its total score agree.*

*Proof.* The first identity gives the common positive normalizing constant. Applying (26.54) to $h_\theta = r_{t+1,\theta}$ shows that its derivative is the sum of the explicit-feature term in (26.60) and the historical tangent term in (26.59). Both agree, so division by the common normalizing constant proves equality of the log-score. □

The proposition applies to the dominated model-filter setting just stated. A moving discrete Contract E–TP cloud must first prove its corresponding tangent representation; matching sampled $c$ and $rc$ rows alone does not prove that its autodiff tangent measure equals $q\,c$. Consequently the compact rows can diagnose and potentially reduce information loss, but their success must still be judged by the total derivative of the executed scalar.

For an LGSSM there is a stronger finite-horizon diagnostic. The remaining-data likelihood

$$R_{t+1:T,\theta}(x) = p_\theta(y_{t+1:T} \mid X_t = x) \tag{26.61}$$

is available from a conditional Kalman recursion and is an exponential quadratic function of $x$. It is one parameter-dependent scalar feature regardless of $p$ and $T$. Writing its stabilized ratio as

$$\frac{R_{t+1:T,\theta}(x)}{R_{t+1:T,\theta}(0)} = \exp\{-\tfrac{1}{2}x^\top J_{t,\theta}x + h_{t,\theta}^\top x\},$$

all pairs $(J_{t,\theta}, h_{t,\theta})$ are obtained in one reverse-time Gaussian information recursion. This avoids rebuilding a length-$T-t$ conditional Kalman graph at every reset: construction is $O(Td_x^3)$, followed by $O(Nd_x^2)$ evaluation per cloud. The stabilization removes only a common positive factor from the feature row and therefore does not change its linear constraint.

**Proposition 26.14** (Finite-horizon continuation identity)**.** *Fix $T$ and a parameter neighborhood on which one positive nonsingular chart is used. If, immediately after reset at time $t$,*

$$\eta_{t,\theta}^S(R_{t+1:T,\theta}) = \eta_{t,\theta}^T(R_{t+1:T,\theta}) \tag{26.62}$$

*as an identity throughout that neighborhood, then teacher and student assign the same exact-model remaining-data likelihood and total derivative to their respective time-$t$ measures.*

*Proof.* For either measure, the displayed expectation is the exact-model conditional factorization of the future observations given its time-$t$ state distribution. The two sides are equal positive differentiable functions of $\theta$ on the fixed-chart neighborhood. Taking logarithms and differentiating the same identity proves equality of their total derivatives, including both measure dependence and explicit parameter dependence of $R_{t+1:T,\theta}$. □

This does not prove equality of the later recursively executed finite LEDH scalar. Its quadrature/proposal operator differs from exact Kalman integration, and subsequent projections are nonlinear cloud maps. The continuation row is therefore an LGSSM oracle diagnostic of reset information loss. It may test whether a compact future-information summary repairs the observed drift, but it cannot be transferred to nonlinear models without a separately justified finite continuation approximation. Neither this row nor the model-filter score field replaces the missing finite-LEDH tangent derivation.

### A bounded ladder of progressive continuation marks

One full remaining-data continuation feature is compact in row count, but its offline construction can be expensive and its approximation can be fragile in a persistent nonlinear model. Replacing it by the entire sequence of all future prefixes would make the chart grow with the horizon. The implemented intermediate design instead freezes a small ordered set of positive look-ahead lengths $\mathcal{L} = \{\ell_1 < \cdots < \ell_r\}$ before preparation. For each available prefix define

$$B_{t,\ell,\theta}(x) = \int \prod_{s=t+1}^{t+\ell} f_\theta(x_s \mid x_{s-1}) g_\theta(y_s \mid x_s)\, dx_{t+1:t+\ell}, \qquad x_t = x. \tag{26.63}$$

For a finite record ending at $T$, the realized length is $\bar{\ell}_t = \min(\ell, T-t)$. If two requested lengths have the same realized length, only one row is kept. This de-duplication depends only on $t, T, \mathcal{L}$, not on particles, parameters, observations, or numerical values, so it is a prepared chart shape rather than a runtime active-set decision.

Let $x_\star(\theta)$ be the declared reference point and set

$$h_{t,\ell,\theta}(x) = \frac{B_{t,\ell,\theta}(x)}{B_{t,\ell,\theta}(x_\star(\theta))}. \tag{26.64}$$

The implementation evaluates numerator and denominator by the same fixed-grid backward target recursion. The denominator is positive and common to a row; it changes neither the row span nor the equality constraint. Its dependence on $\theta$, including dependence through $x_\star(\theta)$, remains in the autodiff graph. For scalar SV the feature vector at time $t$ is

$$\psi_{t,\theta}(x) = \left(1, x, x^2, \{h_{t,\bar{\ell},\theta}(x) : \bar{\ell} \in \overline{\mathcal{L}}_t\}\right)^\top, \tag{26.65}$$

where $\overline{\mathcal{L}}_t$ contains the distinct realized lengths. Thus the reset carries at most $3+r$ weights and scalar locations, independent of $T$; it does not carry a growing score history. The backward preparation cost still grows with each requested continuation window, which is why $\mathcal{L}$ is a capacity ladder rather than an unrestricted feature list.

### Actual-SV amendment: an overcomplete analytical chart

A square chart uses one anchor for each feature. It is algebraically unique when its feature matrix is nonsingular, but its positive domain can be much smaller than the parameter neighborhood on which a score is needed. This is not repaired by adding features: each new feature adds an equality constraint. The amendment here keeps the feature target fixed and adds anchors, thereby adding degrees of freedom to its representation.

The failure that motivates the amendment is concrete. The Actual-SV experiment uses the two unconstrained coordinates

$$\theta = (\Phi^{-1}(\gamma), \log \beta), \qquad \theta_0 = (0.2533471031357997, -0.916290731874155),$$

and, at every nonterminal time, the four features

$$\psi_{t,\theta}(x) = \big(1, x, x^2, h_{t,16,\theta}(x)\big)^\top, \qquad q = 4. \tag{26.66}$$

The center-prepared four-anchor chart at zero-based time index 748 had minimum weight $3.58612667 \times 10^{-7}$. Holding its prepared indices and row scales fixed, changing the first coordinate by $-10^{-5}$ produced $-4.22267303 \times 10^{-6}$, while changing the second coordinate by $+10^{-5}$ produced $-5.23620186 \times 10^{-6}$. Thus the executed square chart correctly failed closed: those moment targets are outside the positive simplex generated by those four frozen columns. This is a chart failure, not a failure of the SV parameter transform, Sinkhorn balancing, or canonical Contract E–Chol.

Choose one anchor count $K > q$, constant across all nonterminal times. At a given time, preparation freezes teacher indices $i_1, \dots, i_K$. The indices are discrete prepared data, but the selected locations $z_k(\theta) = z_{i_k}(\theta)$ are not frozen: they move with the finite teacher program. Consequently their feature columns contribute to the total derivative. Let $D \in \mathbb{R}^{q \times q}$ be a positive diagonal row-scale matrix prepared at the center and then frozen, and define

$$A_\theta = D^{-1} \left[\psi_{t,\theta}(z_1(\theta)) \quad \cdots \quad \psi_{t,\theta}(z_K(\theta))\right], \qquad b_\theta = D^{-1} m_{t,\theta}, \tag{26.67}$$

where $m_{t,\theta}$ is the same finite teacher feature target used by the square chart. Positivity and nonsingularity of $D$ make row scaling an invertible numerical transformation, not a change in the constraints.

**Center reference.** The anchor indices are deterministic center-teacher weighted quantiles. For anchor count $K$, their probability levels are $(k - \frac{1}{2})/K$, $k = 1, \dots, K$. Stable sorting, cumulative teacher mass, and left search select indices. Repeated indices are removed stably and, only to fill duplicates, unused teacher indices are added in descending normalized mass order with stable tie-breaking. Every center teacher point is then assigned to its nearest anchor in state space, again with stable tie-breaking. Aggregating its normalized mass gives a vector $v \in \mathbb{R}^K$. Preparation rejects an empty cell: $v_k > 0$ for every $k$ is a hard condition and is never manufactured by clipping or flooring.

Write $V = \operatorname{diag}(v)$, $A_0 = A_{\theta_0}$, and $b_0 = b_{\theta_0}$. The center reference is the Pearson projection

$$r = v + V A_0^\top (A_0 V A_0^\top)^{-1}(b_0 - A_0 v). \tag{26.68}$$

**Proposition 26.15** (Analytical Pearson center reference)**.** *Suppose $v > 0$ componentwise and $A_0 \in \mathbb{R}^{q \times K}$ has full row rank. Then (26.68) is the unique solution of*

$$\min_{u \in \mathbb{R}^K} \frac{1}{2}(u - v)^\top V^{-1}(u - v) \quad \textit{subject to} \quad A_0 u = b_0. \tag{26.69}$$

*It satisfies $A_0 r = b_0$. Strict positivity of $r$ does not follow from the equality problem and is therefore a separate preparation gate.*

*Proof.* The Hessian $V^{-1}$ is positive definite, so the objective is strictly convex. The Lagrange equations for multiplier $\nu$ are

$$V^{-1}(u - v) + A_0^\top \nu = 0, \qquad A_0 u = b_0.$$

Eliminating $u = v - VA_0^\top \nu$ gives $(A_0 V A_0^\top)\nu = A_0 v - b_0$. Full row rank of $A_0$ and $V \succ 0$ make this Gram matrix positive definite. Substitution yields (26.68); it is feasible and, by strict convexity, unique. Equality feasibility alone imposes no sign constraint, so a nonpositive component must be rejected rather than hidden. □

For a positive accepted $r$, freeze the Pearson precision and its inverse,

$$P = \operatorname{diag}(r_1^{-1}, \dots, r_K^{-1}), \qquad R = P^{-1} = \operatorname{diag}(r). \tag{26.70}$$

The choice is the Hessian of the Pearson chi-square discrepancy centered at $r$; it is a declared metric hypothesis, not a statistical confidence threshold. Runtime computes

$$w_\theta = \arg\min_w \frac{1}{2}(w - r)^\top P(w - r) \quad \text{subject to} \quad A_\theta w = b_\theta. \tag{26.71}$$

**Proposition 26.16** (Runtime solution and frozen-reference total JVP)**.** *Suppose $r > 0$, $A_\theta$ has full row rank, and $A_\theta, b_\theta$ are differentiable along a parameter direction. Set*

$$S = A_\theta R A_\theta^\top, \qquad c = b_\theta - A_\theta r, \qquad \lambda = S^{-1}c, \qquad w_\theta = r + RA_\theta^\top \lambda. \tag{26.72}$$

*Then $w_\theta$ is the unique minimizer of (26.71) and satisfies $A_\theta w_\theta = b_\theta$. With the preparation $r, R, D, i_1, \dots, i_K$ frozen, its directional derivative is*

$$\dot S = \dot A R A^\top + A R \dot A^\top, \qquad \dot c = \dot b - \dot A r, \tag{26.73}$$

$$\dot\lambda = S^{-1}(\dot c - \dot S\lambda), \qquad \dot w = R(\dot A^\top \lambda + A^\top \dot\lambda). \tag{26.74}$$

*In particular,*

$$A\dot w + \dot A w = \dot b, \tag{26.75}$$

*so the same finite program matches both the declared features and their total directional derivatives.*

*Proof.* The Lagrange equations for (26.71), using the sign convention $P(w-r) - A^\top\lambda = 0$, give $w = r + RA^\top\lambda$. Substitution into $Aw = b$ gives $S\lambda = c$. Since $R \succ 0$ and $A$ has full row rank, $S \succ 0$, so the multiplier and the strictly convex minimizer are unique. This proves (26.72) and exact feasibility.

Differentiate $S\lambda = c$ while $r$ and $R$ are fixed: $S\dot\lambda + \dot S\lambda = \dot c$. Solving for $\dot\lambda$, and then differentiating $w = r + RA^\top\lambda$, yields (26.73)– (26.74). Finally, differentiating the already proved identity $Aw = b$ gives (26.75). Here $\dot A$ contains the explicit parameter derivative of each feature, teacher motion, and motion of every selected location $z_{i_k}(\theta)$; freezing an index does not freeze the tensor gathered at that index. □

**Finite-program gate.** The claimed object is the executed TensorFlow solve, not an unknown exact-real solution. Every prepared and runtime case must have finite inputs and outputs, full row rank, condition–roundoff below one, scaled equality residual no larger than its forward-error bound, and strictly positive computed weights. The implementation solves the $q \times q$ Gram system by Cholesky or a linear solve; it never forms a

matrix inverse and never stores or solves a dense $K \times K$ precision system. Failed cases return an invalid flag and poison claim-bearing outputs. No clipping, weight floor, feature deletion, runtime optimization, or active-set switch is allowed.

These diagnostics identify an unreliable finite solve but are not a rigorous componentwise sign certificate for the exact-real solution. For a selected capacity, the weakest design case and weakest held-out case—defined by the smallest finite TensorFlow weight, then time and predeclared point order—are also recomputed at increasing CPU precision. Stable sign is corroborating evidence only, not interval arithmetic and not part of the BayesFilter algorithmic backend.

**Algorithm 4** Actual-SV overcomplete analytical chart preparation and runtime

1: **Prepare:** freeze one global $K > q$, the center $\theta_0$, row scales $D_t$, and the design and held-out parameter points before observing candidate results.
2: **for** each nonterminal time $t$ **do**
3: Select $K$ center-teacher weighted-quantile indices with the deterministic duplicate-fill rule.
4: Aggregate stable nearest-anchor Voronoi masses $v_t$; reject if any component is nonfinite or nonpositive.
5: Form $A_{t,0}, b_{t,0}$ and solve (26.68); reject unless rank, conditioning, residual, finiteness, and $r_t > 0$ gates pass.
6: Freeze indices, $D_t$, $r_t$, and $R_t = \mathrm{diag}(r_t)$.
7: **end for**
8: **Run:** gather the moving locations and feature columns at the frozen indices, form $A_{t,\theta}, b_{t,\theta}$, and solve the $q \times q$ system in (26.72).
9: Apply every finite-program gate; carry the resulting nonuniform positive weights, or poison the claim-bearing recursion on failure.
10: For each parameter direction, propagate (26.73)– (26.74) through the same fixed-shape filtering loop.

For $q$ features and $K$ anchors, forming $ARA^\top$ costs $O(Kq^2)$, its solve costs $O(q^3)$, and temporary storage is $O(Kq+q^2)$. In Actual SV, $q = 4$, so increasing anchor capacity does not increase the dimension of the linear solve. The recursive GPU route must use a fixed-shape `tf.while_loop`; Python horizon unrolling remains a reference diagnostic only.

This amendment proves the analytical projection, equality identities, and JVP of the declared finite chart. It does not prove that a tested finite set of parameter points covers an HMC trajectory, that the four-feature teacher is an exact nonlinear filter, that the resulting score agrees with an independent dense target, or that Contract E–TP is canonical, leaderboard-ready, or superior. Those are separate empirical or scientific questions. In particular, matching one continuation mark and its derivative does not identify the entire filtering tangent measure; recursive same-target and quadrature refinement comparisons remain necessary descriptive evidence.

### Executed Actual-SV selection and certification record

The preceding propositions state what follows mathematically from an accepted fixed preparation. This subsection records separately how the preparation was selected, which finite cases were executed, which derivative references were usable, and what the resulting

evidence can and cannot establish. Numerical observations below are not assumptions of Propositions 26.15 and 26.16.

**Frozen parameter sets and capacity selection.** Write

$$\theta(z) = \theta_0 + 10^{-5}z, \qquad z \in \mathbb{R}^2.$$

Before the capacity ladder was evaluated, the ordered design set was frozen as

$$\mathcal{Z}_{\text{design}} = \{(0,0),(-1,0),(1,0),(0,-1),(0,1),(-1,-1),(-1,1),(1,-1),(1,1)\}, \quad (26.76)$$

and the disjoint audit set was frozen as

$$\mathcal{Z}_{\text{audit}} = \{(-\tfrac{1}{2},-\tfrac{1}{2}),(-\tfrac{1}{2},\tfrac{1}{2}),(\tfrac{1}{2},-\tfrac{1}{2}),(\tfrac{1}{2},\tfrac{1}{2}),(-\tfrac{3}{4},-\tfrac{1}{4}),(-\tfrac{3}{4},\tfrac{1}{4}),(\tfrac{3}{4},-\tfrac{1}{4}),(\tfrac{3}{4},\tfrac{1}{4})\}. \quad (26.77)$$

The audit set did not participate in capacity selection and could not authorize trying another capacity after a failure. One global capacity was required at all 999 nonterminal times. The selection rule was the smallest $K \in \{5,\dots,25\}$ passing every design point at each of $T = 10, 100, 1000$.

Table 26.1: Frozen Actual-SV anchor-capacity ladder. A failure is classified by the first mathematical preparation gate it violates, not by timeout.

| Capacity | Observed disposition | Classification |
|---|---|---|
| $K = 5, 6$ | center preparation fails by $T = 10$ | negative Pearson reference |
| $K = 7,\dots,22$ | center preparation passes $T = 10$, fails by $T = 100$ | negative Pearson reference |
| $K = 23$ | pass all design points at $T = 10, 100, 1000$ | selected smallest pass |
| $K = 24, 25$ | fail at time zero | zero Voronoi/reference mass |

The selected $K = 23$ preparation passed all eight audit points once, without retuning. Its weakest design weight was $5.395467979032687 \times 10^{-165}$, at zero-based ordered design point 7, time zero, anchor 22. Its weakest audit weight was $5.396877738726166 \times 10^{-165}$, at zero-based ordered audit point 2, $z = (\frac{1}{2}, -\frac{1}{2})$, time zero, anchor 22. Re-solving the saved 4-by-23 systems with 50, 100, and 200 decimal digits kept the same weakest anchor and a strictly positive sign. The 200-digit design value begins $5.39546797902344038197 \times 10^{-165}$, and the corresponding audit value begins $5.39687773872662767248 \times 10^{-165}$. This supports the sign of the saved cases; it is not interval arithmetic and does not certify nearby unsampled parameters.

**Proposition 26.17** (Scope of a finite chart audit)**.** *Let $\mathcal{Z}$ be a finite parameter set. If the fixed prepared chart passes every chart finite-program gate at every $z \in \mathcal{Z}$, then the executed chart and its claim-bearing scalar outputs are admissible at those tested points. At a tested point where the separate own-scalar derivative gate also passes, the checked JVP is admissible for that same finite scalar. These observations alone do not imply that either class of gate passes on* $\operatorname{conv}(\mathcal{Z})$*, on any open neighborhood, or along an HMC trajectory.*

*Proof.* The first two statements are exactly the corresponding conjunctions of the pointwise chart and derivative gate outcomes. A finite set contains no information about an untested point without an additional analytic bound, interval certificate, or verified continuity margin. A continuous weight can be positive at every tested point and negative between two of them. Therefore the stronger region claims do not follow from the finite audit. □

**Own-scalar derivative references.** For the two-parameter center score, define the central finite-difference vector with $h = 10^{-5}$ by

$$g_j^{\mathrm{FD}} = \frac{\mathcal{L}(\theta_0 + he_j) - \mathcal{L}(\theta_0 - he_j)}{2h}, \qquad j = 1, 2, \tag{26.78}$$

and report

$$E_{\mathrm{FD}} = \frac{\|g^{\mathrm{manual}} - g^{\mathrm{FD}}\|_2}{\max(\|g^{\mathrm{FD}}\|_2, 1)}. \tag{26.79}$$

The frozen FD-only diagnostic tolerance was $0.05\sqrt{p} = 0.05\sqrt{2} = 0.07071067811865477$. This is an owner-selected engineering tolerance for this deterministic directional FD comparison. It is *not* a 95% confidence interval, a two-standard-deviation rule, or a statistical error probability: neither side of (26.79) is a sample from a declared sampling distribution.

Table 26.2: Actual-SV manual total JVP against central finite differences of the same finite scalar.

| $T$ | $E_{\mathrm{FD}}$ | FD-only gate |
|---|---|---|
| 2 | $2.323339358376968 \times 10^{-11}$ | pass |
| 10 | $1.6187801132686436 \times 10^{-10}$ | pass |
| 100 | $2.247653507298375 \times 10^{-9}$ | pass |
| 1000 | $5.7988816510053705 \times 10^{-8}$ | pass |

Ordinary TensorFlow reverse-mode autodiff agrees at the center through $T = 10$, but one tested $T = 10$ endpoint is already nonfinite and all five tested points are nonfinite at $T \geq 100$, even though the primal, manual JVP, and FD values remain finite. Component-level investigations associate the failure with reverse propagation through extremely small positive-weight paths; they do not prove a unique failing operation. The observed nonfinite result is therefore an implementation failure of that reverse-mode oracle for this finite program. By itself it is evidence neither that the finite scalar is nondifferentiable nor that the manual JVP is correct. A trial custom reverse pullback also returned nonfinite results at $T = 100$; it was removed rather than retained as a fallback.

An independent TensorFlow eager `ForwardAccumulator` differentiated the unmodified primal instead. With

$$E_{\mathrm{FAD}} = \frac{\|g^{\mathrm{manual}} - g^{\mathrm{FAD}}\|_2}{\max(\|g^{\mathrm{manual}}\|_2, \|g^{\mathrm{FAD}}\|_2, 10^{-12})},$$

the observed comparisons were:

Table 26.3: Actual-SV manual total JVP against eager forward autodiff of the same unmodified scalar. The reference gate is $\sqrt{\epsilon_{64}} = 1.4901161193847656 \times 10^{-8}$.

| $T$ | $E_{\mathrm{FAD}}$ | float64 reference gate |
|---|---|---|
| 10 | $1.298650134071988 \times 10^{-13}$ | pass |
| 100 | $4.447444400460831 \times 10^{-12}$ | pass |
| 1000 | $2.6364004959309425 \times 10^{-12}$ | pass |

At $T = 1000$, the manual and eager-forward scores were, respectively,

$$g^{\text{manual}} = (5.664478697500771, -2.566049221886212),$$
$$g^{\text{FAD}} = (5.664478697506889, -2.5660492218710016).$$

Their maximum absolute difference was $1.5210499526574495 \times 10^{-11}$. Automatic forward mode is a CPU reference exception, not the production route. XLA compilation of the transformed nested loop failed on a nonconstant transpose permutation; a non-JIT `tf.function` transformation instead failed a TensorList shape invariant. The production score is consequently the explicit two-direction manual JVP inside the XLA-default fixed-shape loop. This choice is supported by the reported same-scalar checks; it is not justified merely by the failure of the other autodiff implementations.

**Independent dense-filter comparison.** Own-scalar differentiation does not establish that the scalar approximates the desired nonlinear filtering value or score. The selected candidate was therefore compared at the center with a deterministic dense Actual-SV filter using Legendre orders 129 and 257 on radius 10. For scalar score components $a, b$, the reported symmetric-relative difference is

$$d_{\text{sym}}(a,b) = \begin{cases} 2|a-b|/(|a|+|b|), & |a|+|b| > 0, \\ 0, & a = b = 0. \end{cases} \tag{26.80}$$

Table 26.4: Descriptive candidate-minus-order-257 dense-reference comparisons. No scientific-equivalence threshold was declared.

| $T$ | Value difference | Absolute score difference | Componentwise $d_{\text{sym}}$ |
|---|---|---|---|
| 2 | $1.3014182620452175 \times 10^{-8}$ | $(2.6158827104438842 \times 10^{-8}, 2.8765310133849198 \times 10^{-9})$ | $(1.187693931512109 \times 10^{-7}, 5.435712744272272 \times 10^{-9})$ |
| 10 | $2.4500970141616563 \times 10^{-8}$ | $(1.604391193232857 \times 10^{-7}, 7.053514550214857 \times 10^{-9})$ | $(2.2686520413738664 \times 10^{-7}, 1.1943543223104596 \times 10^{-8})$ |
| 100 | $-6.553175069257122 \times 10^{-7}$ | $(1.235704860413911 \times 10^{-5}, 3.52904263500875 \times 10^{-6})$ | $(1.2128187577154448 \times 10^{-5}, 1.192559771676947 \times 10^{-6})$ |

There were no componentwise sign reversals. The order-129 versus order-257 value changes were $4.71 \times 10^{-14}$, $2.34 \times 10^{-13}$, and $2.39 \times 10^{-12}$ at the three horizons; the largest adjacent score change was $1.07 \times 10^{-13}$. Thus the observed candidate differences are not explained by that particular adjacent refinement. They nevertheless remain descriptive: adjacent quadrature orders can share truncation bias, no rigorous dense-reference error bound was proved, and no target-specific equivalence margin was fixed before the run.

**Trusted GPU/XLA execution and failure recording.** The full $T = 1000, K = 23$ manual-score route was executed in TensorFlow 2.19.1 on one logical RTX 4080 SUPER GPU. The logical device was configured before initialization with a hard 8192 MiB limit; memory growth was disabled because TensorFlow does not permit both policies simultaneously. XLA was enabled and all claim-bearing outputs were placed on `GPU:0`. The concrete graph contained three functional `StatelessWhile` nodes, and a source audit found no reachable Python horizon loop.

All center and four FD endpoint charts were valid. The center value and score were

$$\mathcal{L}(\theta_0) = -2286.226010065916, \qquad g^{\text{manual}} = (5.664478697508284, -2.56604922190062).$$

The GPU FD relative error was $6.037244233113808 \times 10^{-8}$, and warm replay, including the score, was bitwise identical. Relative to the bound CPU artifact, the maximum

objective absolute difference was $9.094947017729282 \times 10^{-13}$, and the maximum manual-score absolute difference was $1.4408030324375432 \times 10^{-11}$. These CPU/GPU differences are descriptive; no equivalence threshold was added after seeing them.

TensorFlow reported an allocator peak of 1,995,776 bytes. This field does not include the CUDA context, loaded libraries, or driver allocations and must not be interpreted as total device use. The run took 69.5032 seconds in one attempt; this single timing is engineering telemetry, not a performance distribution or method ranking. TensorFlow's TF32 setting was recorded as enabled, but the candidate tensors were float64, so the artifact makes no TF32 accuracy or throughput claim.

After module initialization, the harness writes a fresh structured failure artifact for catchable Python, TensorFlow, configuration, identity, or allocation exceptions raised by the main evaluation before propagating the error. Failures before that handler is installed, including import/device initialization failures, and native process aborts cannot be serialized by this mechanism. The absence of a failure artifact is therefore not itself proof that no such failure occurred. Successful completion, JSON parsing, the process exit status, and the saved output together establish the recorded successful run.

**Evidence boundary.** The authoritative machine-readable evidence is in the dated Actual-SV overcomplete analytical-chart repair directory under the repository's benchmark evidence tree. The selected preparation has SHA-256

`e7958ba79da7f584b5e761faa22a9ed4fdc53cd90027b0c13389388e130c6a8f.`

Exact commands, environment, device configuration, source state, timings, and artifact paths are collected in the corresponding dated terminal result under the repository's plan directory. It is the Actual-SV overcomplete analytical chart repair result dated 2026-07-17.

The resulting ledger action is deliberately narrow. The former Actual-SV fixed-square item is closed only as an experimental Contract E–TP local-chart and own-scalar engineering defect: the overcomplete route covers the declared finite points and differentiates its own finite scalar. The evidence does not close canonical Contract E–Chol wiring, cross-model propagation, scientific score equivalence, trajectory-region validity, HMC readiness, default readiness, or leaderboard admission.

### 26.12.6 Algorithmic form and complexity

The proposed local step is:

**Algorithm 5** Proposed Contract E–TP local score-aware reset

1: Compute the current LEDH corrected-weight increment $\Delta_t$ and normalized source weights; add $\Delta_t$ to the scalar.
2: Form structurally valid teacher candidates $z_{ij} = F_\theta(x_i, \varepsilon_j)$ and normalized weights $\pi_{ij}$.
3: Evaluate the fixed feature program $\psi_\theta(z_{ij})$ and reduce $b_\theta = \sum_{ij} \pi_{ij}\psi_\theta(z_{ij})$.
4: Use a prepared fixed active set $A$, or a reviewed full-support positive chart, and solve $\Phi_{A,\theta} q_\theta = b_\theta$.
5: Veto nonfinite, rank-deficient, nonpositive, or excessive-residual charts; do not clip weights or drop constraints.
6: Carry $\{s_\ell, q_\ell\}$ to the next LEDH step and initialize its prior log weights with $\log q_\ell$, not $-\log m$.
7: In reverse mode, differentiate teacher construction, feature reduction, the same linear solve, carried log weights, and every later finite LEDH operation.

With $N$ parents, $N$ innovation points, and $K$ retained features, dense teacher reduction costs $O(N^2K)$. A square chart adds $O(K^3)$, while the KKT form adds $O(mK^2+K^3)$. The projection is therefore semi-analytical; the accepted $N^2$ teacher work, not a large per-step optimizer, is expected to dominate. Streaming can reduce teacher storage from $O(N^2d)$ to blocks, provided the selected anchors and feature reductions are reproduced exactly in the pullback.

### 26.12.7 Concrete two-dimensional LGSSM witness

The reference experiment uses

$$x_t = A_\theta x_{t-1} + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, Q_\theta), \tag{26.81}$$

$$y_t = h^\top x_t + \nu_t, \qquad \nu_t \sim \mathcal{N}(0, R_\theta), \tag{26.82}$$

with

$$A_\theta = \begin{bmatrix} \rho & 0.18 \\ -0.12 & 0.83 \end{bmatrix}, \quad Q_\theta = e^{2\ell_q} \begin{bmatrix} 1 & 0.22 \\ 0.22 & 0.65 \end{bmatrix}, \quad h = \begin{bmatrix} 1 \\ 0.35 \end{bmatrix}, \quad R_\theta = e^{2\ell_r}. \tag{26.83}$$

This witness isolates the teacher and compression reset. It uses the bootstrap linear-Gaussian transition and observation potential directly; it does not execute an LEDH migration or PF–PF proposal correction. It can therefore validate the finite teacher-projection propositions and the carried-weight score mechanism, but not the surrounding LEDH flow implementation. At

$$\theta_0 = (\rho, \ell_q, \ell_r) = (0.72, -1.05, -0.70),$$

the prior is

$$x_0 \sim \mathcal{N}\left( \begin{bmatrix} 0.25 \\ -0.30 \end{bmatrix}, \begin{bmatrix} 0.70 & 0.16 \\ 0.16 & 0.45 \end{bmatrix} \right), \qquad (y_1, y_2) = (0.40, -0.15).$$

The realized matrices are

$$A = \begin{bmatrix} 0.72 & 0.18 \\ -0.12 & 0.83 \end{bmatrix}, \quad Q = \begin{bmatrix} 0.1224564283 & 0.0269404142 \\ 0.0269404142 & 0.0795966784 \end{bmatrix}, \quad R = 0.2465969639.$$

An order-nine tensor Gauss–Hermite rule in two dimensions gives 81 prior parents and 81 innovation points, hence a 6561-point teacher at time one. The teacher weights are the Gauss–Hermite product weights multiplied by $g_\theta(y_1 \mid z_{ij})$ and normalized. The seven retained features are

$$\psi_\theta(x) = \big(1, x_1, x_2, x_1^2, x_1x_2, x_2^2, r_{2,\theta}(x)\big)^\top, \tag{26.84}$$

where the linear-Gaussian predictive feature is available analytically:

$$r_{2,\theta}(x) = \mathcal{N}\big(y_2; h^\top A_\theta x, h^\top Q_\theta h + R_\theta\big). \tag{26.85}$$

A deterministic linear-program preparation at $\theta_0$ produced a positive basic feasible set with zero-based teacher indices

$$A = (108, 221, 2317, 2402, 2474, 3942, 4001).$$

Those indices are then frozen; only the seven-by-seven feature solve is differentiated. The resulting support and weights are:

| $\ell$ | $s_{\ell,1}$ | $s_{\ell,2}$ | $q_\ell$ |
|---|---|---|---|
| 1 | -3.4768598774 | -3.5565129989 | 0.0013273172 |
| 2 | -2.2613978461 | -1.2129585132 | 0.0002961221 |
| 3 | -0.5385388272 | -1.9705700614 | 0.0319724624 |
| 4 | -0.4079200865 | -0.1434149819 | 0.0377858212 |
| 5 | -0.6440561084 | 0.3400861366 | 0.0393703609 |
| 6 | 1.3859662461 | -1.8303729322 | 0.0541647250 |
| 7 | 0.4195516400 | -0.0203591973 | 0.8350831912 |

The scaled feature matrix has condition number 84.2606, and the minimum weight is $2.96122 \times 10^{-4}$; hence this center lies inside, rather than on the boundary of, the positive fixed chart.

Table 26.5 separates teacher accuracy from compression accuracy. The Kalman filter is the exact LGSSM oracle. The teacher is a finite quadrature approximation to it. The student is proved and observed to reproduce the finite teacher for the retained predictive feature.

Table 26.5: Two-observation 2D LGSSM value and score at $\theta_0 = (0.72, -1.05, -0.70)$.

| Method | log likelihood | $\partial_\rho$ | $\partial_{\ell_q}$ | $\partial_{\ell_r}$ |
|---|---|---|---|---|
| Kalman oracle | -1.637545116868 | -0.843840883733 | -0.405394763108 | -0.667711074507 |
| 6561-point teacher | -1.637542132400 | -0.843638383670 | -0.405408322446 | -0.667677325884 |
| 7-point student | -1.637542132400 | -0.843638383670 | -0.405408322446 | -0.667677325884 |
| Teacher minus Kalman | $2.98447 \times 10^{-6}$ | $2.02500 \times 10^{-4}$ | $-1.35593 \times 10^{-5}$ | $3.37486 \times 10^{-5}$ |
| Student minus teacher | 0 | 0 | $5.55 \times 10^{-17}$ | $-1.11 \times 10^{-16}$ |

The current teacher increment is $-0.952850708925$, and the retained next increment is $-0.684691423474$ for both teacher and student. In float64, the maximum feature-value residual is $1.30 \times 10^{-15}$, the maximum feature-tangent residual over all seven features and three parameters is $4.05 \times 10^{-15}$, and the largest autodiff versus centered finite- difference residual is $1.83 \times 10^{-10}$ with step $10^{-5}$.

The executable witness and its structured JSON result are stored under the chapter's benchmark and benchmark-artifact directories. Their exact paths, command, and source state are recorded in the dated experiment result note. It intentionally hides CUDA and runs as a small TensorFlow float64 CPU reference. It is implementation evidence for this numerical construction, not GPU/XLA or production evidence.

### 26.12.8 Mathematical and engineering boundary

The propositions prove a precise finite statement: a positive coreset exists at a fixed parameter value; a positive nonsingular frozen chart is locally differentiable; all retained feature totals and tangents match; and retaining the exact next finite predictive contribution makes the next value and score match the teacher. They do not prove any of the following:

- that a low-order or finite teacher equals the exact nonlinear filter;
- that first two moments plus one predictive feature determine the entire tangent filtering measure;
- that one active set stays positive and nonsingular over an HMC trajectory or posterior region;
- that selecting a new active set online is differentiable;
- that a 6561-to-7 compression remains accurate beyond the one retained future observation;
- that full-state Contract E affine anchors preserve the deterministic support of a degenerate NAWM model; or
- that this proposed route is HMC-ready, canonical, default-ready, or computationally feasible at NAWM scale.

The next scientific tests must therefore use, in order, a multi-step LGSSM rollout with Kalman value and score at every step, a singular structural-linear fixture whose deterministic completion is exact, and only then a small pruned DSGE model. Passing the numerical witness above establishes feasibility of the mechanism, not readiness of the algorithm.

## 26.13 Model-agnostic comparison of finite-filter scores

LGSSM experiments are diagnostic because their Kalman oracle exposes failures that would be hidden in a nonlinear model. An LGSSM-specific expected Fisher matrix is nevertheless the wrong general score-comparison interface: the intended particle-filter applications usually do not have an analytic Fisher matrix. This section therefore defines two diagnostics that require only a reference score and its time-local predictive increments. They do not change the finite LEDH scalar, its derivative, or Contract E. They change only how a candidate score error is reported.

Fix the parameter coordinates before comparison. In an HMC experiment these should normally be the unconstrained HMC coordinates. Let

$$\ell_T(\theta) = \sum_{t=1}^{T} \ell_t(\theta), \qquad s_t = \nabla_\theta \ell_t(\theta), \qquad g = \sum_{t=1}^{T} s_t, \tag{26.86}$$

where $\ell_t(\theta) = \log p_\theta(y_t \mid y_{1:t-1})$ for the declared reference filter. If $\widehat{g}$ is a candidate score in the same coordinates, write $\Delta = \widehat{g} - g$.

### 26.13.1 Global score-vector norm diagnostics

The direct global relative error is

$$E_{\text{total}} = \frac{\|\Delta\|_2}{\|g\|_2}. \tag{26.87}$$

It is meaningful when $\|g\|_2 > 0$, and must be reported as undefined rather than zero when the denominator is zero. Unlike a maximum of componentwise relative errors, it does not diverge merely because one realized score component is close to zero. It can, however, hide an inaccurate small component behind a large component, and the denominator can be small because predictive score increments cancel. Report the absolute error $\|\Delta\|_2$ with it.

A cancellation-resistant companion is

$$E_{\text{energy}} = \frac{\|\Delta\|_2}{\left(\sum_{t=1}^{T} \|s_t\|_2^2\right)^{1/2}}. \tag{26.88}$$

This denominator is the square root of the trace of the realized summed OPG matrix, equivalently $\sqrt{T \operatorname{tr}(\overline{S}_T)}$. It is also undefined if every reference increment is zero. Neither (26.87) nor (26.88) is invariant to an arbitrary rescaling of the parameter coordinates. Consequently the coordinate map is part of the artifact identity; a candidate and its reference must use the same map.

### 26.13.2 Average predictive-score OPG metric

Define the realized average outer-product-of-gradients matrix

$$\overline{S}_T = \frac{1}{T} \sum_{t=1}^{T} s_t s_t^\top. \tag{26.89}$$

This is not $gg^\top$. The latter has rank at most one and repeats the near-zero-denominator problem rather than estimating independent parameter scales. Nor is $\overline{S}_T$ the outer product or covariance of scores from repeated particle seeds on one observed data set. Centered variation across those seeds estimates Monte Carlo error; its uncentered second moment mixes the observed score, approximation bias, and Monte Carlo noise. Particle- seed variation belongs in uncertainty intervals, not in the normalization matrix.

For a diagonal-shrinkage coefficient $0 \le \lambda \le 1$, a positive diagonal scale matrix $D$, a base ridge $\epsilon_0 > 0$, and an optional nonnegative numerical floor $\epsilon_{\min}$, set

$$\rho_T = \max\left\{\frac{\epsilon_0}{T}, \epsilon_{\min}\right\}, \tag{26.90}$$

$$M_T = (1-\lambda)\overline{S}_T + \lambda \operatorname{Diag}(\operatorname{diag}\overline{S}_T) + \rho_T D, \tag{26.91}$$

$$J_T = TM_T. \tag{26.92}$$

The matrix $D$ supplies score-squared units and must be frozen independently of the candidate error. In dimensionless unconstrained coordinates, $D = I$ is a possible declared baseline, not a universal scientific default. A reference-derived diagonal scale may instead be frozen on separate simulated trajectories or a tuning split. Choosing $D$, $\lambda$, or either ridge after inspecting which setting makes a candidate pass is forbidden.

The total-score quadratic diagnostic and its diagonal companion are

$$E_{\mathrm{OPG}} = \left(\frac{1}{p}\Delta^\top J_T^{-1}\Delta\right)^{1/2}, \tag{26.93}$$

$$E_{\max,\mathrm{diag}} = \max_{1\le j\le p} \frac{|\Delta_j|}{\sqrt{(J_T)_{jj}}}. \tag{26.94}$$

The first detects errors in correlated directions and the second prevents a small coordinate from being hidden by the global norm. They are diagnostics, not acceptance thresholds until a model-class-specific plan freezes a criterion before candidate results are inspected.

**Proposition 26.18** (Finite-horizon well-posedness)**.** *Suppose $D \succ 0$, $\epsilon_0 > 0$, $T < \infty$, and $0 \le \lambda \le 1$. Then $M_T \succ 0$, even when $T < p$, and (26.93) is finite for every finite $\Delta$.*

*Proof.* For every nonzero $v$,

$$v^\top M_T v = \frac{1-\lambda}{T}\sum_{t=1}^{T}(v^\top s_t)^2 + \frac{\lambda}{T}\sum_{j=1}^{p} v_j^2 \sum_{t=1}^{T} s_{t,j}^2 + \rho_T v^\top D v.$$

The first two terms are nonnegative. The last is strictly positive because $\rho_T \ge \epsilon_0/T > 0$ and $D \succ 0$. Hence $M_T$, and therefore $J_T = TM_T$, is positive definite. □

The proposition makes a short-horizon diagnostic computable; it does not turn the missing directions into observed information. In particular, $\operatorname{rank}(\overline{S}_T) \le \min(T, p)$. For the current $T = 2, p = 5$ LGSSM diagnostic, at least three directions are supplied entirely by shrinkage and ridge. The artifact must report the unregularized rank or eigenvalues, $\rho_T$, and whether the numerical floor is active.

**Proposition 26.19** (Meaning of the $1/T$ ridge)**.** *Assume $\epsilon_{\min} = 0$, $D_T$ is bounded, and $\overline{S}_T \to S$. If $\lambda_T \to \lambda_\star$, then*

$$M_T \longrightarrow (1-\lambda_\star)S + \lambda_\star \operatorname{Diag}(\operatorname{diag} S),$$

*while*

$$J_T = T\left[(1-\lambda_T)\overline{S}_T + \lambda_T \operatorname{Diag}(\operatorname{diag}\overline{S}_T)\right] + \epsilon_0 D_T.$$

*Thus the ridge has constant size in the total-score metric and relative order $1/T$ in every persistently identified direction.*

*Proof.* Substitute $\rho_T = \epsilon_0/T$ into (26.91) and multiply by $T$. Boundedness of $D_T$ makes $(\epsilon_0/T)D_T \to 0$ in the average metric. The stated limits then follow by continuity of the diagonal map. □

Applying $\epsilon_0/T$ directly to the summed OPG would make the ridge only order $1/T^2$ relative to its data term. The average formulation is what gives the intended order $1/T$. A fixed $\lambda > 0$ does not disappear asymptotically: it converges to a diagonally shrunk metric, not to $S$. Recovering the unshrunk OPG requires $\lambda_T \to 0$. Similarly, when $\epsilon_{\min} > 0$ is active, the ridge no longer vanishes relative to the average metric. Persistent floor activity is evidence of weak excitation or numerical scaling, and must be reported rather than hidden as successful normalization.

Under the usual differentiability and interchange conditions at the true parameter, $\mathbb{E}[s_t \mid y_{1:t-1}] = 0$. Predictive score increments are then a martingale-difference sequence, so cross-time score covariances vanish and the expected sum of $s_t s_t^\top$ is the Fisher information of the observed-data score. On one realized trajectory, however, $\overline{S}_T$ remains a data-dependent OPG estimate. For short or weakly excited records it should be pooled over independently simulated or observed trajectories fixed before the candidate comparison. Exact LGSSM Fisher information is useful only as an oracle check that this generic construction behaves sensibly; it is not the normalization required by the nonlinear-model interface.

## 26.14 Implementation contract

A robust canonical implementation should expose two separable functions:

$$\text{reset_forward}(X, \log w, \Xi, \lambda; \eta_{\mathrm{OT}}) = (X^\star, \text{aux}), \qquad \text{reset_vjp}(\text{aux}, G_{X^\star}) = (G_X, G_{\log w}),$$

where $\eta_{\mathrm{OT}}$ contains the declared OT settings: cost convention, epsilon, annealing schedule, iteration budget, the repository-selected row/column chunk extent recorded for reproducibility, stabilization, and stop-gradient policy. For active canonical and candidate routes, the recorded chunk fields must equal the repository chunk-policy selector output from `dpf_transport_exact_divisor_cap3000_v1`; they are provenance fields, not caller-chosen tuning knobs. The selector requires equal row and column chunks, $K = N$ for $N \le 3000$, and for larger particle counts the largest divisor of $N$ not exceeding 3000, with fail-closed behavior if no such divisor exists. The canonical LEDH route also requires a repository-issued per-scope tuning artifact: any claim-bearing run must use controls tuned for the exact model/route/horizon/particle-count/dtype/chunk-policy scope, and a setting borrowed from another scope is only a warm start, not a default. The auxiliary record should contain only mathematical or numerical data needed to reproduce the same finite map. It should not silently change the scalar by switching from streaming to dense semantics, from finite Sinkhorn to another optimizer, or from one random stream to another.

The canonical TensorFlow implementation is owned by `ledh_contract_e_reset_tf.py`, `ledh_contract_e_streaming_tf.py`, and the canonical finite filter callable. The streaming layer carries the augmented payload $[X, 1]$, so its forward and reverse programs retain both $Q$ and $M$ without materializing a dense production matrix. Its VJP exposes source-particle direct and transport terms separately, probability-weight moment terms, normalized-log-weight transport terms, and their correctly composed totals. The checked LGSSM callable packages a primal traversal and a separate five-direction manual-JVP

traversal; pointwise tests match the manual JVP to forward autodiff of the private primal on frozen tiny fixtures. This is not a claim that the manual traversal is mechanically generated from one primal trace or cannot drift.

The following checks belong with any implementation:

(1) row and column residuals for the finite transport object;

(2) equality of dense and streaming VJP on small problems where both are feasible;

(3) finite-difference or raw-autodiff agreement on small fixed-seed cases;

(4) explicit tests for stopped versus differentiated scale, keys, and potentials;

(5) static-shape behavior under the intended compiled path;

(6) finite value and finite score on the target model class before any HMC use.

These checks certify local derivative accounting for the declared finite computation. They do not certify the statistical accuracy of the relaxed filter, posterior convergence, or default production readiness.

## 26.15 Relation to forward-mode JVP

Forward-mode JVP is also mathematically legitimate. For a perturbation $\dot{\theta}$, it propagates tangents through the same filtering loop and returns

$$D_\theta \mathcal{L}_K(\theta)\, \dot{\theta}.$$

This can be memory efficient, and it is useful for diagnostics along a small number of directions. For HMC, however, the sampler usually needs the whole score vector. If the parameter dimension is $d_\theta$, forward mode needs roughly $d_\theta$ directional passes unless it is batched over tangent directions. A VJP gives the full scalar-target score in one reverse pass. That is why the custom-gradient design in this chapter emphasizes the OT VJP. The practical forward-versus-reverse choice also depends on memory traffic, batching, output shape, and compiled-graph structure, not on parameter dimension alone. Separately, if any executed dependency is wrapped in a stop-gradient, the returned cotangent is a partial derivative of the same finite program with that dependency held constant; it equals the total derivative only when the omitted chain contribution is exactly zero, and the text must say which of those two objects is being reported.

## 26.16 What this chapter does and does not claim

This chapter gives a mathematical route for a custom gradient of the executed finite LEDH-PFPF-OT scalar. For the canonical Contract E route it claims:

- the raw barycentric OT product has the local VJP in (26.14), but that object is historical and is not the canonical reset;
- the canonical finite transport uses the row quotient in (26.3) and both quotient cotangents;

- the canonical total pullback includes both direct weighted-moment and streaming-transport terms as in (26.10);
- the normalized finite-transport matrix has the VJP in (26.18)–(26.21);
- finite Sinkhorn or annealed Sinkhorn can be differentiated by a reverse scan through the executed softmin program;
- composing those local VJPs through the filtering loop gives the derivative of the same executed scalar under fixed branch and randomness assumptions.

It does *not* claim that the relaxed OT filter equals a categorical particle filter, that finite Sinkhorn equals unregularized OT, that a custom VJP improves Monte Carlo accuracy, or that the resulting score is automatically HMC-ready for a structural model. It also does not claim that the checked tiny Contract E derivative certificates establish Kalman equivalence, full-time numerical adequacy, nonlinear state-support validity, v2 admission, leaderboard completeness, or default readiness. Those are separate validation questions. The Actual-SV overcomplete result in Section 26.12.5 is likewise an experimental finite-program certificate only. It repairs the declared local fixed-square chart and own-scalar derivative failure; it is not evidence that the separate canonical, HMC-region, scientific-equivalence, or leaderboard gates have passed.

# Part VII

# Learned Transport Extensions

# Chapter 27

# Retained-Teacher Neural OT Warm Starts

The preceding chapters fixed the entropic OT and Sinkhorn layer, together with its recovered custom-gradient bridge, as a fully declared teacher-side computational object. The present chapter introduces the narrowest neural extension of that teacher: do not replace the teacher; learn how to start it better.

This is the main implementation-bearing neural chapter of the OT block. Its question is narrower than "can a network learn transport?" It asks instead: can a network predict a good internal solver state so that a retained corrective Sinkhorn phase needs less work while still protecting the declared teacher object?

This narrowing is deliberate. The monograph begins neural OT with retained- teacher acceleration because it is the least radical learned extension of the existing OT block. The weighted cloud, the entropic teacher, and the barycentric output rule all remain declared in advance; only the route used to reach that teacher object is learned or accelerated.

## 27.1 What problem this chapter solves

Let $X = (x_1, \dots, x_N)^\top \in \mathbb{R}^{N \times d_x}$ be the weighted source cloud and let $w \in \Delta_N$ be its normalized weights. Let $u = (1/N, \dots, 1/N)$ be the equal target marginal, and let $C(X, Z) \in \mathbb{R}^{N \times N}$ be the declared transport cost matrix. The previous chapter already fixed the finite entropic-OT/Sinkhorn route as the reference computational teacher for turning this weighted cloud into an equal-weight cloud.

The practical question now is not whether the teacher should be changed, but whether its execution path can be improved. The chapter therefore asks:

> Can we learn a solver-relevant latent state that makes the retained corrective Sinkhorn solve cheaper while still returning to the same declared finite teacher object after correction?

**How to read the rest of this chapter.** The chapter has one primary lane and one secondary lane. The primary lane is the warm-start story: define the teacher, define the latent state the student predicts, run the corrective retained solver, and judge the corrected cloud against the teacher. Only after that core story is in place does the chapter widen into a shorter survey of broader retained-teacher families such as Nystr"om, positive-feature, and low-rank routes.

| Object | Meaning in this chapter |
|---|---|
| exact entropic teacher $\Pi_\varepsilon^\star$ | optimizer of the declared regularized OT problem |
| finite numerical teacher $\Pi_{\varepsilon,K}$ | entropic teacher after a declared finite number of Sinkhorn repairs |
| latent warm-start target $h_\tau$ | solver-relevant coordinate such as duals, log-scalings, or scaling vectors |
| corrected student output $\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$ | coupling produced after student initialization and retained corrective Sinkhorn steps |
| barycentric output $B(\Pi)$ | equal-weight cloud later code actually consumes |

## 27.2 Running warm-start cell

Keep the same three-particle setting as the previous OT chapters: a weighted source cloud, an equal target marginal, a declared quadratic cost, a regularization level $\varepsilon$, and a finite Sinkhorn budget $K$. The next section will restate the teacher formally; for now the important point is that the retained finite teacher is the coupling $\Pi_{\varepsilon,K}(X, w)$ produced by that declared Sinkhorn route.

A warm-start learner does not try to predict the transported output cloud first. It tries to predict a latent teacher coordinate, such as dual variables, log-scalings, or scaling vectors, that initializes the corrective solve. Write

$$h_\psi = S_\psi(X, w; \varepsilon, N, d_x), \tag{27.1}$$

where $h_\psi$ is later translated into the initialization of the retained corrective Sinkhorn solver.

For a concrete exposition, it is useful to keep one canonical latent target in mind. Suppose the student predicts initial log-scalings

$$h_\psi = (\ell_a^{(0)}, \ell_b^{(0)}), \tag{27.2}$$

which are then exponentiated to give

$$a^{(0)} = \exp(\ell_a^{(0)}), \qquad b^{(0)} = \exp(\ell_b^{(0)}). \tag{27.3}$$

The corrective retained-teacher route then performs the same balancing equations as before,

$$a^{(k+1)} = w \oslash (K_\varepsilon b^{(k)}), \qquad b^{(k+1)} = u \oslash (K_\varepsilon^\top a^{(k+1)}), \tag{27.4}$$

until the declared correction budget is exhausted. This makes the chapter's teacher-preserving logic explicit: the learner may change the starting point of the finite Sinkhorn route, but the corrected route still returns to the same declared kernel, the same marginal constraints, and the same barycentric output rule.

The operational contrast is then simple:

- a naive initialization changes only how quickly the solver reaches the finite teacher,
- a learned initialization also changes only that execution path,
- the corrected coupling and barycentric output are still judged against the same declared finite teacher object.

**Running-example walkthrough.** The warm-start story can now be read in one seven-step ladder:

(1) start from a weighted cloud $(X, w)$, a declared cost, and the equal target marginal $u$,

(2) define the retained finite teacher coupling $\Pi_{\varepsilon,K}$,

(3) choose a latent teacher coordinate to predict, such as log-scalings,

(4) let the student predict that latent coordinate,

(5) translate the prediction into the initialization of the retained corrective Sinkhorn routine,

(6) run the declared number of corrective updates to obtain the corrected coupling $\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$,

(7) form the barycentric equal-weight cloud and compare it against the teacher output through residual checks.

The rest of the chapter merely fills in this ladder mathematically and then asks how adjacent retained-teacher families fit into the same audit posture.

**One concrete warm-start recipe.** To keep the chapter operational, fix one explicit reading. Suppose the teacher latent target is the pair of initial log-scalings $(\ell_a^{(0)}, \ell_b^{(0)})$, the student network predicts exactly that pair, and deployment uses a declared corrective budget $K_{\mathrm{corr}}$. Then one practical recipe is:

(i) solve the retained teacher route on training clouds and store the resulting target log-scalings,

(ii) train the student to predict those log-scalings from $(X, w)$,

(iii) exponentiate the predicted pair to form $(a^{(0)}, b^{(0)})$,

(iv) run $K_{\mathrm{corr}}$ corrective Sinkhorn updates,

(v) compare the corrected barycentric cloud and residuals against the teacher.

This is only one possible retained-teacher design, but it is concrete enough that a first-year reader can see what is stored, what is predicted, and what is still protected by correction.

**Plain-language takeaway.** A warm-start learner is not asked to invent the transport result. It is asked to guess a better place from which the retained teacher solver should begin.

**What the rest of the chapter is for.** A first-year reader should treat everything through the end of the residual hierarchy as the primary implementation story of the chapter. The later Nystr"om, positive-feature, low-rank, and minibatch sections are broader retained-teacher family context: they are useful because they preserve the same audit posture, but they are not new primary objects to learn first. If the notation starts to feel dense, return to the seven-step warm-start ladder above and ask how each later family changes only the route to the teacher rather than the teacher object itself.

## 27.3 What the teacher computes

There is no single teacher unless the variant is named. In this chapter, the main teacher is the retained EOT/Sinkhorn route from the previous chapter. The reader therefore needs one short local recap of that teacher before any learned warm start is introduced.

Given weighted source cloud $X = (x_1, \dots, x_N)^\top$, source marginal $w \in \Delta_N$, equal target marginal $u = (1/N, \dots, 1/N)$, declared cost matrix $C$, and regularization $\varepsilon > 0$, the entropic teacher object is

$$\Pi_\varepsilon^\star = \arg\min_{\Pi \in \mathcal{U}(w,u)} \left\{ \langle \Pi, C \rangle + \varepsilon \sum_{i,j} \Pi_{ij}(\log \Pi_{ij} - 1) \right\}. \tag{27.5}$$

Its Gibbs/scaling form is

$$\Pi_\varepsilon^\star = \operatorname{diag}(a) K_\varepsilon \operatorname{diag}(b), \qquad (K_\varepsilon)_{ij} = \exp(-C_{ij}/\varepsilon), \tag{27.6}$$

with scaling vectors satisfying the balancing equations

$$a = w \oslash (K_\varepsilon b), \qquad b = u \oslash (K_\varepsilon^\top a). \tag{27.7}$$

After a declared finite number $K$ of Sinkhorn repairs, the finite numerical teacher is

$$\Pi_{\varepsilon,K} = \operatorname{diag}(a^{(K)}) K_\varepsilon \operatorname{diag}(b^{(K)}). \tag{27.8}$$

The corresponding barycentric teacher output is

$$Y_\tau(X, w, \varepsilon) = B(\Pi_\tau(X, w, \varepsilon)), \qquad B(\Pi)_j = \frac{1}{u_j} \sum_{i=1}^N \Pi_{ij} x_i. \tag{27.9}$$

In the equal-weight case, this is

$$B(\Pi)_j = N \sum_{i=1}^N \Pi_{ij} x_i. \tag{27.10}$$

These formulas identify the three most important teacher objects in this chapter:

(i) the exact regularized coupling $\Pi_\varepsilon^\star$,

(ii) the finite numerical teacher $\Pi_{\varepsilon,K}$,

(iii) the barycentric teacher output $Y_\tau = B(\Pi_\tau)$.

The chapter's neural route is only intended to accelerate the finite teacher, not to redefine these objects.

**Object audit.** The learned quantity is a latent solver state. The corrected transport object is still the finite teacher-side coupling or its barycentric output. In the vocabulary of the OT block, this is an execution-path approximation, not a transport-object change.

**Canonical reading convention.** For the rest of this chapter, the safest first-year reading convention is:

- $\Pi_\varepsilon^\star$ is the exact entropic teacher,
- $\Pi_{\varepsilon,K}$ is the retained finite numerical teacher,
- $h_\tau$ is the latent teacher coordinate the student is asked to learn,
- $\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$ is the corrected student output,
- all residuals are interpreted relative to that corrected student output versus the chosen finite teacher.

Whenever the notation becomes crowded, return to that five-line convention rather than trying to remember every symbol historically.

**Teacher notation shortcut.** To keep the notation stable, the chapter will use the following shorthand from this point onward:

- exact teacher $= \Pi_\varepsilon^\star$,
- finite teacher $= \Pi_{\varepsilon,K}$,
- generic teacher-side cloud $= Y_\tau = B(\Pi_\tau)$, with $\tau$ always meaning the chosen retained teacher route,
- corrected student output $= \widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$.

So when a later section says "compare against the teacher," the default first-year reading should be: compare the corrected student output against the chosen finite retained teacher unless the exact teacher is explicitly named.

## 27.4 Algorithm: retained-teacher warm-started Sinkhorn

**What the student is allowed to change.** Only the initialization of the retained teacher solver, or another explicitly named latent teacher coordinate.

**What the teacher still protects.** The declared transport constraints, the EOT objective, and the corrected barycentric output after refinement.

**Why this chapter keeps log-scalings as the running latent target.** The chapter names duals, log-scalings, and scaling vectors because all three occur naturally in entropic OT implementations. But the running exposition uses log-scalings because they are concrete, per-particle, and easy to translate back into the retained corrective solver. A first-year reader should therefore treat the log-scaling example as the chapter's canonical student target unless a later section explicitly announces a different latent object.

**Algorithm 6** `train_and_deploy_warmstarted_sinkhorn`

**Require:** training distribution $\mathcal{D}$ over weighted particle clouds, declared cost construction, regularization schedule $\varepsilon$, teacher Sinkhorn solver, architecture class $S_\psi$, deployment-time corrective budget $K_{\mathrm{corr}}$

**Ensure:** trained predictor $S_\psi$, corrected deployment coupling $\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$, barycentric output cloud $\widetilde{X}_{\psi,\mathrm{corr}}$, and declared residual diagnostics

1: Generate teacher problems by solving the retained EOT/Sinkhorn route on sampled clouds from $\mathcal{D}$
2: Extract the latent teacher target to be predicted, e.g. dual/scaling coordinates
3: Fit the predictor $S_\psi$ on this teacher data using the declared loss family
4: For a new weighted cloud $(X, w)$, compute the student prediction $h_\psi = S_\psi(X, w; \varepsilon, N, d_x)$
5: Translate $h_\psi$ into the initial state of the retained corrective Sinkhorn solver
6: Run $K_{\mathrm{corr}}$ corrective Sinkhorn updates to obtain $\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}$
7: Form the barycentric equal-weight output cloud $\widetilde{X}_{\psi,\mathrm{corr}} = B(\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}})$
8: Record row/column residuals, teacher-student discrepancy, and any downstream scalar checks

## 27.5 Evidence contract for retained-teacher learning

Let $h_\tau$ denote the teacher latent target and let $\mathcal{D}(N, d_x, \varepsilon, m, \eta)$ be the training distribution over problem instances. A simple supervised teacher-student fitting problem is

$$\psi^\star \in \arg\min_\psi \mathbb{E}_{(X,w,\varepsilon,m)\sim\mathcal{D}} \left[\mathcal{L}\{S_\psi(X, w; \varepsilon, N, d_x), h_\tau(X, w, \varepsilon)\}\right]. \tag{27.11}$$

In practice, the deployment claim is not about this loss alone. It is about whether the corrected route preserves the teacher output well enough on held-out and stress-test clouds.

### 27.5.1 Permutation symmetry as a design check

Particle labels are arbitrary. For a permutation matrix $P \in \mathbb{R}^{N\times N}$, the predictor should at least satisfy a declared permutation symmetry. In the output-cloud case the natural condition is

$$S_\psi(PX, Pw; \varepsilon, N, d_x) = P\, S_\psi(X, w; \varepsilon, N, d_x) \quad \text{or the corresponding latent-state analogue.} \tag{27.12}$$

The distinction matters because different latent targets transform differently. If the student predicts a per-particle object such as initial log-scalings, then permutation equivariance is the natural symmetry. If the student predicts a compressed global latent statistic, a weaker invariance or gauge-fixed analogue may be more appropriate. In either case, the chapter's main design check is the same: relabeling particles should not change the mathematical content of the predicted teacher state.

### 27.5.2 Residual hierarchy

A retained-teacher route should be judged by a hierarchy of residuals rather than by one discrepancy alone. The first checks are the marginal feasibility residuals of the corrected

route,

$$r_{\mathrm{row}} = \|\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}\mathbf{1} - w\|_1, \qquad r_{\mathrm{col}} = \|\widehat{\Pi}'_{\psi,\varepsilon,K_{\mathrm{corr}}}\mathbf{1} - u\|_1. \tag{27.13}$$

These ask whether the corrected route still satisfies the declared transport constraints. They do *not* by themselves show that the corrected student has reproduced the teacher's transported cloud.

The next deployment residual is the teacher-student discrepancy after correction:

$$R_{\psi,\tau}(X, w, \varepsilon) = B(\widehat{\Pi}_{\psi,\varepsilon,K_{\mathrm{corr}}}) - B(\Pi_\tau(X, w, \varepsilon)). \tag{27.14}$$

This barycentric discrepancy measures how well the corrected student reproduces the chosen teacher cloud. It still does not by itself control filtering or HMC target error. For kernel-compression families such as Nystr"om or positive-feature Sinkhorn, one may also monitor the transported-cloud discrepancy more directly,

$$\|\widetilde{P}X - P_\tau X\|, \tag{27.15}$$

because the transport application is the actual object later code consumes.

**Mini-example of the hierarchy.** The logic of these residuals is easiest to remember by a simple contrast. A route may satisfy the row and column sums almost exactly, so $r_{\mathrm{row}}$ and $r_{\mathrm{col}}$ look excellent, yet still move mass through a factorized or restricted representation whose transported cloud differs visibly from the chosen teacher output. In that case feasibility has been preserved, but teacher parity has not. Conversely, a corrected route might match the teacher cloud very closely while a downstream scalar still drifts because that scalar is more sensitive than the cloud discrepancy alone suggests. This is why the residual hierarchy must be read in order: feasibility first, teacher-cloud parity second, downstream scalar checks third.

A practical retained-teacher claim should therefore keep the following evidence contract explicit:

(1) the predicted latent teacher object is named explicitly,

(2) the corrected route satisfies the declared row/column residual contract,

(3) any factorized kernel or coupling approximation is checked against the teacher at the transported-cloud level,

(4) the corrected cloud discrepancy remains below the declared threshold on the tested envelope,

(5) any downstream scalar or HMC-adjacent target is checked only after the corrected output is formed.

A useful first-year rule of thumb is: if the marginal constraints are good but the transported-cloud discrepancy is visibly non-small on the toy or dense reference problems, the warm start is not yet trustworthy no matter how fast it runs. If the transported cloud is close but a downstream scalar still drifts materially, then the chapter's local parity checks have passed only at the cloud level and the learned route still should not be promoted for that scalar use case.

### 27.5.3 Where broader retained-teacher families fit

The chapter's core implementation story stops with the retained warm-start lane above. The remaining subsections should be read as broader retained-teacher family context: they answer the same teacher-preservation question, but they are not new primary chapter threads. Their role is to show how adjacent scalable-entropic families can still be interpreted through the same retained-teacher audit when the first test remains parity against the declared finite teacher.

Warm-start predictors are the narrowest retained-teacher lane. Other scalable OT families — such as Nystrom, positive-feature, low-rank, or minibatch teacher-side approximations — may still belong to the same lane only when their first audit question is teacher preservation rather than speed alone [Altschuler et al., 2019, Scetbon and Cuturi, 2020, Forrow et al., 2019, Scetbon et al., 2021, Scetbon and Cuturi, 2022, Genevay et al., 2016, 2019, Nguyen et al., 2022, 2021]. Once the approximation is no longer naturally judged by teacher-preservation residuals, it belongs in the later target-changing chapters instead of this one.

**How to read the family context that follows.** If the next subsections start to feel denser than the warm-start core, the safest first-year reading rule is simple: treat them as alternate ways to preserve the same teacher-side audit posture, not as new primary chapter threads. In other words, keep asking the same question: what compressed route is being proposed, and how is its corrected output compared back to the finite retained teacher?

### 27.5.4 Exact online and GPU Sinkhorn as the reference lane

Before turning to approximate retained-teacher families, the reader should keep one semantics-preserving reference lane in view. Exact online or GPU-specialized Sinkhorn methods preserve the same entropic teacher object while changing only how the kernel reductions are executed in memory or on hardware [Feydy et al., 2019, Charlier et al., 2021, Ye et al., 2026, Xiao, 2026]. For squared Euclidean costs,

$$C_{ij} = \|x_i - z_j\|_2^2 = \|x_i\|_2^2 + \|z_j\|_2^2 - 2x_i' z_j, \tag{27.16}$$

and a stabilized potential update can be written schematically as a row-wise log-sum-exp reduction,

$$f_i \leftarrow -\varepsilon \log \sum_j \exp\left((g_j - C_{ij})/\varepsilon + \log u_j\right). \tag{27.17}$$

The same teacher-side barycentric output is then recovered from the resulting finite coupling. This route is the reference lane because it preserves semantics while avoiding full dense materialization where possible. It is not yet a new transport approximation class. It also does not remove the dominant all-pairs interaction structure. That is exactly why the later factorized families matter: they are not introduced before the reader has a semantics-preserving reference route in hand.

### 27.5.5 Nystrom kernel approximations

A natural retained-teacher route is to approximate the Gibbs kernel rather than the final transport object [Altschuler et al., 2019]. For a Gaussian-like kernel, write

$$K_{ij} = k_\eta(x_i, x_j) = \exp(-\eta\|x_i - x_j\|^2). \tag{27.18}$$

The Nystr"om idea is to choose a smaller landmark set

$$X_r = \{\tilde{x}_1, \dots, \tilde{x}_r\} \subset X \tag{27.19}$$

and define the factor matrices

$$V_{ij} = k_\eta(x_i, \tilde{x}_j), \qquad A_{jj'} = k_\eta(\tilde{x}_j, \tilde{x}_{j'}). \tag{27.20}$$

Then the dense kernel is approximated by

$$\widetilde{K} = VA^{-1}V'. \tag{27.21}$$

This is not just a storage identity. It changes how the expensive kernel action is computed. If $A = LL'$ is a Cholesky factorization, then a matrix-vector product is applied as

$$\widetilde{K}v = V\left(L'^{-1}\left(L^{-1}(V'v)\right)\right), \tag{27.22}$$

so the dominant kernel multiply is now factorized rather than dense. After the factorization has been built, each multiply costs $O(Nr)$ rather than $O(N^2)$.

Because Sinkhorn balancing uses kernel-vector products, the same factorization feeds directly into the retained finite teacher route. The approximate teacher-side plan becomes

$$\widetilde{P} = D_1\widetilde{K}D_2 = D_1VA^{-1}V'D_2, \tag{27.23}$$

and the barycentric transport application becomes

$$\widetilde{P}X = D_1VA^{-1}V'D_2X. \tag{27.24}$$

This is the key retained-teacher ladder for the family:

(i) approximate the kernel action,

(ii) run the same teacher-side balancing logic on the factorized action,

(iii) compare the corrected transport output against the dense or streamed teacher.

A practical Nystr"om route therefore needs more than the factorization formula itself. The rank is usually increased adaptively until the approximation is judged good enough. A simple entrywise diagnostic from the scalable-OT note is

$$\|K - \widetilde{K}\|_\infty = 1 - \min_i \widetilde{K}_{ii}, \tag{27.25}$$

which asks whether the approximate kernel still reproduces the self-similarity scale of the original kernel. The deeper lesson is geometric: Nystr"om helps when the post-flow Gibbs kernel has low effective rank, but it can fail when the rank needed to represent the kernel grows quickly with ambient or intrinsic dimension, or when the entropic regularization is too small for low-rank structure to remain visible.

For retained-teacher use, the decisive checks are still teacher-facing rather than speed-facing. In addition to marginal residuals, one should monitor the transported-cloud discrepancy

$$\|\widetilde{P}X - P_\tau X\|, \tag{27.26}$$

because the transported cloud is the actual object later code consumes. The primary diagnostics are therefore not speed alone but kernel-approximation error, marginal residuals after correction, and transported-cloud discrepancy against the chosen teacher route.

### 27.5.6 Positive-feature Sinkhorn

A related route factorizes the kernel through positive features rather than through landmark columns [Scetbon and Cuturi, 2020]. Choose a nonnegative feature map

$$\phi_\theta(x) \in \mathbb{R}_+^r \tag{27.27}$$

and define the induced kernel

$$K_\theta(x_i, x_j) = \langle \phi_\theta(x_i), \phi_\theta(x_j) \rangle. \tag{27.28}$$

The corresponding cost representation is then

$$c_\theta(x_i, x_j) = -\varepsilon \log K_\theta(x_i, x_j). \tag{27.29}$$

At the matrix level, define source and target feature collections by

$$\Xi = [\phi_\theta(x_1), \dots, \phi_\theta(x_N)], \tag{27.30}$$
$$\mathrm{Z} = [\phi_\theta(z_1), \dots, \phi_\theta(z_N)], \tag{27.31}$$

so the sample kernel factorizes as

$$K_\theta = \Xi' \, \mathrm{Z}. \tag{27.32}$$

Balancing steps can therefore be expressed through feature-space products rather than through a materialized dense kernel. After the feature matrices are formed, the dominant kernel-vector products scale like $O(r(N + N))$, so the attraction is again execution-path efficiency rather than a new transport doctrine.

The same factorization also has a useful integral reading. One may view the kernel as arising from a positive feature integral

$$k(x, y) = \int_U \phi(x, u)' \phi(y, u) \, d\rho(u), \tag{27.33}$$

and then approximate that integral by a finite feature family,

$$\phi_\theta(x) = \frac{1}{\sqrt{r}} (\phi(x, u_1), \dots, \phi(x, u_r)). \tag{27.34}$$

This explains why the factorized kernel should still be read as an approximation to the teacher-side kernel action, not automatically as the exact original kernel. The corresponding factored transport application is

$$PX = \mathrm{diag}(u) \Xi' \, \mathrm{Z} \, \mathrm{diag}(v) X. \tag{27.35}$$

This gives the same pedagogical ladder as Nystr"om:

(i) replace dense kernel application by a factorized kernel action,

(ii) run the retained balancing logic through the factors,

(iii) judge the corrected route by teacher-preservation residuals rather than by runtime alone.

One can also write the induced entropic dual in factorized form,

$$W_{\varepsilon,c_\theta}(\mu,\nu) = \max_{\alpha,\beta} a'\alpha + b'\beta - \varepsilon(\Xi e^{\alpha/\varepsilon})' \, \mathrm{Z} e^{\beta/\varepsilon} + \varepsilon, \tag{27.36}$$

which makes the same point from the dual side: the expensive dense interaction is replaced by feature-space products. The caution is sharper than for a pure engineering variant. A positive-feature route changes the effective kernel representation and therefore must be judged by teacher-preservation residuals, marginal checks, and transported-cloud parity, not by runtime alone. It is retained-teacher only to the extent that the corrected route is still explicitly audited against the chosen teacher object.

### 27.5.7 Direct low-rank coupling approximations

Kernel approximations are not the only retained-family option. One may also factorize the coupling itself [Forrow et al., 2019, Scetbon et al., 2021, Scetbon and Cuturi, 2022, Scetbon et al., 2023]. The key conceptual distinction is:

> Approximating the kernel asks how to preserve the same teacher-side transport from a compressed kernel action. Constraining the coupling asks whether a restricted family of couplings is already sufficient.

The second route is more than a fast matvec; it changes the feasible family being searched.

A useful formal entry point is the nonnegative rank of a nonnegative matrix,

$$\mathrm{rank}_+(M) = \min\left\{ r : \; M = \sum_{k=1}^{r} M_k, \; \mathrm{rank}(M_k) = 1, \; M_k \geq 0 \right\}. \tag{27.37}$$

The rank-constrained coupling family is then

$$\Pi_{a,b}(r) = \{P \in \Pi(a,b) : \mathrm{rank}_+(P) \leq r\}. \tag{27.38}$$

A generic factorized parameterization is

$$P = Q \, \mathrm{diag}(1/g) R', \tag{27.39}$$

with factor constraints

$$Q \in \Pi(a,g), \qquad R \in \Pi(b,g), \qquad g \in \Delta_r. \tag{27.40}$$

These constraints mean that the factors themselves carry source mass $a$, target mass $b$, and an intermediate latent mass profile $g$. Equivalently, one may speak of the factored feasible family

$$FC_{a,b}(r) = \{(Q,R,g) : Q \in \Pi(a,g), \; R \in \Pi(b,g), \; g \in \Delta_r\}, \tag{27.41}$$

so the low-rank optimization problem can be read either as a search over rank-constrained couplings or as a search over factored feasible triples. In the same notation, the objective itself is

$$\min_{P \in \Pi_{a,b}(r)} \langle C, P \rangle = \min_{(Q,R,g) \in FC_{a,b}(r)} \langle C, Q \, \mathrm{diag}(1/g) R' \rangle \, . \tag{27.42}$$

Transport application then becomes

$$PX = Q\,\mathrm{diag}(1/g)R'X, \tag{27.43}$$

so storage and barycentric application are both executed through the low-rank factors rather than through a dense coupling.

A richer extension introduces a small latent transport plan $T$ between the two factor families. In schematic form, the induced coupling becomes

$$P = Q\,\mathrm{diag}(1/g_Q)T\,\mathrm{diag}(1/g_R)R', \tag{27.44}$$

which allows mass to route through a compact latent transport instead of only through diagonal latent components. A practical rank-adequacy question then appears: is the chosen rank large enough? The note's stronger low-rank lines use reduced-gradient or dual-slack diagnostics for that purpose. Even without carrying those full certificates into this chapter, the reader should keep the right lesson in view: low-rank coupling is not only a storage trick; it is an optimization over a restricted feasible family, so parity against the teacher must be checked whenever rank is fixed in advance.

For the present chapter, the important boundary is interpretive. If the first test remains teacher-preservation parity after correction, then the family can still be read through the retained-teacher lens. If the prose instead starts defending the restricted family as a different final transport object in its own right, the discussion has crossed toward the changed-object lane treated more naturally in the next chapter.

### 27.5.8 Stochastic and minibatch cautions

Stochastic or minibatch transport ideas are tempting because they reduce local work per update [Genevay et al., 2016, 2019, Nguyen et al., 2022, 2021]. But a minibatch-optimal plan is not automatically a valid full- ensemble deterministic resampling map. Solving transport on sampled subclouds or local batches may be a useful approximation strategy, yet the first question remains whether the resulting corrected route preserves the declared teacher well enough on the full weighted ensemble.

For this reason, minibatch variants belong to the retained-teacher lane only when a later correction stage or explicit audit shows that full-ensemble marginals, transported-cloud behavior, and downstream scalars still track the declared teacher route. The practical validation sequence should therefore stay stricter than a paper-summary reading:

(1) first check small dense transported-particle parity, not only scalar OT values,

(2) then check row and column marginals plus positivity/non-finite failures,

(3) then inspect representation diagnostics such as rank, feature dimension, sparsity, projection count, or block locality,

(4) only after value parity consider gradient smoke tests and larger- $N, d_x$ runtime or memory ladders,

(5) and only after those local checks interpret downstream filtering evidence.

If the minibatch construction is instead promoted as its own randomized or local replacement transport, it should be read through the later structure-changing chapters rather than through the present retained-teacher one.

| Family | What is approximated? | What stays teacher-protected? | First residual to check |
|---|---|---|---|
| Warm start | solver initialization or latent state | finite Sinkhorn teacher and barycentric output | corrected cloud discrepancy |
| Nystrom / positive-feature | kernel action or feature representation | declared teacher-side transport after correction | marginal and teacher-preservation residual |
| Low-rank coupling | feasible coupling representation | declared output object only if parity remains the first test | coupling/cloud parity under restriction |
| Minibatch / stochastic routes | local or sampled transport work | teacher status only if full-ensemble behavior is still audited | full-ensemble marginal and cloud behavior |

### 27.5.9 How the broader retained-teacher approximations fit here

The same residual logic gives the chapter a place to classify the broader scalable-OT approximations from the survey. If a Nystrom factorization, a positive-feature kernel, or a low-rank coupling factorization is proposed as a retained-teacher acceleration, then the first audit question is still teacher preservation: how close is the transported cloud or coupling application to the declared teacher route from Chapter 25?

For example, a kernel-factor route may replace the dense Gibbs kernel by a factorized approximation while still producing a coupling application of the same schematic form. A low-rank coupling route may store and apply transport through structured factors rather than a dense matrix. A stochastic or minibatch route may reduce computation by solving smaller transport subproblems. In all such cases, this chapter's doctrine is the same: they are still retained-teacher candidates only if the approximation error is measured against the teacher object rather than hidden behind speedups alone.

That is why the residual hierarchy should be read broadly. The diagnostic is not restricted to one neural warm-start map. It is the first reusable test for any approximation family that claims to accelerate the teacher without changing the meaning of the transported output. Once the approximation stops being naturally judged by teacher-preservation residuals, it belongs in the later target-changing chapters instead of this retained-teacher one.

## 27.6 Filtering-loop insertion point

Inside the filtering loop, the retained-teacher neural route sits exactly where Chapter 25 placed the finite EOT/Sinkhorn layer:

$$\{x_i, w_i\}_{i=1}^N \xrightarrow{\text{student warm start+retained Sinkhorn}} \{\tilde{x}^{(j)}, 1/N\}_{j=1}^N. \tag{27.45}$$

It accelerates the equal-weighting layer after the weighted cloud has already been formed. It does not redefine the earlier particle-weighting rule itself.

## 27.7 Verification and non-claims

A practical implementation should still check a few local quantities carefully:

(1) teacher generation is declared and reproducible,

(2) the predicted latent teacher object is named explicitly,

(3) the corrected route satisfies the row/column residual contract,

(4) teacher-student cloud discrepancy remains below the declared threshold on the tested envelope,

(5) the downstream scalar is checked whenever the learned route is embedded in a filtering likelihood or HMC target.

These checks are mathematical and numerical, not bureaucratic. Their purpose is only to ensure that the reader can tell whether the implementation is still computing the declared teacher-student object or has drifted to a different one.

This chapter does *not* claim:

- that the student replaces the teacher object,
- that small teacher-student residuals automatically imply posterior or HMC correctness,
- that broader neural OT families are unimportant, only that they answer a different implementation question.

Its role is narrower: fix one teacher-student acceleration route in a form that a reader can nearly implement directly.

# Chapter 28

# ICNN, Brenier Maps, and Monge-Gap Direct Map Learning

The previous chapter studied the narrowest neural extension of the entropic OT teacher: retain the teacher and learn how to start its solver better. The present chapter changes the question. Instead of accelerating a declared solver, it asks how one can learn the transport map itself.

This is a more ambitious route. The potential gain is obvious: if a good map is learned directly, one may bypass repeated iterative solves at deployment time. The cost is equally important: the learned object is no longer an initialization for a retained teacher. It is the transport map itself, so every modeling and training choice now acts directly on the output cloud.

This chapter therefore has one dominant task: teach the reader what direct-map learning is trying to learn, why Brenier/ICNN and Monge-gap methods are natural representatives of that lane [Amos et al., 2017, Makkuva et al., 2020, Uscidda and Cuturi, 2023], and what evidence is needed once no corrective teacher solve remains in control.

## 28.1 What changes when the map itself is learned?

The best way to read this chapter is to compare it with the retained-teacher warm-start chapter.

- In the warm-start chapter, the teacher still decides what the final transport plan and output cloud are; the learner only helps the teacher solve its own problem faster.

- In the present chapter, the learner itself proposes the transport map. There may be no corrective Sinkhorn phase at all.

So the central question is no longer "did we accelerate the same teacher?" It is "what map object have we learned, and under what assumptions should we trust it?"

**Plain-language takeaway.** If the previous chapter was about helping an expensive trusted calculation, this chapter is about replacing that calculation by a learned map. That is why the mathematical structure and the evidence contract both change.

**How to read the rest of this chapter.** The sequence now splits into two direct-map routes. First comes the Brenier/ICNN family, which keeps a strong structural prior by learning a convex potential whose gradient defines the map. Then comes the Monge-gap family, which keeps the same direct-map goal but relaxes the structural prior and asks instead how OT-like the learned endpoint map remains under the chosen cost. Both routes differ from the previous chapter in the same decisive way: there is no corrective teacher solve standing behind the final output object.

## 28.2 Running direct-map cell

A tiny one-dimensional cell is enough to carry the main ideas. Let the source be the two-point cloud

$$X_{\mathrm{src}} = \{-1, 1\} \quad \text{with equal weights } \frac{1}{2},$$

and let the target be

$$X_{\mathrm{tgt}} = \{0, 2\} \quad \text{with equal weights } \frac{1}{2}.$$

The obvious translation map is

$$T^\star(x) = x + 1. \tag{28.1}$$

Applying $T^\star$ to the source cloud sends $-1 \mapsto 0$ and $1 \mapsto 2$, so the pushforward equals the target cloud exactly.

For quadratic cost,

$$\frac{1}{2} \sum_{x \in X_{\mathrm{src}}} |x - T^\star(x)|^2 = \frac{1}{2}(1^2 + 1^2) = 1. \tag{28.2}$$

This simple cell already shows the difference between a direct map and a retained teacher solve. The output cloud is obtained directly by evaluating $T$ on the source particles; there is no intermediate coupling matrix or balancing loop in this picture.

A second map on the same endpoint cloud shows why direct-map evidence is more subtle. Consider

$$T_{\mathrm{good}}(-1) = 0, \qquad T_{\mathrm{good}}(1) = 2, \tag{28.3}$$

$$T_{\mathrm{bad}}(-1) = 2, \qquad T_{\mathrm{bad}}(1) = 0. \tag{28.4}$$

Both maps reach the same endpoint set $\{0, 2\}$, but for quadratic cost the good map has cost

$$\frac{1}{2}(1^2 + 1^2) = 1,$$

whereas the bad map has cost

$$\frac{1}{2}(3^2 + 1^2) = 5.$$

So endpoint fit alone does not settle whether the learned map behaves like a good transport map for the declared cost.

## 28.3 Direct-map lane: object changed, evidence changed

A direct map-learning method tries to produce a deterministic transport map

$$T : \mathbb{R}^{d_x} \to \mathbb{R}^{d_x}. \tag{28.5}$$

The source measure $\mu$ and target measure $\nu$ are related by the pushforward relation

$$T_{\#}\mu = \nu, \tag{28.6}$$

meaning that if one samples $X \sim \mu$ and then computes $T(X)$, the law of that transformed sample is $\nu$. Equivalently, for every test function $f$ for which the integrals exist,

$$\int f(y)\, d(T_{\#}\mu)(y) = \int f(T(x))\, d\mu(x). \tag{28.7}$$

At the empirical-cloud level, this means that if the chapter starts from source particles $x_i \sim \mu$ and evaluates $z_i = T(x_i)$, then the mapped cloud $\{z_i\}$ is the finite-sample object being asked to approximate the declared target law or target cloud.

The important change relative to the warm-start chapter is immediate: the learned object is no longer an internal solver coordinate. It is the transport map itself, or a potential from which that map is derived. In the vocabulary of the OT block, retained-teacher learning changes the execution path while a corrected solve still returns to a declared transport object. Direct-map learning changes the transport object directly and therefore inherits a different evidence contract.

| Quantity | Status in this chapter | Practical burden |
|---|---|---|
| learned map $T$ | primary learned object | must be validated directly, not through a retained teacher residual |
| pushforward $T_{\#}\mu$ | endpoint-law target | should match the declared target law or cloud closely enough on held-out problems |
| transport-cost / Monge-gap behavior | auxiliary OT-structure evidence | helps distinguish a plausible transport map from an arbitrary endpoint fit |
| downstream scalar built from mapped particles | implementation consequence | must be checked for drift once the map is embedded in filtering or HMC-adjacent code |

**Reader checkpoint.** The direct-map route is not just a stronger warm start. It asks the learned map itself to carry the approximation burden.

**Reader map for the technical core.** Before moving deeper into the equations, it helps to keep the two direct-map routes side by side.

| Route | Learned object | Training signal | Evidence required |
|---|---|---|---|
| Brenier / ICNN | convex potential $\psi_\theta$ and endpoint map $T_\theta = \nabla\psi_\theta$ | potential-based objective built from mapped source samples and target-side scoring | endpoint fit, OT-structure plausibility, downstream scalar stability |
| Monge-gap | direct endpoint map $T_\theta$ | endpoint-fit term plus empirical Monge-gap surrogate | endpoint fit, Monge-gap behavior, downstream scalar stability |

The rest of the chapter should be read through those three columns: what object is learned, what training signal drives it, and what evidence would justify trusting that object in later filtering code.

## 28.4 Brenier viewpoint

In the quadratic-cost Euclidean setting, a classical route is to represent the map as the gradient of a convex potential [Villani, 2003, Peyré and Cuturi, 2019, Vesseron and Cuturi, 2024]:

$$T^\star(x) = \nabla\psi^\star(x), \qquad \psi^\star \text{ convex.} \tag{28.8}$$

The quickest intuition is geometric. Quadratic-cost transport prefers maps with a monotone or, more precisely, cyclically monotone structure. Gradients of convex potentials are the canonical way to realize that structure. In one dimension this reduces to the familiar monotone rearrangement picture: sort the mass from left to right and move it without crossings. In higher dimension the theorem-strength statement is subtler, but the guiding lesson is the same: the more the transport is forced to behave like a monotone gradient field, the more a convex potential becomes a natural object to learn. This is why the Brenier viewpoint appears so naturally: instead of learning an arbitrary map, learn a scalar potential whose gradient already lives in a mathematically privileged map family.

This statement should be read with its theorem-level assumptions attached. The classical Brenier characterization is a continuous-measure result for quadratic cost in Euclidean space under regularity conditions such as finite second moments and absolute continuity of the source measure with respect to Lebesgue measure. In the finite empirical-cloud setting used throughout BayesFilter, a Monge map may fail to exist or fail to be unique in that theorem-strength sense. So the present chapter uses the Brenier viewpoint primarily as structured motivation for a learned map family, not as a blanket theorem that every finite cloud transport problem in the particle-filter setting is exactly represented by a gradient of a convex potential.

In one dimension, this means the derivative of the potential defines the map. For the translation cell above, one convenient choice is

$$\psi^\star(x) = \frac{1}{2}x^2 + x. \tag{28.9}$$

Then

$$\nabla\psi^\star(x) = x + 1 = T^\star(x). \tag{28.10}$$

The second derivative of $\psi^\star$ is 1, so the potential is convex. This tiny calculation is the cleanest way to see what the Brenier/ICNN family is trying to exploit: instead of learning the map directly, it learns a structured potential whose gradient gives the map.

A small finite-cloud warning is useful here. Even in tiny empirical problems, the transport object may fail to be a single theorem-strength Monge map. If one source atom of mass 1 must feed two target atoms of mass 1/2 each, the natural finite-cloud transport may split mass across targets rather than assign one unique endpoint to that source atom. This is exactly why the chapter treats Brenier as a structured motivation ladder rather than as an automatic finite-cloud theorem.

**What survives from Brenier theory in this chapter.** The theorem-strength Brenier result is not being imported wholesale into the finite-cloud BayesFilter setting. What survives here is a narrower bridge. The quadratic-cost theory says that convex-potential gradients are a mathematically natural map family when the continuous assumptions hold. The monograph then uses that fact as *structured motivation* for a direct-map parameterization. What does *not* survive automatically is a theorem that every finite empirical-cloud transport problem in this particle-filter setting is already solved exactly by an ICNN potential. The reader should therefore carry forward a three-way separation: continuous-theorem support, structured finite-cloud motivation, and the separate empirical question of whether the learned map behaves well once embedded in the actual BayesFilter computation.

## 28.5 ICNN/Brenier route

Input-convex neural networks (ICNNs) provide a parametric family of scalar functions that remain convex in designated inputs [Amos et al., 2017, Makkuva et al., 2020]. The implementation idea is:

(i) parameterize the potential $\psi$ by an ICNN $\psi_\theta$,

(ii) obtain the transport map by differentiating the network, $T_\theta(x) = \nabla\psi_\theta(x)$,

(iii) train $\psi_\theta$ so that the induced map sends source mass toward the target according to the chosen OT objective.

The middle step is the important one to slow down. The learned object is not yet a teacher-side coupling or a corrected barycentric cloud. It is the potential $\psi_\theta$, together with the gradient-defined endpoint map $T_\theta = \nabla\psi_\theta$ that later code will actually apply to the source particles. So the training objective is not certifying "the same object as the retained teacher, only faster." It is certifying, at best, that this chosen map family behaves plausibly under the declared direct-map evidence contract.

A useful mathematical picture to keep in view is not one universal training formula, but the structural relation

$$\psi^*(y) = \sup_x \{x^\top y - \psi(x)\}, \tag{28.11}$$

between a convex potential and its convex conjugate. In theorem-strength quadratic-cost OT, Brenier structure motivates learning a convex potential whose gradient gives the map. In practical finite-cloud implementations, however, several objective families are used: dual or conjugate-based objectives, empirical pushforward penalties, or regularized map-learning surrogates. The present chapter therefore uses the Brenier/ICNN viewpoint primarily as structured motivation for the map family, not as a claim that one displayed objective automatically covers every finite-cloud training loop.

One good way to read the displayed objective is to track the roles of its two functions separately. The network parameter $\theta$ chooses the source-side potential $\psi_\theta$, whose gradient becomes the deployed map. The function $\varphi$ plays the complementary target-side role: it scores where the mapped samples land. For a *fixed* $\varphi$, however, the final expectation over $Y \sim \nu$ is constant in $\theta$, so target information reaches the inner minimization only through $\varphi$ itself, not through that constant term. The formula is not saying that every direct-map implementation literally fixes this $\varphi$ in advance. It is saying that one common way to

train a Brenier-style map is to balance a source-to-map transport cost against a target-side scoring term that rewards mapped samples for matching the intended target law. For a practical reading, a first-year reader can think of $\varphi$ as the target-side scoring object in the objective: whether it is updated alternately or learned jointly depends on the exact implementation. If it were held fixed, the inner problem would ignore $\nu$ entirely, so a genuinely target-anchored trainer must include some $\varphi$-side update or a target-informed choice of $\varphi$.

The attraction of this route is that the map is not a black-box regressor. It is constrained to come from a convex potential, which is exactly the sort of structure the quadratic-cost theory suggests under its stated assumptions.

A concrete schematic training objective helps fix ideas. One representative potential-based route is to choose a target-side potential $\varphi$ and solve an objective of the schematic form

$$\min_{\theta} \mathbb{E}_{X\sim\mu}\left[\frac{1}{2}\|X - \nabla\psi_\theta(X)\|^2 - \varphi(\nabla\psi_\theta(X))\right] + \mathbb{E}_{Y\sim\nu}[\varphi(Y)]. \tag{28.12}$$

This should not be read as the only ICNN objective in the literature. Its role in this chapter is narrower: it makes explicit what one direct-map learner may optimize. The first expectation penalizes the source-to-map transport cost while evaluating the target-side potential on the mapped samples. The second expectation records the target-side potential on samples from $\nu$; for fixed $\varphi$ it is constant in $\theta$, so by itself it does not anchor the inner minimization to $\nu$. Target anchoring must come from how $\varphi$ is chosen, updated, or learned. At deployment time the object used by later code is still the gradient-defined endpoint map $T_\theta = \nabla\psi_\theta$, not the training objective itself. The display and algorithm below are therefore a schematic direct-map training pattern, not a claim that one fully specified canonical trainer has been derived in this chapter. A first-year reader should therefore read the objective in four rungs:

(i) choose a convex potential family $\psi_\theta$,

(ii) obtain the endpoint map by differentiation,

(iii) optimize a loss built from mapped samples and target-side information,

(iv) deploy only the resulting endpoint map, not the training loss itself.

The cost is also immediate from the equations. One must evaluate gradients of the potential and, depending on the chosen training formulation, may need to evaluate or approximate conjugate or dual terms. So the route trades a retained transport solver for a more structured but more delicate map-learning objective.

**Reader checkpoint.** Three evidential layers should remain separate for this route:

(i) whether the learned map reaches the right endpoint law or cloud well enough,

(ii) whether the learned map still behaves like a plausible OT-style map under the declared cost and structural prior,

(iii) whether a downstream scalar built from the mapped particles remains stable once this learned map is inserted into filtering or HMC-adjacent code.

A good training loss or good endpoint fit may support the first layer without yet settling the second or third.

**Algorithm 7** `train_icnn_brenier_map`

**Require:** source-cloud sampler for $\mu$, target-cloud sampler for $\nu$, ICNN potential family $\psi_\theta$, declared quadratic-cost/Brenier setting, convexity-enforcement policy, optimizer, learning-rate schedule, batch size, stopping rule, and held-out validation clouds

**Ensure:** trained potential $\psi_\theta$, induced map $T_\theta(x) = \nabla\psi_\theta(x)$, and validation diagnostics for pushforward mismatch and scalar drift

1: Initialize ICNN parameters $\theta$ and optimizer state
2: **while** the stopping rule has not been met **do**
3: Draw minibatches $x_b \sim \mu$ and $y_b \sim \nu$
4: Evaluate the potential values $\psi_\theta(x_b)$
5: Evaluate the gradient-defined transport map $T_\theta(x_b) = \nabla\psi_\theta(x_b)$
6: Update the declared target-side potential or conjugate estimate from the current target minibatch, then compute the declared dual or potential-based training objective
7: Add any declared regularization enforcing smoothness, stability, or numerical tractability
8: Backpropagate the total loss and update $\theta$
9: Re-enforce convexity according to the declared ICNN policy if it is not built directly into the parameterization
10: **end while**
11: On held-out validation clouds, compute mapped particles $T_\theta(x_i)$
12: Evaluate endpoint pushforward mismatch, transport-cost proxy, and downstream scalar sensitivity on the held-out clouds
13: Return $\psi_\theta$, the induced map $T_\theta$, and the declared diagnostics

**Approximation enters here.** The learned object is the transport map itself. Errors in the learned potential or its gradient directly change the transported output cloud.

**Why the next section changes emphasis rather than object.** The Brenier/ICNN route and the Monge-gap route are both direct-map answers. The learned endpoint map is still the primary object in both cases. What changes in the next section is not the endpoint ambition but the structural prior: instead of forcing the map to arise from one especially rigid potential family, the chapter asks how to penalize a candidate map when it behaves unlike an OT map under the chosen cost.

## 28.6 Monge-gap route

A more flexible direct-map family regularizes a candidate map by how far it behaves from an OT map rather than forcing a particular architecture from the start [Uscidda and Cuturi, 2023]. For cost $c$ and reference measure $\rho$, the Monge gap is

$$\mathcal{M}^c_\rho(T) = \int c(x, T(x))\, d\rho(x) - W_c(\rho, T_\#\rho). \tag{28.13}$$

Here $\rho$ is an explicitly declared reference measure used to probe how OT-like the learned map is on the region of state space that matters for the intended application. In the simplest reading one may take $\rho = \mu$, so the regularizer is applied on the same source measure used for transport. But a chapter, experiment, or implementation may instead

choose a different reference measure to emphasize a wider training envelope or a different weighting of the state space. The role of $\rho$ must therefore be named explicitly rather than left implicit.

The key structural property is immediate from the definition. The deterministic map $x \mapsto T(x)$ induces a coupling between $\rho$ and $T_{\#}\rho$, so by the definition of optimal transport,

$$W_c(\rho, T_{\#}\rho) \leq \int c(x, T(x))\, d\rho(x). \tag{28.14}$$

In slightly fuller words: the map-induced coupling $\gamma_T = (\mathrm{Id}, T)_{\#}\rho$ is one admissible coupling between the source measure and the pushforward target. The optimal-transport cost $W_c(\rho, T_{\#}\rho)$ is the infimum over all admissible couplings with those same marginals, so it cannot exceed the cost of this specific map-induced one. Therefore

$$\mathcal{M}^c_\rho(T) \geq 0, \tag{28.15}$$

with equality when the map-induced coupling is already optimal for transporting $\rho$ to $T_{\#}\rho$. This is why the Monge gap is a meaningful regularizer: it measures how far the learned map behaves from an OT map under the chosen cost.

As a mathematical definition, the Monge gap is exact. But in a practical training loop, the second term is typically not available in closed form because the pushforward measure $T_{\#}\rho$ itself depends on the current learned map. So the reader should picture a four-step ladder:

(i) define the exact Monge-gap quantity mathematically,

(ii) recognize that the OT reference term is usually unavailable in closed form inside training,

(iii) replace that unavailable term by a declared estimator or surrogate,

(iv) interpret the resulting optimization only as strongly as that surrogate justification allows.

A concrete empirical surrogate is obtained by drawing reference samples $x_i \sim \rho$, forming mapped particles $z_i = T_\theta(x_i)$, and defining

$$\widehat{\mathcal{M}}^c_{\rho,m}(T_\theta) = \frac{1}{m}\sum_{i=1}^m c(x_i, z_i) - W_c\left(\frac{1}{m}\sum_{i=1}^m \delta_{x_i}, \frac{1}{m}\sum_{i=1}^m \delta_{z_i}\right). \tag{28.16}$$

The second term is now a batchwise discrete OT problem on the empirical reference cloud and its mapped image. This is now a finite-sample training quantity, not the exact population Monge gap. So any finite-sample training objective that uses an estimated OT reference term should be read as a surrogate or approximate Monge-gap objective unless the estimator and its properties are stated explicitly. Operationally, the chapter's point is: fit a useful map, but penalize it when it behaves unlike an OT map for the chosen cost. A useful mental model is that the training loop is differentiating through two finite objects at once: the mapped sample cloud and a discrete OT computation on that cloud. If either one is a poor proxy for the intended downstream use case, the empirical Monge-gap objective may still decrease while the later filtering scalar drifts in an undesired direction.

The running cell already shows the idea. The good and bad endpoint maps above both match the endpoint law, but the bad map moves mass much less efficiently. On that tiny example, the direct transport cost of the good map is $\frac{1}{2}(1^2 + 1^2) = 1$, while the bad map has cost $\frac{1}{2}(3^2 + 1^2) = 5$. So even before worrying about population measures or finite-sample estimators, the Monge-gap logic is already visible: two maps can fit the same endpoint cloud while differing sharply in how OT-like their transport behavior is under the chosen cost. The Monge-gap route is attractive precisely because it can discourage such maps without forcing the full ICNN/Brenier structural commitment.

**Algorithm 8** `train_monge_gap_regularized_direct_map`

**Require:** source-cloud sampler for $\mu$, target-cloud sampler for $\nu$, reference-measure sampler for $\rho$, parametric map family $T_\theta$, cost function $c$, endpoint-fit objective, Monge-gap weight $\lambda_{\mathrm{MG}}$, optimizer, stopping rule, and held-out validation suite

**Ensure:** trained direct map $T_\theta$, Monge-gap diagnostic history, endpoint-fit diagnostics, and downstream scalar sensitivity report

1: Initialize map parameters $\theta$ and optimizer state
2: **while** the stopping rule has not been met **do**
3: Draw minibatches from the source, target, and reference distributions
4: Compute mapped source particles $z_i = T_\theta(x_i)$
5: Evaluate the declared endpoint-fit loss between the mapped source batch and the target batch
6: Compute the empirical transport-cost term on the reference samples from $\rho$
7: Estimate the OT reference term needed for the empirical Monge-gap penalty; this estimate defines the practical training surrogate unless a stronger equivalence result is supplied
8: Form the total loss as endpoint-fit term plus $\lambda_{\mathrm{MG}}$ times the empirical Monge-gap penalty, together with any declared regularization
9: Backpropagate the total loss and update $\theta$
10: **end while**
11: On held-out validation clouds, evaluate endpoint mismatch, Monge-gap magnitude, transport-cost proxy, and downstream scalar drift
12: Return the trained map $T_\theta$ and the declared diagnostics

**Reader checkpoint.** The Monge-gap route still learns a direct map, but it relaxes the strong ICNN-only structural commitment. The gain is flexibility. The cost is that the learned map is now judged relative to a regularized OT-like behavior rather than by a retained teacher solver.

## 28.7 Structured variants and boundaries

The same direct-map logic can be combined with additional structural priors, such as sparse displacement or factorized transport structure [Forrow et al., 2019, Chen et al., 2024, Xu et al., 2025, Geuter et al., 2025]. These are worth keeping in view because they may be useful when the filtering update itself has a scientifically justified sparse or low-active-subspace geometry. But they inherit the same chapter-wide burden: the learned endpoint map itself must be validated, rather than interpreted through a retained corrective teacher.

This is why the direct-map route should be read as a different scalable-OT answer from retained-teacher acceleration. It is more radical than teacher-preserving factorized Sinkhorn, because the final map object itself is learned. But it is still narrower than the dynamic or operator families of the next chapter, because it aims to learn one endpoint map rather than an entire transport path or measure-level operator. The next chapter therefore does not continue the same endpoint-map story at larger scale; it changes the learned object again, from one final map to richer path, velocity-field, or operator representations.

| Family | Learned object | Corrective teacher retained? | First burden of proof |
|---|---|---|---|
| Retained-teacher acceleration | solver state or latent teacher coordinate | yes | teacher-preservation residual after correction |
| Direct-map learning | endpoint map or potential-derived map | no | endpoint, OT-structure, and downstream scalar drift |
| Dynamic/operator learners | path, velocity field, or operator family | no | target-status and same-scalar coherence across richer objects |

## 28.8 Why this is not the default first route

The direct-map families are mathematically serious and potentially powerful, but they are not the monograph's default first implementation route for two reasons:

(i) they learn the final map object itself rather than only accelerating a declared teacher solve,

(ii) their strongest mathematical support depends on more specific geometric assumptions than the retained-teacher acceleration route requires.

### 28.8.1 Direct-map families as a different scalable-OT answer

The boundary with the retained-teacher chapter can be made even clearer. Nystrom, positive-feature, and related entropic approximations begin by asking how to preserve or approximate a declared teacher transport object more cheaply. The direct-map route asks a different question: can one learn the final transport map itself well enough that repeated teacher solves are no longer the center of the method?

That difference is not merely architectural. It changes the meaning of the main residual. In the retained-teacher lane, the first residual is naturally a teacher-preservation discrepancy. In the direct-map lane, the primary residuals are pushforward mismatch, Monge-gap behavior, transport-cost proxy, and scalar drift under the learned map. So this chapter should be read as a target-boundary chapter: it explains why a direct-map learner is not just "a stronger warm start," but a new answer to the scalable-OT problem with a different evidence contract.

In practical terms, this chapter occupies the middle position between the retained-teacher approximation lane and the broader replacement/dynamic lane. It is more radical than teacher-preserving factorized Sinkhorn, because the map itself is learned. But it is

still narrower than the dynamic/operator families of the next chapter, because it aims to learn one endpoint map rather than an entire transport path or measure-level operator.

## 28.9 Verification and non-claims

A minimal implementation-facing reading of this chapter should distinguish:

- what map object is being fit,
- what assumptions justify that map family,
- what endpoint, pushforward, transport-cost, or Monge-gap residuals are measured,
- what downstream scalar drift is checked once the learned map is embedded in filtering or HMC-adjacent code.

The practical residuals differ from the retained-teacher lane because no corrective Sinkhorn phase is automatically present to return to a declared teacher object. So endpoint fit, pushforward mismatch, transport-cost proxy, Monge-gap proxy, and downstream scalar sensitivity are part of the primary evidence, not side diagnostics.

This chapter does *not* claim:

- that direct learned maps are interchangeable with retained-teacher EOT acceleration,
- that a map-level fit automatically preserves the same filtering scalar,
- that broader direct-map families are already validated for BayesFilter.

Its role is to slow down the direct-map family itself so the reader can see that it is solving a different problem from the previous chapter.

# Chapter 29

# Dynamic Geodesic and Operator Learners, and the Filtering Target Contract

The previous chapter treated direct map-learning families such as ICNN/Brenier and Monge-gap approaches. The present chapter changes the learned object again. Instead of asking the network to learn one static transport map, it asks what happens when the learned object is a dynamic transport path, a velocity field, or an operator acting on families of measures.

This chapter is intentionally secondary. Its job is not to become another core implementation chapter. Its job is to explain why these broader learned families carry an even heavier target-status burden than retained-teacher acceleration or static direct-map learning.

## 29.1 What changes when the learned object becomes richer?

The easiest way to read this chapter is to keep the following contrast in mind.

- The retained-teacher neural chapter learned how to solve a declared transport teacher faster.
- The direct-map chapter learned one final transport map.
- This chapter asks what happens when the learned object is richer than one final map: a path, a velocity field, or an operator on entire families of measures.

So the main burden is not just implementation difficulty. The chapter's stable question is narrower and more useful:

> If the learned object is richer than one endpoint map, what scalar is downstream code actually using, and what gradient interpretation still remains justified?

For the rest of this chapter, the recurring OT-block terms should be read in one stable way:

- "teacher" means the earlier dense or retained-teacher reference route from the OT block,
- "target-changing" means the learned object is no longer automatically that same reference transport object,
- "same-scalar parity" means the reported gradient must differentiate the exact scalar value that is actually built from the learned object used by the code.

| Family in this chapter | Learned object | What changes first? | First audit question |
|---|---|---|---|
| Dynamic geodesic learners | path or velocity field | endpoint plus trajectory | is the reported scalar tied to that same path object? |
| Operator learners | family-level measure map | one problem becomes a class of problems | what target is induced across the problem family? |
| Target-changing boundary families | projected, local, or replacement transport | the transport object itself | what object is now acting on the filtering ensemble? |

**How to read the rest of this chapter.** The sequence is intentionally staged. First comes a tiny toy contrast between an endpoint map and a richer path object. Next comes the general object/scalar/ gradient audit language that the chapter will reuse. Only after that does the chapter look at dynamic geodesic learners, operator learners, and finally the projected or localized boundary families that should usually be read as target- changing in the BayesFilter monograph. To keep the chapter concrete, the reader should imagine one simple downstream scalar throughout: a filtering-style scalar that reduces the learned transport output into one value, either from the terminal cloud alone or from the full richer object. The chapter keeps asking how that scalar and its gradient change as the learned object becomes richer.

**One repeated filtering-style scalar to keep in mind.** To keep the chapter concrete, imagine one recurring scalar $\widetilde{L}$ built from the learned transport output before a later reduction in the filtering pipeline. In the endpoint-map case, $\widetilde{L}$ might depend only on the terminal equal-weight cloud. In the dynamic-path case, $\widetilde{L}$ might add a path-action penalty. In the operator or projected/localized case, $\widetilde{L}$ might depend on a family-level output or a projected/local block transport summary. The whole chapter is about how the meaning of $\widetilde{L}$ and therefore of its gradient changes when the learned object changes.

## 29.2 Running path-versus-map example

A tiny one-dimensional toy example makes the distinction concrete. Suppose the source cloud is

$$X_{\mathrm{src}} = \{-1, 1\}$$

and the target cloud is

$$X_{\mathrm{tgt}} = \{0, 2\}.$$

A static-map learner tries to learn one rule such as

$$T(x) = x + 1, \tag{29.1}$$

which moves the source points directly to the target points.

A dynamic path learner instead tries to describe how the particles move as a function of pseudo-time $\lambda \in [0, 1]$. Two simple paths with the same endpoint map are

$$x_A(\lambda) = x + \lambda, \qquad 0 \le \lambda \le 1, \tag{29.2}$$
$$x_B(\lambda) = x + \lambda^2, \qquad 0 \le \lambda \le 1. \tag{29.3}$$

Both paths satisfy

$$x_A(0) = x_B(0) = x, \qquad x_A(1) = x_B(1) = x + 1 = T(x).$$

So they agree on the final endpoint transformation. But they disagree as soon as later code cares about more than the endpoint. For example,

$$\Phi_{\text{mid}}(x(\cdot)) = x(1/2) \tag{29.4}$$

gives

$$\Phi_{\text{mid}}(x_A) = x + 1/2, \qquad \Phi_{\text{mid}}(x_B) = x + 1/4,$$

and the path-action functional

$$\mathcal{A}(x(\cdot)) = \int_0^1 |\dot{x}(\lambda)|^2 \, d\lambda \tag{29.5}$$

produces

$$\mathcal{A}(x_A) = 1, \qquad \mathcal{A}(x_B) = \int_0^1 (2\lambda)^2 \, d\lambda = \frac{4}{3}.$$

Thus two learned objects can agree on the endpoint map while still inducing different path-dependent scalars.

This distinction matters because later code need not use only the endpoint. At the level of notation, the endpoint map in both examples is still

$$T(x) = x(1) = x + 1, \tag{29.6}$$

but a path-dependent scalar may instead use the whole trajectory $x(\cdot)$. For a BayesFilter-style reading, imagine a scalar that averages only the terminal mapped particles versus a scalar that also penalizes path action or an intermediate position. Those two scalars may agree on the same endpoint map while still producing different values and different gradients because they consume fundamentally different learned objects. Once intermediate positions, path functionals, or operator outputs enter the scalar that is differentiated downstream, two learners with the same endpoint map can still define different effective targets.

**Plain-language takeaway.** A richer learned transport object can preserve the same endpoint while changing what downstream code means by the value and gradient it consumes.

## 29.3 Richer-object audit: endpoint, path, scalar, gradient

The toy example motivates the general audit ladder. Once the learned object is richer than one endpoint map, the reader should slow down and ask three separate questions: what object is being learned, what scalar later code builds from that object, and what gradient can therefore be interpreted.

**Object inventory.** A dynamic transport learner no longer tries only to produce one final map $T$. Instead it may try to learn:

- a geodesic path $\rho_\lambda$ between source and target measures,
- a velocity field $v_\lambda(x)$ generating that path,
- or an operator $\mathcal{G}$ that maps input measures to output trajectories, output measures, or transport-related latent coordinates.

This is a larger algorithmic change because the learned object is no longer merely a replacement endpoint map. It is a representation of how transport unfolds or how an entire family of transport problems is parameterized.

**What downstream code may consume.** Once a richer object is available, later code may consume:

- only the terminal map or endpoint cloud,
- intermediate positions or path summaries,
- action, regularity, or trajectory penalties,
- or operator outputs that vary across a family of related problems.

The object question therefore comes first. One cannot decide what scalar is being computed until one knows what learned object the scalar is built from.

**Same-scalar warning.** The most important target-contract warning survives every change of learned object. If a downstream routine uses a scalar $\widetilde{L}$ built from a learned map, path, or operator, then the reported gradient must differentiate that same scalar. Even if this value-gradient parity holds, the result is coherent only for the learned target unless a correction returns to the teacher or original target.

**Same-scalar template.** The chapter's main interpretive contract can be written schematically as

$$\mathcal{O}_\theta \longmapsto \widetilde{L}_\theta = \Phi(\mathcal{O}_\theta; \text{data}), \qquad g_\theta = \nabla_\theta \widetilde{L}_\theta, \tag{29.7}$$

where $\mathcal{O}_\theta$ is the learned object (map, path, or operator output), $\widetilde{L}_\theta$ is the actual downstream scalar built from that object, and $g_\theta$ is the reported gradient. A concrete BayesFilter-style example to keep in mind is a scalar that averages a terminal equal-weight cloud before passing it into a later filtering reduction. If a richer learner instead contributes a path penalty or an operator-generated summary to that scalar, then the object feeding $\Phi$

has changed even if the endpoint cloud looks similar. The gradient is interpretable only if it differentiates that same declared scalar built from that same learned object. Even perfect value-gradient parity therefore certifies coherence only for the learned target unless a correction returns to the earlier teacher.

| Learned object | What may change | First audit question |
|---|---|---|
| static map | endpoint cloud | does the learned endpoint preserve the intended downstream scalar? |
| dynamic path / velocity field | endpoint plus whole trajectory | is the reported gradient tied to the same scalar built from that path? |
| operator on measures or clouds | family-level input-output rule | what target is induced across the family of problems, and is it the one later code interprets? |

**Reader checkpoint.** The present chapter has crossed another object boundary. A dynamic learner is not just another static map family. It changes the interpretation burden because more than the endpoint may now matter.

## 29.4 Dynamic geodesic learners

Some neural OT papers learn Wasserstein geodesics or associated velocity fields [Hanebeck, 2023, Gracyk and Chen, 2024, Liu and Chen, 2026]. The minimal mathematical picture is that a dynamic transport object is no longer only an endpoint map $T$. It is a pseudo-time path of measures $(\rho_\lambda)_{0\le\lambda\le1}$ together with a velocity field $v_\lambda(x)$ satisfying a continuity equation of the schematic form

$$\partial_\lambda \rho_\lambda + \nabla \cdot (\rho_\lambda v_\lambda) = 0, \tag{29.8}$$

with endpoints

$$\rho_0 = \mu, \qquad \rho_1 = \nu. \tag{29.9}$$

In a Benamou–Brenier-style reading, one then judges a path by a transport-action objective of the schematic form

$$\int_0^1 \int \|v_\lambda(x)\|^2 \, d\rho_\lambda(x) \, d\lambda. \tag{29.10}$$

The endpoint map is now only one summary of a richer learned object; the path and velocity field are themselves part of what later code could use.

This is mathematically appealing because particle-flow filtering already has a pseudo-time flavor. The attraction is therefore obvious: perhaps one could learn reusable transport dynamics instead of solving a fresh static problem each time.

The caution is equally important. A dynamic geodesic learner does not answer the same question as a retained finite Sinkhorn solver. It learns a richer object and must therefore be judged against a richer scalar-and-gradient interpretation. The first audit question is not merely whether the endpoint fit is good. It is whether the downstream scalar is built from the same path object whose gradient is later reported.

**How to read the algorithm below.** The following routine is schematic. It is included to make the richer learned object concrete, not to promote dynamic-path learning as the monograph's default BayesFilter implementation route.

**Algorithm 9** `train_dynamic_transport_path`

**Require:** source-cloud sampler, target-cloud sampler, pseudo-time grid $0 = \lambda_0 < \cdots < \lambda_K = 1$, learned velocity-field or path parameterization, rollout integrator, endpoint loss, path regularizers, optimizer, stopping rule, and validation suite

**Ensure:** trained path or velocity-field parameters, deployable terminal map or path object, and pathwise validation diagnostics

Initialize the path or velocity parameters and optimizer state
**while** the stopping rule has not been met **do**
    Draw source and target cloud minibatches
    Initialize the source particles at pseudo-time $\lambda_0$
    **for** each pseudo-time step $k = 0, \ldots, K-1$ **do**
        Evaluate the learned velocity field or path increment at $\lambda_k$
        Advance the particles using the declared rollout or integration rule
    **end for**
    Compute the endpoint loss against the target cloud at $\lambda = 1$
    Compute any declared pathwise regularizers, such as smoothness or action penalties
    Backpropagate the total loss and update the parameters
**end while**
On held-out clouds, evaluate endpoint fit, path regularity, discretization sensitivity, and downstream scalar sensitivity
Return the trained path/velocity object together with the declared diagnostics

## 29.5 Operator learners

A related family learns operators on spaces of measures [Lu et al., 2021, Li et al., 2021, Kovachki et al., 2023]. In such approaches, the input is itself a measure or empirical cloud, and the output is either another measure, a path of measures, or a set of latent coordinates describing the transport. A useful empirical-cloud signature is

$$\mathcal{G}_\theta : (\mu, \text{context}) \longmapsto \text{endpoint cloud, path object, or transport latent state.} \tag{29.11}$$

Concretely, one may imagine an operator that takes an empirical cloud together with problem context and returns a terminal transformed cloud, a whole pseudo-time path, or transport-related latent coordinates for a family of related filtering problems. For example, one operator might take a weighted particle cloud together with observation context and output a full pseudo-time path whose terminal state is then reduced into a filtering scalar. Another might output only a terminal cloud. The chapter's point is that these two outputs can induce different scalars and therefore different gradient meanings even when they start from the same input ensemble. In that sense, the operator section should be read as the family-level analogue of the earlier path-vs-endpoint toy example: the repeated scalar $\widetilde{L}$ is still the anchor, but the object fed into it has changed.

The main conceptual advantage is reuse across families of problems. The main conceptual cost is that the learned object is now even less tied to one declared teacher solve. So the first question is no longer only "does this recover one finite teacher route well enough?" It is "what family-level target is this operator inducing across the class of problems to which it is applied?" That makes these methods valuable as research-direction chapters, but not as the default first implementation route in this monograph.

## 29.6 Target-changing boundary families

Several important high-dimensional families fit this chapter because they should not be read as compressed dense-teacher solvers by default. The common theme is simple: they may still produce useful transport surrogates, but the object they compute is no longer automatically the same full-state dense transport object that earler chapters treated as the reference lane.

**Projected families: sliced and subspace OT.** Sliced and subspace OT methods attack high dimension by projecting the measures onto one-dimensional directions or selected subspaces before solving lower- dimensional transport problems [Kolouri et al., 2019, Deshpande et al., 2019, Paty and Cuturi, 2019, Nguyen, 2025]. The crucial warm-up fact is one-dimensional OT itself. For one-dimensional measures, the monotone transport map is

$$T(x) = F_\nu^{-1}(F_\mu(x)), \tag{29.12}$$

and the one-dimensional Wasserstein distance is

$$W_p^p(\mu,\nu) = \int_0^1 \left|F_\mu^{-1}(z) - F_\nu^{-1}(z)\right|^p \, dz. \tag{29.13}$$

For empirical distributions, this one-dimensional transport is computed by sorting projected samples. For a direction $\theta \in S^{d-1}$, let $\theta_\#\mu$ denote the pushforward of $\mu$ under $x \mapsto \langle \theta, x\rangle$. The sliced Wasserstein construction then has the schematic form

$$\mathrm{SW}_p^p(\mu,\nu) = \int_{S^{d-1}} W_p^p(\theta_\#\mu, \theta_\#\nu)\, d\theta, \tag{29.14}$$

with empirical directional approximation

$$\widehat{\mathrm{SW}}_p^p(\mu,\nu) = \frac{1}{L}\sum_{\ell=1}^{L} W_p^p\big((\theta_\ell)_\#\mu, (\theta_\ell)_\#\nu\big). \tag{29.15}$$

A fully empirical one-dimensional term is computed by sorting,

$$W_2^2(D^\theta, F^\theta) = \frac{1}{|D|}\sum_i \left\| D^\theta_{\pi^\theta_D(i)} - F^\theta_{\pi^\theta_F(i)} \right\|_2^2. \tag{29.16}$$

One can also sharpen random directional averaging into a best-direction criterion, for example by the max-sliced quantity

$$\mathrm{MSW}_p^p(\mu,\nu) = \max_{\theta\in S^{d-1}} W_p^p(\theta_\#\mu, \theta_\#\nu). \tag{29.17}$$

In plain language, these methods first solve easier projected transports and then use those projected solutions to define a higher-dimensional surrogate. That is why they belong in this boundary chapter: a projected or reconstructed transport is not automatically the same object as the full-state dense coupling from the earlier OT chapters. The first interpretive question is therefore not only teacher preservation. It is what projected or reconstructed transport object is now acting on the filtering ensemble. In the running toy spirit, the key contrast is: the reference lane earlier in the OT block acted through one full-state transport object, whereas a sliced route acts through many lower-dimensional projected ones whose average or reconstruction must still be interpreted with care. If the recurring scalar $\widetilde{L}$ is built from those projected outputs, then the gradient is only a gradient of that projected-object scalar, not automatically of the earlier dense reference scalar.

### 29.6.1 Sliced, subspace, and localization families as target-changing lanes

The scalable-OT survey gives this chapter a more precise boundary around several important high-dimensional families. Sliced and subspace OT methods attack high dimension by projecting the measures onto one-dimensional directions or selected subspaces before solving lower-dimensional transport problems. Localization and block-structured OT instead exploit state-space structure by transporting local blocks or using localized costs rather than one global cloud-wide coupling.

These methods are mathematically important, but in the BayesFilter monograph they should not be described as mere compressed versions of the dense entropic teacher. A projected one-dimensional or subspace transport, or a block-localized transport, is generally a different transport object from the full-state dense coupling used in the previous chapters. That means the main burden here is target interpretation rather than just solver engineering.

In that sense, sliced/subspace and localization families fit the present chapter better than the retained-teacher chapter. Their primary promise is not simply to reach the same teacher object faster. Their promise is to define a more tractable transport rule in high dimension. The price is that the resulting output cloud must usually be read as a new resampling approximation unless a separate argument shows how the projected or localized transport reconstructs the intended full-state object.

This chapter therefore treats these families as target-changing or structure-changing candidates by default. They may still be useful and even necessary in high dimension, but their first audit question is no longer only teacher-preservation residual. It is: what transport object is now being applied to the filtering ensemble, and what scalar target would a later optimizer or HMC routine actually differentiate if this object were embedded downstream?

**Structured families: localization and block transport.** Localization and block-structured OT instead exploit state-space structure by transporting local blocks or using localized costs rather than one global cloud- wide coupling. A schematic block decomposition is

$$x_i = (x_i^{(1)}, \dots, x_i^{(B)}), \tag{29.18}$$

followed by either blockwise transports

$$P^{(b)} \in \Pi(a^{(b)}, b^{(b)}) \tag{29.19}$$

or shared particle-level couplings with additive local costs

$$C_{ij} = \sum_{b=1}^{B} C_{ij}^{(b)}. \tag{29.20}$$

The first option says: transport each block with its own local rule. The second says: keep one particle-level coupling, but define that coupling through a cost that is itself localized across scientifically meaningful state blocks. In plain language, the computed object may therefore be a structured local transport rule rather than one dense global coupling. This can be scientifically sensible when the state decomposition itself is justified, but it still changes the object whose scalar consequences downstream code will inherit. So

the reader must ask not only whether the method is faster or more stable, but which structured transport object is actually being applied and what stronger dense-parity claim is therefore no longer automatic.

**Boundary markers only.** Several additional families remain useful only as boundary markers here: measure-to-measure regression operators, local Wasserstein regression, and manifold or Riemannian neural OT [Vandergrift et al., 2026, Girshfeld and Chen, 2025, Micheli et al., 2026]. They show how quickly the learned object can drift from the current finite-cloud Euclidean teacher, but they do not by themselves answer the present implementation question.

## 29.7 Same-scalar rule revisited

**What the remaining sections now do.** The previous sections identified richer families by the object they learn and by how quickly that object can drift away from the earlier dense reference route. The remaining sections do not introduce new families. They instead collect the audit questions that help the reader decide what interpretation remains mathematically supported once the learned object is richer than a static endpoint map.

**Mini audit example.** Suppose a learner reports a gradient for a scalar that uses both the terminal cloud and the path-action penalty from the running example. If the code used to compute the scalar includes $\mathcal{A}(x(\cdot))$ but the reported gradient differentiates only the terminal map $T(x) = x(1)$, then same-scalar parity has failed even if the endpoint cloud looks correct. The strongest justified interpretation in that case is only that the endpoint map is reasonable as a transport summary; it is not that the reported gradient is the derivative of the actual executed scalar. The same logic applies to operator learners and projected/localized families: once the filtering-style scalar uses the richer object, the audit must stay tied to that same object all the way through value and gradient evaluation.

## 29.8 How to audit a richer learned object

For richer learned objects, a few mathematical consistency checks are especially important:

(i) whether endpoint or pathwise improvement is accompanied by stability of the downstream scalar,

(ii) whether the learned object behaves robustly under weight, dimension, or particle-count shift,

(iii) whether a reported gradient is the gradient of the same scalar value that is later interpreted downstream,

(iv) whether any claim of target preservation is supported by an explicit correction mechanism rather than by smoothness or speed alone.

These are not governance gates. They are simply the mathematical checks needed to understand what object is being computed, what scalar is built from that object, and what interpretation remains identifiable from the reported gradient.

**Algorithm 10** `audit_dynamic_transport_target_contract`

**Require:** learned output object (map, path, or operator), downstream scalar-construction routine, reported gradient routine, optional teacher or corrected baseline scalar, shift-test suite, and declared tolerances for scalar and gradient mismatch

**Ensure:** a consistency report containing same-scalar parity status, shift-robustness status, any failed conditions, and the strongest mathematically supported interpretation of the learned object

**for** each evaluation problem in the declared suite **do**
    Construct the learned output object for that problem
    Compute the downstream scalar $\widetilde{L}$ from that learned object
    Compute the reported gradient of that same scalar
    **if** a teacher or corrected baseline exists **then**
        Compute the baseline scalar and, where applicable, the baseline gradient
        Record scalar and gradient gaps relative to the baseline
    **end if**
    Check that the reported gradient is tied to the same scalar $\widetilde{L}$
    Evaluate shift robustness under particle-count, weight, and context variation
    Record any failed scalar, gradient, or robustness condition
**end for**
Return the full consistency report and the strongest interpretation of the learned object consistent with those results

**Reader checkpoint.** This procedure does not approve or reject a method in an institutional sense. It simply organizes the chapter's local question: what object was learned, what scalar was built from it, and what gradient interpretation is still justified for that same scalar?

## 29.9 Relation to the full HMC target chapter

This chapter stops at the target-status burden created by broader dynamic and operator learners. It does not settle the HMC question. That full taxonomy still belongs to Chapter 30, which interprets the scalar and gradient contracts rung by rung across the whole DPF ladder. Put more sharply: the present chapter settles the local object-identification question, while the later HMC chapter settles the broader sampler-interpretation question.

**OT-block takeaway.** The OT sequence has now split into three broad implementation postures. The entropic OT chapter fixed the semantics-preserving reference lane. The retained- teacher chapter kept that object but learned how to reach it faster. The direct- map and dynamic/operator chapters then moved into target-changing territory by learning endpoint maps, paths, or measure-level operators directly. The next HMC and debugging chapters therefore do not ask which family is most elegant in the abstract; they ask what scalar each family induces and what evidence is needed before any stronger claim is made.

## 29.10 What this chapter does and does not claim

This chapter explains why the broadest neural OT families should be read as secondary research directions in this monograph. It does *not* claim:

- that geodesic, operator, and adjacent measure/manifold families solve the same implementation problem as retained-teacher acceleration,
- that a richer learned object is automatically more useful for BayesFilter,
- that these families are already ready for the same implementation status as the earlier chapters.

Its role is to close the learned-transport part responsibly by keeping the broadest learned families in view while handing the full target-status question forward to the HMC interpretation and verification chapters.

# Part VIII

# HMC Target Interpretation, Verification, and Filter Choice

# Chapter 30

# DPF HMC Target Correctness and Structural-Model Suitability

The preceding particle, transport, and learned-surrogate chapters built a ladder of increasingly differentiable filtering constructions: raw particle-flow proposals, proposal-corrected PF-PF, soft differentiable resampling, deterministic OT equal-weighting, entropic optimal-transport relaxation with Sinkhorn, custom-gradient LEDH-PFPF-OT differentiation for the declared finite value path, retained-teacher learned transport acceleration, and broader learned-map or dynamic-transport surrogates. This chapter asks a narrower and more severe question. If one of those constructions is used inside Hamiltonian Monte Carlo for a nonlinear DSGE or MacroFinance model, what target is the Markov chain intended to leave invariant?

The answer cannot be inferred from the label "differentiable particle filter." HMC is a target-specific algorithm. Its mathematical interpretation depends on the scalar log target used for accept/reject decisions, the gradient used by the numerical integrator, and the correction mechanism, if any [Neal, 2011, Betancourt, 2017]. Particle MCMC adds a separate target logic: an unbiased, nonnegative likelihood estimator can define a corrected extended-space chain under pseudo-marginal assumptions, but that is not the same object as a smooth surrogate likelihood differentiated by HMC [Andrieu and Roberts, 2009, Andrieu et al., 2010]. The DPF ladder must therefore separate three questions:

(i) whether the scalar value supplied to HMC is the intended posterior value, an unbiased estimator inside a corrected construction, or a deliberately approximate value;

(ii) whether the gradient supplied to the integrator is the derivative of that same scalar value;

(iii) whether the evidence is strong enough for exploratory, research, or broader external claims.

## 30.1 Objects in the HMC target contract

Let $\theta \in \Theta$ be the structural parameter and let $u \in \mathbb{R}^{d_\theta}$ be the unconstrained parameter used by the sampler, with transform $\theta = T(u)$. Let

$$\ell(u) = \log p(T(u)) + \log L(y_{1:T} \mid T(u)) + \log |\det DT(u)|$$

denote the scalar log target in unconstrained coordinates when the filtering likelihood is exact. This expression assumes that the transform is differentiable on the sampler-valid region and that $DT(u)$ is the square Jacobian relevant to the change of variables, or the appropriate constrained- to-unconstrained volume term if the parameterization is more general. In a DPF construction the likelihood term may be replaced by a particle, relaxed, or learned object. The HMC contract is therefore stated for the actual scalar value $\ell_\star(u)$ used by the sampler, not for the method family name.

Table 30.1: Objects that must be named before a DPF construction can be called an HMC target.

| Object | Mathematical role | Reviewer audit question |
|---|---|---|
| Parameter $u$ and transform $T$ | Coordinates integrated by HMC and the map back to structural parameters. | Are support constraints, Jacobian terms, and invalid parameter regions included in the scalar target? |
| Scalar log target $\ell_\star(u)$ | The value whose density $\pi_\star(u) \propto \exp\{\ell_\star(u)\}$ is claimed as the invariant target. | Is $\ell_\star$ exact, pseudo-marginal extended-space, relaxed, or learned? |
| Potential $U_\star(u)$ | $U_\star(u) = -\ell_\star(u)$, up to an irrelevant additive constant. | Is the sign convention consistent between value, gradient, and accept/reject? |
| Momentum $p$ and mass $M$ | Auxiliary Gaussian momentum $p \sim \mathcal{N}(0, M)$ used to define Hamiltonian dynamics. | Is $M$ fixed or adapted under a declared warmup policy, and is it positive definite? |
| Hamiltonian $H_\star(u, p)$ | $H_\star(u,p) = U_\star(u) + \frac{1}{2}p^\top M^{-1}p$ in ordinary separable HMC. | Does the accept/reject step evaluate this same $H_\star$? |
| Integrator $\Phi_{\epsilon,L}$ | Numerical map used to propose $(u', p')$ after $L$ steps of size $\epsilon$. | Is the proposal reversible and volume preserving for the stated correction, or is a different Metropolis ratio supplied? |
| Gradient $g(u)$ | Vector supplied to the integrator in place of $\nabla_u \ell_\star(u)$. | Is $g(u) = \nabla_u \ell_\star(u)$ for the same scalar value, or is the mismatch explicitly corrected and diagnosed? |
| Accept/reject value | Scalar value used in the Metropolis correction. | Does the correction use the intended target, the relaxed target, or a learned surrogate? |
| Target-status category | Label attached to the resulting chain. | Does the label distinguish exact, pseudo-marginal, relaxed, learned, and diagnostic-only use? |

Assume that $\ell_\star : \mathbb{R}^{d_\theta} \to \mathbb{R}$ is differentiable on the interior of the sampler's valid region, and that $g : \mathbb{R}^{d_\theta} \to \mathbb{R}^{d_\theta}$ is the vector field supplied to the integrator in the same coordinates. The mass matrix $M$ in the ordinary separable Hamiltonian is a symmetric positive-definite $d_\theta \times d_\theta$ matrix, so $p^\top M^{-1} p$ is well defined. The minimum value-gradient

consistency condition is

$$g(u) = \nabla_u \ell_\star(u). \tag{30.1}$$

Equation (30.1) is not a claim that $\ell_\star$ is the exact nonlinear filtering likelihood. It is a claim that the gradient used by the integrator and the value used by the target interpretation are the same mathematical object.

If the value is $\ell_\star$ but the integrator uses a vector field $g \neq \nabla \ell_\star$, the usual leapfrog derivation no longer describes the Hamiltonian flow of $H_\star$. A Metropolis correction may still be valid for an arbitrary proposal only after reversibility, volume preservation, and the correct acceptance ratio have been established. Without that separate proposal-level proof, a smooth chain with mismatched value and gradient is not evidence of correct HMC. If both value and gradient are changed to a relaxed or learned scalar, then the chain may be internally coherent, but the target has changed to $\pi_\star$.

## 30.2 Target-status taxonomy

The DPF chapters use the following target-status categories. The categories are deliberately conservative because a reviewer should be able to tell exactly what posterior, extended posterior, or surrogate posterior is being discussed.

**Exact-likelihood HMC**
: HMC evaluates a deterministic scalar log posterior and its exact or audited derivative. Numerical integration error is corrected by the standard Metropolis step, provided the implemented proposal satisfies the usual HMC reversibility and volume-preservation assumptions.

**Pseudo-marginal MCMC**
: A random nonnegative likelihood estimator is included as part of an extended-space target. The claim is not that the estimator is a smooth likelihood surrogate. The claim is that marginal correctness follows from the pseudo-marginal construction under its assumptions [Andrieu and Roberts, 2009, Andrieu et al., 2010].

**Noisy-gradient method**
: A stochastic or approximate gradient is used to move the chain. This is not exact HMC unless a valid correction theory is stated. It may be useful as an optimization, adaptation, or diagnostic device, but that usefulness does not by itself identify the invariant posterior.

**Delayed-acceptance or surrogate-corrected MCMC**
: A surrogate or particle object proposes, screens, or accelerates moves, but a later correction evaluates the intended target. The target claim belongs to the corrected chain, not to the surrogate alone.

**Relaxed-target HMC**
: The scalar log target deliberately replaces a discontinuous or discrete filtering operation by a smooth relaxed operation, such as soft resampling or entropic OT. HMC can be coherent for this scalar if Equation (30.1) holds, but the posterior is the relaxed posterior.

**Learned-surrogate HMC**
: A neural or amortized map defines part of the scalar value. The chain targets the learned scalar only if value and gradient are consistent. It targets the original or teacher posterior only after a separate correction or error-bound argument strong enough for that claim.

## 30.3 Rung-by-rung target analysis

### 30.3.1 EDH and LEDH flow proposals

At the raw EDH or LEDH rung, the value object is a flow-based approximate filtering quantity. The flow is derived from Gaussian closure or local linearization assumptions and is integrated numerically. If HMC differentiates the same scalar flow object that it evaluates, then the construction can satisfy value-gradient consistency for that scalar. Its target status is nevertheless limited:

- exact-target HMC only in recovery cases where the flow construction equals the exact filtering likelihood, such as the linear-Gaussian benchmark;
- approximate-target or value-side benchmark HMC outside those recovery cases;
- diagnostic-only use if the implementation differentiates a path that differs from the scalar value used for accept/reject.

Smoothness of the flow ODE, by itself, does not upgrade a closure-based filtering approximation into an exact nonlinear posterior target.

### 30.3.2 PF-PF proposal correction

At the PF-PF rung, the flow is used as a proposal and the scalar value includes the proposal-to-target correction through importance weights and change-of-variables terms. This is the first rung in the BayesFilter DPF ladder with an explicit proposal-correction interpretation. That statement is not a production or validation claim. It means only that the value-side target story is more defensible than raw flow transport because the proposal density, Jacobian or log determinant, and particle weight enter the same scalar object. The improvement is proposal-accounting clarity for the stated one-step conditional construction in Chapter 22; it is not evidence, by itself, of multi-step posterior fidelity, finite-particle stability, or HMC correctness.

For HMC, the required scalar is the corrected particle objective actually used by the sampler. The required gradient is the derivative of that same corrected objective, including every differentiable term in the proposal density, transition density, measurement density, flow map, and log determinant. The remaining evidence burden is substantial:

- finite-particle variability must be measured rather than hidden by a deterministic seed;
- log-determinant and Jacobian paths must be audited against the same value path;
- flow discretization error must be separated from Monte Carlo error;

- compiled repeated evaluation must remain finite and reproducible on the structural model class being claimed;

- if randomness enters the HMC target, the construction must state whether it is pseudo-marginal extended-space MCMC, a fixed-randomness deterministic approximation, or a noisy-gradient diagnostic.

Thus PF-PF is a plausible first research rung for DPF-HMC experiments, not a full HMC-validity theorem and not by itself a stronger external claim.

### 30.3.3 Soft differentiable resampling

Soft resampling replaces categorical ancestor selection by a differentiable mixture or shrinkage construction [Zhu et al., 2020, Jonschkowski et al., 2018, Karkus et al., 2018]. The scalar value is therefore a soft-resampling filtering value. If HMC differentiates exactly that scalar, the resulting chain can be internally coherent for the soft target. It should not be described as sampling the categorical-resampling posterior unless a separate correction is supplied.

The core mathematical issue is nonlinear test-function bias. Even if a soft-resampling map preserves a weighted mean under special conditions, it does not generally preserve expectations of nonlinear functions, path genealogies, or the likelihood contribution of the original categorical particle filter. The HMC label must therefore be relaxed-target HMC, surrogate-target HMC, or diagnostic use, depending on the scalar value actually corrected by the sampler.

### 30.3.4 OT and entropic OT resampling

At the OT/EOT rung, resampling is written as a transport problem [Reich, 2013, Cuturi, 2013, Peyré and Cuturi, 2019, Corenflos et al., 2021]. Entropic regularization and a finite Sinkhorn solve make the construction differentiable and numerically structured, but they also define a different map from exact categorical resampling. The scalar value must specify the cost, entropy convention, regularization parameter, stopping tolerance, and whether the unrolled solver path or an implicit derivative is used.

The scalable-OT survey sharpens the target-status split inside this rung. Exact online or streamed Sinkhorn implementations can still belong to the same relaxed entropic target class if they preserve the same declared teacher object and the same executed scalar. Their main changes are engineering changes in storage, blocking, or GPU execution, not changes in the target definition itself. By contrast, factorized kernel routes, low-rank coupling routes, sliced/subspace transports, and localized/block transports should not all inherit that same status automatically. Some remain best read as retained-teacher approximations, while others are better read as explicit new transport rules.

For HMC, the target status is relaxed-target HMC when the accept/reject value and gradient both use the same EOT/Sinkhorn scalar. Finite solver residuals, underflow handling, stabilization, and $\varepsilon$ sensitivity are target diagnostics, not mere implementation details. A finite Sinkhorn layer does not become an exact categorical resampling layer because its gradient exists. Nor does a faster implementation become a new target simply because its execution path is different, provided the same scalar and same transport object are still being evaluated.

### 30.3.5 Learned or amortized OT

Learned OT introduces a second approximation layer: a neural or amortized map is trained to imitate, accelerate, or residual-correct a transport teacher [Amos et al., 2017, Makkuva et al., 2020, Uscidda and Cuturi, 2023, Gracyk and Chen, 2024]. The scalar value is then the learned scalar unless a correction evaluates the teacher or original target. The gradient path is the derivative through the learned map and any explicitly retained teacher terms.

The scalable-OT survey helps separate two cases that should not be conflated. Retained-teacher accelerators, such as warm-start predictors or other teacher-preserving approximations, may still be judged first against the same transport teacher object. Their HMC burden is therefore to show that the value and gradient are consistent for the same learned-or-corrected scalar and that teacher-preservation residuals are understood on the tested envelope. Broader transport learners, sliced or localized replacement rules, and dynamic/operator families do not even begin with that teacher-preservation presumption. They are better read as learned or replacement targets unless a separate correction or error argument returns the chain to the teacher or original target.

The target-status label must therefore be learned-surrogate HMC unless the implementation supplies a valid correction or an error analysis strong enough to promote the claim. Runtime reduction, smoothness, and low teacher residual on training clouds are not target-correctness evidence. The missing validation includes out-of-distribution particle-cloud tests, seed sensitivity, posterior comparison against the EOT teacher, and stress tests on structural parameter regions that are invalid, nearly invalid, or geometrically stiff.

## 30.4 Structured target maps

Table 30.2: Rung-by-rung HMC target map.

| Rung | Scalar value | Gradient path | Target status | Missing validation before promotion |
|---|---|---|---|---|
| EDH/LEDH | Gaussian-closure or local-flow filtering scalar. | Derivative of the same flow scalar, if implemented consistently. | Exact only in recovery cases; otherwise approximate benchmark. | Linear-Gaussian recovery, nonlinear likelihood-error studies, flow discretization checks, invalid-region behavior. |
| PF-PF | Proposal-corrected particle scalar with weights and flow Jacobian/log-det terms. | Derivative of the same corrected scalar, including correction terms. | First rung with explicit proposal-correction interpretation; research candidate only. | Finite-particle variance, Jacobian/log-det audit, value-gradient parity, compiled repeated evaluation, randomness policy. |

| Rung | Scalar value | Gradient path | Target status | Missing validation before promotion |
|---|---|---|---|---|
| Soft resampling | Soft or shrinkage resampling scalar. | Derivative through the same soft map. | Relaxed or surrogate target. | Posterior sensitivity against categorical or trusted references, nonlinear test-function bias, seed and temperature sensitivity. |
| OT/EOT | Transport-relaxed scalar with cost, entropy, $\varepsilon$, and solver budget. | Derivative through the same unrolled or implicit solver path. | Relaxed target. | $\varepsilon$ and Sinkhorn-budget sensitivity, residual tolerances, stabilization audit, teacher posterior comparison. |
| Learned OT | Learned approximation to an OT/EOT or residual-corrected scalar. | Derivative through the learned map and retained teacher terms. | Learned surrogate unless corrected. | Teacher residual to posterior-error analysis, out-of-distribution cloud tests, structural stress tests, correction evidence. |

Table 30.2 is a target map, not a performance table. It does not say that the lower rungs are useless or that the higher rungs are invalid. It says that every rung must name the scalar target before an HMC claim can be interpreted.

## 30.5 Pseudo-marginal and surrogate distinctions

The pseudo-marginal result and the DPF surrogate story are often confused because both involve particle approximations. They are different claims. Suppose $\widehat{L}(y \mid \theta, \omega) \geq 0$ is an unbiased estimator of $L(y \mid \theta)$ under auxiliary randomness $\omega$:

$$\mathbb{E}_{\omega|\theta}\left[\widehat{L}(y \mid \theta, \omega)\right] = L(y \mid \theta).$$

Pseudo-marginal MCMC targets an extended density involving $\omega$ and can recover the exact marginal posterior under its assumptions. The validity argument is an extended-target argument. It is not an argument that $\log \widehat{L}$ is a smooth deterministic likelihood whose pathwise gradient is the gradient of $\log L$.

By contrast, a relaxed DPF target usually chooses a deterministic or fixed-randomness scalar $\ell_\star(\theta)$ and differentiates it. HMC may be valid for $\ell_\star$ if the value-gradient contract and proposal correction hold. That does not imply validity for the original posterior $\ell(\theta)$. A delayed-acceptance or surrogate-corrected scheme can bridge the gap only if the final correction evaluates the intended target with the right acceptance probability.

## 30.6 Why nonlinear DSGE models sharpen the issue

Nonlinear DSGE models make target drift especially costly because the filtering law is tied to structural economics rather than only to a generic latent-state transition. The DPF chapter does not need to solve the economics, but it must not hide target changes inside a numerical method. The main DSGE stress points are:

(i) **Determinacy and invalid regions**: the sampler will propose parameters outside or near determinacy regions. The target must return a declared finite penalty or $-\infty$ policy, not an uncaught solver failure.

(ii) **Pruning and approximation convention**: first-order, higher-order, pruned, and simulation-based approximations can define different latent laws. A DPF likelihood is only meaningful after the structural law is fixed.

(iii) **Structural timing**: observables, controls, states, and shocks must enter the transition and measurement equations with the intended timing. A smooth filter for the wrong timing is still the wrong target.

(iv) **Latent-state semantics**: deterministic-completion coordinates and stochastic state coordinates are not interchangeable. Particle clouds must represent the declared latent law.

(v) **Local Jacobian fragility**: EDH, LEDH, and proposal-corrected flow methods inherit local-linearization and derivative fragility near boundaries or stiff regions.

For DSGE work, the correct conclusion is conservative. A DPF rung may be a useful research target after it states its scalar value and diagnostics. It is not bank-facing evidence until the structural law, invalid-region policy, value-gradient parity, and posterior sensitivity have been tested on the named model class.

## 30.7 Why MacroFinance models sharpen the issue

MacroFinance latent-factor systems create a different target-risk profile. Their long panels can amplify small likelihood changes, and their latent factors often interact with volatility, measurement noise, and missing-data patterns. A relaxed or learned resampling map that looks harmless on a short toy filter can move posterior mass materially in a long panel.

The minimum MacroFinance limitations to record are:

- latent dimension and particle degeneracy pressure;
- long observation spans that compound small per-period target changes;
- missing, ragged-edge, or mixed-frequency observation patterns;
- volatility and measurement-noise directions with sharp curvature;
- compiled repeatability across seeds, batches, and hardware settings;
- model-risk governance, including reproducible artifacts and claim boundaries.

These limitations do not rule out DPF research. They rule out treating a smooth differentiable filtering path as operational-claim evidence before the target and validation ladder have been passed.

## 30.8 Relation to surrogate-HMC acceleration

Surrogate-HMC and Hamiltonian-neural-network ideas address a different point in the inference stack [Greydanus et al., 2019, Dhulipala et al., 2022, Hoffman et al., 2019, Parno and Marzouk, 2018]. Chapter 51 gives the complete construction, proves the discrete proposal conditions, and explains why a true endpoint likelihood value is not by itself a universal correction. These methods attempt to improve movement through an already chosen target, or to use a surrogate proposal that is later corrected against that target. DPF methods may instead alter the filtering likelihood itself through flow approximation, relaxed resampling, or learned transport. The distinction is operational:

- a corrected surrogate-HMC proposal can preserve the intended target if the correction evaluates that target and the proposal assumptions hold;
- a relaxed DPF likelihood defines a new target unless a correction returns to the original likelihood;
- a learned DPF transport map defines a learned target unless the sampler corrects against the teacher or original target.

Table 30.3: Method-family distinctions relevant to DPF-HMC claims.

| Method family | What is approximated? | Target implication | Required wording |
|---|---|---|---|
| Classical exact HMC | Hamiltonian trajectory by numerical integration. | Target preserved after Metropolis correction when value, gradient, and proposal conditions hold. | Exact for the named scalar target, not exact because the trajectory is exact. |
| Pseudo-marginal MCMC | Likelihood value by an unbiased nonnegative estimator on extended space. | Marginal posterior can be exact under pseudo-marginal assumptions. | Corrected extended-space chain, not smooth surrogate-gradient HMC. |
| Noisy-gradient method | Gradient or force field used for movement. | No exact posterior claim without correction theory. | Diagnostic or approximate unless corrected. |
| Delayed-acceptance surrogate | Early-stage value or proposal geometry. | Can preserve intended target if final correction evaluates intended target. | Surrogate-assisted, not target-defining, when corrected. |
| Relaxed DPF | Filtering likelihood construction itself. | Posterior changes to relaxed posterior. | Relaxed-target HMC. |
| Learned DPF | Transport, residual, or filtering map by a learned function. | Posterior changes to learned posterior unless corrected. | Learned-surrogate HMC pending correction or validation. |

## 30.9 Evidence ladder for stronger external language

The chapter's recommendation is evidence-gated. A method can be mathematically interesting, credible as a research prototype, and still not yet justify stronger external claims. The following ladder constrains the wording used in this chapter and in later experiments.

Table 30.4: Evidence ladder for DPF-HMC claim language.

| Claim level | Allowed claim | Required evidence | Disallowed shortcut |
|---|---|---|---|
| Exploratory use | The rung is worth studying on controlled examples. | Named scalar value, value-gradient check, finite smoke tests, clear target status. | Calling the method suitable because it is differentiable. |
| Credible research use | The rung has a defensible target interpretation for a named model class. | Source-grounded derivation, repeated fixed-seed and varied-seed tests, posterior comparisons, failure-region diagnostics. | Treating a toy result as evidence for DSGE or MacroFinance deployment. |
| Broader external claim | The rung supports a restricted external research conclusion with stated limitations. | Reproducible demonstrations, model-specific stress tests, challenger comparisons, and documented limitations. | Using runtime improvement as target-correctness evidence. |
| Operational claim | The rung is proposed for controlled use in a defined workflow. | Independent review, change control, monitoring, rollback, data and model stability across releases. | Inferring operational readiness from chapter-level mathematics. |

No rung in this chapter is promoted to an operational claim. The strongest current recommendation is a bounded experimental hypothesis: prioritize proposal-corrected PF-PF first because it is the first rung with an explicit proposal-correction interpretation, then compare relaxed and learned rungs as deliberately changed targets. The hypothesis remains conditional on the finite- particle, Jacobian, randomness, compilation, and posterior-comparison checks stated above.

## 30.10 Recommendation

The disciplined recommendation is:

(i) use EDH and LEDH as recovery and approximation benchmarks, not as generic exact HMC targets;

(ii) treat proposal-corrected PF-PF as the first experimental hypothesis with an explicit proposal-correction interpretation and a defensible value-side target story, subject to finite-particle, Jacobian, randomness, compilation, and posterior-comparison checks;

(iii) label soft resampling and OT/EOT as relaxed-target HMC unless a correction restores the original categorical or trusted target;

(iv) label learned OT as learned-surrogate HMC unless a correction or posterior-error argument justifies stronger wording;

(v) require model-class evidence before any stronger external claim.

For the differentiable particle-flow route studied in this monograph, the serious experimental path is therefore not a no-resampling shortcut. The route to test is Li–Coates Algorithm 1 UKF LEDH with PF-PF correction and a declared OT, Sinkhorn, or annealed-transport resampling relaxation. Carrying the Algorithm 1 covariance state through that transport is an implementation contract for the selected route, because the covariance state belongs to Algorithm 1 while the transport map defines the relaxed resampling layer. A no-resampling graph or fixed-randomness run can still be useful as a kernel, shape, or gradient-plumbing diagnostic, but it is not by itself evidence that the DPF route is filter-ready or HMC-ready.

## 30.11 What this chapter does and does not claim

This chapter claims that DPF-HMC analysis must be organized around named scalar targets and value-gradient consistency. It does not claim:

- that PF-PF has already passed finite-particle, Jacobian, compiled-path, or posterior-comparison checks,
- that relaxed or learned resampling targets are unusable,
- that pseudo-marginal validity transfers to deterministic relaxed gradients,
- that nonlinear DSGE or MacroFinance DPF-HMC has already been fully validated.

Its role is narrower: it clarifies which scalar target is being differentiated, what correction mechanism is being invoked if any, and what mathematical interpretation the resulting chain can support.

# Chapter 31

# DPF Literature-to-Debugging Verification Contract

The preceding chapter turned the DPF and transport staircase into a target-status taxonomy. The present chapter turns that taxonomy into a verification contract. Its purpose is not to validate any implementation by prose. Its purpose is to tell the reader which equation, controlled example, implementation quantity, tolerance, and failure interpretation belongs to each layer.

The contract is deliberately ordered. Target and value-gradient checks come before tuning. Teacher-map diagnostics come before learned-map diagnostics. Equation-local failures should be interpreted narrowly: a failed Sinkhorn marginal test is evidence about a finite solver, not evidence that particle flows are wrong; a learned-map residual is evidence about the student and its training distribution, not evidence that the OT teacher is invalid.

## 31.1 Diagnostic order

Use this order before changing code or tuning numerical parameters:

(1) name the scalar object being estimated, relaxed, learned, or sampled;

(2) verify that the gradient path corresponds to that same scalar object whenever HMC or autodiff claims are made;

(3) test the classical particle-filter recursion and likelihood estimator on a controlled model;

(4) test the particle-flow endpoint and homotopy derivative before testing PF-PF proposal correction;

(5) test PF-PF weights and log-determinant accounting before interpreting resampling or OT behavior;

(6) test categorical, soft, EOT, Sinkhorn, and barycentric objects before testing a learned student map;

(7) only after those checks pass, run posterior comparisons and sampler diagnostics.

This order prevents a common error: making a numerically smooth path smoother without first proving that it is the intended path.

## 31.2 Proposal log-probability autodiff topology

The most expensive debugging lesson in one same-object implementation tie-out was not a floating-point tolerance issue and not a seed issue. It was an autodiff-topology issue in the adapted proposal log probability. The forward numbers can be exactly right while the derivative is wrong if the proposal sample and the proposal log probability are wired through the same distribution object in the computation graph.

Write

$$f_\theta(x_t \mid x_{t-1}) \quad \text{and} \quad g_\theta(y_t \mid x_t)$$

for the transition and observation densities. For an adapted proposal

$$q_\theta(x_t \mid x_{t-1}, y_t),$$

the one-step log-weight update corresponding to (20.12) is

$$\log \tilde{w}_t^{(i)} = \log w_{t-1}^{(i)} + \log f_\theta(x_t^{(i)} \mid x_{t-1}^{(i)}) + \log g_\theta(y_t \mid x_t^{(i)}) - \log q_\theta(x_t^{(i)} \mid x_{t-1}^{(i)}, y_t). \tag{31.1}$$

The last term is not the log probability of a random-number-generation event. It is the density of the realized particle value under the declared proposal law.

In the linear-Gaussian adapted-proposal case used by one representative debug trace, the optimal proposal satisfies

$$q_\theta(x_t \mid x_{t-1}, y_t) = p_\theta(x_t \mid x_{t-1}, y_t) = \frac{f_\theta(x_t \mid x_{t-1}) g_\theta(y_t \mid x_t)}{p_\theta(y_t \mid x_{t-1})}. \tag{31.2}$$

Substituting (31.2) into the signed increment in (31.1) gives

$$\begin{aligned} &\log f_\theta(x_t \mid x_{t-1}) + \log g_\theta(y_t \mid x_t) - \log q_\theta(x_t \mid x_{t-1}, y_t) \\ &\quad = \log p_\theta(y_t \mid x_{t-1}). \end{aligned} \tag{31.3}$$

Thus, when the proposal is the exact adapted proposal, the signed increment is independent of the realized value $x_t$ after cancellation. Equivalently,

$$\frac{\partial}{\partial x_t} \left[ \log f_\theta(x_t \mid x_{t-1}) + \log g_\theta(y_t \mid x_t) - \log q_\theta(x_t \mid x_{t-1}, y_t) \right] = 0.$$

This zero derivative is a cancellation of three density derivatives. It is not obtained by making the proposal-density derivative vanish.

The correct implementation topology therefore has two conceptual steps:

(i) sample the proposed particle from the proposal law;

(ii) evaluate the proposal density at the realized particle as an ordinary density argument.

In pseudocode:

```
sample_dist = make_proposal_dist(previous_state, observation)
proposed_particles = sample_dist.sample(seed)

density_dist = make_proposal_dist(previous_state, observation)
proposal_ll = density_dist.log_prob(proposed_particles)
```

The second distribution has the same numerical parameters as the first, but it is a fresh density-evaluation node. Equivalently, one may call a helper such as `optimal_proposal_log_prob(...)` that rebuilds the proposal mean and covariance before applying `log_prob`. This is the topology used by FilterFlow's executable contract: `propose` first returns a state whose `particles` field contains the sampled particles, and `loglikelihood(proposed_state, state, observation)` then recomputes the proposal distribution and evaluates its density at `proposed_state.particles`.

The tempting but wrong topology is

```
proposal_dist = make_proposal_dist(previous_state, observation)
proposed_particles = proposal_dist.sample(seed)
proposal_ll = proposal_dist.log_prob(proposed_particles)
```

This can be value-correct: the scalar value of `proposal_ll` may match the fresh-density value exactly. But the derivative can be wrong because the autodiff graph is no longer the graph of the mathematical density function $x_t \mapsto \log q_\theta(x_t \mid x_{t-1}, y_t)$. The proposed particle is now also the reparameterized sample produced by the same distribution node. In one archived fixture trace from the BayesFilter/FilterFlow comparison campaign, that wiring made the BayesFilter VJP

$$\frac{\partial}{\partial x_t} \log q_\theta(x_t \mid x_{t-1}, y_t)$$

with unit upstream equal to zero, while FilterFlow's density-at-realized-particle route gave a maximum absolute VJP of

$$26.075772828701865.$$

The forward proposal log-probability values still agreed. The value agreement therefore hid the bug.

The downstream effect was exactly the cancellation failure predicted by (31.3). With the naive wiring, BayesFilter's unit-upstream VJP of

$$\log f_\theta + \log g_\theta - \log q_\theta$$

with respect to the proposed particles had maximum absolute discrepancy

$$26.075772828710043,$$

and the normalized previous-log-weight carry VJP had discrepancy

$$7.930673258915529.$$

After evaluating the proposal log probability through a freshly recomputed proposal density, the proposal-log-probability VJP matched FilterFlow exactly in that probe, the signed-sum discrepancy fell to about

$$1.49 \times 10^{-11},$$

and the normalized carry discrepancy fell to about

$$1.45 \times 10^{-11}.$$

These numbers are evidence from one concrete comparison trace; the general rule is the topology rule above.

The audit implication is strict. A DPF implementation may pass value replay, seed replay, particle replay, and proposal-log-probability value replay while still failing the derivative contract. Every adapted-proposal implementation that is used for gradient comparison must therefore check the proposal log-probability VJP as a density evaluation at the realized particle, not only the scalar log-probability value.

## 31.3 Same-object deterministic tie-out contract

A deterministic implementation tie-out is a same-object comparison, not an oracle test. The only statement it supports is conditional:

> Under a frozen deterministic comparison contract, two implementations can be shown to match on declared density, filter-path, fixed-ancestor, and fixed-branch AD-gradient surfaces.

The contract freezes the objects that would otherwise make a particle-filter comparison ambiguous:

(i) the model row and physical parameter vector;

(ii) density probes for initial, transition, and observation log densities;

(iii) initial particles, observations, and additive transition innovations;

(iv) the scalar, namely the sum of predictive log normalizers for the path comparisons;

(v) branch schedules: no resampling for one path test, and fixed ancestor indices for the branch-replay path test;

(vi) the gradient scalar and physical knobs for the fixed-branch AD-gradient comparison.

The comparison deliberately avoids random-number-generator equality. When an ancestor schedule is needed, it is treated as data. Both sides receive the same schedule and replay the same branch.

The illustrative comparison suite contained six rows:

`lgssm_2d_h25_rich`, `sv_1d_h18_rich`, `range_bearing_4d_h20_rich`, `structural_ar1_quadratic_h16`, `spatial_sir_j3_rk4`, `predator_prey_rk4`.

The closed deterministic comparison reported:

- density tie-out: all six rows matched, with maximum absolute delta 0;
- deterministic no-resampling path tie-out: all six rows matched, with maximum absolute delta 0;
- deterministic fixed-ancestor path tie-out: all six rows matched, with maximum absolute delta 0, and the expected one resampling event was replayed on each executed row;

- fixed-branch AD-gradient tie-out: all five executable gradient rows matched on scalar and AD-gradient values, with maximum scalar delta 0 and maximum AD-gradient delta 0.

The spatial SIR row was not promoted to the gradient comparison. It remains a gradient-contract block under the frozen manifest, not a failure and not a success.

Finite differences are diagnostic-only in this contract. They are useful for finding gross scalar/gradient wiring mistakes, but they are not stable enough to be a gate for the comparison. In one gradient artifact the range-bearing same-implementation finite-difference diagnostic had maximum reported discrepancy about

$$1.16 \times 10^{-4},$$

while the BayesFilter and FilterFlow AD gradients still matched exactly under the fixed-branch scalar. This is recorded as a numerical diagnostic, not as a promotion criterion and not as a veto.

Two robustness diagnostics sit below this gate:

- a seeded fixed-ancestor diagnostic generated three frozen ancestor schedules and replayed them across the full comparison suite. All executed rows matched with maximum absolute delta 0. This is branch-robustness evidence only; it is not a stochastic-resampling distribution claim;

- a controlled one-dimensional LGSSM horizon ladder compared $T = 2$ and $T = 4$. Both scenarios had implementation agreement. The $T = 4$ scenario retained a forward residual veto in an older Sinkhorn residual diagnostic, so the ladder is explanatory and does not change the main deterministic comparison conclusion.

The non-claims are part of the contract. The BayesFilter/FilterFlow tie-out does not prove filter correctness, does not prove that either implementation is mathematically correct, does not validate stochastic resampling distributions, does not validate gradients through random or discrete ancestor selection, and does not compare to student implementations, tensor-train or sparse-grid alternatives, dense quadrature, paper tables, GPU behavior, HMC behavior, DSGE behavior, scalability, deployment, or production readiness.

## 31.4 Equation-to-test matrix

Table 31.1: Equation-indexed verification matrix for the DPF block.

| Layer | Equation or claim | Controlled example | Implementation quantity | Pass signal | Narrow failure interpretation |
|---|---|---|---|---|---|
| Filtering recursion | (20.3), (20.4) | linear-Gaussian state-space model with Kalman reference | predict/update law, normalized weights, filtered mean/covariance | moments match reference within declared Monte Carlo error | recursion, normalization, or model-interface bug; not an OT or learned-map claim |
| Bootstrap likelihood estimator | (20.16), (20.17) | same linear-Gaussian model; fixed seeds and increasing $N$ | per-period average weights and log likelihood estimator | Monte Carlo mean approaches Kalman likelihood; variance decreases with $N$ | particle proposal or weight bug; not evidence about differentiability |

Table 31.1: Equation-indexed verification matrix for the DPF block, continued.

| Layer | Equation or claim | Controlled example | Implementation quantity | Pass signal | Narrow failure interpretation |
|---|---|---|---|---|---|
| Homotopy derivative | (21.1), (21.3) | scalar nonlinear observation with analytic log-likelihood derivative | finite difference in $\lambda$ versus analytic derivative | central difference and analytic derivative agree under step refinement | homotopy algebra or likelihood derivative issue; not yet a PF-PF correction issue |
| EDH endpoint | (21.11), (21.15) | linear-Gaussian recovery with known posterior mean/covariance | flow endpoint particles, mean, covariance, and endpoint residual | endpoint moments match Kalman update within flow and Monte Carlo tolerances | flow ODE or Gaussian-closure implementation issue; not HMC target validity |
| LEDH local endpoint | (21.17), (21.24) | synthetic observation with known local Jacobian variation | particle-local Jacobians, local precision, and endpoint continuity | local quantities are finite and stable under small perturbations | local linearization fragility; not proof that global EDH is wrong |
| PF-PF weight | (22.2), (22.3), (22.8) | synthetic affine flow with known determinant | proposal density, target density, corrected log weight | manual density ratio and implemented weight agree | proposal-density or correction algebra bug; not a resampling diagnosis |
| Log-determinant ODE | (22.31), (22.32), (22.33) | affine flow $x_\lambda = A_\lambda x + b_\lambda$ | integrated log determinant and finite-difference determinant | sign and magnitude match under step-size refinement | Jacobian sign, time direction, or integrator issue; not a likelihood theorem |
| Categorical discontinuity | (23.2) | two-particle resampling with weight crossing | ancestor index path and realized resampled cloud | jump occurs as expected; no false pathwise gradient is reported | discrete resampling is being differentiated incorrectly; not proof soft resampling is valid |
| Soft-resampling bias | (23.5), (23.7) | two-particle nonlinear test function | mean preservation and nonlinear test-function error | affine tests behave as derived; nonlinear bias is visible when expected | surrogate-law effect; not evidence of categorical-target preservation |
| EOT/Sinkhorn marginals | (25.4), (25.14), (25.20) | small cost matrix with prescribed marginals | row residual, column residual, cost, entropy, $\varepsilon$, iteration count | residuals fall below tolerance and are stable under log-domain implementation | finite-solver or stabilization issue; not proof of posterior invariance |
| Barycentric projection | (25.21) | small coupling with known column masses | projected equal-weight support cloud and dimensions | output has declared shape and column-mass normalization | projection or shape bug; not a coupling-optimality result |
| Barycentric moment contract | (25.24) | linear-Gaussian, SV, or SIR fixture with no-OT weighted comparator | pre-reset weighted mean/covariance, post-reset equal-weight mean/-covariance, and downstream value delta | mean preservation and declared covariance contraction are visible; any Kalman or exact-likelihood gate is labeled same-scalar or moment-preserving | finite-$N$ reset changes the filtering scalar; do not interpret a same-scalar gradient as an exact-likelihood gradient |
| Learned-map residual | (27.1), (27.14), (27.12) | permutation-equivariance test and held-out teacher clouds | teacher/student residuals, permutation residual, training-distribution tags | residuals stay below threshold on the declared envelope | student or distribution-shift issue; not a teacher-map refutation |
| HMC value-gradient contract | (30.1) | finite-difference check on a fixed scalar target and fixed random seed policy | scalar value, autodiff gradient, finite-difference gradient, accept/reject value | all paths refer to the same scalar and gradients agree within tolerance | target-interface bug; do not tune HMC before resolving it |

## 31.5 Minimal controlled examples

Table 31.2: Minimal controlled examples for DPF verification.

| Example | Required inputs | Expected output | Does not prove |
|---|---|---|---|
| Linear-Gaussian recovery | Known transition, observation, covariance matrices, Kalman likelihood, and Kalman filtering moments. | Particle recursion, EDH endpoint, and likelihood estimates approach the Kalman reference under increasing particles or decreasing flow step size. | Nonlinear DSGE validity or learned-map generalization. |
| Synthetic affine flow | Invertible affine map, analytic determinant, and known proposal density. | PF-PF density ratio, weight, and log determinant match hand calculation. | Correctness for nonlinear flow fields. |
| Scalar nonlinear observation | One-dimensional observation function with analytic derivatives and controlled curvature. | Homotopy derivative and local-linearization diagnostics show expected error as curvature changes. | High-dimensional stability. |
| Two-particle soft-resampling bias | Two particles, weights crossing 1/2, and nonlinear test function. | Categorical path is discontinuous; soft path has the predicted surrogate bias. | Acceptability of soft resampling as an HMC target. |
| Small Sinkhorn problem | Positive cost matrix, weights, equal target marginal, fixed $\varepsilon$, and known residual tolerance. | Row/column residuals and stabilized objective behave consistently with the solver budget. | Posterior equivalence to categorical resampling. |
| Permutation-equivariance learned-map test | Teacher clouds, learned map, and permutation matrices. | Permuting input rows permutes output rows, and scalar summaries remain invariant when required. | Small teacher/student error or target preservation. |
| Finite-difference HMC gradient test | Fixed scalar target, fixed random seed policy, finite-difference step ladder, and autodiff gradient. | Autodiff and finite-difference gradients agree for the scalar used by accept/reject. | Convergence of the Markov chain or correctness for another scalar target. |

## 31.6 Diagnostic specification

Every diagnostic result should be recorded with the following fields:

- **Equation or claim**: label from the chapter, not only a method name.
- **Inputs**: model, particles, weights, random seed policy, $\varepsilon$, iteration budget, architecture, and scalar target as applicable.
- **Expected output**: scalar, vector, matrix, empirical measure, residual, or posterior summary.
- **Tolerance**: absolute and relative thresholds, Monte Carlo standard-error logic, or convergence slope.
- **Failure interpretation**: the narrow mathematical layer that the failure implicates.
- **Non-implication**: what the failure does not prove.

Table 31.3: Diagnostic contract by layer.

| Diagnostic | Inputs | Tolerance rule | Failure interpretation | Non-implication |
|---|---|---|---|---|
| Filtering recursion parity | Kalman reference, fixed particles, fixed model. | Monte Carlo error decreases with $N$; deterministic quantities match reference when applicable. | State-space interface, normalization, or weight update bug. | Does not reject OT, learned OT, or HMC. |
| Flow endpoint parity | Linear-Gaussian recovery, flow step ladder. | Endpoint error decreases as flow integration is refined. | Flow ODE, closure, or integration issue. | Does not prove PF-PF correction is wrong. |
| PF-PF algebra parity | Affine flow, analytic determinant, explicit proposal density. | Log-weight difference below deterministic tolerance. | Density-ratio, Jacobian, or sign error. | Does not diagnose resampling. |
| Soft-resampling bias check | Two-particle nonlinear test. | Bias sign and magnitude match derived expression within arithmetic tolerance. | Surrogate target is being confused with categorical target. | Does not prove soft resampling is unusable. |
| Sinkhorn residual check | Small cost matrix, fixed $\varepsilon$, solver budget. | Row/column residuals below declared threshold and improve with budget. | Finite-solver or stabilization issue. | Does not prove EOT is a valid posterior target. |

| Diagnostic | Inputs | Tolerance rule | Failure interpretation | Non-implication |
|---|---|---|---|---|
| Learned residual check | Teacher outputs, held-out clouds, distribution tags. | Residuals reported by regime; thresholds tied to training envelope. | Student approximation or extrapolation issue. | Does not invalidate the teacher. |
| HMC value-gradient check | Scalar value, autodiff path, finite-difference ladder. | Gradient errors shrink under step refinement until numerical limits. | Value-gradient mismatch or nondifferentiable scalar path. | Does not prove chain convergence. |

## 31.7 Source and implementation map

Table 31.4: Source, chapter, equation, and implementation links.

| Layer | Chapter location | Equation labels | Source family | Implementation quantity |
|---|---|---|---|---|
| Classical PF | Chapter 20, Sections 20.3–20.7 | (20.4), (20.17) | SMC literature spine | weights, ESS, likelihood estimator, empirical moments |
| Particle flow | Chapter 21, Sections 21.3–21.7 | (21.3), (21.14), (21.24) | Daum–Huang, PF-PF flow literature | flow field, endpoint residual, local Jacobians |
| PF-PF correction | Chapter 22, Sections 22.3–22.11 | (22.8), (22.32) | proposal correction and change-of-variables spine | proposal density, target density, log determinant, corrected weight |
| Resampling and OT | Chapter 23; Chapter 24; Chapter 25 | (23.2), (23.7), (24.12), (25.4) | differentiable resampling, OT, Sinkhorn sources | ancestor path, soft surrogate, coupling residuals, barycentric cloud |
| Learned OT | Chapter 27 | (27.1), (27.14), (27.12) | EOT teacher and set-architecture sources | teacher output, student output, residuals, distribution tags |
| HMC target | Chapter 30, Sections 30.1–30.9 | (30.1) | HMC, pseudo-marginal, surrogate-HMC spine | scalar value, gradient, accept/reject value, target-status label |

**Why the OT doctrine starts here.** The preceding sections were equation-local: they tracked which scalar, Jacobian, log-determinant, and autodiff path the implementation is actually using. The OT question now adds a second layer. Once a candidate transport method is proposed, the issue is no longer only whether one equation was coded correctly, but whether the candidate preserves the same transport object, approximates it, or replaces it with a different one. The next sections therefore move from local debugging into candidate classification and promotion doctrine.

## 31.8 Scalable OT reuse-risk and validation doctrine

The scalable-OT survey sharpens the verification contract for the OT block in three ways. First, code availability is useful but does not by itself justify promotion. Second, exact-online, retained-teacher approximate, and structure-changing transport families should not share the same pass/fail story. Third, the validation ladder must test transported particle objects, not only OT scalars or paper-reported benchmark values.

A practical BayesFilter review should therefore separate three lanes:

(i) **Exact reference lane:** dense or streamed entropic OT/Sinkhorn implementations that preserve the same declared teacher object [Feydy et al., 2019, Charlier et al., 2021, Ye et al., 2026, Xiao, 2026].

(ii) **Retained-teacher approximation lane:** Nystrom, positive-feature, low-rank, or similar accelerators judged first by teacher-preservation residuals [Altschuler et al., 2019, Scetbon and Cuturi, 2020, Forrow et al., 2019, Scetbon et al., 2021].

(iii) **Structure-changing lane:** sliced, localized, block, or other replacement transport rules judged as explicit new resampling approximations [Kolouri et al., 2019, Deshpande et al., 2019, Paty and Cuturi, 2019, Nguyen, 2025].

The first question for any candidate is therefore not "is it faster?" but "which of these lanes does it belong to?" The rest of the diagnostic stack depends on that classification.

| Family | Lane | Object | Change | First diagnostic |
|---|---|---|---|---|
| Exact online / streamed Sinkhorn | Exact reference | finite entropic coupling and barycentric cloud | memory traffic and execution strategy | marginal residual and barycentric parity |
| Nystrom / positive-feature / low-rank routes | Retained-teacher | corrected teacher-side coupling or barycentric output | kernel or coupling execution path | teacher-preservation residual |
| Sparse / screened variants | Boundary case | same finite object only if support restriction is harmless | active pair count or screened work | same-object parity before speed claims |
| Sliced / localized / block replacements | Structure-changing | projected, local, or replacement transport rule | full-state transport semantics | declared replacement criterion plus downstream diagnostics |

### 31.8.1 Code availability and reuse risk are prefilters, not verdicts

Source availability is a practical filter because BayesFilter needs an exposed transport object, not only a scalar loss. But a repository with a readable commit, a visible API, or even a working prototype does not become default-ready for BayesFilter on that basis alone. Reuse risk should therefore be recorded as a narrow implementation note:

(1) whether the code exposes a usable transport object or only a scalar loss,

(2) whether the backend is compatible with the BayesFilter TensorFlow/TFP default,

(3) whether the source is complete and inspectable,

(4) whether the code path matches the mathematical object cited in the literature,

(5) whether any missing piece would force BayesFilter to invent new semantics locally rather than reproduce the cited object.

A failure here does not refute the mathematics. It only limits reuse value and raises implementation-surface risk.

| Source style | Object exposed? | Backend fit | Main reuse risk | BayesFilter implication |
|---|---|---|---|---|
| Dense or streamed entropic OT references | usually yes | strongest when TensorFlow/TFP-aligned | exact semantics but poor scaling if all pairs remain | best correctness baseline |
| Retained-teacher factorized approximations | only if corrected coupling or barycentric output is explicit | acceptable if local correction is implementable | latent approximation may hide semantic drift | useful only with teacher-preservation audit |
| Sparse or screened libraries | sometimes partial only | often specialized tooling | support restriction may change the effective finite object | treat as boundary case, not automatic drop-in |
| Minibatch or stochastic libraries | often partial or local only | mixed | local plans need not define one full deterministic resampling map | concept lane until full-ensemble semantics are declared |

### 31.8.2 Validation ladder before promotion

Before any scalable OT candidate is promoted beyond a literature note or local prototype, the review should follow a fixed ladder. The central rule is that transported-particle parity comes before downstream performance claims.

(1) deterministic dense parity on small fixtures: compare transported particles or transported summaries, not only OT objective values;

(2) marginal checks: source and target mass constraints under weighted inputs;

(3) positivity, non-finite, and numerical-stability checks;

(4) representation diagnostics appropriate to the lane: rank, feature dimension, sparsity, projection count, block locality, or minibatch design;

(5) LEDH-PFPF-OT value parity on existing BayesFilter fixtures;

(6) gradient smoke tests only after value-side parity is understood;

(7) runtime and memory ladder over particle count and state dimension;

(8) downstream filtering diagnostics;

(9) multi-seed or uncertainty-aware comparisons before ranking stochastic candidates;

(10) default-readiness review only after downstream evidence, not from source availability or paper benchmarks alone.

This ladder is deliberately asymmetric. Passing an early rung does not justify a later claim, but failing an early rung can already block promotion.

### 31.8.3 Lane-specific narrow interpretations

The failure interpretation should remain lane-specific.

- In the exact reference lane, a failed marginal or barycentric parity check is evidence about the finite solver or implementation path.

- In the retained-teacher approximation lane, a large teacher-preservation residual is evidence about the approximation family or its tested envelope, not a refutation of the teacher object itself.
- In the structure-changing lane, disagreement with the dense teacher is not automatically failure; the question is whether the replacement criterion and downstream validation have been declared and met.

This prevents a common review error: treating every non-dense transport as if it were trying to prove exact parity with the same teacher object.

### 31.8.4 Recommended research order

The scalable-OT survey also suggests a practical order for future BayesFilter work. That order should follow the three-lane split rather than a single global race for runtime.

(i) **Exact reference first:** keep the dense or streamed entropic Sinkhorn route as the semantics-preserving baseline and measure how far better execution strategy alone can scale.

(ii) **Retained-teacher approximations second:** test Nystrom, positive-feature, low-rank, or related factorized routes by teacher-preservation residuals before treating them as candidate drop-in accelerators.

(iii) **Structure-changing candidates third:** study sliced, localized, block, or other replacement transports only under an explicit replacement criterion and downstream filtering diagnostics.

This order is not a statement that the first lane will ultimately win. It is a statement about evidence discipline. A route that preserves the teacher object should be understood before a route that intentionally changes it.

## 31.9 Promotion thresholds

Table 31.5: Claim thresholds for DPF implementation interpretations.

| Claim level | Required passing evidence | Disallowed interpretation | Gate |
|---|---|---|---|
| Exploratory implementation | Layer-local smoke tests pass on minimal controlled examples; failures are recorded with equation labels. | Treating smoothness or runtime as target correctness. | May run more experiments. |

| Claim level | Required passing evidence | Disallowed interpretation | Gate |
|---|---|---|---|
| Credible research implementation | Classical PF, flow, PF-PF, resampling/OT, learned-map, and HMC value-gradient checks pass on named model classes with sensitivity reports. | Generalizing beyond tested $N$, $d_x$, $\varepsilon$, model class, or target status. | May make bounded research claims. |
| Broader external claim | Research checks plus posterior comparisons, stress regimes, reproducible demonstrations, and documented failure policies. | Claiming deployment suitability from a chapter or benchmark alone. | Requires separate domain review. |
| Operational deployment claim | Independent review, change control, monitoring, rollback, data/-model governance, release stability, and ongoing diagnostics. | Assuming chapter-level mathematics alone is sufficient approval. | Requires a separate operational process. |

### 31.9.1 Closing scalable-OT synthesis

Taken together, the scalable-OT survey and the present monograph support a simple three-lane posture. Exact online or streamed Sinkhorn remains the reference lane because it preserves the same declared entropic teacher object while improving execution strategy. Nystrom, positive-feature, low-rank, and related factorized routes belong to the retained-teacher lane only when they are still judged first by teacher-preservation residuals rather than by runtime alone. Sliced, localized, block, or other replacement transports belong to the structure-changing lane and therefore require an explicit replacement criterion plus downstream filtering evidence.

The main non-claim is equally important. A fast repository, a smooth scalar, a small local residual, or a favorable benchmark number does not by itself promote a scalable-OT candidate to BayesFilter default status. Promotion still requires the same ledger fixed above: transported-particle parity or a declared replacement criterion, marginal checks, numerical-stability checks, and downstream filtering diagnostics matched to the lane being claimed.

## 31.10 Boundary of this contract

This chapter does not validate any DPF implementation. It defines the minimum diagnostic evidence needed to interpret failures and to avoid promoting a method beyond its evidence. The word "validate" is therefore reserved for a specific equation-local check or for a separately administered external review; it is not used here as a blanket claim about DPF, learned OT, or HMC.

# Chapter 32

# Filter Choice

Filter choice is the capstone decision register for the preceding particle, transport, HMC-target, and verification staircase. It is not only a speed decision. For HMC, the selected backend determines the log likelihood, the derivative path, the finite-failure policy, and the compilation surface. One should choose the simplest backend that gives the required target fidelity and a validated gradient.

## 32.1 Decision Register

The decision should be recorded in a short register:

**Exact Kalman, solve, or square-root LGSSM** Best for linear Gaussian likelihoods and regression oracles. The main risks are SPD failures, mask handling, and large-scale factor failures. The relevant check is analytic/custom score parity plus compiled value-gradient parity.

**EKF** Best as a cheap nonlinear baseline and affine recovery check. The main risks are linearization bias and Jacobian fragility. The relevant check is affine Kalman recovery plus finite-gradient stress tests.

**Sigma-point** Best for moderate nonlinear Gaussian approximations. The main risks are an approximate target and covariance-update instability. The relevant check is reference recovery, static sigma-point shapes, and derivative parity.

**Square-root sigma-point** Best for value-side robustness under near-singular covariance. The main risk is false confidence when the implementation reconstructs covariance matrices internally. The relevant check is factor reconstruction, finite gradients, and compiled parity.

**SVD sigma-point** Historical eigenderivative UKF/SVD routes are retained primarily for diagnostic comparison, regression baselines, and bounded small-case oracles. The main risks are close or repeated spectral values and raw autodiff instability. They should not be the preferred HMC-facing route once the strict-SPD principal-square-root backend is available. The remaining role of the historical branch is eigen-gap telemetry, failure-mode diagnosis, and comparison against the promoted principal-square-root branch.

**Particle or differentiable particle filter** Best for non-Gaussian diagnostics and frontier approximations. The main risks are Monte Carlo variance, resampling non-differentiability, and relaxed target bias. The relevant check is an estimator-variance report together with a clearly declared correction policy.

## 32.2 Recommended Order

For a new model, BayesFilter should proceed in this order:

(i) reduce to an LGSSM or affine test case and validate the exact Kalman likelihood and derivatives;

(ii) add masks, missing-data patterns, and mixed-frequency timing while retaining exact references;

(iii) test EKF or sigma-point approximations on low-dimensional nonlinear examples with dense or trusted references;

(iv) only then attempt DSGE-scale sigma-point HMC on the promoted strict-SPD principal-square-root derivative route, using the historical eigenderivative/SVD branch only as a diagnostic comparison or bounded oracle;

(v) treat particle methods and transport methods as separate evidence streams unless a correction policy makes them target-preserving.

## 32.3 Industrial Default

For large industrial applications, the default should be an exact or controlled approximate likelihood with an analytic or custom derivative path. Raw tape gradients through spectral decompositions are acceptable as diagnostic tools and small-case oracles, but they should not be the final route unless spectral-gap, finite-gradient, and compiled parity evidence survives the intended stress regime.

This policy is intentionally conservative. The SVD sigma-point debugging work showed that value robustness and HMC-gradient robustness are different problems. The later high-dimensional, geometry, and case-study chapters use this register as the entry gate for application-specific backend choice rather than discovering those distinctions only after long chains.

# Part IX

# High-Dimensional Nonlinear Filtering

# Chapter 33

# High-Dimensional Nonlinear Filtering Foundations

## 33.1 The Filtering Problem In Its Least Forgiving Form

A high-dimensional nonlinear filter is not merely a large Kalman filter. It is an approximation to a sequence of conditional probability laws whose update can become singular, multimodal, sharply curved, or numerically inaccessible. This chapter fixes the mathematical objects that the later chapters approximate. It also separates exact identities from approximation policies. That separation matters because a filter used inside optimization or Hamiltonian Monte Carlo does not only produce state estimates; it defines a scalar likelihood target and possibly a gradient of that target.

Let $\theta \in \Theta$ be static parameters, $x_t \in \mathbb{R}^n$ a latent state, and $y_t \in \mathbb{R}^m$ an observation. In discrete time a state-space model is

$$x_0 \sim \mu_\theta, \qquad x_t \mid x_{t-1}, \theta \sim f_\theta(\cdot \mid x_{t-1}), \qquad y_t \mid x_t, \theta \sim g_\theta(\cdot \mid x_t). \tag{33.1}$$

The continuous-time diffusion analogue, used by the Zakai/DMZ and tensor-train PDE papers, is commonly written

$$dX_t = a_\theta(X_t, t)\, dt + B_\theta(X_t, t)\, dV_t, \tag{33.2}$$
$$dY_t = h_\theta(X_t, t)\, dt + dW_t, \tag{33.3}$$

with variants for correlated noise. The discrete recursion is the natural contract for production filtering APIs; the continuous formulation exposes the PDE and pathwise robust methods in Davis [1980], Yau and Yau [2000], Yau and Yau [2008], Li et al. [2019], and Meng et al. [2025].

For a reader coming from numerical analysis, physics, or computational chemistry, the central object is the conditional law $p_\theta(x_t \mid y_{1:t})$. The input object at time $t$ is the previous conditional law and a new observation; the output object is a new conditional law plus the normalizer $Z_t$. The exact target is the Bayes recursion below. Every later method is an approximation to one of three objects: the law $p_\theta(x_t \mid y_{1:t})$, the likelihood scalar $\ell_T(\theta)$, or a derivative of that scalar. A diagnostic is useful only when it says which of these objects it is checking.

**Running quadratic-observation cell.** The same teaching example will be used through Chapters 33–40:

$$x_t = \rho x_{t-1} + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q), \qquad y_t = x_t^2 + \epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, R). \tag{33.4}$$

It is deliberately small. Its purpose is to make the objects visible, not to validate a macro-finance model. With a one-step Gaussian prediction $X \sim \mathcal{N}(0, P^-)$, the filtering density after observing $y > 0$ is proportional to

$$\exp\left\{-\frac{x^2}{2P^-} - \frac{(y - x^2)^2}{2R}\right\}. \tag{33.5}$$

When $R$ is small and $y$ is away from zero, the likelihood rewards states near both $+\sqrt{y}$ and $-\sqrt{y}$. The exact filtering object is therefore a conditional law that may have two lobes. A mean alone, a covariance alone, or a finite scalar likelihood alone is only one view of that object.

| Object | Role in the exact recursion | Why the running cell exposes it |
|---|---|---|
| Conditional law $p_t$ | Probability law of $x_t$ given observations | $x^2$ observations can create two separated lobes. |
| Normalizer $Z_t$ | One-step predictive likelihood contribution | It integrates both lobes and is the scalar used in inference. |
| Likelihood $\ell_T$ | Sum of log normalizers over time | Changing $Z_t$ changes the parameter posterior. |
| Sensitivity $s_t$ | Derivative of the law or normalizer with respect to parameters | HMC needs the gradient of the same scalar it reports. |

**Assumption 33.1** (Dominated filtering model)**.** *The kernels in* (33.1) *admit densities with respect to reference measures, the predictive normalizing constants below are finite and positive on the observed path, and every differentiation under an integral sign is justified by a local dominating function. Continuous-time PDE statements additionally use the regularity, ellipticity, growth, and boundary assumptions stated in the cited source.*

## 33.2 Exact Discrete-Time Recursion And Likelihood

Under Assumption 33.1, prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int f_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1})\, dx_{t-1}, \tag{33.6}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{Z_t(\theta)}, \qquad Z_t(\theta) = \int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})\, dx_t. \tag{33.7}$$

The likelihood factors as

$$p_\theta(y_{1:T}) = \prod_{t=1}^{T} Z_t(\theta), \qquad \ell_T(\theta) = \sum_{t=1}^{T} \log Z_t(\theta). \tag{33.8}$$

The derivation is short, but it is the contract that every later approximation must respect. Write the joint density through time $t$ as

$$p_\theta(x_{0:t}, y_{1:t}) = \mu_\theta(x_0) \prod_{s=1}^{t} f_\theta(x_s \mid x_{s-1}) \prod_{s=1}^{t} g_\theta(y_s \mid x_s).$$

Integrating out $x_{0:t-1}$ after conditioning on $y_{1:t-1}$ gives (33.6). Multiplying the predictive density by the likelihood $g_\theta(y_t \mid x_t)$ gives an unnormalized density

$$\gamma_t(x_t) = g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}).$$

Its integral is exactly the one-step predictive density of the observation,

$$Z_t(\theta) = p_\theta(y_t \mid y_{1:t-1}).$$

Bayes' formula then gives (33.7). Finally, the chain rule for the observation density gives

$$p_\theta(y_{1:T}) = p_\theta(y_1) \prod_{t=2}^{T} p_\theta(y_t \mid y_{1:t-1}) = \prod_{t=1}^{T} Z_t(\theta).$$

Thus a filtering implementation that changes the normalizers changes the likelihood, not merely the state estimate.

For actual stochastic-volatility rows, the target discipline is now explicit: there is one cumulative likelihood target, $\ell_T(\theta) = \sum_{t=1}^{T} \log Z_t(\theta)$, and every admissible numerical route must approximate that same scalar. In the transformed observation parameterization used in the code and later chapters,

$$z_t = \log(y_t^2), \qquad z_t - \log(\beta^2) - h_t \sim \log(\chi_1^2),$$

so the one-step normalizer is defined by the exact transformed actual-SV observation law. Direct-likelihood quadrature, fixed-cloud reweighting, and the current Zhao–Cui comparator are therefore comparable only because they are meant to approximate that same normalizer sequence.

The previous actual-SV "Lane B" framing was wrong for this inference problem. It replaced the transformed actual-SV observation law by a Gaussian-closure construction built from predictive observation moments of an augmented variable. That replacement changes the implied one-step normalizer and therefore changes the cumulative likelihood target itself. A filter may still be internally self-consistent for that replacement scalar, but it is not evidence about the intended actual-SV likelihood in (33.8). Any implementation, benchmark, or gradient check must therefore keep two questions separate: whether a method differentiates its own declared scalar correctly, and whether that declared scalar is the right target in the first place.

### 33.2.1 Actual-SV Single-Target Derivation

The corrected actual-SV statement can now be written as a short derivation. Start from the raw observation equation

$$y_t = \beta e^{h_t/2} \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,1), \qquad \beta > 0. \tag{33.9}$$

For nonzero observations define the transformed datum

$$z_t = \log(y_t^2). \tag{33.10}$$

Squaring (33.9) gives $y_t^2 = \beta^2 e^{h_t}\varepsilon_t^2$, and taking logs yields

$$z_t - \log(\beta^2) - h_t = \log(\varepsilon_t^2). \tag{33.11}$$

Since $\varepsilon_t^2 \sim \chi_1^2$, the transformed observation density is therefore

$$g_\theta(z_t \mid h_t) = p_{\log \chi_1^2}\big(z_t - \log(\beta^2) - h_t\big)\,. \tag{33.12}$$

Thus the transformed actual-SV one-step normalizer is

$$Z_t(\theta) = \int g_\theta(z_t \mid h_t) p_\theta(h_t \mid z_{1:t-1})\, dh_t, \tag{33.13}$$

and the target remains the single cumulative likelihood $\ell_T(\theta) = \sum_{t=1}^T \log Z_t(\theta)$.

This identity fixes the admissible same-target routes. A dense exact-transformed reference, a direct-likelihood fixed-cloud rule, or the current factorized scalar Zhao–Cui wrapper may differ numerically, but they remain comparable only when they all evaluate (33.12) and then accumulate the resulting transformed normalizers. The direct-likelihood route has the generic node-reweighting form

$$\widehat{Z}_t^{\mathrm{dir}}(\theta; B) = \sum_{r=1}^M w_r g_\theta\big(z_t \mid h_t^{(r)}\big), \tag{33.14}$$

with node locations $h_t^{(r)}$ and weights $w_r$ supplied by the declared predictive approximation branch $B$. The dense reference and the present Zhao–Cui comparator are same-target for the same reason: their value paths also consume the transformed observations $z_t$ and the exact transformed density in (33.12).

**Proposition 33.1** (Augmented Gaussian closure changes the actual-SV scalar)**.** *Suppose instead that one augments the one-step state with an explicit observation-noise coordinate and uses the raw observation map in* (33.9)*. If the resulting filter propagates that augmented cloud through the raw map, replaces* (33.12) *by predictive observation moments, and accumulates a Gaussian innovation likelihood such as* (36.13)*, then the accumulated scalar is not the transformed actual-SV likelihood target* $\ell_T(\theta)$*.*

*Proof.* The transformed target in (33.13) is defined by integrating the exact transformed observation density $g_\theta(z_t \mid h_t)$. By contrast, the augmented Gaussian-closure route uses the raw map $y_t = \beta e^{h_t/2}\varepsilon_t$, forms predictive observation moments such as $\bar{z}_t$ and $S_t$, and then accumulates a Gaussian innovation scalar. That replacement changes the observation-density object inside the one-step normalizer. Once the one-step normalizer changes, the cumulative likelihood changes as well by (33.8). Hence the Gaussian-closure route may define a different declared approximate scalar, but it is not a same-target approximation to the transformed actual-SV likelihood. □

**Proposition 33.2** (When a Gaussian-closure likelihood surrogate can be reasonable)**.** *Let* $p_t^{\mathrm{obs}}(y)$ *be the predictive observation density induced by the correct augmented input law, and let* $(\bar{z}_t, S_t)$ *be the predictive mean and covariance used in the Gaussian innovation scalar* (36.13)*. Suppose:*

*(i) the sigma-point or fixed-cloud rule is applied to the correct augmented input variable, so that the moment statement in Proposition 19.2 applies;*

*(ii) the local small-uncertainty and smoothness hypotheses behind Proposition 19.2 hold, so the predictive mean and covariance are accurate through the stated second-order regime; and*

*(iii) on a neighborhood $\mathcal{U}_t$ containing the realized observation $y_t$, the predictive observation log density satisfies*

$$\log p_t^{\mathrm{obs}}(y) = \log \varphi_{n_y}(y; \bar{z}_t, S_t) + r_t(y), \qquad y \in \mathcal{U}_t, \tag{33.15}$$

*with remainder bounded by*

$$|r_t(y)| \le \delta_t, \qquad y \in \mathcal{U}_t. \tag{33.16}$$

*Then, provided $y_t \in \mathcal{U}_t$, the one-step Gaussian innovation scalar is a local likelihood surrogate with error bounded by*

$$\left| \log p_t^{\mathrm{obs}}(y_t) - \left[ -\frac{1}{2} \left\{ \log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi) \right\} \right] \right| \le \delta_t. \tag{33.17}$$

*Proof.* By definition of the Gaussian density,

$$\log \varphi_{n_y}(y_t; \bar{z}_t, S_t) = -\frac{1}{2} \left\{ \log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi) \right\},$$

with $v_t = y_t - \bar{z}_t$. Evaluating (33.15) at $y_t$ gives

$$\log p_t^{\mathrm{obs}}(y_t) - \log \varphi_{n_y}(y_t; \bar{z}_t, S_t) = r_t(y_t),$$

and (33.17) follows from (33.16). □

This proposition states the admissible sense in which a UKF-style or fixed-cloud Gaussian closure can still be useful. The structural UKF result controls predictive moments under the correct augmented input law; the extra remainder assumption controls how far the full predictive observation density is from the Gaussian density built from those moments. Both ingredients are needed. Accurate moments alone do not guarantee an accurate likelihood scalar when the predictive observation law is strongly skewed, heavy-tailed, multimodal, or otherwise far from Gaussian near the realized observation.

For actual SV this means the augmented Gaussian-closure route may be a reasonable local surrogate likelihood under a small-uncertainty, near-Gaussian predictive observation regime, but it still does not become the same target as the transformed exact likelihood in (33.13). It is a surrogate scalar, not a second exact route to the same inference problem.

### 33.2.2 Actual-SV SR-UKF Augmented-Noise Adapter

This subsection gives the model-specific adapter for the factor-propagating SR-UKF score contract in Section 17.3. It is deliberately an adapter for the raw-observation Gaussian-closure surrogate in Proposition 33.2; it is not an admission that the surrogate equals the transformed actual-SV target in (33.13).

For one panel coordinate, use the scalar latent recursion

$$H_t = \gamma H_{t-1} + U_t, \qquad U_t \sim \mathcal{N}(0, \sigma^2), \qquad Y_t = \beta \exp(H_t/2) E_t, \qquad E_t \sim \mathcal{N}(0,1). \tag{33.18}$$

The candidate score contract for this surrogate uses the unconstrained parameterization

$$\theta = (\theta_\gamma, \theta_\beta), \qquad \gamma = \Phi(\theta_\gamma), \qquad \beta = \exp(\theta_\beta), \qquad \sigma \text{ fixed}. \tag{33.19}$$

Here $\Phi$ is the standard normal distribution function and $\varphi$ is its density, so

$$\partial_{\theta_\gamma}\gamma = \varphi(\theta_\gamma), \qquad \partial_{\theta_\beta}\beta = \beta, \qquad \partial_{\theta_i}\sigma = 0. \tag{33.20}$$

A sigma-point adapter must place points on the pre-observation uncertainty variable

$$A_t = (H_{t-1}, U_t, E_t)^\top. \tag{33.21}$$

If the previous filtered approximation is

$$H_{t-1} \mid y_{1:t-1} \approx \mathcal{N}(m_{t-1}, c_{t-1}c_{t-1}^\top),$$

then the declared augmented law is

$$A_t \mid y_{1:t-1} \approx \mathcal{N}\left( \begin{bmatrix} m_{t-1} \\ 0 \\ 0 \end{bmatrix}, C_{a,t}C_{a,t}^\top \right), \qquad C_{a,t} = \begin{bmatrix} c_{t-1} & 0 & 0 \\ 0 & \sigma & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{33.22}$$

The exact shape of $c_{t-1}$ follows the scalar factor branch used by the generic SR-UKF handoff. The crucial point is the three-coordinate law: lagged latent state, state innovation, and observation shock are all present.

The state and raw-observation maps sent to the generic SR-UKF backend are

$$X_\theta(A_t) = \gamma H_{t-1} + U_t, \tag{33.23}$$

$$Z_\theta(A_t) = \beta \exp(X_\theta(A_t)/2) E_t. \tag{33.24}$$

The symbol $Z_\theta(A_t)$ in (33.24) is the raw predicted observation coordinate used by the Gaussian closure; it is not the transformed datum $z_t = \log(y_t^2)$ from (33.10).

For a placed point $a^{(j)} = (h_-^{(j)}, u^{(j)}, e^{(j)})$, the local map derivatives required by (17.4) are

$$D_a X_\theta(a^{(j)}) = \begin{bmatrix} \gamma & 1 & 0 \end{bmatrix}, \qquad \partial_{\theta_\gamma} X_\theta(a^{(j)}) = \varphi(\theta_\gamma) h_-^{(j)}, \qquad \partial_{\theta_\beta} X_\theta(a^{(j)}) = 0, \tag{33.25}$$

$$D_a Z_\theta(a^{(j)}) = \beta \exp(x^{(j)}/2) \begin{bmatrix} \frac{1}{2}\gamma e^{(j)} & \frac{1}{2}e^{(j)} & 1 \end{bmatrix}, \tag{33.26}$$

$$\partial_{\theta_\gamma} Z_\theta(a^{(j)}) = \frac{1}{2}\beta \exp(x^{(j)}/2) e^{(j)} \varphi(\theta_\gamma) h_-^{(j)}, \qquad \partial_{\theta_\beta} Z_\theta(a^{(j)}) = \beta \exp(x^{(j)}/2) e^{(j)}, \tag{33.27}$$

where $x^{(j)} = X_\theta(a^{(j)})$. These are ordinary pointwise derivatives of the declared maps. They do not differentiate a principal matrix square root, an eigensystem, an SVD basis, or an autodiff tape.

The initial stationary branch, when used, is

$$H_0 \sim \mathcal{N}\left(0, \frac{\sigma^2}{1-\gamma^2}\right), \qquad |\gamma| < 1, \tag{33.28}$$

with score handoff derivatives

$$\partial_{\theta_\gamma} m_0 = 0, \qquad \partial_{\theta_\beta} m_0 = 0, \qquad \partial_{\theta_\gamma} \frac{\sigma^2}{1-\gamma^2} = \frac{2\sigma^2\gamma}{(1-\gamma^2)^2}\varphi(\theta_\gamma), \qquad \partial_{\theta_\beta} \frac{\sigma^2}{1-\gamma^2} = 0. \tag{33.29}$$

If the factor-propagating backend consumes the scalar stationary factor $c_0 = \sigma(1-\gamma^2)^{-1/2}$ rather than the variance, the corresponding fixed-$\sigma$ branch derivative is

$$\partial_{\theta_\gamma} c_0 = \sigma\gamma(1-\gamma^2)^{-3/2}\varphi(\theta_\gamma), \qquad \partial_{\theta_\beta} c_0 = 0. \tag{33.30}$$

If a nonstationary initialization is chosen, the initialization law and its derivatives must replace (33.28)– (33.30) explicitly.

Given the placed points from (33.22), the generic SR-UKF backend computes

$$\bar{x}_t, \quad \bar{y}_t, \quad P_{xx,t,\star}, \quad S_{t,\star}, \quad P_{xy,t,\star}$$

from the propagated pairs $(X_\theta(a_t^{(j)}), Z_\theta(a_t^{(j)}))$. The declared one-step surrogate contribution is

$$\widehat{\ell}_t^{\mathrm{rawGC}}(\theta) = -\frac{1}{2}\left[\log(2\pi) + \log S_{t,\star} + (y_t - \bar{y}_t)^2 S_{t,\star}^{-1}\right], \tag{33.31}$$

or its vector-panel analogue when several independent coordinates are combined. Its analytical score is exactly the specialization of (17.12) with $v_t = y_t - \bar{y}_t$, followed by the filtered-state derivative handoff (17.14)– (17.16):

$$\partial_{\theta_i} \sum_{t=1}^{T} \widehat{\ell}_t^{\mathrm{rawGC}}(\theta) = \sum_{t=1}^{T} \partial_{\theta_i} \widehat{\ell}_t^{\mathrm{rawGC}}(\theta), \qquad i \in \{\gamma, \beta\}. \tag{33.32}$$

This is the score of the raw actual-SV Gaussian-closure surrogate scalar. It is not the score of $\ell_T(\theta)$ in (33.13).

A two-coordinate collapsed-process specialization may be used only after this three-coordinate adapter has been audited. The required proof obligation is the scalar Gaussian law equivalence

$$(H_t^-, E_t) = (\gamma H_{t-1} + U_t, E_t) \sim \mathcal{N}\left(\begin{bmatrix}\gamma m_{t-1} \\ 0\end{bmatrix}, \begin{bmatrix}\gamma^2 c_{t-1} c_{t-1}^\top + \sigma^2 & 0 \\ 0 & 1\end{bmatrix}\right), \tag{33.33}$$

together with derivative equivalence for the mean and factor handoff. A collapsed route that cannot demonstrate (33.33) and the matching derivatives remains a separate approximation and cannot be used as the admitted SR-UKF analytical-score route.

| |
|---|
| No exact actual-SV likelihood claim; |
| no same-target transformed-likelihood claim; |
| no GradientTape score admission; |
| no historical SVD/eigenderivative score admission; |
| no strict-SPD principal-root derivative admission. |

(33.34)

The practical consequence is strict. Same-scalar derivative checks, fixed-branch audits, or optimizer/HMC experiments are meaningful only after the value path is a route to (33.13). Internal consistency of a Gaussian-closure scalar does not repair the target mismatch. This is why the current transformed direct-likelihood route and the current Zhao–Cui comparator remain admissible same-target comparators, whereas the previous augmented Gaussian-closure framing does not.

**Reader checkpoint.** What should now be clear: filtering updates a probability law and a scalar normalizer at the same time. Object preserved: the conditional law after normalization. Approximation enters at: the representation of the predictive law, the integral for $Z_t$, or both. Exported to synthesis: any industrial method must say whether it preserves the intended likelihood scalar, merely a different surrogate scalar, or only a moment summary.

**Proposition 33.3** (Predictive score identity)**.** *Let $Z_t(\theta) = \int a_t(x,\theta)\,dx$ with $a_t(x,\theta) = g_\theta(y_t \mid x)p_\theta(x \mid y_{1:t-1})$. If $a_t > 0$ on the posterior support and dominated differentiation holds, then*

$$\nabla_\theta \log Z_t(\theta) = \int \nabla_\theta \log a_t(x,\theta)\, p_\theta(x \mid y_{1:t})\, dx. \tag{33.35}$$

*Proof.* Under dominated differentiation,

$$\nabla_\theta \log Z_t(\theta) = \frac{\nabla_\theta Z_t(\theta)}{Z_t(\theta)} = \frac{\int \nabla_\theta a_t(x,\theta)\,dx}{\int a_t(x,\theta)\,dx}.$$

On the support where $a_t > 0$, $\nabla_\theta a_t = a_t \nabla_\theta \log a_t$. Substitution gives

$$\nabla_\theta \log Z_t(\theta) = \int \nabla_\theta \log a_t(x,\theta) \frac{a_t(x,\theta)}{Z_t(\theta)}\, dx.$$

The last factor is the filtering density in (33.7). Hence (33.35) follows. The identity does not say that a numerical score is correct; it says that any reported score must correspond to the derivative of the same normalizer used in the reported likelihood. □

This proposition is exact but not automatically implementable. A numerical filter supplies an approximate normalizer $\widehat{Z}_t$ and an approximate history-dependent distribution. If value and gradient are not derivatives of the same scalar approximation, a downstream optimizer or HMC sampler is solving a different problem from the one reported by the filter.

## 33.3 Analytical Likelihood-Gradient And Filtering Sensitivities

This section is a project derivation for the dominated discrete-time model in (33.1). It is not a continuous-time adjoint theorem. The PDE and robust-DMZ settings require their own sensitivity or adjoint analysis under the regularity assumptions of the cited sources.

Write

$$p_t^-(x_t) = p_\theta(x_t \mid y_{1:t-1}), \qquad p_t(x_t) = p_\theta(x_t \mid y_{1:t}),$$

and let the parameter-gradient sensitivities be

$$s_t^-(x_t) = \nabla_\theta p_t^-(x_t), \qquad s_t(x_t) = \nabla_\theta p_t(x_t).$$

These sensitivities are vector-valued when $\theta$ is vector-valued; the derivation is componentwise. Differentiating the prediction equation (33.6) under Assumption 33.1 gives

$$s_t^-(x_t) = \int \left[\nabla_\theta f_\theta(x_t \mid x_{t-1}) p_{t-1}(x_{t-1}) + f_\theta(x_t \mid x_{t-1}) s_{t-1}(x_{t-1})\right] dx_{t-1}. \tag{33.36}$$

The first term is the direct parameter effect on the transition density. The second term is the inherited sensitivity of the previous filtering law. If the initial distribution depends on $\theta$, the recursion starts from $s_0(x_0) = \nabla_\theta p_\theta(x_0)$, or from the corresponding post-initial-observation sensitivity if an initial update is used.

The one-step normalizer is

$$Z_t(\theta) = \int g_\theta(y_t \mid x_t) p_t^-(x_t)\, dx_t. \tag{33.37}$$

Differentiating it gives

$$\nabla_\theta Z_t(\theta) = \int \left[\nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t)\right] dx_t. \tag{33.38}$$

Therefore

$$\nabla_\theta \ell_T(\theta) = \sum_{t=1}^{T} \nabla_\theta \log Z_t(\theta) = \sum_{t=1}^{T} \frac{\nabla_\theta Z_t(\theta)}{Z_t(\theta)}. \tag{33.39}$$

It remains to differentiate the normalized update. Define

$$n_t(x_t, \theta) = g_\theta(y_t \mid x_t) p_t^-(x_t), \qquad p_t(x_t) = \frac{n_t(x_t, \theta)}{Z_t(\theta)}.$$

The numerator sensitivity is

$$\nabla_\theta n_t(x_t, \theta) = \nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t).$$

Using the quotient rule and $\nabla_\theta Z_t / Z_t = \nabla_\theta \log Z_t$,

$$s_t(x_t) = \frac{\nabla_\theta g_\theta(y_t \mid x_t) p_t^-(x_t) + g_\theta(y_t \mid x_t) s_t^-(x_t)}{Z_t(\theta)} - p_t(x_t) \nabla_\theta \log Z_t(\theta). \tag{33.40}$$

Integrating (33.40) over $x_t$ gives zero, because a normalized density has derivative of total mass equal to zero. This is the sensitivity analogue of probability conservation.

**Proposition 33.4** (Discrete-time likelihood sensitivity recursion)**.** *Under Assumption 33.1, with differentiable transition and observation densities and finite positive $Z_t(\theta)$, equations (33.36)–(33.40) give an analytical recursion for $\nabla_\theta \ell_T(\theta)$ in the exact discrete-time filter.*

*Proof.* Equation (33.36) is differentiation of the prediction integral. Equation (33.38) is differentiation of the predictive normalizer. Equation (33.39) follows from the likelihood factorization (33.8) and the identity $\nabla \log Z = \nabla Z / Z$ for $Z > 0$. Equation (33.40) is the quotient rule applied to $p_t = n_t / Z_t$. Each step is componentwise in $\theta$ and uses dominated differentiation to exchange gradient and integral. □

This recursion is the analytical object needed by Chapter 39. An automatic-differentiation tape through an approximate filter is a different object unless it differentiates exactly the same scalar displayed to the sampler.

## 33.4 Compact Sensitivity Score For Finite LEDH–PFPF–OT

This section specializes the preceding sensitivity principle to the finite LEDH–PFPF–OT scalar used by the benchmark code. The object is not the exact Kalman likelihood and not the derivative of an ideal infinite-iteration transport solve. It is the derivative of the finite computation actually run: fixed base random numbers, a fixed resampling mask, finitely many LEDH flow steps, finitely many Sinkhorn balancing steps, and the displayed accumulated log normalizers. A partial derivative that treats the transport normalization, scale, Sinkhorn potentials, or transported particles as fixed when they change with $\theta$ is not the score of this finite scalar.

Let $\theta \in \mathbb{R}^p$, and let $r_{0,i}$ and $r_{t,i}$ be the fixed standard-normal base draws used to form the initial particles and transition noise. The compact method carries, at each time $t$,

$$X_t = \{x_{t,i}\}_{i=1}^N, \qquad \lambda_t = \{\lambda_{t,i}\}_{i=1}^N, \qquad \dot{X}_t = \partial_\theta X_t, \qquad \dot{\lambda}_t = \partial_\theta \lambda_t,$$

where $\lambda_t$ are normalized log weights. It does not store the full history of LEDH matrices or transport plans. For a diagonal LGSSM row with

$$A_\theta = \mathrm{diag}(\phi_1, \dots, \phi_d), \qquad Q_\theta = q^2 I, \qquad R_\theta = r^2 I,$$

the fixed-randomness prediction is

$$m_{t,i} = A_\theta x_{t-1,i}, \quad x^-_{t,i} = m_{t,i} + q\, r_{t,i}, \quad \dot{m}_{t,i} = A_\theta \dot{x}_{t-1,i} + (\partial_\theta A_\theta) x_{t-1,i}, \quad \dot{x}^-_{t,i} = \dot{m}_{t,i} + (\partial_\theta q) r_{t,i}. \tag{33.41}$$

These are total derivatives under the fixed base draws. They are not derivatives after redrawing the particles.

For a linear observation $y_t = Hx_t + \epsilon_t$, the LEDH affine map used in the row can be written

$$P_\theta = Q_\theta^{-1}, \qquad S_\theta = R_\theta^{-1}, \qquad \Lambda_\theta = P_\theta + H^\top S_\theta H, \qquad C_\theta = \Lambda_\theta^{-1}, \tag{33.42}$$

$$\mu_{t,i} = C_\theta\{P_\theta m_{t,i} + H^\top S_\theta y_t\}, \quad B_\theta = \mathrm{chol}(C_\theta)\,\mathrm{chol}(Q_\theta)^{-1}, \quad z_{t,i} = \mu_{t,i} + B_\theta(x^-_{t,i} - m_{t,i}). \tag{33.43}$$

Here $Q_\theta$ and $R_\theta$ are positive definite, so $P_\theta$, $S_\theta$, $\Lambda_\theta$, $C_\theta$, the Cholesky factors, and the log determinant below are well defined. If a numerical implementation adds jitter or projects a covariance to a positive-definite matrix, the derivative is the derivative of that jittered or projected finite operation, not of the unjittered formula. The tangent of this map is obtained by differentiating these matrix equations:

$$\dot{P}_\theta = -P_\theta \dot{Q}_\theta P_\theta, \qquad \dot{S}_\theta = -S_\theta \dot{R}_\theta S_\theta, \quad \dot{\Lambda}_\theta = \dot{P}_\theta + H^\top \dot{S}_\theta H, \quad \dot{C}_\theta = -C_\theta \dot{\Lambda}_\theta C_\theta, \tag{33.44}$$

$$\dot{\mu}_{t,i} = \dot{C}_\theta\{P_\theta m_{t,i} + H^\top S_\theta y_t\} + C_\theta\{\dot{P}_\theta m_{t,i} + P_\theta \dot{m}_{t,i} + H^\top \dot{S}_\theta y_t\}, \tag{33.45}$$

$$\dot{z}_{t,i} = \dot{\mu}_{t,i} + \dot{B}_\theta(x^-_{t,i} - m_{t,i}) + B_\theta(\dot{x}^-_{t,i} - \dot{m}_{t,i}). \tag{33.46}$$

For $L = \mathrm{chol}(C_\theta)$, the Cholesky tangent is defined by

$$E = L^{-1}\dot{C}_\theta L^{-\top}, \qquad M_{ij} = \begin{cases} E_{ij}, & i > j, \\ E_{ii}/2, & i = j, \\ 0, & i < j, \end{cases} \qquad \dot{L} = LM.$$

Thus $\dot{B}_\theta = \dot{L}\,\mathrm{chol}(Q_\theta)^{-1} + L\,\partial_\theta\{\mathrm{chol}(Q_\theta)^{-1}\}$.

The log-weight correction is differentiated in the same forward pass. With

$$c_{t,i} = \lambda_{t-1,i} + \log f_\theta(z_{t,i} \mid x_{t-1,i}) + \log g_\theta(y_t \mid z_{t,i}) - \log f_\theta(x^-_{t,i} \mid x_{t-1,i}) + \log|\det B_\theta|,$$

define $a_t = \mathrm{logsumexp}_i\, c_{t,i}$ and $\lambda^+_{t,i} = c_{t,i} - a_t$. The one-step likelihood contribution and normalized-log-weight tangent are

$$\dot{a}_t = \sum_{i=1}^{N} \omega^+_{t,i}\dot{c}_{t,i}, \qquad \dot{\lambda}^+_{t,i} = \dot{c}_{t,i} - \dot{a}_t, \qquad \omega^+_{t,i} = \exp(\lambda^+_{t,i}). \tag{33.47}$$

If an implementation floors normalized weights by replacing $\omega^+_{t,i}$ with $\max(\omega^+_{t,i}, \epsilon_{\mathrm{floor}})$, then the derivative must include that branch: active floored entries have zero tangent for the displayed floored log weight. Ignoring this branch is again a different derivative.

It remains to differentiate the finite Sinkhorn equal-weighting map. Write the streaming softmin used by the code as

$$\mathrm{sm}_\varepsilon(q; K, v) = -\varepsilon \log \sum_j \exp\{v_j - \|q - k_j\|^2/(2\varepsilon)\}. \tag{33.48}$$

For tangents $(\dot{q}, \dot{K}, \dot{v}, \dot{\varepsilon})$, define

$$\pi_j(q) = \frac{\exp\{v_j - \|q - k_j\|^2/(2\varepsilon)\}}{\sum_l \exp\{v_l - \|q - k_l\|^2/(2\varepsilon)\}}.$$

The softmin tangent is

$$\partial\,\mathrm{sm}_\varepsilon = -\dot{\varepsilon}\log\sum_j \exp\{v_j - \|q - k_j\|^2/(2\varepsilon)\} - \varepsilon\sum_j \pi_j(q)\dot{h}_j, \tag{33.49}$$

$$\dot{h}_j = \dot{v}_j - \frac{(q - k_j)^\top(\dot{q} - \dot{k}_j)}{\varepsilon} + \frac{\|q - k_j\|^2}{2\varepsilon^2}\dot{\varepsilon}.$$

This equation is local and can be evaluated in blocks. It does not require a dense $N \times N$ matrix to be stored.

The finite Sinkhorn potentials are a fixed composition of the softmin (33.48). With source log weights $\alpha = \lambda^+_t$, target log weights $\beta = -\log N$, scaled particles $u_i = (z_{t,i} - \bar{z}_t)/s_t$, and deterministic annealing values $\varepsilon_k$, the executed code carries four potentials. Starting from the cold-start softmins, one finite balancing step has the form

$$a^{k+1}_y = \frac{1}{2}\left[a^k_y + \mathrm{sm}_{\varepsilon_k}(u; u, \alpha + b^k_x/\varepsilon_k)\right], \tag{33.50}$$

$$b^{k+1}_x = \frac{1}{2}\left[b^k_x + \mathrm{sm}_{\varepsilon_k}(u; u, \beta + a^k_y/\varepsilon_k)\right], \tag{33.51}$$

$$a^{k+1}_x = \frac{1}{2}\left[a^k_x + \mathrm{sm}_{\varepsilon_k}(u; u, \alpha + a^k_x/\varepsilon_k)\right], \tag{33.52}$$

$$b^{k+1}_y = \frac{1}{2}\left[b^k_y + \mathrm{sm}_{\varepsilon_k}(u; u, \beta + b^k_y/\varepsilon_k)\right]. \tag{33.53}$$

After the declared finite number of balancing steps, the transport application uses $a = \mathrm{sm}_\varepsilon(u; u, \alpha + b_x/\varepsilon)$ and $b = \mathrm{sm}_\varepsilon(u; u, \beta + a_y/\varepsilon)$. The tangent recursion is exactly the

derivative of (33.53), using (33.49) at each finite iteration. The scale and initial annealing value are part of the same derivative:

$$\bar{z}_t = N^{-1}\sum_i z_{t,i}, \qquad u_{t,i} = (z_{t,i} - \bar{z}_t)/s_t, \quad \dot{u}_{t,i} = \frac{\dot{z}_{t,i} - \dot{\bar{z}}_t}{s_t} - \frac{z_{t,i} - \bar{z}_t}{s_t^2}\dot{s}_t, \tag{33.54}$$

$$\varepsilon_0 = \max\{(\max_{i,j} u_{t,ij} - \min_{i,j} u_{t,ij})^2, 10^{-6}\}. \tag{33.55}$$

When the max or min is tied, the implementation must choose and document a subgradient convention. Away from ties, this is the ordinary derivative.

Given final finite potentials $a, b$, the barycentric transport row is

$$\tilde{z}_{t,i} = \sum_j \exp\left\{\frac{a_i + b_j - \|u_i - u_j\|^2/2}{\varepsilon} - \rho_j + \log N + \lambda^+_{t,j}\right\} z_{t,j}, \tag{33.56}$$

where $\rho_j$ is the column log normalizer for the same finite potentials. Its tangent is the direct product rule:

$$\dot{\tilde{z}}_{t,i} = \sum_j T_{ij}\left[\dot{z}_{t,j} + z_{t,j}\left(\dot{\eta}_{ij} - \dot{\rho}_j + \dot{\lambda}^+_{t,j}\right)\right], \tag{33.57}$$

where $T_{ij}$ is the finite transport weight in (33.56) and

$$\eta_{ij} = \frac{a_i + b_j - \|u_i - u_j\|^2/2}{\varepsilon}.$$

The column-normalizer tangent is the softmax average of $\dot{\eta}_{ij}$ over rows $i$. Equation (33.57) is the forward-mode counterpart of a manual VJP. It computes the same total derivative of the same finite Sinkhorn map, but it carries tangents forward in time and therefore does not need the full filtering history.

**Proposition 33.5** (Compact finite LEDH–PFPF–OT score recursion)**.** *Assume $Q_\theta$ and $R_\theta$ are positive definite with the stated dimensions, every jittered or projected matrix used by the finite code remains positive definite, and the parameter point is away from nondifferentiable max, floor, active-resampling, covariance-projection, and Cholesky-boundary ties. Assume also that the finite LEDH and Sinkhorn operations above are executed with fixed base random numbers and a fixed resampling mask. If the filter loop carries $(X_t, \lambda_t, \dot{X}_t, \dot{\lambda}_t)$ and updates the tangents by (33.41)– (33.57), then*

$$\dot{\widehat{\ell}}_T(\theta) = \sum_{t=1}^T \dot{a}_t$$

*is the total derivative of the finite scalar $\widehat{\ell}_T(\theta) = \sum_{t=1}^T a_t(\theta)$ computed by that same fixed-randomness LEDH–PFPF–OT run.*

*Proof.* Each displayed update is a differentiable finite-dimensional operation away from the stated branch boundaries: matrix inverse, Cholesky, affine maps, Gaussian log densities, logsumexp normalization, softmin, a fixed finite composition of Sinkhorn updates, column log normalization, and barycentric transport. Equations (33.41)– (33.57) are the chain rule applied to this finite composition in the same order as the value computation. The scalar reported by the filter is the sum of the log normalizers $a_t$. Therefore the carried scalar tangent is the sum of their tangents, which is (33.5). No reverse-time cotangent or stored history is required for this forward-mode proof. □

The memory difference is simple. The full-history reverse route stores time-indexed LEDH auxiliaries, transported states, and weights before scanning backward. The compact sensitivity route stores only the current state and its $p$ parameter tangents. Its leading state memory is

$$O(BNDp + BNp + D^2p)$$

plus local streaming blocks for Sinkhorn, rather than a time-multiplied history. This is the right route for a small parameter dimension such as the LGSSM row. For very large $p$, a checkpointed adjoint may be cheaper, but it must still differentiate the same finite scalar.

**Reader checkpoint.** What should now be clear: the likelihood gradient is not a separate numerical decoration. It is the derivative of the same normalizers that define $\ell_T$. Object preserved: scalar value-gradient consistency. Approximation enters at: any approximate $p_t^-$, $\widehat{Z}_t$, truncation, rounding, or adaptive gate. Exported to synthesis: HMC can start only after the scalar and gradient contract is named.

## 33.5 Zakai, Kushner–Stratonovich, And DMZ Equations

For diffusion models, the normalized conditional distribution satisfies a nonlinear filtering equation, while the unnormalized conditional measure satisfies a linear Zakai equation. We use van Handel [2007] as the standard checked derivation anchor for the normalized and unnormalized equations rather than relying on unchecked historical priority papers. In an informal density notation, the unnormalized density $\sigma_t$ obeys a linear stochastic equation of the form

$$d\sigma_t = \mathcal{L}_\theta^* \sigma_t \, dt + h_\theta^\top \sigma_t \, dY_t, \tag{33.58}$$

where $\mathcal{L}_\theta^*$ is the Fokker–Planck adjoint associated with the signal generator. The normalized density is

$$\rho_t(x) = \frac{\sigma_t(x)}{\int \sigma_t(u)\, du}. \tag{33.59}$$

The Duncan–Mortensen–Zakai viewpoint emphasizes the density/PDE form of (33.58). It is attractive in high dimension because it turns filtering into density propagation; it is frightening for exactly the same reason, since a tensor-product grid has exponentially many degrees of freedom.

**Remark 33.1** (Bibliographic scope)**.** *For the equations used below, this chapter relies on checked technical sources: van Handel for the standard normalized and unnormalized filtering equations, and Davis/Yau papers for the pathwise robust DMZ transformation and memoryless algorithm. Classical priority names are retained for orientation, but a historical citation is not used as a substitute for an inspected theorem or derivation.*

## 33.6 Pathwise Robust DMZ And Memoryless Computation

Davis [1980] studies a multiplicative functional transformation for nonlinear filtering. In the simple scalar-observation diffusion setting, the idea is to absorb observation-path

dependence into a path-dependent transformation of the unnormalized density. The transformed object evolves according to a generator/PDE that can be studied pathwise rather than only as a stochastic differential equation.

Yau and Yau [2000] and Yau and Yau [2008] build a robust DMZ algorithm around this principle. Their central computational move is to freeze observation increments on a time partition and repeatedly solve observation-independent Kolmogorov-type equations, correcting by observation-dependent multiplicative factors. In schematic form, over $[t_j, t_{j+1}]$ one computes

$$\partial_s u_j(s,x) = \mathcal{K}_{y_{t_j}} u_j(s,x), \qquad s \in [t_j, t_{j+1}], \tag{33.60}$$

$$u_{j+1}(t_{j+1},x) = \mathcal{M}(y_{t_{j+1}} - y_{t_j}, x)\, u_j(t_{j+1},x), \tag{33.61}$$

where $\mathcal{K}_{y_{t_j}}$ and $\mathcal{M}$ denote the paper-specific frozen operator and multiplicative observation update. The details are not optional: the convergence and bounded-domain approximation theorems in Yau and Yau [2000, 2008] depend on their growth, regularity, weak-solution, and boundary assumptions.

**Algorithm 11** Robust DMZ offline/online template

**Require:** Partition $0 = t_0 < \cdots < t_K = T$, initial density, checked PR-DMZ assumptions, spatial domain or truncation rule.

1: Build or choose the observation-independent Kolmogorov solver required by the cited theorem.
2: **for** $j = 0, \ldots, K-1$ **do**
3: Freeze the observation path according to the cited rule.
4: Solve the frozen Kolmogorov equation on $[t_j, t_{j+1}]$.
5: Apply the multiplicative observation correction.
6: Normalize only after preserving the unnormalized-density semantics.
7: Record mass, positivity, boundary, and discretization residuals.
8: **end for**

Meng et al. [2025] is a recent arXiv contribution rather than a settled classical reference. It is therefore used here provisionally: it links pathwise robust DMZ regularity estimates to sparse functional tensor-train approximations and a finite-difference QTT algorithm. The durable lesson for the rest of the monograph is methodological rather than evidentiary: a tensor filter must make regularity, rank/compressibility, discretization error, rounding error, and online-assimilation stability visible before small storage can be interpreted as a filtering result.

## 33.7 Approximate Targets And The HMC Boundary

For a prior $\pi(\theta)$, the exact parameter posterior is proportional to

$$\pi(\theta \mid y_{1:T}) \propto \exp\{\ell_T(\theta)\}\pi(\theta). \tag{33.62}$$

If a filter supplies $\widehat{\ell}_T(\theta)$, the posterior implied by the calculation is

$$\widehat{\pi}(\theta \mid y_{1:T}) \propto \exp\{\widehat{\ell}_T(\theta)\}\pi(\theta), \tag{33.63}$$

unless a pseudo-marginal or other exact correction is part of the algorithm. The corresponding gradient is

$$\nabla_\theta \widehat{\ell}_T(\theta) = \nabla_\theta \sum_{t=1}^{T} \log \widehat{Z}_t(\theta),$$

provided the reported gradient is the derivative of the reported scalar. Truncation, resampling, clipping, tensor rounding, adaptive stopping, and discrete accept/reject gates must therefore state which scalar is being differentiated. The HMC chapter returns to this point: exact Hamiltonian dynamics for (33.63) is not exact sampling from (33.62).

## 33.8 Why Dimension Breaks Filters

High dimension is a family of failure mechanisms.

**Particle collapse.**
The bootstrap and SIR filters of Gordon et al. [1993] and Arulampalam et al. [2002] represent the filtering law by particles and weights. Bengtsson et al. [2008] and Snyder et al. [2008] show that, under their assumptions, weight concentration can be governed by the variance of the log likelihood and can require enormous ensembles. These results do not say every structured particle method fails; they say proposal, localization, and correction must be central.

**Grid explosion.**
A density grid with $M$ points per coordinate has $M^n$ entries. Sparse grids, tensor trains, and transport maps are attempts to exploit smoothness, low-order interactions, low rank, or triangular structure. If the posterior lacks that structure, the format fails.

**Gaussian projection bias.**
EKF/UKF/CKF-style filters propagate only selected moments. Multimodality, skewness, sharp likelihoods, and nonlocal curvature can be invisible to the chosen moment rule.

**Sampler geometry.**
HMC and NUTS can fail through divergences, poor energy exploration, invalid regions, or a value/gradient mismatch even when the filter returns finite numbers.

## 33.9 Defects Passed To Synthesis

This chapter passes five defects to the synthesis chapter.

(1) The exact object is a sequence of probability laws and normalizers. A later method that returns a mean but not a defensible $Z_t$ has not supplied a likelihood target for parameter inference.

(2) The approximate posterior in (33.63) is a different target unless an exact correction is part of the method. This is the first same-scalar obligation inherited by HMC.

(3) Analytical score recursions such as (33.39) are available in the dominated discrete-time setting. Continuous-time/PDE score and adjoint formulas are not derived here and remain source-specific obligations.

(4) Density/PDE formulations require mass, positivity, boundary, and regularity checks. A low residual in a transformed PDE does not by itself prove that the normalized density is a valid filtering law.

(5) Source-local PDE assumptions matter. The Davis and Yau robust-DMZ transformations and the Meng tensor approximation results support the statements made under their hypotheses; they do not license an unrestricted high-dimensional filtering theorem for BayesFilter.

The synthesis chapter uses these defects as gates: exact target first, approximation target second, sampler or acceleration only downstream.

**Exported to synthesis.**

| Object | Exact target | Approximation site | Failure mode | Diagnostic/veto | Cost variable | Promotion gate | Citation/derivation anchor |
|---|---|---|---|---|---|---|---|
| Conditional law | $p_\theta(x_t \mid y_{1:t})$ | density, particles, moments, tensor, or map | mean-only or invalid law | mass, positivity, reference moments | state dimension | law semantics named | (33.6)–(33.7) |
| Normalizer | $Z_t = p(y_t \mid y_{1:t-1})$ | quadrature, Monte Carlo, tensor/PDE solve | wrong likelihood | finite positive $Z_t$, scalar parity | integral cost | scalar likelihood reported | (33.7)–(33.8) |
| Likelihood gradient | $\nabla_\theta \ell_T$ | sensitivity approximation or autodiff path | gradient of a different scalar | finite-difference parity, same-scalar check | filter-gradient cost | value and gradient match | Proposition 33.4 |
| DMZ/PDE density | unnormalized filtering density | spatial discretization, operator compression | mass, boundary, or regularity drift | residual, mass, positivity, boundary checks | grid/rank | source assumptions checked | Davis [1980], Yau and Yau [2000, 2008] |

## 33.10 Methodological Boundary

The chapters that follow are literature-grounded design chapters, not production promotion notes. A numerical smoke test can show that a command ran, values were finite, shapes matched, or a stop rule triggered. It cannot establish posterior accuracy, NAWM readiness, tensor validation, HMC convergence, GPU/XLA readiness, or a default production policy.

## 33.11 Bibliographic Limits

Several historical or specialized sources were not available for direct technical inspection in this pass: Savostyanov-specific maxvol/quasioptimality, Stroud's book, Smolyak's original sparse-grid construction, Genz-specific cubature foundations, and Knothe's original rearrangement source. When later chapters use successor or alternative sources, their claims are restricted to what those inspected sources actually state.

# Chapter 34

# Deterministic Gaussian Projection and Point-Rule Foundations

## 34.1 Introduction And Scope

High-dimensional nonlinear filtering forces a choice of computational object. The exact Bayesian filter propagates a full conditional law through time, but in nonlinear models that law usually does not remain available in a finite closed form. In low dimension one may still tolerate heavy deterministic quadrature or a richer non-Gaussian representation. In high dimension, however, both cost and structural complexity escalate quickly. This chapter studies one narrower lane: a deterministic Gaussian-surrogate filter that carries a single Gaussian object, uses explicit moment engines to approximate the nonlinear expectations it needs, and prepares the rule-family language that the next chapter will specialize to fixed sparse-grid quadrature.

This chapter imports the exact-target and same-scalar discipline from Chapter 33. Its task is not yet to build the full sparse-grid algorithm. It first fixes the deterministic Gaussian carried object, the Gaussian projection contract, and the point-rule family comparisons needed before sparse-grid specialization. The next chapter, Chapter 35, then turns this shared Gaussian-rule foundation into the explicit fixed-cloud sparse-grid lane.

The running quadratic-observation cell from Chapter 33 motivates the same Gaussian-projection questions studied here, and it reappears concretely in the next chapter's worked scalar example; the present chapter stays at the level of Gaussian carried objects and deterministic moment engines. Low-dimensional cloud construction, value recursion, and the fixed-cloud scalar belong to the sparse-grid chapter that follows.

The central limitation should be fixed at the outset. A deterministic Gaussian lane does not claim exact nonlinear filtering, exact nonlinear likelihood evaluation, or faithful preservation of arbitrary multimodality, skewness, or high-order global interaction structure. Its purpose is narrower: to ask whether one carried Gaussian, together with a declared deterministic moment engine, can provide a coherent and scientifically useful approximate likelihood lane in the regime under study.

**Background assumed.** The reader is assumed to know multivariate Gaussian notation, state-space-model notation, Jacobian-style derivative notation, and the use of a Cholesky factor for a positive-definite covariance matrix. Sparse-grid merging, same-scalar branch identities, and fixed-cloud derivative obligations are defined where they

first become necessary.

### 34.1.1 Problem Setting And Deterministic Gaussian Lanes

The scientific problem is high-dimensional nonlinear filtering. At time $t$, the exact Bayesian filter propagates and updates the full conditional law $p_\theta(x_t \mid y_{1:t})$. In nonlinear models that law is generally not available in a finite-dimensional closed form, and in high dimension it becomes unrealistic to treat exact function-valued propagation as the practical computational object. The deterministic Gaussian family therefore does not attempt to rescue exactness by notation. It selects one approximate lane whose strengths and boundaries can both be stated explicitly.

Within this family, the carried object is a single Gaussian surrogate $\mathcal{N}(m_t, P_t)$. Different deterministic point-rule lanes then differ not in what object is carried, but in how the nonlinear Gaussian moments needed by the projection are approximated. This chapter fixes that shared carried-object and moment-contract language before the next chapter specializes it to the fixed sparse-grid lane.

### 34.1.2 The Deterministic Gaussian Lane In One Page

The deterministic Gaussian family studied here is defined by two deliberate approximations and one downstream derivative constraint.

1. **Gaussian carried state.** Instead of carrying the full filtering law, the method carries one Gaussian summary $\mathcal{N}(m_t, P_t)$.

2. **Deterministic moment engine.** The nonlinear Gaussian moments that define the prediction and observation projection are approximated by a fixed point-rule family in standardized coordinates.

3. **Same-scalar derivative discipline.** Any later reported gradient must be the derivative of one declared approximate scalar built from that same deterministic moment engine.

The exact object, the carried surrogate, and the eventual approximate scalar are therefore different in kind:

$$\begin{aligned} \text{exact object:} &\quad p_\theta(x_t \mid y_{1:t}), \\ \text{carried surrogate:} &\quad \mathcal{N}(m_t, P_t), \\ \text{declared approximate scalar:} &\quad \widehat{\ell}_T^{(M)}(\theta) \text{ for a chosen deterministic moment engine } M. \end{aligned} \tag{34.1}$$

This distinction is the governing contract of the deterministic Gaussian block. The next chapter will instantiate one specific engine, fixed sparse-grid quadrature, within this larger family.

### 34.1.3 Approximation Hierarchy And Reading Guide

The deterministic Gaussian side of the expanded block unfolds in two chapter movements. First, the exact filtering recursion and the Gaussian projection that replaces it are fixed formally. Second, nearby deterministic point-rule families are compared at the level of their moment engines. Only after that shared foundation is explicit does the next chapter

specialize to sparse-grid construction, merged clouds, the filtering value path, and the fixed-cloud scalar.

The main coordinates are:

$$\xi \in \mathbb{R}^{n_x}, \qquad \xi \sim \mathcal{N}(0, I_{n_x}) \quad \text{standard quadrature coordinate,} \tag{34.2}$$

$$x = m + C\xi, \quad CC^\top = P \quad \text{physical state coordinate under a carried Gaussian,} \tag{34.3}$$

$$a = f_\theta(x), \quad \text{transition image used for prediction,} \tag{34.4}$$

$$z = h_\theta(x), \quad \text{observation image used for the innovation.} \tag{34.5}$$

A deterministic point-rule lane builds a finite rule in $\xi$-space and then places that rule into physical state space through the current Gaussian factor. That sentence is the bridge from the present foundations chapter to the sparse-grid chapter that follows.

## 34.2 Exact Filtering And Gaussian Projection

Use the additive-noise state-space model

$$X_t = f_\theta(X_{t-1}) + \eta_t, \qquad \eta_t \sim \mathcal{N}(0, Q_\theta), \tag{34.6}$$

$$Y_t = h_\theta(X_t) + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \tag{34.7}$$

with $X_t \in \mathbb{R}^{n_x}$, $Y_t \in \mathbb{R}^{n_y}$, and parameter $\theta \in \mathbb{R}^p$. The exact Bayesian prediction and update are

$$p_\theta(x_t \mid y_{1:t-1}) = \int p_\theta(x_t \mid x_{t-1}) p_\theta(x_{t-1} \mid y_{1:t-1}) \, \mathrm{d}x_{t-1}, \tag{34.8}$$

$$p_\theta(x_t \mid y_{1:t}) = \frac{g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1})}{\int g_\theta(y_t \mid x_t) p_\theta(x_t \mid y_{1:t-1}) \, \mathrm{d}x_t}. \tag{34.9}$$

In nonlinear models, these integrals usually do not close in finite-dimensional form. Deterministic Gaussian filters replace those exact densities by a Gaussian moment projection.

### 34.2.1 Object Map And Deterministic Gaussian Conventions

The deterministic Gaussian family uses the following common objects.

| symbol | meaning | shape |
|---|---|---|
| $m_t, P_t$ | carried Gaussian mean and covariance after update | $n_x$, $n_x \times n_x$ |
| $m_t^-, P_t^-$ | predictive Gaussian mean and covariance before update | $n_x$, $n_x \times n_x$ |
| $C_t, C_t^-$ | covariance factors with $P_t = C_t C_t^\top$, $P_t^- = C_t^-(C_t^-)^\top$ | $n_x \times n_x$ |
| $\chi_t^{(r)}$ | deterministic physical rule point | $n_x$ |
| $z_t^{(r)}$ | observation image of a rule point | $n_y$ |
| $\bar{z}_t, S_t, C_{xz,t}$ | predicted observation mean, innovation covariance, cross-covariance | $n_y$, $n_y \times n_y$, $n_x \times n_y$ |
| $v_t, K_t$ | innovation residual and Gaussian projection gain | $n_y$, $n_x \times n_y$ |

Three implementation conventions are fixed chapter-wide.

1. **Factor convention.** Every covariance factor is the lower triangular Cholesky factor with positive diagonal, written $P = CC^\top$, whenever the declared branch is valid.

2. **Solve convention.** Formulas may use $S_t^{-1}$ for compact mathematics, but the implementation contract is to compute such actions by linear solves against the declared factor of $S_t$, not by forming an explicit inverse.

3. **Noise convention.** Additive process noise enters analytically through $Q_\theta$ in predictive moments and additive observation noise through $R_\theta$ in innovation moments. This chapter does not use state augmentation for these additive noises.

### Gaussian Projection From Moments

Assume the predictive state is represented by

$$X_t^- \sim \mathcal{N}(m_t^-, P_t^-). \tag{34.10}$$

Let

$$Z_t = h_\theta(X_t^-) \tag{34.11}$$

be the noiseless predicted observation. The Gaussian projection needs

$$\bar{z}_t = \mathbb{E}[Z_t], \tag{34.12}$$
$$S_t = R_\theta + \mathbb{E}[(Z_t - \bar{z}_t)(Z_t - \bar{z}_t)^\top], \tag{34.13}$$
$$C_{xz,t} = \mathbb{E}[(X_t^- - m_t^-)(Z_t - \bar{z}_t)^\top]. \tag{34.14}$$

Given these moments, define

$$v_t = y_t - \bar{z}_t, \qquad K_t = C_{xz,t} S_t^{-1}, \tag{34.15}$$
$$m_t = m_t^- + K_t v_t, \qquad P_t = P_t^- - K_t S_t K_t^\top. \tag{34.16}$$

The approximation has two layers. First, these moments may be approximated by a deterministic point rule. Second, the resulting posterior information is compressed back to one Gaussian.

### How This Matches The Jia–Xin–Cheng Gaussian Approximation Block

Jia, Xin, and Cheng first write the Gaussian approximation filter before they introduce sparse grids. In their additive-noise setting, the sparse-grid part of the method answers only how to approximate the Gaussian expectations needed for $m_t^-$, $P_t^-$, $\bar{z}_t$, $S_t$, and $C_{xz,t}$. It does not change the fact that the filter stores one Gaussian pair $(m_t, P_t)$.

In BayesFilter notation, that bridge has the generic form

$$\mathbb{E}[F(\xi)] \quad \rightsquigarrow \quad \mathcal{Q}_M[F] \quad \rightsquigarrow \quad \sum_r w_r^{(M)} F(\xi_r^{(M)}), \tag{34.17}$$

where $M$ denotes the chosen deterministic point-rule family. The next chapter specializes this generic expectation-to-rule replacement to the fixed sparse-grid cloud and its stored scalar contract.

## 34.3 Deterministic Point-Rule Families

A deterministic Gaussian-surrogate lane is defined not only by the carried Gaussian object but also by the moment engine it uses. The key families for the expanded high-dimensional block are:

- local linearization (EKF-style moment closure),
- low-order sigma-point rules (UKF/CKF-style closures),
- dense Gauss–Hermite / tensor-product rules,
- sparse-grid rules that the next chapter will specialize.

The decisive question is therefore not whether all these methods are "Gaussian," but what deterministic moment engine they use and what that engine implies for the approximate scalar that later chapters may differentiate.

The standard sigma-point interface already exists elsewhere in the monograph. Chapter 16 gives the generic sigma-point moment-matching interface, while Chapter 17 clarifies square-root backend contracts. The purpose here is not to reteach all sigma-point filters, but to position the deterministic point-rule family as the shared foundations of the sparse-grid specialization.

**Rule-family comparison discipline.** For a deterministic point rule with nodes $\xi_r^{(M)}$, mean weights $w_{r,m}^{(M)}$, and covariance/cross-covariance weights $w_{r,c}^{(M)}$, the family-level operator has the form

$$\bar{z}_t^{(M)} \approx \sum_r w_{r,m}^{(M)} h_\theta(m_t^- + C_t^- \xi_r^{(M)}), \tag{34.18}$$

$$S_t^{(M)} \approx R_\theta + \sum_r w_{r,c}^{(M)} \Delta_{t,r}^{(M)} \Delta_{t,r}^{(M)\top}, \tag{34.19}$$

with the matching cross-covariance operator defined analogously. The one-weight specialization covers symmetric rules such as CKF-like or fixed sparse-grid clouds. Standard scaled-UKF comparisons generally require distinct mean and covariance weights, so rule-family comparisons must preserve that distinction.

## 34.4 Exports To Sparse-Grid Specialization And Later Chapters

This chapter exports four things forward.

(1) the exact-target versus Gaussian-carried-object distinction,

(2) the Gaussian projection contract and its innovation objects,

(3) the deterministic point-rule family language needed before sparse-grid specialization,

(4) the rule-family comparison discipline needed later when sparse-grid, low-order sigma-point, and dense GHQ lanes are compared.

The next chapter, Chapter 35, begins where this one stops: it instantiates one specific deterministic point-rule lane, fixed sparse-grid quadrature, and develops the low-dimensional cloud geometry, Smolyak structure, filtering value path, and fixed-cloud scalar that this shared foundations chapter has intentionally deferred.

# Chapter 35

# Sparse-Grid Quadrature and the Fixed-Cloud Scalar

## 35.1 Purpose And Scope

This chapter is the fixed sparse-grid quadrature lane itself. The preceding chapter fixed the deterministic Gaussian carried object, the Gaussian projection contract, and the rule-family comparison language. The present chapter now commits to one specific deterministic moment engine: a fixed, merged sparse-grid cloud in standardized Gaussian coordinates.

The chapter therefore owns four burdens that should stay together:

1. low-dimensional cloud construction from one-dimensional rules,
2. active bands and Smolyak coefficients,
3. duplicate merging and the stored cloud object,
4. the fixed-cloud scalar object exported to the later SGQF continuation, fixed-branch, and validation chapters.

The later continuation chapter, Chapter 36, takes up the forward value path, the one-step numeric oracle, and the SGQF-specific same-branch validation burden. The present chapter stays on cloud construction mechanics so that the reader can understand the sparse-grid object before it is asked to carry a likelihood scalar.

## 35.2 Low-Dimensional Construction Of FixedSGQF

This chapter develops the sparse-grid construction in the order a reader usually needs it. Start from one nonlinear Gaussian moment in one coordinate. Lift that rule to a tensor-product cloud in two coordinates. Then ask how the same construction becomes too expensive in higher dimension and how the sparse-grid combination reduces that cost without changing the underlying standardized-coordinate logic.

### 35.2.1 One nonlinear Gaussian moment as the basic quadrature problem

Start with a scalar Gaussian input and a simple nonlinear observation map. Let

$$X \sim \mathcal{N}(m, P), \qquad h(X) = X^2. \tag{35.1}$$

Suppose we want the Gaussian-projection observation mean

$$\bar{z} = \mathbb{E}[h(X)] = \mathbb{E}[X^2]. \tag{35.2}$$

Write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, 1)$ and $C^2 = P$. Then

$$\bar{z} = \int_{\mathbb{R}} (m + C\xi)^2 \phi(\xi)\, \mathrm{d}\xi. \tag{35.3}$$

Applying the three-point standard-normal rule $\{(-\sqrt{3}, \frac{1}{6}), (0, \frac{2}{3}), (\sqrt{3}, \frac{1}{6})\}$ to this standardized integral gives

$$\frac{1}{6}(m - \sqrt{3}C)^2 + \frac{2}{3}m^2 + \frac{1}{6}(m + \sqrt{3}C)^2 = m^2 + C^2 = m^2 + P, \tag{35.4}$$

so the rule reproduces this quadratic Gaussian moment exactly. Once the state has been standardized, the moment problem becomes a standard-normal weighted integral. The quadrature rule is therefore a numerical approximation to a Gaussian expectation, not an extra modeling layer with a separate target.

### 35.2.2 Tensor-product growth in two dimensions

Now move from one coordinate to two. Let

$$X \sim \mathcal{N}(m, P), \qquad X \in \mathbb{R}^2, \tag{35.5}$$

and write $X = m + C\xi$ with $\xi \sim \mathcal{N}(0, I_2)$. If each standardized coordinate uses the three-point rule $\{-\sqrt{3}, 0, \sqrt{3}\}$, then the direct tensor product uses all pairs

$$(\xi_1, \xi_2) \in \{-\sqrt{3}, 0, \sqrt{3}\} \times \{-\sqrt{3}, 0, \sqrt{3}\}, \tag{35.6}$$

so already in two dimensions the rule has $3^2 = 9$ points, while in three dimensions it has $3^3 = 27$ points. Sparse-grid construction tries to keep the same standardized-coordinate logic while avoiding the full tensor explosion.

### 35.2.3 One-Dimensional Rules And Their Tensor Products

A one-dimensional standard-normal quadrature rule at level $\ell$ is a weighted sum

$$I_\ell[\psi] = \sum_{a=1}^{s_\ell} \omega_{\ell,a} \psi(\zeta_{\ell,a}) \approx \int_{\mathbb{R}} \psi(\xi)\phi(\xi)\, \mathrm{d}\xi. \tag{35.7}$$

The chapter fixes the odd-point standard-normal GHQ ladder as its concrete one-dimensional family:

$$I_1 = \text{1-point standard-normal GHQ}, \qquad I_2 = \text{3-point standard-normal GHQ}, \qquad I_3 = \text{5-point standar} \tag{35.8}$$

which is stronger than the source theorem's minimal exactness requirement and is particularly useful for transparent 1D/2D/3D teaching examples.

The tensor rule attached to a multi-index $\mathbf{i} = (i_1, \dots, i_b)$ is

$$I_{\mathbf{i}}[F] = (I_{i_1} \otimes \cdots \otimes I_{i_b})[F]. \tag{35.9}$$

This is simply the Cartesian-product lift of the one-dimensional rules.

### 35.2.4 Three-dimensional preview: why sparse grids help

The 3D preview is the first place where sparse grids become visibly different from dense tensor rules. With the 3-point GHQ family in each coordinate, the dense rule has 27 points. The low sparse-grid level keeps only the tensor rules whose total level lies in a narrow active band, then combines them with signed Smolyak coefficients and merges repeated nodes. The practical message is that sparse grids are not "drop some points from a tensor product." They are a signed combination of tensor rules followed by duplicate accumulation into one stored cloud.

**A fully worked two-dimensional tensor example.** Take $b = 2$ and use

$$I_1 : (0, 1), \qquad I_2 : \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{35.10}$$

Then the tensor rules $(1, 1)$, $(1, 2)$, and $(2, 1)$ are completely explicit:

$$(1,1): \ (0,0) \text{ with weight } 1, \tag{35.11}$$

$$(1,2): \ (0,-\sqrt{3}), (0,0), (0,\sqrt{3}) \text{ with weights } \frac{1}{6}, \frac{2}{3}, \frac{1}{6}, \tag{35.12}$$

$$(2,1): \ (-\sqrt{3},0), (0,0), (\sqrt{3},0) \text{ with weights } \frac{1}{6}, \frac{2}{3}, \frac{1}{6}. \tag{35.13}$$

So the sparse-grid level-2 band in two dimensions already shows the key pattern: one origin contribution from $(1, 1)$ and two axis triples from $(1, 2)$ and $(2, 1)$.

**A concrete 3D sparse-grid preview.** For $b = 3$ and $L = 2$, the active band is not the full tensor rule $(2, 2, 2)$. Instead it contains

$$(1,1,1), \qquad (1,1,2), \qquad (1,2,1), \qquad (2,1,1), \tag{35.14}$$

with coefficients

$$c_{(1,1,1)} = -2, \qquad c_{(1,1,2)} = 1, \qquad c_{(1,2,1)} = 1, \qquad c_{(2,1,1)} = 1. \tag{35.15}$$

Before duplicate merging, those four tensor rules contribute

$$1 + 3 + 3 + 3 = 10 \tag{35.16}$$

raw node-weight contributions, not 27. After merging, the origin contribution from $(1, 1, 1)$ and the three level-mixed rules accumulates to

$$-2 + \frac{2}{3} + \frac{2}{3} + \frac{2}{3} = 0, \tag{35.17}$$

so the center cancels completely, leaving the six axis points

$$(\pm\sqrt{3},0,0),\ (0,\pm\sqrt{3},0),\ (0,0,\pm\sqrt{3}), \tag{35.18}$$

each with weight $1/6$. This is the concrete sense in which a low sparse-grid rule can collapse a 27-point dense picture into a 6-point stored cloud while still preserving selected low-order structure.

## 35.3 Sparse-Grid Rule Reconstructed In Source Order

For a declared sparse-grid level $L$, the active band may be written as

$$\mathcal{A}_{b,L} = \{\mathbf{i} \in \mathbb{N}^b : L \le |\mathbf{i}| \le L + b - 1\}, \qquad |\mathbf{i}| = \sum_{j=1}^{b} i_j. \tag{35.19}$$

This notation is the key to the low-dimensional examples: for $b = 2, L = 2$, the active band gives $(1,1),(1,2),(2,1)$, not $(2,2)$. For $b = 3, L = 2$, it gives the four tensor rules that build the low-level 3D sparse-grid preview.

The Smolyak coefficient attached to $\mathbf{i}$ is

$$c_{\mathbf{i}} = (-1)^{L+b-1-|\mathbf{i}|}\binom{b-1}{L+b-1-|\mathbf{i}|}. \tag{35.20}$$

Hence the sparse-grid quadrature approximation is

$$A_{b,L}[F] = \sum_{\mathbf{i}\in\mathcal{A}_{b,L}} c_{\mathbf{i}} I_{\mathbf{i}}[F]. \tag{35.21}$$

This coefficient can be negative. That fact is central: it is why later covariance and innovation objects must be checked rather than assumed safe just because each component tensor rule individually looks Gaussian-friendly.

### 35.3.1 From Formula To Point Cloud

The runtime object used later by the filter is not a symbolic Smolyak expression and not a raw tensor rule. It is the merged, sorted list of standardized nodes and accumulated signed weights:

$$\mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}}. \tag{35.22}$$

That means the cloud-construction contract must specify:

1. the one-dimensional rule family,
2. the active band and Smolyak coefficients,
3. duplicate-merge tolerance,
4. zero-weight pruning rule,
5. deterministic node ordering.

If two implementations use the same nominal level $L$ but merge nearly equal nodes differently, they have not necessarily evaluated the same scalar.

**Raw-to-merged 2D cloud walkthrough.** For $(b, L) = (2, 2)$, the active indices are $(1, 1)$, $(1, 2)$, and $(2, 1)$ with coefficients $-1, +1, +1$. Before merging, the origin appears three times:

$$(0, 0) \text{ from } (1, 1), \qquad (0, 0) \text{ from } (1, 2), \qquad (0, 0) \text{ from } (2, 1). \tag{35.23}$$

Its accumulated signed weight is

$$-1 + \frac{2}{3} + \frac{2}{3} = \frac{1}{3}, \tag{35.24}$$

while the four axis nodes each receive weight $1/6$. So the final merged five-point cloud is

$$\{(0, 0), (\pm\sqrt{3}, 0), (0, \pm\sqrt{3})\} \quad \text{with weights} \quad \left\{\frac{1}{3}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}\right\}. \tag{35.25}$$

This is the minimal nontrivial example showing why the stored object is a merged cloud rather than a symbolic list of tensor rules.

**Raw-to-merged 3D cloud walkthrough.** For $(b, L) = (3, 2)$, the active indices from (35.14) contribute ten raw node-weight terms in total. Seven distinct node locations appear before zero-weight pruning: the origin and the six axis nodes $(\pm\sqrt{3}, 0, 0)$, $(0, \pm\sqrt{3}, 0)$, $(0, 0, \pm\sqrt{3})$. The origin weight cancels by (35.17), leaving six stored nodes after pruning. So the bookkeeping path is

$$10 \text{ raw contributions} \longrightarrow 7 \text{ merged node locations} \longrightarrow 6 \text{ stored nodes after zero-weight pruning.} \tag{35.26}$$

This concrete 3D sparse-grid reduction replaces the 27-point dense tensor picture by a six-point merged cloud because the low-level signed combination preserves the selected low-order axis structure of the declared Gaussian-moment problem. To see what is being approximated, take the explicit test function

$$F(\xi) = \xi_1^2 + \xi_1\xi_2 + \xi_3. \tag{35.27}$$

Every stored node has exactly one nonzero coordinate, so the cross term and the linear term both average to zero on the cloud, while the quadratic term contributes $\frac{1}{6}\cdot 3 + \frac{1}{6}\cdot 3 = 1$. The six-point cloud therefore reproduces the exact standard-normal value $\mathbb{E}[F(\xi)] = 1$ for this declared 3D example.

**Deterministic merge / prune / order policy.** A fixed-cloud builder routine should execute the following deterministic policy:

1. enumerate every active tensor rule and every raw tensor node in a fixed, lexicographic order,
2. for each raw node $\xi^{\mathbf{i},\alpha}$, look for an existing stored node $\xi$ satisfying

$$\|\xi^{\mathbf{i},\alpha} - \xi\|_\infty \le \tau_{\text{merge}} := 10^{-12} \max\{1, \|\xi\|_\infty\}, \tag{35.28}$$

3. if such a node exists, accumulate the signed weight $c_{\mathbf{i}} w^{\mathbf{i},\alpha}$; otherwise add a new dictionary entry,

4. remove entries whose accumulated absolute weight is below the declared numerical-zero threshold,

5. sort the surviving nodes lexicographically and store the final cloud in that order.

For the fixed BayesFilter scalar, the cloud $\mathcal{C}_{b,L}$, the univariate rules, the merge tolerance, the zero-weight rule, the sorting convention, and the accumulated signed weights are all part of the target. If two implementations use the same nominal level but merge nearly equal floating-point nodes differently, they have not necessarily evaluated the same scalar.

**Implementation-facing cloud-builder contract.** The runtime object produced by this section is a stored cloud builder routine of the form `build_fixed_sparse_grid_cloud(`$(b, L, \{I_\ell\}, \tau_{\mathrm{me}}$ that returns the sorted merged cloud $\mathcal{C}_{b,L}$, its weight-total diagnostic, and any signed-weight diagnostics used later in validation. The mathematics above is the authoritative definition of that routine.

### 35.3.2 Exactness, Point Count, And The UKF Relation

The low-level family used above now has an explicit exactness and UKF relation. For $(b, L) = (3, 2)$ with the three-point rule, the merged cloud reproduces every standard-normal moment generated by total-degree-at-most-three monomials and the pure axis quartics $\mathbb{E}[\xi_j^4] = 3$, while it does not reproduce mixed quartics such as $\mathbb{E}[\xi_i^2 \xi_j^2]$ for $i \neq j$; the point count follows the bookkeeping path $10 \to 7 \to 6$ displayed in (35.26). Moreover, for general dimension $b$ at level $L = 2$, the merged cloud is exactly the classical UKF sigma-point set with $\kappa = 3 - b$: the merged origin weight is $1 - b/3 = \kappa/(b + \kappa)$ and each axis node $\pm\sqrt{3}\, e_j$ carries weight $1/6 = 1/(2(b + \kappa))$. This identity is explanatory. It does not justify identifying sparse-grid filtering with UKF filtering in general, and it does not remove Gaussian projection error.

## 35.4 What The Fixed Cloud Exports Forward

This construction chapter now hands the SGQF continuation chapter a fully specified sparse-grid object:

$$\begin{aligned}
&\text{stored cloud: } \mathcal{C}_{b,L} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M_{b,L}},\\
&\text{construction metadata: } \{\text{rule family, } \mathcal{A}_{b,L},\ c_{\mathbf{i}}, \text{merge tolerance, zero-weight rule, node ordering}\},\\
&\text{declared downstream role: } \mathcal{C}_{b,L} \text{ is the fixed quadrature object from which the SGQF scalar is built.}
\end{aligned} \tag{35.29}$$

The next chapter, Chapter 36, uses this already-fixed cloud to define the forward value path, the one-step numeric oracle, and the lane-specific same-branch validation contract.

## 35.5 Methodological Boundary And Non-Claims

The sparse-grid lane should be read as a deterministic Gaussian-surrogate method with a declared fixed cloud, not as a full non-Gaussian posterior method. Its strength is transparency: the cloud, signed weights, and construction metadata are explicit. Its limitation is equally explicit: one Gaussian carried object can still lose multimodality,

strong asymmetry, and broad interaction structure even when the sparse-grid moment engine itself is accurate for the Gaussian moments it is asked to approximate.

Exported forward:

- Chapter 36 takes up the value-path, numeric-oracle, and SGQF validation burden on the fixed cloud defined here.
- Chapter 37 takes up richer retained-object alternatives when one Gaussian is too narrow.
- Chapter 39 may import the fixed-cloud scalar and branch-defining sparse-grid metadata only after the SGQF continuation chapter has stated them explicitly.
- Chapter 40 may import only the declared Gaussian-projection defect, point-explosion / active-dimension defect, and sparse-grid scalar contract stated across this chapter and its continuation.

# Chapter 36

# Fixed-Cloud Filtering, Same-Branch SGQF, and Lane-Specific Validation

## 36.1 Purpose And Scope

The previous chapter fixed the sparse-grid object itself: the one-dimensional rule family, the active band, the signed Smolyak combination, and the merged cloud stored in standardized Gaussian coordinates. That was the construction burden. The present chapter takes up the burden that follows once the cloud has already been fixed: what scalar the fixed cloud computes, how the forward value path is assembled, what counts as the same branch for this lane, and what validation burden must be discharged before the sparse-grid surrogate can be promoted beyond a pedagogical or exploratory object.

This split is deliberate. A reader first needs to understand how the cloud is built. Only after that does it become useful to ask which one-step scalar the cloud defines, which branch metadata must stay fixed when differentiating that scalar, and which diagnostics certify that a BayesFilter implementation is still evaluating the same declared object rather than a nearby adaptive variant.

For actual-SV rows, this chapter must now be read with a stricter boundary. The fixed-cloud construction below defines a deterministic Gaussian-projection scalar: the cloud approximates predictive and observation moments, and the resulting one-step scalar is the Gaussian innovation log likelihood in (36.13). That construction is mathematically valid as a Gaussian-projection filter description, but the previous actual-SV "Lane B" framing was wrong for the intended inference target. For transformed actual SV, the intended one-step normalizer comes from the exact transformed observation law, so a route that replaces that law by Gaussian closure is not a co-equal lane of the same problem. The fixed-cloud machinery remains useful only when the scalar it differentiates is already the right target.

## 36.2 Filtering Value Path On A Fixed Cloud

Fix a standardized sparse-grid cloud

$$\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M}, \qquad \xi^{(r)} \in \mathbb{R}^{n_x}, \quad w_r \in \mathbb{R}. \tag{36.1}$$

and freeze that cloud before likelihood evaluation. The sparse-grid value path then has two quadrature stages at each time: prediction and observation.

### 36.2.1 Prediction Stage

Let $P_{t-1} = C_{t-1}C_{t-1}^\top$ on the declared square-root branch. Place the previous Gaussian cloud by

$$\chi_{t-1}^{(r)} = m_{t-1} + C_{t-1}\xi^{(r)}, \tag{36.2}$$

push points through the transition,

$$a_t^{(r)} = f_\theta(\chi_{t-1}^{(r)}), \tag{36.3}$$

and form

$$m_t^- = \sum_{r=1}^{M} w_r a_t^{(r)}, \tag{36.4}$$

$$P_t^- = Q_\theta + \sum_{r=1}^{M} w_r (a_t^{(r)} - m_t^-)(a_t^{(r)} - m_t^-)^\top. \tag{36.5}$$

After (36.5), BayesFilter symmetrizes $P_t^-$ and checks the declared positive-definite branch. For the same-branch lane studied here there is no adaptive jitter, rank switch, node rewrite, or cloud rebuild inside the scalar. If the predictive branch fails, the value path returns a veto instead of silently changing the object being evaluated.

### 36.2.2 Observation Stage

Factor $P_t^- = C_t^-(C_t^-)^\top$ on the declared branch and place the predictive cloud:

$$\chi_t^{(r)} = m_t^- + C_t^-\xi^{(r)}, \tag{36.6}$$

Propagate through the observation map,

$$z_t^{(r)} = h_\theta(\chi_t^{(r)}), \tag{36.7}$$

and compute

$$\bar{z}_t = \sum_{r=1}^{M} w_r z_t^{(r)}, \tag{36.8}$$

$$\Delta_t^{(r)} = z_t^{(r)} - \bar{z}_t, \tag{36.9}$$

$$S_t = R_\theta + \sum_{r=1}^{M} w_r \Delta_t^{(r)} \Delta_t^{(r)\top}, \tag{36.10}$$

$$C_{xz,t} = \sum_{r=1}^{M} w_r (\chi_t^{(r)} - m_t^-)\Delta_t^{(r)\top}. \tag{36.11}$$

The one-step innovation scalar is then

$$v_t = y_t - \bar{z}_t, \tag{36.12}$$

$$\widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log\det S_t + v_t^\top S_t^{-1} v_t + n_y \log(2\pi)\right\}. \tag{36.13}$$

This is the one-step scalar exported by the Gaussian-projection value path. It is defined by predictive and observation moments $\bar{z}_t$, $S_t$, and $C_{xz,t}$, not by direct quadrature of the exact normalizer $Z_t(\theta)$. That distinction is decisive for actual SV: if the intended observation law is the exact transformed law $z_t = \log(y_t^2), z_t - \log(\beta^2) - h_t \sim \log(\chi_1^2)$, then the fixed cloud can support the intended likelihood only inside a same-target direct reweighting path whose one-step scalar is the exact transformed-law normalizer; the Gaussian innovation scalar (36.13) is not that normalizer. The older actual-SV use of (36.13) as a standalone Lane-B likelihood was therefore wrong, not merely an alternative convention. If $S_t$ fails the declared branch, the value path returns a veto. If it passes, the Gaussian projection update is

$$K_t = C_{xz,t} S_t^{-1}, \tag{36.14}$$

$$m_t = m_t^- + K_t v_t, \tag{36.15}$$

$$P_t = P_t^- - K_t S_t K_t^\top. \tag{36.16}$$

with the final carried covariance again checked on the declared branch.

**Value-path execution order.** For this lane the runtime order is part of the scalar identity:

1. symmetrize and factor $P_{t-1}$,
2. place previous-state points and propagate through $f_\theta$,
3. form $(m_t^-, P_t^-)$ and veto if the predictive branch fails,
4. factor $P_t^-$, place predictive points, and propagate through $h_\theta$,
5. form $\bar{z}_t$, $S_t$, and $C_{xz,t}$, and veto if the innovation branch fails,
6. form $v_t$, $\widehat{\ell}_t^{\mathrm{FSGQ}}$, $K_t$, $m_t$, and $P_t$, then veto if the carried covariance branch fails.

Any implementation that changes the cloud, node ordering, factor branch, or veto policy in the middle of this sequence is no longer evaluating the same fixed-cloud scalar.

## 36.3 Worked One-Step Example With A Numeric Oracle

Use the one-dimensional transition-observation model

$$X_0 \sim \mathcal{N}(m_0, P_0), \qquad m_0 = 0.5, \qquad P_0 = 1, \tag{36.17}$$

$$X_1 = f_\beta(X_0) + \eta_1, \qquad f_\beta(x) = \beta x, \qquad \eta_1 \sim \mathcal{N}(0, Q), \tag{36.18}$$

$$Y_1 = h_\theta(X_1) + \varepsilon_1, \qquad h_\theta(x) = x^2, \qquad \varepsilon_1 \sim \mathcal{N}(0, R), \tag{36.19}$$

with

$$Q = 0.25, \qquad R = 1, \qquad y_1 = 2. \tag{36.20}$$

Evaluate at $\beta = 1$ on the positive scalar Cholesky branch and use the three-point standard-normal cloud

$$\mathcal{C}_{1,2} = \left\{ \left(-\sqrt{3}, \frac{1}{6}\right), \left(0, \frac{2}{3}\right), \left(\sqrt{3}, \frac{1}{6}\right) \right\}. \tag{36.21}$$

**Placed points and moments.** Since $C_0 = \sqrt{P_0} = 1$, the previous-state placed points are

$$\chi_0^{(r)} = m_0 + C_0\xi^{(r)} \in \{0.5 - \sqrt{3},\ 0.5,\ 0.5 + \sqrt{3}\}. \tag{36.22}$$

At $\beta = 1$, the transition images satisfy

$$a_1^{(r)} = f_\beta(\chi_0^{(r)}) = \chi_0^{(r)}. \tag{36.23}$$

The predicted Gaussian moments are therefore

$$m_1^- = \sum_r w_r a_1^{(r)} = 0.5, \qquad P_1^- = Q + \sum_r w_r (a_1^{(r)} - m_1^-)^2 = 1.25 = \frac{5}{4}. \tag{36.24}$$

The declared Cholesky factor is

$$C_1^- = \sqrt{P_1^-} = \sqrt{1.25} = \frac{\sqrt{5}}{2}. \tag{36.25}$$

Hence the predictive placed points are

$$\chi_1^{(r)} = m_1^- + C_1^-\xi^{(r)} \in \left\{0.5 - \frac{\sqrt{15}}{2},\ 0.5,\ 0.5 + \frac{\sqrt{15}}{2}\right\}. \tag{36.26}$$

Passing those points through $h_\theta(x) = x^2$ gives

$$z_1^{(r)} \in \{2.0635083269,\ 0.25,\ 5.9364916731\}. \tag{36.27}$$

The centered observation values are

$$\Delta_1^{(r)} = z_1^{(r)} - \bar{z}_1 \in \{0.5635083269,\ -1.25,\ 4.4364916731\}. \tag{36.28}$$

and the Gaussian update objects are

$$\bar{z}_1 = \sum_r w_r z_1^{(r)} = 1.5 = \frac{3}{2}, \tag{36.29}$$

$$S_1 = R + \sum_r w_r (z_1^{(r)} - \bar{z}_1)^2 = 5.375 = \frac{43}{8}, \tag{36.30}$$

$$C_{xz,1} = \sum_r w_r (\chi_1^{(r)} - m_1^-)(z_1^{(r)} - \bar{z}_1) = 1.25 = \frac{5}{4}, \tag{36.31}$$

$$v_1 = y_1 - \bar{z}_1 = 0.5 = \frac{1}{2}, \tag{36.32}$$

$$K_1 = C_{xz,1} S_1^{-1} = \frac{10}{43} \approx 0.2325581395, \tag{36.33}$$

$$m_1 = m_1^- + K_1 v_1 = \frac{53}{86} \approx 0.6162790698, \tag{36.34}$$

$$P_1 = P_1^- - K_1 S_1 K_1^\top = \frac{165}{172} \approx 0.9593023256. \tag{36.35}$$

The one-step deterministic approximate log likelihood is

$$\widehat{\ell}_1^{\mathrm{FSGQ}} = -\frac{1}{2}\left\{\log S_1 + v_1^2 S_1^{-1} + \log(2\pi)\right\} \approx -1.7830736342. \tag{36.36}$$

**Same-branch derivative with respect to $\beta$.** The same cloud and the same positive scalar Cholesky branch are reused in the derivative lane. Because the parameter enters only through the transition in this example, $\partial_\beta h_\theta(x) = 0$. Since $m_0$ and $P_0$ are fixed in $\beta$, the previous-state placed points satisfy

$$\dot{\chi}_0^{(r)} = 0. \tag{36.37}$$

From the transition derivative rule,

$$\dot{a}_1^{(r)} = D_x f_\beta(\chi_0^{(r)})\dot{\chi}_0^{(r)} + \partial_\beta f_\beta(\chi_0^{(r)}) = \chi_0^{(r)} \in \{0.5 - \sqrt{3},\ 0.5,\ 0.5 + \sqrt{3}\}. \tag{36.38}$$

Hence

$$\dot{m}_1^- = \sum_r w_r \dot{a}_1^{(r)} = 0.5, \qquad \dot{d}_1^{(r)} = \dot{a}_1^{(r)} - \dot{m}_1^- \in \{-\sqrt{3},\ 0,\ \sqrt{3}\}, \tag{36.39}$$

and

$$\dot{P}_1^- = 2. \tag{36.40}$$

Because $C_1^- = (P_1^-)^{1/2}$, the declared branch derivative is

$$\dot{C}_1^- = \frac{\dot{P}_1^-}{2C_1^-} = \frac{2}{\sqrt{5}} \approx 0.8944271910. \tag{36.41}$$

Therefore the predictive point sensitivities are

$$\dot{\chi}_1^{(r)} = \dot{m}_1^- + \dot{C}_1^- \xi^{(r)} \in \{-1.0491933385,\ 0.5,\ 2.0491933385\}. \tag{36.42}$$

Since $h_\theta(x) = x^2$,

$$\dot{z}_1^{(r)} = 2\chi_1^{(r)}\dot{\chi}_1^{(r)} \in \{3.0143149884,\ 0.5,\ 9.9856850116\}. \tag{36.43}$$

The centered observation sensitivities are

$$\dot{\Delta}_1^{(r)} = \dot{z}_1^{(r)} - \dot{\bar{z}}_1 \in \{0.5143149884,\ -2,\ 7.4856850116\}. \tag{36.44}$$

The derived moment sensitivities are

$$\dot{\bar{z}}_1 = 2.5 = \frac{5}{2}, \qquad \dot{v}_1 = -2.5 = -\frac{5}{2}, \tag{36.45}$$

$$\dot{S}_1 = 14.5 = \frac{29}{2}, \qquad \dot{C}_{xz,1} = 3.25 = \frac{13}{4}. \tag{36.46}$$

Let

$$u_1 = S_1^{-1} v_1 = \frac{4}{43} \approx 0.0930232558. \tag{36.47}$$

Then the one-step score ingredients are

$$\operatorname{tr}(S_1^{-1}\dot{S}_1) = \frac{116}{43} \approx 2.6976744186, \tag{36.48}$$

$$2\dot{v}_1 u_1 = -\frac{20}{43} \approx -0.4651162791, \tag{36.49}$$

$$-u_1^2 \dot{S}_1 = -\frac{232}{1849} \approx -0.1254732288. \tag{36.50}$$

The same-branch one-step score is therefore

$$\dot{\widehat{\ell}}_1^{\mathrm{FSGQ}} = -\frac{1}{2}\left[\frac{\dot{S}_1}{S_1} + 2\dot{v}_1 u_1 - u_1^2 \dot{S}_1\right] = -\frac{1948}{1849} \approx -1.0535424554. \tag{36.51}$$

For later propagation,

$$\dot{K}_1 = \dot{C}_{xz,1} S_1^{-1} - K_1 \dot{S}_1 S_1^{-1} = -\frac{42}{1849} \approx -0.0227149811, \tag{36.52}$$

so

$$\dot{m}_1 = \dot{m}_1^- + \dot{K}_1 v_1 + K_1 \dot{v}_1 = -\frac{343}{3698} \approx -0.0927528394, \tag{36.53}$$

$$\dot{P}_1 = \dot{P}_1^- - \dot{K}_1 S_1 K_1 - K_1 \dot{S}_1 K_1 - K_1 S_1 \dot{K}_1 = \frac{2353}{1849} \approx 1.2725797729. \tag{36.54}$$

This makes the chapter's limitation concrete. Every quantity above is deterministic and internally checkable on the declared branch, yet the nonlinear observation law can still generate non-Gaussian posterior structure. FixedSGQF therefore gives a transparent Gaussian-surrogate value-and-gradient oracle, not a full non-Gaussian posterior representation.

**Implementation checklist for this oracle.** A first implementation should be able to reproduce, up to declared floating-point tolerance, at least $m_1^-$, $P_1^-$, $\bar{z}_1$, $S_1$, $C_{xz,1}$, $K_1$, $m_1$, $P_1$, $\widehat{\ell}_1^{\mathrm{FSGQ}}$, and $\dot{\widehat{\ell}}_1^{\mathrm{FSGQ}}$. This is the compact end-to-end numerical oracle for both the value path and the derivative path.

This example is not a global correctness theorem. Its role is narrower and practical: a first BayesFilter implementation should be able to reproduce these numbers before any same-branch derivative claim is trusted.

## 36.4 General Analytical Derivative Framework For FixedSGQF

The worked oracle fixes one concrete same-branch derivative instance. The next task is to state the general derivative framework so that the SGQF lane does not depend on the toy example alone. Differentiate with respect to one parameter component $\theta_i$, and let a dot denote $\partial_i$.

**Why the derivative is operationally delicate.** The one-step scalar depends on the previous carried Gaussian, the covariance factor used to place the cloud, the nonlinear transition and observation images, the innovation covariance, and the posterior update. The derivative must therefore propagate through the entire recursion, and it remains meaningful only when the value and derivative paths stay on the same declared branch.

**Current-score objects versus propagation objects.** For the current likelihood contribution, the derivative closes once $\dot{v}_t$ and $\dot{S}_t$ are known. The derivative $\dot{C}_{xz,t}$ is not needed to close the current score, but it is needed to form $\dot{K}_t$ and hence to propagate $\dot{m}_t$ and $\dot{P}_t$ to the next time step. A correct implementation must keep these two roles separate.

### 36.4.1 Derivative Ledger

The chain is

$$\begin{aligned}\theta \longmapsto (m_{t-1}, P_{t-1}) \longmapsto C_{t-1} \longmapsto \chi_{t-1}^{(r)} \longmapsto a_t^{(r)} \longmapsto (m_t^-, P_t^-) \longmapsto C_t^- \\ \longmapsto \chi_t^{(r)} \longmapsto z_t^{(r)} \longmapsto (\bar{z}_t, S_t, C_{xz,t}) \longmapsto (v_t, K_t, \widehat{\ell}_t^{\mathrm{FSGQ}}, m_t, P_t).\end{aligned} \tag{36.55}$$

The sparse-grid nodes and weights do not move with $\theta$. They are saved branch objects. The Gaussian factors $C_{t-1}$ and $C_t^-$, however, do move with $\theta$, because they are functions of $P_{t-1}$ and $P_t^-$.

### 36.4.2 Square-Root Branch

For the declared lower-triangular Cholesky branch $P = LL^\top$, set

$$A = L^{-1}\dot{P}L^{-\top}. \tag{36.56}$$

Define the lower-triangular operator

$$\Phi(A)_{jk} = \begin{cases} A_{jk}, & j > k, \\ A_{jj}/2, & j = k, \\ 0, & j < k. \end{cases} \tag{36.57}$$

Then

$$\dot{L} = L\Phi(A). \tag{36.58}$$

Every factor derivative in this SGQF lane uses this declared branch recipe.

### 36.4.3 Prediction Sensitivities

From the placed-point equation,

$$\dot{\chi}_{t-1}^{(r)} = \dot{m}_{t-1} + \dot{C}_{t-1}\xi^{(r)}. \tag{36.59}$$

From the transition map,

$$\dot{a}_t^{(r)} = D_x f_\theta(\chi_{t-1}^{(r)})\dot{\chi}_{t-1}^{(r)} + \partial_i f_\theta(\chi_{t-1}^{(r)}). \tag{36.60}$$

The predicted mean derivative is

$$\dot{m}_t^- = \sum_{r=1}^{M} w_r \dot{a}_t^{(r)}. \tag{36.61}$$

Let

$$d_t^{(r)} = a_t^{(r)} - m_t^-, \qquad \dot{d}_t^{(r)} = \dot{a}_t^{(r)} - \dot{m}_t^-. \tag{36.62}$$

Then

$$\dot{P}_t^- = \dot{Q}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{d}_t^{(r)} d_t^{(r)\top} + d_t^{(r)} \dot{d}_t^{(r)\top} \right\}. \tag{36.63}$$

### 36.4.4 Observation Sensitivities And Innovation Score

On the predictive factor branch,

$$\dot{\chi}_t^{(r)} = \dot{m}_t^- + \dot{C}_t^- \xi^{(r)}. \tag{36.64}$$

Observation sensitivities satisfy

$$\dot{z}_t^{(r)} = D_x h_\theta(\chi_t^{(r)}) \dot{\chi}_t^{(r)} + \partial_i h_\theta(\chi_t^{(r)}). \tag{36.65}$$

Then

$$\dot{\bar{z}}_t = \sum_{r=1}^{M} w_r \dot{z}_t^{(r)}, \tag{36.66}$$

$$\dot{v}_t = -\dot{\bar{z}}_t, \tag{36.67}$$

$$\dot{\Delta}_t^{(r)} = \dot{z}_t^{(r)} - \dot{\bar{z}}_t, \tag{36.68}$$

$$\dot{S}_t = \dot{R}_\theta + \sum_{r=1}^{M} w_r \left\{ \dot{\Delta}_t^{(r)} \Delta_t^{(r)\top} + \Delta_t^{(r)} \dot{\Delta}_t^{(r)\top} \right\}, \tag{36.69}$$

$$\dot{C}_{xz,t} = \sum_{r=1}^{M} w_r \left\{ \dot{\chi}_t^{(r)} \Delta_t^{(r)\top} + (\chi_t^{(r)} - m_t^-) \dot{\Delta}_t^{(r)\top} \right\}. \tag{36.70}$$

The one-step fixed-Gaussian innovation score is

$$\partial_i \widehat{\ell}_t^{\mathrm{FSGQ}} = -\frac{1}{2} \left[ \mathrm{tr}(S_t^{-1} \dot{S}_t) + 2 \dot{v}_t^\top S_t^{-1} v_t - v_t^\top S_t^{-1} \dot{S}_t S_t^{-1} v_t \right]. \tag{36.71}$$

This closes the current score role. The propagation role then uses

$$\dot{K}_t = \dot{C}_{xz,t} S_t^{-1} - K_t \dot{S}_t S_t^{-1} \tag{36.72}$$

and

$$\dot{m}_t = \dot{m}_t^- + \dot{K}_t v_t + K_t \dot{v}_t, \tag{36.73}$$

$$\dot{P}_t = \dot{P}_t^- - \dot{K}_t S_t K_t^\top - K_t \dot{S}_t K_t^\top - K_t S_t \dot{K}_t^\top. \tag{36.74}$$

These become the differentiated inputs to the next time step.

## 36.5 Same-Branch Fixed-Cloud Scalar Contract

For the sparse-grid lane, the saved branch identity is not exhausted by the symbol $\mathcal{C}_{b,L}$. A branch-valid finite-difference comparison must reuse the full fixed ledger:

$$\mathfrak{B}_{\mathrm{FSGQ}} = \{\mathcal{C}_{b,L}, \text{ one-dimensional rule family, merge tolerance, zero-weight pruning rule, node ordering} \tag{36.75}$$

For parameter component $i$, the same-branch central difference is

$$D_i(h) = \frac{\widehat{\ell}_T^{\mathrm{FSGQ}}(\theta + h e_i; \mathfrak{B}_{\mathrm{FSGQ}}) - \widehat{\ell}_T^{\mathrm{FSGQ}}(\theta - h e_i; \mathfrak{B}_{\mathrm{FSGQ}})}{2h}. \tag{36.76}$$

A row is branch-valid only if both perturbations reuse the same saved structural branch record and realize the same accepted/failure stage-time pattern through every likelihood contribution included in the comparison. Equal cloud family but different first failure time counts as a branch mismatch; equal first failure time but different accepted-prefix pattern also counts as a branch mismatch.

That stricter logic matters because sparse-grid differentiation is not just "differentiate some nearby Gaussian filter." It is "differentiate the same fixed-cloud scalar." The later shared same-scalar chapter may import that contract, but the sparse-grid lane must state it locally because the cloud, merge policy, and branch-veto discipline are part of the lane-specific scalar identity.

**Frozen versus recomputed objects on a saved branch.** The branch identity freezes structure, not numerical values. Within a fixed cloud branch,

$$\begin{aligned} &\text{frozen structural objects:} && \mathcal{C}_{b,L},\ \tau_{\mathrm{merge}},\ \tau_{\mathrm{zero}},\ \text{ordering},\ C(\cdot),\ \text{preprocessing policy},\ \text{init} \\ &\text{parameter-dependent numerical objects:} && m_t, P_t, C_t, m_t^-, P_t^-, C_t^-, \bar{z}_t, S_t, C_{xz,t}, K_t, \widehat{\ell}_t^{\mathrm{FSGQ}}. \end{aligned} \tag{36.77}$$

Thus same-scalar finite differences recompute the parameter-dependent numerical objects by the same declared branch rule; they do not reuse old numerical values. In particular, $m_0(\theta \pm he_i)$ and $P_0(\theta \pm he_i)$ are recomputed by the same initial-condition policy. What must remain unchanged is the structural route by which those values are produced.

**Finite-difference ladder and pass criterion.** For each parameter component, use a ladder such as

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \tag{36.78}$$

For valid rows, compare the central difference against the analytical score $G_i$ by

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}. \tag{36.79}$$

The diagnostic passes when at least two consecutive valid rows satisfy $e_i(h) \le 10^{-5}$ and the smaller-step row is no worse than ten times the larger-step row. Failure means one of three things is likely: the derivative formula is wrong, the value and gradient paths are not using the same scalar, or the declared branch is too nonsmooth or ill-conditioned at that point.

**Implementation-facing FD checklist.** A routine of the form `same_scalar_fd_check(`$i, h_{\mathrm{ladder}}, \theta, \mathfrak{B}_{\mathrm{F}}$ should:

1. evaluate the value path at $\theta \pm he_i$ using the same saved structural branch metadata,
2. mark a row branch-invalid if either side changes the structural branch or the realized accepted/failure stage-time pattern,
3. compute the central difference only for branch-valid rows,
4. compare that central difference against the analytical score with (36.79), and
5. report whether two consecutive valid rows satisfy the declared tolerance rule.

The equations above remain the authoritative mathematical definition; this list is the operational review contract.

## 36.6 Lane-Specific Validation Burden

Fixed SGQF should be validated as an approximate deterministic likelihood lane, not as an exact posterior filter. The minimum validation suite is:

1. **Construction checks.** Verify duplicate-node accumulation, weight totals, signed-weight counts, and deterministic cloud reproduction for small $(b, L)$ cases.

2. **Linear-Gaussian recovery.** For affine $f_\theta, h_\theta$, the Gaussian projection recursion should match the Kalman filter up to floating-point tolerance.

3. **Worked scalar oracle.** Reproduce the one-step values in Section 36.3 exactly enough that the cloud, factor, innovation, and likelihood assembly are demonstrably aligned.

4. **Same-branch finite differences.** Use (36.76) on representative parameters and step sizes while logging branch-valid versus branch-invalid rows.

5. **Signed-weight stability.** Record the minimum eigenvalues of symmetrized $P_t^-$, $S_t$, and $P_t$, plus any branch veto.

6. **Memory and runtime scaling.** Record $M_{b,L}$, model evaluations, wall time, and peak memory as $b$, $L$, and $T$ change.

7. **Accuracy ladder.** Compare fixed SGQF against tensor-product GHQ in small dimension and against other controlled references where available.

A useful test-model ladder mirrors the p47 source burden:

- **Model A:** a linear-Gaussian recovery case,
- **Model B:** the scalar quadratic-observation cell from Chapter 33,
- **Model C:** a cloud-sensitive two-dimensional nonlinear observation where duplicate merging and fourth moments matter,
- **Model D:** a small high-dimensional block-local model where signed weights, active-dimension growth, and covariance checks become visible,
- **Model E:** a deliberately hard all-coordinate interaction model to expose when sparse-grid savings stop compensating for Gaussian-surrogate bias,
- **Model F:** a Jia–Xin–Cheng-style orbit / sparse-grid benchmark that distinguishes sparse-grid exactness claims from posterior-accuracy claims.

The first three models should be understood concretely, not only as names:

$$\text{Model A: } X_t = A_\theta X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q_\theta), \quad Y_t = H_\theta X_t + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, R_\theta), \tag{36.80}$$

$$\text{Model B: } X_t = \rho X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, q), \quad Y_t = X_t^2 + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, r), \tag{36.81}$$

$$\text{Model C: } X_t = \rho X_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, qI_2), \quad Y_t = \theta^2(X_{t,1}^2 + X_{t,2}^2) + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, r). \tag{36.82}$$

Model A checks exact-reference recovery, Model B exposes posterior-shape failure of one-Gaussian surrogates, and Model C catches duplicate-node or signed-weight assembly errors because the relevant fourth moments change when the cloud is built incorrectly.

The implementation report for this lane should log at least:

$$\mathcal{R}_{\mathrm{SGQF}} = \{M_{b,L},\ \text{signed-weight count},\ \min \lambda(P_t^-),\ \min \lambda(S_t),\ \min \lambda(P_t),\ \text{FD table},\ \text{branch-veto count}, \tag{36.83}$$

Accuracy claims should be made only against a named comparator:

$$\text{error} = \begin{cases} |\widehat{\ell}_T - \ell_T^{\mathrm{Kalman}}|, & \text{linear-Gaussian test}, \\ |\widehat{\ell}_T - \ell_T^{\mathrm{dense}}|, & \text{small dense-quadrature test}, \\ \text{RMSE or posterior diagnostic against PF/TT reference}, & \text{nonlinear large test}. \end{cases} \tag{36.84}$$

These are not mere software counters. They record the places where the declared sparse-grid scalar can fail mathematically or operationally: duplicate merging, signed-weight stability, covariance branch validity, same-branch derivative parity, and cost growth.

Passing this suite supports the claim that the implementation evaluates and differentiates the declared fixed-cloud surrogate scalar. It does not prove exact posterior accuracy, HMC convergence, or default readiness.

## 36.7 Implementation Contract For FixedSGQF

A compact implementation contract makes the SGQF lane auditable without forcing the reader back to the earlier derivation. The key runtime objects are:

| object | definition | shape | reuse |
|---|---|---|---|
| $\xi^{(r)}$ | standardized sparse-grid node | $n_x$ | value and gradient |
| $w_r$ | accumulated signed weight after duplicate merge | scalar | value and gradient |
| $C_{t-1}, C_t^-$ | covariance factors on declared branch | $n_x \times n_x$ | value, gradient, diagnostics |
| $\chi_{t-1}^{(r)}$ | previous placed point | $n_x$ | transition value and derivative |
| $a_t^{(r)}$ | transition image | $n_x$ | prediction moments |
| $\chi_t^{(r)}$ | predictive placed point | $n_x$ | observation value and derivative |
| $z_t^{(r)}$ | observation image | $n_y$ | observation moments |
| $\bar{z}_t, S_t, C_{xz,t}$ | Gaussian projection moments | $n_y$, $n_y \times n_y$, $n_x \times n_y$ | likelihood and update |
| $v_t, K_t, u_t$ | innovation, gain, solve variable with $S_t u_t = v_t$ | $n_y$, $n_x \times n_y$, $n_y$ | likelihood, update, gradient |
| $\dot{\chi}, \dot{a}, \dot{z}$ | point sensitivities | matching value objects | gradient only |
| $\dot{m}, \dot{P}, \dot{S}, \dot{C}_{xz}$ | moment and filter sensitivities | matching value objects | gradient and next step |
| data record | $y_{1:T}$, preprocessing, missing-data policy if any | metadata | value, gradient, finite difference |
| initial law | $m_0(\theta), P_0(\theta), \dot{m}_0, \dot{P}_0$ | $n_x, n_x \times n_x$ | value and gradient start |
| branch record | cloud, rules, merge tolerance, zero-weight rule, node ordering, preprocessing policy, initial-condition policy, factor branch, symmetrize-then-veto rule, thresholds, accepted/failure stage-time pattern | metadata | same-scalar finite difference |

### 36.7.1 Input, Output, Invariant, And Failure Contract

The full lane-level routine has inputs

$$\begin{aligned}\mathcal{I}_{\text{in}} = (&y_{1:T}, \theta, L, \{I_\ell\}_{\ell=1}^{L+b-1}, m_0(\theta), P_0(\theta),\\ &\dot{m}_0^{(1:p)}(\theta), \dot{P}_0^{(1:p)}(\theta), f_\theta, h_\theta, Q_\theta, R_\theta,\\ &D_x f_\theta, \partial_{1:p} f_\theta, D_x h_\theta, \partial_{1:p} h_\theta, \dot{Q}_\theta^{(1:p)}, \dot{R}_\theta^{(1:p)},\\ &\text{merge tolerance, factor map, branch thresholds}).\end{aligned} \tag{36.85}$$

It returns either

$$\left(\widehat{\ell}_T^{\text{FSGQ}}(\theta), \nabla_\theta \widehat{\ell}_T^{\text{FSGQ}}(\theta), \{m_t, P_t\}_{t=0}^T, \mathcal{D}_{\text{diag}}\right) \tag{36.86}$$

or a failure record

$$\mathcal{F} = (\text{time, stage, reason, diagnostic values}). \tag{36.87}$$

The invariants are:

$$\sum_{r=1}^{M} w_r = 1 \qquad \text{up to construction tolerance,} \tag{36.88}$$

$$P_t^- = (P_t^-)^\top, \quad S_t = S_t^\top, \quad P_t = P_t^\top \qquad \text{after explicit symmetrization,} \tag{36.89}$$

$$S_t \succ 0 \qquad \text{on every accepted likelihood step,} \tag{36.90}$$

$$\mathcal{B}_{\text{value}} = \mathcal{B}_{\text{gradient}} \qquad \text{same saved branch.} \tag{36.91}$$

The routine fails rather than repairs if

$$\lambda_{\min}(S_t) < \epsilon_S, \qquad \lambda_{\min}(P_t^-) < \epsilon_P, \qquad \lambda_{\min}(P_t) < \epsilon_P, \tag{36.92}$$

or if the declared factor map cannot be evaluated on the symmetrized covariance. A smoothed repair would define a different scalar and therefore would need a new derivative contract.

### 36.7.2 One-Step Value-Path Contract

At one accepted time step, the value path is the ordered map

$$\begin{aligned}(m_{t-1}, P_{t-1}, y_t, \theta, \mathcal{C}) &\longmapsto C_{t-1} \longmapsto \chi_{t-1}^{(r)} \longmapsto a_t^{(r)} \longmapsto (m_t^-, P_t^-) \\ &\longmapsto C_t^- \longmapsto \chi_t^{(r)} \longmapsto z_t^{(r)} \longmapsto (\bar{z}_t, S_t, C_{xz,t}) \\ &\longmapsto (v_t, \widehat{\ell}_t^{\text{FSGQ}}, K_t) \longmapsto (m_t, P_t).\end{aligned} \tag{36.93}$$

The accepted map is conditioned on two successful branch checks: the predictive covariance $P_t^-$ must pass the declared factor and positive-definite test before observation-point placement, and the innovation covariance $S_t$ must pass the declared branch test before the Gaussian projection update is accepted. The carried outputs propagated to the next step are only $(m_t, P_t)$ together with the global fixed branch record $\mathcal{B}$. Point clouds, moment arrays, gains, and diagnostics may be cached for derivative reuse, but they are not part of the carried filtering state itself.

### 36.7.3 Default Numerical Constants

The constants below are defaults for diagnostics, not claims of universal validity:

| quantity | default | role |
|---|---|---|
| duplicate merge tolerance | $10^{-12}\max(1, \|\xi\|_\infty)$ | groups standardized nodes that should be identical under the declared rule |
| weight zero tolerance | $10^{-14}$ | removes accumulated numerical-zero weights after merge |
| $\epsilon_S$ | $10^{-10}$ | veto threshold for innovation covariance positive definiteness |
| $\epsilon_P$ | $10^{-10}$ | veto threshold for carried and predictive covariance factors |
| $\epsilon_\ell$ | $10^{-3}\max(1, \|\widehat{\ell}_T\|)$ | pilot level-ladder change threshold |
| finite-difference relative tolerance | $10^{-5}$ | same-scalar gradient diagnostic |
| maximum branch-invalid FD rows | 0 in final audit | branch changes mean the derivative is not being tested |

These values are deliberately conservative. They are meant to reveal when the fixed branch is fragile. They are not tuning knobs for making a failing branch look smooth.

## 36.8 End-To-End Mathematical Algorithm

**FixedSGQF value and gradient, with all branch exits.**

1. Build the sparse-grid cloud from $b = n_x$, level $L$, and $\{I_\ell\}$. Save $\mathcal{C} = \{(\xi^{(r)}, w_r)\}_{r=1}^{M}$.
2. Check the weight-sum invariant. If it fails, return $\mathcal{F} = (0, \text{cloud}, \text{weight-sum failure}, \cdots)$.
3. Initialize $\widehat{\ell}_T^{\mathrm{FSGQ}} \leftarrow 0$, $G_i \leftarrow 0$ for $i = 1, \ldots, p$, and compute $m_0, P_0, \dot{m}_0^{(i)}, \dot{P}_0^{(i)}$.
4. For $t = 1, \ldots, T$, symmetrize $P_{t-1}$. If the factor map fails, return a factor-branch failure. Otherwise set $C_{t-1} = C(P_{t-1})$ and compute the matching factor derivatives.
5. Place previous points, compute transition images, and form $(m_t^-, P_t^-)$ together with their derivatives.
6. Symmetrize $P_t^-$. If the predictive covariance fails the branch, return predictive covariance failure.
7. Factor $P_t^-$, place observation points, evaluate observation values, and form $(\bar{z}_t, S_t, C_{xz,t}, v_t)$.
8. Symmetrize $S_t$. If the innovation covariance fails the branch, return innovation covariance failure.
9. Only on that accepted innovation branch, form $\widehat{\ell}_t^{\mathrm{FSGQ}}$, update $\widehat{\ell}_T^{\mathrm{FSGQ}}$, and accumulate the componentwise score.
10. Form $K_t$, then propagate $m_t, P_t$ and their sensitivities. If the carried covariance branch fails, return carried covariance failure.
11. Return $(\widehat{\ell}_T^{\mathrm{FSGQ}}, G_{1:p}, \{m_t, P_t\}, \mathcal{D}_{\mathrm{diag}})$.

The algorithm is deliberately mathematical rather than Pythonic. Its purpose is to say what an implementation must compute, save, and refuse to change if the reported gradient is to be the derivative of the reported value.

## 36.9 Exports To Later Chapters

This SGQF continuation chapter exports five specific objects forward:

1. the fixed-cloud value-path recursion,
2. the one-step numeric oracle,
3. the lane-specific branch-identity record,

4. the branch-valid finite-difference contract for SGQF,

5. the SGQF validation and non-claim boundaries.

The later fixed-branch chapter may now treat the sparse-grid lane as an already specified scalar object rather than re-explaining the cloud mechanics. The closing validation/promotion chapter may consume the SGQF diagnostics and veto logic without needing to rederive the lane's runtime map.

# Chapter 37

# Low-Rank Density Filters, KR Maps, and Retained Objects

## 37.1 Orientation And Scope

This chapter studies a high-dimensional nonlinear filtering problem through the lens of one specific computational object: a low-rank non-Gaussian retained representation built from tensor-train density approximations, conditional Knothe–Rosenblatt maps, and squared-density repair. The exact target remains the filtering law and normalizer from Chapter 33; the approximation here is the retained low-rank density or retained-object representation that numerically replaces that law.

The scientific tension is the same one that appears across many high-dimensional filtering problems. The exact Bayesian recursion propagates a full conditional law, but once the model is nonlinear that law is usually not available in a finite-dimensional closed form. Zhao and Cui answer that tension with a non-Gaussian density-approximation lane based on tensor trains, squared-density repair, conditional KR maps, and preconditioning. The purpose of this chapter is to make that retained-object construction readable in source order and to state clearly what object is being carried, normalized, updated, and later handed to the shared fixed-branch derivative chapter.

**What this chapter does not claim.** This chapter does not claim that adaptive TT/KR algorithms are globally smooth, that moderate TT rank is always sufficient in high dimension, or that a retained low-rank object automatically yields an exact posterior target for HMC. Its purpose is narrower: define the retained object, explain when that object is scientifically meaningful, and state the diagnostics that later chapters must use before promotion.

## 37.2 Running Example And Notation

The running nonlinear example should stay conceptually continuous with the rest of the block, but the retained-object lane needs a slightly richer notation than the deterministic Gaussian lane. The key distinction is between:

- the exact Bayesian target law and its normalizer,
- the adaptive low-rank approximation family that numerically replaces it,

- and the fixed-branch scalar/derivative objects that later chapters may define from a frozen member of that family.

The method uses state variables $x_t$, observation variables $y_t$, and a static parameter $\alpha$. Coordinate group notation such as $x_{t,<j}, x_{t,>j}, x_{t,\leq j}, x_{t,\geq j}$ matters because conditional transport and conditional density construction are coordinate-local operations. For TT/KR methods, the essential bookkeeping objects are coordinate groups, conditional densities, retained marginal/filter objects, and any Jacobian terms needed when the retained object is moved into preconditioned coordinates.

## 37.3 Exact Filtering Target And Recursive Bottleneck

The retained-object lane starts from the same target discipline as the rest of the block. At time $t$, the filtering problem is to update

$$p(x_t, \alpha \mid y_{1:t}),$$

and the path problem is to update

$$p(x_{0:t}, \alpha \mid y_{1:t}).$$

After observing $y_t$, the exact adjacent-state target has the three-block form

$$q_t(x_t, \alpha, x_{t-1}) = p(x_{t-1}, \alpha \mid y_{1:t-1}) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t),$$

and the next filter is obtained by integrating out $x_{t-1}$ and normalizing. The bottleneck is therefore always the same: how to represent this object and how to integrate the represented object without returning to an infeasible full grid.

Implementation object. A minimal implementation must expose evaluators for the previous retained filter, the transition density, and the observation density, then build an evaluator for the three-block target. The first failure check is that this target must be finite and nonnegative at all fitting points; otherwise neither a square root nor an importance weight is meaningful.

## 37.4 Tensor-Train Approximation Toolkit

A tensor train is a compressed representation of a multidimensional array or function. For a grid tensor $A(i_1, \ldots, i_D)$, the TT format writes

$$A(i_1, \ldots, i_D) = G_1(i_1) G_2(i_2) \cdots G_D(i_D), \tag{37.1}$$

where the $G_k(i_k)$ are small matrices with connecting ranks. If the mode size is $M$ and ranks are bounded by $r$, storage is $O(DMr^2)$ instead of $O(M^D)$. The key condition is rank stability: if ranks grow with the effective dimension, the compression disappears.

The chapter uses this toolkit for filtering objects, not for abstract storage arithmetic alone. The object being approximated may be a density, an unnormalized adjacent-state target, a retained marginal, or a conditional map. The approximation only helps if the represented object remains interpretable as a probability or retained filtering quantity after truncation, marginalization, and normalization.

### 37.4.1 Rank-Count Plausibility

The method is plausible when the posterior geometry has limited interaction width after ordering and preconditioning. If every coordinate uses $p$ basis functions and the function has $D$ coordinates, a full tensor stores $p^D$ coefficients, while a TT with moderate ranks stores approximately $DpR^2$ coefficients. This is the theorem-to-implementation bridge: low-rank structure is not decoration, but the empirical condition under which the method escapes the full-grid curse of dimensionality.

Rank diagnostics therefore belong to the method contract. If fitted ranks grow persistently toward the cap or if conditioning deteriorates at the same time, then the chosen ordering, basis, or preconditioner is not expressing the target compactly.

## 37.5 Coordinate Systems, Conditional Maps, And Retained Objects

The retained-object lane uses several coordinate systems because different operations are easiest in different spaces. Let

$$r = (x_t, \alpha, x_{t-1}) \tag{37.2}$$

denote physical filtering coordinates. A domain map sends a reference cube coordinate $z \in [-1, 1]^D$ to physical coordinates:

$$r = \Psi_t(z), \qquad J_{\Psi,t}(z) = \left|\det \nabla \Psi_t(z)\right|. \tag{37.3}$$

The target seen by the TT in reference coordinates is

$$\widetilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \tag{37.4}$$

Thus $z$ is the coordinate in which basis functions, fitting points, and mass matrices are declared.

Preconditioning introduces another coordinate $u$. Let $T_t$ be an invertible map from physical coordinates to a preconditioned coordinate:

$$u = T_t(r), \qquad r = T_t^{-1}(u). \tag{37.5}$$

If $\eta(u)$ is the reference density used in $u$-space, then

$$q_t(r)\,\mathrm{d}r = q_t(T_t^{-1}(u)) \left|\det \nabla T_t^{-1}(u)\right| \,\mathrm{d}u. \tag{37.6}$$

The retained coordinate $z_t$ is the part of the reference coordinate retained after marginalizing the previous state:

$$z = (z_t, z_{t-1}), \qquad a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}, \tag{37.7}$$

and

$$\widehat{p}_t(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \tag{37.8}$$

Most implementation mistakes in this lane are coordinate mistakes: using a physical density in reference coordinates without the Jacobian, applying a KR map in $u$-space to a point still in $r$-space, or carrying a retained filter in $z_t$ while evaluating it as if it were already a physical density.

### 37.5.1 One Observation Step Through All Coordinate Systems

The symbols $r, z, u, z_t$ become easier to track when one observation update is written as a single chain. Begin in physical coordinates

$$r = (x_t, \alpha, x_{t-1}), \qquad q_t(r) = \widehat{p}_{t-1}(x_{t-1}, \alpha) f_\alpha(x_t \mid x_{t-1}) g_\alpha(y_t \mid x_t). \tag{37.9}$$

A domain map

$$r = \Psi_t(z), \qquad z \in [-1, 1]^D, \qquad J_{\Psi,t}(z) = |\det \nabla \Psi_t(z)| \tag{37.10}$$

turns the physical density into a reference-coordinate density

$$\widetilde{q}_t(z) = q_t(\Psi_t(z)) J_{\Psi,t}(z). \tag{37.11}$$

The squared-TT route fits a source-scale square root in reference coordinates and then returns to the unshifted evidence scale through

$$q_{t,\mathrm{src}}(z) = \phi_t(z)^2 + \tau_t \lambda_t(z), \qquad \widehat{q}_t(z) = e^{-c_t} q_{t,\mathrm{src}}(z). \tag{37.12}$$

The carried filter is not the full joint object. If $z = (z_t, z_{t-1})$, then

$$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\, \mathrm{d}z_{t-1}, \qquad \widehat{p}_t^{\mathrm{ref}}(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \tag{37.13}$$

If a later formula needs the filter as a physical density, write

$$(x_t, \alpha) = \Psi_{t,\mathrm{ret}}(z_t), \qquad \widehat{p}_t^{\mathrm{phys}}(x_t, \alpha) = \widehat{p}_t^{\mathrm{ref}}(\Psi_{t,\mathrm{ret}}^{-1}(x_t, \alpha)) \left|\det \nabla \Psi_{t,\mathrm{ret}}^{-1}(x_t, \alpha)\right|. \tag{37.14}$$

The bookkeeping rule is: $\widetilde{q}_t(z)$, $a_t(z_t)$, $\widehat{Z}_t$, and $\widehat{p}_t^{\mathrm{ref}}(z_t)$ are reference-coordinate objects integrated with $\mathrm{d}z$; $\widehat{p}_t^{\mathrm{phys}}$ is the corresponding physical object after the declared Jacobian correction.

Preconditioning inserts one additional coordinate $u$. A bridge density $\rho_t(r)$ and map $T_t : r \mapsto u$ are chosen so that the dominant shape of $q_t$ is easier in $u$-space. The residual target is

$$q_t^\sharp(u) = \frac{q_t(T_t^{-1}(u))}{\widehat{\rho}_t(T_t^{-1}(u))} \eta(u), \tag{37.15}$$

and the pullback relation after fitting is

$$\widehat{q}_t(r) = \frac{\widehat{\nu}_t^\sharp(T_t(r))}{\eta(T_t(r))} \widehat{\rho}_t(r). \tag{37.16}$$

Thus the full coordinate story of one observation step is

$$q_t(r) \to \widetilde{q}_t(z) \to \widehat{q}_t(z) \to \widehat{p}_t^{\mathrm{ref}}(z_t) \to \widetilde{q}_{t+1}(z_{t+1}, z_t), \tag{37.17}$$

with the preconditioned route adding

$$q_t(r) \to q_t^\sharp(u) \to \widehat{\nu}_t^\sharp(u) \to \widehat{q}_t(r). \tag{37.18}$$

The filtering task does not change; only the coordinate system in which the hard part of the density is approximated changes.

## 37.6 TT Sequential Learning And Conditional KR Transports

Zhao and Cui [2024] is the central discrete-time TT source for this lane. Its checked role here is direct: the chapter uses the paper's sequential state-and-parameter learning setup, its posterior-density approximation view, and its conditional Knothe–Rosenblatt map construction from a TT approximation.

A nonnegative TT approximation supplies conditional density factors or conditional cumulative distribution functions. A triangular conditional map can then be constructed from one-dimensional conditional CDFs, which is the KR transport idea in operational form. For BayesFilter this is attractive because it connects low-rank density approximation to transport preconditioning. It remains conditional because normalization, monotonicity, rank error, and Jacobian stability are part of the method, not afterthoughts.

## 37.7 What This Chapter Exports Forward

At the end of this front-half chapter, the retained-object lane has fixed the objects that later operational chapters are allowed to manipulate:

1. the adjacent-state target and its coordinate systems,
2. the TT approximation toolkit and rank-count plausibility logic,
3. the retained numerator $a_t$ and normalized carried filter $\widehat{p}_t$,
4. the KR / preconditioning map perspective,
5. the core diagnostics that decide whether the retained object remains scientifically meaningful.

The next chapter, Chapter 38, then takes up the operational middle: squared-TT recursion, fixed-branch likelihood construction, stored fields, and the exact scalar object exported to the same-scalar derivative chapter.

## 37.8 Tensor-Network Kalman And Square-Root Caution

Tensor-network covariance compression is included here only as a cautionary adjacent lane. If a covariance-like object is compressed, the method must expose either a PSD reconstruction check or a factor/square-root representation whose product is positive semidefinite by construction. This material remains secondary to the retained TT/KR density lane; it is kept because it clarifies one important boundary between density compression and covariance compression.

## 37.9 Complexity, Diagnostics, And Exports

The retained-object lane exports five primary diagnostics:

(1) rank growth,

(2) normalization/mass error,

(3) positivity / nonnegativity construction,

(4) support / Jacobian / map validity,

(5) retained-object consistency for the next time step.

These diagnostics are not side comments; they are the mechanism by which the chapter decides whether the retained object remains scientifically meaningful.

**Exported forward.** The next operational chapter may import only the retained-object scalar ingredients and branch-defining structural choices that this chapter makes explicit. The later validation/promotions chapter may import only the defects, diagnostics, promotion gates, and source-risk boundaries stated here. Neither later chapter should silently strengthen TT/KR transport existence, branch-local differentiability, or retained-object feasibility beyond what the checked sources support.

## 37.10 Methodological Boundary And Sources

The substantive claims in this chapter are tied to TT, KR, transport, and adjacent tensor-network sources where their checked constructions are used. Savostyanov maxvol theory and Knothe's original rearrangement source were not inspected directly here; the chapter therefore uses Oseledets–Tyrtyshnikov for TT-cross and Rosenblatt plus modern transport-filtering papers for triangular maps. No source here is used to claim production readiness, NAWM readiness, posterior accuracy, tensor-method validation, transport-method validation, HMC convergence, GPU/XLA readiness, or default readiness.

# Chapter 38

# Squared-TT Recursion and Fixed-Branch Likelihood Construction

## 38.1 Purpose And Scope

The previous chapter fixed the retained-object front half of the non-Gaussian lane: the exact adjacent-state target, the TT approximation toolkit, the coordinate systems, the KR/preconditioning map logic, and the meaning of the retained marginal $a_t$ and normalized carried filter $\widehat{p}_t$. Those objects answer what the method is trying to represent. The present chapter answers the operational question that follows: once the branch has committed to a finite-dimensional TT/KR construction, what exact approximate likelihood object is produced, what must be saved for the next step, and which numerical fields are part of the scalar identity rather than merely transient implementation detail?

This chapter is therefore the operational middle of the p50 lane. It is not yet the full derivative chapter. It builds the squared-TT recursion, the retained marginal and normalizer, the fixed least-squares branch specialization, and the stored-field contract that later allows the shared same-scalar gradient chapter to differentiate one declared scalar rather than an adaptively changing routine.

Coefficient or audit discrepancy against SGQF diagnoses representation error; it does not alone determine whether a TT is useful for filtering. When the TT supplies a conditional particle proposal, the downstream quantities are the importance-corrected filtering moments and log evidence, particle effective sample size, weight concentration, resampling and ancestor counts. These are compared with an independent reference and simple proposals. SGQF supplies a Gaussian-joint guide and comparator, so a worse fit discrepancy can motivate a repair without precluding that filtering comparison. Particle-proposal evidence must be distinguished from validation of the uncorrected retained TT density and Gram normalizer developed in this chapter: importance corrections change the computation being assessed.

## 38.2 Squared-TT Filtering Object And Recursive Closure

The retained-object lane must answer two questions at the same time:

$$\text{How do we represent } q_t? \qquad \text{How do we integrate the represented object?} \tag{38.1}$$

The exact adjacent-state target is a many-variable function rather than a finite-dimensional Gaussian summary. In the TT/KR lane, the target is first moved into reference coordinates and then approximated by a nonnegative low-rank object, typically through a square or squared-density representation.

Write the represented adjacent-state object in retained coordinates as

$$\widehat{q}_t(z_t, z_{t-1}), \tag{38.2}$$

and define the retained numerator and normalizer by

$$a_t(z_t) = \int \widehat{q}_t(z_t, z_{t-1})\,\mathrm{d}z_{t-1}, \qquad \widehat{Z}_t = \int a_t(z_t)\,\mathrm{d}z_t. \tag{38.3}$$

The carried filter is then

$$\widehat{p}_t(z_t) = \frac{a_t(z_t)}{\widehat{Z}_t}. \tag{38.4}$$

This is the recursive closure the lane must preserve: a represented adjacent object, a retained numerator, a normalizer, and a normalized carried filter that can be queried at the next time step. Saving only samples, only a low-rank core without normalization metadata, or only $\widehat{Z}_t$ is not enough to close the same branch recursively.

## 38.3 What Must Be Saved For The Next Time Step

The next subtlety is what must survive from one time step to the next. If too little is stored, the next target cannot be reconstructed on the same branch. If the wrong object is stored, the implementation may look internally coherent while no longer evaluating the same scalar as the fixed branch.

For a fixed-branch likelihood implementation, the forward object that must be available at time $t+1$ is not merely a cloud or a normalizer. It is the retained filter evaluator and the ingredients required to differentiate that retained filter on the same branch. A useful stored-field contract is

$$\mathcal{F}_t = \{C_{t,k},\ M_{t,>k},\ M_{t,<k},\ \widehat{Z}_t,\ a_t,\ \widehat{p}_t, \Psi_t,\ \lambda_t,\ \tau_t\}_{k=1}^{D}, \tag{38.5}$$

with the derivative chapter later augmenting it by the corresponding dotted objects.

For the derivative-aware fixed branch, the stronger stored-field contract is

$$\dot{\mathcal{F}}_t = \{\dot{C}_{t,k}^{(i)},\ \dot{M}_{t,>k}^{(i)},\ \dot{M}_{t,<k}^{(i)},\ \dot{\widehat{Z}}_t^{(i)},\ \dot{a}_t^{(i)},\ \dot{\widehat{p}}_t^{(i)}\}_{k=1}^{D}, \tag{38.6}$$

so that the next transformed target may be evaluated together with its product-rule sensitivity on the same branch. The carried-filter derivative is the quotient object

$$\partial_{\beta_i}\widehat{p}_t(x_t;\beta) = \frac{\partial_{\beta_i} a_t(x_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(x_t;\beta)\partial_{\beta_i}\widehat{Z}_t(\beta)}{\widehat{Z}_t(\beta)^2}, \tag{38.7}$$

which is precisely the object needed by the next-step fixed-branch derivative pass.

The point is conceptual before it is algorithmic: the next-step target depends on evaluating the previous carried filter at new fitting points. Therefore the retained-object lane must store the represented numerator/filter object in a form that the next-step evaluator can query without rebuilding an adaptive branch. This is what makes the fixed-branch recursion meaningfully recursive rather than a sequence of disconnected local fits.

## 38.4 Fixed-Branch Likelihood Construction

The adaptive Zhao–Cui algorithm contains discrete choices—ranks, fitting points, sweep decisions, domain maps, and stabilization choices—that should not be treated as globally differentiable with respect to the scientific parameter. The fixed-branch specialization freezes those discrete choices first, then asks what scalar is defined when the corresponding numerical equations are recomputed on that same frozen structure.

The fixed branch is therefore more than a density formula. At time $t$, the finite-dimensional branch data may be summarized as

$$\mathcal{B}_t = \{\Omega_{t,k}, \Psi_{t,k}, \psi_{t,k,\ell}, C_{t,k}, R_{t,k}, B_{t,k}, \lambda_{t,k}, \tau_t, c_t, \text{mass contractions}, \text{map data}\}. \tag{38.8}$$

Changing domains, bases, rank tuples, TT cores, reference densities, defensive mass conventions, or log-scale shifts changes the approximate likelihood. Those objects are part of the scalar identity, not optional implementation metadata.

### 38.4.1 What A Core Object Looks Like

For coordinate $k$, store

$$C_k \in \mathbb{R}^{R_{k-1}\times p_k\times R_k}. \tag{38.9}$$

Given a scalar coordinate value $r_k$, first compute the basis vector

$$b_k(r_k) = \big(\psi_{k,0}(r_k), \dots, \psi_{k,p_k-1}(r_k)\big)^\top. \tag{38.10}$$

Then form the matrix slice

$$H_k(r_k)[a,b] = \sum_{\ell=0}^{p_k-1} C_k[a,\ell,b]\, b_k(r_k)[\ell]. \tag{38.11}$$

Evaluation of the TT is the loop

$$v_0 = 1, \qquad v_k = v_{k-1}H_k(r_k), \quad k = 1,\dots,D, \qquad \widehat{h}(r) = v_D. \tag{38.12}$$

The square-density object additionally stores mass contractions

$$M_{>k} = \int H_{k+1}(r_{k+1})\cdots H_D(r_D)H_D(r_D)^\top\cdots H_{k+1}(r_{k+1})^\top \,\mathrm{d}r_{k+1:D}, \tag{38.13}$$

To make the retained marginal explicit, the branch must first fix an ordering convention. Let the current-state block occupy the first $m$ coordinates, so $z_{\text{cur}} = z_{1:m}$ and $z_{\text{prev}} =$

$z_{m+1:D}$. In the adjacent-state order used below, $D = 2m$. Define the retained prefix row vector

$$G_m(z_{1:m}) = H_1(z_1)\cdots H_m(z_m) \in \mathbb{R}^{1\times R_m}. \tag{38.14}$$

The required right contractions integrate only the previous-state suffix:

$$\begin{aligned} M_{>D} = 1, \qquad M_{>j-1}[a,a'] = \sum_{b,b',\ell,\ell'} C_j[a,\ell,b]C_j[a',\ell',b']B_j[\ell,\ell']M_{>j}[b,b'], \\ j = D,\dots,m+1, \end{aligned} \tag{38.15}$$

where $B_j = \int b_j(z)b_j(z)^\top \,\mathrm{d}\mu_j(z)$ is the frozen basis mass matrix under the declared reference measure. Writing

$$\lambda_{t,\mathrm{ret}}(z_{1:m}) = \int \lambda_t(z_{1:m}, z_{m+1:D})\,\mathrm{d}\mu_{m+1:D}, \tag{38.16}$$

the retained numerator is the finite-dimensional contraction

$$a_t(z_{1:m};\beta) = e^{-c_t}\left\{G_m(z_{1:m})M_{>m}G_m(z_{1:m})^\top + \tau_t\lambda_{t,\mathrm{ret}}(z_{1:m})\right\}, \tag{38.17}$$

which is the saved carried object queried by the next step. The defensive term is on the same represented-density scale as the squared contraction because the declared branch target is $\overline{q}_t = e^{-c_t}(\phi_t^2 + \tau_t\lambda_t)$, not a mixed-scale sum.

The same-branch derivative used here freezes the bases, mass matrices, reference measure, domain and coordinate map, $c_t$, $\tau_t$, and the defensive density. Thus

$$\dot{B}_j = 0, \quad \dot{b}_j = 0, \quad \dot{c}_t = 0, \quad \dot{\tau}_t = 0, \quad \dot{\lambda}_{t,\mathrm{ret}} = 0. \tag{38.18}$$

The retained-prefix derivative is

$$\dot{G}_m = \sum_{k=1}^{m} H_1\cdots H_{k-1}\dot{H}_k H_{k+1}\cdots H_m. \tag{38.19}$$

If a different branch makes any object in (38.18) parameter-dependent, the following expression is not its total derivative until the missing product-rule terms are added.

$$\begin{aligned} \dot{M}_{>j-1}[a,a'] = &\sum_{b,b',\ell,\ell'} \dot{C}_j[a,\ell,b]C_j[a',\ell',b']B_j[\ell,\ell']M_{>j}[b,b'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b]\dot{C}_j[a',\ell',b']B_j[\ell,\ell']M_{>j}[b,b'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b]C_j[a',\ell',b']B_j[\ell,\ell']\dot{M}_{>j}[b,b'], \qquad j = D,\dots,m+1, \end{aligned} \tag{38.20}$$

starting from $\dot{M}_{>D} = 0$. The retained-numerator derivative is therefore

$$\dot{a}_t(z_{1:m};\beta) = e^{-c_t}\left\{\dot{G}_m M_{>m}G_m^\top + G_m\dot{M}_{>m}G_m^\top + G_m M_{>m}\dot{G}_m^\top\right\}. \tag{38.21}$$

These are not decorative arrays; together with the complementary sweep contractions used in the updates, they are the finite-dimensional objects through which marginals, normalizers, retained evaluators, and later same-branch derivatives are actually computed.

**Vector retained-prefix certificate.** For the genuine retained-prefix case $m = 2, D = 4$, write

$$G_2(z_{1:2}) = H_1(z_1)H_2(z_2),$$

so the retained numerator is the explicit first-two-block contraction

$$a_t(z_{1:2};\beta) = e^{-c_t}\{G_2(z_{1:2})M_{>2}G_2(z_{1:2})^\top + \tau_t\lambda_{t,\mathrm{ret}}(z_{1:2})\}.$$

A direct integration of the same concrete branch and the right-side recursion must give the same coefficient map for this retained-prefix form. The stored right-sided mass contraction is the branch-specific object that the later fixed-branch derivative chapter differentiates.

A deterministic certificate used the basis $b_j(z) = (1, z)^\top$ under the normalized uniform reference measure on $[-1, 1]$, so $B_j = \mathrm{diag}(1, 1/3)$, together with $e^{-c_t} = 2/3$, $\tau_t = 1/5$, fixed defensive density one, and nontrivial rank-two cores. Direct multivariate-polynomial integration and the right-contraction recursion gave exactly the same coefficient maps. In the scalar case they both gave

$$\begin{aligned} a_t(z_1) &= \frac{196}{45} - \frac{8}{9}z_1 + \frac{26}{9}z_1^2, \\ \dot{a}_t(z_1) &= -8 + \frac{256}{9}z_1 - \frac{80}{9}z_1^2. \end{aligned} \tag{38.22}$$

For the genuinely vector-retained case $m = 2, D = 4$, write $a_t(z_1, z_2) = \sum_{i,j=0}^{2} V_{ij} z_1^i z_2^j$ and similarly for $\dot{a}_t$. Both independent routes gave

$$\begin{aligned} V &= \begin{pmatrix} 8878/135 & 6560/27 & 6644/27 \\ -176/27 & -3488/27 & -6560/27 \\ 68/27 & 176/27 & 1772/27 \end{pmatrix}, \\ \dot{V} &= \begin{pmatrix} 256/3 & 3152/9 & 2644/9 \\ 1264/27 & 3088/27 & 1552/27 \\ 28/27 & -1376/27 & -3368/27 \end{pmatrix}. \end{aligned} \tag{38.23}$$

The exact checker and its JSON output are retained with the rewrite audit artifacts. This certificate proves the finite retained-prefix identities for the two declared fixtures; it does not certify the TT fitting algorithm or an adaptive implementation route.

**Chapter-37 interface note.** Chapter 39 must use the same retained/reference/Jacobian ownership story as this certificate. The saved evaluator consumes retained reference coordinates, the physical-to-reference conversion is owned by the target-construction path, and the route-audit note remains a convention boundary rather than a theorem.

## 38.5 Consolidated Fixed Least-Squares Formulation

The implementable fixed-branch route in BayesFilter should be stated narrowly. It is not adaptive TT-cross. It is a fixed least-squares specialization with fixed points, fixed ranks, fixed sweep order, and deterministic stabilization. In symbols,

fixed-branch TT likelihood route = fixed points+fixed ranks+fixed sweep order+deterministic least sq (38.24)

This narrower statement matters because it isolates the branch whose value and later derivative can be audited. If the implementation silently switches pivot sets, changes rank caps, or changes stabilization rules under perturbation, then it has left the fixed-branch scalar described here.

A representative one-step input/output contract is:

$$\mathcal{I}_t = \{y_t, \beta, \widehat{p}_{t-1}, f_\beta, g_\beta, \Psi_t, Z_{\mathrm{fit}}^{(t)}, W^{(t)}, \mathrm{basis}, \mathrm{ranks}, \mathrm{sweeps}, \rho_t, c_t, \tau_t\}, \tag{38.25}$$

$$\mathcal{O}_t = \{C_{t,k}, M_{t,>k}, M_{t,<k}, \widehat{Z}_t, a_t, \widehat{p}_t\}_{k=1}^{D}. \tag{38.26}$$

The important point is not the exact symbol list but the discipline: the branch must state what is input, what is output, and which outputs are required to close the next-step recursion on the same branch.

A concrete shape contract helps keep this branch auditable. One useful table is

| object | shape |
|---|---|
| $C_k$ | $R_{k-1} \times p_k \times R_k$ |
| $L_{j,k}$ | $1 \times R_{k-1}$ |
| $R_{j,k}$ | $R_k \times 1$ |
| $A_k$ | $n_{\mathrm{fit}} \times (R_{k-1} p_k R_k)$ |
| $g_k = \mathrm{vec}(C_k),\ d_k$ | $R_{k-1} p_k R_k$ |
| $N_k$ | $(R_{k-1} p_k R_k) \times (R_{k-1} p_k R_k)$ |

(38.27)

with vectorization convention

$$\iota_k(a, \ell, b) = (a-1) p_k R_k + \ell R_k + b, \qquad g_k[\iota_k(a, \ell, b)] = C_k[a, \ell, b]. \tag{38.28}$$

This is the level at which the later derivative chapter can say, concretely, what it means to recompute the same least-squares solve rather than copying a stale core from the base parameter.

### 38.5.1 One Concrete Branch, Fully Instantiated

A branch record is easier to audit when one concrete instantiation is stated. For a declared run, that record should pin down:

1. coordinate order and domain policy,
2. basis family and basis degree in each coordinate,
3. deterministic fitting points and weights,
4. rank vector and deterministic rank ladder before branch freezing,
5. sweep order and number of sweeps,
6. defensive reference density, defensive mass, and any frozen shift convention,
7. the boxed convention that the fit is performed in source scale while the reported scalar returns to the unshifted evidence scale.

This list is not busywork. If one of these items changes under perturbation, then the implementation has left the fixed-branch scalar even if all later notation still looks similar.

A useful first mathematical instantiation is intentionally narrow. For a scalar state, use the two-block coordinate order

$$r_t = (x_t, x_{t-1}) \in \mathbb{R}^2. \tag{38.29}$$

For vector states, concatenate coordinates as

$$r_t = (x_{t,1}, \ldots, x_{t,m_x}, x_{t-1,1}, \ldots, x_{t-1,m_x}). \tag{38.30}$$

Choose a fixed interval for each coordinate,

$$r_{t,k} \in [\ell_{t,k}, u_{t,k}], \qquad r_{t,k} = a_{t,k} + h_{t,k} z_k, \qquad a_{t,k} = \frac{u_{t,k} + \ell_{t,k}}{2}, \qquad h_{t,k} = \frac{u_{t,k} - \ell_{t,k}}{2}, \tag{38.31}$$

so the Jacobian is constant,

$$J_t = \prod_{k=1}^{D} h_{t,k}. \tag{38.32}$$

The branch stores $\ell_{t,k}, u_{t,k}, a_{t,k}, h_{t,k}$, and these remain fixed through same-branch finite differences.

A deterministic domain-selection policy should be declared before branch freezing. Let a pilot location and scale be

$$\mu_{t,k}^{\text{pilot}}, \qquad s_{t,k}^{\text{pilot}} > 0, \tag{38.33}$$

and set

$$[\ell_{t,k}, u_{t,k}] = [\mu_{t,k}^{\text{pilot}} - \gamma s_{t,k}^{\text{pilot}}, \mu_{t,k}^{\text{pilot}} + \gamma s_{t,k}^{\text{pilot}}], \tag{38.34}$$

with fixed expansion factor $\gamma$. In the current fixed-variant fitting story, these pilot moments may come from a Gaussian scout such as a UKF warm start. The scout provides a local frame and an initialization region; it is not a certificate that the represented density is already correct. The out-of-domain diagnostic at fitting or proposal points is

$$\delta_{\text{out}} = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}\{r_{t,k}^{(j)} \notin [\ell_{t,k}, u_{t,k}] \text{ for some } k\}, \tag{38.35}$$

and the branch is accepted only if

$$\delta_{\text{out}} \le \delta_{\max}. \tag{38.36}$$

If this fails, the branch must be rebuilt before differentiation rather than expanded during a same-branch derivative check.

Use one declared basis family and fixed point design. A useful first choice is normalized Legendre basis of common degree $p$ in every coordinate together with deterministic Halton-type fitting points and equal weights. The rank vector is then selected by a small deterministic ladder before branch freezing, for example

$$\mathcal{R}_{\text{ladder}} = \{(1, \ldots, 1), (2, \ldots, 2), (4, \ldots, 4), (6, \ldots, 6), (8, \ldots, 8)\}, \tag{38.37}$$

with the first candidate passing the declared residual, conditioning, and defensive-mass tests becoming the frozen branch rank. If no candidate passes, the correct outcome is branch failure, not an unrecorded adaptive rank change.

The defensive reference and shift convention should also be fixed explicitly. A minimal deterministic choice is uniform reference density on $[-1,1]^D$, a source-scale floor $\epsilon_{\text{floor}} > 0$, defensive mass $\tau_t = 2^D \epsilon_{\text{floor}}$, and shift $c_t$ computed once at the base parameter and then reused for same-branch finite differences. This is the meaning of the chapter's boxed convention that the fit is carried out in source scale while the reported scalar returns to the unshifted evidence scale.

### 38.5.2 Square-Root Fitting Object Versus Density Audit Object

The fixed-variant fitting lane needs one further distinction. The TT sweep may be written in square-root form, but the scientific object being evaluated is a density. Write the represented nonnegative object as

$$q_\theta(z) = h_\theta(z)^2 + \tau q_0(z), \qquad Z_\theta = \int q_\theta(z)\,\mathrm{d}z, \qquad p_\theta(z) = \frac{q_\theta(z)}{Z_\theta}, \tag{38.38}$$

where $h_\theta$ is the trainable square-root TT field, $q_0$ is the fixed defensive reference density, and $\tau > 0$ is the defensive mixture weight. If the target square-root field is $s(z)$, then the target density is

$$p_\star(z) \propto s(z)^2. \tag{38.39}$$

Therefore a raw square-root residual,

$$h_\theta(z_i) \approx s(z_i), \tag{38.40}$$

is not yet the primary heldout criterion. The reported density depends on the square, the defensive component, and the normalizer $Z_\theta$. Sign, scale, and local amplitude errors in $h_\theta$ need not map one-for-one to density quality after normalization. Square-root residuals are therefore fitting diagnostics, not promotion criteria by themselves.

For a weighted batch

$$B = \{(z_i, w_i, s_i)\}_{i=1}^n, \tag{38.41}$$

the target-only empirical measure is

$$\alpha_i^{\text{hold}} = \frac{w_i s_i^2}{\sum_j w_j s_j^2}, \tag{38.42}$$

and the corresponding density cross-entropy is

$$\widehat{\mathcal{L}}_{\text{hold}}(\theta) = -\sum_i \alpha_i^{\text{hold}} \log q_\theta(z_i) + \log Z_\theta. \tag{38.43}$$

This target-only measure is the corrected heldout audit object for the fixed-variant lane. The historical helper

$$\alpha_i^{\text{old}} \propto w_i\big(s_i^2 + \tau q_0(z_i)\big) \tag{38.44}$$

mixes the defensive reference density into the empirical target. That helper may still be useful as a boundary comparator or internal training-support convention, but it is not

the heldout target measure and should not be reported as if it were the scientific density criterion.

This correction also changes how fitting evidence should be narrated. A UKF warm start can supply local geometry and a mini-batch training run can improve the represented density over that geometry, but success is judged by the target-only heldout cross-entropy in (38.43), not by the old $\tau q_0$-smoothed helper and not by square-root residuals alone.

## 38.6 Initialization And Sweep Order

The branch must also save the deterministic initialization and sweep rule. A constant-channel initialization is a useful mechanical baseline, but it should not be confused with the preferred fixed-variant fitting narrative once a deterministic geometric warm start is available. When a Gaussian scout such as a UKF provides the pilot frame, the current mathematical story is that the branch record should freeze a warm start derived from that scout rather than revert to random or purely constant initialization. Let one bidirectional sweep mean updates in the order

$$1, 2, \dots, D, D-1, \dots, 1. \tag{38.45}$$

Fix the number of bidirectional sweeps $S$. The exact deterministic initialization rule matters less than saving it in the branch record, because changing initialization changes the fixed fitting map.

### 38.6.1 Forward Core Update

At fitting point $z^{(j)}$, define environments for core $k$ by

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \qquad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}). \tag{38.46}$$

For endpoint ranks this means $L_{j,k} \in \mathbb{R}^{1\times R_{k-1}}$ and $R_{j,k} \in \mathbb{R}^{R_k\times 1}$. The design row for coefficient $(a, \ell, b)$ is

$$A_k[j, (a, \ell, b)] = L_{j,k}[1, a]\, b_k(z_k^{(j)})[\ell] R_{j,k}[b, 1]. \tag{38.47}$$

The local ridge system is

$$N_k g_k = d_k, \qquad N_k = A_k^\top W A_k + \rho I, \qquad d_k = A_k^\top W y. \tag{38.48}$$

After solving (38.48), reshape $g_k$ back into the core by

$$C_k[a, \ell, b] = g_k[(a, \ell, b)]. \tag{38.49}$$

This is the forward fitting rule whose identity later same-branch finite differences must preserve.

### 38.6.2 Derivative Core Update

The derivative environments are obtained by differentiating the matrix products:

$$\dot{L}_{j,k} = \sum_{r=1}^{k-1} H_1 \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_{k-1}, \tag{38.50}$$

$$\dot{R}_{j,k} = \sum_{r=k+1}^{D} H_{k+1} \cdots H_{r-1} \dot{H}_r H_{r+1} \cdots H_D. \tag{38.51}$$

Because basis functions and fitting points are frozen, the derivative design row contains only environment terms:

$$\begin{aligned} \dot{A}_k[j,(a,\ell,b)] = {} & \dot{L}_{j,k}[1,a] b_k(z_k^{(j)})[\ell] R_{j,k}[b,1] \\ & + L_{j,k}[1,a] b_k(z_k^{(j)})[\ell] \dot{R}_{j,k}[b,1]. \end{aligned} \tag{38.52}$$

The derivative normal equations are

$$\dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k, \qquad \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}, \tag{38.53}$$

and differentiating $N_k g_k = d_k$ gives

$$N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{38.54}$$

After solving (38.54), reshape

$$\dot{C}_k[a,\ell,b] = \dot{g}_k[(a,\ell,b)]. \tag{38.55}$$

This is the derivative fitting rule. A later implementation must use this same stored computational path rather than inventing a separate approximate derivative solve.

### 38.6.3 Alternating-Sweep Recomputations

The symbols $L_{j,k}$, $R_{j,k}$, $A_k$, $N_k$, and $d_k$ are not static arrays during an alternating sweep. They are recomputed from the current cores at the moment a core is updated. Let

$$H_{j,k} = H_k(z_k^{(j)}; C_k), \qquad \dot{H}_{j,k} = H_k(z_k^{(j)}; \dot{C}_k), \tag{38.56}$$

where basis values and fitting points are fixed and only the coefficients change. For a current core collection $C^{\mathrm{cur}}$, define prefix and suffix products by

$$L_{j,1}^{\mathrm{cur}} = 1, \qquad L_{j,k}^{\mathrm{cur}} = L_{j,k-1}^{\mathrm{cur}} H_{j,k-1}^{\mathrm{cur}}, \qquad k = 2, \ldots, D, \tag{38.57}$$

and

$$R_{j,D}^{\mathrm{cur}} = 1, \qquad R_{j,k}^{\mathrm{cur}} = H_{j,k+1}^{\mathrm{cur}} R_{j,k+1}^{\mathrm{cur}}, \qquad k = D-1, \ldots, 1. \tag{38.58}$$

The dotted prefix and suffix products follow the same recursions with one product-rule term added. At a core update $k$, build the current design matrix and normal equations from the current environments, solve for the new core, then build the dotted objects and solve the derivative system. After updating one core, every cached prefix with index $\ell > k$ and every cached suffix with index $\ell < k$ is stale and must be recomputed before use.

This stale-cache rule is part of the branch definition. It is what makes the phrase “the same fitting rule” operational rather than rhetorical.

### 38.6.4 Stopping Rule And Failure Exits

After each full sweep compute

$$\varepsilon_s = \frac{\|A(C^{(s)}) - y\|_W}{\max\{\|y\|_W, \epsilon_{\text{abs}}\}}, \qquad \Delta_s = \frac{\|C^{(s)} - C^{(s-1)}\|}{\max\{\|C^{(s-1)}\|, \epsilon_{\text{abs}}\}}, \tag{38.59}$$

The sweep stops successfully only if

$$\varepsilon_s \le \epsilon_{\text{fit}} \quad \text{and} \quad \Delta_s \le \epsilon_{\text{sweep}}. \tag{38.60}$$

If this does not happen by $S$ sweeps, the branch reports sweep failure rather than silently accepting a different scalar. The full accepted/failure path must therefore distinguish at least the following exits:

| Failure condition | Mathematical test | Report field |
|---|---|---|
| Domain miss | $\delta_{\text{out}} > \delta_{\max}$ | domain failure |
| Rank ladder fail | no $R$ satisfies residual/conditioning/floor tests | rank failure |
| Ill-conditioned solve | $\text{cond}(N_k) > \kappa_{\max}$ | solve failure |
| Sweep stagnation | $\varepsilon_s > \epsilon_{\text{fit}}$ or $\Delta_s > \epsilon_{\text{sweep}}$ after $S$ sweeps | sweep failure |
| Nonpositive target for smooth derivative | $\min_j \widetilde{q}_{t,j} \le 0$ without declared floor | target failure |
| Mass failure | $\widehat{Z}_t \le 0$ or not finite | normalizer failure |
| Finite-difference branch mismatch | frozen fields or accepted/failure path differ across $\beta - h, \beta, \beta + h$ | branch mismatch |

A failed branch is not a successful derivative with a warning attached; it is a declared failure of the fixed scalar. This accepted/failure vocabulary is what staged *ch*37 later consumes when it says a branch-valid FD row requires the same realized acceptance/failure path on both sides of the perturbation.

## 38.7 One Observation Step Through The Stored Objects

A reader should be able to narrate one time step in the following order:

1. evaluate the previous carried filter on the current branch's fitting points,
2. combine it with the transition and observation densities to form the adjacent-state target in the declared coordinates,
3. fit the squared-TT / low-rank object on the fixed branch,
4. contract out the previous-state block to form $a_t$,
5. compute $\widehat{Z}_t$ and normalize to $\widehat{p}_t$,

6. save the branch fields needed for the next-step forward evaluation and later same-branch derivative evaluation.

The dependency flow should be explicit enough to implement:

$$\begin{aligned}
\text{reference points} &\longrightarrow z^{(j)} \text{ from the fixed point design},\\
\text{physical points} &\longrightarrow r^{(j)} = \Psi_t(z^{(j)}),\\
\text{target values} &\longrightarrow \widetilde{q}_{t,j} = \widehat{p}_{t-1}(x_{t-1}^{(j)};\beta) f_\beta(x_t^{(j)} \mid x_{t-1}^{(j)}) g_\beta(y_t \mid x_t^{(j)}) J_t,\\
\text{square-root fit values} &\longrightarrow y_j = e^{c_t/2}\sqrt{\widetilde{q}_{t,j}},\\
\text{initial cores} &\longrightarrow \text{deterministic constant-channel initialization},\\
\text{fixed sweep objects} &\longrightarrow L_{j,k}, R_{j,k}, A_k, N_k, d_k, C_k,\\
\text{mass objects} &\longrightarrow M_{>k}, M_{<k}, R_t, \widehat{Z}_t,\\
\text{carried objects} &\longrightarrow a_t, \widehat{p}_t,\\
\text{saved objects for } t+1 &\longrightarrow \mathcal{F}_t \text{ and, when needed, } \dot{\mathcal{F}}_t.
\end{aligned} \tag{38.61}$$

The forward environments are recomputed from the current cores during each sweep; they are not static arrays. For fitting point $z^{(j)}$, the environments are

$$L_{j,k} = H_1(z_1^{(j)}) \cdots H_{k-1}(z_{k-1}^{(j)}), \qquad R_{j,k} = H_{k+1}(z_{k+1}^{(j)}) \cdots H_D(z_D^{(j)}), \tag{38.62}$$

and the local ridge system is built from those current environments. After a core update, cached prefixes and suffixes on the wrong side of that core are stale and must be recomputed before use. That bookkeeping is part of the fixed fitting map.

The same principle governs next-step closure. The next transformed target is not formed from a vague "previous TT object," but from the saved carried evaluator:

$$\widetilde{q}_{t+1}(z;\beta) = \widehat{p}_t(x_t(z);\beta) f_\beta(x_{t+1}(z) \mid x_t(z)) g_\beta(y_{t+1} \mid x_{t+1}(z)) J_{t+1}(z). \tag{38.63}$$

If the derivative lane is active, the saved dotted object from (38.6) is what makes the next-step product-rule derivative possible on the same branch. Without that saved object, a later derivative pass would be differentiating a different computational route.

This is the operational middle that the monograph must carry so that the later derivative chapter can start from a declared scalar object rather than a vague reference to "the TT filter."

## 38.8 Exports To The Same-Scalar Derivative Chapter

This chapter exports the following objects forward:

1. the represented adjacent-state object and retained marginal recursion,
2. the stored-field contract for next-step closure,
3. the fixed-branch scalar $\widehat{\ell}_T^{\mathrm{TT/KR}}(\beta; B)$,
4. the fixed-branch least-squares specialization,
5. the lane-specific statement that domains, points, ranks, maps, stabilization rules, and stored-field conventions belong to the scalar identity.

The next chapter may now focus on derivative theory, branch-valid finite differences, and HMC-facing same-scalar discipline, because the present chapter has already fixed what scalar object is being differentiated.

# Chapter 39

# Fixed-Branch Approximate Likelihoods and Same-Scalar Gradients

## 39.1 Why Fixed-Branch Discipline Is Shared Across The Two Lanes

Both method chapters in the restarted high-dimensional block face the same structural problem. The exact Bayesian target is a filtering law and normalizer sequence, but the implemented approximation uses discrete numerical structure that may change with parameters: cloud level, merge policy, factor branch, coordinate domains, ranks, pivots, grids, fitting points, or map choices. If those structural choices move while the parameter moves, then the reported derivative is not automatically the derivative of one declared scalar.

This chapter therefore treats fixed-branch discipline as a cross-method target contract, not as an appendix to one particular algorithm.

## 39.2 A Canonical Branch Record

A branch record should be read as a three-ledger object,

$$\mathcal{H}(B) = \{\mathcal{H}_{\text{fixed}}, \mathcal{H}_{\text{diff}}, \mathcal{H}_{\text{diag}}\}, \tag{39.1}$$

where:

- $\mathcal{H}_{\text{fixed}}$ contains the structural choices that define the scalar on the saved branch,
- $\mathcal{H}_{\text{diff}}(\theta)$ contains the parameter-dependent numerical values recomputed on that branch,
- $\mathcal{H}_{\text{diag}}$ contains residuals, condition numbers, rank vectors, branch-failure markers, and other diagnostic quantities.

For the sparse-grid lane, the fixed ledger includes the cloud, one-dimensional rule family, merge/order policy, factor family, thresholds, preprocessing policy, and initial-condition policy. For the retained-object lane, the fixed ledger includes coordinate domains, basis

families, fitting points, weights, ranks, sweep order, shifts, stabilization choices, retained-object construction policy, and the defensive mass / reference-density conventions that define the same branch.

Two evaluations use the same branch if and only if their fixed ledgers match:

$$B(\theta_a) = B(\theta_b) \quad \Longleftrightarrow \quad \mathcal{H}_{\text{fixed}}(\theta_a) = \mathcal{H}_{\text{fixed}}(\theta_b). \tag{39.2}$$

The differentiable ledger is allowed to change with the parameter; otherwise the same-scalar derivative would not be testing the actual fixed branch.

## 39.3 Fixed-Branch Approximate Likelihoods Across The Three Operational Layers

The restarted high-dimensional part now separates the operational burdens that the old compressed block used to mix together. The deterministic sparse-grid lane exports a declared fixed-cloud scalar from Chapter 36. The retained-object TT/KR lane exports a declared fixed-branch scalar from Chapter 38. The present chapter sits above those lane-specific constructions and states the common same-scalar derivative logic they must both obey.

The three operational layers are:

$$\begin{aligned} \text{exact target:} \quad & \ell_T(\theta) = \sum_{t=1}^{T} \log Z_t(\theta), \\ \text{declared approximate scalar:} \quad & \widehat{\ell}_T(\theta; B), \\ \text{same-scalar derivative requirement:} \quad & \nabla_\theta \widehat{\ell}_T(\theta; B) \text{ differentiates the same declared } \widehat{\ell}_T. \end{aligned} \tag{39.3}$$

For the sparse-grid lane, $B$ contains the fixed cloud, merge/order policy, factor branch, and lane-specific veto rules. For the retained-object lane, $B$ contains the fixed domains, fitting points, ranks, sweeps, shifts, stabilization choices, and stored-field conventions that define the TT/KR scalar. The chapter therefore treats fixed-branch discipline as a cross-method target contract, not as an appendix to one particular algorithm.

For the actual-SV family, the ordering of obligations is now explicit. First one declares the target-correct scalar. For the transformed actual-SV problem that scalar is the cumulative likelihood built from the exact transformed normalizers,

$$\widehat{\ell}_T^{\,\text{sv}}(\theta; B) = \sum_{t=1}^{T} \log \widehat{Z}_t^{\,\text{sv}}(\theta; B), \qquad \widehat{Z}_t^{\,\text{sv}}(\theta; B) \approx Z_t(\theta).$$

Only after that declaration does the same-scalar derivative contract become meaningful. Direct-likelihood quadrature, fixed-cloud reweighting, and Zhao–Cui comparisons are legitimate only when they approximate this same normalizer sequence. The previous actual-SV "Lane B" framing violated that ordering by differentiating a Gaussian-closure replacement scalar built from predictive observation moments of an augmented variable. That scalar was not a second admissible target for the intended inference problem; it was the wrong scalar. A finite-difference or autodiff check on that wrong scalar can show only internal consistency of the replacement construction, not correctness for the actual-SV likelihood.

## 39.4 Finite-Difference Contract And Branch-Validity Logic

The finite-difference contract is

$$D_i(h) = \frac{\widehat{\ell}_T(\theta_0 + he_i; B) - \widehat{\ell}_T(\theta_0 - he_i; B)}{2h}, \tag{39.4}$$

with both perturbed evaluations reusing the same fixed ledger while recomputing all parameter-dependent numerical values through the same declared equations.

For the retained-object lane, this means the domains, points, ranks, shifts, and sweep order are unchanged, but the fitted core values

$$C_{t,k}(\theta_0 + he_i; B) \quad \text{and} \quad C_{t,k}(\theta_0 - he_i; B) \tag{39.5}$$

are recomputed from the same saved fitting rule. They are not copied from the base parameter. A row is branch-valid only if the two perturbations share:

(i) the same saved fixed ledger, and

(ii) the same realized acceptance/failure path through the declared branch logic.

If either side changes the accepted/failure pattern, the row is branch-invalid and is not evidence about the derivative of the same scalar.

**Copied-core and branch-identity audit protocol.** The operational diagnostic should promote these prose rules into explicit indicators. For a step ladder $h_i$, define

$$I_{\pm}(h_i) = \mathbf{1}\{\mathcal{H}_{\text{fixed}}(\theta_0 \pm h_i e_j) = \mathcal{H}_{\text{fixed}}(\theta_0)\}, \tag{39.6}$$

which checks branch-identity equality of the frozen fields, and

$$K_{\pm}(h_i) = \mathbf{1}\{C_{t,k}(\theta_0 \pm h_i e_j; B) \text{ was recomputed by the fixed fitting rule}\}, \tag{39.7}$$

which checks that perturbed cores were not copied from the base parameter. Record a decreasing-window indicator

$$W_i = \mathbf{1}\{e(h_i) > e(h_{i+1}) > e(h_{i+2})\}, \tag{39.8}$$

and classify the diagnostic status as one of:

$$\text{status} = \begin{cases} \text{branch identity failure}, & \min_{i,\pm} I_{\pm}(h_i) = 0, \\ \text{copied-core failure}, & \min_{i,\pm} K_{\pm}(h_i) = 0, \\ \text{decreasing-window agreement}, & \max_i W_i = 1, \\ \text{inconclusive: no decreasing window}, & \text{otherwise.} \end{cases} \tag{39.9}$$

This makes the same-branch contract auditable. If a frozen field differs, the centered difference is comparing different functions. If a differentiable field is copied rather than recomputed, the centered difference is not evaluating the fixed fitting rule.

## 39.5 Value Path, Derivative Path, And What Is Actually Differentiated

The implementation burden of a same-scalar chapter is not just to state that a gradient exists. It must say which objects belong to the value path and which objects belong to the derivative path. A generic fixed-branch workflow has the shape

$$\begin{aligned} &\text{value path:} && \text{frozen structure} \to \widehat{Z}_t \to \widehat{\ell}_T, \\ &\text{derivative path:} && \text{same frozen structure} \to \dot{\widehat{Z}}_t \to \partial_i \widehat{\ell}_T. \end{aligned} \tag{39.10}$$

For the sparse-grid lane this means differentiating the same fixed cloud and the same branch-conditioned Gaussian value path. For the retained-object lane it means differentiating the same retained-object fitting equations, mass contractions, quotient updates, and carried-filter update equations without changing domains, points, ranks, shifts, or sweep order.

The most important implementation distinction is between:

- objects needed to close the current scalar derivative, and
- objects needed only to propagate the branch-consistent state to the next step.

This distinction is what keeps a same-scalar derivative from being confused with a numerically similar but structurally different adaptive algorithm.

**Retained-object derivative mechanics on the frozen branch.** For the TT/KR lane, the forward and derivative paths should be read as the same finite sequence of fixed equations with different outputs. A useful teaching layer is:

| Forward scalar path | Derivative path |
|---|---|
| $\widetilde{q}_{t,j} = \widehat{p}_{t-1,j} f_j g_j J_j$ | $\dot{\widetilde{q}}_{t,j} = J_j(\dot{\widehat{p}}_{t-1,j} f_j g_j + \widehat{p}_{t-1,j} \dot{f}_j g_j + \widehat{p}_{t-1,j} f_j \dot{g}_j)$ |
| $y_j = e^{c_t/2}\sqrt{\widetilde{q}_{t,j}}$ | $\dot{y}_j = e^{c_t/2}\dot{\widetilde{q}}_{t,j}/(2\sqrt{\widetilde{q}_{t,j}})$ |
| $A_k \to N_k = A_k^\top W A_k + \rho I,\ d_k = A_k^\top W y$ | $\dot{A}_k \to \dot{N}_k = \dot{A}_k^\top W A_k + A_k^\top W \dot{A}_k,\ \dot{d}_k = \dot{A}_k^\top W y + A_k^\top W \dot{y}$ |
| $N_k g_k = d_k$ | $N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k$ |
| $g_k \to C_k \to M_{>k} \to \widehat{Z}_t$ | $\dot{g}_k \to \dot{C}_k \to \dot{M}_{>k} \to \dot{\widehat{Z}}_t$ |
| $\widehat{p}_t = a_t/\widehat{Z}_t$ | $\dot{\widehat{p}}_t = \dot{a}_t/\widehat{Z}_t - a_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2$ |

(39.11)

The fixed linear solve is the key finite-dimensional derivative step:

$$N_k g_k = d_k \quad \Longrightarrow \quad N_k \dot{g}_k = \dot{d}_k - \dot{N}_k g_k. \tag{39.12}$$

Computation should solve (39.12) with the same numerical linear algebra used in the forward pass; it should not form an explicit inverse. After the core derivative is obtained, the remaining objects follow by repeated product-rule and quotient-rule propagation through the same frozen branch.

**Mass-contraction and normalizer block.** The squared-TT normalizer is not a separate conceptual object from the fitted cores; it is a contraction of those same cores against the frozen basis mass matrices. Let the right mass contraction satisfy a recursion of the form

$$M_{>j-1}[a,a'] = \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'], \qquad j = D,\dots,m{+}1, \tag{39.13}$$

for a retained prefix of dimension $m$, with basis mass matrix $B_j$ frozen by the branch. Continuing the same recursion through $j = 1$ gives the full square mass. Differentiate the suffix recursion by the product rule:

$$\begin{aligned}\dot{M}_{>j-1}[a,a'] = &\sum_{b,b',\ell,\ell'} \dot{C}_j[a,\ell,b] M_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] \dot{M}_{>j}[b,b'] C_j[a',\ell',b'] B_j[\ell,\ell'] \\ &+ \sum_{b,b',\ell,\ell'} C_j[a,\ell,b] M_{>j}[b,b'] \dot{C}_j[a',\ell',b'] B_j[\ell,\ell'], \qquad j = D,\dots,m+1.\end{aligned} \tag{39.14}$$

Starting from the terminal contraction, this recursion produces the square mass $R_t$ and its derivative $\dot{R}_t$. Assume here that the fixed defensive density is normalized under the declared full reference measure, $\int \lambda_t \, \mathrm{d}\mu_{1:D} = 1$. If

$$\zeta_t = R_t + \tau_t, \qquad \widehat{Z}_t = e^{-c_t} \zeta_t, \tag{39.15}$$

then, with $c_t$ and $\tau_t$ frozen on the branch,

$$\dot{\widehat{Z}}_t = e^{-c_t} \dot{R}_t. \tag{39.16}$$

This is the mass-contraction block the derivative must preserve. Omitting one of the product-rule terms in (39.14) means the reported gradient no longer differentiates the same scalar normalizer.

**Carried-filter and next-step closure block.** The retained numerator is another contraction of the same represented adjacent-state object. Write

$$a_t(z_t;\beta) = \int \widehat{q}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}, \qquad \dot{a}_t(z_t;\beta) = \int \dot{\widehat{q}}_t(z_t, z_{t-1};\beta) \, \mathrm{d}z_{t-1}. \tag{39.17}$$

The normalized carried filter then satisfies the quotient rule

$$\dot{\widehat{p}}_t(z_t;\beta) = \frac{\dot{a}_t(z_t;\beta)}{\widehat{Z}_t(\beta)} - \frac{a_t(z_t;\beta)\dot{\widehat{Z}}_t(\beta)}{\widehat{Z}_t(\beta)^2}. \tag{39.18}$$

This is the crucial bridge between time steps. At time $t+1$, the fixed-branch target contains $\widehat{p}_t$, so its derivative contains $\dot{\widehat{p}}_t$. Without saving the carried-filter derivative, the next-step gradient is incomplete even if the current-step score is numerically correct.

**Saved retained-filter object and query rule.** The phrase "save the carried object" must denote a finite-dimensional evaluator, not an informal function name. For retained state dimension $m > 1$, let the retained block use the product basis

$$b_t(z_t) = b_{t,1}(z_{t,1}) \otimes \cdots \otimes b_{t,m}(z_{t,m}) \in \mathbb{R}^{p_{\mathrm{ret}}}, \qquad p_{\mathrm{ret}} = \prod_{j=1}^{m} p_j. \tag{39.19}$$

The link from the retained TT prefix to a coefficient matrix must be explicit. Because the bases form a tensor-product basis, there is a matrix $A_t \in \mathbb{R}^{p_{\mathrm{ret}} \times R_m}$ satisfying

$$G_m(z_t;\beta) = b_t(z_t)^\top A_t(\beta), \qquad \dot{G}_m(z_t;\beta) = b_t(z_t)^\top \dot{A}_t(\beta). \tag{39.20}$$

The current fixed branch also requires the retained defensive density to have a declared coefficient matrix

$$\lambda_{t,\mathrm{ret}}(z_t) = b_t(z_t)^\top \Lambda_t^{\mathrm{ret}} b_t(z_t), \qquad \dot{\Lambda}_t^{\mathrm{ret}} = 0. \tag{39.21}$$

If the selected defensive density has no exact representation of this form, the implementation must store it as a separate evaluator; it may not silently drop the defensive contribution from a $Q_t$-only saved object.

With $c_t, \tau_t, \Lambda_t^{\mathrm{ret}}$, the bases, and the reference measure frozen, the coefficient matrices derived from the Chapter 36b contraction are

$$\begin{aligned} &Q_t, \dot{Q}_t, \Lambda_t^{\mathrm{ret}} \in \mathbb{R}^{p_{\mathrm{ret}} \times p_{\mathrm{ret}}}, \\ &Q_t = e^{-c_t}\left(A_t M_{>m} A_t^\top + \tau_t \Lambda_t^{\mathrm{ret}}\right), \\ &\dot{Q}_t = e^{-c_t}\left(\dot{A}_t M_{>m} A_t^\top + A_t \dot{M}_{>m} A_t^\top + A_t M_{>m} \dot{A}_t^\top\right). \end{aligned} \tag{39.22}$$

Consequently, the retained numerator and its derivative use the matching quadratic evaluators. Here $b_t(z_t) \in \mathbb{R}^{p_{\mathrm{ret}}}$ is a column vector, so every product below is scalar:

$$a_t(z_t;\beta) = b_t(z_t)^\top Q_t(\beta) b_t(z_t), \qquad \dot{a}_t(z_t;\beta) = b_t(z_t)^\top \dot{Q}_t(\beta) b_t(z_t), \tag{39.23}$$

Normalize the stored evaluator by

$$P_t = \frac{Q_t}{\widehat{Z}_t}, \qquad \dot{P}_t = \frac{\dot{Q}_t}{\widehat{Z}_t} - \frac{Q_t \dot{\widehat{Z}}_t}{\widehat{Z}_t^2}. \tag{39.24}$$

For next-step query points $z_t^{\mathrm{query}}(j)$, build the retained-basis matrix

$$B^{\mathrm{query}}[j,:] = b_t(z_t^{\mathrm{query}}(j))^\top, \qquad b_t(z_t^{\mathrm{query}}(j))^\top \in \mathbb{R}^{1 \times p_{\mathrm{ret}}}, \qquad B^{\mathrm{query}} \in \mathbb{R}^{M \times p_{\mathrm{ret}}}, \tag{39.25}$$

and evaluate the saved carried object by

$$\widehat{p}_t^{\mathrm{ref}}[j] = B^{\mathrm{query}}[j,:] P_t B^{\mathrm{query}}[j,:]^\top, \qquad \dot{\widehat{p}}_t^{\mathrm{ref}}[j] = B^{\mathrm{query}}[j,:] \dot{P}_t B^{\mathrm{query}}[j,:]^\top. \tag{39.26}$$

The next-step query rule must also state the coordinate system and Jacobian ownership explicitly. In the current fixed-adjacent route, the stored carried marginal is a density under the retained-block *reference* measure, and the saved evaluator therefore consumes retained reference coordinates directly. Physical fitting points are converted to reference coordinates by the route's coordinate map before the saved evaluator is queried. The Jacobian of that physical-to-reference conversion is owned by the step-(t+1) target-construction path, not by the stored evaluator itself. Thus the next-step query rule is that $z_t^{\mathrm{query}}(j)$ denotes the previous-state block of the fixed branch's fitting point *after conversion into the retained reference coordinates on which* (39.26) *is defined*. A later implementation may store $Q_t$ in TT, low-rank, or sparse form, but only if it preserves the same evaluator and derivative evaluator in (39.26). This is the stored-field convention that closes the derivative recursion on the same branch.

**Route-audit note.** The release-oriented text above is a contract statement rather than a source or code audit. A final promotion should verify that the actual fixed-branch route uses the same reference measure and conversion ownership before this paragraph is promoted beyond a documented convention.

**Branch-manifest reporting hook.** Derivative and finite-difference reports for this lane should therefore carry a TT/KR branch-manifest identifier together with the saved-field convention, so that branch-valid rows certify equality of the actual retained evaluator rather than only superficial notation.

**Compact derivative-reading checklists.** A later implementation review should be able to verify the derivative lane block by block:

$$\begin{aligned}
&\text{squared-density block:} && \overline{q}_t = e^{-c_t}(\phi_t^2 + \tau_t\lambda_t),\ \partial(\phi_t^2) = 2\phi_t\dot{\phi}_t;\\
&\text{mass-contraction block:} && R_t = \textstyle\int \phi_t^2,\ \dot{R}_t \text{ from product-rule contractions;}\\
&\text{fixed-solve block:} && N_k g_k = d_k \ \mapsto\ N_k\dot{g}_k = \dot{d}_k - \dot{N}_k g_k;\\
&\text{carried-filter block:} && \widehat{p}_t = a_t/\widehat{Z}_t,\ \dot{\widehat{p}}_t = \dot{a}_t/\widehat{Z}_t - a_t\dot{\widehat{Z}}_t/\widehat{Z}_t^2.
\end{aligned} \tag{39.27}$$

The corresponding failure-mode summaries are:

- **squared-density failure:** differentiating a different fitted square root than the one actually squared,
- **mass-contraction failure:** omitting a product-rule term in the contraction recursion,
- **fixed-solve failure:** changing the normal equations or copying a stale core at perturbed parameters,
- **carried-filter failure:** feeding time $t+1$ without the saved dotted retained object or with an undeclared query rule.

These lists are not replacement derivations; they are compact audit objects that make it harder for a future implementation to silently drift away from the mathematical branch defined here.

## 39.6 Approximate Targets, Exact-Target Corrections, And HMC Admissibility

For an unconstrained coordinate $q \in \mathbb{R}^d$ and transform $\theta = \tau(q)$, the exact-target HMC potential is

$$U(q) = -\ell_T(\tau(q)) - \log \pi(\tau(q)) - \log\left|\det D\tau(q)\right|. \tag{39.28}$$

If the filter supplies only $\widehat{\ell}_T$, then ordinary HMC samples the declared approximate posterior defined by $\widehat{\ell}_T$. Recovering the exact posterior from an approximate or randomized likelihood would require a separately defined exact-target extended-state or Metropolis correction scheme; such a scheme is outside the scope of this chapter.

The HMC admissibility burden is therefore downstream of same-scalar discipline:

(1) the filter must define the scalar likelihood or approximate likelihood before any sampler can be interpreted,

(2) the gradient used by HMC must be the gradient of the same scalar used in the Metropolis correction and posterior interpretation,

(3) transformed targets, including TT/KR-style maps, require the Jacobian-corrected target and valid support.

## 39.7 Exports To Validation And Promotion Chapters

The later validation chapter may import only the following consequences from the present chapter:

- the branch-valid finite-difference contract,
- the distinction between fixed structural ledgers and recomputed differentiable ledgers,
- the exact-target versus approximate-target HMC admissibility boundary,
- the rule that divergences, nonfinite gradients, and same-scalar mismatch are veto diagnostics rather than efficiency datapoints.

## 39.8 Implementation Boundary

BayesFilter has no nonlinear HMC convergence result in this lane, no NeuTra backend, no RMHMC backend, no tensor/KR transport-preconditioned HMC backend, and no NAWM posterior recovery benchmark. This chapter defines a shared branch, scalar, and derivative contract. It does not promote any sampler or backend as production-ready for nonlinear state-space models.

# Chapter 40

# Validation, Defect Calculus, and Method Promotion

This chapter closes the restarted high-dimensional block. It synthesizes the validation and promotion logic shared by the deterministic Gaussian/sparse-grid lane and the non-Gaussian low-rank retained-object lane. Its purpose is not to re-derive the methods, but to consume their exported defects, diagnostics, approximate-target contracts, and HMC-admissibility consequences.

The previous chapters have already separated:

- exact target from approximate target,
- carried object from moment or retained-object engine,
- frozen branch-defining structure from recomputed numerical values.

The role here is therefore downstream and comparative: which diagnostics veto a method, which diagnostics only explain it, and under what conditions a chapter- local approximation becomes an industrially credible component rather than a merely interesting computation.

## 40.1 Validation Architecture And Benchmark Roles

The restarted high-dimensional block uses three ledgers of evidence that must not be conflated:

(1) **Engineering correctness.** Did the declared scalar build, factor, and update without violating its own branch rules?

(2) **Numerical or sampler validity.** Did finite-difference parity, ESS, PSD, rank, support, Jacobian, or divergence diagnostics pass?

(3) **Scientific interpretation.** Given the surviving diagnostics, what may be concluded about approximation quality, target fidelity, or method promotion?

A result in one ledger cannot be silently upgraded into the next. A green build is engineering evidence. A valid same-scalar finite-difference row is numerical evidence. Neither by itself is a scientific proof that the approximation is the right industrial method.

The benchmark roles are therefore tiered:

- exact or controlled references test whether the declared scalar behaves as intended on small models,
- local diagnostics test whether the approximation object stays meaningful,
- promotion ladders decide whether a method deserves more expensive downstream comparison.

## 40.2 Finite-Difference Verification As The First Shared Gate

The first shared validation gate is finite-difference parity on a fixed branch. For a base parameter $\theta_0$ and coordinate direction $e_i$, define

$$D_i(h) = \frac{\widehat{\ell}_T(\theta_0 + he_i; B) - \widehat{\ell}_T(\theta_0 - he_i; B)}{2h}, \qquad G_i = \partial_i \widehat{\ell}_T(\theta_0; B), \tag{40.1}$$

where both perturbed evaluations reuse the same fixed branch ledger and recompute the parameter-dependent numerical objects through the same declared equations. The row is branch-valid only if the branch identity is unchanged on both sides.

A useful reporting contract is:

$$e_i(h) = \frac{|D_i(h) - G_i|}{\max\{1, |D_i(h)|, |G_i|\}}, \tag{40.2}$$

with a small ladder such as

$$h \in \{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\} \max\{1, |\theta_i|\}. \tag{40.3}$$

The diagnostic supports the same-scalar claim only when branch-valid rows show a stable decreasing error window. Failure may indicate a wrong derivative, a branch mismatch, or a nonsmooth/ill-conditioned value path. It is a validation gate, not a posterior-accuracy theorem.

## 40.3 Why The Synthesis Must Start With Defects

The preceding chapters explain high-dimensional nonlinear filtering methods. An industrial macro-finance program needs a harsher object: a calculus of failure. A method is useful for a global-bank or central-bank model only when the failure mode is explicit, the mitigation has a mathematical contract, and the cost variable that can kill the method is visible before implementation. Academic novelty is not the target here. The target is a defensible composition of known methods under stated assumptions.

This chapter therefore treats every candidate as a conditional component. A Gaussian scaffold is useful if it exposes local curvature and covariance failure. Sparse-grid or high-degree cubature is useful if it diagnoses local nonlinearity without pretending to be a global filter. Tensor methods are useful if economic structure keeps ranks stable and preserves probability or covariance semantics. Transport maps are useful if they reduce geometry while leaving a checkable target. HMC is useful only downstream of a same-scalar likelihood contract.

The reader-facing synthesis map is this: start with the exact conditional law and likelihood normalizers from Chapter 33; replace them only by named approximations; attach one diagnostic to each object being approximated; and promote a method only after the previous object exports the input required by the next. The chapter's exact target is not a new algorithm. It is an auditable decision procedure for composing existing methods without losing the scalar likelihood, probability-law, support, Jacobian, rank, or covariance semantics needed by industrial inference.

**Running quadratic-observation cell.** The scalar cell

$$x_t = \rho x_{t-1} + \eta_t, \qquad y_t = x_t^2 + \epsilon_t$$

has now served five roles. Chapter 33 used it to show that the exact filtering law can have two lobes. Chapter 35 used it to show why Gaussian projection can preserve moments while losing shape. Chapter 36 used it to turn one fixed sparse-grid cloud into a concrete value-path and numeric-oracle contract. Chapter 37 and Chapter 38 used it to separate retained-object semantics from fixed-branch likelihood construction. Chapter 39 used it to insist that HMC needs a scalar likelihood and a same-scalar gradient. Here the same cell is not evidence for a macro-finance model. It is the small mechanism that tells a mixed numerical reader what the architecture is trying not to lose.

**Imports from the preceding chapters.**

| Source chapter | Exported object | Exported defect | Exported diagnostic/veto | Consumed here as |
|---|---|---|---|---|
| Ch. 33 | conditional law, normalizer, likelihood gradient | wrong target, missing normalizer, value-gradient mismatch | finite $Z_t$, mass, positivity, same-scalar check | target and likelihood gates |
| Ch. 35 | Gaussian moment pair and point-rule family | projection bias, exactness without accuracy, point explosion | innovation, PSD, active dimension, comparator residual | scaffold and local curvature diagnostic |
| Ch. 36 | fixed sparse-grid scalar, numeric oracle, branch record | merge mismatch, signed-weight instability, branch-invalid finite differences | oracle reproduction, branch-valid FD, minimum-eigenvalue veto, cloud reproduction | deterministic Gaussian lane validation |
| Ch. 37 | retained object, coordinate map, KR / preconditioning semantics | support failure, Jacobian mismatch, rank/semantic failure | support/logdet, rank, mass, positivity | retained-object meaning gate |
| Ch. 38 | fixed-branch TT/KR scalar, stored-field contract | wrong saved object, branch-identity loss, incomplete next-step closure | stored-field audit, retained-marginal closure, branch-fixed structural check | operational middle gate |
| Ch. 39 | scalar potential, gradient, transformed target | same-scalar mismatch, divergences, invalid transform | scalar parity, finite support/Jacobian, energy diagnostics | downstream sampler gate |

## 40.4 Lane-Specific Validation Ladders Before Promotion

The high-dimensional part now contains two distinct approximation lanes with different local failure modes, so the validation ladder must stay lane-aware before the closing promotion comparison begins.

### 40.4.1 FixedSGQF ladder

The sparse-grid lane should not be promoted from elegant construction alone. Before any cross-method comparison, it should pass a ladder that includes:

1. deterministic cloud reproduction and duplicate-merge checks,
2. the one-step numeric oracle from Chapter 36,
3. branch-valid finite differences on a saved fixed-cloud branch,
4. signed-weight stability checks through $P_t^-$, $S_t$, and carried covariance objects,
5. controlled GHQ / linear-Gaussian comparators where available.

These diagnostics answer whether the implementation is evaluating the declared fixed-cloud scalar, not whether Gaussian projection has become an exact high-dimensional posterior method.

### 40.4.2 Retained-object TT/KR ladder

The TT/KR lane carries a richer retained object and therefore faces richer local failure modes. Before cross-method promotion, it should pass a ladder that includes:

1. coordinate-system and Jacobian-consistency checks,
2. positivity / nonnegativity and normalization checks for the represented object,
3. rank-growth and conditioning diagnostics,
4. stored-field and retained-marginal closure checks for the next step,
5. branch-valid finite differences on the fixed-branch likelihood route.

These diagnostics do not prove that moderate TT rank is always sufficient or that the adaptive source algorithm is globally smooth. They only establish whether the frozen retained-object branch remains a scientifically interpretable scalar object on the tested route.

**Copied-core and branch-status reporting.** When the derivative claim is part of the result, the TT/KR lane should report not only a finite-difference error table but also the branch-status fields imported from Chapter 39: branch-identity indicators, copied-core indicators, decreasing-window indicators, and the resulting status classification. A small error alone is insufficient if the perturbed values were computed on different frozen fields or with copied fitted cores.

A useful derivative reporting object is:

$$\mathcal{R}_{\mathrm{FD}} = \{\widehat{\ell}_T(\theta_0; B),\ G_i,\ (h_i, D_i(h_i), e_i(h_i))_i, I_{\pm}(h_i), K_{\pm}(h_i), W_i, \mathrm{status}\}. \tag{40.4}$$

Here the report should also retain the scalar values $\widehat{\ell}_T(\theta_0 + h_i e_j; B)$ and $\widehat{\ell}_T(\theta_0 - h_i e_j; B)$ for each $h_i$, not merely the final error summary. The derivative table is meaningful only when the branch record is identical in the three columns base / plus / minus.

**Benchmark-specific interpretation.** The benchmark ledger should stay explicit about what each model can and cannot establish. A linear-Gaussian benchmark can establish exact-reference agreement for the declared run contract, but not nonlinear posterior accuracy. A stochastic-volatility or predator-prey benchmark can establish long-horizon nonlinear stability or preconditioning benefit under the declared run contract, but not universal scalability. A spatial SIR benchmark can establish whether the method remains useful at the tested state dimension, horizon, rank ladder, and basis ladder, but not that all spatial epidemic models admit moderate TT rank. One-seed demonstrations remain diagnostic unless the result artifact explicitly justifies a stronger interpretation.

**Runtime, memory, and veto interpretation.** Runtime and memory should be interpreted only after veto diagnostics have passed. When reported, they should be split into mathematically meaningful components: forward target evaluation, fitting solves, mass contractions, KR inversion or sampling work, retained-object storage work, and derivative overhead. Likewise, peak memory should separate value-only storage from value-plus-gradient storage. These are not mere profiling counters; they are the places where the fixed-branch approximation can become unaffordable or mathematically fragile.

## 40.5 Mathematical Veto Ledger Before Interpretation

The validation report should mark a run as failed before interpreting posterior plots, trajectory summaries, or method rankings if any mathematical veto fires. Let

$$\mathcal{F}_t = \{Z_t, \widehat{Z}_t, \widehat{p}_t, C_{t,k}, \dot{C}_{t,k}, w_i, D_h, G_{\mathrm{FB}}\} \tag{40.5}$$

collect the scalar, tensor, weight, and derivative objects used at time $t$. The finite-value veto is

$$V_{\mathrm{finite}} = \mathbf{1}\{\exists t, \exists a \in \mathcal{F}_t : \ a \text{ contains NaN or Inf}\}. \tag{40.6}$$

The normalization veto is

$$V_{\mathrm{norm}} = \mathbf{1}\{\exists t : \ \widehat{Z}_t \le 0 \text{ or } |\int \widehat{p}_t(z)\,\mathrm{d}z - 1| > \varepsilon_{\mathrm{norm}}\}. \tag{40.7}$$

The least-squares conditioning veto is

$$V_{\mathrm{cond}} = \mathbf{1}\{\max_{t,k} \kappa(N_{t,k}) > \kappa_{\max}^{\mathrm{allowed}}\}. \tag{40.8}$$

The rank-saturation veto is

$$V_{\mathrm{rank}} = \mathbf{1}\{\exists t : \ R_t = R_{\max} \text{ and residual}_t > \varepsilon_{\mathrm{fit}}\}. \tag{40.9}$$

The KR monotonicity veto is

$$V_{\mathrm{KR}} = \mathbf{1}\{\exists j, z_{<j}, a < b : \ F_j(b \mid z_{<j}) < F_j(a \mid z_{<j})\}. \tag{40.10}$$

Finally, the fixed-branch derivative veto is

$$V_{\mathrm{grad}} = \mathbf{1}\{B(\beta{+}h) \neq B(\beta) \text{ or } B(\beta{-}h) \neq B(\beta) \text{ or no branch-stable decreasing window holds}\}. \tag{40.11}$$

A run should be interpreted only if

$$V_{\text{finite}} + V_{\text{norm}} + V_{\text{cond}} + V_{\text{rank}} + V_{\text{KR}} + V_{\text{grad}}^{\text{required}} = 0, \tag{40.12}$$

where $V_{\text{grad}}^{\text{required}} = V_{\text{grad}}$ only when the claim includes validating the derivative. The tolerances in these vetoes must be declared before a run. A minimal default table is:

| quantity | default | meaning |
|---|---|---|
| $\varepsilon_{\text{norm}}$ | $10^2\epsilon_{\text{mach}} + 10\,\varepsilon_{\text{quad}}$ | normalization tolerance |
| $\kappa_{\max}^{\text{allowed}}$ | $10^{10}$ in double precision | least-squares conditioning ceiling |
| $\varepsilon_{\text{fit}}$ | $\max(10^{-8}, 10\,\text{tol}_{\text{ALS}})$ | rank-saturation residual ceiling |
| $\varepsilon_{\text{KR}}$ | $10^2\epsilon_{\text{mach}} + 10\,\varepsilon_{\text{quad}}$ | monotonicity/inversion tolerance |

(40.13)

These are implementation tolerances, not scientific outcomes. Changing them after seeing posterior plots invalidates the benchmark interpretation.

## 40.6 Defect 1: Log-Weight Dispersion Can Become Exponential Without Structure

The bootstrap and SIR filters in Chapter 37 are honest baselines because their collapse mechanism is visible. The aim of this section is not to prove the strongest available particle-collapse theorem. It is to record a heuristic defect calculus showing why, without locality, guided proposals, or tempering, the variance scale entering the weights can grow rapidly with effective dimension. The sharper asymptotic statements belong to the cited particle-filter literature; the present derivation is a warning model, not a complete theorem of exponential collapse.

$$\text{ESS} = \frac{1}{\sum_{i=1}^{N} \bar{W}_i^2}. \tag{40.14}$$

Suppose the log-weight decomposes across conditionally independent observation blocks,

$$L_i = \sum_{k=1}^{d_{\text{eff}}} \ell_{ik}, \qquad \mathbb{E}[\ell_{ik}] = \mu, \qquad \text{Var}(\ell_{ik}) = \sigma_\ell^2, \tag{40.15}$$

with weak dependence sufficient for $\text{Var}(L_i) \asymp d_{\text{eff}}\sigma_\ell^2$. This is the informal calculus behind the high-dimensional particle-filter warnings of Bengtsson et al. [2008] and Snyder et al. [2008].

**Proposition 40.1** (Log-weight variance warning)**.** *If the log-weight variance grows like* $\text{Var}(L_i) = v_d \asymp d_{\text{eff}}\sigma_\ell^2$*, then the associated importance-weight dispersion scale grows exponentially in* $v_d$ *under the lognormal heuristic below. In that regime, a particle filter with proposal independent of the current observation can enter a practically severe collapse regime unless ensemble size grows very aggressively with that dispersion scale.*

*Derivation.* For a lognormal heuristic, $\mathbb{E}[W] = \exp(\mu_d + v_d/2)$ and $\mathbb{E}[W^2] = \exp(2\mu_d + 2v_d)$, so

$$\frac{\mathbb{E}[W^2]}{\mathbb{E}[W]^2} = \exp(v_d).$$

The usual importance-sampling variance factor is therefore exponential in $v_d$. Since the denominator of (40.14) is driven by the largest normalized weights, stable ESS typically requires sample growth that offsets this exponential dispersion. The cited papers give sharper asymptotic statements under stronger assumptions; the monograph uses this calculation only to justify a heuristic defect warning about log-weight dispersion, not to claim a full theorem of exponential particle collapse for every proposal/model pair. □

## 40.7 Defect 2: Fixed-Box Row Designs Collapse Against Concentrated Step Targets

The second defect was measured, not anticipated: it was diagnosed on the squared-TT value ladder of Chapter 38 (execution artifacts 2026-08-17 through 2026-08-20) and is recorded here because its *signature* — an error plateau that ignores every resolution knob while in-sample residuals improve — is exactly the kind of misleading evidence this chapter's ledgers exist to catch.

### 40.7.1 The mechanism

The fixed least-squares step fits $h$ so that $h^2$ tracks the step target $f_t$ on the reference box $B = [-1, 1]^{2n}$, from $N$ frozen rows $z_i$ with weights $w_i$, while the likelihood increment uses the *exact* Gram normalizer $Z_h = \int_B h^2 \, d\mu$. Two structural facts collide as $n$ grows.

First, the transition-step target concentrates on the intersection of two kernel manifolds, $x_t \approx Ax_{t-1}$ (transition) and $x_t \approx y_t$ (observation), whose thickness is set by the noise scales and does *not* grow with the box. For a uniform-in-$\mu$ row design the effective sample size against the target,

$$\mathrm{ESS}_f = \frac{\left(\sum_i w_i f_t(z_i)\right)^2}{\sum_i w_i^2 f_t(z_i)^2}, \tag{40.16}$$

therefore decays like the volume ratio of a correlated $O(\sigma/\text{halfwidth})^{2n}$ slab: geometrically in $n$. Measured on the linear-Gaussian ladder fixture at $N = 8192$ rotated-Sobol rows and halfwidth 3: $\mathrm{ESS}_f \approx 219$ at $n = 2$ but $\approx 11$ at $n = 4$ — the fit determines $\sim$208 core columns from $\sim$11 informative rows.

Second, the exact normalizer converts *unseen* target mass directly into likelihood bias. Writing the fitted square as $h^2 = f_t + e$, the increment carries $\log \int_B h^2 \, d\mu = \log\big(\int_B f_t \, d\mu + \int_B e \, d\mu\big)$, and rows that never visit the mass-bearing manifold produce a fit whose error term $e$ removes exactly the mass the rows never saw. The measured signature at $n = 4$: a per-step under-count of 2–3 nats at *every* transition step (the $t = 0$ marginal, whose target is not concentrated, was exact to $3 \times 10^{-3}$), essentially constant in rank ($r = 2 \ldots 8$), basis degree ($12 \ldots 16$), and row budget ($8192 \ldots 32768$ bought 21%), while the in-sample weighted residual *improved* monotonically. In-sample residuals are computed under the same defective row measure and are therefore structurally blind to this defect: they are an explanatory diagnostic only, never a promotion criterion.

### 40.7.2 Why importance rows make it worse

The textbook repair — draw rows from a proposal $q$ concentrated near the manifold and reweight by $1/q$ — was implemented and *refuted* at the step level. With conjugate

(exact) Gaussian proposals, defensive uniform mixtures from 10% to 75%, and both weighted-$L^2(\mu)$ and $L^2(q\mu)$ fit objectives, the increment error flipped from a $-1.7$-nat under-count to $+7$ to $+62$-nat over-counts, degrading *monotonically* with basis degree. The mechanism is a representation constraint, not a sampling one: once rows resolve the manifold, the square-root target carries the target's full dynamic range across the fixed box (relative amplitudes $e^{10+}$ after the smooth shift), a moderate-rank polynomial TT cannot represent that spike, its off-manifold oscillation is unconstrained precisely where rows are sparse, and the exact $Z_h = \int h^2$ integrates the oscillation into spurious mass. The uniform design had merely *hidden* the spike. No row design can repair this: the defect lives in the coordinates, not in the quadrature.

### 40.7.3 The repair: bridged coordinates, not better rows

The source construction already contains the answer. Zhao and Cui [2024] (their Section 5) precondition the step: a bridging density $\rho_t$ built from already-computed state defines a Knothe–Rosenblatt change of coordinates $T_t$ with $(T_t)_\sharp \rho_t \propto \eta$ for a tensor-product reference $\eta$, and the TT fits the *ratio* pushforward

$$q_{\sharp,t}(u) \;=\; \frac{q_t\big(T_t^{-1}(u)\big)}{\rho_t\big(T_t^{-1}(u)\big)}\,\eta(u), \tag{40.17}$$

which is $\eta$ perturbed by the (far less concentrated) ratio $q_t/\rho_t$. Their linear instance (their Section 5.2) takes $\rho_t$ Gaussian from estimated moments, so $T_t$ is a per-step affine map — recentring and rescaling the box around where the filter actually is. A step-level probe of the affine instance on the $n = 4$ fixture (block-diagonal maps from filtered moments at $\kappa = 3$ standard deviations, rows uniform in the new box, Jacobian carried in the target) moved the step error from $-1.71$ to $-0.30$ nats and the coverage (40.16) from 11 to 174 — the only lever, among rows, rank, degree, halfwidth, and importance weighting, that moved the error at all. The over-widened $\kappa = 4$ variant degraded again, as the coverage mechanism predicts.

### 40.7.4 Ledger consequences

Three rules join the promotion calculus. (i) A row-design scope must record a target-coverage diagnostic of the form (40.16); a fit with collapsed $\mathrm{ESS}_f$ is invalid as evidence regardless of its residual. (ii) In-sample weighted residuals never gate promotion for scattered-row fits; only same-target references or resolved-quadrature cross-checks do. (iii) Fixed-box scopes do not extrapolate in $n$: coverage decays geometrically, so each dimension rung requires either a re-validated coverage diagnostic or the preconditioned coordinates of (40.17).

The engine-level form of this repair — per-step frozen triangular maps propagated through the retained object, with a quadrature correction for increment mass truncated by the box — subsequently closed the $n = 2$ rung outright: $5.4\times10^{-5}$ nats per step against the declared $2.5 \times 10^{-3}$ bar, with $r^*(2) = 6$ confirmed across seeds and ranks (execution artifacts 2026-08-20 through 2026-08-22). The same engine at $n = 4$, however, exposed a third defect — not in the rows, and not repaired by the maps — in the bounded domain itself. Section 40.8 derives it, and Section 40.9 resolves the construction question it forces.

## 40.8 Defect 3: Chained Bounded Domains Truncate The Filter Itself

The bridged-coordinate repair of Section 40.7 places every step's fit on its own moment box. Doing so exposes a third defect that is invisible at a single step because it is created by the *recursion*: the retained polynomial of step $t-1$ is defined only on the step-$t-1$ box, so the step-$t$ construction is forced to keep its previous-block domain inside the old one, and the nested domains truncate filter mass at a geometric rate in the dimension. This section derives the defect in proposition–proof form. Throughout, $n$ is the state dimension (the joint fit variable has dimension $2n$), $\Phi$ is the standard normal distribution function, and "the engine" is the adapted squared-TT value engine with per-step frozen affine maps validated at $n = 2$.

### 40.8.1 Why the domains must nest

**Lemma 40.1** (Containment is forced by retention)**.** *Let the step-$t-1$ retained object be a polynomial density representation defined on the image $B_{t-1} = m_{t-1} + L_{t-1}[-1,1]^n$ of its own coordinate map, with no declared extension outside $B_{t-1}$. Then any step-$t$ fit whose previous-block rows are drawn in a new box $B_t^p = m_t^p + L_t^p[-1,1]^n$ is well defined only if $B_t^p \subseteq B_{t-1}$.*

*Proof.* The step-$t$ target evaluates the retained density of step $t-1$ at every previous-block row location $x_p \in B_t^p$. By hypothesis the retained object assigns no value to $x_p \notin B_{t-1}$: the polynomial expression exists there formally but does not represent the retained density, and the engine's fail-closed assertion rejects such rows. A well-defined step therefore requires every point of $B_t^p$ to lie in $B_{t-1}$. □

The constraint would be harmless if the boxes could remain wide. They cannot. The engine's measured containment behavior (wide-$\kappa$ grid, 2026-08-21, two independent measurements) is that the *effective* previous-block half-width — the requested $\kappa$ times the shrink factor that containment imposes — is pinned at $\kappa^* \approx 2.0$–$2.2$ standard deviations regardless of the requested $\kappa \in [3,6]$: the containment bound involves the ratio of new-box to old-box widths, both of which scale with the same requested $\kappa$, so the request cancels. We record $\kappa^*$ as a measured engine constant, not a theorem; the propositions below take it as an input and are exact given it.

### 40.8.2 The truncated mass is geometric in dimension

**Proposition 40.2** (In-box mass of a whitened Gaussian block)**.** *Let $X \sim \mathcal{N}(m, \Sigma)$ on $\mathbb{R}^n$ with $\Sigma = LL^\top$ nonsingular, and let $B_\kappa = \{x : \|L^{-1}(x-m)\|_\infty \le \kappa\}$ be the moment box with half-width $\kappa$ in whitened coordinates. Then*

$$\Pr\{X \in B_\kappa\} = \big(2\Phi(\kappa) - 1\big)^n. \tag{40.18}$$

*Proof.* Set $U = L^{-1}(X - m)$. Then $U \sim \mathcal{N}(0, I_n)$, the event $\{X \in B_\kappa\}$ is $\{|U_i| \le \kappa\ \forall i\}$, the coordinates $U_i$ are independent standard normals, and $\Pr\{|U_i| \le \kappa\} = \Phi(\kappa) - \Phi(-\kappa) = 2\Phi(\kappa) - 1$. The product over $i = 1, \dots, n$ gives (40.18). □

**Proposition 40.3** (Geometric growth of the truncated fraction)**.** *Under the hypotheses of Proposition 40.2, the truncated fraction $\varepsilon_{\text{trunc}}(n,\kappa) = 1 - \big(2\Phi(\kappa)-1\big)^n$ is strictly increasing*

*in $n$ and satisfies $\varepsilon_{\mathrm{trunc}}(n,\kappa) \geq 1 - e^{-2n\overline{\Phi}(\kappa)}$ with $\overline{\Phi}(\kappa) = 1 - \Phi(\kappa)$; in particular it does not vanish for any fixed $\kappa$ as $n$ grows, and at the measured containment cap it is already material at $n = 4$:*

| | $n = 2$ | $n = 4$ | $n = 8$ |
|---|---|---|---|
| $\kappa = 2.0$ | 0.089 | 0.170 | 0.311 |
| $\kappa = 2.2$ | 0.055 | 0.107 | 0.202 |
| $\kappa = 3.0$ | 0.005 | 0.011 | 0.021 |

*Proof.* Monotonicity in $n$ is immediate since $0 < 2\Phi(\kappa) - 1 < 1$. For the bound, write $2\Phi(\kappa)-1 = 1-2\overline{\Phi}(\kappa)$ and use $\log(1-x) \leq -x$: $(1-2\overline{\Phi}(\kappa))^n \leq e^{-2n\overline{\Phi}(\kappa)}$. The tabulated values evaluate (40.18) directly (diagnostic script `check_reference_domain_selection_20260823.py`). □

The measured engine telemetry sits *above* this ideal bound, as it must: the containment shrink is anisotropic and the box is not perfectly centered. On the $n = 4$ ladder cells (attempt04, 2026-08-22) the ratio of increment mass recovered outside the box to mass inside was 0.36–1.85 across six cells — in three of six, more step mass lay outside the box than inside — against the ideal 0.11–0.17 of the table. At $n = 2$ the same telemetry read 0.03–0.11, which is why the $n = 2$ rung passes with margin.

### 40.8.3 Truncation is a bias floor, not a resolution problem

The decisive point is that the truncated mass is not an approximation error that better fitting reduces. Even a *perfect* fit on the box retains the wrong law.

**Proposition 40.4** (Total-variation cost of conditioning)**.** *Let $P$ be a probability measure and $B$ an event with $P(B) = 1 - \varepsilon > 0$, and let $P_B = P(\cdot \mid B)$ be the conditioned law. Then* $\mathrm{TV}(P_B, P) = \sup_A |P_B(A) - P(A)| = \varepsilon$.

*Proof.* For any measurable $A$, decompose $P(A) = P(A \cap B) + P(A \cap B^c)$ and $P_B(A) = P(A \cap B)/(1 - \varepsilon)$. Hence

$$P_B(A) - P(A) = P(A \cap B)\Big(\tfrac{1}{1-\varepsilon} - 1\Big) - P(A \cap B^c) = \tfrac{\varepsilon}{1-\varepsilon} P(A \cap B) - P(A \cap B^c).$$

The first term lies in $[0, \varepsilon]$ because $P(A \cap B) \leq 1 - \varepsilon$; the second lies in $[0, \varepsilon]$. Their difference therefore lies in $[-\varepsilon, \varepsilon]$, so the supremum is at most $\varepsilon$, and $A = B^c$ attains it: $P_B(B^c) - P(B^c) = 0 - \varepsilon = -\varepsilon$. □

**Remark 40.1** (Zero-fit-error bias floor of the chained-box program)**.** *Combine the three statements. Containment (Lemma 40.1) pins the effective previous-block box at $\kappa^* \approx$* 2–2.2 *(measured); at that width a Gaussian block loses the geometric mass fraction of Proposition 40.3; and by Proposition 40.4 the step's retained law then differs from the true one by at least that fraction* in total variation even at zero fit error. *The floor is independent of TT rank, basis degree, and row budget — which is exactly the measured attempt04 signature at $n = 4$: errors of* 0.34–0.47 *nats per step, nearly flat from rank* 6 *to* 8*, with the truncation-ratio telemetry identifying the cause. The one-step statement is exact; the multi-step compounding (each step conditions anew on a new box) is measured rather than bounded in closed form here, and it can only worsen the one-step floor. A scalar quadrature correction can and does repay the* increment *mass (it moved the $n = 4$*

*value error from* $\sim$2.5 *nats to* 0.34–0.47*), but it cannot repair the* retained shape*: the recursion propagates a minority slice of the posterior. This is the precise sense in which the bounded-box program is insufficient at* $n \geq 4$*, and it discharges the rôle the truncation-ratio flag was designed for: those cells are invalid as rank evidence, so no* $r^*(4)$ *statement of any kind is licensed by them.*

### 40.8.4 Ledger consequences

Two rules join the calculus. (i) Any bounded-domain scope whose retained object chains across steps must record the truncation-ratio diagnostic (recovered outside-mass over inside-mass); a cell whose ratio exceeds the declared threshold is invalid as rank or accuracy evidence, whatever its residuals. (ii) A domain policy is a structural assumption in the sense of this chapter's promotion calculus: it must be re-justified at every dimension rung, because Proposition 40.3 makes "worked at $n$" evidence that expires geometrically in $n$.

## 40.9 Reference-Domain Selection For The Squared-TT Program

Defect 3 forces a construction question that must not be settled by preference: on which reference domain, with which basis family and which reference measure, should the squared-TT recursion live? This section states the admissible candidates, defines the properties a construction must satisfy or optimize, and derives the status of every candidate against every property. The conclusion — recorded here with its supporting derivations because it redirects the high-dimensional program — is that the incumbent bounded construction is *eliminated* by three independent arguments, and that among the two source-faithful replacements the Gaussian-reference Hermite construction is selected on the derived criteria, with the compactified construction retained as the documented fallback.

### 40.9.1 Candidates and properties

**Definition 40.1** (Candidate constructions)**.** *All candidates share the per-step frozen affine (triangular) preconditioner* $T_t(x) = L_t^{-1}(x - m_t)$ *built from deterministic moment hints, the squared representation with defensive term, the exact Gram-chain normalizer, and the manual-adjoint derivative program. They differ only in the triple (reference measure* $\mu$*, basis, domain):*

***C1*** *(incumbent)* $\mu$ *= uniform on the moment box* $B_t \subset \mathbb{R}^{2n}$*; Legendre basis; retained domains chained by Lemma 40.1.*

***C2*** $\mu = \mathcal{N}(0, I_{2n})$ *on* $\mathbb{R}^{2n}$ *in the preconditioned coordinates* $u = T_t(x)$*; normalized probabilists' Hermite basis; no boxes.*

***C3*** $\mu$ *= uniform on* $[-1, 1]^{2n}$ *in coordinates* $z_i = u_i/\sqrt{s^2 + u_i^2}$ *(the per-axis algebraic compactification used by the reference implementation of Zhao and Cui [2024]); Legendre or Lagrange basis on the box; no truncation, since the compactification is a diffeomorphism of* $\mathbb{R}$ *onto* $(-1, 1)$*.*

**Definition 40.2** (Requirements and objectives)**.** *A construction must satisfy the hard requirements*

***R1*** Exact normalizer and analytic adjoint*: the one-dimensional mass matrices* $B_{ij} = \int b_i b_j \, d\mu$ *are closed-form constants, so the Gram-chain normalizer and the manual adjoint of the declared finite program exist exactly;*

***R2*** Well-defined recursion*: the step-t row law is supported inside the domain of the step-*$(t-1)$ *retained object;*

*and is scored on the objectives*

***O1*** *structural truncation mass* $\varepsilon_{\text{trunc}}$ *(target mass outside the representable domain at zero fit error);*

***O2*** *the zero-fit-error bias floor of the retained law;*

***O3*** *per-axis basis cardinality* $\ell(\varepsilon)$ *needed for relative* $L^2(\mu)$ *accuracy* $\varepsilon$ *on the program's calibration family (the whitened near-Gaussian step targets; the linear-Gaussian model is the exactly-Gaussian member);*

***O4*** *TT rank demand on the same family;*

***O5*** *conditioning of basis evaluations on the effective row support, and the* $\ell$*-driven cost channels (design-matrix width* $\ell r^2$*, mass work* $\ell^2$*, compiled-graph size);*

***O6*** *existence of the exact parameter score of the declared program with closed-form adjoint nodes;*

***O7*** *the defensive floor performs its declared function: the floor density dominates the target (*$\sup_x q/\lambda < \infty$*), which is the hypothesis under which Zhao and Cui [2024, eq. (13) and Lemma 1] prove the bounded target-to-approximation ratio.*

*Engineering re-derivation cost is deliberately* not *an objective; it is a budget input recorded separately, because allowing it to stand in for a mathematical property is how an inherited construction survives scrutiny it should not survive.*

### 40.9.2 The reference algebra is closed-form in all three candidates

**Proposition 40.5** (Hermite mass identity)**.** *Let* $\mathrm{He}_k$ *denote the probabilists' Hermite polynomials and* $\widetilde{\mathrm{He}}_k = \mathrm{He}_k/\sqrt{k!}$*. Under* $\mu = \mathcal{N}(0,1)$*,* $\int \widetilde{\mathrm{He}}_j \, \widetilde{\mathrm{He}}_k \, d\mu = \delta_{jk}$*: the mass matrix is the identity, exactly, at every degree.*

*Proof.* The generating function of the family is $G(z,t) = e^{zt - t^2/2} = \sum_{k\geq 0} \mathrm{He}_k(z)\, t^k/k!$. With $\varphi$ the standard normal density,

$$\int G(z,s)\, G(z,t)\, \varphi(z)\, dz = e^{-(s^2+t^2)/2} \int e^{z(s+t)} \varphi(z)\, dz = e^{-(s^2+t^2)/2}\, e^{(s+t)^2/2} = e^{st},$$

using the normal moment generating function $\int e^{za}\varphi(z)\, dz = e^{a^2/2}$. Expanding both sides,

$$\sum_{j,k} \frac{s^j t^k}{j!\, k!} \int \mathrm{He}_j \, \mathrm{He}_k \, d\mu = \sum_m \frac{(st)^m}{m!},$$

and matching coefficients of $s^j t^k$ gives $\int \mathrm{He}_j \mathrm{He}_k \, d\mu = \delta_{jk}\, k!$. Dividing by $\sqrt{j!\,k!}$ yields the claim. (Numerical confirmation: the degree-$\leq 12$ mass matrix computed by 60-node Gauss–Hermite quadrature matches the identity to $1.3 \times 10^{-15}$.) □

**Proposition 40.6** (C3 leaves the incumbent reference algebra unchanged)**.** *Under C3 the reference measure is the uniform probability measure on $[-1,1]^{2n}$, identical to C1's; every mass matrix, Gram chain, retention contraction, and adjoint node of the incumbent program is unchanged. The compactification enters only through the fit target: with $u_i = sz_i/\sqrt{1-z_i^2}$ per axis and Jacobian $du_i/dz_i = s(1-z_i^2)^{-3/2}$, the density of a physical target $q$ with respect to $\mu$ in $z$-coordinates is*

$$q_\mu(z) \;=\; q\big(u(z)\big) \prod_{i=1}^{2n} s\,(1-z_i^2)^{-3/2} \Big/ 2^{-2n}, \tag{40.19}$$

*a row-evaluable factor with no effect on the basis-side algebra.*

*Proof.* The reference algebra of the incumbent program depends only on the pair (basis, $\mu$); C3 keeps both. Equation (40.19) is the change-of-variables formula for densities under the product diffeomorphism $z \mapsto u(z)$, with $2^{-2n}$ the uniform reference density. All $z$-dependence introduced by the map sits inside the target evaluation at rows, which the fit already treats as a black box. □

**Remark 40.2** (The derivative requirement never forced the box)**.** *R1 holds for all three candidates — Proposition 40.5 for C2 (where the identity mass matrix in fact* simplifies *every Gram chain), Proposition 40.6 for C3, and the existing certified program for C1. R6 below is likewise candidate-independent. The program's historical premise that the analytic-derivative requirement selected the bounded Legendre box is therefore* wrong relative to that claim*: the only place the derivative requirement genuinely forces a deviation from Zhao and Cui [2024] is the preconditioner's moment estimate — their Section 5.2 estimates $(\mu_t, \Sigma_t)$ by a particle step, which is not differentiable, so this program uses frozen deterministic hints — and that deviation is identical across C1, C2, and C3.*

### 40.9.3 Domain validity eliminates the incumbent

**Proposition 40.7** (Domain scorecard)**.** *(i) C1 fails R2 at the measured containment cap for every unbounded-support state model, with truncation mass growing geometrically in $n$ (Propositions 40.2–40.3) and a zero-fit-error retained bias of at least that mass (Proposition 40.4). (ii) C2 and C3 satisfy R2 with $\varepsilon_{\mathrm{trunc}} \equiv 0$ and zero structural bias floor: their domains are all of $\mathbb{R}^{2n}$ (directly for C2; via the diffeomorphism for C3), so the previous-step retained object is defined at every possible row, containment never arises, and at zero fit error the retained law equals the true one up to the defensive-term mixture quantified in Proposition 40.8.*

*Proof.* (i) is the content of Defect 3 (Section 40.8). For (ii), the retained object under C2 or C3 is a density on the full space; the step-$t$ row law, wherever it is supported, lies in that domain, so Lemma 40.1 imposes no constraint. With no excluded region, $\varepsilon_{\mathrm{trunc}} = 0$ by definition, and the zero-fit-error retained density equals $(q_\mu + \tau Z_h \lambda)/((1+\tau)Z_h)$, whose distance from the truth is exactly the mixture term treated below. □

### 40.9.4 The defensive floor: bias cost and domination

The squared representation carries the defensive term of Zhao and Cui [2024, eq. (13), eq. (16)]: the represented $\mu$-density is $\bar{q} = h^2 + \tau Z_h \lambda$ with $Z_h = \int h^2 d\mu$, $\lambda \geq 0$, $\int \lambda \, d\mu = 1$, so the increment $\int \bar{q} \, d\mu = (1+\tau) Z_h$ remains exact. The floor exists to keep the target absolutely continuous with respect to the representation — bounded log-ratios, importance weights satisfying a central limit theorem — and its two costs are quantified next.

**Proposition 40.8** (Defensive mixture bias)**.** *At zero fit error ($h^2 = q_\mu$, the $\mu$-density of the true step target with mass $Z_h$), the normalized represented law is the mixture $\hat{p} = (1-w)\, p + w\, \lambda$ with $p = q_\mu / Z_h$ and $w = \tau/(1+\tau)$, and*

$$\mathrm{TV}(\hat{p}, p) = w \, \mathrm{TV}(\lambda, p) \leq \frac{\tau}{1+\tau},$$

*independently of the dimension. Under the source coupling $\tau_t \leq \|\phi_t - \sqrt{q_t}\|_{L^2}^2$ [Zhao and Cui, 2024, Lemma 1 and Proposition 11] the bias is second order in the fit error, so the zero-fit-error limit of the retained law is exactly the truth; under this program's current fixed policy $\tau = 10^{-6}$ it is a per-step constant three orders below the $2.5 \times 10^{-3}$ accuracy bar.*

*Proof.* Normalizing $\bar{q} = q_\mu + \tau Z_h \lambda$ by its mass $(1+\tau) Z_h$ gives $\hat{p} = \frac{1}{1+\tau} p + \frac{\tau}{1+\tau} \lambda$. Then $\hat{p} - p = w(\lambda - p)$, and integrating absolute values, $\mathrm{TV}(\hat{p}, p) = \frac{1}{2} \int |\hat{p} - p| = w \cdot \frac{1}{2} \int |\lambda - p| = w \, \mathrm{TV}(\lambda, p) \leq w$. The mixture weight $w$ contains no dimensional quantity. The two $\tau$ policies substitute directly. □

**Proposition 40.9** (Bounded domains defeat the floor's own hypothesis)**.** *Let the step target $q$ have full support on $\mathbb{R}^{2n}$ (as for every Gaussian-family fixture). Then for any defensive density $\lambda$ supported in a bounded box $B$, $\sup_x q(x)/\lambda(x) = \infty$, so the domination hypothesis of Zhao and Cui [2024, eq. (13)] fails and the guarantee $\sup_x p/\hat{p} < \frac{\hat{z}}{\tau z} \sup_x q/\lambda$ is vacuous. Under C2 and C3 the hypothesis is satisfiable by design: $\lambda$ may be any full-support tensor-product density with closed-form mass (the reference density itself, or a heavier-tailed product chosen to dominate a declared target tail class), so the floor's tail order is a free design parameter rather than a property of the construction.*

*Proof.* On $B^c$, $q > 0$ while $\lambda = 0$, so the ratio is unbounded. Under C2/C3 take $\lambda$ to be any product density positive on $\mathbb{R}^{2n}$ with $\int \lambda \, d\mu = 1$; positivity everywhere makes $\sup q/\lambda$ finite for any continuous $q$ with tails dominated by $\lambda$'s, which is a choice of $\lambda$, not a constraint of the domain. □

**Remark 40.3** (What the floor regularizes, and what it does not)**.** *The floor splits the tail question into two channels that must not be conflated.* Recursion stability*: with domination, the represented log-density is floored at $\log(\tau Z_h \lambda)$, so zeros of the fitted polynomial and un-fitted tail regions produce bounded, tail-mass-weighted errors at later rows instead of divergent log-targets or unbounded ratios — this is exactly the guard's declared function, available to C2 and C3 equally through the choice of $\lambda$, and structurally unavailable to C1 by Proposition 40.9.* Increment accuracy*: the floor's mass is $\lambda$-shaped, not target-shaped; Lemma 1 of Zhao and Cui [2024] prices it as additional $L^2$ error, and target mass the fit does not capture still biases the increment one for one. The floor therefore does not relax any approximation-efficiency requirement below; it converts tail* failures *from catastrophic to graded, which is a safety property, not an accuracy subsidy.*

### 40.9.5 Approximation efficiency separates the survivors

**Proposition 40.10** (Degree collapse at the Gaussian calibration point of C2)**.** *Let the step target be $q(x) = Z\,\mathcal{N}(x; m, \Sigma)$ on $\mathbb{R}^d$ ($d = 2n$), and let the preconditioner use the exact moments: $T(x) = L^{-1}(x - m)$, $LL^\top = \Sigma$. Then the C2 fit target — the density of the pushforward $T_\sharp q$ with respect to $\eta = \mathcal{N}(0, I_d)$ — is the constant $Z$, so the exact fit is $h \equiv \sqrt{Z}$: per-axis degree 0, TT rank 1.*

*Proof.* The pushforward density with respect to Lebesgue measure is $q(T^{-1}(u))\,|\det L| = Z\,\mathcal{N}(m + Lu; m, \Sigma)\,|\det L|$. Since $\mathcal{N}(m + Lu; m, \Sigma) = (2\pi)^{-d/2}|\det \Sigma|^{-1/2} e^{-\frac{1}{2}u^\top u}$ and $|\det L| = |\det \Sigma|^{1/2}$, this equals $Z\,(2\pi)^{-d/2} e^{-\|u\|^2/2} = Z\,\eta(u)$. Dividing by the reference density $\eta$ leaves the constant $Z$. □

For the linear-Gaussian calibration model the joint step target *is* Gaussian in $(x_t, x_{t-1})$ — the product of a Gaussian retained law, a linear-Gaussian transition, and a linear-Gaussian observation has a jointly quadratic exponent — so Proposition 40.10 applies exactly under exact hints, and near-Gaussian targets perturb it continuously: under C2 the basis burden is set by the *deviation* from Gaussianity, which is the rank-collapse mechanism the source paper demonstrates for its preconditioned fits (their Figure 2) and the regime its delegated weighted-approximation theory addresses [Cui and Dolgov, 2022, Cui et al., 2023].

**Proposition 40.11** (No geometric Legendre rate for the compactified Gaussian)**.** *Under C3 with map scale $s > 0$, the one-axis fit target at the Gaussian calibration point is $f(z) = c\,(1 - z^2)^{-3/4} \exp\!\big(-a\,\frac{z^2}{2(1-z^2)}\big)$ on $[-1, 1]$, with $a = s^2/2$ and $c > 0$ (the square root of the compactified Gaussian–Jacobian product). Its Legendre coefficients $c_k$ satisfy $\limsup_{k\to\infty} |c_k|^{1/k} = 1$: the expansion converges at no geometric rate, in contrast to the entire (supergeometric) fit targets of C1 and the constant fit target of C2 at the same calibration point.*

*Proof.* Suppose, for contradiction, $\limsup_k |c_k|^{1/k} = 1/\rho < 1$. By the Laplace integral representation $P_k(z) = \frac{1}{\pi}\int_0^\pi \big(z + \sqrt{z^2 - 1}\cos\theta\big)^k d\theta$, every Legendre polynomial obeys $|P_k(z)| \le \big(|z| + |\sqrt{z^2 - 1}|\big)^k \le \rho'^k$ on the closed Bernstein ellipse $E_{\rho'}$ (foci $\pm 1$, parameter $\rho'$), for any $\rho' < \rho$. Hence $\sum_k |c_k|\,|P_k(z)|$ converges uniformly on $E_{\rho'}$, and the Legendre series defines a function $F$ analytic on the open ellipse $\operatorname{int} E_\rho$, which contains the points $\pm 1$ in its interior for every $\rho > 1$, and which agrees with $f$ on $(-1, 1)$. Then $F^4$ is analytic on $\operatorname{int} E_\rho$ and agrees on $(-1, 1)$ with

$$f(z)^4 = c^4\,(1 - z^2)^{-3} \exp\!\Big(-\tfrac{2a\,z^2}{1 - z^2}\Big),$$

which is single-valued and analytic on $\mathbb{C}\setminus\{\pm 1\}$ (the algebraic factor is a rational function; the exponential factor is analytic wherever $z \neq \pm 1$). By the identity theorem on the connected open set $\operatorname{int} E_\rho \setminus \{\pm 1\}$, the two agree there, so $f^4$ extends analytically to $z = 1$. But along $z = 1 + \delta e^{i\pi/4}$, $\delta \downarrow 0$, one has $1 - z^2 = -2\delta e^{i\pi/4} + O(\delta^2)$, hence

$$-\frac{2a\,z^2}{1 - z^2} = \frac{a}{\delta} e^{-i\pi/4} + O(1), \qquad \operatorname{Re}\Big(-\tfrac{2az^2}{1-z^2}\Big) = \frac{a}{\sqrt{2}\,\delta} + O(1) \to +\infty,$$

so $|f^4| \ge C\delta^{-3} e^{a/(\sqrt{2}\delta)} \to \infty$: $f^4$ is unbounded near $z = 1$, contradicting analyticity there. Therefore no $\rho > 1$ exists. □

**Remark 40.4** (Quantitative verification and cost consequences)**.** *The proposition rules out a geometric rate; the practical degrees were computed exactly (Gauss–Legendre quadrature of the coefficient integrals, 2026-08-23 diagnostic script). For relative $L^2$ accuracy $10^{-3}$ on one axis at the Gaussian calibration point, C3 needs per-axis cardinality $\ell = 33$ at map scale $s = 1$ (63 at $s = 0.5$, 21 at $s = 2$); the C1 comparator (entire $\sqrt{\text{truncated Gaussian}}$) needs $\ell = 7$; C2 needs $\ell = 1$ by Proposition 40.10. The reference implementation of Zhao and Cui [2024] corroborates the C3 figure independently: its compactified route runs 33 degrees of freedom per axis. The $\ell$ burden propagates through every cost channel of O5 — design-matrix width $\ell r^2$, mass work $\ell^2$, and compiled-graph size, the channel whose rank-driven variant already produced measured compile-time failures at nine axes — while C2's recorded O5 cost is the tail growth of normalized Hermite values, bounded by $\sim$26 at $|z| = 4$ for degrees up to 16, a region Gaussian-reference rows essentially never exceed.*

### 40.9.6 Selection, decisions, and what remains open

| Property | C1 (box) | C2 (Hermite) | C3 (compactified) |
|---|---|---|---|
| R1 exact normalizer/adjoint | pass | pass | pass |
| R2 recursion well defined | **fail**, $n \geq 4$ | pass | pass |
| O1 truncation mass | geometric in $n$ | 0 | 0 |
| O2 bias floor | irreducible | $O(\tau)$, controlled | $O(\tau)$, controlled |
| O3 $\ell(10^{-3})$, calibration | 7 (via truncation) | 1 | 21–63 |
| O5 cost channel | moot | bounded tail factor | $\ell$-inflated |
| O7 floor domination | **fail**, intrinsic | satisfiable | satisfiable |

C1 is eliminated on three independent grounds: R2 (Lemma 40.1 with the measured cap), the geometric truncation identity with its conditioning bias floor (Propositions 40.2–40.4), and the intrinsic failure of the defensive floor's domination hypothesis (Proposition 40.9). Between the survivors, every derived criterion either ties (R1, R2, O1, O2, O6, O7) or favors C2 (O3 decisively at and near the calibration point; O5 through the $\ell$-driven channels), so the program adopts **C2**, with C3 recorded as the fallback should the C2 derivation or implementation encounter a blocking obstacle — C3 reuses the incumbent algebra (Proposition 40.6) at a permanent per-axis cost that the reference implementation itself pays.

Three program decisions are recorded with this selection. First, the defensive constant moves from the fixed convenience value to the source coupling $\tau_t \leq \|\phi_t - \sqrt{q_t}\|^2_{L^2}$ of Zhao and Cui [2024, Proposition 11], which makes the mixture bias of Proposition 40.8 second order; the fixed value remains for smoke tests only. Second, the defensive density $\lambda$ defaults to the reference density, with the domination requirement of Proposition 40.9 to be re-verified against the declared tail class of each non-Gaussian target, escalating to a heavier-tailed product $\lambda$ where domination demands it. Third, Proposition 40.10 changes what the rank ladder measures: under C2 with exact hints the linear-Gaussian ladder collapses by construction, so it becomes a machine-precision correctness oracle, and the feasibility question — the smallest rank achieving declared accuracy as dimension grows — moves to targets that are genuinely non-Gaussian after preconditioning (the stochastic-volatility family) or to deliberately degraded hints. A ladder that kept the old question under the new construction would pass for the wrong reason.

What remains open is stated so that it cannot be silently promoted: the rank demand of C2 on non-Gaussian targets is an empirical question with no prediction recorded here; the practical Hermite conditioning constant at $n \geq 8$ and the compiled-kernel behavior of Hermite evaluations are engineering measurements yet to be made; and the delegated weighted-approximation theory [Cui and Dolgov, 2022, Cui et al., 2023] is cited as the source of the unbounded-domain error analysis and must be read against the C2 derivation before that derivation is treated as reviewed.

## 40.10 Validation Benchmarks And Promotion Logic

The validation ladder should combine exact or controlled references, local object-validity diagnostics, and promotion rules. Exact or reference-density metrics such as Hellinger distance, relative $L^1$ error, moment error, trajectory RMSE, coverage, and ESS quantiles are meaningful only when the target and branch contracts have already been named. Their role is not to replace the same-scalar contract, but to test whether a declared approximation remains useful after the engineering and branch-validity gates have passed.

For the restarted high-dimensional block, the benchmark architecture should at least cover:

(1) an exact or controlled linear-Gaussian reference where available,

(2) a nonlinear toy case that reveals posterior-shape or retained-object limitations,

(3) a branch-valid finite-difference check on the declared approximate scalar,

(4) structural diagnostics specific to the lane: rank, mass, positivity, PSD, support, Jacobian, or ESS as appropriate.

These ladders are not optional appendices. They are what stops a chapter-local approximation from being promoted on speed or elegance alone.

The benchmark backbone should be named explicitly so the reader does not have to reconstruct it from isolated examples:

(V1) exact-reference linear Gaussian benchmark,

(V2) long-horizon stochastic-volatility benchmark,

(V3) spatial SIR benchmark for high-dimensional partial-observation filtering,

(V4) predator-prey benchmark for nonlinear preconditioning,

(V5) derivative validation table whenever a fixed-branch gradient is part of the claim.

A useful compact benchmark table should pair the resource ledger with the model-specific accuracy ledger and, when derivatives are claimed, with a separate finite-difference table containing $D_h$, $G_{\mathrm{FB}}$, $|D_h - G_{\mathrm{FB}}|$, and the branch-status fields for each $h$. The derivative table is meaningful only when the branch record is identical in the three columns $\theta - h$, $\theta$, and $\theta + h$.

**What each benchmark can and cannot establish.** The linear-Gaussian benchmark can establish exact-reference agreement for the declared run contract, but it cannot establish nonlinear posterior accuracy. The stochastic-volatility benchmark can establish long-horizon nonlinear stability and credible agreement with references or synthetic truth under the declared run contract, but it cannot by itself establish high-dimensional scalability. The spatial SIR benchmark can establish whether the method remains useful at the tested latent dimension, horizon, rank ladder, and basis ladder, but it cannot prove that all spatial graphs or epidemic models admit moderate TT rank. The predator-prey benchmark can establish that nonlinear preconditioning yields a measurable benefit on the tested nonlinear dynamical system, but it cannot prove that such preconditioning is always worth its cost. One-seed demonstrations are diagnostic unless the result artifact explicitly justifies stronger interpretation.

## 40.11 Defect Calculus And Promotion Rules

The chapter closes with a simple rule: runtime, memory, ESS per gradient, rank, and point count can compare methods only after the target, semantic, and numerical veto diagnostics have passed. The industrial promotion logic is therefore conditional:

$$V \text{ passes } \Longrightarrow \text{compare performance}; \qquad V^c \Longrightarrow \text{veto, not ranking.} \tag{40.20}$$

This chapter does not conclude that high-dimensional nonlinear filtering is solved for BayesFilter, that any method is NAWM-ready, that HMC converges, or that any lane should become a production default. It concludes only that a candidate method must pass the declared validation and defect calculus before speed, elegance, or novelty are allowed to matter.

# Part X

# HMC, Geometry, and Diagnostics

# Chapter 41

# HMC for State-Space Likelihoods

This chapter develops fixed-trajectory Hamiltonian Monte Carlo for filtering-based posterior targets. The mathematical foundations follow the standard HMC literature [Neal, 2011, Betancourt, 2017], but implementation claims remain gated by BayesFilter's target, gradient, privacy, and compilation contracts.

## 41.1 Target Contract

For BayesFilter, an HMC transition is valid only relative to the target in Definition 3.1. The sampler needs the scalar log target and its gradient in the coordinates used by the integrator. If the filter supplies a custom score, that score must correspond to the same log likelihood value used in the Metropolis correction.

The HMC chapter therefore depends on the earlier derivative chapters. A filtering backend that can evaluate $\ell(\theta)$ but has no declared gradient policy is value-only evidence. It can be used for optimization, diagnostics, or likelihood comparison, but it should not be promoted to HMC production use.

## 41.2 Implementation Position

TensorFlow Probability provides useful reference implementations of HMC and NUTS, but BayesFilter does not treat TFP NUTS as the production tuning backend. This is a project implementation decision, not a mathematical criticism of NUTS [Hoffman and Gelman, 2014]. The fixed-trajectory HMC path is the promoted TensorFlow/TFP route for model-owned tuning because it keeps the value, gradient, mass-matrix, step-size, and leapfrog-count contracts inspectable.

The reasons to keep TFP NUTS as a reference rather than the default production path are:

- full-trajectory NUTS control flow is hard to keep reliably XLA-compiled across model-specific likelihoods, adaptation logic, and diagnostics;
- nested TensorFlow Probability kernels can obscure whether the complete value-and-gradient pipeline is compiled or only partially traced;
- debugging numerical failures inside TFP kernel internals is slower than debugging a small BayesFilter-owned fixed-trajectory HMC transition;

- large DSGE targets require explicit policies for finite log density, finite gradients, static shapes, filter failure handling, and custom derivative backends.

Consequently, BayesFilter's production direction is a small, inspectable HMC layer whose contracts are designed around filtering likelihoods, analytic/custom gradients, and XLA/JIT discipline. Any future NUTS implementation would need its own reviewed filter-aware contracts before it is promoted beyond reference or diagnostic use.

## 41.3 Promoted Fixed-Trajectory Tuning Algorithm

BayesFilter's promoted tuning lifecycle uses fixed-trajectory HMC rather than a single one-dimensional search over integration time. Ordinary coordinates and one frozen nonlinear transport prepare different scopes, but both pass the same candidate-set controller. The controller is deliberately explicit:

1. Build an initial mass matrix from posterior, pilot, or local covariance information.
2. Regularize that covariance toward a symmetric positive-definite matrix; if dense regularization is not credible, use a recorded diagonal fallback.
3. Choose a broad, finite grid of leapfrog counts $L$, with its coverage rationale and proposal budget recorded in the scope.
4. Qualify one or more step-size proposals independently for every $L$, measure every resulting pair with the same frozen geometry, and retain all non-vetoed candidates.
5. Advance the complete measured cohort through declared validation and fresh verification stages. A passing candidate does not end the cohort.
6. Use acceptance, movement, and numerical health to classify evidence. A typed one-sided directional result may create a new child with the same $L$, mass, target, start bank, and warmup protocol; the child receives fresh verification before not-yet-admitted work for another $L$.
7. Extend inconclusive evidence through the declared finite allocation sequence. At the cap, record `inconclusive_at_cap`. Neither pending nor capped evidence qualifies an unmeasured repair.
8. Return the verified set and immutable parent/child records. A caller chooses a member explicitly; the controller issues no nominee.

The step-size search and the trajectory length must refer to the same HMC kernel. In particular, a reasonable-step calculation performed with one leapfrog step is not valid evidence for a later fixed trajectory with $L > 1$. Preparation screens may seed later proposals, but the shared search measures and verifies each pair at its own fixed $L$.

Operational warmup also distinguishes a screened step from an unscreened proposal. If the bootstrap stage has already passed a fixed-kernel screen at $(L, \epsilon_{\text{screen}})$, the first warmup chart treats $\epsilon_{\text{screen}}$ as a qualified upper bound. Dual averaging may adapt below that value, but the step actually supplied to the inner HMC kernel and the retained handoff step are clamped so that

$$0 < \epsilon_t \leq \epsilon_{\text{screen}}.$$

The artifact records the raw dual-averaging proposal, the bounded next step, the step consumed by HMC, and the number of ceiling hits. Ceiling hits are explanatory diagnostics: they show pressure toward an unqualified region, but do not by themselves prove that the bound is optimal.

When a windowed mass update changes coordinates, the old step bound does not transfer automatically. BayesFilter maps the endpoint into the candidate chart and qualifies a new fixed-$L$ step using four independent momentum probes per candidate. All probes must execute with finite, target-valid states; their acceptance probabilities are aggregated for the bracket. A model-domain failure, such as a non-positive-definite covariance at a proposed parameter value, rejects that epsilon candidate and drives the bracket downward. An unrelated TensorFlow runner or device error remains an implementation failure rather than being misclassified as low acceptance.

If a candidate chart cannot obtain a qualified step, the mass update is rejected and the incumbent chart is retained. This is a non-promoting metric decision, not permission to weaken target-domain assertions or replace the declared target.

This procedure is a tuning and diagnostics policy, not a posterior-validity proof. A passing member means that its own fixed HMC kernel survived the declared fresh verification. The result also records incomplete campaign states: `repair\_budget\_exhausted` means a required directional repair was identified but could not be funded, while `partial\_budget` means other required work remains. Neither status is a scientific rejection. Chapter 42 gives the public interfaces and the exact distinction between tuning evidence and retained-chain evidence. Scientific validity, sampler superiority, default readiness, and GPU readiness require separate evidence. R-hat is reporting-only throughout tuning, including when its computation is unavailable. Posterior warmup and cumulative R-hat/ESS assessment follow the explicit selection of a verified member.

## 41.4 Public And Private Handoffs

The local candidate-set result records every pair, its evidence, and its remaining work. Numerical checkpoints additionally preserve geometry, states, and traces so an interrupted search can resume. An exported member contains the verified kernel and the evidence needed for checked replay. These local files may contain target-specific numerical information; consumers choose what to include in an externally shared report.

Earlier single-kernel archives used separate sanitized public summaries and private mechanics payloads. Their readers preserve that historical split. The current public API returns the full candidate set; retained execution still uses a repository-owned validator and an explicitly selected verified member.

## 41.5 Diagnostic Use of NUTS

NUTS remains valuable as an algorithmic reference and as a diagnostic sampler on small controlled targets. The bounded BayesFilter decision is narrower: the project should not treat TFP NUTS as the default answer to every convergence, geometry, or compilation problem. Larger filtering targets add the hard parts omitted by small benchmarks: spectral derivative risk, missing-data branches, model-specific invalid regions, and backend fallbacks.

# Chapter 42

# HMC Tuning Interfaces

HMC tuning determines which frozen kernels are suitable for further sampling. For each declared trajectory length $L$, BayesFilter measures proposed step sizes $\epsilon$, independently verifies the surviving pairs, and retains every pair that passes. A failed pair can motivate another measured proposal. A passing pair does not end the search or establish posterior convergence.

## 42.1 One candidate-set procedure

The public functions `tune_hmc_kernel` and `tune_fixed_transport_hmc_kernel` share `HMCTuningCandidateSetController`. Both return `HMCTypedCandidateSetRun`, whose `result` contains the full `HMCTuningCandidateSetResult`. Their preparation differs with the target and coordinates; their candidate scheduling and retention do not.

Ordinary `HMCKernelTuningConfig` prepares a windowed mass matrix or an explicit identity metric. `FixedTransportHMCKernelTuningConfig` prepares identity mass in the coordinates of a reconstructable frozen transport. A caller with already prepared geometry supplies `HMCControllerConfig` and the numerical binding's `typed_adapter`. The conditional `TensorFlowHMCKernelTuningConfig` proposal-field route also uses the shared scheduler, but retains its weaker mechanics authority.

The capability registry `HMC_TUNING_INTERFACE_CAPABILITIES` identifies the supported target and coordinate combinations. Chain runners, stage helpers, discovery routines, and historical selectors cannot independently issue new candidate-set tuning evidence. Unsupported legacy runner customizations fail before execution. Historical single-kernel results remain readable, with their original identities; they are not the result of the current procedure.

Listing 42.1: Inspecting supported target and coordinate routes.

```
"""Select an HMC tuning route from BayesFilter's capability registry."""

from __future__ import annotations

import sys
from pathlib import Path

REPOSITORY_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(REPOSITORY_ROOT))

from bayesfilter.inference import (
```

```
    active_hmc_tuning_routes,
    hmc_tuning_interface_capability,
)


def require_artifact_authority(interface_name: str) -> str:
    """Return the qualified public tuner name or reject a lower-level route."""

    capability = hmc_tuning_interface_capability(interface_name)
    if capability.interface_kind != "public_tuner" or not capability.
    artifact_authority:
        replacement = capability.replacement or "no supported replacement"
        raise ValueError(
            f"{capability.qualified_name} is {capability.interface_kind}, not a "
            f"public artifact-authority tuner; use {replacement}"
        )
    return capability.qualified_name


ordinary = require_artifact_authority("tune_hmc_kernel")
fixed_transport = require_artifact_authority("tune_fixed_transport_hmc_kernel")
assert {record.interface_name for record in active_hmc_tuning_routes()} == {
    "tune_hmc_kernel",
    "tune_fixed_transport_hmc_kernel",
}

try:
    require_artifact_authority("run_full_chain_neural_force_hmc")
except ValueError as error:
    expected_rejection = str(error)
else:  # pragma: no cover - the registry contract requires rejection
    raise AssertionError("a chain runner was incorrectly accepted as a tuner")

try:
    require_artifact_authority("tune_hmc_kernel_robust_broad_grid")
except ValueError as error:
    expected_diagnostic_rejection = str(error)
else:  # pragma: no cover - the registry contract requires rejection
    raise AssertionError("a diagnostic helper was incorrectly accepted as a tuner")

print(ordinary)
print(fixed_transport)
print(expected_rejection)
print(expected_diagnostic_rejection)
```

## 42.2 Preparation and immutable geometry

Geometry is a shared responsibility rather than an unspecified tuner default. The caller supplies the initial center and may supply a local geometry hypothesis; the tuner validates that hypothesis and owns the subsequent affine construction and adaptation. Under windowed adaptation, it tries a supplied negative log-posterior Hessian, an initial covariance, and positive diagonal parameter scales, in that order, before falling back to identity. Thus a negative Hessian means

$$H(\theta_0) = -\nabla^2_\theta \ell(\theta_0), \qquad \Sigma_0 \approx H(\theta_0)^{-1}, \tag{42.1}$$

in the same unconstrained coordinates as the supplied center. With geometry fallback enabled, a rejected higher-priority hypothesis is recorded before the next is tried; with fallback disabled, it is a hard error. The explicit fixed- identity policy ignores all hints. Identity is a useful mechanics baseline, not evidence that an anisotropic posterior has identity geometry.

When no hint is supplied, the initializer uses unit curvature frequencies; it does not estimate the target Hessian. A finite density or a near-zero score at the initial point therefore does not establish a stable initial step. Strongly scaled targets can fail the bootstrap before windowed adaptation or candidate search begins. Inspect that preparation failure and supply checked geometry in the active coordinates when available. Increasing the subsequent candidate budget does not repair a bootstrap that never reaches its handoff.

Bootstrap epsilon decisions use the mean Metropolis probability over all recorded proposals, including rejected proposals and excluding discarded burnin:

$$\bar{\alpha} = \frac{1}{n} \sum_{t=1}^{n} \exp\{\min(\log r_t, 0)\}.$$

The realized accepted fraction includes the additional uniform accept/reject draw and remains a separate movement diagnostic. For example, a recorded mean probability of 0.6846 lies within the configured $[0.65, 0.75]$ band even when $13/16 = 0.8125$ proposals were accepted. That binary fraction must not cause an upward epsilon repair. Missing, nonfinite or incomplete probability traces stop the screen. The ordinary fixed-kernel band and the optional startup floor have different preparation roles; neither establishes final candidate or posterior validity.

A target-domain exception permits a bounded smaller bootstrap trial only if the adapter specifically classifies it and the repository failure record locates it in a proposal target callback after a finite pre-transition state. Initial-state, retained-state, unattributed and unrelated execution failures remain terminal. The failed round retains its original assertion and first failure; it receives no invented acceptance probability. A smaller finite parent and the failed epsilon can nominate a logarithmic midpoint; otherwise the configured multiplier reduces epsilon. The next trial recomputes $L$, records any clamp and consumes the existing repair budget. A failed reusable runner is replaced, and only a subsequent completed screen can supply the repaired preparation pair. Target assertions and Metropolis correction remain part of every trial.

An optional finite startup probe is available through `HMCKernelTuningConfig(bootstrap_initialization_rounds=20)`. Zero is the default; a positive integer bounds the number of rounds. Each round tests four independent momentum proposals and shrinks the step when proposals are invalid or acceptance is below the configured lower repair floor. The probe preserves the target, affine mass, failed proposals and separate seed stream. Its selected pair must still pass fresh bootstrap and operational adaptation. In this mode, bootstrap uses its `warmup_startup_only` role: high acceptance does not trigger upward refinement before adaptation, and the starting step is not declared a stability ceiling.

Probe wall times remain in the geometry's full artifact hash for audit, but are excluded from the separate numerical geometry hash used by new candidate scopes. Otherwise, two identical preparations could receive different candidate seeds merely because a probe took longer. Numerical probe outcomes, seed provenance, mass and starts remain bound to the numerical identity. Existing checkpoints retain their saved scopes and streams.

The probe follows the declared target-status policy. With `none`, it checks finite retained and proposed states, values and scores without manufacturing status data. With `per_chain_step`, complete valid telemetry is also required. Invalid initial or retained states stop preparation. A finite startup nomination neither qualifies a final candidate nor establishes convergence.

Preparation success need not mean that the mass changed. The `allow_valid_incumbent` policy permits zero empirical updates; inspect `operational_metric_update_count` when the initial geometry is poorly scaled. A regression diagnosis retained identity geometry throughout preparation and subsequently failed posterior equilibration despite verified tuning candidates. Ordinary operational preparation runs one chain; it constructs the four-chain bank for candidate verification afterwards. Its metric assessment preserves temporal order. For explicitly supplied multiple chains, temporal information is estimated separately within each chain. Split R-hat is reported when available and never vetoes or delays a metric proposal.

The proposed position covariance is the finite-window centered second moment, divided by $N - 1$, with correlations shrunk toward zero while preserving marginal variances. It is a geometry hypothesis during adaptation, not an unbiased posterior covariance estimate. State-count and temporal-information floors, numerical rank, condition and regularization-discrepancy checks screen this hypothesis. Positive-definiteness, coordinate reconstruction, target value/score parity and a fresh reasonable-step check are required before using its transform.

Ordinary preparation records each completed window's metric decision and failed checks in `preparation_progress.json`. It also reports whether the scheduled windows can meet the state-count floors. The `standard` preset is a local diagnostic allocation: with the default metric policy, its 150 transitions in six dimensions contain slow windows of 30, 60 and 30, each shorter than the dense minimum of 64. The explicit `serious` preset funds a larger preparation. Inspect its actual decisions as well; neither a larger allocation nor an accepted metric update establishes posterior equilibration. Rejections preserve the incumbent and identify whether evidence, transform construction, affine parity or step qualification failed.

At a metric change, the fresh step search starts from the previous coordinate system's bounded dual-averaging average. Clipping in log space before exponentiation prevents a large unconstrained adaptation proposal from exhausting the search before any usable step is tested. This old bound limits only the initial hypothesis: the search may expand in the new coordinates and must independently qualify its step. Rejected probes preserve their starting value and attempts. A nonfinite warmup trace identifies its failed fields and first indices; the failure still vetoes preparation.

The optional `metric_evidence_policy="finite_window"` keeps the same covariance formula and numerical checks while treating temporal ESS as explanatory. The default `temporal_information` retains the ESS floor. With the optional policy, automatic preparation allocates slow windows large enough to reach the existing dense state-count floor when the declared budget permits, or the diagonal floor when only that is affordable. It preserves the total transitions and initial/final buffers, merging an undersized final slow remainder. Insufficient total capacity remains visible. This schedule change removes avoidable count failures; it does not establish adequate covariance information or posterior equilibration. Directly supplied schedules and the default policy retain their declared window lengths. This option permits an early geometry proposal before the chain has explored well enough to estimate posterior covariance precisely. For any fixed nonsingular

affine factor $A$, $\theta = c + Az$ has transformed score $A^\mathsf{T}\nabla_\theta \ell$; exact-score Metropolis-adjusted sampling in the transformed coordinates does not require $AA^\mathsf{T}$ to equal the posterior covariance. Its usefulness still depends on exploration and numerical health. Adaptation must finish before candidate verification and retained draws.

The option remains experimental. Its first matched regression preparation applied two updates and later failed a nonfinite warmup trace. The existing reasonable-step probe checks a few momenta at one state and cannot guarantee stability everywhere visited later. Preparation failures remain failures, and neither policy turns metric qualification into posterior convergence.

The optional `metric_probe_num_results` extends each of the four independent metric-boundary probes to a short chain at fixed epsilon and fixed leapfrog count. Its default is one transition. A larger allocation, for example sixteen, tests evolving positions and checks every proposed and retained endpoint, score, momentum and energy correction before the existing acceptance bracket can qualify the step. The evidence records the probe length. Retained-state inconsistency or unclassified execution failure stops preparation; a failed proposal rejects that probe. This remains an experimental preparation option: short chains cannot guarantee global stability. Later warmup windows, candidate measurement and independent verification still require their own numerical checks. It neither adds R-hat admission nor retries an invalid warmup window.

The separate optional `preparation_max_restarts` bounds recovery from rejected nonfinite proposals; its default is zero. Recovery requires finite retained states, values and scores, consistent acceptance decisions and valid declared telemetry. The entire failed preparation attempt is archived and its adaptation statistics discarded. A full new schedule starts from the failed window's independently validated starting state and qualified transform, with fresh streams and reset covariance and dual-averaging statistics. A new probe may only shrink from half the consumed failing step. High acceptance can nominate this conservative preparation step; it does not relax final candidate acceptance or independent verification. A changed metric still requires fresh qualification in its new coordinates. All discarded work and probe work counts toward finite attempt and execution budgets. Retained or shared invalidity and unclassified execution errors remain fatal. This ordinary-preparation option is experimental: three restarts is a development allocation, not a universal robustness guarantee.

Listing 42.2: Binding a local covariance estimate in the target coordinates.

```
"""Bind an accepted local covariance to the ordinary public HMC tuner.

The executable ‘‘main‘‘ uses stubs, so this listing can be checked without
estimating a covariance or launching HMC.
"""

from __future__ import annotations

import sys
from collections.abc import Callable, Mapping, Sequence
from pathlib import Path
from types import SimpleNamespace
from typing import Any

REPOSITORY_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(REPOSITORY_ROOT))
```

```
from bayesfilter.inference import (
    HMCKernelTuningConfig,
    estimate_sequential_map_covariance,
    tune_hmc_kernel,
)


def tune_from_fresh_local_covariance(
    *,
    adapter: Any,
    value_and_score_fn: Callable[[Any], tuple[Any, Any]],
    initial_positions: Sequence[Any],
    config: HMCKernelTuningConfig,
    covariance_kwargs: Mapping[str, Any] | None = None,
    covariance_estimator: Callable[..., Any] = estimate_sequential_map_covariance,
    tuner: Callable[..., Any] = tune_hmc_kernel,
) -> Any:
    """Estimate, accept, and bind one center/covariance pair in one coordinate system.
    """

    covariance_result = covariance_estimator(
        value_and_score_fn,
        initial_positions,
        **dict(covariance_kwargs or {}),
    )
    if not covariance_result.accepted:
        raise RuntimeError("local covariance was not accepted")
    if covariance_result.map_candidate is None or covariance_result.covariance is None
    :
        raise RuntimeError("accepted covariance result is incomplete")
    return tuner(
        adapter=adapter,
        initial_position=covariance_result.map_candidate,
        initial_covariance=covariance_result.covariance,
        config=config,
    )


def main() -> Mapping[str, Any]:
    """Exercise only the covariance-to-tuner argument binding."""

    accepted = SimpleNamespace(
        accepted=True,
        map_candidate=(0.25, -0.25),
        covariance=((2.0, 0.1), (0.1, 0.75)),
    )
    observed: dict[str, Any] = {}

    def covariance_stub(*_args: Any, **_kwargs: Any) -> Any:
        return accepted

    def tuner_stub(**kwargs: Any) -> Mapping[str, Any]:
        observed.update(kwargs)
        return {"status": "arguments_bound_without_hmc"}

    result = tune_from_fresh_local_covariance(
        adapter=object(),
        value_and_score_fn=lambda position: (position, position),
```

```
        initial_positions=((0.0, 0.0),),
        config=HMCKernelTuningConfig.smoke(
            target_scope="docs_covariance_first_binding"
        ),
        covariance_estimator=covariance_stub,
        tuner=tuner_stub,
    )
    assert observed["initial_position"] == accepted.map_candidate
    assert observed["initial_covariance"] == accepted.covariance
    assert result["status"] == "arguments_bound_without_hmc"
    return result


if __name__ == "__main__":
    print(main()["status"])
```

`prepare_operational_windowed_mass_handoff` freezes the operational metric and actual post-warmup start bank. The numerical factory `bind_hmc_candidate_set_execution_from_preparation` preserves both its bootstrap and final affine layers. The explicit `bind_hmc_candidate_set_execution` factory instead accepts a frozen mass or supported frozen transport payload. A bound scope records target, data/prior lineage, source dependencies, coordinates, metric, starts, warmup, dtype, backend, and XLA settings. Changing them requires a new scope.

For the proposal-field preparation, four raw-coordinate starts $q_i$ and positive parameter scales $s$ give the initial factor $A = \mathrm{diag}(s)$:

$$c = \frac{1}{4}\sum_{i=1}^{4} q_i, \qquad z_i = (q_i - c)A^{-\mathsf{T}}, \qquad q_i = c + z_i A^{\mathsf{T}}. \tag{42.2}$$

Here starts are represented as row vectors. Caller order is preserved. A diagnostic may explicitly replicate one start; candidate preparation requires its declared four-chain bank. Affine metric windows use $L = 1$ and the configured adaptation counts. Their proposals receive fresh fixed-kernel measurements.

The ordinary preparation config and numerical binding default to GPU/XLA execution. TensorFlow memory growth is configured before device initialization. Small CPU/non-XLA checks are explicit reference or debugging exceptions. Execution policy does not qualify an arbitrary target: the binding still checks its full-chain capability and the consumer must establish value/score fidelity.

## 42.3 Broad search, repairs, and further evidence

The ordinary convenience grid is $L \in \{3, 5, 9, 13, 18, 25\}$, and its legacy preparation config fixes the maximum at 25. The proposal-field config declares its own maximum. Supplied initial, refinement and expansion grids must obey the applicable maximum. An explicit `epsilon_by_l` may supply several proposals at each $L$; `initial_epsilon` is a warm start when no grid is supplied. Every pair is measured independently. A shared numeric proposal at two values of $L$ transfers no acceptance evidence between them. The final-metric epsilon from operational preparation supplies the starting step and the inherited search ceiling. The short probe does not establish a stability limit or guarantee a suitable pair below that ceiling for every $L$. An explicit `candidate_search_bound_expansion_steps=1` widens the exploration cap by the existing repair factor, preserving the final

metric and coordinates. Zero retains the inherited ceiling. Every explored pair still requires its own health checks, measurement and fresh verification. This optional setting is a search hypothesis whose calibration remains separate from its implementation.

Direct preparation bindings expose `preparation_bound_expansion_steps`. They intersect the requested domain with the finite expanded cap, save the original bound and final metric/coordinate identities, and issue a new search identity. Existing scopes and verification receipts are immutable. Preparation failures preserve their stage and available veto causes in `preparation_progress.json` before raising an exception. Nonfinite diagnostic values are marked explicitly in the JSON record, preserving the original failure even when its numerical summaries cannot be finite.

A multiplicative repair can jump from high acceptance directly to nonfinite trajectories. That nonfinite result supplies no valid acceptance direction, so it cannot form the opposing-evidence interval used by refinement. The optional `explore_failed_intervals=True` controller setting allows funded refinement rounds to explore the unvisited geometric interior between a valid directional parent and its nearest numerically rejected same-$L$ child. Each proposal is a new same-family child of the valid parent; both source receipts are recorded, and the failed endpoint stays rejected. This interval is an exploration hypothesis, not a stability bound or an assertion of monotone acceptance. Missing telemetry, unknown invalidity, invalid retained states and shared corruption cannot supply such endpoints. Family, candidate and work limits still apply, and every proposed pair needs measurement and independent verification. The option is disabled by default. An explicit intermediate grid in a fresh scope remains available. Neither procedure exhausts the continuous epsilon domain.

The automatic preparation translation enables one pilot per $L$ and one bounded refinement round. A direct controller config can disable these optional stages for an explicitly supplied grid. A pilot proposes a frozen pair; measurement and fresh verification qualify it. Refinement applies declared epsilon factors and additional $L$ hypotheses to every surviving family. Exact pairs are deduplicated, and all proposals remain subject to candidate and work limits. The reciprocal convenience factors $(0.8, 1.25)$ are hypotheses about useful nearby proposals, not universal numerical optima.

Work proceeds in closed cohorts. Every funded admitted peer receives its reserved stage before a repair child enters the next cohort. A peer whose full required work cannot fit the remaining budget is explicitly deferred, permitting other affordable reserved work to finish. Its evidence allocation is preserved and the search remains incomplete. A first pass does not remove later candidates. Directional evidence creates an immutable child with the same $L$, mass, target, coordinates, and initial bank. The child has a new epsilon and must receive its own measurement and verification. A parent may fail movement because an oversized step rejects nearly everything; that promotion veto does not suppress a supported smaller-epsilon repair. A trajectory alert can propose only declared additional $L$ values, as separate candidates. It never silently changes an existing candidate's $L$.

The distinction between proposal and evidence matters for fixed-length HMC. For a transition from $x$, the acceptance probability is

$$a(\epsilon, L \mid x) = \mathbb{E}[\min\{1, \exp(-\Delta H)\} \mid x]\,. \tag{42.3}$$

The reported probability mean averages the computed Metropolis probabilities over visited states and sampled momenta. The realized acceptance rate also contains the uniform accept/reject decisions: for each proposal with probability $a$, that indicator has expec-

tation $a$. Both summaries describe acceptance in the explored region; neither measures convergence. For the unit harmonic oscillator, one leapfrog matrix has determinant one and trace $2 - \epsilon^2$. In the stable range its conjugate eigenvalues therefore give the phase

$$\vartheta(\epsilon) = \arccos(1 - \epsilon^2/2), \qquad 0 < \epsilon < 2. \tag{42.4}$$

The trajectory phase is $L\vartheta(\epsilon)$. Near a return phase, acceptance can be high with little net movement. A multiplicative epsilon repair is thus another hypothesis to measure. The controller remembers the latest valid directional evidence for each exact pair and prioritizes unvisited geometric interior points toward nearby opposing observations. It retains this evidence across restart. Finite refinement also investigates unresolved directional intervals when no original family survives. It does not assume monotone acceptance or infer a pass at an unmeasured point. When a proposal crosses the declared domain, its unvisited boundary can be measured before that direction is exhausted.

Inconclusive measurement or verification receives the predeclared `evidence_rungs`; the convenience multipliers are $(1, 2, 4)$. Each rung is an independently seeded run of the unchanged candidate. Unfinished work stays resumable; exhaustion gives `inconclusive_at_cap`. The reported compatibility intervals support an operational screen, not a claim of nominal confidence coverage across repeated looks. A cap does not turn an inconclusive candidate into a verified member.

## 42.4 Acceptance, numerical health, and posterior diagnostics

Acceptance probabilities fluctuate even under a fixed kernel. A temporal block below the practical band and another above it therefore do not by themselves show a change in the chain's acceptance distribution. The default raw crossing screen is a heuristic, and its separate four-chain compatibility interval does not calibrate the crossing decision. Independent random seeds also do not make four differently initialized chain means identically distributed.

For a stationary acceptance sequence $a_{it}$ with an applicable central limit theorem, uncertainty in its mean depends on the long-run variance $\sigma_i^2 = \gamma_i(0) + 2\sum_{k\geq 1}\gamma_i(k)$, where $\gamma_i(k)$ is its lag-$k$ autocovariance. Independence between $m$ chains of length $n$ then gives the approximation

$$\widehat{\mathrm{Var}}(\bar{a}) = \frac{1}{m^2 n}\sum_{i=1}^{m}\widehat{\sigma}_i^2.$$

This follows by adding the variances of the independent chain means and dividing by $m^2$. Batch means and lugsail estimate these long-run variances; the formulas and assumptions are discussed in the diagnostics chapter. A positive estimate cannot establish adequate batch length, stationarity or sufficient burn-in.

An experimental acceptance diagnostic now preserves the original tuning decision while computing these uncertainty estimates separately. It compares all temporal windows within each chain, so opposing drifts cannot cancel by averaging across chains. It subtracts simultaneous marginal interval endpoints: if both means are covered, their difference is covered regardless of their covariance. A difference interval wholly outside a declared tolerance supports a material discrepancy; an interval wholly inside supports compatibility for that contrast; overlap with the tolerance boundary remains unresolved.

The approximate marginal intervals still require calibration, including across the declared evidence looks and candidate search.

The October 1 validation did not support a new default. Under strongly persistent stationary simulations, the tested batch rules underestimated uncertainty and reported false conflicts. Increasing the batch length reduced some conflicts without establishing adequate coverage. The new diagnostic therefore has no tuning-artifact authority. Its finite-start model checks and BGS replays are separate from posterior convergence, and R-hat remains reporting-only during tuning.

The default `HMCAcceptancePolicy` uses four independent chain means, temporal blocks, and a 90% compatibility interval. Its first allocation requires at least four blocks of sixteen decisions per chain. After health, chain-conflict, temporal-conflict and trajectory checks, acceptance passes when the compatibility interval overlaps the practical band, lies inside the repair band, and every chain mean lies inside the repair band. For example, mean 0.76 and interval $[0.73, 0.79]$ can pass with practical band $[0.65, 0.75]$ and repair band $[0.55, 0.85]$. The rule tests compatibility; it does not rank candidates by closeness to 0.70. Acceptance, movement and numerical health have distinct roles. Finite accepted/proposed states and targets, endpoint score finiteness, momentum, Metropolis state consistency and declared target status determine whether the numerical evidence is usable. Candidate-local failure rejects that pair; shared execution corruption stops the scope and disables all its replay. A declared target-domain exception preserves the attempted seeds and failure without inventing an acceptance value, so other candidates can continue. Shared invalidity preserves the history of earlier verification while revoking current replay. Released reservations cause affordable deferred work to be reconsidered within the same invocation. Available native divergence vetoes promotion of the current pair. When the acceptance evidence is valid and supports a directional repair, that veto does not prevent a newly measured child. Acceptance compatibility and promotion eligibility are separate typed fields, so a rejected pair's in-band acceptance receipt remains readable alongside its verified peers. Both routes check numerical health over discarded warmup as well. The proposal-field route checks finite states, log acceptance and energy errors before excluding warmup from acceptance statistics.

$\widehat{R}$ is reporting-only during tuning. High, missing, nonfinite or failed-to-compute values cannot reject, rank, repair or delay a candidate. ESS and short-chain timing are descriptive diagnostics. They do not silently replace the acceptance/health screen or discard a viable member. Posterior warmup and cumulative model-coordinate $\widehat{R}$/ESS assessment remain separate requirements for inference.

## 42.5 A frozen nonlinear transport

Let the base log density in model coordinates be $\ell_\theta(\theta)$, and let one frozen differentiable bijection map latent coordinates into those model coordinates,

$$\theta = T(z), \qquad J_T(z) = \frac{\partial T(z)}{\partial z}.$$

The fixed-transport HMC target is

$$\ell_z(z) = \ell_\theta(T(z)) + \log \left|\det J_T(z)\right|, \tag{42.5}$$

and its matching score is

$$\nabla_z \ell_z(z) = J_T(z)^\mathsf{T} \nabla_\theta \ell_\theta(T(z)) + \nabla_z \log \left|\det J_T(z)\right|. \tag{42.6}$$

Dropping either Jacobian term changes the target or its derivative. Such a calculation is wrong relative to the transformed-target claim.

The fixed-transport preparation reconstructs the full diagonal-affine or dense-IAF map from `frozen_transport_payload`, checks its live identity, and uses identity mass in latent coordinates. Starts are supplied in those coordinates. Every proposed $(L, \epsilon)$ then enters the same measurement, repair and fresh-verification procedure. Arbitrary transport callbacks without a supported reconstruction are rejected. Changing a map changes the scope; receipts or chain states from different maps are never pooled.

Positive funnel tuning tests assume a supplied frozen whitening or partially whitening map. Learning a useful map is a separate preparation problem. For the model $v \sim \mathcal{N}(0, \sigma^2)$ and $x_i \mid v \sim \mathcal{N}(0, e^v)$ with two children, the exact map $v = \sigma z_0$, $x_i = e^{v/2} z_i$ has log determinant $\log \sigma + v$. Substitution in the transformed density cancels both conditional scale terms and gives three independent standard normals. This provides an algebraic control without assuming that a learner discovers the map. A partial map $x_i = e^{s(v)} z_i$ instead leaves conditional standard deviation $e^{v/2 - s(v)}$ in the latent coordinates; it tests tuning with known residual curvature.

Bounding $s(v)$ alone does not bound the residual stiffness: if $s$ remains bounded as $v \to -\infty$, the latent conditional precision $e^{2s(v)-v}$ diverges. A useful diagnostic partial-whitening family instead sets $s(v) = v/2 + \delta(v)$ with $\delta(v) = a \tanh(v/(2\sigma))$. For $S = z_1^2 + z_2^2$, its log determinant is $\log \sigma + v + 2\delta(v)$, and substitution in the normalized funnel density yields

$$U(z) = \tfrac{1}{2} z_0^2 + \tfrac{1}{2} e^{2\delta(\sigma z_0)} S - 2\delta(\sigma z_0) + \tfrac{3}{2} \log(2\pi).$$

The child-coordinate curvature lies in $[e^{-2a}, e^{2a}]$ for $a \geq 0$; the zero-amplitude limit recovers the exact control. The entire Hessian is still unbounded because derivatives with respect to $z_0$ contain $S$. The validation fixture uses $\sigma = 3$ and $a = 1/2$ as an explicit development hypothesis, not a learned-map or sampling default. Even the exact latent Gaussian control can require many draws for precise means of the model's heavy-tailed children.

The untransformed centered funnel remains a stress test for bounded search and honest numerical rejection. An empty candidate set there does not by itself establish a tuning defect. For a supplied map, tuning searches the declared step sizes and trajectory lengths in the frozen coordinates and independently verifies every survivor. A finite search need not succeed for every imperfect map. Comparisons at common model starts must apply the inverse chart and inverse map before passing latent starts to the tuner, then check the forward roundtrip. Posterior precision and exploration in the original model remain separate from candidate qualification.

Listing 42.3: Fixed-transport candidate-set tuning.

```
"""Small CPU/reference candidate-set example; no posterior or ranking claim."""
from __future__ import annotations
import os
import sys
from pathlib import Path
os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1")
os.environ.setdefault("TF_FORCE_GPU_ALLOW_GROWTH", "true")
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
import tensorflow as tf
from bayesfilter.inference import (
    HMCControllerConfig, HMCCandidateExecutionConfig, HMCAcceptancePolicy,
    HMCKernelTuningConfig, FixedTransportHMCKernelTuningConfig,
```

```
    ValueScoreCapability, tune_hmc_kernel, tune_fixed_transport_hmc_kernel,
)

class AnalyticGaussianAdapter:
    parameter_dim = 2
    def adapter_signature(self):
        return "docs-candidate-set-gaussian-v1"
    def value_score_capability(self):
        return ValueScoreCapability(value_score_authority="graph_native",
    xla_hmc_ready=False,
            runtime_backend="tensorflow", target_scope="docs_candidate_set",
            evidence_path=__file__, nonclaims=("small CPU reference fixture",))
    def log_prob_and_grad(self, position):
        position = tf.convert_to_tensor(position, tf.float64)
        return -.5 * tf.reduce_sum(position**2, axis=-1), -position

adapter = AnalyticGaussianAdapter()
starts = tf.constant([[-1.,-.5],[-.3,.2],[.4,-.2],[1.,.5]], tf.float64)
# This reduced grid bounds the documentation smoke; the ordinary convenience
# policy uses L=(3,5,9,13,18,25) and optional pilots/refinement.
search = HMCControllerConfig(primary_l_grid=(3,5),
    epsilon_by_l=((3,(1.1,1.3)),(5,(1.1,1.3))), evidence_rungs=(1,))
execution = HMCCandidateExecutionConfig(measurement_num_results=64,
    verification_num_results=64, num_warmup_steps=8, seed=(20260914,71),
    acceptance_policy=HMCAcceptancePolicy(), target_status_trace_policy="none",
    use_xla=False, non_xla_reason="small documentation CPU reference")

from bayesfilter.inference.neutra_artifacts import load_frozen_neutra_artifact
frozen = {"schema":"bayesfilter.neutra.frozen_affine_diag.v1",
    "transport_id":"docs-identity", "dimension":2,
    "target_signature":adapter.adapter_signature(), "log_jacobian_available":True,
    "shift":[0.,0.], "raw_scale":[0.,0.]}
loaded = load_frozen_neutra_artifact(frozen, expected_target_signature=adapter.
    adapter_signature())
run = tune_fixed_transport_hmc_kernel(base_adapter=adapter, fixed_transport=loaded.
    transport,
    frozen_transport_payload=frozen, initial_position=starts,
    config=FixedTransportHMCKernelTuningConfig(initial_step_size=1.3,
        leapfrog_grid=(3,5), use_xla=False, target_scope="docs_candidate_set"),
    search_config=search, execution_config=execution,
    target_lineage={"model":"standard Gaussian", "data":"none", "prior":"standard
    normal"},
    source_paths=[__file__])
assert run.result.payload()["nominee_id"] is None
assert len(run.result.candidates) >= 4
print(run.result.completion_status, run.result.verified_candidate_ids)
```

## 42.6 Typed deterministic proposal-field mechanics

`bind_neural_force_hmc_tuning_runner` binds a declared position-only field to its coordinate-consistent endpoint potential. The public facade accepts that binding with `TensorFlowHMCKernelTunin` Affine preparation remains specific to this transition, while the shared controller schedules its candidate measurements, repairs and verification. The result is conditional mechanics evidence and cannot become an exact-score numerical retained member. The endpoint function still needs target-specific validation.

The typed objects use potential, not log-density, signs. If $\ell(\theta) = \log \pi(\theta)$, the exact endpoint object is

$$U(\theta) = -\ell(\theta). \tag{42.7}$$

For the scalar-potential-gradient semantic, the proposal force is $F(\theta) = \nabla U(\theta) = -\nabla \ell(\theta)$. The kick uses

$$p_{n+1/2} = p_n - \frac{\epsilon}{2} F(q_n), \qquad q_{n+1} = q_n + \epsilon M^{-1} p_{n+1/2}, \tag{42.8}$$

followed by the symmetric terminal kick and momentum flip.

The v2 binding also permits an honestly labeled deterministic position-only proposal field $B(q)$ that need not equal $\nabla U(q)$. Position-only kicks remain volume-preserving shears, and the symmetric schedule remains reversible. The exact endpoint Metropolis rule is

$$\alpha = \min\left\{1, \exp\left[-U(q') - K(p') + U(q) + K(p)\right]\right\}. \tag{42.9}$$

Under these conditions, the endpoint potential determines the invariant target; a poor or sign-reversed proposal field can destroy efficiency without by itself changing that target. A wrong endpoint potential, missing coordinate change, momentum-dependent field, asymmetric schedule, or direct state update invalidates that conclusion.

For an affine mass coordinate $\theta = c + Fz$, the repository wrapper applies the chain rule to the proposal field and endpoint potential. The affine log-Jacobian is constant and is deliberately omitted from energy differences under the declared `constant_omitted` convention. Both raw objects and that convention are bound into runner schema `bayesfilter.hmc_tuning_runner_binding.v2`.

Listing 42.4: Typed proposal field and exact endpoint-potential binding.

```
"""Construct the typed deterministic-field tuning inputs without launching HMC."""

from __future__ import annotations

import sys
from collections.abc import Mapping
from pathlib import Path
from typing import Any

REPOSITORY_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(REPOSITORY_ROOT))

import tensorflow as tf

from bayesfilter.inference import (
    DETERMINISTIC_POSITION_ONLY_PROPOSAL_FIELD_SEMANTICS,
    FourChainMeanBandAcceptancePolicy,
    FrozenPositionOnlyForce,
    FrozenTargetPotential,
    TensorFlowHMCKernelTuningConfig,
    bind_neural_force_hmc_tuning_runner,
    tune_hmc_kernel,
)


def build_binding() -> Any:
    """Bind a deterministic non-gradient proposal field to an exact potential."""

    def proposal_field(position: tf.Tensor) -> tf.Tensor:
```

```
        position = tf.convert_to_tensor(position, tf.float64)
        return tf.stack(
            (
                position[..., 0] + 0.2 * position[..., 1],
                0.4 * position[..., 1],
            ),
            axis=-1,
        )

    def exact_endpoint_potential(position: tf.Tensor) -> tf.Tensor:
        position = tf.convert_to_tensor(position, tf.float64)
        return 0.5 * tf.reduce_sum(tf.square(position), axis=-1)

    return bind_neural_force_hmc_tuning_runner(
        force=FrozenPositionOnlyForce(
            function=proposal_field,
            identity="docs-deterministic-proposal-field-v1",
            semantics=DETERMINISTIC_POSITION_ONLY_PROPOSAL_FIELD_SEMANTICS,
            coordinate_system="raw",
        ),
        target=FrozenTargetPotential(
            function=exact_endpoint_potential,
            identity="docs-exact-endpoint-potential-v1",
            coordinate_system="raw",
        ),
        target_scope="docs_neural_force_binding",
    )


def build_mechanics_config() -> TensorFlowHMCKernelTuningConfig:
    """Declare an intentionally tiny, non-authoritative documentation budget."""

    return TensorFlowHMCKernelTuningConfig(
        parameter_dimension=2,
        evidence_role="diagnostic_only",
        mass_window_results=(1,),
        step_adaptation_results=64,
        verification_results=64,
        max_leapfrog_steps=5,
        initial_step_size=0.1,
        budget_provenance="bounded documentation mechanics diagnostic",
        initial_step_size_provenance="documentation convenience hypothesis",
        geometry_provenance="unit-scale documentation diagnostic",
        target_scope="docs_neural_force_binding",
        acceptance_policy=FourChainMeanBandAcceptancePolicy(
            overall_band=(0.65, 0.75),
            per_chain_band=(0.55, 0.85),
        ),
        target_accept_prob=0.70,
        verification_repair_rounds=0,
        step_repair_factor=2.0,
        mass_shrinkage=0.10,
        covariance_jitter=1.0e-9,
        eigenvalue_floor=1.0e-9,
        max_condition_number=1.0e8,
        seed=(20260905, 1),
    )
```

```
def tune_deterministic_field(
    *,
    adapter: Any,
    initial_position: Any,
    parameter_scales: Any,
    output_dir: str | Path,
) -> Any:
    """Show the only valid public dispatch for this mechanics-only branch."""

    return tune_hmc_kernel(
        adapter=adapter,
        initial_position=initial_position,
        parameter_scales=parameter_scales,
        config=build_mechanics_config(),
        runner_binding=build_binding(),
        output_dir=output_dir,
    )


def main() -> Mapping[str, Any]:
    """Return both typed payloads; no tuner or chain runner is called."""

    binding_payload = build_binding().payload()
    config_payload = build_mechanics_config().payload()
    assert binding_payload["artifact_authority"] is False
    assert binding_payload["force_semantics"] == (
        DETERMINISTIC_POSITION_ONLY_PROPOSAL_FIELD_SEMANTICS
    )
    assert binding_payload["endpoint_target_coordinate_system"] == "raw"
    assert config_payload["artifact_authority"] is False
    assert config_payload["trajectory_candidate_policy"] == (
        "powers_of_two_then_explicit_cap"
    )
    return {"binding": binding_payload, "config": config_payload}


if __name__ == "__main__":
    print(main()["binding"]["binding_hash"])
```

## 42.7 Persistence, resource limits, and replay

`verified_candidate_ids` retains every verified member with no implicit nominee. `viable_candidate_ids` also includes validating and terminally inconclusive candidates; their individual states must be inspected. A verified member remains usable when another candidate exhausts its budget, unless the scope itself is invalid. Descriptive differences among members do not establish a statistically supported ranking.

A fresh output directory preserves `execution_spec.json`, immutable numerical evidence and chunk files, and the atomic `tuning_checkpoint.json`. All observations, including failed measurements and zero-verified searches, are retained. Complete results use `candidate_set_result.json` and are never overwritten. Resource failures pause with their reason; unexpected exceptions preserve state and propagate. Optional diagnostic errors cannot erase completed numerical evidence. Malformed provider data is recorded

as an execution error before it enters the observation history; retry retains the attempted cost. Valid shared-failure observations remain recorded. If the same live binding later records shared invalidity, even a member verified earlier becomes unavailable for replay.

`resume_hmc_candidate_set_tuning` reconstructs the same geometry, policy, evidence, candidates and work queue using the original target. It checks source and target drift, tensor checksums, numerical versions, and scope identity. Position-field mechanics separately preserve preparation and observations for `resume_position_field_candidate_tuning`, which also requires the original runner binding. Their `controller_checkpoint.json` also references immutable completed chunks, permitting restart within a longer evidence stage. Controller-only reload cannot create missing numerical evidence.

Before frozen execution exists, `preparation_progress.json` records preparation phases, elapsed time and failures. Ordinary preparation checks time at available progress boundaries; proposal-field affine preparation is one compiled graph checked before and after its call. Failed preparation must be retried in a fresh directory. Numerical resume requires a frozen scope with the same source and controller policy; historical results remain readable when those identities change, but cannot be silently resumed under new rules.

Call budgets count attempted work. Candidate reservations protect mandatory stages; extensions and retries can use remaining free budget. A separate transition-times-$(L+1)$ estimate accounts conservatively for gradient work. The scheduler quotes unfinished work, and each chunk is charged and checkpointed before its native call. Resume does not recharge saved completed chunks; an interrupted native call remains conservatively charged even when it returned no usable evidence. Elapsed time includes recorded preparation. Numerical chunks are bounded by `chunk_max_results`, whose convenience default is 256. Time limits are checked between chunks; Python cannot preempt a native operation or compilation. A hard process limit must be applied externally. When preparation and search time limits are both supplied, the smaller limit applies. The first-call timing includes any tracing/compilation rather than pretending that it measures steady-state kernel cost.

Select a member by explicit ID and supply its result and numerical binding to `build_retained_bound_hmc_archive_runner_from_candidate_set_result`. The two frozen-kernel variants share these checks, with the claim-eligible variant additionally requiring actual GPU/XLA evidence. A child's own passing verification is required; its failed ancestor need not pass. Export with `runner.export(path)` and reconstruct with `load_hmc_candidate_retained_runner(path,adapter=original_target)`. Fresh retained blocks continue from the verified or preceding endpoint with frozen settings and fresh seeds. Freshness checks include every tuning stage and chunk, attempted partial chunks and predecessor retained blocks. Attempted seeds are reconstructed from the durable work/accounting records after member export and reload, including failed native calls that returned no trace. Tuning draws remain excluded. `runner.run_sequential` connects the explicitly chosen member to the shared cumulative posterior controller, preserving the member's transition, chain topology and target-status requirements. The generic entry point is `run_hmc_posterior(member=runner,config=config)`. Diagnostics use model coordinates and any declared scientific functionals. Posterior rejection does not change tuning membership. Chapter 47 distinguishes adaptation, discarded equilibration and retained precision.

The status `complete` means the declared search reached terminal dispositions; its verified set may be empty. It does not prove the absence of a useful kernel. The recorded limiting reason distinguishes the candidate cap, the family repair count, the epsilon do-

main and exhaustion of representable unvisited proposals. At a refinement cap, the remaining count includes only eligible pairs not yet admitted in that round. Reporting collections resolve explicit scope/search/member identities and require every included search to complete, independently of their ordering.

Public option conflicts are rejected before numerical preparation. An issued binding rejects redundant search, execution, provenance or transport overrides. Automatic routes may accept the shared search and execution configs. For the proposal-field route, legacy `step_adaptation_results` supplies the fixed-pair pilot count, while `verification_results` supplies measurement and verification counts. The shared execution configuration can replace these allocations and supply an explicit `pilot_num_results`. Pilots never qualify a pair. The legacy adaptive selector remains historical. Config payloads label their legacy candidate-policy fields as historical helper metadata; the active result's shared configs determine candidate stages. Proposal-field execution defaults to XLA; a diagnostic exception supplies `non_xla_reason`.

Exact-score TFP bindings optionally share a compiled runner across leapfrog counts at each chunk size with `execution_config.reuse_leapfrog_graphs=True`; the default is false. Each cache belongs to one frozen target and geometry with fixed chain topology, trace policy and backend. Independent scalar chains keep their original seed folding. The complete execution artifact records this option and restart preserves it. Within the same source snapshot it does not change numerical seeds; a source change still changes scope identity and can change seeds. The proposal-field route rejects the option. Reuse can reduce first-call compilation while increasing warmed execution cost, so assess its cost on complete fits. Candidate qualification and posterior diagnostics are unchanged.

Retained-member exports share one checksummed evidence bundle beside their compact member files. Copy the bundle when moving the members, or use `runner.export(path, portable=True)` for a standalone file. Numerical analyses are cached by current evidence content; mutations and changed source identities still invalidate reuse. Predecessor history is checked iteratively. These mechanics tests do not qualify target-scale memory, restart latency, posterior convergence, or scientific validity. Consumer evidence must also establish complete target/data/prior identity and any intermediate integrator telemetry required by that target.

Listing 42.5: Ordinary candidate-set invocation.

```
"""Small CPU/reference candidate-set example; no posterior or ranking claim."""
from __future__ import annotations
import os
import sys
from pathlib import Path
os.environ.setdefault("CUDA_VISIBLE_DEVICES", "-1")
os.environ.setdefault("TF_FORCE_GPU_ALLOW_GROWTH", "true")
sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
import tensorflow as tf
from bayesfilter.inference import (
    HMCControllerConfig, HMCCandidateExecutionConfig, HMCAcceptancePolicy,
    HMCKernelTuningConfig, FixedTransportHMCKernelTuningConfig,
    ValueScoreCapability, tune_hmc_kernel, tune_fixed_transport_hmc_kernel,
)

class AnalyticGaussianAdapter:
    parameter_dim = 2
    def adapter_signature(self):
        return "docs-candidate-set-gaussian-v1"
```

```
    def value_score_capability(self):
        return ValueScoreCapability(value_score_authority="graph_native",
    xla_hmc_ready=False,
            runtime_backend="tensorflow", target_scope="docs_candidate_set",
            evidence_path=__file__, nonclaims=("small CPU reference fixture",))
    def log_prob_and_grad(self, position):
        position = tf.convert_to_tensor(position, tf.float64)
        return -.5 * tf.reduce_sum(position**2, axis=-1), -position

adapter = AnalyticGaussianAdapter()
starts = tf.constant([[-1.,-.5],[-.3,.2],[.4,-.2],[1.,.5]], tf.float64)
# This reduced grid bounds the documentation smoke; the ordinary convenience
# policy uses L=(3,5,9,13,18,25) and optional pilots/refinement.
search = HMCControllerConfig(primary_l_grid=(3,5),
    epsilon_by_l=((3,(1.1,1.3)),(5,(1.1,1.3))), evidence_rungs=(1,))
execution = HMCCandidateExecutionConfig(measurement_num_results=64,
    verification_num_results=64, num_warmup_steps=8, seed=(20260914,71),
    acceptance_policy=HMCAcceptancePolicy(), target_status_trace_policy="none",
    use_xla=False, non_xla_reason="small documentation CPU reference")

run = tune_hmc_kernel(adapter=adapter, initial_position=starts,
    config=HMCKernelTuningConfig.smoke(mass_policy="fixed_identity",
        target_scope="docs_candidate_set"),
    search_config=search, execution_config=execution,
    target_lineage={"model":"standard Gaussian", "data":"none", "prior":"standard
    normal"},
    source_paths=[__file__])
assert run.result.payload()["nominee_id"] is None
assert len(run.result.candidates) >= 4
print(run.result.completion_status, run.result.verified_candidate_ids)
```

Listing 42.6: Numerical candidate evidence and explicit retained-member replay.

```
"""Numerical candidate-set/reload smoke; engineering evidence, not a posterior.

GPU/XLA is the default. --cpu-reference is a deliberate CPU/non-XLA exception.
The small Gaussian grid, widened acceptance band, and draw counts are fixture
choices that bound test cost; consumers must specify their own tuning policy.
"""
from __future__ import annotations

import argparse
import json
import os
from pathlib import Path
import subprocess
import sys
import time

REPO_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(REPO_ROOT))


class GaussianTarget:
    parameter_dim = 2

    def adapter_signature(self):
        return "docs-candidate-retained-gaussian-v1"
```

```
    def value_score_capability(self):
        from bayesfilter.inference import ValueScoreCapability
        return ValueScoreCapability(value_score_authority="graph_native",
            xla_hmc_ready=True, full_chain_xla_diagnostic_ready=True,
            target_scope="docs_candidate_retained", runtime_backend="tensorflow",
            evidence_path=__file__, nonclaims=("known Gaussian mechanics fixture",))

    def log_prob_and_grad(self, position):
        import tensorflow as tf
        position = tf.convert_to_tensor(position, tf.float64)
        return -0.5 * tf.reduce_sum(position * position, axis=-1), -position

    def target_status_telemetry(self, position):
        import tensorflow as tf
        shape = tf.shape(position)[:-1]
        return {"status_code": tf.zeros(shape, tf.int32),
            "valid_pre_regularized_score": tf.ones(shape, tf.bool),
            "floor_count_value": tf.zeros(shape, tf.int32)}


def run_example(output_dir: Path, *, cpu_reference: bool = False):
    os.environ["TF_FORCE_GPU_ALLOW_GROWTH"] = "true"
    if cpu_reference:
        os.environ["CUDA_VISIBLE_DEVICES"] = "-1"
    import tensorflow as tf
    from bayesfilter.runtime.gpu_memory_policy import
    configure_tensorflow_gpu_memory_growth
    memory = configure_tensorflow_gpu_memory_growth(tf, require_gpu=not cpu_reference)
    from bayesfilter.inference import (
        HMCAcceptancePolicy, HMCCandidateExecutionConfig, HMCControllerConfig,
        PrecomputedMassArtifact, bind_hmc_candidate_set_execution, tune_hmc_kernel,
        build_retained_bound_hmc_archive_runner_from_candidate_set_result,

    build_claim_bearing_retained_frozen_kernel_hmc_adapter_from_candidate_set_result,
        load_hmc_candidate_retained_runner,
    )
    from bayesfilter.inference.hmc import FullChainHMCConfig,
    build_independent_chain_tfp_hmc_runner

    target = GaussianTarget()
    mass = PrecomputedMassArtifact(position=[0.,0.], covariance=tf.eye(2, dtype=tf.
    float64),
        factor=tf.eye(2, dtype=tf.float64), adapter_signature=target.adapter_signature
    (),
        position_role="reference_center", covariance_source="known Gaussian fixture")
    binding = bind_hmc_candidate_set_execution(adapter=target,
        initial_position=tf.constant([[-1.,-.5],[-.3,.2],[.4,-.2],[1.,.5]], tf.float64
    ),
        mass_artifact=mass, target_scope="docs_candidate_retained",
        target_lineage={"model": "standard_normal_2d", "data": "none", "prior": "
    target_itself"},
        source_paths=[__file__], scope_id="docs-bridge", search_id="example-1",
        epsilon_domain=(.01,1.95), repair_factor=1.1, max_repairs_per_family=0,
        config=HMCCandidateExecutionConfig(measurement_num_results=128,
            verification_num_results=128, num_warmup_steps=8, seed=(20260914,11),
            acceptance_policy=HMCAcceptancePolicy(practical_region=(.55,.85),
    repair_region=(.5,.9)),
```

```
        target_status_trace_policy="per_chain_step", use_xla=not cpu_reference,
        non_xla_reason="CPU reference smoke" if cpu_reference else None))
run = tune_hmc_kernel(adapter=target, initial_position=binding.
initial_active_state,
    candidate_set_adapter=binding.typed_adapter, output_dir=output_dir / "tuning",
    config=HMCControllerConfig(primary_l_grid=(2,3),
        epsilon_by_l=((2,(1.1,1.3,1.5)),(3,(1.1,1.3,1.5))),
        total_budget_units=40, repair_reserve_units=3))
if not run.result.verified_candidate_ids:
    raise RuntimeError("no verified member in the bounded engineering fixture")
candidate_id = run.result.verified_candidate_ids[0]  # Representative, no ranking
claim.
builder = (build_retained_bound_hmc_archive_runner_from_candidate_set_result if
cpu_reference
           else
build_claim_bearing_retained_frozen_kernel_hmc_adapter_from_candidate_set_result)
runner = builder(candidate_set_result=run.result, candidate_id=candidate_id,
retained_binding=binding)
member_path = runner.export(output_dir / "member.json")
first = runner.run(num_results=12, seed=(71,1), output_dir=output_dir / "
retained_1")
restored = load_hmc_candidate_retained_runner(member_path, adapter=GaussianTarget
(),
                                              claim_eligible=not cpu_reference)
second = restored.run(num_results=12, seed=(71,2), output_dir=output_dir / "
retained_2",
                      previous_archive=first["archive_path"])
# Match both compilation mode and device: stateless seeds do not promise
# identical random draws between XLA and non-XLA implementations.
with tf.device("/CPU:0" if cpu_reference else "/GPU:0"):
    reference = build_independent_chain_tfp_hmc_runner(binding._active_adapter,
        binding.initial_active_state, FullChainHMCConfig(num_results=12,
            num_burnin_steps=0, step_size=runner.candidate.epsilon,
            num_leapfrog_steps=runner.candidate.leapfrog_steps, seed=(71,2),
            use_xla=not cpu_reference, target_scope="docs_candidate_retained"))
    reference_result = reference.run(current_state=first["final_active_state"],
mode="serial")
max_error = float(tf.reduce_max(tf.abs(second["samples"] - reference_result.
samples)))
if max_error > 1e-10:  # Existing float64 reference-comparison tolerance.
    raise AssertionError(f"retained transition parity failed: {max_error}")
from bayesfilter.inference.hmc_candidate_set_execution import _trace_from_payload,
 _tensor_from_payload
archive = json.loads(Path(second["archive_path"]).read_text())
trace = _trace_from_payload(archive["trace"])
samples = _tensor_from_payload(archive["active_samples"])
initial = _tensor_from_payload(archive["initial_active_state"])
# Independent deterministic leapfrog reference, using the actual captured
# momentum. This checks CPU versus GPU arithmetic without comparing RNGs.
with tf.device("/CPU:0"):
    q0 = tf.concat([initial[None], samples[:-1]], axis=0)
    p0 = tf.identity(trace["initial_momentum"])
    q, p = q0, p0
    eps = tf.constant(runner.candidate.epsilon, tf.float64)
    for _ in range(runner.candidate.leapfrog_steps):
        p = p - .5 * eps * q
        q = q + eps * p
        p = p - .5 * eps * q
```

```
        expected_log_accept = .5 * tf.reduce_sum(q0*q0 + p0*p0 - q*q - p*p, axis=-1)
        position_error = float(tf.reduce_max(tf.abs(q - trace["proposed_state"])))
        momentum_error = float(tf.reduce_max(tf.abs(p - trace["final_momentum"])))
        acceptance_error = float(tf.reduce_max(tf.abs(expected_log_accept - trace["
    log_accept_ratio"])))
    if max(position_error, momentum_error, acceptance_error) > 1e-10:
        raise AssertionError("deterministic CPU leapfrog/energy reference failed")
    compiled_counts = [r._runner.experimental_get_tracing_count()
        for ensemble in binding._runners.values() for r in ensemble._runners]
    if not all(count == 1 for count in compiled_counts):
        raise AssertionError("unbounded/repeated runner tracing")
    return {"status": "passed", "verified_candidate_ids": run.result.
    verified_candidate_ids,
        "candidate_count": len(run.result.candidates), "selected_member": candidate_id
    ,
        "member_hash": runner.member_hash, "member_path": str(member_path),
        "retained_archives": [first["archive_path"], second["archive_path"]],
        "maximum_same_backend_transition_error": max_error,
        "cpu_reference_errors": {"position": position_error, "momentum":
    momentum_error, "log_accept": acceptance_error},
        "compiled_runner_trace_counts": compiled_counts,
        "memory_policy": memory, "runtime_policy": binding._runtime,
        "source_closure": binding._spec["source_closure"],
        "execution_config": binding.config.payload(),
        "gpu_allocator": tf.config.experimental.get_memory_info("GPU:0") if not
    cpu_reference else None,
        "inference_status": "engineering parity only; no statistical ranking or
    posterior admission"}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output-dir", type=Path, required=True)
    parser.add_argument("--cpu-reference", action="store_true")
    args = parser.parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=False)
    started = time.monotonic()
    manifest = {"command": sys.argv, "git_commit": subprocess.check_output(
        ["git","rev-parse","HEAD"], cwd=REPO_ROOT, text=True).strip(),
        "python": sys.executable, "cpu_reference": args.cpu_reference,
        "gpu_intentionally_hidden": args.cpu_reference, "data_version": "
    known_Gaussian_fixture_v1",
        "plan": "docs/plans/bayesfilter-typed-candidate-retained-bridge-plan
    -2026-09-14.md"}
    try:
        manifest["result"] = run_example(args.output_dir, cpu_reference=args.
    cpu_reference)
    except Exception as exc:
        manifest["result"] = {"status": "failed", "error": type(exc).__name__ + ": " +
     str(exc)}
        raise
    finally:
        manifest["wall_seconds"] = time.monotonic() - started
        (args.output_dir / "validation_manifest.json").write_text(json.dumps(manifest,
     indent=2) + "\n")
    print(json.dumps({"status": manifest["result"]["status"], "wall_seconds": manifest
    ["wall_seconds"]}))
```

```
if __name__ == "__main__":
    main()
```

## 42.8 What the validation experiments establish

The tuning procedure is tested through several distinct experiments. Numerical oracles compare target values, scores, coordinate Jacobians and leapfrog updates with independent calculations. Frozen-kernel experiments test invariance using independent reference draws and a reversible random-position construction. The latter uses Algorithm 2 and Proposition 2.3 of Gandy and Scott [2021]; it applies to frozen reversible kernels. An experiment may replace its kernel $K$ by $K^s$, composing $s$ complete Metropolis transitions with independent randomness at each substep. Repeated composition of this same reversible kernel preserves reversibility; section 2.2 of that paper proposes this extension to improve sensitivity to subtle errors. Sensitivity still needs measurement for the chosen alternative. The option `kernel_power` applies only to the frozen invariance experiment. It does not make posterior draws independent. Finite states and log acceptance ratios are checked at every substep, including earlier substeps when the final one is finite.

A separate optional wrapper implements Algorithm 3 of Gandy and Scott [2021]. Each look uses an independently seeded complete experiment, with the sample count increased once after the first look. For overall level $\alpha$, at most $k$ looks and a fixed family of $d$ component tests, initialize $\beta = \alpha/k$ and $\gamma = \beta^{1/k}$. At each look, set $q = d \min_j p_j$. Reject when $q \leq \beta$, stop without rejection when $q > \gamma + \beta$, and otherwise continue with $\beta \leftarrow \beta/\gamma$. Theorem 3.1 bounds the probability of rejection under the null by $\alpha$ when the look vectors are independent and each component p-value is superuniform. Fresh experiments are essential: repeatedly testing cumulative posterior draws would not satisfy that independence condition. Numerical failures invalidate the affected experiment; they do not count as defect detection. This wrapper is a validation experiment and does not alter the tuning acceptance screen, warmup checks or posterior precision stopping.

The Gaussian mechanics experiment also reconstructs the Metropolis log ratio from an independent analytic density and the actual TFP endpoint momenta. It checks that the returned state agrees with the recorded accept/reject decision. This arithmetic check can expose a reversed energy correction even when a short distributional experiment has little power to detect its effect. For an identity-metric Gaussian, an additional fixed-look test uses the sum of squared standardized endpoint coordinates. Starting from independent exact Gaussian draws makes the endpoints independent under the correct frozen kernel, so this sum has a chi-squared distribution with degrees of freedom equal to the number of draws times the dimension. The experiment adjusts for all declared radius, rank and energy tests. Adjacent states from one chain cannot replace independent endpoints in this calculation. Null controls and complete repeated experiments measure size and power for the declared defect; one detected error does not establish broad sensitivity. Controller tests check that every qualified pair is retained, each repair keeps its own L, and fresh verification and restart preserve the numerical record. Full-procedure experiments then call the public tuner and run the separate posterior controller. Small integration cases reconstruct every verified member. Larger replicated experiments may predeclare one member by tuning identity before sampling; all other verified members remain recorded as unassessed for that experiment.

Simulation-based calibration generates a parameter from a proper prior and a dataset conditional on that parameter. If the fitted outputs are independent exact posterior draws, the generating parameter's randomized rank among them is uniform. This supplies the null distribution for the test. The implemented design uses independent complete fits within each dataset and one fixed output per fit. It includes data-dependent likelihood quantities because parameter ranks alone can miss a procedure that ignores the observations. Failed fits remain in the denominator; conditional ranks from successful fits cannot establish calibration for a procedure that sometimes fails. Candidate siblings share a dataset and are not independent calibration replications. The rank argument follows Talts et al. [2020]; the likelihood quantities follow the sensitivity analysis of Modrák et al. [2023].

The later M15 experiment obtained all 246 planned fitted outputs over 82 fresh datasets, using three independently fitted outputs per dataset. Its rank tests did not reject, but three rank draws give a coarse test. Independent analytic controls in M16 showed that the 32-dataset normal design detected a shift of one quarter of a posterior standard deviation in only 27 of 256 experiments, and a half-standard-deviation shift in 64 of 256. The corresponding nonrejection therefore supplies little evidence against those errors. These controls measure the rank statistic's sensitivity; they do not measure defect power of the complete HMC procedure. The full denominators, uncertainty and source versions are recorded in the M15 and M16 result notes under `docs/plans/`.

An analytic posterior permits an additional direct check of fitted outputs. For a zero-mean normal prior with variance $\tau^2$ and independent observations with known variance $\sigma^2$, the conditional posterior has variance $s_y^2 = (\tau^{-2}+n\sigma^{-2})^{-1}$ and mean $\mu_y = s_y^2 \sum_i y_i/\sigma^2$. If a predetermined output $X_j$ from each independent complete fit has that conditional posterior law, then $Z_j = (X_j - \mu_y)/s_y$ are independent standard normals. Consequently, $\sum_j Z_j/\sqrt{J}$ is standard normal and $\sum_j Z_j^2$ has a $\chi^2_J$ distribution. Two two-sided tests, each at half the declared size, check location and second moment. The union bound controls their combined size under this correct-output null. The second check can expose errors that leave the mean unchanged.

The diagnostic fixes the member, chain and terminal-draw rule before seeing posterior outcomes. Missing, capped or unqualified fits make its planned experiment incomplete; they are never counted as successful defect detections. Shared-stream controls cannot count as independent fits. The normal laws above describe the hypothesis under test. Adaptive stopping does not prove that a finite HMC endpoint follows them, and these tests do not establish coverage of intervals for stopped posterior means. Arithmetic adversaries and activated whole-fit defects answer different questions and are reported separately.

An invariant kernel can remain motionless or revisit only a few states. Identity and two-cycle controls therefore accompany the distributional tests. Similarly, an MCSE estimator can perform adequately on fixed-length stationary arrays while intervals based on it have poor coverage at a data-dependent stopping time. Separate experiments compare the actual controller's stopped means and medians with available exact references, retaining cap and unavailable outcomes. A fixed-length arm reuses the same verified kernel with independent randomness and predeclared discarded and retained counts. Mixture mode probabilities check global exploration that a local mean or small reported MCSE may miss. Missing arms remain in the comparison denominator; pointwise uncertainty intervals from a small number of replications do not establish a ranking or nominal sequential coverage. No R-hat, ESS or MCSE assessment changes tuning membership.

Coverage is reported both over all planned fits and conditional on an available interval. Repeating a fit to one fixed dataset measures conditional behavior; it is distinct from prior-predictive simulation-based calibration. A longer fixed-count arm can also have inaccurate uncertainty estimates, so disagreement with nominal coverage does not by itself identify optional stopping as the cause.

The September 2026 campaign observed this distinction directly. A Gaussian mixture run passed its local posterior checks after 6,500 discarded and 2,000 retained transitions per chain, but all retained draws occupied the right mode. Its estimated mean was 4.956, against the exact mean 2, with reported MCSE 0.038. Thus a small within-mode uncertainty estimate did not measure error relative to the full posterior. Target-specific posterior assessment should include relevant mode or other global quantities and examine sensitivity to starting locations. The saved result and its limited replication evidence are documented in `docs/plans/bayesfilter-inference-validation-24h-result-2026-09-16.md`. Validation designs may now declare the mixture probability $P(x < 0)$ as a monitored posterior mean. It receives the same diagnostic and precision checks as the other declared quantities; a constant event indicator supplies no usable precision estimate. A separate experiment starts chains in both known modes. These checks address the saved failure mechanism without establishing discovery of unknown modes. Synthetic acceptance calibration separately measures the candidate screen near its practical and repair boundaries, including repeated evidence allocations. It does not establish posterior validity. The separate numerical acceptance experiment calls the public tuner on a declared identity-metric Gaussian pair. Every repeated search measures that exact pair from the fixed tuning starts, with no repair children. Each measurement survivor receives its own fresh verification. Independent exact Gaussian anchors supply a stationary acceptance reference using actual Metropolis transitions and a separate endpoint-energy check. That reference neither supplies tuning starts nor determines qualification. Stationary acceptance and acceptance over a finite window from fixed starts are different quantities; their difference is part of the experiment. Observed qualification rates therefore describe the operational compatibility screen, not nominal sequential coverage or automatic-preparation reliability. The M16 experiment completed 384 CPU-reference and 192 GPU/XLA searches around stationary acceptance values from .64 to .76. Pairs on both sides of the nominal [.65,.75] band could qualify. This is consistent with the implemented uncertainty-aware compatibility rule, whose finite-start evidence measures a different quantity from the independent stationary reference. Separately, 128 complete sequential validation experiments per device detected the declared reversed-Metropolis-ratio defect every time. Three of the four baseline/no-op null intervals were still too wide for the declared size-precision screen. That result supports detection of this defect while leaving null-size precision and sensitivity to other defects unresolved. Stopped-interval reporting compares the monitored mode probability with its independent mixture CDF. That quantity remains in the planned denominator when warmup ends without retained draws.

Automatic initialization and supplied epsilon proposals are distinct subjects of validation. A supplied proposal can exceed the final prepared metric's bound; native initialization uses preparation's own starting epsilon. Either search can return no verified member under its finite domain and budget. A complete calibration experiment must preserve those missing fits, even when all observed ranks among successful fits look uniform. Parallel dataset groups are combined only when declared together with identical settings and frozen source; pilot and repaired experiments remain separate.

A complementary diagnostic uses all retained draws from one qualified fit to test its posterior mean. With $\theta \sim N(0, \tau^2)$ and conditionally independent observations $y_i \mid \theta \sim N(\theta, \sigma^2)$, completing the square in the prior times likelihood gives the independent reference

$$s_*^2 = (\tau^{-2} + n\sigma^{-2})^{-1}, \qquad m_* = s_*^2 \sigma^{-2} \sum_{i=1}^{n} y_i.$$

The validation engine `reference_mean` compares $|\widehat{m} - m_*|/\widehat{\mathrm{MCSE}}$ with the declared normal critical value. It recomputes lugsail MCSE independently from the archived model draws; the reference never enters tuning. The normal approximation defines the alarm, but its achieved false-alarm and detection rates require independent complete fits with fresh data. A capped, missing or unqualified fit is a possible alarm in the conservative null rate and a nondetection in the defect rate. This answers a location-error question distinct from endpoint invariance and SBC; arithmetic and execution tests alone do not establish its power.

Matched consumer comparisons use the same public ordinary tuner and posterior controller. The explicit noncentered eight-schools and Bayesian regression adapters reproduce the pinned posteriordb Stan laws, observations and coordinate Jacobians. A member is selected by tuning identity before posterior sampling; every other verified member remains recorded. The ten-chain Stan reference is used only for the subsequent comparison, with lugsail uncertainty from both finite samples. Agreement within a declared practical margin is a target-specific accuracy screen. It neither makes the reference exact nor establishes calibration for another dataset or consumer. Exact inputs, commands and results are recorded in `docs/plans/bayesfilter-hmc-repair-m8-result-2026-09-18.md`.

The later M17 matrix completed 21 model/route fits and four matched-reference fits. Eleven matrix members and three reference members passed their full declared posterior checks. Centered funnels produced no verified member; mixtures reached the warmup cap; rotated Gaussian, noncentered funnel and CPU eight-schools fits reached retained precision caps. Their verified kernels remain in the tuning set. All four reference fits met the declared mean agreement margins, illustrating why reference mean agreement and requested precision must be reported separately. Six conditional position-field cases also exercised fresh verification and durable restart. Fixed nonlinear maps in this matrix were not trained. Exact model/device outcomes and source versions are recorded in `docs/plans/bayesfilter-hmc-repair-m17-result-2026-09-21.md`. The original MacroFinance target and matched posterior reference remain a separate missing comparison.

Cost pilots can save a host profile of automatic preparation, candidate search, export/reload and posterior assessment. The profile includes failed attempts; a framework call may include both compilation and execution time. Such a pilot identifies costs to investigate before funding more replications. It does not establish statistical calibration or rank the viable kernels by efficiency. Use the validation CLI's `run` command with `--profile-execution` for paired comparisons: it preserves the numerical design and random streams. With isolated fits, the profiler runs in the numerical child after framework imports, with setup time recorded separately. It records Python call boundaries without native builtins tracing, avoiding an observed Python 3.13/TensorFlow interaction that loses outer calls. This is host cost attribution, not device kernel timing. The run index records profile availability. A completed fit is never repeated to replace a missing profile, and profiling failures do not replace numerical outcomes.

Repeated validation fits can use a fresh process for each complete fit, with an explicit per-fit timeout inside the total experiment budget. This bounds the lifetime of process-local framework caches. Each child preserves its full candidate set, posterior chunks and independent assessment before normal exit; the coordinator records process failures separately from saved numerical results. Missing or abnormally terminated fits remain in the replication denominator. Numerical search, accuracy, stopping and reference-mean designs enable this optional mode with `options.isolate_fits` and `options.fit_process_timeout_seconds`. The setting does not change the HMC transition, tuning criteria or posterior thresholds.

A bounded sibling study can instead predeclare several members using `member_rule="shortest_verified_l"`, `posterior_members="selected"` and an explicit `posterior_member_count`. The rule orders distinct verified leapfrog counts increasingly and takes the first candidate identifier within each. It records the list before posterior sampling and preserves any shortage. Each ordinal member is assessed separately across complete fits; siblings do not increase the number of independent replications. A failed member does not suppress the next member or remove a tuning candidate. This is a bounded cost hypothesis, not an established mixing ranking, and posterior outcomes cannot choose a winner from the same assessment draws.

The executable suite definitions are documented in `docs/validation/README.md`. The following table is generated from their saved reports. C, H and P denote current-source executions, historical-source executions and planned designs, including retries. These are counts of designs, not models declared correct or independent replications. CPU reference work and GPU/XLA work remain separate. A completed controller call with no retained draws does not establish posterior coverage; missing stopping intervals remain in the denominator. Affine and dense nonlinear transports occupy separate rows. Each experiment declares its evidence scope. Incomplete confirmation cannot establish calibration; statistical uncertainty and unresolved coverage are reported with the underlying observations.

The table covers suite reports. Separately executed phase diagnostics, including the matched posteriordb comparisons above, are recorded in their phase results. An unavailable external eight-schools row refers to that original suite's missing reference bundle; it does not negate the later explicitly bound eight-schools comparison. Neither supplies the absent MacroFinance reference.

Automatically inferred suite requirements identify each planned design, so a completed step-size or kernel-power setting cannot cover an unrun setting in the same category. Explicit category requirements may request any matching assessed design. Older saved plans without design identifiers retain that category-level meaning; their individual rows determine whole-suite completion.

| Target | Route/device | Question | C / H / P | Findings |
|---|---|---|---|---|
| Banana | Dense IAF / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Banana | Dense IAF / GPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Banana | Affine transport / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Banana | Affine transport / GPU | Accuracy | 0 / 1 / 1 | pipeline assessed |

| Target | Route/device | Question | C / H / P | Findings |
|---|---|---|---|---|
| Banana | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Banana | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Banana | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Banana | Automatic / GPU | Accuracy | 0 / 2 / 2 | pipeline assessed |
| Beta | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Beta | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Beta | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Beta | Prepared / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Beta-binomial | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Beta-binomial | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Beta-binomial | Automatic / CPU | SBC | 0 / 65 / 66 | calibration incomplete, no discrepancy detected, timed out |
| Beta-binomial | Automatic / CPU | Stopping | 0 / 49 / 49 | pipeline assessed |
| Beta-binomial | Automatic / GPU | Accuracy | 0 / 2 / 3 | pipeline assessed, timed out |
| Beta-binomial | Automatic / GPU | SBC | 0 / 7 / 8 | no discrepancy detected, not run |
| Beta-binomial | Automatic / GPU | Stopping | 0 / 34 / 34 | pipeline assessed |
| Beta-binomial | Reference / CPU | SBC | 0 / 2 / 2 | no discrepancy detected |
| Cauchy | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Cauchy | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Cauchy | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Cauchy | Automatic / CPU | Accuracy | 0 / 2 / 2 | pipeline assessed |
| Cauchy | Automatic / GPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Dirichlet | Dense IAF / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Dirichlet | Dense IAF / GPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Dirichlet | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Dirichlet | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |

| Target | Route/device | Question | C / H / P | Findings |
|---|---|---|---|---|
| Dirichlet | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Dirichlet | Prepared / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Dirichlet | Prepared / GPU | Accuracy | 0 / 1 / 3 | not run, pipeline assessed, timed out |
| Eight schools | External / CPU | Accuracy | 0 / 0 / 2 | unavailable |
| Funnel | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Funnel | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Funnel | Frozen kernel / GPU | Mechanics | 0 / 2 / 3 | mechanics passed, not run |
| Funnel | Automatic / CPU | Accuracy | 0 / 4 / 5 | failed, no verified members |
| Funnel | Automatic / GPU | Accuracy | 0 / 6 / 8 | failed, no verified members, not run |
| Funnel noncentered | Automatic / CPU | Accuracy | 0 / 5 / 5 | pipeline assessed, posterior incomplete |
| Funnel noncentered | Automatic / GPU | Accuracy | 0 / 6 / 7 | pipeline assessed, posterior incomplete, timed out |
| Gamma | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Gamma | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Gamma | Frozen kernel / GPU | Mechanics | 0 / 2 / 3 | mechanics passed, not run |
| Gamma | Prepared / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Gaussian | Controller / CPU | Acceptance | 0 / 22 / 22 | acceptance rates estimated |
| Gaussian | Controller / CPU | Search | 0 / 2 / 2 | inventory passed |
| Gaussian | Dense IAF / GPU | Accuracy | 0 / 3 / 3 | no verified members, pipeline assessed |
| Gaussian | Affine transport / CPU | Accuracy | 0 / 2 / 2 | pipeline assessed |
| Gaussian | Affine transport / GPU | Accuracy | 0 / 2 / 2 | pipeline assessed |
| Gaussian | Frozen kernel / CPU | Acceptance | 0 / 10 / 10 | actual hmc acceptance measured |
| Gaussian | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Gaussian | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Gaussian | Frozen kernel / CPU | Power | 0 / 9 / 9 | power estimated |
| Gaussian | Frozen kernel / GPU | Invariance | 0 / 1 / 1 | no discrepancy detected |
| Gaussian | Frozen kernel / GPU | Mechanics | 0 / 5 / 6 | mechanics passed, not run |

| Target | Route/device | Question | C / H / P | Findings |
|---|---|---|---|---|
| Gaussian | Frozen kernel / GPU | Power | 0 / 18 / 21 | not run, power estimated |
| Gaussian | Automatic / CPU | Accuracy | 0 / 1 / 2 | pipeline assessed, timed out |
| Gaussian | Automatic / GPU | Accuracy | 0 / 2 / 4 | pipeline assessed, timed out |
| Gaussian | Prepared / CPU | Acceptance | 0 / 6 / 6 | acceptance rates estimated |
| Gaussian | Prepared / CPU | Accuracy | 0 / 1 / 2 | pipeline assessed, timed out |
| Gaussian | Prepared / CPU | Stopping | 0 / 4 / 5 | pipeline assessed, timed out |
| Gaussian | Prepared / GPU | Acceptance | 0 / 22 / 22 | acceptance rates estimated |
| Gaussian | Prepared / GPU | Stopping | 0 / 3 / 5 | not run, pipeline assessed |
| Gaussian | Reference / CPU | Stopping | 0 / 6 / 6 | diagnostics assessed |
| LGSSM location | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| LGSSM location | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| LGSSM location | Automatic / CPU | SBC | 0 / 18 / 18 | no discrepancy detected |
| LGSSM location | Automatic / CPU | Stopping | 0 / 16 / 16 | pipeline assessed |
| LGSSM location | Automatic / GPU | Accuracy | 0 / 2 / 3 | pipeline assessed, timed out |
| LGSSM location | Automatic / GPU | SBC | 0 / 1 / 2 | calibration incomplete, not run |
| LGSSM location | Automatic / GPU | Stopping | 0 / 1 / 1 | pipeline assessed |
| LGSSM location | Reference / CPU | SBC | 0 / 2 / 2 | no discrepancy detected |
| Macrofinance | External / CPU | Accuracy | 0 / 0 / 2 | unavailable |
| Mixture | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Mixture | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Mixture | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Mixture | Prepared / CPU | Accuracy | 0 / 2 / 2 | posterior incomplete |
| Mixture | Prepared / CPU | Stopping | 0 / 3 / 4 | incomplete, pipeline discrepancy, timed out |
| Mixture | Prepared / GPU | Accuracy | 0 / 2 / 2 | posterior incomplete |
| Mixture | Prepared / GPU | Stopping | 0 / 4 / 6 | not run, pipeline discrepancy, posterior incomplete |
| Mixture | Reference / CPU | Stopping | 0 / 2 / 2 | diagnostics assessed |
| Normal-normal | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |

| Target | Route/device | Question | C / H / P | Findings |
|---|---|---|---|---|
| Normal-normal | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Normal-normal | Automatic / CPU | SBC | 0 / 91 / 91 | calibration incomplete, no discrepancy detected |
| Normal-normal | Automatic / CPU | Stopping | 0 / 49 / 49 | pipeline assessed |
| Normal-normal | Automatic / GPU | SBC | 0 / 11 / 22 | calibration incomplete, cancelled, no discrepancy detected, not run, timed out |
| Normal-normal | Automatic / GPU | Stopping | 0 / 34 / 34 | pipeline assessed |
| Normal-normal | Prepared / CPU | SBC | 0 / 17 / 17 | no discrepancy detected |
| Normal-normal | Reference / CPU | Power | 0 / 17 / 17 | power estimated |
| Normal-normal | Reference / CPU | SBC | 0 / 2 / 2 | no discrepancy detected |
| Regression | External / CPU | Accuracy | 0 / 0 / 2 | unavailable |
| Rotated gaussian | Affine transport / CPU | Accuracy | 0 / 1 / 1 | posterior incomplete |
| Rotated gaussian | Affine transport / GPU | Accuracy | 0 / 1 / 2 | not run, posterior incomplete |
| Rotated gaussian | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Rotated gaussian | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Rotated gaussian | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Rotated gaussian | Automatic / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |
| Rotated gaussian | Automatic / GPU | Accuracy | 0 / 3 / 3 | pipeline assessed |
| Student t | Frozen kernel / CPU | Invariance | 0 / 2 / 2 | no discrepancy detected |
| Student t | Frozen kernel / CPU | Mechanics | 0 / 3 / 3 | mechanics passed |
| Student t | Frozen kernel / GPU | Mechanics | 0 / 1 / 2 | mechanics passed, not run |
| Student t | Automatic / CPU | Accuracy | 0 / 1 / 1 | pipeline assessed |

## 42.9 Historical readers and complete interface inventory

Initial geometry and bootstrap screening are implemented in `hmc_geometry.py` and `hmc_bootstrap.py`, respectively. Windowed preparation, its timeout policy and the frozen-mass/start-bank handoff are implemented in `hmc_mass_adaptation.py`. Public imports

and historical `hmc_kernel_tuning` aliases resolve to the same definitions. Automatic preparation calls these implementations directly; public presets, configuration translation and geometry-scaled budgets live in `hmc_configuration.py`. The old module retains historical orchestration and readers, with aliases for the extracted definitions. These helpers do not issue candidate-set tuning authority. Numerical bindings include their source files and shared preparation helpers. Seed identifiers remain stable, while live seed records name the current implementation files. Historical results retain their original source provenance.

Earlier single-kernel ordinary, fixed-transport and typed TensorFlow results retain their original identities and readers. Their compatibility replay roles and older `claim_bearing_blockers` remain historical evidence; a caller-edited authority flag or blocker list cannot grant authority. The active procedure uses the candidate-set result and the member's own numerical evidence. Discovery, refinement, `select_fixed_transport_candidate_set`, and stage helpers remain diagnostics. The generated registry below identifies their replacement public route and their limits.

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.hmc_tuning_dispatch.tune_hmc_kernel` | public tuner; tested supported; authority | Target: artifact authority requires an ordinary exact value/score target; the typed position-field branch additionally requires a repository-issued binding and exact endpoint target but remains mechanics-only. Coordinates: ordinary adapter coordinates with repository-owned affine mass coordinates. Mass: windowed adaptation by default or explicit fixed identity from config. Epsilon: shared per-L pilot, exact-pair measurement, bounded directional repair, and independently seeded evidence rungs. L: shared primary L grid (3, 5, 9, 13, 18, 25), explicit optional all-survivor refinement and expansion; retain every verified pair. Verification: fresh fixed-kernel verification; default TFP runner requires typed acceptance, health, and minimum draws; rank-normalized split/folded R-hat is retained as an explanatory diagnostic with threshold metadata (1.01), not a tuning handoff gate. ESS: disabled for ordinary tuning admission; retained posterior ESS is separate. Forbidden: arbitrary bare runner callback; engineering_probe_covariance_multiplier as a public ordinary mode; transferring epsilon qualification across different L candidates; fixed nonlinear transport without its transformed target contract; retained posterior convergence claim. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.fixed_transport_hmc_tuning_tf.tune_fixed_transport_hmc_kernel` | public tuner; tested supported; authority | Target: identity-bound frozen transport with exact transformed value including Jacobian and matching score. Coordinates: fixed transport latent z coordinates. Mass: fixed identity mass in z; no ordinary windowed mass adaptation. Epsilon: shared per-L pilot or explicit epsilon proposals, exact-pair measurement and bounded independently verified repairs. L: all declared (epsilon, L) pairs measured; all-survivor refinement and retention. Verification: fresh fixed-kernel verification for every survivor; declared evidence rungs; R-hat reporting-only. ESS: disabled for kernel tuning admission; cumulative posterior ESS is separate. Forbidden: arbitrary position-only force without a frozen transport; ordinary mass adaptation claim; reuse after transport identity changes; legacy directional diagnostic policy as an artifact-authority handoff. |
| `bayesfilter.inference.hmc_candidate_set_tuning.tune_hmc_candidate_set` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.fixed_transport_candidate_selection.select_fixed_transport_candidate_set` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_fixed_transport_hmc_kernel. |
| `bayesfilter.inference.hmc_robust_broad_grid.tune_hmc_kernel_robust_broad_grid` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.hmc_fixed_metric_grid_search.run_fixed_metric_grid_search` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_operational_broad_grid.run_operational_broad_grid` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_operational_broad_grid.run_operational_broad_grid_process_parallel` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_budget_ladder.run_fixed_mass_hmc_tuning_budget_ladder` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.hmc_tuning.run_fixed_mass_step_tuning_diagnostic` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_tuning.run_windowed_mass_adaptation_diagnostic` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_tuning.run_fixed_trajectory_tuning_diagnostic` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_tuning.run_gaussian_dual_averaging_diagnostic` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.hmc_kernel_tuning.run_hmc_start_bank_diagnostic` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.fixed_transport_hmc_candidate_discovery_tf.discover_fixed_transport_hmc_candidates` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_fixed_transport_hmc_kernel. |
| `bayesfilter.inference.fixed_transport_hmc_candidate_discovery_tf.run_fixed_transport_hmc_candidate_campaign` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_fixed_transport_hmc_kernel. |
| `bayesfilter.inference.fixed_transport_hmc_candidate_discovery_tf.refine_fixed_transport_hmc_candidates` | diagnostic helper; diagnostic only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_fixed_transport_hmc_kernel. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.generic_hmc_tuning.run_generic_hmc_tuning_orchestration` | historical helper; historical only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.generic_hmc_tuning.orchestrate_generic_hmc_tuning` | historical helper; historical only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.fixed_trajectory_hmc_tuning_v2.run_tiny_gaussian_fixed_trajectory_hmc_tuning_v2` | historical helper; historical only; no authority | Target: not authoritative; use the active replacement target contract. Coordinates: not authoritative; use the active replacement coordinates. Mass: not authoritative for canonical handoff. Epsilon: not authoritative for canonical handoff. L: not authoritative for canonical handoff. Verification: cannot issue canonical fresh-verification admission. ESS: not authoritative for canonical handoff. Forbidden: canonical tuning artifact; retained-kernel handoff; public tuning recommendation. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc_kernel_tuning.run_hmc_tune_verify_repair_loop` | stage helper; internal only; no authority | Target: prevalidated ordinary adapter, geometry, and bootstrap artifacts. Coordinates: same adapter and mass-coordinate lineage as public tuner. Mass: internal windowed adaptation and repair. Epsilon: internal fixed-mass tuning and repair. L: internal leapfrog-count selection and repair. Verification: internal fresh fixed-kernel verification. ESS: disabled for ordinary tuning admission. Forbidden: final one-call API; canonical artifact authority. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.hmc.run_full_chain_tfp_hmc` | chain runner; internal only; no authority | Target: adapter exact value and matching score. Coordinates: caller-supplied fixed mass coordinates. Mass: not owned; consumes supplied coordinates. Epsilon: fixed or config-requested dual averaging only. L: fixed caller-supplied leapfrog count. Verification: none by itself. ESS: none by itself. Forbidden: standalone full-tuning claim; canonical artifact authority. Replacement: tune_hmc_kernel. |

| Interface | Status | Target, tuning, verification, and use |
|---|---|---|
| `bayesfilter.inference.fixed_transport_hmc_mechanics_tf.run_fixed_transport_full_chain_tfp_hmc` | chain runner; internal only; no authority | Target: preconstructed exact transformed value/score adapter. Coordinates: frozen-transport latent z coordinates. Mass: not owned; consumes the supplied fixed coordinate system. Epsilon: fixed or config-requested dual averaging only. L: fixed caller-supplied leapfrog count. Verification: none by itself. ESS: none by itself. Forbidden: standalone full-tuning claim; candidate-selection claim; canonical artifact authority. Replacement: tune_fixed_transport_hmc_kernel. |
| `bayesfilter.inference.neural_force_hmc.run_full_chain_neural_force_hmc` | chain runner; diagnostic only; no authority | Target: frozen position-only force plus exact deterministic endpoint potential. Coordinates: native affine mass coordinates when bound; direct identity fallback is diagnostic only. Mass: not owned; identity in supplied mass coordinates. Epsilon: fixed or config-requested dual averaging only. L: fixed caller-supplied leapfrog count. Verification: none by itself. ESS: none by itself. Forbidden: standalone full-tuning claim; mass-tuning claim; leapfrog-count-tuning claim; canonical artifact authority. Replacement: tune_hmc_kernel. |
| `bayesfilter.inference.neural_force_hmc.bind_neural_force_hmc_tuning_runner` | runner binding factory; tested supported; no authority | Target: coordinate-consistent frozen force and exact raw-coordinate endpoint potential. Coordinates: TensorFlow tuning branch native affine mass wrapper; identity fallback for a direct low-level call is diagnostic only. Mass: owned by the typed TensorFlow mechanics branch. Epsilon: owned by the typed TensorFlow mechanics branch. L: shared broad L grid (3, 5, 9, 13, 18, 25) within the declared maximum; explicit all-survivor and unresolved-interval refinement; retain every verified pair. Verification: fresh injected-runner fixed-kernel health and acceptance verification. ESS: disabled for ordinary tuning admission. Forbidden: standalone tuning authority; bare callable injection; transformed target masquerading as ordinary raw coordinates. Replacement: tune_hmc_kernel. |

# Chapter 43

# Mass Matrices and Local Geometry

Mass matrices translate derivative information into sampler geometry. In a linear Gaussian validation problem, a local posterior Hessian or observed information matrix can be a useful preconditioner. In a nonlinear or weakly identified model, the same matrix can be misleading unless it is regularized and checked against diagnostics.

## 43.1 Sources of Curvature

BayesFilter recognizes several curvature sources:

- analytic likelihood Hessian plus prior curvature;
- observed-information or expected-information approximations;
- finite-difference Hessian diagnostics at a MAP point;
- empirical covariance from warmup draws;
- diagonal fallbacks when dense curvature is unstable.

The source must be recorded because each source has a different failure mode. MacroFinance mass-matrix tests check eigenvalue regularization, dense versus diagonal choices, whitening finiteness, and covariance construction from Hessian helpers. These are geometry utilities; they are not identification proofs.

## 43.2 Regularization Policy

A mass matrix used by HMC must be symmetric positive definite. If a curvature estimate is indefinite or ill-conditioned, BayesFilter should regularize it, record the regularization, and expose the eigenvalue diagnostics. Silently using an indefinite curvature matrix as if it were a covariance is a target geometry error.

## 43.3 Geometry-Scaled Tuning Budgets

Mass-matrix diagnostics can inform preparation budgets. A fixed number of samples is a policy choice, not a property of the model. BayesFilter's geometry summary records:

- parameter dimension $d$;
- covariance condition number, when available;
- effective dimension, computed from the covariance eigenvalue spectrum;
- anisotropy ratio $d/d_{\text{eff}}$;
- regularization pressure, including eigenvalue clipping, non-positive raw eigenvalues, and diagonal fallback use.

Historical stage schedules use this summary to allocate local work through `HMCGeometryScaledBudge` Its first attempt budget is

$$B_0 = \text{clamp}\left(\lceil c_d \, d \, m_{\text{geom}} \rceil, B_{\min}, B_{\max,0}\right),$$

where $m_{\text{geom}}$ is a clipped multiplier built from condition, anisotropy, and regularization pressure. Later attempts grow as

$$B_k = \min(B_{\max}, B_0 2^k).$$

Bootstrap screens use a smaller square-root dimension rule,

$$N_{\text{boot}} = \text{clamp}\left(\left\lceil c_{\sqrt{d}} \sqrt{d} \, m_{\text{geom}} \right\rceil, N_{\min}, N_{\max}\right),$$

with a discarded initialization segment chosen as a fixed fraction of the screen length. This preparation allocation does not establish sufficient posterior equilibration; the separate assessment is described in Chapter 47.

These formulas are heuristics, not posterior convergence criteria. They explain historical stage budgets and the geometry-based preparation policies that remain available. The current candidate-set search freezes its own measurement and verification allocations and finite `evidence_rungs`. The convenience initial allocation comes from the acceptance policy's minimum block evidence; target-specific adequacy must still be checked. Geometry-derived preparation does not silently change that declared search allocation.

## 43.4 Timing And Progress

Wall-clock limits are treated differently from statistical tuning budgets. A stage may run for a long time because the model is large but still making meaningful progress. BayesFilter therefore separates:

- progress evidence, such as stage transitions, updated mass summaries, changed step-size hashes, changed trajectory hashes, repair decisions, and fresh verification summaries;
- emergency caps, which are machine-protection limits rather than claims that the sampler has failed scientifically.

Current candidate-set tuning records attempted calls, reserved mandatory work, a conservative gradient-work estimate, and elapsed time including preparation. It checks the declared wall limit at preparation progress boundaries and between bounded numerical chunks; native calls require an external process limit for hard preemption. Attempted chunks are charged before execution, including a failed native call; completed saved chunks are not charged again on resume. Unfundable peers remain pending while affordable reserved work can proceed. The older staged timeout settings belong to historical schedules. Neither kind of timeout is evidence against the posterior unless a predeclared continuation veto says so; incomplete searches preserve their evidence and remaining work.

## 43.5 Fixed-Center Score Curvature

Let $c$ be an unchanged reviewed center and $z$ a standardized local offset. Writing $g_z$ for the score in those coordinates, BayesFilter fits

$$g_z(c) - g_z(c+z) \approx P_z z, \qquad P_z = C_z^{-1}. \tag{43.1}$$

This relation identifies a local score derivative only when its fit and stability diagnostics pass. It does not require $g_z(c) = 0$, so it is not a Hessian-at-MAP statement. Score magnitude is explanatory telemetry; curvature stability is the geometry target for this API.

The dense comparator estimates a symmetric $P_z$ directly and preserves it before SPD projection. The structured route parameterizes $C_z$ and predicts with its inverse. Covariance parameters preserve SPD by construction, whereas the score equation consumes precision.

## 43.6 One- and Two-Factor Covariance Geometry

For $q \in \{1, 2\}$, let $D = \operatorname{diag}(\sigma_1, \ldots, \sigma_N)$ and let row $L_i$ contain the factor loadings of coordinate $i$. The covariance is

$$C_z = DRD, \qquad R = \operatorname{diag}\big(1 - \|L_i\|_2^2\big) + LL^\top, \qquad \|L_i\|_2^2 < 1. \tag{43.2}$$

The row-ball transform enforces strict interior loadings. For independent standard normals $F_k$ and $e_i$, the same correlation follows from

$$x_i = \sum_{k=1}^{q} \rho_{ki} F_k + \sqrt{1 - \sum_{k=1}^{q} \rho_{ki}^2}\, e_i, \qquad \operatorname{corr}(x_i, x_j) = \sum_{k=1}^{q} \rho_{ki}\rho_{kj}. \tag{43.3}$$

The implementation decodes unconstrained variables to $C_z$, obtains $P_z$ by a Cholesky solve inside every objective evaluation, and predicts $P_z z$. It returns both matrices so callers cannot silently reverse them.

One factor has $N$ deviation coordinates and $N$ loadings, hence $2N$ continuous parameters. The largest-absolute loading is constrained positive to remove the discrete sign ambiguity. Two factors begin with $N + 2N$ coordinates. A two-row triangular anchor forces one loading to zero, leaving $3N - 1$ continuous parameters; positive anchor diagonals remove discrete sign symmetries. A prediction-Jacobian rank check rejects an unidentified second factor. Factor columns and rotations are nuisance representations: covariance and precision are the downstream objects.

## 43.7 Cloud Size, Reuse, and Escalation

The implemented search-count rule is

$$m(N) = \begin{cases} N^2, & N \le 10, \\ 100 + (N-10)\log(N-10), & N > 10, \end{cases} \qquad m_{\text{even}} = 2\left\lceil m(N)/2 \right\rceil. \tag{43.4}$$

It is a user-selected computational policy, not a sample-complexity theorem. The search uses stateless randomized orthogonal frames and antithetic pairs. Finite nonzero earlier rows within the current standardized radius may be reused for training only. Every structured replicate still supplies at least $4N$ fresh rows, split between training and selection holdout. Fixed-center selection adds at least $2N$ fresh audit rows that are used only after selection.

Both APIs try one factor before two. The fixed-center route fits two factors only when one-factor fit, holdout, or cross-replicate stability fails, or when the caller explicitly requests a two-factor shrinkage target. The sequential proposal path follows the same one-before-two order. Its trust-region quadratic proposes a step, but the exact target value and score determine actual improvement and acceptance; regression never replaces the posterior. The fixed-center path does not propose a new center.

## 43.8 Stability, Consensus, and Fallback

Every requested replicate retains raw eigenvalues and inertia. Pairwise diagnostics include generalized-eigenvalue spread, trace-normalized Frobenius and operator differences, and principal angles for a caller-declared dominant subspace. Projection burden and selection-holdout error are separate gates. Omitted caps make these quantities descriptive and prohibit HMC-canary eligibility; the DZ5 case does not define universal thresholds.

For admissible SPD precision replicates $K_s$, convex consensus shrinkage is

$$\bar{K} = S^{-1}\sum_{s=1}^{S} K_s, \qquad K_\lambda = (1-\lambda)\bar{K} + \lambda T, \qquad 0 \le \lambda \le 1. \tag{43.5}$$

This is plain convex shrinkage of designed score-regression fits, not the classical Ledoit–Wolf estimator. No closed-form Ledoit–Wolf intensity is used. The caller supplies a finite weight grid containing zero and one. Training and selection-holdout evidence select the weight; a disjoint audit cloud can veto but cannot alter it.

The fail-closed order is:

(i) a stable, identified, holdout-passing one-factor precision;

(ii) a stable, identified two-factor precision when one factor is inadequate;

(iii) a stable dense consensus combined with an explicitly selected, stable, identified structured target;

(iv) if no structured target is supported, a stable dense consensus combined with its diagonal;

(v) diagonal-only output as an explicitly reviewed diagnostic or operational artifact, never an automatic scientific promotion.

An inadmissible raw dense family cannot be repaired by diagonalizing its projected consensus. Identity is not an automatic fallback. If no branch passes, BayesFilter returns `geometry_readiness_blocked`.

# Chapter 44

# Boundary Handling and Safe Gradients

Boundary handling is part of the posterior target. Parameters can leave the stationary, positive-definite, or economically admissible region during HMC warmup. A robust implementation must handle those proposals without crashing the chain or returning non-finite gradients.

## 44.1 Finite Invalid Returns

The preferred policy is:

(i) detect invalid regions before fragile factorizations where possible;

(ii) return either the exact support-rejection value ($-\infty$) or a declared finite fallback scalar for a modified target;

(iii) if a finite fallback scalar is used, return the exact derivative of that same scalar;

(iv) record the invalid reason in diagnostics;

(v) keep the behavior stable under JIT compilation.

This policy is not a mathematical trick to make an invalid model valid. The exact support-rejection contract and the finite modified-target contract are different mathematical objects and must be named as such. A finite fallback may keep sampler control flow in a compiled finite-valued regime, but it does not by itself guarantee rejection or preserve the exact support-restricted target.

## 44.2 Boundary Checklist

Every HMC target should state its policy for prior support, parameter transforms, stationarity or determinacy restrictions, covariance pivots, solve residuals, non-finite likelihoods, and non-finite gradients.

## 44.3 Rank, Chart, and Affine-Support Boundaries

A singular Gaussian score is a smooth local object only on a fixed-rank, fixed-chart, fixed-support stratum. Rank changes, pivot changes, QR sign changes, support changes, and signed-weight PSD boundaries are not ordinary interior points of that score program. BayesFilter therefore distinguishes:

- an on-support fixed branch, which returns the affine-support value and its analytical derivative;
- an off-support observation, which has exact value $-\infty$ under the singular law and no admitted classical score; and
- a rank or chart crossing, which invalidates the frozen score branch and requires a new value-only preflight before another differentiated run.

The implementation reports these events instead of adding an undeclared nugget or differentiating through SVD rank discovery. Consequently, the fixed-rank score is suitable only for HMC targets whose trajectories remain inside a predeclared stratum or whose outer target logic handles boundary rejection. This is a bounded differentiability claim, not a claim of a globally smooth singular-covariance posterior.

# Chapter 45

# Occasionally Binding Constraints, Gradient Discontinuities, and a Failed Hypothesis

## 45.1 Purpose and scope

This chapter documents a research hypothesis that was *mathematically wrong*, the rigorous audit that exposed it, and the lessons learned. It is preserved here not as a correct method but as an instructive failure: an example of how plausible-sounding geometric reasoning, when not grounded in explicit derivations, can lead to a research program built on a false premise.

The hypothesis was that a softplus-smoothed occasionally binding constraint, when composed with an Unscented Kalman Filter (UKF) recursion inside an HMC target, creates gradient discontinuities that require specialized event-aware integration or neural transport. An adversarial audit proved this claim false: the composed map is $C^\infty$ wherever the UKF covariance remains positive definite, so no gradient discontinuity exists. The real difficulty—if any—is stiffness (high curvature), which belongs to the smooth geometry and calls for step-size reduction or preconditioning, not for event detection.

The chapter structure follows the scientific autopsy method:

1. State the wrong hypothesis and its narrative motivation.

2. Present the rigorous mathematical rebuttal, with full derivations.

3. Identify the specific errors: where intuition diverged from proof, where claims lacked anchors, and where terminology conflated distinct geometric cases.

4. Extract durable lessons for research discipline and claim verification.

**Reader responsibility.** This chapter does not present a correct sampling method. Its value is diagnostic: it shows what a wrong claim looks like when stated precisely, how to audit one rigorously, and how to distinguish "seems like it should cause trouble" from "proven to cause trouble."

## 45.2 The failed hypothesis

### 45.2.1 Narrative form

In a shadow-rate term-structure model or New Keynesian model with an occasionally binding zero lower bound, the forward yield curve or policy rate is bounded below by some $\ell$ (typically 0 or a small negative effective lower bound). A hard censoring rule

$$i_t = \max\{\ell, i_t^\star\} \tag{45.1}$$

is continuous but has a kinked gradient at $i_t^\star = \ell$. Hamiltonian Monte Carlo remains valid for continuous kinked targets when the nondifferentiable set has Lebesgue measure zero[1], but leapfrog integration may be inefficient near the kink.

A natural smoothing strategy replaces the hard max with a softplus approximation:

$$s(u) = \ell + a \log(1 + e^{(u-\ell)/a}) = \ell + a \cdot \operatorname{softplus}\left(\frac{u-\ell}{a}\right), \tag{45.2}$$

where $a > 0$ is a scale parameter. As $a \to 0$, $s(u) \to \max\{\ell, u\}$ pointwise. The softplus is smooth: it is a composition of $C^\infty$ functions (translation, scaling, exponential, logarithm), so $s \in C^\infty(\mathbb{R})$.

**The wrong hypothesis (retracted).** In a nonlinear state-space model where the observation map involves (45.2), if the marginal likelihood is computed via an Unscented Kalman Filter, the composed parameter-to-log-likelihood map can exhibit gradient discontinuities when the latent shadow rate crosses the bound deeply. Specifically, when the state drifts far below the bound, the UKF covariance collapses, sigma points cluster, and the gradient of the log-likelihood with respect to parameters jumps discontinuously. This creates a "continuous-kink" geometry that ordinary HMC handles inefficiently and that motivates event-aware integration or neural transport.

This hypothesis was recorded in a survey manuscript ("Bayesian Inference for Nonlinear Models with a Genuine Zero Lower Bound," §7, August 2026 draft) and motivated an 11-week, 1000 GPU-hour research plan targeting event-aware HMC (Tran & Kleppe reflection/refraction), UKF variance inflation, and neural surrogate training.

### 45.2.2 Why it seemed plausible

The hypothesis rested on several intuitively appealing but unverified steps:

1. **Softplus curvature.** For small $a$, the second derivative $s''(u) = O(a^{-1})$ grows large, suggesting stiffness.

2. **UKF sensitivity loss.** When the latent rate is far below the bound, $s'(u) \approx 0$, so the observation Jacobian $H_t \approx 0$ and the Kalman gain $K_t \approx 0$. The filtered covariance $P_{t|t}$ approaches the predicted $P_{t|t-1}$, and the observation becomes uninformative.

3. **Sigma-point clustering.** As $P_t \to 0$, unscented sigma points collapse toward the mean, and the UKF's quadrature-based likelihood approximation changes character.

[1] Afshar and Domke (2015), "Reflection, Refraction, and Hamiltonian Monte Carlo."

4. **Parameter sensitivity amplification.** If $P_t$ is small and parameter-sensitive, and if the innovation covariance $S_t = H_t P_t H_t^\top + R$ is also small, then $\|\nabla_\theta \log p(y_t \mid y_{1:t-1}, \theta)\|$ could amplify rapidly.

5. **Analogy to hard kinks.** Since the softplus "approximates" the hard max, and hard max has a kinked gradient, perhaps the softplus inherits a "near-kink" that becomes a discontinuity under composition with the UKF recursion.

None of these steps is individually absurd. Together, they painted a picture of a smooth function that, when embedded in a recursive filter and differentiated with respect to parameters, might develop jump discontinuities. The error was *not checking the conclusion with a derivation.*

## 45.3 The rigorous rebuttal

An adversarial audit (Codex, September 2026) subjected the hypothesis to mandatory derivations and fixed-verdict vocabulary. The audit proved the central mathematical claim false. We reproduce the key results here.

### 45.3.1 Softplus smoothness and curvature bound

**Proposition 45.1** (Softplus is $C^\infty$, not kinked)**.** *The softplus map (45.2) is infinitely differentiable with bounded derivatives on compact sets. Its first and second derivatives are*

$$s'(u) = \sigma(z), \qquad s''(u) = \frac{\sigma(z)(1-\sigma(z))}{a}, \tag{45.3}$$

*where $z = (u-\ell)/a$ and $\sigma(z) = 1/(1+e^{-z})$ is the logistic function. The supremum of the second derivative is*

$$\sup_{u\in\mathbb{R}} s''(u) = \frac{1}{4a}. \tag{45.4}$$

*Proof.* Write $s(u) = \ell + a\log(1+e^z)$ where $z = (u-\ell)/a$. The logarithm, exponential, and affine maps are all $C^\infty$ on their domains, and the composition is $C^\infty$ on $\mathbb{R}$.

For the first derivative, use the chain rule:

$$\frac{d}{dz}\log(1+e^z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}} = \sigma(z).$$

Since $dz/du = 1/a$,

$$s'(u) = a\cdot\sigma(z)\cdot\frac{1}{a} = \sigma(z).$$

For the second derivative,

$$\frac{d\sigma}{dz} = \sigma(z)(1-\sigma(z)),$$

so

$$s''(u) = \frac{d\sigma}{du} = \frac{d\sigma}{dz}\cdot\frac{dz}{du} = \frac{\sigma(z)(1-\sigma(z))}{a}.$$

The product $\sigma(1-\sigma)$ achieves its maximum at $\sigma = 1/2$ (i.e., $z = 0$), where $\sigma(1-\sigma) = 1/4$. Thus

$$\sup_u s''(u) = \frac{1}{4a}.$$

□

**Consequence.** The softplus map has no kink, no jump, and no discontinuous derivative of any order. Its curvature scales as $O(1/a)$, which is *stiffness*, not a discontinuity. For small $a$, the second derivative is large but finite and continuous.

### 45.3.2 UKF composition: no gradient jump

**Proposition 45.2** (UKF composition remains smooth)**.** *Let $\theta \in \mathbb{R}^p$ be parameters, and let the UKF recursion compute a filtered mean $m_t(\theta)$ and covariance $P_t(\theta)$ at each time $t$, assuming $P_t(\theta) \succ 0$ for all $\theta$ in a neighborhood. Let the observation map be $h_t(x_t, \theta) = s(g(x_t, \theta))$, where $s$ is the softplus (45.2) and $g$ is smooth. If the UKF operations (Cholesky factorization, matrix inversion, sigma-point propagation) are smooth in $(m_t, P_t, \theta)$ on the open cone $P_t \succ 0$, then the log-likelihood $\ell(\theta) = \log p(y_{1:T} \mid \theta)$ computed via the UKF is smooth in $\theta$ wherever all intermediate $P_t(\theta)$ remain positive definite.*

*Proof sketch.* The UKF recursion is a finite composition of:

1. Smooth parameter-dependent functions: $m_t(\theta)$, $P_t(\theta)$, $g(\cdot, \theta)$, $s(\cdot)$.

2. Smooth matrix operations: addition, multiplication, Cholesky factorization $LL^\top = P$ (smooth on $P \succ 0$), matrix inversion (smooth on invertible matrices).

3. Smooth scalar operations: weighted sums (unscented mean), quadratic forms, log determinant $\log |S|$ (smooth when $S \succ 0$).

At each time $t$:

- Predicted mean and covariance: smooth in $\theta$ if the previous filtered estimates are smooth.

- Sigma points: $\mathcal{X}_t^{(j)} = m_t + (\sqrt{(n+\lambda)P_t})_j$, smooth in $(m_t, P_t)$ via the Cholesky factor.

- Propagated sigma points: $\mathcal{Y}_t^{(j)} = h_t(\mathcal{X}_t^{(j)}, \theta)$, smooth because $s$ and $g$ are smooth.

- Unscented observation mean: $\hat{y}_t = \sum_j w_j \mathcal{Y}_t^{(j)}$, a smooth weighted sum.

- Innovation covariance: $S_t = \sum_j w_j (\mathcal{Y}_t^{(j)} - \hat{y}_t)(\mathcal{Y}_t^{(j)} - \hat{y}_t)^\top + R$, smooth in the sigma points.

- Log-likelihood increment: $-\frac{1}{2}[\log |S_t| + (y_t - \hat{y}_t)^\top S_t^{-1} (y_t - \hat{y}_t)]$, smooth when $S_t \succ 0$.

- Kalman gain and filtered update: smooth matrix operations.

By induction over $t$, the entire log-likelihood $\ell(\theta) = \sum_{t=1}^T \ell_t(\theta)$ is smooth in $\theta$ on the open set where all $P_t(\theta) \succ 0$. □

**Consequence.** There is no composition-induced discontinuity. If the UKF covariance stays positive definite, the gradient $\nabla_\theta \ell(\theta)$ is continuous. The hypothesis that "UKF filtering creates gradient jumps" is **wrong relative to the stated softplus target**.

### 45.3.3 The variance collapse mechanism is backwards

The survey manuscript claimed that when the latent shadow rate drifts far below the bound ($u \ll \ell$), the UKF covariance collapses, amplifying parameter sensitivity and creating gradient instability. The audit proved this mechanism backwards.

**Proposition 45.3** (Variance preservation under insensitive observations)**.** *In the scalar observation case, let $m_t^-$, $P_t^-$ be the predicted mean and variance, $h_t = s(g(m_t^-, \theta))$ the observation function, and $H_t = s'(g(m_t^-)) \cdot g'(m_t^-)$ the observation Jacobian. The Kalman update is*

$$S_t = H_t P_t^- H_t + R, \quad K_t = P_t^- H_t S_t^{-1}, \quad P_{t|t} = (1 - K_t H_t) P_t^-. \tag{45.5}$$

*If $s'(g(m_t^-)) \to 0$ (deep below bound), then $H_t \to 0$, $K_t \to 0$, and $P_{t|t} \to P_t^-$.*

*Proof.* As $H_t \to 0$, the innovation covariance $S_t = H_t P_t^- H_t + R \to R > 0$. The Kalman gain $K_t = P_t^- H_t / S_t \to P_t^- \cdot 0 / R = 0$. The filtered variance $P_{t|t} = (1 - K_t H_t) P_t^- \to (1 - 0) P_t^- = P_t^-$.

The observation is uninformative about the shadow depth: it provides no information to shrink the variance. The filtered variance equals the predicted variance, which can grow under the state dynamics if $\Phi \neq 0$ and $Q > 0$. □

**Consequence.** The correct implication is:

$$\boxed{s' \to 0 \implies H_t \to 0 \implies K_t \to 0 \implies P_{t|t} \to P_t^- \quad \text{(variance preserved or grows)}}$$

not

$$\textit{(Wrong claim, retracted)}\text{: } s' \to 0 \implies \text{variance collapses} \implies \text{gradient jumps.}$$

The survey's §7.1 narrative inverted cause and effect. Insensitive observations preserve variance; they do not collapse it.

### 45.3.4 Innovation covariance bound: no unbounded inverse

The survey claimed that variance collapse could make the innovation covariance $S_t$ small, causing its inverse $S_t^{-1}$ to amplify parameter derivatives unboundedly. This is also false.

**Proposition 45.4** (Innovation covariance lower bound)**.** *For $S_t = H_t P_t^- H_t^\top + R$ with $R \succ 0$, we have $S_t \succeq R$, hence $\|S_t^{-1}\| \leq \|R^{-1}\|$ in operator norm.*

*Proof.* Since $H_t P_t^- H_t^\top \succeq 0$ (positive semidefinite) and $R \succ 0$ (positive definite), $S_t = H_t P_t^- H_t^\top + R \succeq R$. For positive definite matrices, $A \succeq B \succ 0$ implies $A^{-1} \preceq B^{-1}$, so $S_t^{-1} \preceq R^{-1}$, giving $\|S_t^{-1}\| \leq \|R^{-1}\|$. □

**Consequence.** The measurement noise covariance $R$ (fixed at $5 \times 10^{-4}$ in the fixture) bounds the innovation inverse from above, regardless of $P_t$ collapse. The survey's §7.2 amplification mechanism does not follow from variance collapse alone.

### 45.3.5 Sigma-point limit: convergence to linearization

The survey suggested that as $P_t \to 0$, the UKF sigma points undergo a "regime flip" or "simultaneous cluster contribution" that creates discontinuous parameter sensitivity. The audit showed this is a misunderstanding of the sigma-point limit.

**Proposition 45.5** (Unscented mean converges to the observation function)**.** *For the standard symmetric UKF with sigma points $\mathcal{X}_t^{(j)} = m_t \pm \sqrt{(n+\lambda)P_t}\, e_j$, the unscented observation mean is*

$$\hat{y}_t = w_0 h(m_t) + \sum_{j=1}^{n} w_j [h(m_t + c_j) + h(m_t - c_j)],$$

*where $c_j = \sqrt{(n+\lambda)P_t}\, e_j$. As $P_t \to 0$, $c_j \to 0$ and $\hat{y}_t \to h(m_t)$.*

*Proof.* Taylor expand $h(m_t \pm c_j)$ around $m_t$:

$$h(m_t \pm c_j) = h(m_t) \pm J_h(m_t)c_j + \tfrac{1}{2}c_j^\top H_h(m_t)c_j + O(\|c_j\|^3).$$

The odd terms cancel under symmetric weights, and the weights sum to one, so

$$\hat{y}_t = h(m_t) + \tfrac{1}{2}\sum_{j=1}^{n} w_j c_j^\top H_h(m_t)c_j + O(\|P_t\|^{3/2}).$$

As $P_t \to 0$, the second-order correction vanishes, giving $\hat{y}_t \to h(m_t)$. □

**Consequence.** All sigma points converge to the mean $m_t$, and the unscented observation mean converges to $h(m_t)$ (the linearization). There is no "cluster flip" or "simultaneous regime contribution." The UKF gracefully becomes an extended Kalman filter in the limit $P_t \to 0$.

### 45.3.6 Leapfrog stability for smooth stiff potentials

The survey cited a kink-crossing energy-error bound (eq:56a in the survey manuscript) as motivation for event-aware integration. For a smooth potential, this bound is vacuous.

**Proposition 45.6** (Leapfrog stability bound)**.** *For a smooth potential $U(\theta)$ with Hessian eigenvalues bounded by $\lambda_{\max}$, leapfrog integration is stable when*

$$\epsilon < \frac{2}{\sqrt{\lambda_{\max}}}. \tag{45.6}$$

*For the softplus-composed Hessian with $\sup s'' = 1/(4a)$, if the dominant curvature is $\lambda_{\max} = O(1/a)$, then the stability bound is $\epsilon = O(\sqrt{a})$.*

*Proof.* For the harmonic mode with Hessian eigenvalue $\lambda$, the linearized leapfrog update has stability only when $\epsilon\sqrt{\lambda} < 2$. For a general smooth potential, the worst-case mode sets the bound.

If the observation Jacobian $H_t = s'(g(m_t)) \cdot g'(m_t)$ and the parameter Hessian involves $s'' = O(1/a)$, then $\lambda_{\max} = O(1/a)$ is plausible. This gives $\epsilon = O(\sqrt{a})$ as the stability scale. □

**Consequence.** For smooth stiff potentials, the mathematics selects step-size reduction, mass-matrix preconditioning, or Riemannian metric adaptation—not event detection. The survey's eq:56a bound states

$$\Delta H = O(\epsilon \|\nabla U^+ - \nabla U^-\|).$$

For smooth softplus, $\nabla U^+ = \nabla U^-$ (the gradient is continuous), so $\|\nabla U^+ - \nabla U^-\| = 0$ and the bound is $\Delta H = O(\epsilon \cdot 0) = 0$—vacuous. Event-aware integration addresses actual gradient jumps (hard kinks, regime switches), not smooth high curvature.

## 45.4 Specific errors in the failed hypothesis

We now catalog the precise points where the hypothesis diverged from mathematical reality.

### 45.4.1 Error 1: Confusing stiffness with discontinuity

**What was claimed:** "The softplus creates gradient discontinuities when composed with UKF filtering."

**What is true:** The softplus has curvature $s'' = O(1/a)$, which for small $a$ is large but finite and continuous. This is *stiffness*, a property of smooth geometry. Stiffness makes small step sizes necessary for leapfrog stability; it does not create a discontinuity.

**Why the error persisted:** The informal language "the gradient becomes very sensitive" was interpreted as "the gradient jumps," and the limiting behavior $s(u) \to \max\{\ell, u\}$ as $a \to 0$ was misread as implying that the finite-$a$ softplus inherits the hard max's kink. Neither inference is valid without a proof.

### 45.4.2 Error 2: Inverting the variance collapse mechanism

**What was claimed:** "When the latent shadow rate drifts far below the bound, the UKF covariance collapses, creating gradient amplification."

**What is true:** When $s'(u) \to 0$ (deep below bound), the observation Jacobian $H_t \to 0$, the Kalman gain $K_t \to 0$, and the filtered covariance $P_{t|t} \to P_t^-$ (preserved, not collapsed). The observation is uninformative, so it neither shrinks nor amplifies the variance.

**Why the error persisted:** The narrative confused two different regimes: (i) informative observations near the kink, which can shrink variance and make the filter sensitive, versus (ii) uninformative observations far below the bound, which leave variance unchanged. The survey conflated these and concluded that deep-below-bound implies collapse.

### 45.4.3 Error 3: Unbounded inverse claim without the $S \succeq R$ bound

**What was claimed:** "Variance collapse makes the innovation covariance $S_t$ small, causing $S_t^{-1}$ to amplify gradients unboundedly."

**What is true:** $S_t = H_t P_t H_t^\top + R \succeq R > 0$ always, so $\|S_t^{-1}\| \leq \|R^{-1}\|$ is bounded by the measurement noise covariance, regardless of $P_t$.

**Why the error persisted:** The derivation in the survey computed $dS$ and $d(\log|S|)$ but did not check the $S \succeq R$ lower bound before asserting amplification. The mechanism was plausible-sounding but not derived.

### 45.4.4 Error 4: Misunderstanding the sigma-point limit

**What was claimed:** "As $P_t \to 0$, sigma points undergo a simultaneous regime flip or cluster jump that creates discontinuous parameter sensitivity."

**What is true:** As $P_t \to 0$, all sigma points converge to the mean $m_t$, and the unscented mean $\hat{y}_t \to h(m_t)$ smoothly. The UKF becomes an extended Kalman filter in this limit; there is no flip.

**Why the error persisted:** The intuition that "collapsing sigma points must do something discontinuous" was never tested with a Taylor expansion. The correct limit is a smooth convergence, not a jump.

### 45.4.5 Error 5: Convention collision in scaling statements

The survey manuscript used the symbol $\alpha$ with two different meanings:

- In §7 (UKF discontinuities): $\alpha$ as temperature parameter, where the softplus is written $s(u) = (1/\alpha)\log(1 + e^{\alpha(u-\ell)})$, so $\alpha = 1/a$ in our notation.
- In §13 (model specification): $\alpha$ as the scale parameter itself, matching our $a$.

These are reciprocals: $\alpha_{\text{temp}} = 1/\alpha_{\text{scale}}$. Every statement of the form "curvature is $O(\alpha)$" is correct under one convention and wrong under the other. The survey's $\mathrm{O}(\alpha)$ scaling claims are internally inconsistent, making it impossible to verify whether stiffness grows or shrinks with the stated parameter.

**Why the error persisted:** The two sections were written at different times, and no global symbol audit caught the reuse. This is a cautionary example of how notation drift can corrupt quantitative claims.

### 45.4.6 Error 6: Taxonomy confusion

The survey manuscript (§2) correctly defines four posterior geometries:

1. Case 1: Continuous kinked target, $i_t = \max\{\ell, i_t^\star\}$, $\Delta U = 0$.
2. Case 2: Deterministic branch jump, $\Delta U \neq 0$ from selection between equilibria.
3. Case 3: Mixed discrete-continuous target (stochastic regime variable).
4. Case 4: Multiplicity or nonexistence (no well-defined posterior until a selection rule is added).

The survey's §7 then claims the softplus UKF creates a Case 1 (kinked) or Case 3 (value jump) geometry, contradicting the §13 case-study text that explicitly classifies the UKF route as smooth (no kink, no jump). This internal contradiction was not noticed until the audit.

**Why the error persisted:** Different sections were drafted with different claims in mind, and no cross-section consistency check was performed before the hypothesis hardened into a research plan.

## 45.5 What the executed evidence actually shows

An 8-parameter joint HMC run (G2.3, filter-free, hard-max target `mf_c1_k40_hardmax`) was executed and passed a convergence gate at max $\hat{R} = 1.0090$ after warmup 8000 / draws 8000. The bottleneck was parameter $\theta_8$ (log FX measurement noise scale) with ESS 848.7, about 5× slower than the drift parameters.

The G2.3 result document (September 2026) offers two candidate explanations for the $\theta_8$ bottleneck:

1. **Funnel geometry:** Log noise scales in a non-centered parameterization sit in a funnel against the latent innovations. The conditional curvature of the latent-to-parameter map varies with the noise scale itself, so a fixed dense mass matrix cannot flatten the geometry.

2. **Trajectory-length periodicity:** In a stiff direction, leapfrog trajectories can return near their starting point after an integer multiple of the period, producing high acceptance but poor exploration. This would explain the non-monotone mixing (L=64 worse than L=32 and L=128).

Neither explanation invokes UKF filtering, gradient discontinuities, or softplus kinks, because:

- The executed target used the hard max (C1), not softplus (S1).
- The executed target was filter-free joint HMC over latent states, not UKF-marginalized.
- No UKF implementation exists in the `bayesfilter/hardbound/` package to test.

The measured evidence does not support the UKF discontinuity hypothesis. It supports a different bottleneck (funnel or periodicity) in a different target (hard-max joint HMC) that passed its convergence gate without requiring event-aware integration or neural transport.

## 45.6 The research plan that followed the wrong hypothesis

Based on the false discontinuity claim, an 11-week research plan was drafted with the following phases:

1. **Phase 1 (weeks 1-2, 20 GPU-hours):** Measurement. Build a UKF-filtered softplus target, run toy-model and G2.3 diagnostics, and measure gradient discontinuities at kink crossings.

2. **Phase 2 (weeks 3-4, 100 GPU-hours):** Event-aware integration. Implement Tran & Kleppe (2025) reflection/refraction HMC (GRHMC) for the measured discontinuities.

3. **Phase 3 (weeks 5-6, 80 GPU-hours):** UKF variance inflation. Inflate UKF covariance to prevent collapse, retune HMC, and measure impact on mixing.

4. **Phase 4 (weeks 7-10, 800 GPU-hours):** Neural surrogate. Train a Hamiltonian Neural Network on the discontinuous UKF target and use it as an HMC proposal or warping.

5. **Phase 5 (week 11):** Production candidate selection. Rank methods by ESS/gradient ¿ 0.1 threshold and promote to production.

Total budget: 1000 GPU-hours over 11 weeks.

### 45.6.1 Why the plan was abandoned

The adversarial audit (M1-M25, September 2026) delivered 17 "wrong relative to stated target" verdicts, 6 "unsupported" verdicts, 1 "correct," and 1 "heuristic only." The five mandatory mathematical derivations (M1, M3, M4, M5, M6) all contradicted the hypothesis. Specifically:

- M1: Softplus is $C^\infty$; gradient jump claim is false.
- M4: Variance collapse mechanism is backwards.
- M5: Innovation covariance bound prevents unbounded amplification.
- M6: Sigma-point limit is smooth convergence, not a flip.
- M15: Plan contradicts master program's binding non-goals (filter-free, no event-aware HMC).
- M16: G2.3 evidence uses hard-max joint HMC, not UKF-filtered softplus.
- M20: Surrogate-target HMC is biased, not exact.

The plan was retracted in full, the survey §7 scheduled for removal, and the research direction closed. No phase was executed.

## 45.7 Lessons for research discipline

This failure offers several durable lessons for structuring research claims and avoiding similar errors.

### 45.7.1 Lesson 1: Derive, don't narrate

**The failure mode.** The hypothesis was built from a chain of plausible-sounding steps: "softplus is steep" → "UKF becomes sensitive" → "gradients jump." Each local step seemed reasonable, but the composition was never verified with an end-to-end derivation.

**The corrective discipline.** For any mathematical claim that motivates expensive work (measurement, tuning, training, 1000 GPU-hours):

1. Write the claim as a proposition with explicit hypotheses and conclusion.
2. Derive the conclusion from the hypotheses, or cite a theorem that does so.
3. If the derivation is "obvious," write it anyway—errors hide in the obvious.

4. If the derivation is blocked by missing definitions or assumptions, treat that as a signal that the claim is not yet ready.

Narrative momentum ("it makes sense that $X$ would cause $Y$") is not evidence. A correct derivation or a measurement artifact is evidence.

### 45.7.2 Lesson 2: Distinguish stiffness from discontinuity

**The failure mode.** The softplus second derivative $s'' = O(1/a)$ grows large as $a \to 0$, and the pointwise limit $\lim_{a\to 0} s(u) = \max\{\ell, u\}$ is kinked. These facts were conflated into "the softplus is almost kinked, so it acts like a kink."

**The corrective discipline.** Stiffness (large but finite and continuous derivatives) and discontinuity (actual jumps) are distinct geometric classes with distinct samplers:

- **Stiffness** calls for step-size reduction ($\epsilon = O(\lambda_{\max}^{-1/2})$), mass-matrix preconditioning, Riemannian metrics, or trust-region methods. Leapfrog remains valid; it is merely expensive.
- **Discontinuity** calls for event detection, reflection/refraction, or discontinuous HMC variants that explicitly handle the jump in potential or gradient.

A function that is $C^k$ for finite $k \geq 1$ is not discontinuous. Treating smooth high-curvature functions as discontinuous is a category error that leads to the wrong sampler.

### 45.7.3 Lesson 3: Check mechanisms in both directions

**The failure mode.** The narrative was "$s' \to 0$ causes variance collapse." The Kalman equations say $s' \to 0 \implies K_t \to 0 \implies P_{t|t} \to P_t^-$ (variance preserved). The correct implication is the opposite of the claimed one, but the error was not caught because the mechanism was stated in words, not equations.

**The corrective discipline.** For any claimed causal mechanism $A \implies B$:

1. Write the explicit equations or inequalities connecting $A$ and $B$.
2. Derive $B$ from $A$ using those equations.
3. Derive the contrapositive: $\neg B \implies \neg A$. If the contrapositive is false, the original claim is false.
4. Check whether $B \implies A$ is also true, false, or independent. If "thus" appears in the narrative, verify that the implication actually points in the stated direction.

Equations are bidirectional; they can be solved for any variable. A verbal "thus" can point in the wrong direction without triggering an error signal.

### 45.7.4 Lesson 4: Anchor claims to code, not to plans

**The failure mode.** The hypothesis referenced "the UKF route" as if it existed. The survey described a UKF implementation (§13), and the research plan budgeted GPU-hours for UKF experiments. Neither the description nor the plan constituted an implementation. The actual 'bayesfilter/hardbound/' package contained no UKF recursion, no

'tf.where' branching on parameter-dependent conditions, and no Cholesky factorization inside an observation loop—none of the elements required for a UKF-marginalized target.

**The corrective discipline.** Before claiming that "method $X$ exhibits property $Y$":

1. Locate the implementation of method $X$: file path, function name, call chain from entry point to the relevant operation.
2. Run a minimal test that exercises the claimed property $Y$.
3. Record the artifact (test output, diagnostic value, plot) as evidence.

A plan to implement $X$ is not an implementation of $X$. A survey section describing $X$ is not an implementation of $X$. An implementation is code that compiles and runs. If the claim is about code that does not yet exist, state that explicitly: "If we implement $X$, we conjecture $Y$."

### 45.7.5 Lesson 5: Adversarial review catches what friendly review misses

**The failure mode.** The hypothesis was developed incrementally: a survey section here, a research plan there, informal discussions, and gradual narrative convergence. Each local decision seemed reasonable, and no single-author review caught the composition error.

**The corrective discipline.** For research directions that commit significant resources (time, compute, opportunity cost of alternative directions):

1. Write a single self-contained document stating the hypothesis, the evidence for it, the research plan, and the expected outcome.
2. Subject that document to adversarial review: a reviewer whose task is to find flaws, not to approve the plan.
3. Require the reviewer to deliver mandatory derivations for key mathematical claims, not just a thumbs-up/thumbs-down verdict.
4. Use fixed verdict vocabulary (correct, wrong relative to stated target, unsupported, not checked) to prevent evasive language ("reasonable," "approximately," "in some sense").
5. Treat "unsupported" the same as "wrong" for go/no-go decisions: if the evidence is missing, the claim is not ready.

Friendly review surfaces misunderstandings and unclear exposition. Adversarial review surfaces wrong claims that sound right. Both are necessary, but they are not the same service.

### 45.7.6 Lesson 6: "Not checked" is not "probably fine"

**The failure mode.** Several sub-claims in the hypothesis were never verified: the UKF implementation did not exist, the Tran & Kleppe (2025) paper was a dangling citation (not in the bibliography, no full text available), the 800 GPU-hour Phase 4 budget had no cost model, and the ESS/gradient ¿ 0.1 promotion threshold had no calibration to the

measured ladder. Each of these was noted as "to be done later," but the research plan proceeded as if they were already validated.

**The corrective discipline.** Distinguish three evidence states:

- **Verified:** A derivation, measurement, or cited theorem explicitly supports the claim.
- **Unsupported:** No evidence has been checked, but the claim might be true.
- **Wrong:** A derivation or measurement explicitly contradicts the claim.

For go/no-go decisions, treat "unsupported" as a block, not as "probably fine." A hypothesis with three verified steps and two unsupported steps is not 60% validated; it is unvalidated. The unsupported steps are gaps, not rounding errors.

## 45.8 What remains correct

Not every statement in the original hypothesis was wrong. We record the surviving claims to avoid overcorrecting.

### 45.8.1 Stiffness is real

The softplus second derivative bound $\sup s'' = 1/(4a)$ is correct. For small $a$, the curvature is large, and the leapfrog stability bound $\epsilon < 2/\sqrt{\lambda_{\max}}$ can force small step sizes. This is a genuine implementation challenge for HMC, even though it is not a discontinuity.

**What works for stiffness:**

- Step-size adaptation (dual averaging with a conservative target acceptance rate).
- Dense or Riemannian mass-matrix adaptation to precondition the geometry.
- Trust-region or implicit integrators for severely stiff problems.

### 45.8.2 Hard kinks are different

The hard max $i_t = \max\{\ell, i_t^\star\}$ has a derivative that jumps from 0 to 1 at the boundary; the derivative is undefined at equality. This can produce a continuous potential with a discontinuous gradient. The validity of a Metropolis-corrected leapfrog proposal rests on its reversible, volume-preserving composition of momentum kicks and position drifts, with a fixed deterministic force convention and the exact endpoint energy. The measure-zero boundary alone does not establish accurate integration or efficient exploration.

Boundary methods must match the type of boundary. Pakman and Paninski [2014, Section 2] solve the quadratic Hamiltonian exactly between reflections at truncation constraints. Their Bayesian Lasso extension (Section 4, equations 4.12–4.22) instead crosses a continuous kink with continuous position and velocity, updating the region's quadratic dynamics. That special construction does not supply an integrator for an arbitrary hard-max state-space posterior. Afshar and Domke [2015, Algorithm 1 and Section 5] incorporate reflection or refraction at a jump $\Delta U$ in potential energy, with their volume-preservation proof covering affine partition boundaries. When a crossing has $\Delta U = 0$ and nonzero normal momentum, their refraction rule leaves that momentum unchanged.

Thus this rule alone does not correct the integration error caused by a continuous gradient kink.

**Key distinction:** The hard max is a Case 1 geometry (continuous kink, $\Delta U = 0$). The softplus is smooth (no kink, $\Delta U = 0$, but large curvature). They are not the same, and methods for one do not automatically apply to the other.

### 45.8.3 Filter choice matters for HMC targets

For nonlinear state-space models, the choice of filter (particle filter, extended Kalman filter, unscented Kalman filter, ensemble Kalman filter) determines the marginal likelihood approximation and its gradient. Different filters have different:

- Bias-variance tradeoffs (bootstrap particle filter is unbiased but high-variance; UKF is deterministic but biased for highly nonlinear models).
- Differentiability (ordinary resampling is discontinuous; optimal-transport resampling is smooth but adds regularization).
- Computational cost (particle filter scales with particle count $N$; UKF scales with state dimension $d$).

The hypothesis was wrong about *where* the discontinuity arises (not in the softplus-UKF composition), but correct that filter geometry matters for HMC.

### 45.8.4 Funnel geometry is a real bottleneck

The G2.3 leapfrog ladder result identified $\theta_8$ (log FX noise scale) as the bottleneck, with ESS 848.7 vs 4000-5100 for drift parameters. The candidate explanation is funnel geometry: in a non-centered parameterization, the conditional curvature of latent innovations varies with the noise scale, creating a funnel that dense mass matrices cannot flatten.

This is a real geometric challenge, independent of the failed UKF discontinuity hypothesis. Standard repairs for funnel geometry include:

- Non-centered vs centered parameterization (already used in the executed G2.3 target).
- QR reparameterization of the latent-to-observable Jacobian to decorrelate scales from rotations.
- Hierarchical priors on log scales to regularize extreme values.
- Partial centering chosen before sampling. Variationally inferred parameterization learns the interpolation between centered and non-centered coordinates during variational preprocessing, then fixes it for HMC [Gorinova et al., 2020, Section 5.2 and Appendix C]. The variational objective does not guarantee that all posterior modes are represented.

The funnel is a smooth geometry problem, not a discontinuity problem. It is orthogonal to the retracted UKF hypothesis.

## 45.9 Archival status and relation to other chapters

### 45.9.1 This chapter in the BayesFilter monograph

This chapter is an *autopsy*, not a method. It documents a hypothesis that failed adversarial review and was retracted. Its purpose is pedagogical: to show what a wrong claim looks like when stated precisely, how to audit it rigorously, and how to avoid similar errors.

Readers implementing HMC for occasionally binding constraints should read:

- Chapter 44: Boundary handling and safe gradients (correct general treatment).
- Chapter 41: HMC for state-space models (correct filter-HMC integration).
- Pakman and Paninski [2014] for truncated Gaussian and piecewise-quadratic dynamics, and Afshar and Domke [2015] for potential jumps across affine boundaries, with the distinctions above for a continuous kink.
- This chapter only for the cautionary example.

### 45.9.2 Relation to the retracted survey section

The failed hypothesis originated in §7 of the survey manuscript "Bayesian Inference for Nonlinear Models with a Genuine Zero Lower Bound" (August 2026 draft). That section is scheduled for retraction, and the survey will note:

> "An earlier draft (August 2026) included §7, which claimed that softplus-smoothed occasionally binding constraints create gradient discontinuities when composed with UKF filtering. This section was retracted in September 2026 after an adversarial audit proved the central mathematical claim false. The softplus-UKF composition is $C^\infty$ wherever the covariance remains positive definite; the claimed discontinuity does not exist. See BayesFilter monograph Chapter 45 for the full autopsy."

The survey's correct Case 1-4 taxonomy (§2), particle-filter treatment (§6, §9-12), and New Keynesian multiplicity analysis (§8) are unaffected and remain in the final version.

### 45.9.3 Relation to the abandoned research plan

The research plan "UKF Discontinuous-Gradient HMC Program" (August 26, 2026) was archived without execution. Its five phases (measurement, event-aware integration, UKF variance inflation, neural surrogate, production selection) were all predicated on the false discontinuity hypothesis. The plan is preserved in the repository at

```
docs/plans/ukf-discontinuous-gradient-hmc-research-plan-2026-08-26.md
```

with a header stating "RETRACTED: This plan was based on a mathematically false hypothesis. See the Codex audit reply for the rebuttal."

### 45.9.4 Relation to executed work

The "Hard-Bound Kink-Target HMC Master Program" (Program A, August-September 2026) executed filter-free joint HMC on the hard-max target `mf_c1_k40_hardmax` and passed the G2.3 convergence gate. This work is unaffected by the failed UKF hypothesis, because:

- Program A used the hard max (C1), not softplus (S1).
- Program A was filter-free (joint HMC over latent states), not UKF-marginalized.
- Program A's binding non-goals explicitly excluded particle filters, UKF marginal likelihood, and event-aware HMC.

The executed work stands as correct. The abandoned plan was a separate, unexecuted proposal.

## 45.10 Final summary

**The hypothesis:** Softplus-smoothed occasionally binding constraints, when composed with UKF filtering inside an HMC target, create gradient discontinuities that require event-aware integration or neural transport.

**The verdict: Wrong relative to the stated target.** The softplus is $C^\infty$, UKF composition with smooth maps is smooth, and no gradient discontinuity exists. The real difficulty—if any—is stiffness (high curvature $O(1/a)$), which is a smooth geometry problem.

**The specific errors:**

1. Confusing stiffness with discontinuity.
2. Inverting the variance collapse mechanism ($s' \to 0$ preserves variance, does not collapse it).
3. Ignoring the $S \succeq R$ bound (innovation inverse is bounded by measurement noise).
4. Misunderstanding the sigma-point limit (smooth convergence to linearization, not a flip).
5. Convention collision ($\alpha$ used as both $1/a$ and $a$, inverting all scaling claims).
6. Taxonomy confusion (softplus UKF is smooth, not Case 1 kinked or Case 3 value-jump).

**The lessons:**

1. Derive, don't narrate.
2. Distinguish stiffness from discontinuity.
3. Check mechanisms in both directions.
4. Anchor claims to code, not to plans.

5. Adversarial review catches what friendly review misses.

6. "Not checked" is not "probably fine."

**The corrective action:** Survey §7 retracted, research plan archived, hypothesis closed. No phases executed, no GPU-hours spent. The failure was caught by adversarial audit before resources were committed.

**The durable value:** A worked example of how to structure, audit, and reject a wrong hypothesis—and how to extract lessons that prevent the next one.

# Chapter 46

# XLA and JIT Compilation

BayesFilter uses JIT compilation as a production tool, not as a correctness argument. A sampler is not fully compiled merely because an outer function is wrapped in `tf.function`. The complete value-and-gradient path must be inside the compiled region, with static shapes and compatible device placement.

## 46.1 Full-Pipeline Requirement

For HMC, the compiled path includes:

- parameter transform;
- model-provider maps;
- filtering likelihood;
- custom or autodiff gradient path;
- invalid-region branch;
- leapfrog transition or equivalent integrator.

If any component falls back to Python, retraces dynamically, or executes an opaque noncompiled kernel, the implementation should be described as partially compiled.

## 46.2 Device and Shape Constraints

The DSGE XLA source notes emphasize two lessons that carry over to BayesFilter. First, XLA compiles a function for a target platform, so mixing CPU-only custom calls with GPU-targeted operations inside one JIT block can fail. Second, XLA requires static shapes. Dynamic metadata should be encoded as values or opaque compile-time metadata, not as changing tensor ranks.

The practical policy is phase separation: solve or model-construction phases may be compiled separately from the filtering/HMC phase if they have different device requirements. A monolithic JIT block is valuable only when it actually contains compatible operations.

## 46.3 Explicit CPU Multiprocess Cloud Evaluation

Independent fixed-center score rows may be evaluated through `CPUXLACloudEvaluator`. This is an explicitly selected CPU route, not an automatic fallback after a GPU failure. The evaluator creates a persistent spawn-based process pool on its first cloud, after the row count is known, and reuses that pool for later calls. Pool creation temporarily supplies the CPU-only environment inherited before spawn can re-import the caller's main module, and then restores the parent environment. A lightweight bootstrap module that has no framework imports fails closed unless it entered with `CUDA_VISIBLE_DEVICES=-1` and memory growth enabled. It records any framework modules already imported under that inherited environment, then compiles and warms one static $B = 1$ value-and-score function with XLA for reuse. Results are restored to input order, child exceptions propagate, and each child returns its bootstrap provenance.

The user-directed worker baseline is

$$W = \max(1, \lfloor C/3 \rfloor) ,$$

clamped to the first cloud's task count. Here $C$ is the physical-core count from `psutil.cpu_count(logical=False)` when available; otherwise the logical `os.cpu_count()` fallback and its provenance are recorded. Callers may override $W$. This policy is not a performance optimum.

Each completed row advances `completed_rows`. A heartbeat emitted while waiting has `semantic_progress=false` and is not completed work. Time budgets remain safety guards rather than mandatory kill times for a process that continues to publish semantic progress.

The child-process XLA smoke proves only that the tested spawn, import, compile, and evaluation route executes. It does not establish scaling or CPU/GPU superiority. Target-only XLA parity remains separate from full-chain HMC XLA. GPU execution has an independent allocator contract: `TF_FORCE_GPU_ALLOW_GROWTH=true` before import and verified memory growth on every visible GPU. These CPU tests do not establish GPU readiness.

# Chapter 47

# Diagnostics and Failure Taxonomy

Diagnostics are evidence about failures and geometry. They are not proofs of convergence. BayesFilter should report diagnostics at three levels: filter evaluation, derivative evaluation, and sampler behavior.

## 47.1 Filter and Derivative Diagnostics

Filter diagnostics include factor pivots, reconstruction errors, solve residuals, condition numbers, invalid-region counts, and finite-value checks. Derivative diagnostics include score/Hessian finite checks, symmetry errors, backend parity deltas, finite-difference residuals, and custom-gradient parity. For SVD or eigendecomposition paths, singular-value or eigenvalue gap telemetry is mandatory before HMC claims are made.

## 47.2 Sampler Diagnostics

Sampler diagnostics include acceptance rate, divergences, energy behavior, effective sample size, split R-hat, mass-matrix conditioning, and tree depth if NUTS is used diagnostically. Short clean runs are smoke tests. They can justify the next experiment, but they do not establish convergence for an industrial target.

## 47.3 Warm-up and posterior precision

A tuning allocation answers how much computation to spend learning geometry and qualifying a frozen transition. It does not answer how long the resulting chain must run to forget its initialization. BayesFilter therefore keeps all verified $(L, \epsilon)$ pairs and asks the caller to choose a member before posterior assessment. No R-hat, ESS or MCSE threshold changes tuning membership.

The selected member runs a separate discarded warm-up, followed by cumulative retained draws. The shared keyword configuration inherits a minimum of 2,000 warm-up transitions per chain, a recent window of 1,000, modern R-hat at most 1.05, and a cap of 10,000. These numbers are operational owner-policy choices; they are not estimates of a sufficient burn-in for every target. Stan's windowed adaptation likewise allocates configured warm-up to fast step-size and slow metric updates. Finishing that schedule is not a proof of stationarity.

`HMCPosteriorAssessmentPolicy` makes the additional requirements explicit: minimum bulk and tail ESS during warm-up and retention, the number of consecutive successful warm-up checks, and accuracy targets for the retained estimators. Every eligible look reports rank-normalized split and folded R-hat, bulk/tail ESS, original-scale mean ESS and mean MCSE. Bulk and tail ESS use the Stan/ArviZ initial-positive, initial-monotone recursion, with an explicit method identifier in the assessment report. The separate precision-policy autocorrelation estimator retains the TFP convention described below. Custom callbacks add requirements; they cannot replace core health and R-hat checks. The archived wrapper retains its historical strict inequality and declared coordinate/ESS screens, while using the same diagnostic arithmetic. The generic entry point for a verified member is `run_hmc_posterior`.

Repeated successful windows are useful evidence of stability in the region visited. Overlapping windows are not independent replications, and all chains can remain in one unrepresentative mode. A small R-hat cannot detect a mode no chain visits. Model-specific functionals, event indicators and log density can be monitored with a named `quantities_fn` and a stable `quantities_id`. If the warm-up cap is exhausted, the result reports inconclusive equilibration and does not start retained inference. Warm-up draws remain archived and are always excluded from posterior estimates.

A short recent window can itself contain too little effective information for the readiness screen, even after initialization bias has decayed. Increasing the warm-up cap while keeping that window fixed does not increase the information available at each check. A target-specific study should therefore declare both its recent-window length and its total allowance, using pilot dependence estimates to plan them and fresh replications to assess delivery. Changing these settings defines a new posterior policy and checkpoint; it does not revoke or retune the verified kernel.

The shared configuration also permits an explicitly justified finite budget exception: `max_results_per_chain` sets the bound and `count_budget_reason` records why a bound above 10000 is needed. Warm-up and retained maxima must lie within it. The serialized posterior policy labels the nondefault allocation; the canonical 10000 default and all health, R-hat and precision requirements are unchanged.

For a scalar functional $g(\theta)$, let $X_{it} = g(\theta_{it})$ be draw $t$ of independent chain $i$, with $m$ chains of length $n$. Under stationarity and an applicable central limit theorem, define the chain's long-run variance by

$$\sigma_i^2 = \gamma_i(0) + 2\sum_{k\geq 1}\gamma_i(k), \qquad \gamma_i(k) = \mathrm{Cov}(X_{i,t}, X_{i,t+k}). \tag{47.1}$$

Since the pooled estimate is $m^{-1}\sum_i \bar{X}_i$ and chains are independent, its approximate Monte Carlo variance is

$$\widehat{\mathrm{Var}}_{\mathrm{MC}}(\bar{X}) = \frac{1}{m^2 n}\sum_{i=1}^{m}\widehat{\sigma}_i^2, \qquad \mathrm{MCSE}(\bar{X}) = \sqrt{\widehat{\mathrm{Var}}_{\mathrm{MC}}(\bar{X})}. \tag{47.2}$$

Serial dependence enters through $\sigma_i^2$, while independence between chains permits addition of their variances. Concatenating chains into a single serial process would introduce artificial transitions between their endpoints.

For batch size $b$, let $a = \lfloor n/b \rfloor$ and let $\bar{X}_{ik}^{(b)}$ denote the mean in complete batch $k$. Ordinary batch means estimates

$$\widehat{\sigma}_{i,b}^2 = \frac{b}{a-1}\sum_{k=1}^{a}\left(\bar{X}_{ik}^{(b)} - a^{-1}\sum_{j=1}^{a}\bar{X}_{ij}^{(b)}\right)^2. \tag{47.3}$$

The lugsail estimate combines two batch sizes [Vats and Flegal, 2021, section 4, equation (7)]:

$$\widehat{\sigma}^2_{i,L} = \frac{\widehat{\sigma}^2_{i,b} - c\widehat{\sigma}^2_{i,\lfloor b/r \rfloor}}{1-c}, \qquad r \geq 1, \quad 0 \leq c < 1. \tag{47.4}$$

For positively correlated chains, this construction can offset downward bias in the ordinary estimate. It can also increase variability and yield a nonpositive estimate. BayesFilter reports such estimates as unavailable precision evidence instead of clipping them into a passing result. The literature values $r = 3, c = 1/2$ and $b = \lfloor \sqrt{n} \rfloor$ are configurable baselines, not universal recommendations. The minimum of twenty complete batches is an operational stability floor, not a theorem about coverage. Terminal incomplete batches are excluded from long-run variance estimation; their count is recorded, and all retained draws still enter the posterior mean.

Batch length must also resolve the dependence in the chain. For a stationary unit-variance Gaussian AR(1) with correlation $\rho$, define $V(k) = k + 2\sum_{j=1}^{k-1}(k-j)\rho^j$, the variance of a sum of $k$ draws. When $n = ab$, centering the batch means gives

$$\mathbb{E}[\widehat{\sigma}^2_{i,b}] = \frac{V(b) - V(n)/a^2}{b(1-1/a)}. \tag{47.5}$$

This follows by subtracting the variance of the overall batch mean from the sum of the batch second moments. Substituting the two batch sizes into (47.4) shows why a positive estimate can still be biased downward. With $\rho = 0.995$, $n = 10000$, $b = 100$, $r = 3$ and $c = 1/2$, its expected mean-variance estimate is only about 34% of the exact finite-count variance. The smaller batch calculation uses its own complete-batch count. Twenty or more batches therefore cannot certify adequate bandwidth. Dependence, longer-chain evidence and sensitivity to justified batch lengths must be examined when the required accuracy is consequential.

The same calculation gives the exact variance of the pooled mean of $m$ independent stationary chains as $V(n)/(mn^2)$. For fixed $|\rho| < 1$ and long chains this is approximately

$$\mathrm{Var}(\bar{X}) \simeq \frac{1}{mn}\frac{1+\rho}{1-\rho}. \tag{47.6}$$

At $\rho = 0.995$, four unit-variance chains therefore need about 39900 retained draws each to attain mean MCSE 0.05 in this planning approximation. A cap of 10000 is inadequate for that typical precision requirement even when the transition and uncertainty calculation are correct. The actual stopping rule also depends on noisy diagnostic estimates and requires its own calibration; this count is neither a burn-in estimate nor a universal lower bound for HMC.

The autocorrelation option retains the identified TFP 0.25 positive-pairs ESS calculation. Rank-normalized bulk ESS is useful for exploration diagnostics, but original-scale mean uncertainty needs the ESS of the actual functional. For a quantile, BayesFilter follows the indicator-ESS and order-statistic construction in Vehtari et al. [2021, section 4.4], with the explicitly named TFP positive-pairs indicator ESS. The pooled percentile preserves equal interpolation endpoints exactly: otherwise rounding a cutoff below a repeated value can change every indicator at that value and hence its estimated ESS. The corrected quantile method has a versioned identifier in both the report and precision policy, so a checkpoint from the earlier arithmetic cannot be silently reused. A constant rare-event indicator or a tied zero-width quantile interval supplies no reliable accuracy evidence.

The reported quantile MCSE approximates a standard error using central order statistics. Multiplying it by 1.96 gives a symmetric diagnostic interval; this is not the same finite-sample calculation as propagating the .025 and .975 probabilities directly to empirical order statistics. Independent formula agreement checks the arithmetic, not the coverage of either interval under actual HMC stopping. Finite diagnostics do not establish the moments or mixing assumptions needed by any of these estimators.

Each `HMCPrecisionTarget` specifies a mean or a quantile and an absolute MCSE tolerance, an MCSE-to-posterior-SD tolerance, or both. Accuracy is assessed on cumulative retained draws. No supplied tolerance means `precision_not_requested`; a successful diagnostic screen then carries no claim of sufficient estimator accuracy. At the retained cap, unmet targets remain insufficient. Repeated MCSE screening is an operational stopping rule: the implementation does not claim finite-sample or anytime-valid confidence coverage. The asymptotic fixed-width results in Flegal and Gong [2013, sections 2–3.1 and Appendix A] require additional regularity and estimator-consistency conditions. Lugsail estimates uncertainty in retained means; it does not estimate how many burn-in draws are enough. Experiments that compare stopping policies must declare the batch size, minimum batch count and lugsail coefficients together with the recent-window length, effective-sample-size requirements and persistence of readiness checks. The fixed-count comparison uses the same declared estimator. A change to any of these choices requires a fresh fit identity, including when a previous fit has already saved its final summary. Otherwise a cached result could silently be attributed to a policy that never produced it.

Absolute accuracy requirements must be interpreted in the units of the functional. For independent normal observations with standard deviation $s$ and total count $N$, the mean has MCSE $s/\sqrt{N}$. The usual quantile approximation gives variance $p(1-p)/(Nf(q_p)^2)$; substituting $p = 1/2$ and normal density $f(q_{1/2}) = 1/(\sqrt{2\pi}s)$ yields median MCSE $\sqrt{\pi/2}\,s/\sqrt{N}$. Thus a broad normal coordinate can need more draws to estimate its median than its mean at the same absolute tolerance. These independent-draw calculations help plan cost; correlated HMC can have different variances, including smaller ones under negative dependence. A retained cap therefore requires inspecting quantity-specific uncertainty and trajectory behavior before diagnosing a tuning failure. An unmet accuracy requirement remains unmet even when R-hat is favorable.

The diagnostic version `bayesfilter.hmc_diagnostic_math.v2` corrects two prior arithmetic errors: normal-score ranks use $S + 1/4$ in the denominator, and modern R-hat is the square root of the variance ratio. Earlier reported threshold results require recomputation from draws before current use. Adjacent-epoch drift statistics remain descriptive: the old normalization omits covariance between adjacent epoch means and is not a calibrated test. The legacy Phase 29 warmup screen still compares these statistics with its configured heuristic thresholds. Those comparisons can reject that legacy screen; they are not part of the common posterior assessment and do not have calibrated significance levels.

Listing 47.1: A verified Gaussian member with explicit posterior accuracy targets.

```
"""Verified tuning member to posterior precision: a small engineering example.

Counts, tolerances and seeds below are explicit Gaussian fixture choices. They
are not defaults for a scientific model. GPU/XLA is the normal mode;
--cpu-reference selects a small CPU/non-XLA reference check.
"""
from __future__ import annotations
import argparse
```

```
import json
import os
from pathlib import Path
import sys
import time

REPO_ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(REPO_ROOT))


def run_example(root: Path, *, cpu_reference=False):
    os.environ['TF_FORCE_GPU_ALLOW_GROWTH'] = 'true'
    if cpu_reference:
        os.environ['CUDA_VISIBLE_DEVICES'] = '-1'
    import tensorflow as tf
    from bayesfilter.runtime.gpu_memory_policy import
    configure_tensorflow_gpu_memory_growth
    memory = configure_tensorflow_gpu_memory_growth(tf, require_gpu=not cpu_reference)
    from bayesfilter.inference import (
        HMCAcceptancePolicy, HMCCandidateExecutionConfig, HMCControllerConfig,
        PrecomputedMassArtifact, bind_hmc_candidate_set_execution, tune_hmc_kernel,
        build_retained_bound_hmc_archive_runner_from_candidate_set_result,
        HMCPrecisionTarget, HMCPrecisionPolicy, HMCPosteriorAssessmentPolicy,
        SequentialNeuTraHMCConfig, run_hmc_posterior,
    )
    from bayesfilter.runtime.durable_tensor_checkpoint import DurableTensorCheckpoint
    from docs.examples.hmc_candidate_set_retained import GaussianTarget

    target = GaussianTarget()
    binding = bind_hmc_candidate_set_execution(adapter=target,
        initial_position=tf.constant([[-1.,-.5],[-.3,.2],[.4,-.2],[1.,.5]], tf.float64
    ),
        mass_artifact=PrecomputedMassArtifact(position=[0.,0.],
            covariance=tf.eye(2,dtype=tf.float64), factor=tf.eye(2,dtype=tf.float64),
            adapter_signature=target.adapter_signature(), position_role='
    reference_center',
            covariance_source='known standard Gaussian fixture'),
        target_scope='docs_candidate_retained', scope_id='docs-posterior', search_id='
    example-1',
        target_lineage={'model':'standard_normal_2d', 'data':'none', 'prior':'
    target_itself'},
        source_paths=[__file__], epsilon_domain=(.1,1.9), repair_factor=1.1,
    max_repairs_per_family=0,
        config=HMCCandidateExecutionConfig(measurement_num_results=128,
            verification_num_results=128, num_warmup_steps=8, seed=(20260914,11),
            acceptance_policy=HMCAcceptancePolicy(practical_region=(.55,.85),
    repair_region=(.5,.9)),
            use_xla=not cpu_reference, target_status_trace_policy='per_chain_step',
            non_xla_reason='Gaussian documentation CPU reference' if cpu_reference
    else None))
    tuning = tune_hmc_kernel(adapter=target, initial_position=binding.
    initial_active_state,
        candidate_set_adapter=binding.typed_adapter, output_dir=root/'tuning',
        config=HMCControllerConfig(primary_l_grid=(3,), epsilon_by_l=((3,(1.3,)),),
            total_budget_units=10, repair_reserve_units=3))
    if not tuning.result.verified_candidate_ids:
        raise RuntimeError('bounded fixture produced no verified member')
    # This fixture has one L. For a real grid select a member explicitly;
```

```
    # descriptive acceptance or ESS differences do not establish a ranking.
    member = build_retained_bound_hmc_archive_runner_from_candidate_set_result(
        candidate_set_result=tuning.result, candidate_id=tuning.result.
    verified_candidate_ids[0],
        retained_binding=binding)
    member.export(root/'member.json')
    policy = HMCPosteriorAssessmentPolicy(warmup_bulk_ess_min=50., warmup_tail_ess_min
    =50.,
        warmup_consecutive_checks=2, retained_bulk_ess_min=100., retained_tail_ess_min
    =100.,
        precision=HMCPrecisionPolicy((HMCPrecisionTarget('x', mcse_absolute_max=.10),
            HMCPrecisionTarget('y', kind='quantile', probability=.9, mcse_absolute_max
    =.15)),
            method='lugsail', jit_compile=not cpu_reference))
    config = SequentialNeuTraHMCConfig(step_size=member.step_size,
        num_leapfrog_steps=member.num_leapfrog_steps, jit_compile=not cpu_reference,
        warmup_seed=(20260915,17), retained_seed=(20260915,19),
        warmup_chunk_results=256, warmup_min_results=256, warmup_check_window_results
    =256,
        warmup_max_results=1024, retained_chunk_results=512, retained_min_results=512,
        retained_max_results=2048, assessment_policy=policy)
    with DurableTensorCheckpoint(root/'posterior_chunks', {
        'member_hash':member.member_hash, 'transform':'identity', 'quantities':'x_y_v1
    ',
        'parameter_names':['x','y'], 'consumer_diagnostic':'none', 'policy':policy.
    payload(),
    }) as store:
        result = run_hmc_posterior(member=member, config=config, parameter_names=('x',
    'y'),
            checkpoint_store=store)
    return {**{k:v for k,v in result.items() if not k.startswith('private_')},
        'memory_policy':memory, 'runtime_policy':binding._runtime,
        'source_closure':binding._spec['source_closure'],
        'interpretation':'Gaussian engineering smoke; no model-specific burn-in or
    default promotion'}


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('--output-dir', type=Path, required=True)
    parser.add_argument('--cpu-reference', action='store_true')
    args = parser.parse_args()
    args.output_dir.mkdir(parents=True, exist_ok=False)
    started = time.monotonic()
    result = run_example(args.output_dir, cpu_reference=args.cpu_reference)
    result['wall_seconds'] = time.monotonic()-started
    (args.output_dir/'result.json').write_text(json.dumps(result,indent=2,allow_nan=
    False)+'\n')
    print(json.dumps({k:result[k] for k in ('passed','equilibration_status','
    precision_status','wall_seconds')}))


if __name__ == '__main__':
    main()
```

## 47.4 Failure Taxonomy

Common failure classes are:

- target-domain failures;
- factorization or solve failures;
- non-finite value or gradient failures;
- spectral derivative instability;
- JIT retracing or partial compilation;
- sampler geometry failures;
- unsupported missing-data or mask patterns.

Each failure should point to a next action: fix the model contract, change the backend, add jitter or regularization, strengthen derivative validation, or downgrade the backend to diagnostic use.

## 47.5 Curvature Diagnostic Roles

Fixed-center curvature diagnostics have explicit roles:

**required gate** finite raw matrices, caller-declared raw inertia, selection-holdout error, projection burden, complete pairwise stability caps, and the independent audit error;

**promotion veto** a failed required gate blocks nomination for an exact HMC mechanics canary;

**repair trigger** failed one-factor holdout can trigger an identified two-factor fit, while unstable dense curvature triggers structured or consensus analysis;

**explanatory** center score magnitude, optimizer iterations, descriptive runtimes, and any stability metric for which no cap was supplied.

SPD projection does not convert failed raw curvature into a pass. Raw inertia and matrices remain visible. Likewise, a low score is not a curvature gate and a stable nonzero-score center is not a MAP. A locally useful geometry for an exact Metropolis-corrected HMC canary remains unproved as a posterior covariance or global-mode Hessian.

`geometry_readiness_blocked` rejects the current evidence and mass handoff. It does not imply that the model, target, or curvature-preconditioning research direction has failed.

# Chapter 48

# Transport Maps and Surrogate Geometry

Transport maps and surrogates are downstream geometry tools. They can make an HMC target easier to explore, but they cannot repair an invalid likelihood, an inconsistent custom gradient, or a non-finite filter boundary. This chapter therefore depends on the target, derivative, and diagnostics contracts already defined in the earlier chapters.

## 48.1 Transported Targets

Let $q \in \mathbb{R}^d$ denote the original unconstrained parameter and let

$$q = T_\psi(z)$$

be an invertible differentiable map from transported coordinates $z$ to the original coordinates. If the original target density is

$$\pi_Q(q) \propto \exp[-U_Q(q)],$$

then the transported target is

$$U_Z(z) = U_Q(T_\psi(z)) - \log \left| \det J_{T_\psi}(z) \right| . \tag{48.1}$$

Running HMC in $z$-space is target-preserving only when the sampler evaluates (48.1) and its gradient. The Jacobian term is not optional. A learned map that is imperfect can still be useful as a preconditioner, but omitting the change-of-variables correction changes the target.

## 48.2 NeuTra Position

The Phase 2B literature gate accepted NeuTra only as transport and geometry support. That is exactly the right status for BayesFilter. A NeuTra-like map can improve posterior geometry by making the transformed target closer to a simple reference distribution. It is not a correctness guarantee, and it is not a substitute for a valid filtering likelihood or a derivative provider that matches the Metropolis value.

The strategic use is:

(i) certify the filtering target and gradient first;

(ii) train or choose a transport map to improve geometry;

(iii) run HMC on the Jacobian-corrected transported target;

(iv) compare diagnostics and effective sample size per unit time to the untransported or affine baseline.

If the map has poor tail behavior, bad inverse stability, or large Jacobian curvature, it can make sampling worse even when it improves a training loss.

## 48.3 Surrogate Targets

A surrogate likelihood or surrogate Hamiltonian is a different object from a transport map. If the surrogate changes the scalar log target used in the Metropolis correction, the Markov chain targets the surrogate posterior unless a correction mechanism is added. BayesFilter should record one of three statuses for every surrogate:

**diagnostic surrogate** used only for exploration, initialization, or comparison; no posterior correctness claim is made;

**corrected surrogate** used inside a delayed-acceptance or Metropolis-corrected scheme that evaluates the intended target before a sample is accepted;

**target-defining surrogate** deliberately defines the approximate posterior being sampled, with approximation error documented separately.

This distinction prevents a common failure mode: using a fast approximate filter or neural surrogate for gradients and then interpreting the resulting chain as if it sampled the exact filtering posterior.

## 48.4 Diagnostics

Transport diagnostics should be tied to sampler performance and target integrity:

- transformed-target finite value and finite gradient checks;
- log-Jacobian finite checks and inverse/forward reconstruction residuals;
- tail probes for large $\|z\|$;
- acceptance rate, divergences, E-BFMI, and effective sample size per second relative to an affine baseline;
- shell-level Hessian or force diagnostics where available.

These diagnostics explain why a map helps or fails. They do not override a failed target contract.

## 48.5 BayesFilter Policy

Transport and surrogate methods should enter BayesFilter after the exact or controlled approximate likelihood path has been validated. They are valuable for large models because geometry can dominate HMC cost, but they sit on top of the filter contract rather than underneath it. In particular, they should not be proposed as a fix for SVD spectral-gradient instability, missing derivative guards, or partial XLA compilation. Those problems must be solved at the filter and sampler-contract layer first.

# Chapter 49

# Neural Transport Hamiltonian Monte Carlo

This chapter gives a self-contained account of neural-transport Hamiltonian Monte Carlo (NeuTra HMC) and documents the BayesFilter implementation and experiments available in July 2026, followed by a September 2026 investigation of weak nonlinear learning in the q20 target. The construction was introduced by Hoffman et al. [2019]; the derivation and implementation below do not require the reader to consult that paper. The HMC and transport mathematics, diagnostics, and five common truth-tail targets are derived explicitly. Eight additional historical configurations are reported under their preserved protocols. Four of those are inherited DSGE targets whose complete economic model systems belong to their dedicated chapters and repositories; they are identified here at the resolution needed to interpret the NeuTra evidence, but are not presented as self-contained re-derivations of those economic models.

## 49.1 Motivation: geometry is often the HMC bottleneck

Let $\vartheta$ denote scientifically meaningful parameters and let $y$ denote the observations. Bayesian computation targets

$$p(\vartheta \mid y) = \frac{p(y \mid \vartheta)p(\vartheta)}{p(y)}. \tag{49.1}$$

The evidence $p(y)$ is normally unavailable, but HMC needs only an unnormalized log posterior and its gradient. Constraints are first removed by a model chart $\vartheta = C(\theta)$, $\theta \in \mathbb{R}^d$. Thus the source-coordinate target used by computation is

$$\widetilde{\pi}_\Theta(\theta) = p(y \mid C(\theta))p(C(\theta)) \left|\det J_C(\theta)\right|, \tag{49.2}$$

$$\ell_\Theta(\theta) = \log \widetilde{\pi}_\Theta(\theta). \tag{49.3}$$

The tilde emphasizes that normalization is unnecessary.

Euclidean HMC is efficient when typical posterior directions have comparable scales and curvature. It can be inefficient when a single global mass matrix cannot accommodate strong correlations, changing curvature, funnels, or nonlinear ridges. A step size small enough for the stiffest region then makes movement elsewhere slow. An affine

whitening map repairs constant covariance but cannot generally straighten changing geometry. NeuTra learns an explicit nonlinear coordinate map and runs ordinary HMC in the learned coordinates.

Three facts motivate the method and delimit its claims.

1. The learned variational distribution need not be exact. Its purpose is to provide useful coordinates, not final posterior draws.
2. The Jacobian-corrected HMC target is exact for the declared source-coordinate posterior whenever the frozen map is a bijection and HMC is implemented correctly.
3. A poor map can still reduce efficiency or hide a mode from finite runs. Exact invariance is not a finite-time convergence guarantee.

## 49.2 Hamiltonian Monte Carlo from first principles

For a target $\widetilde{\pi}(q)$ on $q \in \mathbb{R}^d$, introduce momentum $p \sim \mathcal{N}(0, M)$ with positive-definite mass $M$. Define

$$U(q) = -\log \widetilde{\pi}(q), \qquad K(p) = \tfrac{1}{2} p^\top M^{-1} p, \qquad H(q,p) = U(q) + K(p). \tag{49.4}$$

Hamilton's equations are

$$\dot{q} = M^{-1} p, \qquad \dot{p} = \nabla_q \log \widetilde{\pi}(q). \tag{49.5}$$

Along an exact trajectory,

$$\frac{dH}{dt} = \nabla_q H^\top \dot{q} + \nabla_p H^\top \dot{p} \tag{49.6}$$

$$= \nabla U(q)^\top M^{-1} p + (M^{-1}p)^\top [-\nabla U(q)] = 0. \tag{49.7}$$

The phase-space velocity is $v(q,p) = (\nabla_p H, -\nabla_q H)$, whose divergence is

$$\nabla_q \cdot \nabla_p H - \nabla_p \cdot \nabla_q H = 0 \tag{49.8}$$

because the mixed derivatives cancel. Liouville's formula therefore gives a unit Jacobian determinant for the exact flow, so phase-space volume is preserved. If $(q(t), p(t))$ solves the equations, then $(q(-t), -p(-t))$ does also, proving reversibility after a momentum flip.

Numerically, one leapfrog step of size $\epsilon$ is

$$p_{n+1/2} = p_n + \tfrac{\epsilon}{2} \nabla \log \widetilde{\pi}(q_n), \tag{49.9}$$

$$q_{n+1} = q_n + \epsilon M^{-1} p_{n+1/2}, \tag{49.10}$$

$$p_{n+1} = p_{n+1/2} + \tfrac{\epsilon}{2} \nabla \log \widetilde{\pi}(q_{n+1}). \tag{49.11}$$

Each substep is a shear: it changes one block while holding the other fixed, and therefore has block-triangular Jacobian determinant one. Their composition is volume-preserving. Replacing $\epsilon$ by $-\epsilon$ and reversing the substep order recovers the starting state; because the kick–drift–kick order is symmetric, a momentum flip supplies the required proposal reversibility. After $L$ steps, the proposal $(q', -p')$ is accepted with probability

$$\alpha = \min\{1, \exp[-H(q',p') + H(q,p)]\}. \tag{49.12}$$

Indeed, for the joint density $\rho(q,p) \propto e^{-H(q,p)}$ and the deterministic reversible proposal map $F$, the accepted forward probability mass is

$$\rho(x)\min\left\{1, \frac{\rho(Fx)}{\rho(x)}\right\} = \min\{\rho(x), \rho(Fx)\} = \rho(Fx)\min\left\{1, \frac{\rho(x)}{\rho(Fx)}\right\}, \tag{49.13}$$

where $x = (q,p)$. This is detailed balance; the unit Jacobian means no proposal volume factor is missing. The Metropolis correction therefore removes leapfrog discretization bias. A high acceptance rate alone is not evidence of good exploration: $\epsilon$ can be so small that proposals are nearly unchanged.

## 49.3 Exact neural change of variables

Let $z \in \mathbb{R}^d$ be latent HMC coordinates and let

$$\theta = T_\phi(z) \tag{49.14}$$

be a differentiable bijection with frozen parameters $\phi$. For a measurable set $A$ in source space,

$$\Pr(\Theta \in A) = \int_A \pi_\Theta(\theta)\, d\theta \tag{49.15}$$

$$= \int_{T_\phi^{-1}(A)} \pi_\Theta(T_\phi(z))|\det J_{T_\phi}(z)|\, dz. \tag{49.16}$$

Therefore the pulled-back target is

$$\widetilde{\pi}_Z(z) = \widetilde{\pi}_\Theta(T_\phi(z))|\det J_{T_\phi}(z)|, \tag{49.17}$$

$$\ell_Z(z) = \ell_\Theta(T_\phi(z)) + \log|\det J_{T_\phi}(z)|. \tag{49.18}$$

If a Markov kernel leaves $\pi_Z$ invariant, pushing its stationary draws through $T_\phi$ produces draws from $\pi_\Theta$. Hence approximation error in training does not change the stationary posterior. It changes only the geometry, initialization, and finite-compute efficiency.

By the chain rule, the latent score is

$$\nabla_z \ell_Z(z) = J_{T_\phi}(z)^\top \nabla_\theta \ell_\Theta(T_\phi(z)) + \nabla_z \log|\det J_{T_\phi}(z)|. \tag{49.19}$$

Both terms are derivatives of the same scalar target. Dropping the log Jacobian changes the probability measure; dropping its derivative gives the wrong force for that measure. During HMC, $\phi$ is fixed, so no optimizer state participates in (49.19).

Artifacts distinguish latent $z$, unconstrained source $\theta$, and physical $\vartheta = C(\theta)$. HMC integrates $z$; the filter target consumes $\theta$; scientific summaries use $\vartheta$.

## 49.4 The historical BayesFilter dense IAF

The implementation and numerical settings in this section are historical and unfaithful to the original author's code. The current author-based IAF uses a free scale bias outside the conditional cap and nonzero author initialization; see Section 50.8 and the implementation

reference. The historical formulas remain here to explain the preserved experiments, not as the active architecture or training recipe.

The tested map is a stack of three MADE-style dense inverse autoregressive flow (IAF) stages, reverse-order permutations between stages, and a fixed full affine lift. This section derives that exact implementation.

For one stage, masked dense networks produce vectors $a(z)$ and $b(z)$ such that coordinate $i$ depends only on $z_{1:i-1}$. BayesFilter bounds the log scale by

$$s_i(z_{1:i-1}) = s_{\max} \tanh\left(\frac{a_i(z_{1:i-1})}{s_{\max}}\right), \qquad s_{\max} = 1, \tag{49.20}$$

and applies

$$u_i = z_i \exp\{s_i(z_{1:i-1})\} + b_i(z_{1:i-1}). \tag{49.21}$$

Because $s_i$ and $b_i$ do not depend on $z_i$,

$$\frac{\partial u_i}{\partial z_i} = e^{s_i}, \qquad \frac{\partial u_i}{\partial z_j} = 0 \quad (j > i). \tag{49.22}$$

The Jacobian is lower triangular, so

$$\det J_f(z) = \prod_{i=1}^{d} e^{s_i(z_{1:i-1})}, \qquad \log|\det J_f(z)| = \sum_{i=1}^{d} s_i(z_{1:i-1}). \tag{49.23}$$

Invertibility follows constructively. The first coordinate satisfies $u_1 = e^{s_1} z_1 + b_1$, where $s_1, b_1$ are constants, so $z_1 = e^{-s_1}(u_1 - b_1)$. After $z_{1:i-1}$ have been recovered, evaluate $s_i(z_{1:i-1})$ and $b_i(z_{1:i-1})$ and set

$$z_i = e^{-s_i(z_{1:i-1})}\{u_i - b_i(z_{1:i-1})\}. \tag{49.24}$$

This recursion gives a unique inverse for every $u \in \mathbb{R}^d$. The positive diagonal derivative agrees with that inverse construction. The scale bound prevents a single stage from creating arbitrarily large or small local volume changes.

Let $f_1, \ldots, f_K$ denote the IAF stages and $P$ the reverse permutation. For $K = 3$, define

$$h_1 = f_1(z), \quad h_2 = f_2(Ph_1), \quad h_3 = f_3(Ph_2), \quad T_\phi(z) = c + Lh_3, \tag{49.25}$$

where $L$ is a fixed nonsingular full matrix. Since $|\det P| = 1$,

$$\log|\det J_T(z)| = \log|\det L| + \sum_{k=1}^{K}\sum_{i=1}^{d} s_{k,i}. \tag{49.26}$$

The affine $(c, L)$ supplies target-specific location and scale; only the neural stages are trained. Output-layer weights are initialized at zero, so the residual stack begins as identity. Hidden weights use small stateless-normal initialization, and reverse permutations let later stages model dependencies hidden by a single ordering. These are tested implementation choices, not universal optimal defaults.

BayesFilter also exposes a distinct pure-IAF family. For that family there is no fixed outer $(c, L)$ component:

$$T_\phi^{\text{pure}}(z) = f_3(Pf_2(Pf_1(z))), \qquad \log|\det J_{T^{\text{pure}}}(z)| = \sum_{k=1}^{3}\sum_{i=1}^{d} s_{k,i}. \tag{49.27}$$

When heterogeneous physical units make identity initialization numerically unsuitable, the final IAF output biases may be initialized so that the stack has $T_\phi^{\text{pure}}(0) = c_0$ and its leading diagonal scale is $\exp(s_0)$. Both biases remain trainable members of $\phi$; $c_0$ and $s_0$ are not serialized as a fixed affine chart and do not contribute an extra Jacobian term. A bounded-scale recipe must strictly contain every entry of $s_0$; the paper-style recipe instead uses the MADE log-scale output directly, as in the reference implementation. Applying this initialization at the final stage keeps the preceding IAFs in latent scale; putting it at the first stage would make every later network consume heterogeneous physical-unit activations. This warm start changes the optimizer's initial state, not the change-of-variables formula or the target density.

## 49.5 Reverse-KL training and its gradient

Let $r(z) = \mathcal{N}(0, I)$ and let $q_\phi$ be the pushforward distribution of $\theta = T_\phi(z)$. Change of variables gives

$$\log q_\phi(T_\phi(z)) = \log r(z) - \log|\det J_T(z)|. \tag{49.28}$$

The normalized posterior is $\pi_\Theta(\theta) = \widetilde{\pi}_\Theta(\theta)/Z$. Thus

$$\mathrm{KL}(q_\phi \| \pi_\Theta) = \mathbb{E}_{z\sim r}\left[\log r(z) - \log|\det J_T(z)| - \ell_\Theta(T_\phi(z))\right] + \log Z. \tag{49.29}$$

Neither $\log r(z)$ nor $\log Z$ depends on $\phi$, so the optimized loss is

$$\mathcal{L}(\phi) = -\mathbb{E}_{z\sim r}\left[\ell_\Theta(T_\phi(z)) + \log|\det J_T(z)|\right]. \tag{49.30}$$

For a batch $z^{(1:B)}$, the unbiased Monte Carlo estimator is the corresponding sample mean. Reparameterization yields

$$\nabla_\phi \mathcal{L} = -\mathbb{E}_r\left[J_{T,\phi}(z)^\top \nabla_\theta \ell_\Theta(T_\phi(z)) + \nabla_\phi \log|\det J_T(z)|\right]. \tag{49.31}$$

This is differentiated through the same declared target used later by HMC.

Reverse KL penalizes mass placed where the posterior density is low but may underrepresent separated modes or tails. Consequently, heldout loss can select a training recipe, and nonfinite loss or gradients can veto training, but loss cannot establish posterior convergence. HMC corrects density error only if finite computation actually explores all relevant latent regions.

BayesFilter uses stateless Gaussian batches, one leading-batch target call, float64 TensorFlow/TFP, XLA, GPU memory growth, and Adam with bias correction. Gradient clipping and learning-rate schedules are named-procedure choices: some procedures use global-norm clipping and linear or piecewise decay, while the SSL–LSTM capacity families use per-variable clipping and an explicitly controlled constant learning rate. The run manifest must identify the exact choice; neither policy is a universal NeuTra default. No NumPy or Python sample-axis loop belongs to the active training/HMC route. A valid segmented resume restores weights, both Adam moments, completed step, seed schedule, and learning-rate schedule.

## 49.6 When transport training learns scale but little nonlinear shape

The September 2026 q20 investigation asks why reverse-KL training can produce an almost affine map while substantial nonlinear posterior geometry remains. The target is

the four-parameter, thirty-observation UKF approximate posterior of the SSL–LSTM example. The "q20" designation refers to its latent-state configuration, not a twenty-dimensional parameter vector. All statements in this section concern that specified approximate posterior; accurate sampling from it would not establish agreement with the exact state-space posterior. The earlier three-stage examples in this chapter and the two/four-stage q20 recipes are distinct experiments.

### 49.6.1 What failed, and what has already been checked

Let $z \sim r = \mathcal{N}(0, I_4)$, let $\theta = T_\phi(z)$, and define the unnormalized transformed log density and its Gaussian-score residual as

$$\ell_Z(z) = \ell_\Theta(T_\phi(z)) + \log|\det J_T(z)|, \qquad g(z) = \nabla_z \ell_Z(z) + z. \tag{49.32}$$

An exact Gaussianizing map has $g = 0$. A useful HMC map need not make this identity exact, so the residual explains geometry and can nominate a repair; posterior estimation still requires the sampling checks described elsewhere in this chapter.

On the saved bank of 1,000 base-normal points, affine regressions explain 99.866–99.999% of the coordinate variation of the depth-four map. Of the observed squared residual $\sum_n \|g(z^{(n)})\|^2$, 89.02% lies in latent coordinate 2. A constant-plus-linear regression on all four latent coordinates explains only 0.573% of that component's residual energy; its root-mean-square magnitude is 4.2638. That latent direction principally controls the physical observation weight, approximately $\theta_3 = 0.2743 + 0.4958 z_2$ on this bank. These descriptive observations localize a failure to learn nonlinear shape. They are not a bound on derivatives of the fitted map, proof of separability, or evidence that the target has multiple modes.

The corresponding source audit checked the loss

$$-\mathbb{E}_r[\ell_\Theta(T_\phi(z)) + \log|\det J_T(z)|] \tag{49.33}$$

and verified that its first parameter derivative includes both the supplied analytic target score and the log determinant. The previously checked finite differences do not support a missing-score explanation. The reported loss near 43 is not a KL divergence of 43: the additive terms $\mathbb{E}_r \log r + \log Z$ are not included in that number.

Clipping alone also cannot explain the remaining failure. The continuation control clipped 99.02% of its updates, whereas the clipping repair clipped 0.68% and the depth-four continuation clipped none. Each principal repair added 1,024 updates with batch 32. The added layers consequently have only 1,024 updates, and the existing layers have 2,048 lifetime updates. The late depth-four losses are flat and noisy; improvement from parent to endpoint does not establish continuing improvement at the endpoint or optimizer convergence.

These results and their frozen-source identities are preserved in the September 23 mathematical audit. They motivate three distinct mechanisms: release overall scale from a saturating parameterization, reduce noise in the same reverse-KL gradient, and expose direct scalar shape changes. The observations do not yet determine which mechanism matters most.

### 49.6.2 Why reverse-KL stationarity need not remove the score residual

The objective averages log density, whereas $g$ measures its derivative. Even an exact optimum within an affine family can retain nonlinear score error. Consider $p(x) \propto \exp(-x^4/12)$ and $x = m + \sigma z$. The nonconstant reverse-KL objective is

$$L(m, \sigma) = \frac{m^4 + 6m^2\sigma^2 + 3\sigma^4}{12} - \log \sigma. \tag{49.34}$$

For fixed positive $\sigma$, its minimum in $m$ is zero. At $m = 0$, $\partial_\sigma L = \sigma^3 - \sigma^{-1}$, so the optimum is $\sigma = 1$. Nevertheless $g(z) = z - z^3/3$, and the Gaussian moments give

$$\mathbb{E}[g] = \mathbb{E}[zg] = 0, \qquad \mathbb{E}[g^2] = 1 - \tfrac{2}{3}(3) + \tfrac{1}{9}(15) = \tfrac{2}{3}. \tag{49.35}$$

Thus affine directions can be stationary while nonlinear error remains. This is an analytic example, not a fitted description of the q20 posterior.

Adding $\mathbb{E}\|g\|^2$ to the objective would change the optimization problem. Its parameter derivative generally requires derivatives of the target score, which the existing stopped-score bridge does not supply. The following path-gradient mechanism instead preserves reverse KL and needs only the first target score.

### 49.6.3 Mechanism one: separate global scale from conditional deformation

The q20 conditional log scale is $s(a) = 2\tanh(a/2)$. At the observed mean scale near $-1.9936$,

$$\frac{ds}{da} = 1 - \left(\frac{s}{2}\right)^2 \simeq 0.0064. \tag{49.36}$$

This is a measured local attenuation. It does not prove the composed map cannot contract: other stages can compensate, and Adam's update is not proportional to this derivative alone. Still, the fixed outer scale of four requires contraction somewhere, so making every global scale change pass through the cap is an avoidable optimization constraint.

One parameterization supported by the original NeuTra author implementation is

$$s_j(x_{<j}) = b_j + c\tanh\left(\frac{h_j(x_{<j})}{c}\right), \qquad \frac{\partial s_j}{\partial b_j} = 1. \tag{49.37}$$

The bounded term controls input-dependent variation and the free bias controls global contraction. A final cap on their sum would restore the obstruction. The inspected `neutra/utils.py` implementation applies its optional `log_scale_clip_pre` before adding `log_scale_bias`; this is an available source implementation, not a claim that every published experiment used that option. Andrade's stable-flow construction similarly separates conditional clamps from a final trainable affine transformation [Andrade, 2024, Sections 3–5].

The smallest map-preserving implementation is a separate affine correction

$$A_\eta(x) = m + \operatorname{diag}(e^b)x, \qquad \log|\det J_A| = \sum_j b_j, \qquad m = b = 0 \text{ at initialization.} \tag{49.38}$$

Then $A_\eta \circ T_0 = T_0$ initially. This adds unconstrained global scale without rewriting the old checkpoint or optimizer slots. Its finite-state checks must reject overflow rather than clip the posterior or omit offending draws. It can improve scale conditioning; by itself it does not supply the missing nonlinear scalar shape.

### 49.6.4 Mechanism two: an exact alternative reverse-KL gradient

Write $s_q(\theta) = \nabla_\theta \log q_\phi(\theta)$ and $s_\pi(\theta) = \nabla_\theta \ell_\Theta(\theta)$. With a differentiable full-support map and integrability sufficient to interchange differentiation and integration, the chain rule gives

$$\nabla_\phi \mathrm{KL}(q_\phi \| \pi_\Theta) = \mathbb{E}_r\big[(\partial_\phi T)^\top (s_q - s_\pi)\big] + \mathbb{E}_q\big[\partial_\phi \log q_\phi(\theta)|_\theta\big], \tag{49.39}$$

$$\mathbb{E}_q\big[\partial_\phi \log q_\phi(\theta)|_\theta\big] = \partial_\phi \int q_\phi(\theta)\, d\theta = 0. \tag{49.40}$$

The path estimator retains the first expectation. At $q = \pi$ it vanishes for each draw, whereas the ordinary estimator can remain noisy. Removing a zero-mean term does not guarantee lower variance away from the optimum, because covariance between terms also matters [Roeder et al., 2017, Section 2]. A finite minibatch therefore need not give equal standard and path gradients. Their expectations, rather than their pointwise values, must be checked against the same objective.

The proposal score can be obtained without numerical inversion. Differentiate the change-of-variables identity at a known base draw:

$$\log q_\phi(T_\phi(z)) = \log r(z) - \log|\det J_T(z)|, \tag{49.41}$$

$$J_T(z)^\top s_q(T_\phi(z)) = -z - \nabla_z \log|\det J_T(z)|. \tag{49.42}$$

Solving this transposed-Jacobian system supplies $s_q$. Applied one layer at a time it is the score recursion of Vaitl et al. [2024, Proposition 3.2 and Appendix B.1]. Their optimized coupling-flow timing results do not establish the cost of our autoregressive implementation. The first optional implementation uses a bounded small dense Jacobian assembled by coordinate-wise vector-Jacobian products. That loop is over the four parameter coordinates, not training samples; all sample rows remain batched. It uses no TensorFlow pfor and makes no coupling-flow speed claim.

After computing $v = s_q - s_\pi$, hold $v$ fixed and differentiate $B^{-1}\sum_n T_\phi(z^{(n)})^\top v^{(n)}$ with respect to $\phi$. This scalar carries the intended gradient but is not the reverse-KL loss; the actual $-\ell_\Theta - \log|\det J_T|$ must be reported separately. Stopping the complete forward log determinant would discard necessary terms. The inverse-density construction of Vaitl et al. [2022] and the forward recursion above are two ways to compute the required proposal score correctly.

Combining (49.32) and (49.42) yields

$$J_T^\top (s_q - s_\pi) = -g, \qquad \nabla_\phi \mathrm{KL} = -\mathbb{E}_r\big[(J_T^{-1}\partial_\phi T)^\top g\big]. \tag{49.43}$$

If available parameter variations have little projection onto $g$, the expected gradient can be small despite large nonlinear error. Variance reduction helps resolve an existing direction; it does not create a missing one.

### 49.6.5 Mechanism three: direct scalar deformation with an inexpensive log determinant

Huang et al. [2018, Sections 2–3 and 6.1.2] identify poor fits and local minima with conditional affine flows and replace the scalar affine function by a monotone neural transformation. For a deep sigmoid flow (DSF), let

$$S(x) = \sum_{k=1}^{K} w_k \sigma(a_k x + b_k), \quad \tau(x) = \log S(x) - \log(1 - S(x)),$$
$$a_k > 0, \quad w_k > 0, \quad \sum_k w_k = 1. \tag{49.44}$$

Here $\sigma$ is logistic. Positivity makes the derivative strictly positive; the limits are $\tau(x) \to \pm\infty$ as $x \to \pm\infty$. For $K = 1$, $\tau(x) = a_1 x + b_1$, giving an exact affine control.

Let $p_k = \sigma(a_k x + b_k)$ and $N = \sum_k w_k a_k p_k (1 - p_k)$. Direct differentiation gives

$$\tau'(x) = \frac{N}{S(1 - S)}, \qquad \log \tau'(x) = \log N - \log S - \log(1 - S). \tag{49.45}$$

Thus there is a fast analytic determinant formula. When the scalar parameters depend only on preceding coordinates, the full autoregressive Jacobian remains triangular and $\log |\det J| = \sum_j \log \tau'_j$. A masked conditioner produces all coordinate parameters in one pass. A single DSF scalar layer costs $O(BdK)$ beyond the conditioner; a deep dense scalar network also incurs its hidden matrix-product costs. These operation counts are not measured GPU timings.

Directly forming $1 - S$ loses precision in the tail. With $u_k = a_k x + b_k$, calculate instead

$$A = \mathrm{LSE}_k(\log w_k + \log \sigma(u_k)), \tag{49.46}$$
$$B_0 = \mathrm{LSE}_k(\log w_k + \log \sigma(-u_k)), \tag{49.47}$$
$$C = \mathrm{LSE}_k(\log w_k + \log a_k + \log \sigma(u_k) + \log \sigma(-u_k)), \tag{49.48}$$
$$\tau = A - B_0, \qquad \log \tau' = C - A - B_0. \tag{49.49}$$

Here $\mathrm{LSE}(v) = \log \sum_k \exp v_k$ is evaluated stably. This algebra is consistent with the log-domain approach in the paper's Appendix C. The inspected author code additionally shrinks $S$ to $(1 - \delta)S + \delta/2$ for numerical stability. That yields finite output limits and differs from the full-range map required here; the proposed implementation uses (49.49) without that shrinkage.

HMC and geometry diagnostics need the derivative of the log determinant too. Let $\rho_k = w_k a_k p_k (1 - p_k)/N$. Then

$$\frac{\tau''}{\tau'} = \sum_k \rho_k a_k (1 - 2p_k) - \exp(C - A) + \exp(C - B_0). \tag{49.50}$$

The first term is $N'/N$, and the last two differentiate $-\log S$ and $-\log(1 - S)$. This remains a batched scalar reduction. For a conditional map, derivatives through the conditioner also contribute; the scalar formula alone is not the entire multidimensional score.

NAF's significant cost distinction is its inverse. Given $y = \tau(x)$, each component logit crosses $y$ at $r_k = (y - b_k)/a_k$. Therefore

$$\min_k r_k \;\le\; \tau^{-1}(y) \;\le\; \max_k r_k. \tag{49.51}$$

At the lower bound every sigmoid is at most $\sigma(y)$; at the upper bound every sigmoid is at least $\sigma(y)$, proving the bracket. A bounded bisection can solve inside it, with explicit input/output tolerances, an iteration cap, and a failure result. Training in the forward reverse-KL direction and ordinary latent HMC steps do not require this inverse on every evaluation. Physical-space proposal densities, forward-KL training, and ensemble cross densities can do so. Fast forward determinants consequently do not imply a cheap complete ensemble pipeline. The first implementation does not differentiate through bisection or offer inverse-based training.

There is also an initialization trap. At $a_k = 1, b_k = 0$ for all components, with $w = \mathrm{softmax}(\eta)$,

$$\partial_{a_k}\tau = w_k x, \qquad \partial_{b_k}\tau = w_k, \qquad \partial_{\eta_k}\tau = 0. \tag{49.52}$$

The scalar parameter directions are initially only affine. Small distinct components expose nonlinear directions, but their initial map displacement must be measured. Zero output initialization is not generally a defect: successful NAF and Glow recipes start near identity [Huang et al., 2018, Appendix E][Kingma and Dhariwal, 2018, Section 3.3]. The question is whether the chosen features and parameter directions can respond usefully on this target.

The smallest shape experiment freezes the current map and uses $T_\psi = T_0 \circ D_\psi$, where $D_\psi(z) = (z_1, \tau_\psi(z_2), z_3, z_4)$. It includes $\log \tau'_\psi(z_2)$ in the loss and keeps the posterior unchanged. The observed concentration of error motivates coordinate 2, but does not guarantee a marginal correction will suffice. Residual dependence on other coordinates would motivate a conditional transform next.

Explicit inverses are available for rational-quadratic splines, along with inexpensive determinants [Durkan et al., 2019, Section 3]. Their usual construction is $C^1$ but generally not $C^2$. Since the scalar transformed force contains $\tau''/\tau'$, knot behavior requires a separate HMC energy and derivative check. They remain an alternative, not an automatically qualified replacement.

### 49.6.6 How the three mechanisms will be distinguished

First test the arithmetic on analytic targets: affine identities, Gaussian pointwise cancellation of the path gradient, equality of expected standard and path gradients under converged quadrature, finite differences of scalar values and log determinants, and inverse roundtrips including tails. Checkpoint reconstruction must preserve all mechanism parameters and never imply HMC admission. Tiny CPU fixtures with hidden GPUs answer these engineering questions only. A few successful optimizer steps are not training evidence for the q20 posterior.

The target experiment then starts from the same saved endpoint for isolated scale, gradient-estimator, and scalar-shape arms, with unchanged reverse KL. An affine-only scalar control distinguishes shape from location and scale. Pair the latent streams where meaningful and charge actual worker time; equal target-row counts need not imply equal optimizer steps or equal runtime. Keep true losses, per-block gradient/update measurements, scale derivatives, inverse work where used, and checkpoints. A failed candidate triggers the corresponding next repair; it does not alone reject NeuTra.

Every numerical choice needs a local basis. The coordinate follows the saved residual decomposition. The minimum nonlinear component count follows from $K = 1$ being affine, but sufficient capacity is uncalibrated. Positive slopes and weights follow from invertibility. Initial component spread is judged by map displacement and nonlinear

response. Batch, learning rate, continuation length, replication, and any revised conditional cap must be priced and calibrated on the actual target. Published values such as the original NeuTra batch 4,096 and 5,000 updates are experimental settings, not q20 defaults.

Fresh heldout objective estimates and the standard 1,000-point score report can support nomination. They cannot establish mode coverage, convergence, or superiority among stochastic candidates without the corresponding evidence. A new scalar family also requires supported frozen-map reconstruction, transformed-score checks, and fresh public-tuner qualification before HMC. The implementation and experiment sequence is recorded in the three-mechanism repair plan. The strongest competing explanation remains that the current IAF family could learn adequate nonlinear shape under a better calibrated continuation. The three tests are designed to distinguish that explanation from limited effective parameter directions, rather than assume an architecture replacement is necessary.

### 49.6.7 A shared implementation of the proposed mechanisms

The September 24 implementation makes the affine autoregressive map, its inverse, and its score derivatives common to ordinary, weighted, legacy, and frozen-map consumers. Historical checkpoints retain their original scale conventions. A separate source-based IAF configuration implements the NeuTra author code's conditional cap with a free bias at every layer, together with the paper's three-stage, two-hidden-layer ELU architecture. The fixed cap is not a bound on the free global scale. The source initializer uses fan-in variance scaling with factor 0.02, as selected by the author's `MakeIAFBijectorFn`; neither that factor nor the paper's training schedule establishes a calibrated q20 protocol.

The new DSF configuration is a full conditional neural autoregressive flow: the earlier coordinates determine each coordinate's positive slopes, shifts, and mixture weights. Its source profile retains the author's cMADE conditioner, final conditioner weight normalization, and shared output projection, specialized to a constant context for an unconditional target. The log-domain transformer preserves the paper's full real support. It omits the author implementation's output shrinkage, which would bound that support. This numerical distinction and the fixed random-number conventions are documented explicitly in the implementation reference.

Layerwise proposal-score propagation is connected to the Adam training update. It uses Vaitl's general score recursion and autoregressive triangular solves, not the coupling-specific linear-cost formula. Inverse-density gradients use implicit differentiation of the same conditional map. Training handoff includes the standard 1,000-point residual report; a finite report alone establishes neither adequate whitening nor posterior convergence. These changes close implementation gaps. Their numerical regression and GPU checks are engineering evidence; a freshly calibrated q20 training run and downstream sampler validation are still required.

## 49.7 Geometry induced in source space

NeuTra may also be understood as a learned position-dependent metric. With identity latent mass, its Hamiltonian is

$$H_Z(z,p) = -\ell_\Theta(T(z)) - \log|J(z)| + \tfrac{1}{2}p^\top p. \tag{49.53}$$

Let $J = J_T(z)$ and define source momentum $p_\theta = J^{-\top} p$. Then $p = J^\top p_\theta$ and

$$\tfrac{1}{2} p^\top p = \tfrac{1}{2} p_\theta^\top J J^\top p_\theta, \tag{49.54}$$

$$-\log |J| = \tfrac{1}{2} \log |(JJ^\top)^{-1}|. \tag{49.55}$$

Hence the same scalar Hamiltonian in $\theta$ has metric

$$G(\theta) = (J(z)J(z)^\top)^{-1}, \tag{49.56}$$

which explains how a nonlinear map adapts effective scale and orientation with position. Unlike general Riemannian HMC, computation remains an explicit, separable Euclidean HMC problem in $z$. The benefit exists only where the learned $J$ approximates useful posterior geometry.

## 49.8 Complete BayesFilter algorithm

For a valid batched target, the operational algorithm is:

1. Define the physical model, prior, source chart, likelihood or filter-defined approximate likelihood, scalar log target, score, and target health telemetry. Verify value/score and batch identity before training.

2. Choose a fixed affine lift $(c, L)$ and a small, target-specific candidate set of IAF widths and learning rates. Train each screen candidate with (49.30); compare on common heldout latent noise.

3. Train a fresh final transport from independent initialization using the nominated recipe. Freeze and serialize its exact component order, weights, biases, masks, affine lift, target signature, and training-state hash.

4. Construct (49.18) from the frozen map. Check trainable/frozen parity, finite values, scores, and target telemetry.

5. Screen a broad set of fixed HMC pairs $(\epsilon, L)$ with disjoint seeds through the shared candidate controller. Acceptance evidence and numerical health qualify each exact pair; retain every verified member. Short-chain ESS and modern R-hat are reporting-only during kernel tuning.

6. Retain warm-up chunks separately until the warm-up gate passes or the cap is reached. Never pool warm-up into posterior summaries.

7. Accumulate fresh retained chunks until the joint R-hat, ESS, and health gates pass or the retained cap is reached.

8. Transform retained $z$ draws to source and physical coordinates and apply the predeclared scientific diagnostic. Archive all three coordinates, seeds, settings, telemetry, hashes, and wall time.

## 49.9 Selecting among frozen transports

Training restarts are candidate transports, not automatically permanent estimation arms. After each transport has been frozen and independently bound to the same target, prepare one immutable scope and run the shared candidate-set controller described in Section 42.3. BayesFilter owns the fixed-kernel validation rungs, candidate-local rejection, generic hard screens, and retention of every viable transport. The client owns training receipts, transport provenance, and the declared score used to nominate among viable candidates. A separate frozen transport is a separate scope; its states and verification receipts never enter another scope's queue.

The recommended sequence is: freeze the transport and scope; measure the broad fixed-kernel grid; validate every retained pair on fresh, disjoint seeds; create same-$(L)$ epsilon children only from typed directional evidence; verify each child independently; retain all candidates that pass; and nominate one only after recording the score ordering and its uncertainty status. A low reverse-KL or good whitening receipt can nominate a candidate, but it does not prove global Gaussianity, posterior correctness, or sampler superiority. R-hat belongs to separate posterior assessment and cannot reject, rank or delay tuning members. An inconclusive result receives a declared evidence extension or an explicit inconclusive disposition at its cap; it does not silently replace $L$.

Do not rank restarts solely by training loss, silently discard viable candidates, or treat a p-value as a universal quality score. The candidate receipt must say whether the ordering is descriptive or statistically supported and must preserve the hard vetoes for every rejected candidate.

## 49.10 Convergence and scientific diagnostics

For $M$ chains of length $N$, each chain is split in half, producing $2M$ sequences of length $n = N/2$. Pool the draws, replace ranks $r$ by normal scores

$$z = \Phi^{-1}\left(\frac{r - 3/8}{S + 1/4}\right), \tag{49.57}$$

where $S = MN$. If $\bar{z}_m$ and $s_m^2$ are split-chain means and variances,

$$W = \frac{1}{2M}\sum_m s_m^2, \tag{49.58}$$

$$B = \frac{n}{2M-1}\sum_m (\bar{z}_m - \bar{z})^2, \tag{49.59}$$

$$\widehat{V} = \frac{n-1}{n}W + \frac{1}{n}B, \qquad \widehat{R}_{\text{rank}} = \sqrt{\widehat{V}/W}. \tag{49.60}$$

Folded R-hat repeats the calculation for $|x - \text{median}(x)|$, detecting scale discrepancies. BayesFilter reports

$$\widehat{R}_{\text{modern,j}} = \max\{\widehat{R}_{\text{rank,j}}, \widehat{R}_{\text{folded,j}}\}. \tag{49.61}$$

Thus folded R-hat is required, but it is not the sole reported R-hat.

For an estimand with autocorrelations $\rho_t$, the conceptual effective sample size is

$$N_{\text{eff}} = \frac{MN}{1 + 2\sum_{t\geq 1}\rho_t}, \tag{49.62}$$

The preserved-transition diagnostic API uses biased lag autocovariances, unbiased within-chain variance and Geyer's initial positive and monotone sequences, including the final even-lag improvement used by Stan `posterior` and ArviZ. Its estimated integrated autocorrelation time is bounded below by $1/\log_{10}(S)$ for $S$ input draws. This finite-sample safeguard prevents unstable negative ESS estimates; it is not a theoretical upper bound on an antithetic chain's true ESS. Constant pooled samples remain non-promotable.

Bulk ESS uses split, rank-normalized draws. Tail ESS is the minimum of the split-chain ESS values for the quantile-CDF indicators $I(x \le q_{0.05})$ and $I(x \le q_{0.95})$. The latter complements $I(x > q_{0.95})$, whereas an inclusive upper-tail indicator differs when rejected proposals repeat a cutoff value. Linear percentile interpolation preserves equal endpoints exactly. These are finite-run diagnostics, not proofs of convergence.

The preserved-transition API retains its unsplit-chain convention for pooled posterior-mean ESS and MCSE, $\mathrm{SD}(x)/\sqrt{N_{\mathrm{eff,mean}}}$. Its per-chain MCSE uses the same estimator with one chain. ArviZ's public mean-ESS diagnostic instead splits chains, so a reference comparison must state which convention it checks. Bulk/tail diagnostics still use split chains. This reporting correction does not change HMC transitions or retroactively alter archived diagnostic reports.

The public `rank_normalized_hmc_diagnostics` entry point uses these same bulk/-tail and split/folded R-hat implementations. Its inputs retain the order draw, chain, parameter; the preserved-transition API uses chain, draw, parameter. Thresholds and output schemas remain unchanged. The separately named TFP mean/quantile precision option and the temporal-ESS heuristic used during covariance adaptation have their own estimator conventions; this reporting repair does not change them.

The historical case-study protocol below used four chains, warm-up modern R-hat at most 1.05, retained modern R-hat at most 1.01, minimum bulk ESS 1000, minimum tail ESS 400, and caps of 10,000 warm-up and retained draws per chain. The value 1000 is a conservative convenience rule, not a theorem. The structural result below missed it narrowly and is labeled with the disclosed post-result owner adjudication rather than called an automated pass.

New posterior runs declare ESS and precision requirements using the assessment policy in Chapter 47. The rank and R-hat calculations have version `bayesfilter.hmc_diagnostic_math.v2`; bulk and tail ESS use `bayesfilter.stan_initial_positive_monotone_ess.v1`. Historical numerical reports below have not been recomputed with these estimators and cannot establish current threshold passage.

TFP's fixed HamiltonianMonteCarlo kernel did not expose native divergence flags for these runs. The preserved hard screen defines an energy failure as a nonfinite log acceptance ratio or one below $-1000$. This narrower telemetry is reported honestly and is not renamed as a native divergence diagnostic.

For synthetic data with known truth $\vartheta_j^*$ and $S$ pooled retained draws, define

$$\widehat{F}_j(\vartheta_j^*) = \frac{n_{<,j} + \frac{1}{2}n_{=,j} + \frac{1}{2}}{S+1}, \qquad p_{\mathrm{truth},j} = 2\min\{\widehat{F}_j, 1-\widehat{F}_j\}. \tag{49.63}$$

This posterior-tail diagnostic is not a frequentist p-value. The cost-bounded rule runs one dataset seed first: pass if every value is at least 0.05; repeat one fresh seed for a marginal $[0.003, 0.05)$ value; classify and investigate immediately below 0.003. A sampler veto makes truth recovery inconclusive. Truth at the prior mean is deliberately favorable; noncentral fixtures are labeled.

## 49.11 Filter-defined posterior targets used below

The nonlinear experiments do not use an unspecified "UKF likelihood" or "SGQF likelihood." This section defines the scalar approximation inserted into (49.3).

Suppose a filtering rule represents $x_{t-1} \mid y_{1:t-1}, \vartheta$ by mean $m_{t-1}$ and covariance $P_{t-1}$. Let standardized points $\xi_r$ and weights $w_r^{(m)}, w_r^{(c)}$ be fixed. Given a square-root factor $S$ with $SS^\top = P_{t-1}$, place $x_{t-1}^{(r)} = m_{t-1} + S\xi_r$. For an additive-noise transition $f_\vartheta$ and observation $h_\vartheta$, form

$$\widetilde{x}_t^{(r)} = f_\vartheta(x_{t-1}^{(r)}), \tag{49.64}$$

$$m_t^- = \sum_r w_r^{(m)} \widetilde{x}_t^{(r)}, \tag{49.65}$$

$$P_t^- = Q_t + \sum_r w_r^{(c)} (\widetilde{x}_t^{(r)} - m_t^-)(\widetilde{x}_t^{(r)} - m_t^-)^\top. \tag{49.66}$$

When process noise enters nonadditively, the rule is placed on the block Gaussian $(x_{t-1}, \varepsilon_t)$ and the transition consumes both parts; this is the structural-UKF route used for (49.96). No artificial variance is then added to deterministic completion coordinates.

Refactor $P_t^- = S_t^- (S_t^-)^\top$, place $\widehat{x}_t^{(r)} = m_t^- + S_t^- \xi_r$, and set

$$y_t^{(r)} = h_\vartheta(\widehat{x}_t^{(r)}), \tag{49.67}$$

$$\mu_t = \sum_r w_r^{(m)} y_t^{(r)}, \tag{49.68}$$

$$F_t = R_t + \sum_r w_r^{(c)} (y_t^{(r)} - \mu_t)(y_t^{(r)} - \mu_t)^\top, \tag{49.69}$$

$$C_t = \sum_r w_r^{(c)} (\widehat{x}_t^{(r)} - m_t^-)(y_t^{(r)} - \mu_t)^\top, \tag{49.70}$$

$$K_t = C_t F_t^{-1}, \qquad m_t = m_t^- + K_t(y_t - \mu_t), \tag{49.71}$$

$$P_t = P_t^- - K_t F_t K_t^\top. \tag{49.72}$$

The filter-defined log likelihood is the Gaussian prediction-error scalar

$$\ell_{\mathrm{filt}}(\vartheta) = -\frac{1}{2} \sum_{t=1}^{T} \left[ d_y \log(2\pi) + \log |F_t| + v_t^\top F_t^{-1} v_t \right], \quad v_t = y_t - \mu_t. \tag{49.73}$$

For any scalar source coordinate $a$, differentiation of one increment gives

$$\partial_a \ell_t = -\frac{1}{2} \left[ \mathrm{tr}(F_t^{-1} \partial_a F_t) + 2(\partial_a v_t)^\top F_t^{-1} v_t - v_t^\top F_t^{-1} (\partial_a F_t) F_t^{-1} v_t \right]. \tag{49.74}$$

Propagating derivatives through (49.66)–(49.72), including the square-root derivative, supplies the score of the same scalar (49.73). BayesFilter uses this checked manual or custom-gradient recursion rather than a different force.

For the principal-square-root UKF, in dimension $d$ the unscented rule is

$$\lambda = \alpha^2(d + \kappa) - d, \tag{49.75}$$

$$\xi_0 = 0, \quad \xi_i = \sqrt{d + \lambda}\, e_i, \quad \xi_{i+d} = -\sqrt{d + \lambda}\, e_i, \tag{49.76}$$

$$w_0^{(m)} = \frac{\lambda}{d + \lambda}, \quad w_0^{(c)} = w_0^{(m)} + 1 - \alpha^2 + \beta, \tag{49.77}$$

$$w_i^{(m)} = w_i^{(c)} = \frac{1}{2(d + \lambda)} \quad (i > 0). \tag{49.78}$$

The tested rule fixes $(\alpha, \beta, \kappa) = (1, 2, 0)$ and uses the unique symmetric positive-definite principal square root. Here $\lambda = 0$, so the mean center weight is zero and the covariance center weight is two.

For the fixed level-2 SGQF in dimension $d$, the merged Smolyak/Gauss–Hermite cloud has the closed form

$$\xi_0 = 0, \quad w_0 = 1 - \frac{d}{3}, \qquad \xi_{j,\pm} = \pm\sqrt{3}e_j, \quad w_{j,\pm} = \frac{1}{6}, \quad j = 1, \dots, d. \tag{49.79}$$

The normalization follows from

$$w_0 + \sum_{j=1}^{d}(w_{j,+} + w_{j,-}) = 1 - \frac{d}{3} + 2d\frac{1}{6} = 1. \tag{49.80}$$

Symmetry makes the weighted mean zero. Off-diagonal second moments vanish, and coordinate $j$ has second moment

$$\frac{1}{6}(\sqrt{3})^2 + \frac{1}{6}(-\sqrt{3})^2 = 1, \tag{49.81}$$

so the weighted covariance is $I_d$. Predator–prey has $d = 2$, hence five points and center weight $1/3$. SIR has $d = 18$, hence 37 points and center weight $-5$; that negative weight is part of the declared rule and is retained. Both filters use (49.66)–(49.73); only their fixed point clouds and square-root branches differ.

All bounded-uniform physical priors below use the explicit probit chart

$$\vartheta_i = l_i + (u_i - l_i)\Phi(\theta_i), \tag{49.82}$$

$$\frac{d\vartheta_i}{d\theta_i} = (u_i - l_i)\phi(\theta_i), \tag{49.83}$$

$$\log p(\vartheta_i) + \log\left|\frac{d\vartheta_i}{d\theta_i}\right| = \log\phi(\theta_i), \qquad \partial_{\theta_i}\log\left[p(\vartheta_i)\left|\frac{d\vartheta_i}{d\theta_i}\right|\right] = -\theta_i. \tag{49.84}$$

The interval width cancels the uniform density exactly. These chart terms are part of the source target before the NeuTra Jacobian is added.

## 49.12 Models in the common truth-tail experiment

Five retained archives are comparable under (49.63). Each has four chains and 4000 retained draws per chain, hence 16,000 pooled draws. Warm-up is separate.

### 49.12.1 Four-dimensional exact LGSSM

The model is

$$x_t = Ax_{t-1} + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q), \qquad y_t = x_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, R), \tag{49.85}$$

with $T = 120$, stationary Gaussian initialization, diagonal $Q, R$, and lower triangular

$$A = \begin{pmatrix} .62 & 0 & 0 & 0 \\ .18 & .48 & 0 & 0 \\ -.10 & .14 & .30 & 0 \\ .06 & -.08 & .11 & .16 \end{pmatrix}. \tag{49.86}$$

The process standard deviations are $(.30, .26, .22, .18)$ and observation standard deviations $(.12, .11, .10, .09)$. The 18 inferred quantities are ten free elements of $A$ and eight standard deviations. The likelihood is the exact Kalman prediction-error likelihood. In raw coordinates, the prior is independent Gaussian and centered at the frozen truth, with standard deviation .5 for the four diagonal transition coordinates, .6 for the six strict lower-triangular coordinates, and .35 for each log process or observation standard deviation. Warm-up/retained counts were 2000/4000 per chain; maximum modern R-hat was 1.00215, minimum bulk/tail ESS 4571.6/3977.0, and the energy-error count was zero.

### 49.12.2 Predator–prey with UKF and SGQF likelihoods

For prey $X$ and predator $Y$, the continuous dynamics are

$$\dot{X} = rX(1 - X/K) - s\frac{XY}{a + X}, \tag{49.87}$$

$$\dot{Y} = u\frac{XY}{a + X} - vY. \tag{49.88}$$

They are integrated by RK4 over intervals of length two using step 0.1. The initial state is $\mathcal{N}((50, 5)^\top, I_2)$; process and observation covariance are both $4I_2$; both state coordinates are observed; and $T = 20$. Independent uniform priors use the box

$$r \in [.1, 1.1],\ K \in [110, 130],\ a \in [20, 30],\ s \in [.1, 1.1],\ u, v \in [0, 1], \tag{49.89}$$

implemented by six probit charts with their Jacobians. The generating vector is

$$(r, K, a, s, u, v) = (.6, 114, 25, .3, .5, .5). \tag{49.90}$$

Thus $K$ and $s$ are not at prior means, requiring the noncentral label. The two posterior targets differ only in the declared approximate filter likelihood: principal-square-root UKF versus fixed SGQF. Both used 2000/4000 warm-up/retained draws per chain and zero recorded energy failures. UKF had maximum modern R-hat 1.00081 and minimum bulk/tail ESS 27623.6/13394.1; SGQF had 1.00033 and 26978.5/12974.7.

### 49.12.3 Parameterized spatial SIR with SGQF likelihood

Nine regions have susceptible-infectious pairs $(S_j, I_j)$, giving an 18-dimensional state. With graph adjacency $A_{jk}$ and degree $d_j$, the continuous dynamics are

$$\dot{S}_j = -\kappa S_j I_j + \tfrac{1}{2}\sum_k A_{jk} S_k - \tfrac{1}{2} d_j S_j, \tag{49.91}$$

$$\dot{I}_j = \kappa S_j I_j - \nu I_j + \tfrac{1}{2}\sum_k A_{jk} I_k - \tfrac{1}{2} d_j I_j. \tag{49.92}$$

All nine infectious coordinates are observed. The frozen $T = 20$ fixture uses unit process covariance and base observation covariance $100 I_9$. Three source parameters have independent $\mathcal{N}(0, .5^2)$ priors and scale

$$\kappa = .1e^{\theta_1}, \qquad \nu = 18e^{\theta_2}, \qquad \mathrm{sd}(e) = 10e^{\theta_3}. \tag{49.93}$$

The truth $(.1, 18, 10)$ is therefore prior-centered. The target uses a fixed level-2, 37-point SGQF observed-data likelihood. Warm-up/retained counts were 2000/4000; maximum modern R-hat was 1.00007, minimum bulk/tail ESS 16358.5/14568.5, and the energy-error count was zero.

### 49.12.4 Structural deterministic dynamics with UKF likelihood

The structural model is

$$m_t = \rho m_{t-1} + \sigma\varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0,1), \tag{49.94}$$
$$k_t = \phi k_{t-1} + \gamma m_t^2, \tag{49.95}$$
$$y_t = m_t + k_t + e_t, \quad e_t \sim \mathcal{N}(0,R). \tag{49.96}$$

There is no artificial innovation in $k_t$. The initial state has mean zero and covariance $\mathrm{diag}(.04,.09)$. A principal-square-root structural UKF supplies the approximate likelihood for $T = 100$. Independent uniform priors, written in the reporting order $(\rho,\sigma,\phi,\gamma,R)$, are

$$\rho \in [.05,.98], \quad \sigma \in [.05,1.25], \quad \phi \in [.05,.98], \quad \gamma \in [.02,1], \quad R \in [.02,1], \tag{49.97}$$

again represented by probit charts. The truth $(\rho,\sigma,\phi,\gamma,R) = (.8,.5,.7,.4,.25)$ differs from prior mean $(.515,.65,.515,.51,.51)$ in the same order, so this is noncentral.

The final transport was a fresh 5000-step, three-stage dense IAF with hidden widths $(30,30)$ and learning rate $5\times10^{-3}$, selected from four 500-step screens. Heldout reverse KL was 137.734, versus 195.591 for the identity/source-prior baseline. This is training evidence only. The admitted fixed HMC kernel used step size .05 and 12 leapfrog steps. Verification had zero energy failures and modern R-hat 1.03240 under the historical 1.05 tuning gate. The current procedure reports tuning R-hat without gating on it. Warm-up stopped at 4000 draws per chain when the last 1000-draw window reached 1.04742. Retained maximum rank/folded/modern R-hat were 1.00668/1.00156/1.00668; minimum bulk/tail ESS were 971.06/2354.53. The original convenience gate of bulk ESS 1000 was missed by 2.9%. After seeing the result, the owner accepted it under an explicitly disclosed threshold of 900. The result is therefore qualified and owner-adjudicated, not an automated pass of the original rule.

## 49.13 Complete common experimental results

Table 49.1 gives the configuration-level screen. All 38 generating values lie inside their empirical 95% intervals, and no truth-tail value is below .05, so no second seed was triggered.

| Configuration | $d$ | Max $\widehat{R}$ | Min bulk ESS | Min $p_{\text{truth}}$ | Result |
|---|---|---|---|---|---|
| LGSSM exact | 18 | 1.00215 | 4571.6 | .06781 | central pass |
| Predator–prey UKF | 6 | 1.00081 | 27623.6 | .21255 | noncentral pass |
| Predator–prey SGQF | 6 | 1.00033 | 26978.5 | .21505 | noncentral pass |
| SIR SGQF | 3 | 1.00007 | 16358.5 | .37266 | central pass |
| Structural UKF | 5 | 1.00668 | 971.1 | .28442 | qualified noncentral pass |

Table 49.1: Common one-seed truth-tail experiment; 16,000 retained draws per row.

## 49.14 Additional historical model tests

The broader ledger counts a family once for shared scientific dynamics and a configuration separately when its posterior likelihood or solution/filter construction changes. Seeds,

| Model | Parameter | Truth | Mean | 95% interval | $p_{\text{truth}}$ |
|---|---|---|---|---|---|
| LGSSM | $a_{11}$ | .62 | .5303 | [.3252,.7328] | .4062 |
| | $a_{22}$ | .48 | .5361 | [.3420,.7153] | .5373 |
| | $a_{33}$ | .30 | .2021 | [-.0055,.4320] | .3663 |
| | $a_{44}$ | .16 | .2385 | [.0297,.4646] | .4609 |
| | $a_{21}$ | .18 | .1114 | [-.0444,.2768] | .4285 |
| | $a_{31}$ | -.10 | -.2239 | [-.3237,-.0892] | .0686 |
| | $a_{32}$ | .14 | .2176 | [.0665,.3224] | .2603 |
| | $a_{41}$ | .06 | -.0276 | [-.1697,.1188] | .2159 |
| | $a_{42}$ | -.08 | -.0607 | [-.2088,.0906] | .7969 |
| | $a_{43}$ | .11 | .2200 | [.0689,.3251] | .1409 |
| | $q_1$ | .30 | .2543 | [.1822,.3160] | .1657 |
| | $q_2$ | .26 | .2159 | [.1657,.2632] | .0678 |
| | $q_3$ | .22 | .1985 | [.1337,.2465] | .4345 |
| | $q_4$ | .18 | .1772 | [.1260,.2156] | .9688 |
| | $r_1$ | .12 | .1475 | [.0734,.2279] | .5507 |
| | $r_2$ | .11 | .1089 | [.0572,.1662] | .9464 |
| | $r_3$ | .10 | .1120 | [.0540,.1823] | .7870 |
| | $r_4$ | .09 | .0933 | [.0460,.1546] | .9844 |
| PP–UKF | $r$ | .6 | .5351 | [.4424,.6461] | .2185 |
| | $K$ | 114 | 115.2047 | [113.3208,117.1686] | .2125 |
| | $a$ | 25 | 25.0175 | [20.2462,29.7582] | .9968 |
| | $s$ | .3 | .2859 | [.1053,.7426] | .7156 |
| | $u$ | .5 | .4588 | [.0228,.9628] | .8756 |
| | $v$ | .5 | .5010 | [.0900,.9378] | .9810 |
| PP–SGQF | $r$ | .6 | .5352 | [.4429,.6464] | .2160 |
| | $K$ | 114 | 115.2186 | [113.3183,117.1946] | .2150 |
| | $a$ | 25 | 25.0450 | [20.2774,29.7543] | .9855 |
| | $s$ | .3 | .2861 | [.1057,.7283] | .7183 |
| | $u$ | .5 | .4568 | [.0221,.9595] | .8734 |
| | $v$ | .5 | .4991 | [.0913,.9286] | .9769 |
| SIR–SGQF | $\kappa$ | .1 | .09976 | [.09752,.10202] | .8319 |
| | $\nu$ | 18 | 17.8364 | [17.4812,18.2052] | .3727 |
| | observation sd | 10 | 10.3291 | [9.2950,11.5276] | .5959 |
| Structural | $\rho$ | .8 | .7668 | [.5440,.9281] | .8035 |
| | $\sigma$ | .5 | .5093 | [.3107,.7545] | .9986 |
| | $\phi$ | .7 | .4656 | [.0761,.8132] | .2844 |
| | $\gamma$ | .4 | .5080 | [.1387,.9413] | .7160 |
| | $R$ | .25 | .1962 | [.0357,.4092] | .5505 |

Table 49.2: All parameter-level common truth-tail results in physical coordinates.

dimensions, transports, and tuning candidates do not inflate the count. Including the five configurations above, learned NeuTra reached transformed HMC in ten families and thirteen configurations. The following eight configurations used their original protocols and must not be interpreted as common truth-tail tests.

The historical terminology is fixed as follows. "Second-moment R-hat" and "second-moment ESS" apply the run's original convergence estimators to the coordinate-wise estimands $x_j^2$; they were intended to emphasize scale and tail exploration and are not the modern rank/folded diagnostics defined above. A "quality pass" in the local paper-suite required finite selected chains and median-over-seed second-moment R-hat below the threshold declared for that run. The synthetic dimension-100 ladder used two seeds, four chains, 2048 retained draws per chain, threshold 2.0, and a retained-draw budget ratio 0.001. The ratio is

$$\frac{(\text{local chains})(\text{local retained draws per chain})}{16384 \times 500}, \tag{49.98}$$

because the source paper ran 1000 transitions and discarded the first 500. Thus "paper-scale" below means the same target family and dimension, not the same sampling budget. A historical configuration is called "clean/strong" only when its own declared finite, sampler, and completion screens passed; it is "qualified/marginal" when a named budget, convergence, telemetry, or parent-artifact condition remained open.

**Neal funnel, dimension 100.**

The exact unnormalized density was

$$\log \widetilde{\pi}(y, x) = -\tfrac{1}{2}y^2 - \tfrac{1}{2}e^{-2y}\sum_{j=1}^{99} x_j^2 - 99y, \tag{49.99}$$

equivalently $y \sim \mathcal{N}(0, 1)$ and $x_j \mid y \sim \mathcal{N}(0, e^{2y})$. It was tested with diagonal, triangular-affine, and three-stage dense-IAF transports. At the preserved local budget, IAF gave acceptance .838, second-moment R-hat 1.01867, and minimum second-moment ESS 376.3, whereas affine funnel controls had second-moment ESS below 10. This passed its preserved quality screen; its diagnostics and budget differ from the modern common protocol.

**Ill-conditioned Gaussian, dimension 100.**

The target was $\mathcal{N}(0, \Sigma)$. A seed-10 Gaussian matrix supplied an orthogonal $Q$; sorted draws $\lambda_i \sim \text{Gamma}(.8, 1)$ defined

$$\Sigma = Q\,\text{diag}(\lambda_i^{-1})Q^\top. \tag{49.100}$$

It tested constant and neural geometry. Triangular affine and IAF were both strong: second-moment R-hat 1.00277/1.00244 and minimum second-moment ESS 2716.5/2724.5. This shows that nonlinear transport was not necessary for a target whose geometry a full affine map could remove.

**German-credit sparse logistic regression, dimension 51.**

The dataset had 1000 observations and 25 raw columns; 24 predictors were range-scaled, mapped by $2x - 1$, and augmented by an intercept, giving $d = 25$. Binary outcomes obeyed

$$y_n \sim \text{Bernoulli}\big(\text{logit}^{-1}(x_n^\top[\tau z \circ \lambda])\big)\,, \tag{49.101}$$

with $z_j \sim \mathcal{N}(0,1)$ and independent Gamma(.5, .5) global $\tau$ and local $\lambda_j$ scales. Log scales make the unconstrained dimension $2d+1 = 51$. A three-stage IAF with step .05 and 32 leapfrogs achieved acceptance .918, second-moment R-hat 1.01564, and minimum second-moment ESS 414.7 across three seeds, six chains, and 768 retained draws per chain. Its quality threshold was 1.2. It is qualified because budget ratio .0005625 remained below the predeclared .001 gate, not because the chain exhibited a hard numerical failure.

**NK-like analytic surrogate.**

In local coordinates $x \in \mathbb{R}^9$, let

$$r = \|x\|, \qquad a = .35\tanh(x_1)s(r), \qquad \lambda_s = 1 + .25(r/3)^2, \tag{49.102}$$

and let $\lambda_w$ be the smooth lower-bounded version of $.15[1-.6(r/3)^2]$. If $R(a)$ rotates the first two coordinates and $P_2(x) = R(a)\,\mathrm{diag}(\lambda_w, \lambda_s)R(a)^\top$, the target is

$$\log \widetilde{\pi}(x) = -\tfrac{1}{2}x_{1:2}^\top P_2(x) x_{1:2} - \tfrac{1}{2}\sum_{j=3}^{9} x_j^2. \tag{49.103}$$

Here $s(r)$ smoothly caps $r/3$ at one and the weak eigenvalue is smoothly floored at $10^{-3}$, with softplus scales .05 and .002. This nine-dimensional changing-Hessian target used three IAF stages with $(9, 9)$ hidden layers, 5000 reverse-KL steps, batch 4096, and a 49-candidate fixed HMC grid. The selected $.5 \times 3$ kernel had acceptance .757, zero recorded divergences, preserved $x$-coordinate R-hat 1.00158, and minimum preserved $x$-coordinate ESS 2541.5. These preserved diagnostics are not silently promoted to the modern rank/folded definitions. This is one-seed strong diagnostic evidence, not a method ranking.

**Small/real NK local posterior.**

This is a nine-parameter Small-NK state-space posterior: the model solution and observed data define an exact linear-Gaussian Kalman likelihood, combined with the model's declared prior in unconstrained coordinates. It is a real local posterior, not the analytic surrogate in the preceding item. It used a 3000-step three-stage IAF and 49 CPU HMC candidates. All worker results were recovered; the selected $.5 \times 4$ kernel had acceptance .918, zero divergences, preserved $x$-coordinate R-hat 1.00077, and minimum preserved $x$-coordinate ESS 9011.3. These are not asserted to be modern rank/folded diagnostics. The parent timed out before canonical reduction, so the result is qualified despite strong worker-level diagnostics. "Canonical reduction" means the parent process did not write its normal aggregate JSON after all 49 workers; the selected row was rebuilt by the same deterministic key from hash-checked worker outputs.

**Small/real NK principal-square-root SVD–UKF.**

The same economic model was paired with a nonlinear principal-square-root SVD–UKF approximate prediction-error likelihood rather than the linear Kalman likelihood. A fixed diagonal transport trained with batch 4096 for 5000 steps and an HMC kernel with step .5, 16 leapfrogs, four chains, and 1536 retained draws per chain reached acceptance .703, preserved per-parameter bulk/tail ESS 550.4/615.8, but had preserved per-parameter R-hat $1.01774 > 1.01$ and lacked native divergence telemetry. The artifact does not establish that this R-hat was the modern

rank/folded maximum. It remains a marginal continuation candidate, not a clean pass.

**Rotemberg NK linear-Kalman posterior.**
The linearized 15-parameter economic solution with exact Kalman likelihood completed calibrated learned-NeuTra HMC. Four chains used 1536 burn-in and 1536 retained draws; the selected step .65 and 25-leapfrog kernel had acceptance .743, preserved maximum R-hat 1.00250, minimum preserved per-parameter bulk/tail ESS 3793.4/1484.5, and finite samples. The record does not identify that R-hat as the modern rank/folded maximum. It is a clean historical baseline, not a claim about the nonlinear Rotemberg posterior.

**Rotemberg NK second-order SVD posterior.**
A four-shock minimal principal-square-root second-order state construction, coupled to an SVD sigma-point approximate likelihood, produced a 15-dimensional HMC state. Training and a 612-candidate fixed grid selected step .72917 and two leapfrogs, with acceptance .732, zero selected-candidate energy failures, maximum preserved $x$-coordinate R-hat 1.00324, and minimum preserved $x$-coordinate ESS 2248.3. These are not asserted to be modern rank/folded diagnostics. This is a completed-valid serious baseline; no comparative ranking or economic-model adequacy follows.

Nine of the thirteen configurations are clean or strong under their preserved screens. Four are qualified or marginal: German budget ratio, real-NK parent timeout, NK SVD–UKF convergence/telemetry, and structural post-result ESS adjudication. Exact/KSC stochastic volatility and several Zhao–Cui cells stopped before retained learned-NeuTra HMC and are therefore not counted.

## 49.15 Interpretation, failure modes, and nonclaims

The evidence supports the bounded statement that learned NeuTra has produced viable transformed-HMC sampling across a broad set of difficult geometries, including exact linear-Gaussian, nonlinear biological, epidemic, structural, hierarchical, regression, and DSGE-derived targets. In the five comparable synthetic configurations, all 38 inspected truths were non-extreme and inside empirical 95% intervals for one frozen dataset each.

The strongest alternative explanations are favorable fixture choice, reverse KL missing a mode not visited by finite HMC, approximate filter likelihoods defining a different posterior from the exact scientific model, and heterogeneous historical protocols. The results therefore do not establish:

- repeated-sampling calibration or 95% frequentist coverage;
- exactness of UKF, SGQF, or SVD-filter posteriors relative to the latent data-generating model;
- universal mode or tail recovery by reverse KL;
- statistical superiority over plain HMC, NUTS, RMHMC, or another map;
- convergence merely because R-hat and ESS cross finite thresholds;

- universal architecture, optimizer, HMC, production, or default policy.

The engineering, sampler, and scientific ledgers remain separate. A correct map can train poorly; a low loss can yield bad HMC; valid HMC can place truth in an extreme tail; and one-fixture success can fail under repeated datasets. That separation is not procedural caution: it is the mathematical distinction between implementing the intended density, exploring it adequately, and learning something scientifically defensible from the resulting posterior.

# Chapter 50

# Modern Importance Sampling and Coverage-Aware Flow Training

A posterior can have several economically distinct explanations of the same observations. A proposal fitted around one explanation may generate smooth, well-behaved trajectories while missing the others. Importance sampling makes this failure precise: a proposal must assign sufficient probability wherever the posterior contributes to the expectation being estimated. A tractable unnormalized posterior is enough to compute weights, but it does not make unvisited regions visible. Modern methods address this difficulty by adapting mixtures, moving through intermediate distributions, learning transport maps, or learning stochastic dynamics. Their common purpose is to make the change of probability measure less variable.

This chapter develops these ideas for a posterior on unconstrained parameters $x \in \mathbb{R}^d$. Write its unnormalized density as $\gamma(x)$, its normalizing constant as $Z = \int \gamma(x)\,dx \in (0, \infty)$, and its normalized density as $p(x) = \gamma(x)/Z$. In a state-space application, $\gamma$ includes the prior, likelihood and any parameter-chart Jacobian. If a filter supplies an approximate likelihood, all subsequent identities concern that declared approximate posterior. Learning a better proposal cannot remove the filter approximation.

## 50.1 What importance weights measure

Let $q$ be a normalized proposal with $q(x) > 0$ wherever $\gamma(x) > 0$, up to sets of measure zero. Draw independent $X_i \sim q$ and define $W_i = \gamma(X_i)/q(X_i)$. For an integrable quantity $f$, direct substitution of the proposal density gives

$$\mathbb{E}_q[W] = \int q(x)\frac{\gamma(x)}{q(x)}\,dx = Z, \tag{50.1}$$

$$\mathbb{E}_q[W f(X)] = Z\mathbb{E}_p[f(X)]. \tag{50.2}$$

Thus $\widehat{Z} = N^{-1}\sum_i W_i$ is unbiased. When $Z$ is unknown, the self-normalized estimate is

$$\widehat{\mu} = \frac{\sum_i W_i f(X_i)}{\sum_i W_i}, \qquad \mu = \mathbb{E}_p[f(X)]. \tag{50.3}$$

The ratio is generally biased at finite $N$. Under a law of large numbers it is consistent. Under the appropriate second-moment condition, applying the delta method to the

numerator and denominator yields

$$\sqrt{N}(\widehat{\mu}-\mu) \;\Rightarrow\; \mathcal{N}(0,\sigma_f^2), \qquad \sigma_f^2 = \int \frac{p(x)^2}{q(x)}[f(x)-\mu]^2\,dx. \tag{50.4}$$

Indeed, the first-order ratio error is $N^{-1}\sum_i W_i[f(X_i)-\mu]/Z$, whose mean is zero and whose variance is $\sigma_f^2/N$. This derivation explains why a proposal useful for the posterior mean need not be equally useful for a tail event or a rare default probability. For a single known $f$, Cauchy–Schwarz shows that the variance-minimizing self-normalized proposal is proportional to $p|f-\mu|$, when this expression is meaningful. It depends on the unknown answer. Approximating $p$ itself is a more general-purpose choice [Owen, 2013, Chapter 9].

For normalizer estimation, the variance has a direct divergence interpretation:

$$\frac{\mathrm{Var}_q(W)}{Z^2} = \int \frac{p(x)^2}{q(x)}\,dx - 1 = \chi^2(p\|q), \tag{50.5}$$

$$\frac{\mathrm{Var}(\widehat{Z})}{Z^2} = \frac{\chi^2(p\|q)}{N}. \tag{50.6}$$

The familiar sample diagnostic $\widehat{\mathrm{ESS}} = (\sum_i W_i)^2/\sum_i W_i^2$ estimates a weight-concentration quantity. With suitable moment laws, $\widehat{\mathrm{ESS}}/N \to [1+\chi^2(p\|q)]^{-1}$. It is neither a count of independent posterior draws nor a certificate that all modes were visited. If every draw lies in one of two separated modes, nearly equal observed weights can give an apparently excellent ESS while the omitted mode remains unknown.

## 50.2 Why dimension and missing modes matter

Suppose both densities factorize over their coordinates: $p(x) = \prod_j p_j(x_j)$ and $q(x) = \prod_j q_j(x_j)$. Tonelli's theorem then gives

$$1+\chi^2(p\|q) = \prod_{j=1}^{d}\int \frac{p_j(x_j)^2}{q_j(x_j)}\,dx_j = \prod_{j=1}^{d}[1+\chi^2(p_j\|q_j)]. \tag{50.7}$$

A small mismatch repeated across many coordinates can therefore produce exponential weight variance. For $p=\mathcal{N}(m,I_d)$ and $q=\mathcal{N}(0,I_d)$, $W/Z=\exp(m^\top X-\|m\|^2/2)$ and Gaussian integration gives $\mathbb{E}_q[(W/Z)^2]=\exp(\|m\|^2)$. A fixed mean error per coordinate has $\|m\|^2$ proportional to $d$.

A related sample-size analysis is due to Chatterjee and Diaconis [2018, Theorems 1.1–1.2]. They connect the threshold to $\exp\{D_{\mathrm{KL}}(p\|q)\}$. Their bounds involve the concentration of $\log(p/q)$ under $p$ and the integrand, with a separate self-normalized result. The statement is not an unconditional rule that exactly $\exp(D_{\mathrm{KL}})$ samples suffice. It explains why the posterior's typical set can be extremely rare under a superficially plausible proposal. The second-moment identity above and the KL threshold answer related but distinct questions: variance control and the onset of typical-set coverage.

Modes alone do not determine probabilities. At an isolated interior mode $m_k$ with positive-definite $H_k=-\nabla^2\log\gamma(m_k)$, the local quadratic approximation is

$$\gamma(x) \simeq \gamma(m_k)\exp\{-\tfrac{1}{2}(x-m_k)^\top H_k(x-m_k)\}, \qquad Z_k^{\mathrm{Lap}} = \gamma(m_k)(2\pi)^{d/2}|H_k|^{-1/2}. \tag{50.8}$$

The determinant measures local volume. Ranking modes by height discards it. Even the Laplace mass can fail for curved ridges, boundaries, heavy tails or nonquadratic basins. Multistart optimization and mode merging, as developed within the adaptive multimodal MCMC framework of Pompe et al. [2020, Section 2.2], provide useful candidate locations. Finding all modes of an unrestricted nonlinear density remains a separate problem. A finite optimizer search cannot establish exhaustiveness.

A proposal does not need one component per mode. In a product of $d/2$ double-well densities, the number of modes is $2^{d/2}$, while the formula for the density remains short. Exploiting structure can be far more economical than enumerating modes. This distinction is central to interpreting high-dimensional multimodal demonstrations.

## 50.3 From weighted kernels to adaptive parametric mixtures

West [1993, Sections 2.1–2.2] uses importance-weighted kernel mixtures to refine posterior approximations. Given weighted locations, a normalized kernel estimate has the form

$$q_{\text{next}}(x) = \sum_{i=1}^{N} \overline{W}_i K_H(x - X_i), \qquad \overline{W}_i = W_i / \sum_j W_j, \tag{50.9}$$

where $K_H$ integrates to one. Gaussian or Student kernels produce an evaluable proposal and can follow skewness or several basins. Every next sample must be weighted using the density of the proposal that actually generated it. Recycling an adaptive cloud requires corresponding multiple-importance or sequential weighting; retroactively dividing all old draws by the latest proposal is not automatically correct.

The classical kernel density calculation exposes a limitation. Under the usual second-order smoothness assumptions and an isotropic bandwidth $h$, squared integrated bias is of order $h^4$, while integrated variance is of order $(Nh^d)^{-1}$. Balancing the two terms gives

$$\text{MISE}(h) \simeq C_1 h^4 + \frac{C_2}{Nh^d}, \qquad h_{\text{opt}} = \left(\frac{dC_2}{4C_1 N}\right)^{1/(d+4)}, \qquad \text{MISE}(h_{\text{opt}}) = O(N^{-4/(d+4)}). \tag{50.10}$$

This is the standard fixed-dimensional density-estimation calculation, not a theorem covering arbitrary dependent adaptive weights. Its slow exponent explains why adding local kernels alone becomes expensive. Estimating one integral can be easier than reconstructing a whole density; the two tasks must not be conflated.

Consider a finite mixture $q_\eta(x) = \sum_{k=1}^{K} a_k q_k(x; \eta_k)$. Forward-KL fitting minimizes $D_{\text{KL}}(p \| q_\eta)$. Equivalently, it maximizes $\mathbb{E}_p \log q_\eta$. Importance weights approximate this expectation without requiring normalized $p$. The current responsibilities are

$$r_{ik} = \frac{a_k q_k(X_i; \eta_k)}{q_\eta(X_i)}. \tag{50.11}$$

The Gaussian EM updates are

$$a_k^+ = \sum_i \overline{W}_i r_{ik}, \qquad m_k^+ = \frac{\sum_i \overline{W}_i r_{ik} X_i}{a_k^+}, \tag{50.12}$$

$$\Sigma_k^+ = \frac{\sum_i \overline{W}_i r_{ik} (X_i - m_k^+)(X_i - m_k^+)^\top}{a_k^+}. \tag{50.13}$$

Jensen's inequality applied to the latent component label gives the EM lower bound; maximizing it yields these weighted moments. The weights remain fixed during each empirical EM fit. Cappé et al. [2008, Section 2] derive adaptive updates for general mixture classes. Covariance regularization, component removal and stopping remain numerical choices; none creates evidence about an unvisited region.

Raftery and Bao [2010, Section 3] develops incremental mixture importance sampling (IMIS). It adds local Gaussian components near points with large importance weights, estimates local covariance from neighboring points, simulates new draws and recomputes weights against the resulting proposal mixture. It was designed for difficult epidemiological posteriors, including nonlinear ridges. Its residual-weight trigger identifies where the current sampled proposal is inadequate; it cannot add a component at a region that no search or sample has reached.

MitISEM replaces unrestricted kernel proliferation with adaptive Student-$t$ mixtures and importance-weighted EM [Hoogerheide et al., 2012, Sections 2 and 5; Appendix A]. For one $t_\nu(m, \Sigma)$ component, the Gaussian scale-mixture representation adds a latent precision with conditional mean $(\nu + d)/(\nu + (x - m)^\top \Sigma^{-1}(x - m))$. This supplies robust weighted location and scatter updates. More explicitly, with $\tau_{ik} = (\nu_k + d)/(\nu_k + \delta_{ik})$ and $\delta_{ik} = (X_i - m_k)^\top \Sigma_k^{-1}(X_i - m_k)$, one EM step at fixed $\nu_k$ gives

$$m_k^+ = \frac{\sum_i \overline{W}_i r_{ik} \tau_{ik} X_i}{\sum_i \overline{W}_i r_{ik} \tau_{ik}}, \tag{50.14}$$

$$\Sigma_k^+ = \frac{\sum_i \overline{W}_i r_{ik} \tau_{ik} (X_i - m_k^+)(X_i - m_k^+)^\top}{\sum_i \overline{W}_i r_{ik}}. \tag{50.15}$$

Here $\Sigma_k$ is a scatter parameter, not the covariance; for $\nu_k > 2$ the covariance is $\nu_k \Sigma_k/(\nu_k - 2)$. Optimizing the degrees of freedom adds a scalar nonlinear update involving the latent precision's expected logarithm. Large-weight points motivate extra components. The partial MitISEM extension factors marginal and conditional distributions, reducing the dimension of each approximation problem when that factorization is useful. The locally inspected source is the 2011 working paper preceding the 2012 publication; its extra sequential, permutation and partial-mixture constructions should be distinguished from a claim to reproduce every published experiment.

## 50.4 Defensive and adaptive proposals

A defensive mixture protects against a fitted proposal becoming too narrow:

$$q_\lambda(x) = (1 - \lambda) q_{\text{fit}}(x) + \lambda q_0(x), \qquad 0 < \lambda \le 1. \tag{50.16}$$

If $p(x) \le C q_0(x)$ everywhere, then $p/q_\lambda \le C/\lambda$ and

$$\int \frac{p^2}{q_\lambda} \le \frac{C}{\lambda} \int p = \frac{C}{\lambda}. \tag{50.17}$$

The domination assumption is the substantive protection; the word "heavy-tailed" by itself is insufficient. A very small $\lambda$ can still make finite-sample exploration ineffective.

Delyon and Portier [2021, Theorems 7–9] study a kernel estimate mixed with a safe density and give convergence and integral-estimation results under conditions on domination, bandwidth and the declining safe share. Their root-$N$ integral asymptotics concern

increasing sample size at fixed dimension under the stated regularity assumptions. They do not remove the bandwidth's dimension dependence or guarantee small finite-sample cost in high dimension. For example, the related regularized safe scheme of Korba and Portier [2022] requires, among its conditions, $\lambda_n \to 0$, $h_n \to 0$ and $\log n/(nh_n^d\lambda_n) \to 0$. It also requires $q_0 \geq cp$, boundedness and regularity of the density and kernel, and controls such as $(1-\eta_n)\log h_n \to 0$ when powered weights are used. The factor $h_n^d$ shows explicitly where dimension remains in the convergence requirements.

Korba and Portier [2022, Sections 2–3] regularize importance weights using an exponent $\eta \in (0, 1]$. The population calculation is

$$q(x)\left(\frac{p(x)}{q(x)}\right)^{\eta} = p(x)^{\eta}q(x)^{1-\eta}. \tag{50.18}$$

Lower $\eta$ reduces concentration but changes the distribution fitted by those weights. Their mirror-descent interpretation and safe kernel mixture balance this bias against variance, with asymptotic conditions driving $\eta$ toward one. Regularized adaptation and final estimation are different operations: an estimate of $\mathbb{E}_p f$ still requires weights for $p$, unless a different estimand is explicitly intended.

Gradient-based adaptive importance samplers (GRAMIS) use target gradients, local curvature and interactions among proposal components to direct an adaptive mixture [Elvira et al., 2023, Section 3 and Table 2]. Local preconditioning can move a component toward a high-density region; repulsion reduces redundant concentration. Their samples retain evaluable proposal densities and importance correction. For equally allocated samples from $M$ proposals, their deterministic-mixture weight denominator is $M^{-1}\sum_{j=1}^{M} q_j(x)$, so every sampled point is judged against the entire current proposal mixture. The mean update combines a backtracked preconditioned score step with repulsion:

$$m_i^{+} = m_i + \theta_i \Sigma_i \nabla \log \gamma(m_i) + \sum_{j \neq i} r_{ij}. \tag{50.19}$$

The paper specifies the scaling of $r_{ij}$ from a Poisson-field construction and a distance-dependent interaction. Backtracking tests the score step without the repulsion term, and the covariance uses local negative inverse Hessian information only when positive definiteness permits it, retaining valid prior information otherwise. Thus the method is not an unconditional Newton step through an indefinite Hessian. The reported high-dimensional banana examples exploit strong structure. Such examples support investigation of the mechanism, not unrestricted mode discovery in every 50-dimensional posterior.

Gaussian-mixture variational inference (GMMVI) addresses another objective. Arenz et al. [2023, Sections 2–4 and Appendix A] unify natural-gradient updates, trust regions, sample reuse and component adaptation for $D_{\mathrm{KL}}(q\|p)$. A natural gradient has the form $-F(\eta)^{-1}\nabla_\eta D_{\mathrm{KL}}$, where $F$ is the Fisher information in the chosen variational coordinates. It accounts for how parameter changes alter distributions. Reverse KL still weights errors under $q$, so efficient optimization does not itself prevent missing modes. The paper's warning that ELBO-based tuning can impair other distributional metrics is directly relevant to evaluating a learned transport.

## 50.5 Annealing and sequential importance sampling

Let $\gamma_0 = q_0$ be normalized and let $\gamma_K = \gamma$. Intermediate unnormalized densities may be geometric bridges $\gamma_k = q_0^{1-\beta_k}\gamma^{\beta_k}$ for $0 = \beta_0 < \cdots < \beta_K = 1$. An AIS realization begins

at $X_0 \sim q_0$. At stage $k$, multiply its weight by $\gamma_k(X_{k-1})/\gamma_{k-1}(X_{k-1})$ and then apply a Markov kernel $K_k$ invariant for $\pi_k = \gamma_k/Z_k$. With fixed kernels,

$$W_{\mathrm{AIS}} = \prod_{k=1}^{K} \frac{\gamma_k(X_{k-1})}{\gamma_{k-1}(X_{k-1})}, \qquad \mathbb{E}[W_{\mathrm{AIS}} f(X_K)] = \int \gamma(x) f(x)\, dx. \tag{50.20}$$

The last mutation can be omitted, in which case $f$ is evaluated at $X_{K-1}$. To see the identity, suppose the weighted marginal before stage $k$ is $\gamma_{k-1}(x)\,dx$. Multiplication changes it to $\gamma_k(x)\,dx$. Integrating the next state against $K_k$ preserves this measure by invariance. Induction from $q_0$ proves the result. An equivalent path-density derivation uses reverse kernels; it does not require each finite chain to reach equilibrium [Neal, 2001]. It does require correct invariance and support. Adapting a kernel from the same interacting batch needs separate justification and must not silently inherit the fixed-kernel unbiasedness statement.

Log weights accumulate as a sum, avoiding products that overflow. For the geometric bridge the increment is $(\beta_k - \beta_{k-1})[\log \gamma(X_{k-1}) - \log q_0(X_{k-1})]$. A small temperature change limits the immediate variation of this increment. Good mutation is still necessary: many intermediate densities with negligible movement can preserve a bad particle configuration at high cost.

Sequential Monte Carlo adds resampling. Given normalized weights, draw parent indices with those probabilities, reset weights appropriately and apply mutation. Conditional unbiasedness of the resampling operation preserves the weighted measure in expectation. Products of the incremental normalizer estimates give the usual SMC normalizer estimator under standard fixed-design conditions [Chopin and Papaspiliopoulos, 2020]. Resampling trades weight degeneracy for dependence and possible loss of ancestry. It cannot manufacture a mode that neither the initial particles nor mutation reach.

## 50.6 Annealed flow transport and continual training

A learned bijection $T_k$ can move particles before they are weighted. If $X \sim \pi_{k-1}$, change of variables gives the incremental factor

$$G_k(X) = \frac{\gamma_k(T_k(X))\,|\det DT_k(X)|}{\gamma_{k-1}(X)}. \tag{50.21}$$

Multiplying the pushed-forward law by this factor produces $(Z_k/Z_{k-1})\pi_k$. Approximation error in the map affects weight variance; a missing Jacobian changes the measure.

AFT learns each map by minimizing a local reverse KL. Substituting the density of $T_k(X)$ into that KL yields, up to $\log Z_k - \log Z_{k-1}$,

$$\mathcal{L}_k(T_k) = \mathbb{E}_{\pi_{k-1}}[\log \gamma_{k-1}(X) - \log \gamma_k(T_k(X)) - \log |\det DT_k(X)|]. \tag{50.22}$$

Arbel et al. [2021, Equations 7–9 and Algorithm 1] combine transport, importance correction, resampling and MCMC mutation. Their law of large numbers and central limit theorem require explicit smoothness, moment and bounded-weight conditions, with further assumptions for the CLT. Data-dependent fitted maps cannot simply be treated as deterministic maps when asserting finite-sample unbiasedness; the paper supplies an asymptotic analysis for its adaptive scheme.

CRAFT repeatedly runs the annealed SMC procedure and uses renewed particle approximations to update the stage maps [Matthews et al., 2022, Equation 5, Algorithms 1–3 and supplement]. Replenishing particles avoids optimizing each stage indefinitely against one small fixed cloud. The maps learn progressively while corrected particles still represent the annealed sequence. This is a different training organization from fitting one whole-target flow by forward KL or FAB. Its practical advantage depends on target cost, number of stages and mutation; neither repeated fitting nor the transport correction eliminates finite-particle mode loss. One complete pass therefore starts with a fresh base cloud, computes each stage's current weighted training loss, moves points with the current map, updates their weights using (50.21), resamples when the declared criterion calls for it, and mutates with the intermediate target. The next pass uses the updated maps. Optimizer updates and particle transport must follow the chosen algorithm's ordering; a map fitted using a particle cloud cannot be silently replaced in the weight denominator by a later map.

## 50.7 FAB: minimizing the importance-weight second moment

Let $q_\phi$ be the density of a trainable flow. FAB minimizes the alpha-two objective

$$J(\phi) = \int \frac{\gamma(x)^2}{q_\phi(x)}\,dx = Z^2[1 + \chi^2(p\|q_\phi)], \qquad g_\phi(x) = \frac{\gamma(x)^2}{q_\phi(x)J(\phi)}. \tag{50.23}$$

Assume $J < \infty$, $q_\phi > 0$ on the target support, and continuous differentiability of $q_\phi$ with respect to $\phi$. Assume also an integrable envelope for the differentiated integrand in a neighborhood of $\phi$, so that differentiation under the integral is valid. Since $\gamma$ does not depend on the flow parameters,

$$\nabla_\phi J = -\int \frac{\gamma^2}{q_\phi^2}\nabla_\phi q_\phi\,dx = -\int \frac{\gamma^2}{q_\phi}\nabla_\phi \log q_\phi\,dx, \tag{50.24}$$

$$\nabla_\phi \log J = -\mathbb{E}_{g_\phi}[\nabla_\phi \log q_\phi(X)]. \tag{50.25}$$

The negative density score is emphasized where $p$ is large and $q$ is too small. This is the gradient of a chi-square objective, not ordinary forward KL, whose gradient instead averages the same score under $p$. Minimizing $J$ or $\log J$ has the same minimizers and descent directions, but different gradient scales. The unknown $Z$ is irrelevant to those minimizers.

Midgley et al. [2023, Section 3.1, Equations 4–7 and Appendix A] approximate expectations under $g_\phi$ by AIS. At outer iteration $t$, hold $q_t = q_{\phi_t}$ fixed during the forward sampling pass and use

$$\gamma_{t,\beta}(x) = q_t(x)^{1-\beta}\{\gamma(x)^2/q_t(x)\}^\beta = q_t(x)^{1-2\beta}\gamma(x)^{2\beta}, \tag{50.26}$$

$$\log \gamma_{t,\beta}(x) = (1-2\beta)\log q_t(x) + 2\beta \log \gamma(x), \tag{50.27}$$

$$\nabla_x \log \gamma_{t,\beta}(x) = (1-2\beta)\nabla_x \log q_t(x) + 2\beta \nabla_x \log \gamma(x). \tag{50.28}$$

At $\beta = 0$ the density is $q_t$; at $\beta = 1/2$ it is the unnormalized posterior; at $\beta = 1$ it is $\gamma^2/q_t$. A posterior-tempering parameter inside a likelihood is a different quantity from this FAB

bridge parameter. They must not be identified merely because both are denoted beta in software. The AIS log-weight increment is

$$\Delta \log W_i = 2\Delta\beta[\log \gamma(X_i) - \log q_t(X_i)]. \tag{50.29}$$

The flow density is therefore needed at arbitrary locations reached by mutation, not just at samples generated by the flow. HMC mutations also require its spatial score. The author implementation offers a symmetric Gaussian random-walk Metropolis alternative, which uses density values alone.

With AIS terminal points $X_i$ and weights $W_i$, the practical loss is

$$\widehat{S}_t(\phi) = -\sum_i \operatorname{stopgrad}\left(\frac{W_i}{\sum_j W_j}\right) \log q_\phi(\operatorname{stopgrad}(X_i)). \tag{50.30}$$

Differentiation acts only on the final log density. Differentiating through the AIS trajectory, proposal distribution or weights computes additional terms and is wrong relative to this estimator. With fixed valid AIS kernels, unnormalized weights can estimate $\nabla J$ unbiasedly; the ratio in (50.30) generally introduces finite-sample bias. Reverse-KL path-gradient estimators such as Roeder's and Vaitl's solve a different reparameterized-gradient problem [Roeder et al., 2017, Vaitl et al., 2024]; they cannot replace this detached-density gradient unchanged.

The target $g_t$ changes when $q_t$ changes. Fitting a single frozen $g_t$ to completion is therefore not the adaptive FAB procedure. New AIS passes and corrected replay retain the connection to the current objective. At the ideal solution $q = p$, $g = p$ and the auxiliary emphasis disappears.

### 50.7.1 Replay corrections and their limits

Suppose a stored AIS point was weighted for $\gamma^2/q_{\text{old}}$. The corresponding unnormalized target ratio is

$$c_\phi(x) = \frac{\gamma(x)^2/q_\phi(x)}{\gamma(x)^2/q_{\text{old}}(x)} = \frac{q_{\text{old}}(x)}{q_\phi(x)}, \qquad \log c_\phi = \log q_{\text{old}} - \log q_\phi. \tag{50.31}$$

Store the point, its AIS log weight and the log density used for that weight. Sample replay indices proportional to their stored weights, then use the minibatch loss $-B^{-1}\sum_i \operatorname{stopgrad}(c_i) \log q_\phi(X_i)$. Before the optimizer changes $\phi$, add $\log c_i$ to each selected priority and replace its stored log density by the currently evaluated one. Successive corrections telescope: $(q_a/q_b)(q_b/q_c) = q_a/q_c$.

This is Algorithm 1 and Section 3.2 of Midgley et al. [2023]. It saves expensive posterior evaluations because old points need only a new flow density. It does not permit uniform sampling from the buffer followed by the same correction: that would omit the original AIS priorities. Likewise, inserting detached corrections into a trainer that automatically renormalizes every minibatch changes the stated replay loss.

The paper's practical sampling without replacement introduces bias. The inspected author JAX implementation additionally caps $c_i$ in the loss, with a configurable default of ten, while refreshing stored log priorities with the uncapped correction. Capping bounds a stochastic update but changes its expectation. Its activation frequency and effect require measurement. Pooled replay, finite AIS, stale priorities and dependence are further reasons not to call practical replay gradients exactly unbiased. A useful implementation must preserve the distinction between the underlying objective and its finite training procedure.

### 50.7.2 Tail integrability is stronger than full support

A Gaussian example demonstrates the issue. If $p = \mathcal{N}(0, \sigma_p^2)$ and $q = \mathcal{N}(0, \sigma_q^2)$, then

$$\frac{p(x)^2}{q(x)} = C \exp\left[-x^2\left(\frac{1}{\sigma_p^2} - \frac{1}{2\sigma_q^2}\right)\right]. \tag{50.32}$$

Its integral is finite precisely when $\sigma_q^2 > \sigma_p^2/2$. Both densities have full support even when this condition fails. A highly contracted reverse-KL warm start may therefore create an invalid FAB auxiliary density. Radial evaluations and weight diagnostics can expose trouble but cannot prove a global tail bound. A defensive mixture with a proved dominating component can guarantee a finite second moment by (50.17); it also changes the proposal family and must be accounted for explicitly.

FAB's Appendix C studies favorable dimensional scaling under factorized $p, q$, independent coordinate parameters and perfect intermediate transitions. Its gradient-variance calculation motivates increasing intermediate steps with dimension. It is not a guarantee for arbitrary dependent posteriors with local HMC. The 32-dimensional Many-Well benchmark has 65,536 modes through a product construction; it demonstrates exploitation of structure, not an exhaustive optimizer search over arbitrary modes.

## 50.8 Combining FAB with IAF and NeuTra

FAB supplies the training procedure; an inverse autoregressive flow supplies the density family. For stage input $z$, use

$$u_j = z_j \exp\{s_j(z_{<j})\} + \mu_j(z_{<j}), \qquad s_j = b_j + c\tanh\{h_j(z_{<j})/c\}. \tag{50.33}$$

Masked neural networks ensure that $s_j$ and $\mu_j$ depend only on preceding coordinates. The free bias $b_j$ lies outside the conditional cap. This is the option in Hoffman's author code used by the current BayesFilter IAF; the conditional contribution is bounded, while the complete log scale need not be. The cap $c = 2$, width sixteen and four-parameter application are local choices. The source architecture uses ELU conditioners, author block masks, nonzero variance-scaled kernels and coordinate reversal between three stages [Hoffman et al., 2019, Section 4.1.1 and accompanying code].

The Jacobian of each stage is triangular with diagonal $\exp(s_j)$, hence $\log|\det Df| = \sum_j s_j$. For a composition $T_\phi$ with a fixed invertible outer affine map $a + Az$, stage log determinants add and the affine contributes $\log|\det A|$. With standard-normal base density $\varphi_d$,

$$q_\phi(x) = \varphi_d(T_\phi^{-1}(x))|\det DT_\phi(T_\phi^{-1}(x))|^{-1}. \tag{50.34}$$

Sampling evaluates each autoregressive stage in parallel across its coordinates. Evaluating the density at an arbitrary $x$ requires inversion: first recover $z_1$, then use it to recover $z_2$, and continue. The diagonal solve is $z_j = [u_j - \mu_j(z_{<j})]\exp[-s_j(z_{<j})]$. There is no dense determinant, but FAB makes the inverse cost more prominent than reverse-KL training does. Batches remain parallel; the coordinate dependency is intrinsic to this IAF orientation. A nonlinear NAF [Huang et al., 2018] is a different scalar transformation and is not required to combine FAB with IAF.

At fixed $x$, implicit differentiation of $T_\phi(z) = x$ gives

$$\partial_\phi z = -[D_z T_\phi(z)]^{-1}\partial_\phi T_\phi(z). \tag{50.35}$$

Therefore the parameter gradient of $\log q_\phi(x) = \log \varphi_d(z) - \log|\det D_z T_\phi(z)|$ includes both explicit parameter dependence and dependence through $z$. Holding this inverse latent value fixed would omit terms. Autodifferentiating the native inverse recursion computes this total derivative without materializing a dense Jacobian. Finite differences of selected parameter directions provide an independent check. FAB's detached $x$ is fully compatible with differentiating $z = T_\phi^{-1}(x)$ with respect to $\phi$.

Once training is complete, freeze $\phi$ and define the NeuTra target

$$\widetilde{p}_Z(z) = \gamma(T_\phi(z))|\det DT_\phi(z)|, \tag{50.36}$$

$$\nabla_z \log \widetilde{p}_Z(z) = DT_\phi(z)^\top \nabla_x \log \gamma(T_\phi(z)) + \nabla_z \log|\det DT_\phi(z)|. \tag{50.37}$$

Changing variables shows that an invariant chain for this density pushes forward to $p$, regardless of how accurately $q_\phi$ approximates $p$. Identity latent mass is a valid choice; mass adaptation is not a prerequisite. The auxiliary FAB target $\gamma^2/q$ never replaces the posterior in this formula. Better importance-weight coverage may still coexist with sharp latent curvature, so training success does not establish efficient HMC.

For a perfectly Gaussianized posterior, $\nabla_z \log \widetilde{p}_Z(z) = -z$ and $r(z) = \log \widetilde{p}_Z(z) + \|z\|^2/2$ is constant. The standard post-training check evaluates the score residual and $r$ on one thousand fresh normal-base points, preserving finite/status failures and the residual distribution. This checks geometry where the map generates points. Independent region occupancy and target-weighted evidence are needed for coverage; full retained chains and their uncertainty diagnostics are needed for posterior estimation.

## 50.9 Stein methods and stochastic transport

Stein variational gradient descent uses kernels in a different role from a kernel mixture proposal. For a smooth vector field $v$, transform $X \mapsto X + \epsilon v(X)$. Integration by parts gives

$$\frac{d}{d\epsilon} D_{\mathrm{KL}}(q_\epsilon \| p)\Big|_{\epsilon=0} = -\mathbb{E}_q[\nabla \log p(X)^\top v(X) + \nabla \cdot v(X)]. \tag{50.38}$$

Maximizing the negative derivative over a unit ball in a vector-valued reproducing-kernel Hilbert space yields the particle direction

$$v^*(x) \propto \frac{1}{N} \sum_i [k(X_i, x) \nabla \log p(X_i) + \nabla_{X_i} k(X_i, x)]. \tag{50.39}$$

The first term attracts toward posterior mass and the second repels particles [Liu and Wang, 2016]. The evolved empirical measure does not automatically supply an evaluable normalized density for importance weighting. Kernel bandwidth and high-dimensional distance concentration remain material issues.

Projected SVGD exploits a likelihood-informed subspace [Chen and Ghattas, 2020]. A sensitivity matrix built from likelihood scores identifies directions where the posterior differs most strongly from the prior. For a Gaussian prior with covariance $C$, one common form is

$$H = \mathbb{E}_p[(\nabla \log L(X))(\nabla \log L(X))^\top], \qquad H\psi_j = \lambda_j C^{-1} \psi_j, \tag{50.40}$$

where $L$ is the likelihood and the eigenvectors are orthonormal in the prior precision metric. The associated optimal conditional-expectation approximation averages $L$ over discarded prior coordinates. Under the functional-inequality and prior assumptions used

in that literature, its KL approximation error is controlled by a constant times $\sum_{j>r} \lambda_j$. This statement concerns the projected target approximation; finite particles and variational training add errors of their own. It is not a bound on an arbitrary implementation merely using the top $r$ eigenvectors. The method transports coordinates in that subspace and represents the complement using the prior conditional structure. Its error bounds depend on assumptions and the neglected eigenvalue tail. If many directions are strongly informed, projection has little advantage; nominal dimension alone does not determine its suitability.

Continuous flows and controlled diffusions instead learn trajectories. For a continuous normalizing flow $\dot{X}_t = v_\phi(X_t, t)$, conservation of probability implies

$$\frac{d}{dt} \log q_t(X_t) = -\nabla \cdot v_\phi(X_t, t). \tag{50.41}$$

Wu and Xie [2025, Proposition 3.1, Equation 10 and Appendix A] train annealed flow segments with a reverse-KL endpoint term and an integrated squared-velocity penalty. Substituting (50.41) gives the segment objective, up to input-density constants,

$$\mathbb{E}[-\log \gamma_k(X_{t_k}) - \int_{t_{k-1}}^{t_k} \nabla \cdot v_\phi(X_t, t)\, dt + \lambda \int_{t_{k-1}}^{t_k} \|v_\phi(X_t, t)\|^2\, dt]. \tag{50.42}$$

The paper also examines discretized penalties, trace estimators, a first-order training modification and optional Langevin refinement. These approximations and its infinitesimal theory must be distinguished from an exact finite-step transport. The present application retains IAF's explicit triangular Jacobian instead of substituting this continuous-flow architecture.

Sequential controlled Langevin diffusions (SCLD) combine a learned stochastic control with sequential importance correction and resampling [Chen et al., 2025, Section 2, Algorithms 1–4 and Appendix A]. A finite path-space construction makes the correction understandable. Let $F_k(x_k \mid x_{k-1})$ be the simulated forward transition and $B_{k-1}(x_{k-1} \mid x_k)$ a normalized backward transition. The proposal path density is $q_0(x_0) \prod_k F_k$ and the unnormalized target path density is $\gamma_K(x_K) \prod_k B_{k-1}$. Their ratio factors as

$$W_{0:K} = \prod_{k=1}^{K} \frac{\gamma_k(x_k) B_{k-1}(x_{k-1} \mid x_k)}{\gamma_{k-1}(x_{k-1}) F_k(x_k \mid x_{k-1})}, \qquad \gamma_0 = q_0. \tag{50.43}$$

The intermediate densities telescope; integration of the backward transitions leaves terminal density $\gamma_K$. For nondegenerate Euler diffusion steps, the $F_k$ are Gaussian and can be evaluated. Continuous-time versions use Radon–Nikodym derivatives between path measures under the requisite absolute-continuity and integrability conditions. The SCLD paper derives these weights and trains controls using subtrajectory log-variance losses, including off-policy sampling. An additive unknown log normalizer disappears from a variance. For a detached reference path law $R_n$ with sufficient support, the subtrajectory objective has the form

$$\mathcal{L}_n(\phi) = \mathrm{Var}_{R_n}[\log W_{n,\phi}] = \mathbb{E}_{R_n}[(\log W_{n,\phi} - \mathbb{E}_{R_n} \log W_{n,\phi})^2]. \tag{50.44}$$

With the reference law held fixed while differentiating, its gradient is $2\mathbb{E}_{R_n}[(\log W_{n,\phi} - \mathbb{E}_{R_n} \log W_{n,\phi}) \nabla_\phi \log W_{n,\phi}]$. The centering term's derivative cancels because the centered

log weight has mean zero. Off-policy training separates the sampling law from the control being updated. Section 2.3 and Theorem A.2 explain this loss and its off-policy use. Proposition 2.3 instead establishes an exponential relative-error lower bound for a particular KL estimator on product path measures, motivating the alternative. Zero empirical variance on a finite buffer is not equality of complete path measures. Resampling between segments limits concentration over a whole long path, while optional MCMC refinement improves terminal locations.

A weighted path sampler is not automatically a tractable terminal density $q(x)$. It can provide training samples, but distilling them into an IAF is an additional approximation. SCLD's reported multimodal examples include a 50-dimensional, forty-component mixture. Its 1,600-dimensional log-Gaussian Cox example is primarily evaluated through an ELBO; it is not evidence of exhaustive unknown-mode coverage in 1,600 dimensions.

## 50.10 Evaluation and implementation

The broad evaluation of Blessing et al. [2024, Sections 7–8 and Appendices D–F] compares eleven variational and sampling methods across synthetic and Bayesian targets. Its mixture examples vary dimension, including 2, 50 and 200, and show that strong performance on an ELBO or one weight diagnostic can coexist with poor coverage. The evidence depends on the authors' architectures, tuning and budgets; it establishes no universal winner. It supports using several measurements that answer different questions.

For a learned posterior proposal, correctness of density and derivatives comes first. Next, inspect region coverage using independent information where available, importance-weight reliability on untouched draws, and consistency across seeds. For a NeuTra map, inspect transformed scores, numerical validity and downstream mixing. An apparently superior extreme quantile from a short run is descriptive until its uncertainty is assessed. A short successful test can retain a candidate; it cannot establish a production training protocol.

The author-maintained `fab-jax` repository separates sampling, replay and training from its bundled coupling-layer flow. Its MIT license permits adaptation with the notice retained. BayesFilter ports those FAB components to TensorFlow while keeping the existing IAF and batch-native posterior evaluator. This preserves a single transport implementation and avoids a second target implementation solely for a different autodiff backend. The port follows the paper's normalized-weight sum in (50.30); the inspected JAX no-replay function takes a mean of those weighted terms, adding a factor $1/N$. The replay loss, correction cap and without-replacement choice are recorded separately. These are explicit source differences, not interchangeable normalizations under arbitrary optimizer settings.

Three remaining costs determine the application. First, AIS requires repeated posterior evaluations, with scores at every intermediate HMC step or values alone for a Metropolis mutation. Second, IAF density evaluation at arbitrary points requires autoregressive inversion. Third, replay initialization must provide enough distinct points to avoid fitting a tiny static cloud. The author's recommendation is to use as few annealing and mutation steps as can make the resulting samples usefully better than the flow's initial samples, then calibrate on the actual problem. An example configuration is a starting hypothesis. Source faithfulness reduces accidental algorithm changes; target-specific measurement determines whether the resulting training procedure is affordable and scientifically useful.

# Chapter 51

# Metropolis-Corrected Neural-Force Hamiltonian Monte Carlo

This chapter studies a tempting acceleration of Hamiltonian Monte Carlo for state-space models. A filtering likelihood and its score may require one complete recursion through the observations. Ordinary HMC calls that score at every leapfrog kick. A learned force can make the interior trajectory cheap; one then evaluates the declared filtering posterior at the proposed endpoint and applies a Metropolis correction.

That summary is attractive but incomplete. A true endpoint posterior value does not correct an arbitrary neural proposal. The simple HMC acceptance formula is valid only after the proposal map has the required discrete reversibility and volume properties, and its endpoint energy must include the final momentum. A stochastic filtering likelihood adds a separate target problem: a fresh particle-filter estimate is not a deterministic potential and cannot be inserted naively into that formula.

The chapter therefore does four things. It derives a sufficient construction for a frozen learned force, proves its invariant target, identifies precisely why broader learned-Hamiltonian constructions need additional proof or correction, states the filtering boundary for every relevant target class, and ends with an implementation and validation contract for BayesFilter. The presentation is self-contained. The proposal of Dhulipala et al. [2022] and its official implementation motivate the audit, but no theorem below depends on accepting that paper's stationarity argument.

## 51.1 The computational question

Let $u \in \mathbb{R}^d$ be unconstrained sampler coordinates and let $y_{1:T}$ denote the observations. For a deterministic filtering backend $F$, define

$$\ell_F(u) = \log p(u) + \log L_F(y_{1:T} \mid u) + J(u), \qquad U_F(u) = -\ell_F(u), \tag{51.1}$$

where $J$ contains every parameter-transform log Jacobian. The subscript $F$ is essential. If $F$ is a Kalman filter, $L_F$ may be the exact likelihood of the declared linear Gaussian model. If $F$ is a UKF or fixed SGQF, $L_F$ is normally an approximate filtering likelihood. In either case, (51.1) is a deterministic scalar once the model, data, numerical branch, and filter settings are frozen.

With mass matrix $M \succ 0$, ordinary HMC uses

$$H_F(u,p) = U_F(u) + K(p), \qquad K(p) = \tfrac{1}{2} p^\top M^{-1} p, \tag{51.2}$$

and repeatedly evaluates $\nabla U_F$. Write $C_V$ for one new evaluation of $U_F$, $C_G$ for one evaluation of $\nabla U_F$ together with whatever value the implementation returns, and $C_S$ for one learned-force call. For $L$ leapfrog steps, the leading proposal costs are approximately

$$C_{\text{ordinary}} \simeq LC_G, \tag{51.3}$$
$$C_{\text{neural}} \simeq LC_S + C_V. \tag{51.4}$$

The current state's true value can be cached, so the corrected neural proposal usually needs only one new endpoint value. Equations (51.3)–(51.4) are accounting identities, not speedup theorems: acceptance, autocorrelation, fallback, training, compilation, batching, and device utilization determine actual ESS per second.

**Proposition 51.1** (When a neural-force proposal can reduce filtering work)**.** *Suppose ordinary and neural-force kernels use the same number $L$ of interior steps. Ignore one-time compilation and training, and suppose each kernel produces the same effective sample size per accepted transition. Then the neural-force kernel has lower leading cost per transition exactly when*

$$LC_S + C_V < LC_G. \tag{51.5}$$

*If training costs $C_{\text{train}}$ and the per-transition saving is $\Delta C = LC_G - (LC_S + C_V) > 0$, at least $C_{\text{train}}/\Delta C$ transitions are required to amortize training under these assumptions.*

*Proof.* Subtract (51.4) from (51.3). The neural route is cheaper precisely when the difference $\Delta C$ is positive. After $N$ transitions the accumulated saving is $N\Delta C$; it covers training if and only if $N\Delta C \geq C_{\text{train}}$. □

The equal-efficiency premise in Proposition 51.1 is deliberately strong. A useful experiment must instead compare ESS per wall-clock second after charging training and fallback. The proposition only explains why expensive filtering scores make the method worth investigating.

## 51.2 Phase-space Metropolis correction

The target of one HMC transition is the joint density

$$\rho_F(u,p) = Z_H^{-1} \exp\{-H_F(u,p)\}. \tag{51.6}$$

Marginalizing $p$ recovers the posterior proportional to $e^{-U_F(u)}$. Before each transition, momentum is refreshed from $\mathcal{N}(0,M)$. Proposal validity is easiest to state for a deterministic phase-space map.

Let $R(u,p) = (u,-p)$ be the momentum flip. Let $\Phi$ be a differentiable bijection of phase space and define the proposal map $F = R \circ \Phi$. The following proposition gives the exact conditions behind the familiar HMC acceptance probability.

**Proposition 51.2** (Deterministic involutive Metropolis proposal)**.** *Assume $F : \mathbb{R}^{2d} \to \mathbb{R}^{2d}$ is a differentiable involution, $F \circ F = I$, and $|\det DF(x)| = 1$ for every $x$. From state $x = (u,p)$, propose $x' = F(x)$ and accept with*

$$\alpha(x) = \min\{1, \rho_F(x')/\rho_F(x)\} = \min\{1, \exp[-H_F(x') + H_F(x)]\}. \tag{51.7}$$

*Then the accept/reject kernel is reversible with respect to $\rho_F$ and hence leaves $\rho_F$ invariant.*

*Proof.* For any measurable sets $A, B$, the accepted flow from $A$ to $B$ is

$$\int_A \rho_F(x)\alpha(x)\mathbf{1}\{F(x) \in B\}\,dx = \int_A \min\{\rho_F(x), \rho_F(Fx)\}\mathbf{1}\{F(x) \in B\}\,dx. \tag{51.8}$$

Change variables $y = F(x)$. Involution gives $x = F(y)$ and the unit-Jacobian assumption gives $dx = dy$. The expression becomes

$$\int_B \min\{\rho_F(Fy), \rho_F(y)\}\mathbf{1}\{F(y) \in A\}\,dy, \tag{51.9}$$

which is the accepted flow from $B$ to $A$. Rejections remain at the same state and are automatically balanced. Thus detailed balance holds. Taking $A$ to be the full state space proves invariance. □

**Remark 51.1** (Terms in the endpoint correction)**.** *Equation* (51.7) *contains* $K(p') - K(p)$ *as well as* $U_F(u') - U_F(u)$*. A correction based only on* $\ell_F(u') - \ell_F(u)$ *is generally wrong. Momentum cancels only for a proposal that preserves kinetic energy exactly, which ordinary leapfrog does not guarantee.*

**Proposition 51.3** (Omitting endpoint kinetic energy is wrong in general)**.** *There exist volume-preserving involutive proposals for which the position-only acceptance*

$$\alpha_U(x) = \min\{1, \exp[-U_F(u') + U_F(u)]\} \tag{51.10}$$

*does not leave* (51.6) *invariant.*

*Proof.* Take $d = 1$, $U_F(u) = u^2/2$, $K(p) = p^2/2$, and the involution $F(u, p) = (p, u)$. Its Jacobian determinant is $-1$. Because $H_F(Fx) = H_F(x)$, the correct acceptance is one. The position-only rule is $\min\{1, \exp[-p^2/2 + u^2/2]\}$ and rejects a set of positive Gaussian measure, for example whenever $|p| > |u|$. The reverse move from $(p, u)$ uses the reciprocal position ratio. The accepted forward mass is proportional to

$$e^{-(u^2+p^2)/2} \min\{1, e^{(u^2-p^2)/2}\}, \tag{51.11}$$

while the accepted reverse mass is proportional to

$$e^{-(u^2+p^2)/2} \min\{1, e^{(p^2-u^2)/2}\}. \tag{51.12}$$

They differ whenever $|u| \neq |p|$, so detailed balance fails. The missing kinetic term is therefore not optional. □

## 51.3 A sufficient learned-force construction

The cleanest construction learns only a position potential $\widehat{U}_\phi(u)$. Its force

$$g_\phi(u) = \nabla \widehat{U}_\phi(u) \tag{51.13}$$

may be inaccurate. It is frozen before warmup used for inference and remains fixed throughout the retained chain. Define a kick, drift, and one symmetric step by

$$K_a(u, p) = (u, p - a g_\phi(u)), \tag{51.14}$$
$$D_b(u, p) = (u + bM^{-1}p, p), \tag{51.15}$$
$$\Psi_\epsilon = K_{\epsilon/2} \circ D_\epsilon \circ K_{\epsilon/2}. \tag{51.16}$$

The rightmost map acts first.

**Lemma 51.1** (Kick and drift are volume-preserving)**.** *If $g_\phi$ is continuously differentiable, both maps in (51.14)–(51.15) have Jacobian determinant one. Consequently, $\Psi_\epsilon$ and every finite composition of its steps are volume-preserving.*

*Proof.* Their Jacobians have block form

$$DK_a(u,p) = \begin{pmatrix} I & 0 \\ -aDg_\phi(u) & I \end{pmatrix}, \qquad DD_b(u,p) = \begin{pmatrix} I & bM^{-1} \\ 0 & I \end{pmatrix}. \tag{51.17}$$

Both are block triangular with identity diagonal blocks, hence determinant one. Determinants multiply under composition. □

**Lemma 51.2** (Momentum-flip reversibility)**.** *For the momentum flip $R(u,p) = (u,-p)$,*

$$R \circ \Psi_\epsilon \circ R = \Psi_\epsilon^{-1} = \Psi_{-\epsilon}. \tag{51.18}$$

*The same identity holds for $\Phi_{\epsilon,L} = \Psi_\epsilon^L$.*

*Proof.* Direct substitution gives $RK_aR = K_{-a} = K_a^{-1}$ and $RD_bR = D_{-b} = D_b^{-1}$. Therefore

$$R\Psi_\epsilon R = K_{-\epsilon/2} \circ D_{-\epsilon} \circ K_{-\epsilon/2} \tag{51.19}$$

$$= \left(K_{\epsilon/2} \circ D_\epsilon \circ K_{\epsilon/2}\right)^{-1}. \tag{51.20}$$

For $L$ steps, $R\Psi_\epsilon^L R = (R\Psi_\epsilon R)^L = (\Psi_\epsilon^{-1})^L = (\Psi_\epsilon^L)^{-1}$. □

**Proposition 51.4** (Exact invariance for a frozen learned position force)**.** *Let $\Phi_{\epsilon,L} = \Psi_\epsilon^L$ use any fixed continuously differentiable $g_\phi : \mathbb{R}^d \to \mathbb{R}^d$ in the symmetric construction above. The force need not equal $\nabla U_F$. Define $F = R \circ \Phi_{\epsilon,L}$ and accept $F(u,p)$ with the true joint-energy probability (51.7). Then the transition leaves (51.6) invariant. After momentum refresh, its position marginal leaves the deterministic filter-defined posterior proportional to $e^{-U_F(u)}$ invariant.*

*Proof.* Lemma 51.1 and the definition of $R$ give

$$|\det D\Phi_{\epsilon,L}| = 1, \qquad |\det DR| = 1. \tag{51.21}$$

By Lemma 51.2,

$$F^2 = R\Phi_{\epsilon,L}R\Phi_{\epsilon,L} = \Phi_{\epsilon,L}^{-1}\Phi_{\epsilon,L} = I. \tag{51.22}$$

Proposition 51.2 therefore proves joint invariance. Momentum refresh draws from the conditional momentum factor of (51.6) and also preserves the joint density. Marginalization over $p$ gives the stated posterior invariance. □

**Remark 51.2** (Conservative learning is useful but not required for this proof)**.** *Writing $g_\phi = \nabla \widehat{U}_\phi$ guarantees a conservative learned force and permits scalar value/gradient training. The shear and reversibility proof above actually needs only a frozen position-only $g_\phi$. The stronger potential representation remains preferable because it constrains learning, supports energy diagnostics, and prevents accidental momentum dependence.*

**Proposition 51.5** (Surrogate error affects efficiency, not this invariant target)**.** *Under Proposition 51.4, replacing $g_\phi$ by any other fixed finite position-only force does not change the invariant target. However, if the resulting proposal has large true energy error $\Delta H_F = H_F(Fx) - H_F(x)$ on typical states, its acceptance is small for positive $\Delta H_F$, so it can have arbitrarily poor efficiency.*

*Proof.* The invariance proof uses only the map properties, not equality between $g_\phi$ and $\nabla U_F$. Acceptance is $\min\{1, e^{-\Delta H_F}\}$. For any $B > 0$, states with $\Delta H_F \geq B$ have acceptance at most $e^{-B}$. Thus invariance does not bound mixing away from zero. □

## 51.4 Limits of the broader learned-Hamiltonian argument

The method in Dhulipala et al. [2022] learns a scalar Hamiltonian from both position and momentum, obtains its vector field by differentiation, advances that field using a synchronized-leapfrog formula, and applies a true endpoint Hamiltonian test. Its Sections 3.2–3.3 and Algorithms 3.2–3.3 are the relevant source anchors. The paper appeals to Liouville's theorem and observed forward/reverse trajectories. Those facts do not by themselves prove the discrete proposal required by Proposition 51.2.

The issue is mathematical, not terminological. There are three different claims:

1. a Hamiltonian vector field has zero divergence and its exact flow preserves phase volume;
2. an exact Hamiltonian flow is reversible under momentum flip only when the Hamiltonian has the appropriate momentum symmetry;
3. a numerical map must itself be volume-preserving and reversible, or its actual proposal/Jacobian terms must enter MH.

None of these claims implies the next without additional assumptions.

**Proposition 51.6** (Liouville volume preservation does not prove time reversal)**.** *There are smooth Hamiltonians whose exact flows preserve phase volume but are not reversible under* $R(u,p) = (u,-p)$.

*Proof.* Take one dimension and

$$\widehat{H}(u,p) = \tfrac{1}{2}u^2 + \tfrac{1}{2}p^2 + cp, \qquad c \neq 0. \tag{51.23}$$

Hamilton's equations are $\dot{u} = p + c$ and $\dot{p} = -u$. Their vector field has divergence $\partial_u(p+c) + \partial_p(-u) = 0$, so Liouville's formula gives unit volume. But $\widehat{H}(u,-p) = \widehat{H}(u,p) - 2cp$, so the Hamiltonian is not even in momentum. If $(u(t),p(t))$ is a solution, the putative reversed path $(u(-t),-p(-t))$ has first derivative $-\dot{u}(-t) = -(p(-t)+c)$, whereas the equation evaluated on the reversed state requires $-p(-t)+c$. They differ by $2c$. Momentum-flip reversibility fails although volume is preserved. □

**Proposition 51.7** (Momentum evenness implies exact-flow reversibility)**.** *If a smooth Hamiltonian satisfies* $\widehat{H}(u,-p) = \widehat{H}(u,p)$*, then, for every* $t$ *for which its exact flow* $\varphi_t$ *is defined,*

$$R\varphi_t R = \varphi_{-t} = \varphi_t^{-1}. \tag{51.24}$$

*Proof.* Evenness in $p$ makes $\nabla_p \widehat{H}(u,p)$ odd in $p$ and $\nabla_u \widehat{H}(u,p)$ even in $p$. If $z(t) = (u(t),p(t))$ solves Hamilton's equations, define $\widetilde{z}(t) = Rz(-t) = (u(-t),-p(-t))$. Differentiating and applying those parities shows

$$\dot{\widetilde{u}}(t) = -\nabla_p \widehat{H}(u(-t),p(-t)) = \nabla_p \widehat{H}(\widetilde{u}(t),\widetilde{p}(t)), \tag{51.25}$$

$$\dot{\widetilde{p}}(t) = \dot{p}(-t) = -\nabla_u \widehat{H}(\widetilde{u}(t),\widetilde{p}(t)). \tag{51.26}$$

Thus $\widetilde{z}$ is the Hamiltonian solution starting from $Rz(0)$. Uniqueness of the ODE solution yields the identity. □

The latent-HNN architecture in the cited paper takes $(u, p)$ jointly as input and does not impose exact evenness in $p$. Consequently, Proposition 51.7 is unavailable without an added architectural constraint or a different reversal operation.

There is also an integrator mismatch. The position-Verlet formula used in the paper's Algorithm 3.2 is derived for a separable Hamiltonian $\widehat{U}(u) + K(p)$ with known quadratic $K$. If a learned $\widehat{H}(u, p)$ has general momentum dependence, then $\dot{u} = \nabla_p \widehat{H}(u, p)$ need not equal $M^{-1}p$. In the paper's notation, the executed position update is

$$u_i(t + \epsilon) = u_i(t) + \frac{\epsilon}{m_i} p_i(t) - \frac{\epsilon^2}{2m_i} \frac{\partial \widehat{H}}{\partial u_i}(u(t), p(t)). \tag{51.27}$$

Its leading velocity is $p_i/m_i$, not the general learned velocity $\partial \widehat{H}/\partial p_i$. Unless $\nabla_p \widehat{H}(u, p) = M^{-1}p$, the executed update is not an integrator of the learned Hamiltonian vector field it claims to approximate. A label such as "synchronized leapfrog" does not prove that the resulting discrete map is symplectic or reversible for that general learned Hamiltonian.

**Proposition 51.8** (The ordinary energy ratio is incomplete for a general map)**.** *Let $F$ be a differentiable involution but do not assume it preserves volume. The deterministic Metropolis acceptance that balances $\rho_F$ is*

$$\alpha_J(x) = \min\left\{1, \frac{\rho_F(Fx)|\det DF(x)|}{\rho_F(x)}\right\}. \tag{51.28}$$

*Thus the energy-only ratio (51.7) is justified only when the absolute Jacobian determinant is one.*

*Proof.* The accepted mass density sent from $x$ to $y = F(x)$ is $\rho_F(x)\alpha_J(x)dx$. Since $dy = |\det DF(x)|dx$, balance with the reverse accepted mass requires

$$\rho_F(x)\alpha_J(x) = \rho_F(Fx)\alpha_J(Fx)|\det DF(x)|. \tag{51.29}$$

Involution implies $|\det DF(Fx)| = |\det DF(x)|^{-1}$. Substituting (51.28) on both sides gives the same minimum of $\rho_F(x)$ and $\rho_F(Fx)|\det DF(x)|$, proving balance. Removing the Jacobian is valid only when it equals one. □

For a proposal that is neither an involution nor otherwise supplied with a tractable reverse proposal density, even (51.28) is not enough. One must construct an involutive extension, randomize over forward/reverse directions with the right law, or use the full Hastings ratio. Therefore "evaluate the true posterior at the endpoint" is not a general correctness theorem.

## 51.5 True-energy and delayed-acceptance kernels

Two valid algorithms are often conflated.

**One-stage true-energy correction.** Generate the deterministic learned-force proposal of Section 51.3 and apply only (51.7). This is the preferred first BayesFilter implementation because its proof is direct.

**Two-stage delayed acceptance.** Let $\widehat{\rho}(x) \propto e^{-\widehat{H}(x)}$ be a cheap surrogate joint density, and let $q(y \mid x)$ be a proposal kernel that is reversible with respect to $\widehat{\rho}$. Stage one accepts the proposal using the surrogate kernel; conditional on that acceptance, stage two accepts with

$$\alpha_2(x, y) = \min\left\{1, \frac{\rho_F(y)\widehat{\rho}(x)}{\rho_F(x)\widehat{\rho}(y)}\right\}. \tag{51.30}$$

**Proposition 51.9** (Two-stage delayed-acceptance balance)**.** *Suppose the stage-one accepted proposal kernel satisfies*

$$\widehat{\rho}(x)q(y \mid x)\alpha_1(x, y) = \widehat{\rho}(y)q(x \mid y)\alpha_1(y, x). \tag{51.31}$$

*Then accepting a stage-one move with* (51.30) *yields a kernel reversible with respect to* $\rho_F$*.*

*Proof.* For $x \neq y$, multiply the forward accepted flow by its reverse counterpart's denominator:

$$\rho_F(x)q(y \mid x)\alpha_1(x, y) \min\left\{1, \frac{\rho_F(y)\widehat{\rho}(x)}{\rho_F(x)\widehat{\rho}(y)}\right\} \tag{51.32}$$

$$= q(y \mid x)\alpha_1(x, y) \min\left\{\rho_F(x), \frac{\rho_F(y)\widehat{\rho}(x)}{\widehat{\rho}(y)}\right\}. \tag{51.33}$$

Use (51.31) to replace $q(y \mid x)\alpha_1(x, y)$ by $\widehat{\rho}(y)q(x \mid y)\alpha_1(y, x)/\widehat{\rho}(x)$. The result is

$$q(x \mid y)\alpha_1(y, x) \min\left\{\frac{\rho_F(x)\widehat{\rho}(y)}{\widehat{\rho}(x)}, \rho_F(y)\right\}, \tag{51.34}$$

which is the reverse accepted flow. Rejections balance on the diagonal. □

**Proposition 51.10** (A stage-two ratio cannot replace a missing stage one)**.** *The correction* (51.30) *alone does not generally target* $\rho_F$ *when the proposal kernel is not already reversible with respect to* $\widehat{\rho}$*.*

*Proof.* Take $\rho_F = \widehat{\rho}$ on a three-state space $\{0, 1, 2\}$. Then (51.30) equals one for every move. Let the proposal deterministically cycle $0 \mapsto 1 \mapsto 2 \mapsto 0$. For a nonuniform $\rho_F$, for example masses $(1/2, 1/3, 1/6)$, the resulting chain merely rotates those masses and does not preserve $\rho_F$. Thus the stage-two ratio cannot repair an arbitrary proposal. The missing premise is exactly the stage-one balance identity (51.31). □

This proposition identifies the local historical implementation problem. In the audited earlier `dsge_hmc.estimation.DelayedAcceptanceHMC`, the executed logic is schematically

$$y, \widehat{\alpha}_1 \leftarrow \text{surrogate_leapfrog}(x), \tag{51.35}$$

$$\text{adapt}(\widehat{\alpha}_1) \quad \text{during warmup}, \tag{51.36}$$

$$\alpha_{\text{executed}}(x, y) = \min\left\{1, \frac{\rho_F(y)\widehat{\rho}(x)}{\rho_F(x)\widehat{\rho}(y)}\right\}, \tag{51.37}$$

with no Bernoulli accept/reject using $\widehat{\alpha}_1$ between the first and third lines. Thus the surrogate statistic affects adaptation but is not the stage-one kernel assumed by (51.31). The documentation claims the two-stage theorem while the executed kernel omits a premise of Proposition 51.9. The implementation is therefore wrong relative to that exact-posterior claim. The clean repair is either the one-stage true joint-energy kernel of Proposition 51.4, or a complete two-stage kernel that retains both endpoint momenta and performs both acceptances.

## 51.6 The filtering boundary

The proposal proof and the target definition are separate obligations. Even a perfect proposal kernel is exact only for the scalar placed in its MH ratio. This section states that boundary precisely.

Let the latent state-space model be

$$x_0 \sim \mu_u, \tag{51.38}$$
$$x_t \mid x_{t-1} \sim f_u(\cdot \mid x_{t-1}), \tag{51.39}$$
$$y_t \mid x_t \sim g_u(\cdot \mid x_t), \tag{51.40}$$

with exact likelihood

$$L(u) = \int \mu_u(dx_0) \prod_{t=1}^{T} f_u(dx_t \mid x_{t-1}) g_u(y_t \mid x_t). \tag{51.41}$$

The exact unconstrained posterior is

$$\pi(u \mid y) \propto p(u) L(u) e^{J(u)}. \tag{51.42}$$

### 51.6.1 Deterministic exact and approximate filters

Let a deterministic filter $F$ return a positive scalar $L_F(u)$. Define

$$\pi_F(u \mid y) \propto p(u) L_F(u) e^{J(u)}. \tag{51.43}$$

**Proposition 51.11** (Exactness relative to the declared deterministic filter)**.** *If the end-point correction in Proposition 51.4 evaluates* $U_F(u) = -\log p(u) - \log L_F(u) - J(u)$ *deterministically, the kernel leaves* $\pi_F$ *invariant. It leaves the exact posterior* (51.42) *invariant by this argument only if* $L_F(u) = cL(u)$ *almost everywhere for a constant* $c > 0$ *independent of* $u$*.*

*Proof.* The first claim is Proposition 51.4 with the named potential. The normalized densities $\pi_F$ and $\pi$ are equal if their unnormalized likelihood factors differ by a positive $u$-independent constant. Conversely, if the priors and charts agree and the two normalized posterior densities are equal wherever the prior is positive, then their likelihood ratio is the ratio of their normalizing constants and is therefore constant almost everywhere. □

Thus a Kalman filter for the declared linear Gaussian model can supply the exact $L$. A UKF, SGQF, assumed-density filter, deterministic particle flow, or fixed low-rank filter ordinarily supplies a named $L_F \neq L$. Corrected neural-force HMC can be exact for that approximate-filter posterior, not for the latent model posterior. The MH step corrects the trajectory surrogate; it does not correct the filtering approximation.

### 51.6.2 Fixed randomness and relaxed filters

Suppose a randomized program returns $\widehat{L}(u;\omega)$ for random numbers $\omega$. Freezing one realization $\omega_0$ makes

$$\pi_{\omega_0}(u) \propto p(u) \widehat{L}(u;\omega_0) e^{J(u)} \tag{51.44}$$

a deterministic target, provided the executed branch is well defined.

**Proposition 51.12** (What frozen randomness does and does not prove)**.** *Under the conditions of Proposition 51.4, using the same $\omega_0$ for every endpoint value can produce a valid HMC kernel for $\pi_{\omega_0}$. Unbiasedness $\mathbb{E}_\omega[\widehat{L}(u;\omega)] = L(u)$ does not imply $\pi_{\omega_0} = \pi$ for any fixed realization, nor does it imply $\mathbb{E}[\log \widehat{L}] = \log L$.*

*Proof.* With $\omega_0$ frozen, the previous deterministic-target proof applies to (51.44). For the nonimplications, take $\omega \sim \mathrm{Unif}[-1, 1]$ and

$$\widehat{L}(u;\omega) = L(u)\{1 + a(u)\omega\}, \qquad |a(u)| < 1, \tag{51.45}$$

where $a(u)$ is nonconstant. The estimator is positive and unbiased because $\mathbb{E}\omega = 0$. For any fixed $\omega_0 \neq 0$, its realized ratio to $L(u)$ is $1 + a(u)\omega_0$, which is nonconstant, so the realized posterior generally differs from $\pi$. Moreover, strict concavity of log gives $\mathbb{E}\log \widehat{L} \leq \log \mathbb{E}\widehat{L} = \log L$, with strict inequality for a nondegenerate estimator. Neither desired conclusion follows from likelihood unbiasedness alone. □

A continuous relaxation of resampling, a deterministic optimal-transport replacement, or a frozen branch may improve differentiability. It defines the scalar it executes. Unless that scalar equals the original likelihood up to a constant, it is a relaxed or branch-conditional target; the neural-force MH step cannot silently upgrade it.

### 51.6.3 Pseudo-marginal likelihoods

Assume $\widehat{L}(u;\omega) \geq 0$. Let $\nu(d\omega)$ be a dominating measure and write $m(\omega \mid u)$ for the density, with respect to $\nu$, of the auxiliary random variables used by the estimator. Define the extended density with respect to $du\,\nu(d\omega)$ by

$$\widetilde{\pi}(u,\omega) = \frac{p(u)e^{J(u)}\widehat{L}(u;\omega)m(\omega \mid u)}{Z}. \tag{51.46}$$

**Proposition 51.13** (Pseudo-marginal marginalization)**.** *If $\widehat{L}$ is nonnegative and unbiased,*

$$\int \widehat{L}(u;\omega)m(\omega \mid u)\nu(d\omega) = L(u), \tag{51.47}$$

*then the $u$-marginal of (51.46) is the exact posterior (51.42).*

*Proof.* Integrating (51.46) over $\omega$ gives $Z^{-1}p(u)e^{J(u)}L(u)$ by unbiasedness, which is the normalized exact posterior. □

This proposition does not authorize naive HMC on $-\log \widehat{L}(u;\omega)$ with fresh $\omega$ at each force or endpoint evaluation. The Markov state is $(u,\omega)$, and a valid kernel must preserve the extended density. If $m(\omega \mid u)$ depends on $u$, its log density is also part of the conditional $u$-energy. More explicitly, conditional on fixed $\omega$, that energy contains

$$-\log p(u) - J(u) - \log \widehat{L}(u;\omega) - \log m(\omega \mid u). \tag{51.48}$$

Thus using only $-\log \widehat{L}(u;\omega)$ differentiates and corrects the wrong extended scalar. Discrete resampling variables do not automatically admit Hamiltonian dynamics; refreshing them requires an MH/Gibbs/pseudo- marginal step with the appropriate forward and reverse laws. A neural force may propose $u$ conditional on retained $\omega$, but correctness then depends on a complete extended-state kernel, not on Proposition 51.4 applied only to $u$.

**Proposition 51.14** (Fresh noisy endpoint values do not define the deterministic MH ratio)**.** *Replacing $L_F(u)$ in (51.7) by independent fresh estimates at the current and proposed states is not, in general, a valid implementation of either deterministic HMC for $\pi$ or pseudo-marginal MCMC for $\widetilde{\pi}$.*

*Proof.* The deterministic proof fails because the acceptance probability is random conditional on $(u, p)$ and is not a function of one fixed target density. The pseudo-marginal proof fails because the estimator realization at the current state is part of the extended state and must be retained on rejection; redrawing it changes the denominator without an extended-state proposal ratio. Only special noisy-MCMC constructions with their own balance proof avoid this conclusion for $\widetilde{\pi}$. □

### 51.6.4 Filtering target taxonomy

The boundary can be summarized without ambiguity:

Table 51.1: What a corrected neural-force proposal targets.

| Endpoint scalar | Markov state | Supported target | Unsupported upgrade |
|---|---|---|---|
| Exact deterministic $L(u)$ | $(u, p)$ | Exact latent-model posterior | None beyond finite-run convergence |
| Deterministic approximate $L_F(u)$ | $(u, p)$ | Exact for the named filter-defined posterior | Exact latent-model posterior |
| Fixed $\widehat{L}(u; \omega_0)$ | $(u, p)$ with frozen $\omega_0$ | Exact for the fixed-randomness scalar | Pseudo-marginal exactness |
| Relaxed or learned filter scalar | $(u, p)$ | Exact for the relaxed/learned target | Original-filter or latent-model posterior |
| Unbiased nonnegative $\widehat{L}$ | $(u, p, \omega)$ under a proved extended kernel | Exact marginal posterior under pseudo-marginal assumptions | Naive HMC with fresh estimator noise |
| Fresh noisy log likelihood | Incompletely specified | No general exactness claim | Deterministic or pseudo-marginal HMC |

## 51.7 Training a useful position potential

Correctness does not require an accurate force, but efficiency does. A robust design uses a known baseline plus a learned residual. In affine or NeuTra coordinates $z$, write

$$\widehat{U}_\phi(z) = \tfrac{1}{2} z^\top M_0^{-1} z + r_\phi(z). \tag{51.49}$$

The zero residual is already a meaningful no-training baseline. It tests whether the chosen coordinates have made the target nearly Gaussian. Learning $r_\phi$ should be attempted only if this baseline leaves a material force or acceptance gap.

Given training triples $(z_i, U_F(z_i), \nabla U_F(z_i))$, a typical scalar potential loss is

$$\mathcal{L}(\phi) = \lambda_V \frac{1}{N} \sum_i \Big( \widehat{U}_\phi(z_i) - U_F(z_i) - c \Big)^2 + \lambda_G \frac{1}{N} \sum_i \Big\| \nabla \widehat{U}_\phi(z_i) - \nabla U_F(z_i) \Big\|_{W_i}^2, \tag{51.50}$$

where $c$ absorbs the irrelevant potential offset and $W_i \succeq 0$ records the declared scaling or robust weighting. Training on values alone may fit a potential while missing its force. Training on force alone determines the potential only up to a constant, which is harmless for leapfrog but still requires heldout energy diagnostics.

**Proposition 51.15** (Potential offsets do not affect the learned-force kernel)**.** *Replacing $\widehat{U}_\phi$ by $\widehat{U}_\phi + c$ for any constant $c$ leaves every learned-force proposal and the true endpoint acceptance unchanged.*

*Proof.* The gradient of a constant is zero, so every kick is unchanged. The MH ratio uses $H_F$, not $\widehat{U}_\phi$, so its value is also unchanged. □

Training points must cover posterior-relevant trajectories, not merely a large axis-aligned box. Useful sources include tuned ordinary-HMC warmup states, NeuTra base draws mapped into source coordinates, accepted and rejected endpoints, and selected interior points. Validation must be split by trajectory or chain rather than randomly splitting highly correlated points. Tail and mode coverage remain explicit failure modes.

The network is frozen before the kernel is used for inference. Online updates during retained sampling make the proposal time-inhomogeneous and invalidate the fixed-kernel proof unless an adaptive-MCMC theorem with diminishing adaptation and containment is supplied. The safe operational response to a large energy-error alarm is to use a separately valid fallback transition and record the event, not silently retrain inside the retained chain.

**Proposition 51.16** (Mixtures of valid fallback kernels remain valid)**.** *Let $P_1$ be the corrected learned-force kernel and $P_2$ an ordinary HMC or random-walk MH kernel, each invariant for the same $\pi_F$. For a fixed $a \in [0,1]$, the mixture $P = aP_1 + (1-a)P_2$ is invariant for $\pi_F$.*

*Proof.* Linearity of integration gives $\pi_F P = a\pi_F P_1 + (1-a)\pi_F P_2 = a\pi_F + (1-a)\pi_F = \pi_F$. □

A state-dependent fallback probability is not covered by this proposition; it requires its own balance argument. One safe design draws the kernel identity independently before the transition. A second safe pattern rejects a proposal using a pair-symmetric veto, for example $\mathbf{1}\{|H_F(Fx) - H_F(x)| \le B\}$, because the absolute energy difference is the same for $x$ and $Fx$ when $F$ is an involution; the ordinary MH balance proof then applies on the surviving pairs. A one-sided alarm or a fallback selected after inspecting only the proposed endpoint is not covered. After a diagnostic rejection, a separately scheduled fixed-mixture fallback may be used on a later transition.

## 51.8 Algorithm and implementation contract

For a deterministic filter-defined potential, the minimal valid algorithm is:

The implementation must preserve the following contracts.

1. **Target identity.** Endpoint values include the same prior, parameter chart, Jacobian, data, filter, branch policy, and numerical settings as the declared posterior.

2. **Joint energy.** Initial and final momenta are retained until $\Delta H$ is computed. A posterior-only correction is forbidden.

**Algorithm 12** True-energy-corrected neural-force HMC

**Require:** Current $u$ and cached $U_F(u)$; frozen $g_\phi$; $M \succ 0$; step size $\epsilon$; steps $L$.

Draw $p \sim \mathcal{N}(0, M)$ and store $p_0 \leftarrow p$, $u_0 \leftarrow u$.
$p \leftarrow p - (\epsilon/2) g_\phi(u)$.
**for** $j = 1, \ldots, L$ **do**
    $u \leftarrow u + \epsilon M^{-1} p$.
    **if** $j < L$ **then**
        $p \leftarrow p - \epsilon g_\phi(u)$.
    **end if**
**end for**
$p \leftarrow p - (\epsilon/2) g_\phi(u)$.
Set $(u', p') \leftarrow (u, -p)$ and evaluate the new true value $U_F(u')$.
$\Delta H \leftarrow U_F(u') + K(p') - U_F(u_0) - K(p_0)$.
Accept $(u', U_F(u'))$ with probability $\min\{1, e^{-\Delta H}\}$; otherwise retain $(u_0, U_F(u_0))$.

3. **Frozen proposal.** Network weights, normalization, affine or NeuTra map, mass matrix, $\epsilon$, and $L$ are fixed during retained sampling. Warmup adaptation ends before evidence draws.

4. **Symmetric map.** The executed update is exactly the symmetric kick–drift–kick composition. Early termination, clipping, projection, state-dependent step size, non-symmetric finite-value rescue, or a direct neural state-update map needs a new proof.

5. **Deterministic endpoint.** Deterministic filters must reproduce the same value on repeat calls. Stochastic filters must use a separately specified extended-state construction.

6. **Precision and failure.** A mathematically declared out-of-support endpoint has $U_F = +\infty$ and rejects. By contrast, an undefined learned force, failed factorization, overflow, or numerically indeterminate filter value means that the executed proposal or target is not the proved map. It must emit a failed-transition diagnostic and block promotion; it must not be hidden as ordinary rejection. Replacing any failed true value by a convenient neural value changes the kernel and is forbidden.

7. **Batching and XLA.** Chain and training batches should be TensorFlow/XLA operations on the BayesFilter route. Batching is an efficiency requirement; it does not alter the proof.

## 51.9 Tests that answer the mathematical question

Unit tests should target the proof assumptions rather than only finite output.

**Map tests.** For random finite phase points, verify

$$\|R\Phi_{\epsilon,L} R\Phi_{\epsilon,L}(x) - x\| \tag{51.51}$$

at a dtype-appropriate tolerance. Check a small-dimensional autodiff Jacobian against determinant magnitude one. These tests do not prove the property in all dimensions, but they catch implementation drift from the proved formula.

**Acceptance tests.** On a Gaussian target with an exact force, acceptance should approach one as $\epsilon \to 0$, but need not equal one for finite $\epsilon$. Tests that require identically one because the surrogate equals the target are testing the wrong statement: leapfrog still has discretization error. A deliberately biased force should be tested for agreement with Gaussian moments under a long-enough, irreducible corrected-chain experiment, while recognizing that poor mixing can make a finite run inconclusive.

**Counterexample tests.** Test three deliberate defects: remove the final kinetic term; make the force momentum-dependent; and omit stage-one delayed acceptance. Small reference tests should then detect stationary-moment or detailed-balance failure. These negative tests guard the exact mistakes documented in this chapter.

**Filtering tests.** For deterministic filters, repeat endpoint values and compare direct versus cached evaluations. For an exact LGSSM, compare the corrected chain with the analytic/Kalman posterior. For UKF or SGQF, label results as filter-defined and compare to an independent higher-fidelity likelihood only as a sensitivity study. For a particle filter, reject any harness that refreshes estimator randomness inside a nominally deterministic HMC ratio without an extended- state manifest.

## 51.10 Scientific evaluation ladder

The smallest informative comparison is not HNN-HMC versus an untuned HMC. Use the following matched ladder:

1. tuned ordinary HMC on $U_F$;
2. affine or NeuTra coordinates with ordinary true gradients;
3. the same coordinates with the zero-residual Gaussian force from (51.49);
4. the trained residual potential with true-energy correction;
5. the trained route plus a predeclared fixed-probability valid fallback, if fallback is needed.

All arms use the same target, retained-draw policy, diagnostic thresholds, and hardware class. Training data generation and network training are charged in the amortized comparison. Report at least true endpoint values, learned-force calls, true-gradient calls, acceptance, modern rank/folded $\widehat{R}$, bulk and tail ESS, energy-error tails, fallback rate, wall time, and ESS per second.

Pointwise value RMSE, force cosine similarity, and validation loss are explanatory diagnostics. They can reject a clearly broken learner but cannot promote it. The promotion criterion is downstream corrected-chain validity and amortized efficiency against the tuned matched baselines. Passing one filter-defined posterior does not establish latent-model exactness or broad cross-model superiority.

## 51.11 Historical evidence and its proper interpretation

The earlier work in `/home/chakwong/python` contains several reusable artifacts. These include a scalar `PotentialEnergySurrogate`, a historical `DelayedAcceptanceHMC`, contract tests, and SGU result files. They provide useful knowledge transfer, but not a successful result for the method proposed here.

The preserved `sgu_surrogate_hmc.npz` records four chains of 500 draws, zero acceptance, ESS near four, extreme reported $\widehat{R}$, and negative mean learned-versus-true gradient cosine agreement. It is a failed surrogate candidate. The later `sgu_surrogate_dahmc.npz` preserves samples and wall times but omits the acceptance, $\widehat{R}$, ESS, energy, and proposal telemetry needed to establish correctness or efficiency. Neither artifact tests the corrected one-stage joint-energy kernel of Algorithm 12.

The cited paper reports large reductions in numerical target-gradient calls on several synthetic distributions, as well as improved ESS per gradient. It also uses a fallback for NUTS based on error monitoring. Those experiments support the research motivation. They do not prove that the discrete general L-HNN map is reversible and volume-preserving, nor do they cover filtering-target identity or supply BayesFilter evidence.

The official repository is archived. Its HMC code evaluates a true endpoint Hamiltonian, while its learned map uses the general $(u, p)$ network and the synchronized update discussed above. The implementation is a valuable source for auditing what was computed; it is not a proof of the missing discrete-map properties.

## 51.12 BayesFilter corrected-kernel experiments

**Retrospective tuning correction.** The experiments in this section are historical mechanics and fixed-kernel observations, not valid tuned performance comparisons. A later audit found that their fixed-grid selector did not use BayesFilter's native dual-averaging tuner, used short-chain $\widehat{R}$ in candidate selection, and did not bind a native mass artifact. Consequently every tuned runtime, seconds-per-ESS, speed, break-even, and performance-pass claim in the historical tables below is unsupported pending successful native retuning. In the native PP–UKF canary, both arms reached the prescribed acceptance interval, but the learned arm failed folded rank-normalized split $\widehat{R}$ and the exact arm failed modern $\widehat{R}$ plus an energy-tail veto. Nothing in this section promotes either PP–UKF kernel.

The July 2026 BayesFilter campaign tested the narrower construction proved in this chapter, not the historical delayed-acceptance implementation. Every tested arm used a frozen target-specific NeuTra chart, a frozen position-only scalar residual force, the symmetric map in Algorithm 12, and the complete true endpoint energy in the transformed coordinate. Each transition paid for one new value-only endpoint evaluation; archived traces reconstructed the full kinetic-plus- potential energy difference. Warm-up draws were retained. The sampler gate used the maximum of rank-normalized split and folded rank-normalized split $\widehat{R}$, with final thresholds $\widehat{R} \leq 1.01$, bulk ESS at least 1000, and tail ESS at least 400.

The campaign produced corrected-kernel results for five posterior-target configurations in four model families. Table 51.2 reports the learned-residual arm. The zero-residual Gaussian-force control also passed every one of these five cells. The truth-tail

column is the minimum parameter-wise two-sided empirical posterior tail probability of the generating value; it is a one-seed centrality diagnostic, not a frequentist calibration test.

| Target | $\epsilon, L$ | Accept. | max $\widehat{R}$ | bulk/tail ESS | min truth tail |
|---|---|---|---|---|---|
| LGSSM–KF | $0.8, 6$ | 0.678 | 1.00673 | 2069/3005 | 0.0664 |
| Predator–prey UKF | $0.2, 10$ | 0.970 | 1.00534 | 5188/2198 | 0.2062 |
| Predator–prey SGQF | $0.2, 10$ | 0.970 | 1.00179 | 6680/3187 | 0.2177 |
| Spatial SIR SGQF | $0.8, 10$ | 0.898 | 1.00055 | 5770/3519 | 0.3697 |
| Structural UKF | $0.2, 8$ | 0.896 | 1.00543 | 2204/2121 | 0.2777 |

Table 51.2: Historical one-seed learned-force observations produced with the superseded non-native tuner. They are preserved for provenance and are not promoted tuning, convergence, accuracy, or performance evidence.

Endpoint parity against the complete transformed target was at most $4.27 \times 10^{-14}$ across the five cells, and the archived full-energy reconstruction error was zero. The SIR and predator–prey cells passed their admitted physical-posterior-mean comparisons with tuned plain HMC. The structural raw-coordinate comparator remained unavailable because of the preserved source-geometry blocker; no substitute comparator was invented. Its scientific pass therefore rests on the predeclared truth-tail and sampler criteria plus the deterministic structural identity $k_t - \phi k_{t-1} - \gamma m_t^2 = 0$. No artificial process noise was added to $k_t$.

Only the LGSSM cell contained the complete matched performance ledger required by Section 51.10. For its learned arm, reuse-scenario seconds per minimum bulk ESS were 0.2483, versus 0.6193 for same-chart true-gradient HMC; sampling-only values were 0.1098 and 0.2279. This passes the campaign's descriptive performance screen. The zero-residual arm was descriptively better still, at 0.1367 and 0.06767, so the experiment does not establish that residual learning is needed once the NeuTra chart is strong. The other four cells did not preserve a matched true-gradient cost and amortization ledger. Their performance status is therefore *not demonstrated because the matched ledger is missing*, not an inferred performance pass or a claim that the candidate is intrinsically slow.

Eight further historical NeuTra configurations were audited: funnel, ill-conditioned Gaussian, German logistic regression, three NK targets, and two Rotemberg targets. They were not counted as corrected-kernel tests. Their exact historical targets or result notes often survived, but their selected frozen chart tensors did not. Retraining a new chart would answer a different question. The campaign consequently recorded eight cell-local `REQUALIFICATION_BLOCKED` outcomes rather than fabricating a replay. This is a knowledge-transfer and artifact-preservation failure, not evidence that the historical NeuTra results or the corrected kernel are wrong.

These historical experiments were later found to use an ad hoc fixed-grid HMC tuner rather than BayesFilter's native dual-averaging tuner. Their tuned runtime, seconds-per-ESS, speed, break-even, and performance-pass claims are therefore unsupported pending native retuning. A native PP–UKF canary did reach the prescribed acceptance interval for both arms, but the learned arm failed folded rank-normalized split R-hat and the exact arm failed both modern R-hat and an energy-tail veto. The historical fixed-kernel observations do not establish a valid tuned comparison. The resulting scientific statement is consequently narrower: the corrected position-only proposal mechanism is implemented

and worth further investigation, but current nonlinear evidence does not establish a promoted HNN–NeuTra sampler, calibrated performance, universal reliability, statistically supported superiority, residual-learning necessity, or default readiness.

## 51.13 Conclusions and nonclaims

Metropolis-corrected neural-force HMC is a good research direction for filtering posteriors because it can replace $L$ expensive filtering scores by $L$ cheap neural-force calls and one new true endpoint value. The valid core is narrower than the slogan:

$$\boxed{\text{frozen position-only symmetric proposal} + \text{ true joint endpoint energy.}} \tag{51.52}$$

Under the proved assumptions, force approximation affects efficiency rather than the invariant deterministic target. The result does not extend to any of the following constructions without additional analysis:

- an arbitrary neural state-update map;
- a momentum-dependent learned Hamiltonian without proved symmetry and integrator properties;
- a Metropolis correction that drops endpoint kinetic energy; or
- a nominal delayed-acceptance kernel that omits stage one.

Most importantly, exactness is relative to the endpoint scalar. Exact Kalman likelihood yields the corresponding latent-model posterior. UKF, SGQF, fixed branch, relaxed, or learned filtering likelihoods yield their named filter-defined posteriors. An unbiased particle likelihood can recover the exact posterior only through a valid extended-state pseudo-marginal kernel; fresh noisy endpoint estimates are not a substitute. No amount of accuracy in the neural force crosses that filtering boundary.

# Part XI

# Industrial Applications and Case Studies

# Chapter 52

# LGSSM Validation Ladder

The linear Gaussian state-space model is the first validation target for BayesFilter because it supplies an exact likelihood, controlled derivatives, and tractable regression tests. Every nonlinear or DSGE backend should be reduced to an LGSSM special case before it is trusted in HMC.

## 52.1 Evidence Already Available

The MacroFinance project provides the strongest local evidence stream:

- exact prediction-error likelihood formulas and analytic derivative notes;
- covariance, solve, Cholesky square-root, QR square-root, TensorFlow, and SVD-flavored backends;
- finite-difference and autodiff parity tests on small models;
- large-scale backend-ladder tests that report reconstruction errors, solve residuals, QR pivots, and finite derivative deltas;
- missing-data policy tests that fail unsupported sparse panels before the likelihood silently changes meaning.

This evidence justifies using the LGSSM as BayesFilter's regression oracle. It does not yet replace the BayesFilter-specific packaging and public API tests that will be needed when the code is moved into this project.

## 52.2 Validation Rungs

The LGSSM ladder is:

(i) fixed-parameter likelihood agreement against an independent Kalman implementation;

(ii) score agreement against finite differences and small-model autodiff;

(iii) Hessian agreement where the backend claims observed-information support;

(iv) backend parity among solve, Cholesky, QR, and TensorFlow paths;

(v) mask and mixed-frequency parity against dense special cases;

(vi) compiled/eager parity for the full value-and-gradient target;

(vii) short HMC plumbing checks with finite diagnostics;

(viii) medium and strict HMC runs only after the preceding rungs pass.

## 52.3 Interpretation

Passing the LGSSM ladder means the sampler interface, derivative convention, and linear algebra backends are coherent on an exact target. It does not prove that a nonlinear approximation is accurate, and it does not prove that a DSGE posterior has good HMC geometry. It is the floor, not the ceiling.

# Chapter 53

# Nonlinear SSM Validation Ladder

The nonlinear SSM ladder tests whether approximate nonlinear filtering can be used inside HMC before DSGE perturbation solvers and economic boundaries enter the picture. It should use small synthetic models with known parameters and independent references, not large application models.

## 53.1 Controlled Models

The first nonlinear validation suite is deliberately small. It is designed to test the filtering law, structural deterministic completion, and sigma-point moment approximation before DSGE perturbation solvers, client adapters, or HMC geometry enter the picture. The suite contains three first-rung models.

### 53.1.1 Model A: Affine Gaussian Structural Oracle

The affine oracle is a BayesFilter-local synthetic model:

$$x_t = (m_t, \ell_t)^\top, \qquad m_t = 0.35 m_{t-1} - 0.10 \ell_{t-1} + 0.25 \varepsilon_t, \qquad \ell_t = m_{t-1},$$

$$y_t = m_t + \eta_t, \qquad \varepsilon_t \sim N(0,1), \qquad \eta_t \sim N(0, 0.15^2).$$

The initial law is Gaussian and the deterministic coordinate $\ell_t$ stores the previous $m$. The oracle is the exact linear Gaussian Kalman likelihood after converting the structural law to its full-state LGSSM. The test must verify that SVD cubature, the historical diagnostic SVD-UKF branch, the promoted strict-SPD principal-square-root UKF branch, and SVD-CUT4 collapse to the same prediction-error likelihood and that deterministic-completion residuals are zero up to numerical tolerance.

### 53.1.2 Model B: Nonlinear Accumulation

The nonlinear accumulation model is a smooth BayesFilter-local structural fixture:

$$m_t = \rho m_{t-1} + \sigma \varepsilon_t, \qquad k_t = \alpha k_{t-1} + \beta \tanh(m_t),$$

$$y_t = m_t + k_t + \eta_t, \qquad \varepsilon_t \sim N(0,1), \qquad \eta_t \sim N(0, \sigma_\eta^2).$$

The first V1 defaults are

$$\rho = 0.70, \qquad \sigma = 0.25, \qquad \alpha = 0.55, \qquad \beta = 0.80, \qquad \sigma_\eta = 0.30.$$

This model tests nonlinear deterministic completion without a time-indexed transition hook. The reference oracle is a dense one-step Gaussian moment-projection quadrature: it integrates the augmented predictive Gaussian, computes the transformed prediction and observation moments, and then applies the same Gaussian update used by the sigma-point filters. It is not an exact nonlinear likelihood.

### 53.1.3 Model C: Nonlinear Growth Benchmark

The scalar nonlinear growth benchmark is the standard model associated with Kitagawa's Monte Carlo filtering examples [Kitagawa, 1996] and widely reused in sequential Monte Carlo discussions. Its time-inhomogeneous law is

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1+x_{t-1}^2} + 8\cos(1.2(t-1)) + \sigma_x\varepsilon_t,$$

$$y_t = \frac{x_t^2}{20} + \sigma_y\eta_t, \qquad \varepsilon_t, \eta_t \sim N(0,1).$$

Because the current BayesFilter structural TensorFlow transition does not pass an explicit time index, the V1 testing fixture uses an autonomous deterministic phase coordinate:

$$\tau_t = \tau_{t-1} + 1, \qquad \tau_1 = 1,$$

$$x_t = \frac{x_{t-1}}{2} + \frac{25x_{t-1}}{1+x_{t-1}^2} + 8\cos(1.2\tau_{t-1}) + \sigma_x\varepsilon_t.$$

The observation equation remains $y_t = x_t^2/20 + \sigma_y\eta_t$. The first defaults are $x_1 \sim N(0, 0.2)$, $\sigma_x = 1$, and $\sigma_y = 1$. As in Model B, the first oracle is dense one-step Gaussian moment projection, not the exact nonlinear likelihood.

### 53.1.4 Deferred Models

**Bearings-only tracking.** Gordon et al. [1993] and Arulampalam et al. [2002] are useful sources, but the test should wait until angle residual and wrapping policy are fixed.

**Radar range-bearing tracking.** This should follow the bearings-only residual policy and should add mixed-scale observation diagnostics.

**Stochastic volatility.** This row is admitted only after it declares the one cumulative likelihood target being benchmarked. For transformed actual SV that target is the exact transformed likelihood built from the normalizers of $z_t = \log(y_t^2)$, $z_t - \log(\beta^2) - h_t \sim \log(\chi_1^2)$. Same-target direct-likelihood quadrature and Zhao–Cui comparisons are admissible because they approximate that scalar. Generic sigma-point rows that assume additive observation covariance after $h(x_t)$, or that replace the observation law by Gaussian closure, do not benchmark the intended actual-SV target and are therefore not acceptable stand-ins for it.

Deferred means that the model is useful later, not that it is rejected. The first V1 suite stops at Models A–C so that value, score, branch, and benchmark tests share a stable set of fixtures.

## 53.2 Current V1 Filter Evidence

The first CPU nonlinear benchmark compares three current promoted-or-reference sigma-point value filters plus the historical diagnostic comparator:

$$\mathcal{B} = \{\text{SVD cubature}, \text{historical SVD-UKF}, \text{principal-square-root UKF}, \text{SVD-CUT4}\}.$$

For Model A the reference is the exact Kalman prediction-error likelihood after structural-to-LGSSM conversion. The benchmark rows must satisfy

$$|\ell_b(y_{1:T}) - \ell_{\mathrm{KF}}(y_{1:T})| \approx 0, \qquad b \in \mathcal{B},$$

up to floating-point tolerance, and the filtered means and covariances must match the exact Kalman recursion. This verifies that the nonlinear sigma-point code collapses to the correct affine Gaussian law.

For Models B and C the benchmark uses only a dense one-step Gaussian projection reference. If $Q_{\mathrm{dense}}$ denotes a tensor-product Gaussian rule and $\Pi_b$ denotes one of the sigma-point rules, the artifact reports

$$\left|\ell^{(1)}_{\Pi_b} - \ell^{(1)}_{Q_{\mathrm{dense}}}\right|, \qquad \left\|\hat{x}^{(1)}_{\Pi_b} - \hat{x}^{(1)}_{Q_{\mathrm{dense}}}\right\|_2, \qquad \left\|\hat{P}^{(1)}_{\Pi_b} - \hat{P}^{(1)}_{Q_{\mathrm{dense}}}\right\|_F.$$

These are approximation-quality diagnostics for the first filtering step, not exact full-likelihood errors. The current artifact therefore records the full Models B–C log likelihoods as filter outputs and blocks any claim that those values have been certified against an exact nonlinear likelihood.

The same artifact records the shared branch diagnostics

$$r_{\mathrm{det}}, \quad r_{\mathrm{supp}}, \quad g_{\mathrm{place}}, \quad g_{\mathrm{innov}}, \quad c_{\mathrm{floor}}, \quad N_{\mathrm{points}},$$

where $r_{\mathrm{det}}$ is the deterministic-completion residual, $r_{\mathrm{supp}}$ is the residual in the inactive SVD support, $g_{\mathrm{place}}$ and $g_{\mathrm{innov}}$ are placement and innovation spectral-gap diagnostics, $c_{\mathrm{floor}}$ counts active covariance floors, and $N_{\mathrm{points}}$ is the rule size. These diagnostics keep model-law failures separate from numerical regularization. They are also the gate before analytic score or HMC claims.

The analytic first-order score now has a second rung beyond the affine fixture. For Model B we use

$$\theta_B = (\rho, \sigma, \beta)$$

with $\alpha$, the initial law, the innovation variance, and the observation variance held fixed. The derivative provider supplies

$$\frac{\partial F}{\partial x_{t-1}} = \begin{bmatrix} \rho & 0 \\ \beta(1-\tanh^2 m_t)\rho & \alpha \end{bmatrix}, \qquad \frac{\partial F}{\partial \varepsilon_t} = \begin{bmatrix} \sigma \\ \beta(1-\tanh^2 m_t)\sigma \end{bmatrix},$$

and parameter partials

$$F_\rho = \begin{bmatrix} m_{t-1} \\ \beta(1-\tanh^2 m_t)m_{t-1} \end{bmatrix}, \quad F_\sigma = \begin{bmatrix} \varepsilon_t \\ \beta(1-\tanh^2 m_t)\varepsilon_t \end{bmatrix}, \quad F_\beta = \begin{bmatrix} 0 \\ \tanh(m_t) \end{bmatrix}.$$

The observation derivative is $H_x = [1, 1]$. Centered finite differences of the implemented SVD cubature, the historical diagnostic SVD-UKF path, the promoted strict-SPD

principal-square-root UKF path, and the SVD-CUT4 likelihoods match the corresponding analytic scores on the selected smooth branch.

For Model C we use

$$\theta_C = (\sigma_u, \sigma_y, P_{0,x}).$$

The derivative provider supplies

$$F_x = \begin{bmatrix} \frac{1}{2} + 25\frac{1-x^2}{(1+x^2)^2} & -9.6\sin(1.2\tau) \\ 0 & 1 \end{bmatrix}, \qquad F_\varepsilon = \begin{bmatrix} \sigma_u \\ 0 \end{bmatrix},$$

$$F_{\sigma_u} = (\varepsilon_t, 0)', \qquad \partial_{\sigma_y} R = 2\sigma_y, \qquad \partial_{P_{0,x}} P_0 = \mathrm{diag}(1, 0),$$

and $H_x = [x/10, 0]$. Score parity is certified on a nondegenerate phase-state testing variant with positive phase variance. The default Model C value benchmark still has zero phase variance. Under the smooth simple-spectrum/no-active-floor SVD score branch, that default law remains blocked by an active placement floor. This is an intentional diagnostic. The structural fixed-support branch in Chapter 18 instead keeps the phase direction as a declared structural null direction: the factor column and its derivative are zero, the pre-transition sigma-point variable is $(x_{t-1}, \varepsilon_t)$, and the propagated phase coordinate is completed pointwise by the transition map. On that branch, default Model C is now a score-ready testing target for SVD cubature, SVD-UKF, and SVD-CUT4, subject to the fixed-null covariance, fixed-null derivative, active-gap, and finite-difference diagnostics.

Consequently, GPU/XLA and HMC evidence for nonlinear Models B–C remains deferred until the score-ready parameter boxes and branch diagnostics have been selected and tested.

## 53.3 Required Outputs

A nonlinear SSM HMC validation run should write:

- JSON diagnostics with chain counts, warmup/draw counts, step sizes, acceptance rate, divergence counts, R-hat, ESS, MCSE/SD, and runtime;
- NPZ chain artifacts for independent audit;
- true-parameter coverage when synthetic truth exists;
- filter failure counts, conditioning-guard counts, and finite-gradient failures.

Medium runs are sampler-usability evidence. Strict runs require at least four chains, finite log targets, zero or justified divergences, R-hat below 1.01, bulk and tail ESS above the planned thresholds, and saved artifacts.

## 53.4 Stop Rules

If the LGSSM rung fails, the nonlinear rung should not start. If the nonlinear SSM rung fails while LGSSM passes, the failure should be attributed first to the approximation, nonlinear target geometry, transition hook, or derivative path. DSGE/NK debugging should wait until this simpler explanation is ruled out.

# Chapter 54

# Neural Networks and State-Space Model Applications

Neural state-space models combine a learned nonlinear transition with an explicit probabilistic state and observation model. This combination is useful when the transition must retain memory that is richer than a low-order linear recursion, but the application still requires filtering, forecasting, likelihood-based parameter inference, and uncertainty propagation. It also creates a validation problem. A neural transition can be strongly overparameterized, so two posterior approximations may differ in parameter space while inducing nearly the same forecast distribution. Conversely, two samplers can agree on a few parameter summaries and still disagree on the predictive quantities for which the model is used.

This chapter develops a running BayesFilter application based on a stochastic state-space LSTM (SSL-LSTM). It then defines a posterior-predictive equivalence criterion for comparing ordinary HMC with exact-corrected neural-transport HMC. The model description follows the current BayesFilter TensorFlow implementation. The predictive-equivalence criterion is a BayesFilter `extension_or_invention`; it is not a Zhao–Cui source-faithfulness claim.

## 54.1 A Stochastic State-Space LSTM

Let $z_t \in \mathbb{R}^k$ denote the stochastic latent state, $a_t \in \mathbb{R}^h$ the LSTM hidden state, and $c_t \in \mathbb{R}^h$ the LSTM cell state. The augmented filtering state is

$$x_t = (z_t^\top, a_t^\top, c_t^\top)^\top \in \mathbb{R}^{k+2h}. \tag{54.1}$$

Only $z_t$ receives a new structural innovation. Conditional on the previous state, $a_t$ and $c_t$ are deterministic completions. This partition is important for sigma-point filtering because it avoids pretending that the memory coordinates contain independent process noise.

For gate $g \in \{i, f, o, r\}$, define

$$\eta_{g,t} = W_g z_{t-1} + U_g a_{t-1} + b_g, \tag{54.2}$$

$$i_t = \mathrm{sigmoid}(\eta_{i,t}), \tag{54.3}$$

$$f_t = \mathrm{sigmoid}(\eta_{f,t}), \tag{54.4}$$

$$o_t = \mathrm{sigmoid}(\eta_{o,t}), \tag{54.5}$$

$$r_t = \tanh(\eta_{r,t}). \tag{54.6}$$

Here

$$W_g \in \mathbb{R}^{h\times k}, \qquad U_g \in \mathbb{R}^{h\times h}, \qquad b_g \in \mathbb{R}^h,$$

so every gate vector lies in $\mathbb{R}^h$. The latent and observation maps satisfy $A \in \mathbb{R}^{k\times h}$, $d \in \mathbb{R}^k$, $C \in \mathbb{R}^{d_y\times k}$, and $e \in \mathbb{R}^{d_y}$. The candidate gate is denoted by $r$ to avoid confusing it with the cell state. The memory recursion is

$$c_t = f_t \odot c_{t-1} + i_t \odot r_t, \tag{54.7}$$
$$a_t = o_t \odot \tanh(c_t). \tag{54.8}$$

The stochastic latent transition and observation law are

$$z_t = Aa_t + d + \Sigma_z^{1/2}\epsilon_t, \qquad \epsilon_t \sim \mathcal{N}(0, I_k), \tag{54.9}$$
$$y_t = Cz_t + e + \Sigma_y^{1/2}\nu_t, \qquad \nu_t \sim \mathcal{N}(0, I_{d_y}), \tag{54.10}$$

with independent innovations across time and between the two equations. The current implementation uses diagonal $\Sigma_z$ and $\Sigma_y$. Initial states follow

$$x_0 \sim \mathcal{N}(m_0, \operatorname{diag}(s_0^2)). \tag{54.11}$$

Positive standard deviations are constructed from unconstrained coordinates by a softplus transform plus a small positive floor.

Equations (54.6)–(54.10) have the usual LSTM memory mechanism [Hochreiter and Schmidhuber, 1997], but they are embedded in a generative state-space model rather than used only as a deterministic predictor. The transition produces a probability law, the filter integrates over latent uncertainty, and Bayesian inference propagates parameter uncertainty into future observations.

### 54.1.1 Parameter chart

The unconstrained BayesFilter parameter vector contains

$$\theta = \left(\{W_g, U_g, b_g\}_{g\in\{i,f,o,r\}}, A, d, C, e, m_0, \widetilde{s}_0, \widetilde{\sigma}_z, \widetilde{\sigma}_y\right). \tag{54.12}$$

For dimensions $(k, h, d_y)$, the implementation stores the four input-gate matrices, four recurrent matrices, four biases, the latent and observation maps, the initial mean and standard deviations, and the process and observation standard deviations in one fixed order. Static dimensions and a fixed chart are part of the compiled filtering and HMC target contract.

### 54.1.2 Structural SVD-UKF likelihood

The running application evaluates the likelihood with BayesFilter's structural SVD sigma-point filter. Sigma-point integration is performed in the $k$-dimensional innovation space. Each innovation point is lifted through Equations (54.6)–(54.10), and the LSTM hidden and cell states are completed deterministically. This preserves the state partition in Equation (54.1).

Let $\ell_{\text{SVD-UKF}}(\theta; y_{1:T})$ denote the resulting filtering log likelihood. The posterior target is

$$\pi(\theta \mid y_{1:T}) \propto \exp\{\ell_{\text{SVD-UKF}}(\theta; y_{1:T})\}\, p(\theta). \tag{54.13}$$

This is an exact scalar target relative to the fixed SVD-UKF recursion, but the SVD-UKF likelihood is an approximation to the nonlinear model likelihood. Sampler validation must not erase that distinction.

## 54.2 The Scalar Four-Parameter Application

The scalar validation problem sets

$$k = h = d_y = 1, \qquad \dim(x_t) = 3, \qquad T = 30. \tag{54.14}$$

The full scalar chart has 24 entries. The current validation target holds 20 entries fixed at the inherited fixture values and estimates

$$\psi = (A, d, C, e)^\top \in \mathbb{R}^4. \tag{54.15}$$

In repository names these are

- `latent_mean_weight.0.0`;
- `latent_mean_bias.0`;
- `observation_weight.0.0`;
- `observation_bias.0`.

The prior used by the scalar filtering target is

$$\psi \sim \mathcal{N}(\psi_0, 4^2 I_4), \tag{54.16}$$

where $\psi_0$ is the fixed simulation center. The observation path is generated with a fixed stateless seed and is shared by every sampler arm.

This restriction creates a deliberately small but non-Gaussian filtering target. It is useful for testing geometry, transport, HMC, and predictive validation without first solving full 24-parameter identification. It does not establish that the four-parameter restriction is scientifically adequate, that the remaining neural-network parameters are known, or that the fixture is source-faithful to an external SSL-LSTM implementation.

## 54.3 Why Parameter Agreement Is Not the Only Estimand

Traditional sampler checks often compare posterior parameter means, variances, quantiles, or an independently computed parameter posterior. Those checks remain useful when the parameter posterior itself is the scientific object. They can be unnecessarily strong for prediction when multiple parameter configurations induce nearly the same transition and observation law. Neural networks make this issue especially visible because symmetries, weak identification, and compensating weights can create broad or curved parameter sets with similar functional behavior.

The converse warning is equally important. Predictive agreement is weaker than equality of parameter posteriors. If $F$ maps a parameter and filtered state distribution into a forecast law, then

$$F_\# \pi_A = F_\# \pi_B \quad \not\Longrightarrow \quad \pi_A = \pi_B. \tag{54.17}$$

The pushforward can discard directions that matter for structural interpretation even when they do not matter for the selected forecast. The appropriate claim is therefore *predictive-functional equivalence*, not full posterior correctness or parameter identification.

## 54.4 Posterior-Predictive Law

Fix the observed history $y_{1:T}$ and a maximum forecast horizon $H$. For sampler arm $s$, let $\pi_s(d\psi, dx_T \mid y_{1:T})$ be the joint parameter and terminal-state approximation induced by the fixed filtering target and that arm's posterior draws. Its joint predictive path law is

$$P_s^H(d\widetilde{y}) = \int p_\psi(d\widetilde{y}_{T+1:T+H} \mid x_T)\pi_s(d\psi, dx_T \mid y_{1:T}). \tag{54.18}$$

Every sampler comparison must use the same data, parameter chart, terminal filter convention, forecast equations, horizon, and innovation law. A frozen stateless innovation bank may be shared between arms to reduce conditional simulation noise, but uncertainty calculations must retain the chain and forecast-cluster dependence that this design creates. Because this coupling is not part of either marginal predictive law, the omnibus distributional gate should use independently generated arm-specific innovation banks; the shared bank is a paired precision and robustness diagnostic.

### 54.4.1 Horizon-specific moments

For $h = 1, \ldots, H$, define

$$\mu_{s,h} = \mathbb{E}_s[Y_{T+h} \mid y_{1:T}], \tag{54.19}$$

$$v_{s,h} = \mathrm{Var}_s(Y_{T+h} \mid y_{1:T}), \tag{54.20}$$

$$\gamma_{s,h}^{(r)} = \mathbb{E}_s[(Y_{T+h} - \mu_{s,h})^r \mid y_{1:T}], \qquad r \geq 3, \tag{54.21}$$

$$\Gamma_{s,hk} = \mathrm{Cov}_s(Y_{T+h}, Y_{T+k} \mid y_{1:T}). \tag{54.22}$$

The first central moment is identically zero, so the first comparison is the raw predictive mean. Variance is the second central moment. Third and fourth central moments can reveal skewness and tail-shape differences, but their Monte Carlo uncertainty is substantially larger. Cross-horizon covariance matters because equality of all marginal forecast laws does not imply equality of the joint path law.

When the filter can propagate conditional means and covariances deterministically for each posterior draw, the law of total expectation and variance gives a lower-noise estimator:

$$\mu_{s,h} = \mathbb{E}_{\pi_s}[m_h(\psi, x_T)], \tag{54.23}$$

$$v_{s,h} = \mathbb{E}_{\pi_s}\left[V_h(\psi, x_T) + m_h(\psi, x_T)^2\right] - \mu_{s,h}^2. \tag{54.24}$$

BayesFilter should prefer these Rao–Blackwellized moments when the forecast recursion and state approximation make them available. Simulated forecast paths are still required for general path-distribution comparisons.

For implementation, let $m_{cnrh}$ and $s_{cnrh}^2$ be the conditional mean and variance of $Y_{cnrh}$ after fixing a retained parameter–state draw and the terminal/process innovations used by forecast replication $r$. Define

$$\widehat{\mu}_h = \frac{1}{CNR}\sum_{c,n,r} m_{cnrh}, \tag{54.25}$$

$$\widehat{q}_h = \frac{1}{CNR}\sum_{c,n,r}\{s_{cnrh}^2 + m_{cnrh}^2\}, \tag{54.26}$$

$$\widehat{v}_h = \widehat{q}_h - \widehat{\mu}_h^2. \tag{54.27}$$

The corresponding draw-cluster influence functions are

$$\xi_{cn}^{\mu_h} = R^{-1} \sum_r m_{cnrh} - \widehat{\mu}_h, \tag{54.28}$$

$$\xi_{cn}^{\log v_h} = \frac{R^{-1} \sum_r \{s_{cnrh}^2 + (m_{cnrh} - \widehat{\mu}_h)^2\}}{\widehat{v}_h} - 1. \tag{54.29}$$

Equation (54.29) follows by applying the delta method to $v_h = q_h - \mu_h^2$ and then to $\log v_h$. Averaging it over $(c, n)$ gives zero exactly for the plug-in moments above. This Rao–Blackwellization integrates observation noise analytically; it does not integrate terminal-state, parameter, or process-innovation uncertainty unless those variables are also conditioned out by the forecast recursion.

## 54.5 From Moment Generating Functions to Stable Distribution Features

Whenever the MGF is finite on an open neighborhood of zero and termwise differentiation is justified, its Taylor expansion is

$$M_s(t) = \mathbb{E}_s[\exp(tY)] = \sum_{r=0}^{\infty} \frac{t^r}{r!} \mathbb{E}_s[Y^r] \tag{54.30}$$

This expansion provides an attractive formal connection between a weighted collection of moments and the full predictive distribution. It is not, however, a robust default empirical weight. The MGF need not be finite away from zero, and a small number of tail draws can dominate $\exp(tY)$.

The characteristic function is safer:

$$\varphi_s(\omega) = \mathbb{E}_s[\exp(i\omega^\top \widetilde{Y})] = \mathbb{E}_s[\cos(\omega^\top \widetilde{Y})] + i\mathbb{E}_s[\sin(\omega^\top \widetilde{Y})]. \tag{54.31}$$

It always exists, its empirical features are bounded, and the joint characteristic function identifies the joint predictive law. A weighted distance is

$$D_w(P_A^H, P_B^H) = \int |\varphi_A(\omega) - \varphi_B(\omega)|^2 \, w(d\omega). \tag{54.32}$$

For suitable spectral weights, this is the same population discrepancy as a kernel maximum mean discrepancy (MMD) [Gretton et al., 2012, Sriperumbudur et al., 2010]. A Gaussian spectral weight corresponds to a Gaussian radial-basis kernel. A fixed mixture of bandwidths is preferable to choosing a bandwidth after inspecting which choice makes two samplers agree.

## 54.6 What the Weights Mean

Three different weights enter a predictive comparison and should not be collapsed into one tuning parameter.

**Scientific horizon weights.** Weights $\lambda_h \geq 0$ with $\sum_h \lambda_h = 1$ state whether near-term and long-term forecasts have equal scientific value. Equal horizon weights are the neutral default. Discounted weights are legitimate only when the application declares that shorter horizons matter more; they must not be estimated to hide a long-horizon discrepancy.

**Feature scaling.** Means, variances, fourth moments, and cross-products have different units. They must be standardized by a frozen external scale or by a baseline-only pilot whose draws are excluded from confirmation. Scaling is a numerical and interpretive operation, not evidence that a discrepancy is small.

Concretely, if $a_h > 0$ is the frozen predictive standard-deviation scale, then $m^*_{cnrh} = m_{cnrh}/a_h$ and $s^{2*}_{cnrh} = s^2_{cnrh}/a^2_h$ must be supplied to Equations (54.27)– (54.29). The scale must come from an independent calibration-only bank, external scientific units, or a model quantity fixed before confirmation. Estimating $a_h$ from the G/H confirmation comparison would condition the test on the outcome being judged; if no independent scale is available, its uncertainty must instead be carried through the covariance calculation.

**Statistical precision weights.** Let $\widehat{\Delta}$ collect real and imaginary empirical characteristic features, or a fixed vector of standardized moment differences. If $\widehat{\Delta} \in \mathbb{R}^q$, $\widehat{S} \in \mathbb{R}^{q\times q}$ is a symmetric positive-semidefinite estimate of its long-run covariance, and $\lambda > 0$, then $\widehat{S} + \lambda I_q$ is positive definite. The regularized quadratic form

$$Q_\lambda = \widehat{\Delta}^\top (\widehat{S} + \lambda I_q)^{-1} \widehat{\Delta} \tag{54.33}$$

uses the efficient GMM weighting principle for a fixed feature vector [Hansen, 1982]. Here $\widehat{S}$ must account for MCMC autocorrelation, separate chains, repeated forecast innovations, and any common-random-number coupling. The ridge $\lambda$ and feature grid must be chosen in calibration and frozen before the confirmatory arm comparison.

Inverse-covariance weighting does not replace scientific equivalence margins. A very precisely estimated but scientifically negligible feature should not dominate the decision merely because its Monte Carlo variance is small.

## 54.7 Dependence-Aware Estimation of Predictive Features

Write the standardized co-primary feature vector for arm $s$ as

$$\beta_s = (\mu_{s,1}, \ldots, \mu_{s,H}, \log v_{s,1}, \ldots, \log v_{s,H})^\top \in \mathbb{R}^{2H}, \qquad \delta = \beta_A - \beta_B. \tag{54.34}$$

Suppose retained draw $n$ in chain $c$ has $R$ forecast replications $Y_{cnrh}$. With $\overline{Y}_{cnh} = R^{-1}\sum_r Y_{cnrh}$ and population mean and variance $(\mu_h, v_h)$, the draw-cluster influence functions are

$$\xi^{\mu_h}_{cn} = \overline{Y}_{cnh} - \mu_h, \tag{54.35}$$

$$\xi^{\log v_h}_{cn} = \frac{R^{-1}\sum_r (Y_{cnrh} - \mu_h)^2}{v_h} - 1. \tag{54.36}$$

The second expression follows from the delta method: the influence function of $v_h = \mathbb{E}(Y_h - \mu_h)^2$ is $(Y_h - \mu_h)^2 - v_h$, and the derivative of $\log v_h$ is $1/v_h$. Averaging replications

inside a draw cluster preserves their common parameter-state dependence. Replacing population quantities by consistent empirical quantities changes the expansion only by $o_p(N^{-1/2})$ under the conditions below.

For each arm, assume that its $C$ chains are independent copies of a stationary, ergodic Markov chain after warmup; the vector influence sequence has finite $2+\eta$ moments for some $\eta > 0$; its strong-mixing coefficients $\alpha(k)$ satisfy $\sum_{k\geq 1} \alpha(k)^{\eta/(2+\eta)} < \infty$; and the remaining regularity conditions for a multivariate Markov-chain central limit theorem hold. This mixing-and-moment condition also implies absolute summability of the relevant autocovariances under the usual covariance inequality. Then, for $N$ retained draws per chain,

$$\sqrt{CN}(\widehat{\beta}_s - \beta_s) \Rightarrow \mathcal{N}(0, \Omega_s), \tag{54.37}$$

$$\Omega_s = \sum_{k=-\infty}^{\infty} \Gamma_s(k), \qquad \Gamma_s(k) = \mathrm{Cov}(\xi_{s,c,0}, \xi_{s,c,k}). \tag{54.38}$$

For independent arms, $\widehat{V} = \widehat{\Omega}_A/(C_A N_A) + \widehat{\Omega}_B/(C_B N_B)$ estimates the covariance of $\widehat{\delta} = \widehat{\beta}_A - \widehat{\beta}_B$.

### 54.7.1 Why a fixed batch length is not enough

The historical Phase 8 implementation used nonoverlapping batches of fixed length $b = 16$. Even with infinitely many retained draws, the corresponding spectral target is generally

$$\Omega_b = \Gamma(0) + \sum_{0<|k|<b} \left(1 - \frac{|k|}{b}\right) \Gamma(k), \tag{54.39}$$

not $\Omega$. This is a nonvanishing truncation and tapering bias. For a unit-variance AR(1) sequence with correlation $\phi = 0.6$,

$$\Omega = 1 + 2\sum_{k=1}^{\infty} \phi^k = 4, \qquad \Omega_{16} = 1 + 2\sum_{k=1}^{15} \left(1 - \frac{k}{16}\right) \phi^k \simeq 3.53138. \tag{54.40}$$

Thus the fixed-16 estimator understates long-run variance by about 11.7% in this elementary case. More draws do not remove that bias. Historical confidence intervals and sample-size extrapolations based on that estimator are useful engineering and feasibility diagnostics, but they are not a generally consistent inferential basis.

The prospective replacement is the chain-aware Bartlett/Newey–West estimator [Newey and West, 1987, Flegal and Jones, 2010]

$$\widehat{\Omega}_N = \widehat{\Gamma}(0) + \sum_{k=1}^{\ell_N} \left(1 - \frac{k}{\ell_N + 1}\right) \{\widehat{\Gamma}(k) + \widehat{\Gamma}(k)^\top\}, \tag{54.41}$$

where centering occurs separately within each chain and autocovariances are then averaged across chains. A prospective rule

$$\ell_N = \max\{1, \lfloor \kappa_{\mathrm{HAC}} N^{1/3} \rfloor\}, \qquad 0 < \kappa_{\mathrm{HAC}} < \infty, \tag{54.42}$$

satisfies $\ell_N \to \infty$, $\ell_N/N \to 0$, and the stronger $\ell_N^2/N \to 0$. Those bandwidth conditions are sufficient for the usual Bartlett-HAC asymptotic regime, but this monograph does

*not* yet claim a theorem-level consistency result for Equation (54.41) because the exact primary-source conditions remain an explicit source-support blocker. The constant $\kappa_{\mathrm{HAC}}$, any eigenvalue ridge, and the finite-sample admission checks must be selected without inspecting confirmation outcomes. A covariance used for confirmation must select the zero-ridge candidate, or use a separately proved loading that vanishes fast enough not to change the limiting covariance; a fixed positive loading cannot inherit an asymptotic theorem merely because it makes inversion possible. This paragraph is therefore an implementation and audit direction, not a closed asymptotic proof.

Finite-sample bias can remain material even when the bandwidth rule is consistent. In the controlled Gaussian law used below, the draw effect has weight 0.65, two forecast replications contribute independent noise with total weight 0.35, and the draw effect has AR coefficient $\phi = 0.6$. The mean-influence autocovariances are therefore

$$\gamma_0 = 0.65 + \frac{0.35}{2} = 0.825, \qquad \gamma_k = 0.65(0.6)^k \quad (k \geq 1), \tag{54.43}$$

so the exact long-run variance is

$$\Omega = 0.825 + 2(0.65)\frac{0.6}{1 - 0.6} = 2.775. \tag{54.44}$$

At $N = 8192$, $\kappa_{\mathrm{HAC}} = 1$ gives $\ell_N = 20$ and the population Bartlett target is 2.542862, an 8.365% underestimate. This calculation does not prove that a larger multiplier is optimal: it proves that the finite-sample multiplier must be calibrated prospectively on development seeds and then frozen before a fresh audit.

## 54.8 A Proper-Score Definition of Practical Equivalence

Arithmetic midpoints between convenient stress cases do not define scientific negligibility. A more principled contract starts from predictive loss. For the local standardized Gaussian forecasts $P = \mathcal{N}(0, 1)$ and $Q = \mathcal{N}(d, \exp(\ell))$, define symmetric logarithmic-score regret as the average of the two directed Kullback–Leibler divergences. Direct expansion gives

$$\frac{1}{2}\{\mathrm{KL}(P\|Q) + \mathrm{KL}(Q\|P)\} = \frac{1}{4}\{e^{-\ell}(1 + d^2) - 1 + \ell + e^{\ell} + d^2 - 1 - \ell\} \tag{54.45}$$

$$= \frac{1}{2}d^2 + \frac{1}{4}\ell^2 + o(d^2 + \ell^2). \tag{54.46}$$

The logarithmic score is strictly proper [Gneiting and Raftery, 2007]; the quadratic expression is its local regret, not a global Gaussian approximation to an arbitrary predictive law.

For $H$ horizons, choose scientific weights $\lambda_h > 0$ with $\sum_h \lambda_h = 1$ and define

$$L(\delta) = \sum_{h=1}^{H} \lambda_h \left\{ \frac{1}{2}\delta_{\mu,h}^2 + \frac{1}{4}\delta_{\log v,h}^2 \right\} = \delta^{\top} W_L \delta, \tag{54.47}$$

where $W_L = \mathrm{diag}(\lambda_1/2, \ldots, \lambda_H/2, \lambda_1/4, \ldots, \lambda_H/4)$. The application must declare an acceptable regret budget $K > 0$. Scientific equivalence means $L(\delta) < K$; $K$ cannot be

learned from the same sampler outcome being judged. Notice that $W_L$ encodes scientific loss, whereas $\widehat{V}^{-1}$ below encodes Monte Carlo precision. They answer different questions.

An average loss alone can hide a scientifically material discrepancy at one horizon. Define the unweighted horizon regret

$$R_h(\delta) = \frac{1}{2}\delta_{\mu,h}^2 + \frac{1}{4}\delta_{\log v,h}^2, \qquad L_{\max}(\delta) = \max_{1\le h\le H} R_h(\delta). \tag{54.48}$$

For the controlled design with $H = 10$, a persistent negligible standardized mean shift 0.05 has equal-weight average loss 0.00125. A material variance ratio 1.25 at only one horizon has average loss $\log(1.25)^2/40 \simeq 0.0012448$, which is slightly smaller. No scalar threshold on the average loss can classify the first discrepancy as negligible and the second as material. This is a mathematical incompatibility, not a tuning problem.

The prospective repair uses the average and horizonwise losses together. Let the declared negligible anchors be mean shift 0.05 and variance ratio 1.05, and let the declared material anchors be mean shift 0.20 and variance ratios 1.25 and 0.80. On the local proper-score scale define

$$K_N = \max\left\{\tfrac{1}{2}(0.05)^2, \tfrac{1}{4}\log(1.05)^2\right\} = 0.00125, \tag{54.49}$$

$$K_M = \min\left\{\tfrac{1}{2}(0.20)^2, \tfrac{1}{4}\log(1.25)^2, \tfrac{1}{4}\log(0.80)^2\right\} = \tfrac{1}{4}\log(1.25)^2 \simeq 0.0124481, \tag{54.50}$$

$$K_* = \frac{K_N + K_M}{2} \simeq 0.0068491. \tag{54.51}$$

The midpoint is taken in scientific loss, not separately in mean and variance coordinates. It maximizes the smaller additive clearance from the nearest declared anchor. Equal-weight practical equivalence now requires both

$$\sup_{d\in\mathcal{C}_{\text{avg}}} L(d) < K_* \quad \text{and} \quad \sup_{z\in\mathcal{C}_h} R_h(z) < K_* \quad \text{for every } h. \tag{54.52}$$

A material difference is declared if the corresponding lower bound for the average loss or at least one horizon loss exceeds $K_*$. Every extremum uses an authenticated covariance projection from the same feature estimator. The regions are not assumed independent; simultaneous control comes from the prospective error allocation below.

Let $q = 2H$, let $P_h$ select coordinates $(h, H + h)$, and allocate

$$\alpha_{\text{avg}} = 0.025, \qquad \alpha_h = 0.0025 \quad (h = 1, \dots, 10), \qquad \alpha_{\text{avg}} + \sum_{h=1}^{10} \alpha_h = 0.05. \tag{54.53}$$

Construct one average region and ten horizon-marginal regions:

$$\mathcal{C}_{\text{avg}} = \left\{d : (\widehat{\delta} - d)^\top \widehat{V}^{-1}(\widehat{\delta} - d) \le \chi^2_{2H,1-\alpha_{\text{avg}}}\right\}, \tag{54.54}$$

$$\mathcal{C}_h = \left\{z : (P_h\widehat{\delta} - z)^\top (P_h\widehat{V}P_h^\top)^{-1}(P_h\widehat{\delta} - z) \le \chi^2_{2,1-\alpha_h}\right\}. \tag{54.55}$$

Define the three-way decision

$$\sup_{d\in\mathcal{C}} J(d) < K \quad \Longrightarrow \quad \text{equivalent}, \tag{54.56}$$

$$\inf_{d\in\mathcal{C}} J(d) > K \quad \Longrightarrow \quad \text{materially different}, \tag{54.57}$$

$$\text{otherwise} \quad \Longrightarrow \quad \text{inconclusive}. \tag{54.58}$$

Here $(J, \mathcal{C})$ is $(L, \mathcal{C}_{\text{avg}})$ for the average decision and $(R_h, \mathcal{C}_h)$ for horizon $h$. If $\widehat{V} = AA^\top$, substitute $d = \widehat{\delta} + Au$ with $\|u\|_2^2 \le \chi^2_{p,1-\alpha}$, where $p = 2H$ for the average region and $p = 2$ for a horizon region; both extrema are standard quadratic trust-region problems and can be solved from an eigendecomposition and a one-dimensional secular equation. The same calculation applies to each $2 \times 2$ marginal covariance. For $H = 10$, the old horizon calculation used $\chi^2_{20,0.95} = 31.4104$ in every horizon direction, which is a 99.9999849% two-dimensional region. The prospective horizon radius is instead $\chi^2_{2,0.9975} = 11.9829$; the average region uses $\chi^2_{20,0.975} = 34.1696$.

The familywise error-control argument is short. Let $E_{\text{avg}}$ be the event $\delta \in \mathcal{C}_{\text{avg}}$ and let $E_h$ be $P_h\delta \in \mathcal{C}_h$. On $E_{\text{avg}} \cap \bigcap_h E_h$, a true loss at or above $K$ belongs to its corresponding region, so its supremum cannot be below $K$; likewise a true loss at or below $K$ prevents its corresponding infimum from exceeding $K$. Consequently either type of false decisive result implies at least one noncoverage event, and

$$\Pr(\text{false decisive result}) \le \Pr(E^c_{\text{avg}}) + \sum_{h=1}^{H} \Pr(E^c_h) \le \alpha_{\text{avg}} + \sum_{h=1}^{H} \alpha_h + o(1) = 0.05 + o(1). \tag{54.59}$$

No independence between regions is used. If $\widehat{\delta} \to_p \delta$, $\widehat{V} \to_p 0$ at the correct rate, and the average and every relevant horizon loss are separated from $K$ by positive amounts, all regions shrink to their truths and the correct decision has probability tending to one. These conclusions require the central limit theorem and consistent HAC estimator above. At an exact boundary, either decisive result is boundary leakage; the correct three-way outcome is inconclusive. Without a scientifically chosen $K$, no consistent binary conclusion is claimed.

## 54.9 An Equivalence Gate, Not a Failure-to-Reject Gate

Let $\delta_{\mu,h}$ be a predictive-scale-standardized mean difference and let $\delta_{\log v,h}$ be the difference in log predictive variance. The log scale makes the variance comparison dimensionless and treats reciprocal variance ratios symmetrically. Let $\mathcal{I}_{\mu,h}$ and $\mathcal{I}_{\log v,h}$ denote their simultaneous confidence intervals. The primary interpretable gate requires

$$\mathcal{I}_{\mu,h} \subset (-\varepsilon_{\mu,h}, \varepsilon_{\mu,h}), \tag{54.60}$$

$$\mathcal{I}_{\log v,h} \subset (-\varepsilon_{\log v,h}, \varepsilon_{\log v,h}), \qquad h = 1, \dots, H. \tag{54.61}$$

The historical design additionally required an uncertainty upper bound for $D_w$ or MMD to lie below a predeclared practical tolerance. Prospectively, MMD remains an omnibus shape diagnostic until both its tolerance and its own growing-bandwidth uncertainty estimator pass calibration; only then may a reviewed design add it as a co-primary gate. Merely failing to reject equality is insufficient: short chains and noisy high-order moments can make a materially different pair of predictive laws look statistically indistinguishable.

Mean and log variance are the initial co-primary interpretable features. Third and fourth central moments, quantiles, and cross-horizon covariance begin as explanatory diagnostics. They can become promotion criteria only after a reviewed calibration shows adequate finite-sample power and stable uncertainty estimation.

Componentwise TOST remains a useful interpretable sensitivity analysis. With simultaneous $(1 - \alpha)$ intervals, declaring equivalence only when every interval lies inside

its margin is an intersection–union test: under any non-equivalent coordinate null, a false pass requires rejection of that true coordinate null and is controlled at the interval family's declared level. Conversely, declaring a material difference when any Bonferroni interval lies entirely outside its margin controls the familywise false-difference rate. These statements are conditional on valid simultaneous intervals; they do not rescue the historical fixed-16 covariance estimator. The joint proper-score rule in Equation (54.58) is the prospective primary decision because it connects the margins across horizons to one declared forecast-loss budget.

## 54.10 Ordinary HMC and Exact-Corrected NeuTra-HMC

Let $u$ denote the MAP-local coordinates of the four-parameter filtering target, and let $T_\phi : \mathbb{R}^4 \to \mathbb{R}^4$ be an invertible learned transport. NeuTra trains $T_\phi$ so that a standard Gaussian base is pushed toward the target geometry [Hoffman et al., 2019]. After training, the transport is frozen and HMC targets

$$\log \pi_z(z) = \log \pi_u(T_\phi(z)) + \log \left|\det J_{T_\phi}(z)\right| . \tag{54.62}$$

The Jacobian correction and Metropolis correction preserve the intended filtering target even when the learned flow is imperfect, subject to the usual numerical and implementation checks. Training loss, validation loss, or raw flow samples are not posterior-correctness evidence.

The functional comparison is therefore

$$\text{ordinary MAP-local HMC} \quad \text{versus} \quad \text{frozen-transport exact-corrected NeuTra-HMC}, \tag{54.63}$$

with both arms passed through the same forecast operator in Equation (54.18). Each arm must be tuned in its own geometry. Reusing the ordinary-HMC step size or trajectory length for NeuTra-HMC is not a fair comparison.

## 54.11 What the NeuTra Experiments Established

The experiment sequence is important because several failures were failures of procedure, capacity, optimization, or statistical design rather than evidence against neural transport.

**Procedure transfer and capacity.** The first one-stage `tanh` dense IAF and affine-control experiments did not reproduce the established local Rotemberg/SGU training procedure. The corrected local-source transfer used three dense IAF stages, ELU conditioners, reverse-coordinate mixing, a fixed reference translation, standard-normal base draws, reverse KL, batch size 480, at most 5,000 steps, the inherited piecewise learning-rate schedule, and per-variable gradient clipping. Direct forward-map, log-Jacobian, gradient, and one-step Adam comparisons tied out to the local source. The inherited $(4, 4)$ conditioners were then found too small for the SSL-LSTM geometry. Expanding to $(32, 32)$ while retaining learning rate 0.01 saturated the scale outputs at the first 100-step check, showing that width and optimization could not be repaired independently.

**Tuning and adaptive training.** A bounded Optuna search varied the initial learning rate, initialization scale, and clipping norm for the $(32, 32)$ architecture. Training validation occurred every 100 steps. After 500 steps without a meaningful support-eligible improvement, the learning rate was halved; after two such patience periods, training stopped, subject to the 5,000-step cap and learning-rate floor. A candidate that passed on one fresh seed but saturated on the second was rejected. The predeclared trial-0 alternative was then trained on untouched streams G and H. Both passed their transport screens: G selected step 1,000 and stopped at 2,000; H selected step 900 and stopped at 1,900; each used one learning-rate repair. This supports viability under two streams, not optimizer superiority or complete mode coverage.

For the subsequent state-complexity ladder, the prospective final-training protocol is deliberately shorter and more explicit. Its maximum is 2,000 optimizer steps, with a fixed heldout batch checked every 250 steps. A meaningful improvement requires the paired one-sided upper confidence bound for candidate-minus-best loss to lie below the declared minimum delta; numerical equality of aggregate losses is not the criterion. At the first 250-step cycle without such improvement, the trainer restores the best transport and Adam state and halves the learning rate. With no subsequent improvement it continues for two further 250-step cycles, stopping 750 steps after the best checkpoint. A new meaningful improvement replaces the best checkpoint and restarts this clock. The 2,000-step cap and 250-step cadence are reviewed experimental defaults, not proofs of optimal training duration; downstream transport and HMC gates remain necessary.

**Exact target and retained sampling.** The frozen G/H transports passed value, analytic score, finite-difference, round-trip, serialization, scalar/batch, and trusted GPU/XLA checks for Equation (54.62). Separate HMC tuning froze identity momentum covariance, step size 0.8, and four leapfrog steps for both transports. Four chains per transport were admitted at 512 retained draws per chain. Mean acceptance was 0.6689 for G and 0.7036 for H; maximum reported R-hat was about 1.04, and the predeclared ESS and MCSE screens passed. The kernel did not expose native divergence telemetry, so "zero divergences" was not established. G and H are independent peer replications, not posterior oracles.

**Predictive-design failure and its interpretation.** Formal designs using 448 and then 1,984 draws per chain failed prospective power screens before any G/H confirmation forecast was opened. A later sample-size preflight found no feasible design at 1,984 draws and only descriptive feasibility signals at 4,096 or 8,192 draws. Those calculations used the fixed-16 uncertainty estimator and margins partly motivated by stress-case midpoints. Equations (54.39)– (54.58) explain why neither element is a sufficient confirmatory foundation. The experiments established working forecast and sampler machinery and identified a statistical-design defect; they did not establish G/H predictive equivalence, material difference, posterior correctness, or failure of NeuTra.

**Direct dual-loss controlled calibration.** The repaired procedure was then evaluated end to end on internally generated controlled laws without reading retained HMC samples or G/H confirmation forecasts. Each replication generated two independent four-chain forecast banks, estimated the full 20-feature mean/log-variance vector, fit the per-chain-centred growing Bartlett HAC estimator with $\kappa_{\mathrm{HAC}} = 1$ and zero ridge, formed one 95% joint Wald ellipsoid, and computed exact extrema for the average loss and all ten horizon

losses. The prospective ladder used 256 replications per family at 4,096 and 8,192 draws per chain, with independent seed domains and sequential stopping.

The implementation evidence was clean: both rungs had zero invalid numerical replications and zero false decisive results, finite condition numbers, and finite KKT residuals. The current sample ladder nevertheless failed the required-decision gate. At 8,192 draws, the persistent negligible mean family was declared equivalent in only $5/256 = 1.95\%$ of replications. The local variance-ratio-1.25 family was declared materially different in only $76/256 = 29.69\%$. The local mean families reached $224/256 = 87.5\%$ and $209/256 = 81.64\%$ material decisions, but their simultaneous one-sided lower bounds remained below the prospective 80% target. Persistent material mean and variance families reached 100% material decisions at 8,192, while the IID and dependent nulls reached 100% equivalence decisions.

The Monte Carlo coverage gate itself was underpowered. It used one-sided Clopper–Pearson bounds with Bonferroni tail probability $0.05/88$ for eleven families, four operating claims, and two looks. With only 256 replications, a procedure having exactly 95% coverage has only about 26% probability of certifying a lower bound of at least 90%. Pooled across the eleven required families, observed coverage was 90.84% at 4,096 and 92.54% at 8,192; these pooled descriptive rates do not replace family-specific certification. Thus the run supports a sample-size/power failure for the current candidate, not a conclusion that the HAC implementation or predictive-validation idea is invalid. A prospective repair must increase controlled replication count and repair the region geometry and finite-sample HAC calibration before considering more retained draws, without changing $K_*$ or tuning the HAC multiplier on this locked result.

**Prospective directional-region remedy.** The next controlled program replaces the over-conservative horizon geometry with Equations (54.54)– (54.55), while preserving $K_*$, equal horizon weights, and zero-ridge inference. HAC multipliers $\{1, 1.5, 2, 3\}$ are compared only on development seeds, with validity and coverage considered before decision power; one candidate is then frozen for a fresh-seed audit. The audit contains exact-boundary families, for which any decisive result is leakage, and just-inside/outside guard families, for which only a decision in the wrong direction is an error. High power is not required arbitrarily close to the boundary.

The same program compares simulated-path moments with Equation (54.29). In the controlled laws the predictive scale is known to be one. Target confirmation instead requires scales frozen from an independent calibration-only bank, and remains closed until that provenance and the controlled audit both pass. With no more than 120 simultaneous operating claims, the locked audit uses 1,536 replications per family: at one-sided tail probability $0.05/120$, a true event rate of 2.5% has about 96.1% probability of certifying an upper bound of 5%. This is an evidence-capacity calculation, not a guarantee that the candidate will pass.

**Controlled remedy result.** The development comparison confirmed that the July 17 failure was primarily a region-geometry and forecast-noise problem, not evidence against predictive validation. At 8,192 draws, the historical full-region/path candidate again declared the persistent negligible mean equivalent in only 1/96 runs and the local variance-ratio-1.25 family material in 19/96. Every split-region candidate repaired those failures descriptively. The development-only rule nominated the split region, Rao–Blackwellized

features, and $\kappa_{\mathrm{HAC}} = 3$; this nomination does not establish that those settings are statistically superior to the other viable split candidates.

The first capacity screen mistakenly required every one of 22 family coverage rates, each estimated from only 96 replications, to exceed 93%. Under independent exact 95% coverage, that rule falsely blocks with probability about 99.35%. Post-run review therefore removed the minimum-across-families screen, preserved the immutable failed receipt, and used pooled coverage only as a development diagnostic. The 12,288-draw rung then nominated the fixed audit size because it had 95.12% pooled gated-family coverage, at least 98.96% required decisions in every primary family, at most 1.04% wrong-direction or boundary decisions, and zero invalid procedures.

The single fresh-seed audit used 1,536 replications for each family and passed all 77 prospective operating claims. All eleven primary families certified coverage above 90%, required-decision probability above 80%, false decision probability below 5%, and invalid probability below 5%. The weakest simultaneous coverage lower bound was 91.07% for the persistent negligible mean family; its required-decision lower bound was 97.93%. Every invalid count was zero. The largest simultaneous upper bound for exact boundary leakage was 1.106%. This validates the controlled finite-sample procedure for the declared laws. It does not validate retained HMC samples, G/H predictive equivalence, posterior correctness, or model adequacy; target confirmation remains closed pending independently frozen scales and target forecast conditional-variance wiring.

## 54.12 Three Evidence Ledgers

| Ledger | Question | Required evidence |
|---|---|---|
| Engineering correctness | Does the implementation evaluate the intended model and forecast? | Equation-to-code checks, deterministic recursions, linear-Gaussian forecast oracles, scalar/batch parity, transform round trips, finite values, and XLA parity. |
| Sampler validity | Does each chain explore its declared target? | Native divergence telemetry, finite log probabilities and gradients, acceptance, movement, chain-aware R-hat/ESS/MCSE, independent seeds, and frozen artifacts. |
| Predictive-functional equivalence | Do the two valid samplers induce the same practically relevant forecast law? | Simultaneous mean/log-variance equivalence, joint-path characteristic-function or MMD equivalence, and independent confirmation with frozen weights and tolerances. |

A failure in the first or second ledger invalidates interpretation of the third. A pass in the third ledger does not retroactively prove the first two. Two samplers can agree because they share a target bug or a forecast bug. Independent forecast-oracle tests and, where feasible, repeated synthetic-data calibration remain necessary.

## 54.13 Model Adequacy Is a Separate Question

Predictive equivalence asks whether two posterior computations give the same forecast law. Predictive accuracy asks whether that law describes future observations. Held-out log predictive density, CRPS, coverage, and calibration are proper model-evaluation tools [Gneiting and Raftery, 2007], but they do not replace the sampler-equivalence gate. The two questions should use separate artifacts:

1. same-data, same-model predictive equivalence for computation;
2. held-out or rolling-origin proper scores for model adequacy;
3. parameter or structural checks when scientific interpretation depends on identification rather than prediction alone.

## 54.14 Claim Boundary

A successful predictive-equivalence result may support the statement that two validated sampler arms induce practically equivalent $H$-step predictive laws for the fixed scalar SSL-LSTM filtering target, data set, forecast operator, and declared feature/tolerance design. It does not by itself establish:

- equality or correctness of the parameter posterior;
- global parameter identification;
- exactness of the SVD-UKF approximation to the nonlinear likelihood;
- empirical adequacy on unseen data;
- broad HMC, NeuTra, GPU, or default-method readiness;
- sampler superiority or a statistically supported ranking;
- Zhao–Cui source faithfulness.

The repaired implementation and finite-sample validation program for this chapter is specified in the live predictive-test repair plan under `docs/plans`. Until its HAC, trust-region, controlled-coverage, forecast, and independent-confirmation gates pass, this chapter defines a research design and reports bounded engineering history rather than a completed predictive equivalence result.

## 54.15 Direct Visual Validation and the Complexity Ladder

The formal predictive design is deliberately conservative, but it is also useful to inspect the trained result directly. A bounded July 18 experiment therefore produced launch trajectories, predictive fan charts, and horizon-by-horizon moment differences. The data boundary matters. Each of the G and H arms contained four admitted chains with 512 retained draws, but the visualization read only draws 0–63 of segment 0. This is the

prefix that the earlier pilot had already opened and permanently excluded from any later confirmation. Segment 1 and the confirmation suffix were authenticated by hash without deserializing their tensor values. Thus the graphs did not spend or contaminate a future confirmation sample.

Figure 54.1 shows all four launch chains in the transformed NeuTra coordinates. Figure 54.2 maps the same draws through each frozen invertible transport and shows the four free physical coordinates. The observation weight has a visibly smaller range than the other coordinates, so its common-axis trace appears nearly flat; this is a scale effect rather than a claim that it was fixed. The interwoven launch colors are useful evidence against gross initialization memory in this prefix. They are not proof of stationarity, mode coverage, or posterior correctness.

Each arm then used an independent, disjoint innovation bank and the frozen ten-step TensorFlow/TFP SVD–UKF forecast. Four chains, 64 parameter draws, and two forecast replications produced 512 complete paths per arm. Figure 54.3 displays the pooled 5–95 percent path band, median, and launch-specific forecast means. The bands and means overlap substantially by eye. The exact numerical summaries are reported in Table 54.1 so the conclusion does not depend on a graphical scale.

For an interpretable difference plot, the program formed draw-level mean and log-variance influence values, preserved each chain, averaged consecutive blocks of length 16, and estimated a standard error from the 16 block means per arm. It then applied a Bonferroni Gaussian multiplier over the 20 plotted coordinates. These are deliberately labeled approximate explanatory bands: the small number of blocks, absence of a practical-loss margin, and use of the excluded pilot prefix prevent them from emitting an equivalence or material- difference decision. The largest absolute mean difference was 0.08497 at horizon 7. The largest absolute log-variance difference was 0.27390 at horizon 5. Only the latter approximate band excluded zero. This localized descriptive difference is precisely why visual overlap cannot replace the prospective directional-region test.

The strongest direct procedure-level evidence still comes from laws whose truth is known by construction. The locked synthetic audit used 1,536 replications per family. Among the 22 displayed gated and boundary/guard families, minimum empirical coverage was 0.93359, the minimum required- decision rate among equivalence/material families was 0.99023, the maximum false or boundary-decision rate was 0.00326, and the invalid-procedure rate was zero. All prospective family gates passed. Figure 54.5 shows those operating characteristics. This validates the declared testing procedure on the controlled laws; it does not validate retained chains or the empirical SSL–LSTM model.

### 54.15.1 Successive state and memory complexity

A separate ladder tested the general structural adapter without reusing the scalar retained posterior. Its rung variable $q$ was defined prospectively by

$$k = h = q, \qquad d_y = 1, \qquad \dim(x_t) = 3q. \tag{54.64}$$

From the parameter blocks in Equations (54.6)– (54.11), the number of unconstrained parameters is

$$p(q) = 4q^2 + 4q^2 + 4q + q^2 + q + q + 1 + 3q + 3q + q + 1 \tag{54.65}$$

$$= 9q^2 + 13q + 2. \tag{54.66}$$

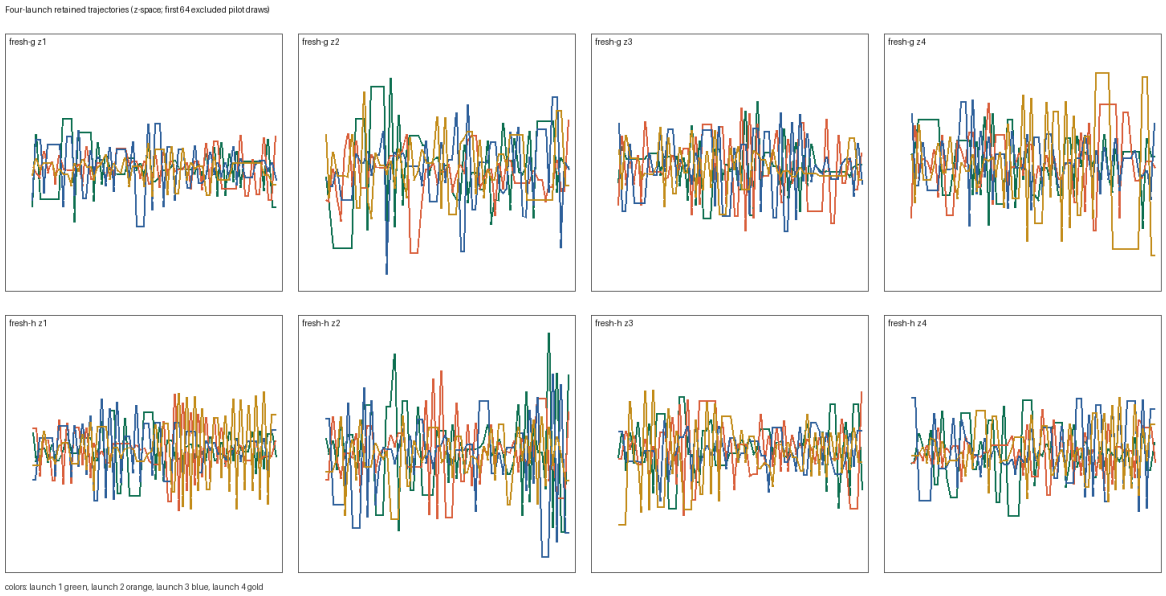



Figure 54.1: Four-launch trajectories in transformed coordinates for the first 64 retained, confirmation-excluded pilot draws of each G and H chain. Colors identify launches. Overlap is an initialization-robustness diagnostic only; neither arm is a posterior oracle.

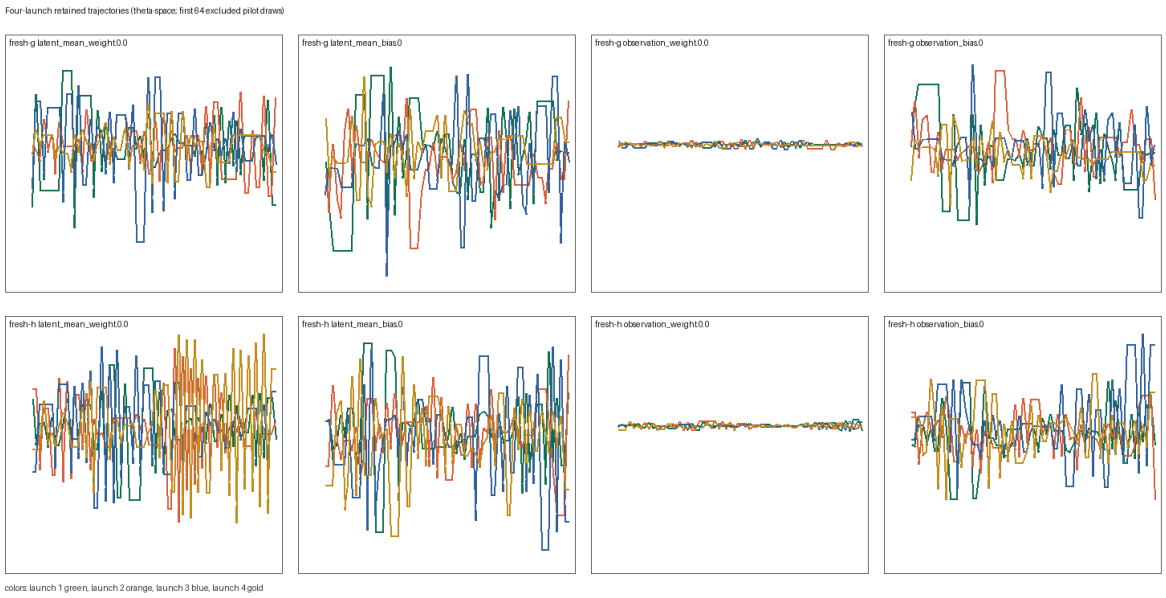



Figure 54.2: The same retained pilot draws mapped to the four free SSL–LSTM parameters: latent mean weight, latent mean bias, observation weight, and observation bias. The common vertical scale makes the narrower observation-weight trace appear nearly flat.

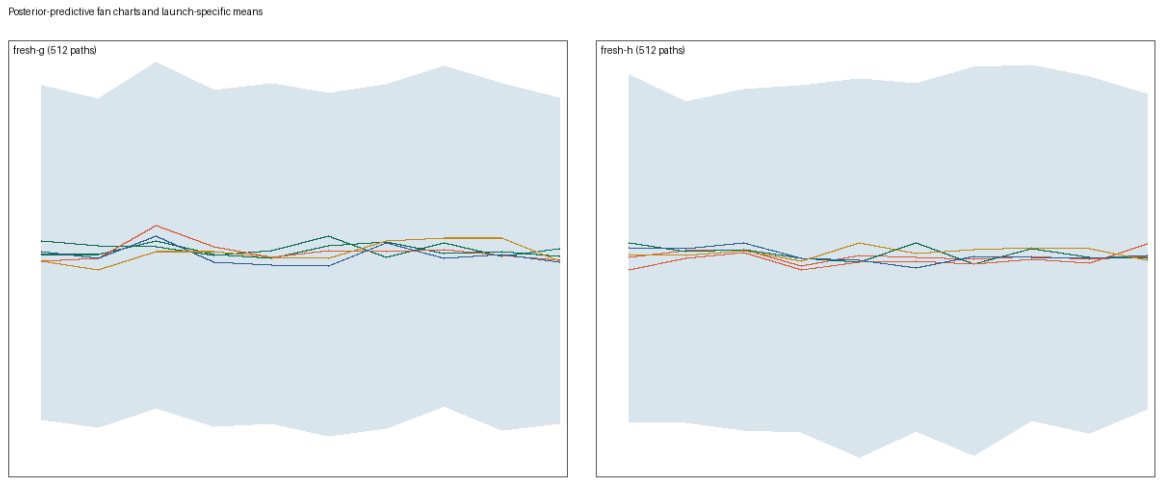



Figure 54.3: Ten-step posterior-predictive fan charts for G and H, using 512 forecast paths per arm. The shaded region is the empirical 5–95 percent path band, and colored lines are launch-specific means. This is a descriptive visualization of a short excluded prefix, not a formal predictive-equivalence region or a model-adequacy test.

Table 54.1: Direct pilot-prefix predictive summaries by horizon.

| Horizon | G mean | H mean | G variance | H variance |
|---|---|---|---|---|
| 1 | 0.03075 | 0.03019 | 0.72734 | 0.79356 |
| 2 | -0.00693 | 0.03452 | 0.74389 | 0.73390 |
| 3 | 0.14740 | 0.07245 | 0.82605 | 0.75009 |
| 4 | 0.02890 | -0.03841 | 0.74680 | 0.78278 |
| 5 | -0.00039 | 0.00548 | 0.70704 | 0.92982 |
| 6 | 0.04054 | 0.01020 | 0.77710 | 0.81790 |
| 7 | 0.07624 | -0.00873 | 0.77616 | 0.87938 |
| 8 | 0.08654 | 0.03440 | 0.82297 | 0.78793 |
| 9 | 0.05153 | 0.00773 | 0.80703 | 0.77792 |
| 10 | -0.00501 | 0.02347 | 0.71073 | 0.70391 |

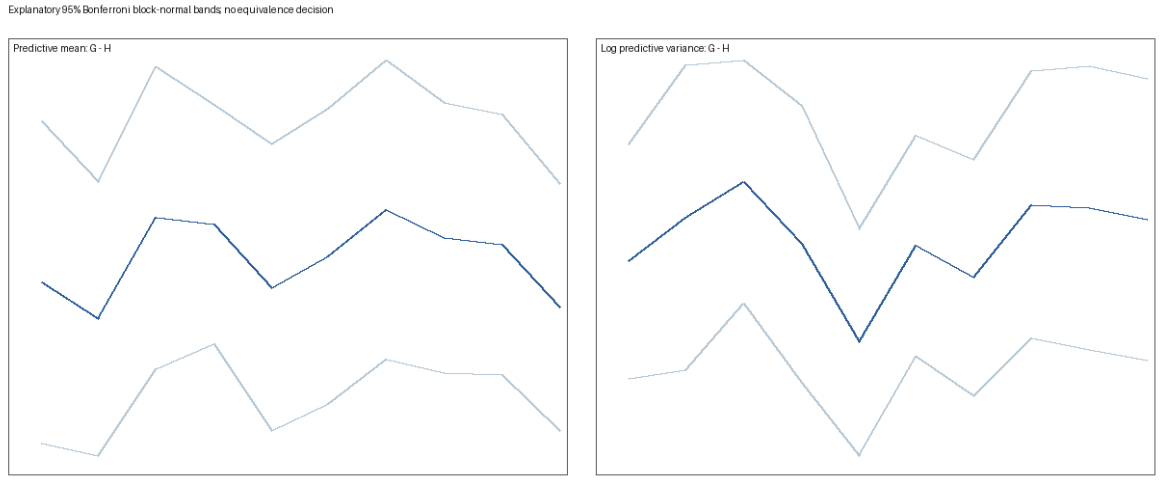



Figure 54.4: G-minus-H predictive mean and log-variance differences with approximate 95 percent Bonferroni block-normal bands. The plot is an explanatory pilot diagnostic and emits no equivalence or material- difference decision.

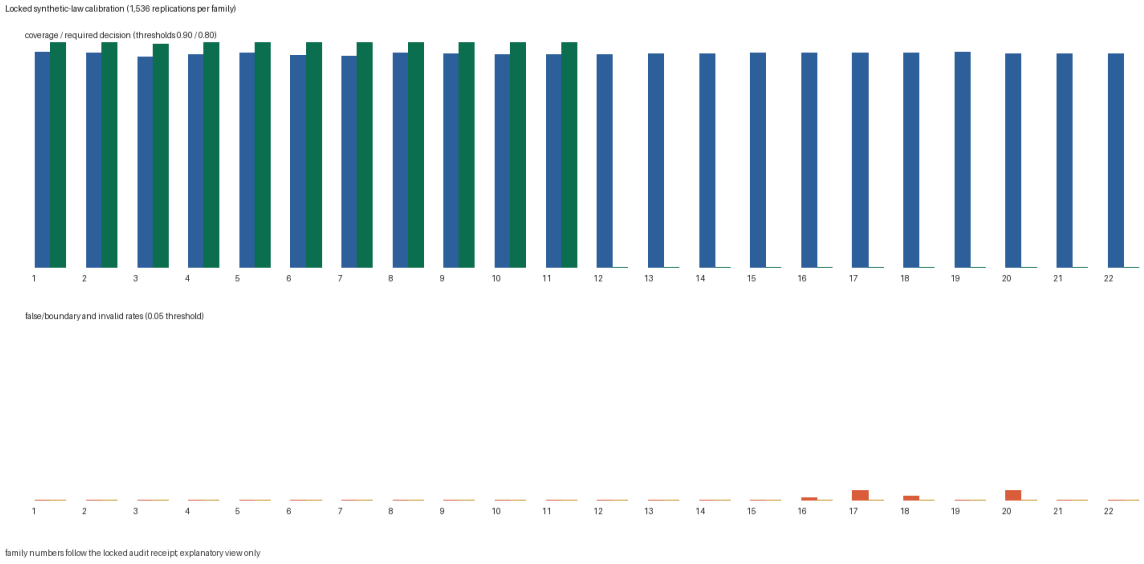



Figure 54.5: Operating characteristics from the locked synthetic-law audit. Blue and green bars are empirical coverage and required-decision rates; orange and gold bars are false/boundary and invalid rates. Family numbers follow the immutable audit receipt. This figure concerns controlled-law calibration, not the G/H retained posterior.

The observation dimension and three-step deterministic fixture were held fixed so the ladder isolated latent/memory and filter scaling. Each rung checked source-derived parameter layout, transition and observation derivatives against central finite differences, repeated compiled SVD–UKF likelihood and score, selected score finite differences, and finite positive-semidefinite filtered covariances. Table 54.2 gives the complete outcome.

Rungs 1, 2, 5, and 10 passed every declared engineering and numerical check. Across all attempted rungs, the largest transition-state Jacobian residual was $3.62 \times 10^{-11}$, the largest selected transition-parameter residual was $9.33 \times 10^{-12}$, and the largest selected analytic-score finite-difference residual was $2.68 \times 10^{-7}$. The $q = 20$ outputs were finite, repeatable, positive semidefinite within tolerance, and within the repaired derivative tolerances. Nevertheless, $q = 20$ is not admitted: its 1,291.90-second rung drove the run beyond the 600-second program cap and its reported 36.17-GB peak process RSS exceeded the 2-GB warning threshold. This is a resource/capacity veto, not evidence that the equations or the neural-state-space idea are wrong. The supervisor was subsequently changed so future rungs run in killable subprocesses with the remaining wall budget; $q = 20$ was not rerun or retroactively promoted.

### 54.15.2 Matrix-free differentiation for nonlinear filters

The $q = 20$ resource veto motivated a derivative-level redesign. The number of sigma points is not, by itself, the central problem. The expensive pattern is to construct a dense local Jacobian for every point and then use that Jacobian only through products with a small number of sensitivity directions. This pattern appears in sigma-point filters, fixed particle replays, and other deterministic filtering recursions. It can be removed without changing the filter scalar.

Let a local transition and observation map be

$$x_r^+ = f_\theta(x_r, \epsilon_r), \tag{54.67}$$

$$y_r^\star = g_\theta(x_r^+), \tag{54.68}$$

where $r = 1, \ldots, R$ indexes sigma points, quadrature points, or fixed replay particles. Suppose the score calculation carries $p$ parameter directions $v_1, \ldots, v_p$. Write $\dot{x}_{jr}$, $\dot{\epsilon}_{jr}$, and $\dot{x}_{jr}^+$ for the tangents in direction $v_j$. A dense implementation first constructs $D_x f_\theta$ and $D_\epsilon f_\theta$ and then evaluates

$$\dot{x}_{jr}^+ = D_x f_\theta(x_r, \epsilon_r)\dot{x}_{jr} + D_\epsilon f_\theta(x_r, \epsilon_r)\dot{\epsilon}_{jr} + D_\theta f_\theta(x_r, \epsilon_r)v_j. \tag{54.69}$$

The matrix-free form evaluates the right-hand side directly as a Jacobian–vector product (JVP):

$$\mathrm{JVP}_f(x_r, \epsilon_r; \dot{x}_{jr}, \dot{\epsilon}_{jr}, v_j) := \frac{d}{ds} f_{\theta+sv_j}(x_r + s\dot{x}_{jr}, \epsilon_r + s\dot{\epsilon}_{jr})\Big|_{s=0}. \tag{54.70}$$

Equations (54.69) and (54.70) are algebraically identical. The difference is computational: the second expression need not store or compute entries of the Jacobian that are never contracted with a tangent. The observation tangent is treated in the same way,

$$\dot{y}^s tar_{jr} = D_x g_\theta(x_r^+)\dot{x}_{jr}^+ + D_\theta g_\theta(x_r^+)v_j. \tag{54.71}$$

### Direct SSL–LSTM tangent recursion

For the state $x = (z, a, c)$ in Equation (54.1), consider first a state tangent $(\dot{z}, \dot{a}, \dot{c})$ with parameters held fixed. The four gate preactivation tangents are

$$\dot{\eta}_g = W_g \dot{z} + U_g \dot{a}, \qquad g \in \{i, f, o, r\}. \tag{54.72}$$

The gate tangents follow elementwise:

$$\dot{i} = i \odot (1 - i) \odot \dot{\eta}_i, \qquad \dot{f} = f \odot (1 - f) \odot \dot{\eta}_f, \tag{54.73}$$

$$\dot{o} = o \odot (1 - o) \odot \dot{\eta}_o, \qquad \dot{r} = (1 - r \odot r) \odot \dot{\eta}_r. \tag{54.74}$$

Differentiating the memory, hidden, and latent updates gives

$$\dot{c}^+ = \dot{f} \odot c + f \odot \dot{c} + \dot{i} \odot r + i \odot \dot{r}, \tag{54.75}$$

$$\dot{a}^+ = \dot{o} \odot \tanh(c^+) + o \odot \{1 - \tanh^2(c^+)\} \odot \dot{c}^+, \tag{54.76}$$

$$\dot{z}^+ = A\dot{a}^+. \tag{54.77}$$

Thus the state JVP is obtained using matrix–vector products and elementwise operations only. For the structural UKF, an innovation tangent is added to the first $k$ coordinates of $(\dot{z}^+, \dot{a}^+, \dot{c}^+)$. The linear observation map has

$$\dot{y}^s tar = C\dot{z}. \tag{54.78}$$

Direct parameter terms are then added exactly as in Equations (54.69) and (54.71). Separating propagated state tangents from direct parameter tangents is an implementation convention; their sum is the total directional derivative.

### Why the order changes

For the complexity ladder, $k = h = q$, the filtering-state dimension is $n = 3q$, and the UKF augments the state by the $q$-dimensional innovation. Consequently,

$$n_{\text{aug}} = n + q = 4q, \qquad R = 2n_{\text{aug}} + 1 = 8q + 1. \tag{54.79}$$

The pointwise dense state-Jacobian tensor contains

$$Rn^2 = (8q + 1)(3q)^2 = 72q^3 + 9q^2 \tag{54.80}$$

entries per time step. At $q = 20$, this is a tensor of shape $[161, 60, 60]$ with 579,600 float64 entries. Its raw size is not a complete memory model: compiler temporaries, multiple gate blocks, derivative directions, covariance derivatives, and values retained across a compiled loop can make the process high-water mark much larger.

More importantly, constructing every output–input derivative block of the LSTM and multiplying latent maps through those blocks can require $O(Rq^3)$ work per time step. Since $R = O(q)$, this is $O(q^4)$. Directly propagating $p$ tangents through Equations (54.72)– (54.77) costs $O(pRq^2)$. The Gaussian moment, covariance, solve, and principal-square-root derivative operations still cost up to $O(pq^3)$. Therefore the expected selected-direction filter order is

$$O(TpRq^2) + O(Tpq^3) = O(Tpq^3), \tag{54.81}$$

and with the current fixed $p = 4$ it is cubic rather than quartic in $q$. This is an operation-count result. Kernel launch overhead, layout, XLA fusion, and hardware utilization determine the measured crossover and must be benchmarked separately.

### Forward JVP versus reverse VJP

Forward propagation is appropriate when the number $p$ of required directions is small. That is the present ladder: it estimates four homologous coordinates even though the full chart grows as Equation (54.66). If the full parameter chart is estimated, carrying one tangent per parameter can itself become the bottleneck. For a scalar log likelihood $\mathcal{L}$, reverse mode instead propagates an output cotangent $\bar{x}^+$ through a vector–Jacobian product

$$\mathrm{VJP}_f(\bar{x}^+) = \left(D_x f_\theta^\top \bar{x}^+, D_\epsilon f_\theta^\top \bar{x}^+, D_\theta f_\theta^\top \bar{x}^+\right). \tag{54.82}$$

A full filter VJP also requires reverse rules for weighted moments, covariance updates, matrix solves, log determinants, square roots, normalized weights, resampling or fixed replay, and any transport. It normally needs checkpointing or recomputation of the forward filtering states. The practical dispatch rule is therefore:

$$\begin{aligned} \text{small selected block} &\Rightarrow \text{batched forward JVP},\\ \text{large chart and scalar target} &\Rightarrow \text{reverse VJP}. \end{aligned} \tag{54.83}$$

Matrix-free does not mean "no matrices." Covariances, factors, and small linear solves remain matrices. It means that large derivative operators are applied to the directions actually required instead of being materialized as dense Jacobians.

### Transfer across filter families

The reusable design is a local derivative contract, not one universal filter:

`transition`, `transition_jvp`, `transition_vjp`,
`observe`, `observe_jvp`, `observe_vjp`.

Each filter retains its own probability approximation and recursion.

- The structural UKF uses the transition JVP on augmented state/innovation points, then applies its existing weighted-moment, covariance, and principal-square-root derivative recursion.
- Fixed-SGQF uses the same point-map JVP on its fixed sparse quadrature cloud. Its quadrature weights, Cholesky branch, failure semantics, and likelihood remain unchanged.
- The fixed Zhao–Cui SSL–LSTM adapter carries particle state scores. Replacing a pointwise $D_x f$ multiplication by the identical JVP avoids an $[N, n, n]$ tensor while preserving fixed seeds, log-mean-exp likelihood, and weighted recentering. This is a BayesFilter `fixed_hmc_adaptation` derivative detail relative to Zhao and Cui [2024]; it is not a source-faithful SSL–LSTM claim.
- LEDH has a different dominant operator: streaming particle transport. Its appropriate matrix-free form is a chunked transport JVP or VJP rather than a sigma-point callback. The existing streaming LEDH operators already follow this pattern; Section 26.15 explains why the full scalar score favors a VJP when the parameter chart is large.

Weighted means and covariances, Cholesky or principal-root derivatives, solves, log determinants, normalized weights, and frozen-randomness replay are the common primitive layer. Point or particle clouds should be processed in chunks when their leading dimension is large, and primal/JVP calculations should be fused when doing so shortens tensor lifetimes. These implementation choices can reduce repeated work, but they do not make UKF, SGQF, Zhao–Cui, and LEDH statistically equivalent.

### Equivalence and validation requirements

A matrix-free route is admissible only if it differentiates the same executed scalar. Required checks are:

(1) dense Jacobian products versus direct JVPs on small cases;

(2) central directional finite differences;

(3) the adjoint identity $\langle \bar{y}, Jv \rangle = \langle J^\top \bar{y}, v \rangle$ whenever both JVP and VJP are implemented;

(4) end-to-end value and score parity on the same point rule, weights, factor branch, randomness, and regularization;

(5) eager/XLA parity and explicit shape checks;

(6) a guard proving the matrix-free route does not invoke the dense Jacobian builder.

Passing these checks establishes derivative equivalence for the tested fixed computation. It does not establish that the filter approximates the exact nonlinear likelihood well, that posterior sampling has converged, that NeuTra is adequate, or that the $q = 20$ ladder now fits its resource budget. Runtime and memory improvement require a separate trusted repeated benchmark, and a full-chart model requires the reverse/VJP development not completed by the selected-direction JVP.

**Observed GPU/XLA engineering evidence.** A subsequent fresh-process benchmark compared the dense and JVP forms on the identical four-direction principal-root UKF score at $q \in \{5, 10, 20\}$, using three alternating-order repetitions and five fixed warm points per process. Across all paired points, the value difference was zero and the largest score difference was $1.78 \times 10^{-15}$. At $q = 20$, the post-compile warm TensorFlow allocator peak fell from 12.614 MiB for the dense route to 4.050 MiB for the JVP route, while the cold compile/first-call peak was unchanged at 363.892 MiB. Process high-water RSS fell only from about 2.39–2.41 GiB to 2.30 GiB. Thus the JVP removes steady derivative allocation but does not explain the historical 36-GiB process report.

The median paired warm-time ratios JVP/dense were 1.895, 1.086, and 0.898 at $q = 5, 10, 20$, respectively. GPU 0 background utilization exceeded the prospective contamination threshold in at least one arm of every $q = 10$ and $q = 20$ pair. Moreover, the $q = 20$ median missed the predeclared 0.80 threshold for nominating an expensive downstream rerun. The evidence therefore supports exactness and lower warm allocation, but not a sufficiently large uncontaminated end-to-end speedup or renewed $q = 20$ NeuTra/HMC execution. No statistical-superiority, posterior, or default-readiness claim follows from three timing repetitions.

### 54.15.3 Execution and test evidence

The direct visual package ran on an NVIDIA RTX 4080 SUPER with TensorFlow 2.20.0, TensorFlow Probability 0.25.0, float64 tensors, and XLA enabled. It completed in 290.00 seconds under a 900-second cap. Its focused orientation, seed-separation, moment, and target-adapter tests reported ten passes, including five passes from the existing target-adapter suite. Every generated PNG and PDF was opened and visually inspected.

The ladder's q=1 full reference contract and dimension/fixture suite reported 12 passes. After adding the killable-subprocess supervisor, the focused dimension and timeout suite reported 12 passes with one expensive q=1 test deselected. A combined intermediate suite reported 22 passes and exposed one temporary-directory path-reporting defect in the new timeout test; that defect was repaired and the focused suite passed. Python compilation and `git diff --check` passed for both runners after their repairs. TensorFlow AutoGraph emitted Python-3.13 deprecation warnings, and XLA emitted register-spill performance warnings at the larger rungs; neither warning was treated as numerical-validity or correctness evidence.

The machine-readable receipts, source hashes, exact commands, seeds, device settings, figure hashes, failed first ladder, repaired ladder, and nonclaims are preserved below the SSL–LSTM NeuTra artifact root under `docs/plans/artifacts`. In prose, the direct package passed while confirmation remained closed; the ladder admitted q=1,2,5,10 and applied a resource veto at q=20. The exact machine status strings remain in the JSON receipts and result notes. Neither status is a posterior-correctness, HMC-readiness, NeuTra-superiority, or model-adequacy claim.

Table 54.2: Trusted GPU/XLA SSL–LSTM structural-adapter complexity ladder.

| $q$ | State | Parameters | Points | Status | Wall (s) | Peak RSS |
|---|---|---|---|---|---|---|
| 1 | 3 | 24 | 9 | passed | 18.45 | 1.39 GB |
| 2 | 6 | 64 | 17 | passed | 23.18 | 1.91 GB |
| 5 | 15 | 292 | 41 | passed | 63.40 | 3.01 GB |
| 10 | 30 | 1,032 | 81 | passed | 236.81 | 8.88 GB |
| 20 | 60 | 3,862 | 161 | resource veto | 1,291.90 | 36.17 GB |

Table 54.3: Dense and matrix-free local derivative objects for the SSL–LSTM complexity target. The dimensions suppress the time axis.

| Object | Dense route | Matrix-free route |
|---|---|---|
| Transition state derivative | $[R, n, n]$ | not materialized |
| State tangents | $[p, R, n]$ | $[p, R, n]$ |
| Transition derivative output | $[p, R, n]$ | $[p, R, n]$ |
| Observation state derivative | $[R, d_y, n]$ | not materialized |
| Covariance derivatives | $[p, n, n]$ | $[p, n, n]$ |
| Principal-root derivatives | $[p, n_{\text{aug}}, n_{\text{aug}}]$ | $[p, n_{\text{aug}}, n_{\text{aug}}]$ |

# Chapter 55

# NK SVD Case Study

The NK SVD case study is a warning label with useful engineering evidence. It shows how far a numerically hardened SVD sigma-point path can get, and it also shows why BayesFilter should not equate bounded HMC output with convergence or industrial readiness.

## 55.1 What Passed

The source reset memos record several meaningful passes:

- focused SVD, target-boundary, adjoint, and generic nonlinear SSM contracts passed during the post-refactor work;
- generic SSM SVD likelihoods and gradients compile under production XLA;
- NK SVD fixed-solution and full-solve likelihood/gradient paths compile under XLA with explicit opt-in;
- tiny NK SVD production-XLA HMC smoke tests run to completion with finite output;
- generic nonlinear SSM medium HMC writes diagnostics with finite chains and zero divergences under tuned short-run defaults.

These results are valuable sampler-usability and compilation evidence.

## 55.2 What Remains Blocked

The same reset memos record why NK convergence remains blocked. Earlier bounded stress runs produced nonfinite adjoint and lam1 inputs, SVD conditioning guard activations, nonfinite intermediate arrays, and extreme but finite transformed-theta states. The later production-XLA smoke gates do not erase that artifact history, because they are tiny runs and not clean stress experiments.

Therefore the next NK claim must be a clean stress gate, not a convergence claim. A valid clean gate requires no checked-nonfinite adjoint captures, no nonfinite lam1 inputs, no nonfinite SVD-filter captures, no unexplained transformed-theta scale explosion, and zero or separately justified divergences.

## 55.3 Lesson for BayesFilter

The case study separates four statuses:

**filter-correct** likelihood and gradients agree with controlled references;

**sampler-usable** bounded HMC runs produce finite chains and useful diagnostics;

**converged** strict multi-chain diagnostics pass on the intended target;

**production-XLA-gated** the full value-and-gradient path compiles and runs under the declared production policy.

NK SVD has meaningful production-XLA and smoke evidence, but it is not yet a converged case study. That distinction should stay visible in every future BayesFilter report.

# Chapter 56

# CIP/AFNS Case Study

The CIP/AFNS material is useful to BayesFilter because it is a large structured linear Gaussian application with economically meaningful observation equations, missing-data pressure, and analytic derivative requirements. The economics belongs in the source CIP monograph. BayesFilter should import only the filtering lessons.

## 56.1 Filtering Lessons

The reusable lessons are:

- affine or linearized macro-finance measurement systems can become large LGSSMs with structured observation matrices;
- yield-curve blocks create dense measurement panels with heterogeneous maturities and noise scales;
- missing observations must be represented by explicit masks, not by silent imputation or dynamic shape changes;
- analytic derivatives and backend parity matter because HMC mass matrices and MAP curvature depend on the same likelihood being sampled.

## 56.2 BayesFilter Boundary

BayesFilter should not re-derive AFNS economics, currency-basis theory, or asset-pricing restrictions in this monograph. It should define the state-space objects that an application supplies:

$$F_\theta, \quad Q_\theta, \quad H_\theta, \quad R_\theta, \quad m_{0|0,\theta}, \quad P_{0|0,\theta},$$

and then validate that the filter, derivative provider, masks, and HMC diagnostics behave as promised. This boundary keeps BayesFilter reusable and prevents application monographs from becoming maintenance forks of the filtering code.

## 56.3 Readiness Use

CIP/AFNS should be used as a large LGSSM readiness case once the BayesFilter package exists. The required evidence is backend parity, missing-data policy, finite analytic gradi-

ents and Hessians, MAP/mass-matrix diagnostics, and short HMC plumbing checks. It is not the right first target for SVD sigma-point HMC claims, because its main reusable strength is exact or controlled linear Gaussian filtering.

## 56.4 DZ5 Fixed-Center Curvature Evidence

A 2026-07-16 MacroFinance diagnostic shows why score stationarity and curvature stability must be separated. The foreign-binding center had prior-scaled score 0.970879, yet at radius 0.015625 three independent dense fits had positive raw curvature, negligible SPD projection, independent holdout relative error below 0.007, and trace-normalized differences near 0.1%. Small $\pm 0.01$ prior-standard-deviation center shifts preserved the geometry. This nominates a separate exact HMC mechanics canary; it does not establish a MAP or convergence.

The both-binding center had the smaller score 0.094590 but failed reproducibility. Raw nonpositive counts were $(2, 2, 1)$, projected generalized- eigenvalue spreads were approximately $4.3 \times 10^{11}$ to $1.7 \times 10^{12}$, and positive-subspace principal-angle disagreement reached about 70.6 degrees. Its projected dense matrix is blocked for HMC use; projection did not repair weak-direction uncertainty.

These are deterministic local diagnostics on one preserved center per regime and three fit seeds. They support the score-versus-curvature correction and the structured and consensus repair. They do not establish posterior validity, identification, recovery, HMC behavior, or default readiness. Applying the new BayesFilter fixed-center API to the both-binding target and running exact HMC remain separate reviewed experiments.

# Chapter 57

# NAWM-Scale Design Target

NAWM-scale models define the industrial target for BayesFilter. They should not be presented as completed evidence in this monograph pass. Instead, they set the design requirements for robust filtering and HMC on high-dimensional state-space systems.

## 57.1 Why NAWM Changes the Standard

As state dimension, parameter dimension, and observation dimension grow, several risks become routine rather than exceptional:

- covariance spectra become clustered or nearly singular;
- HMC proposals visit invalid or barely valid structural regions;
- raw autodiff through eigendecompositions or SVDs becomes fragile;
- compile boundaries and dynamic branches become performance bottlenecks;
- short smoke runs become less informative about long-chain behavior.

This is why BayesFilter should prefer analytic or custom derivative paths for production filtering wherever feasible.

## 57.2 Design Requirements

A NAWM-grade BayesFilter backend needs:

(i) a fixed, documented target contract and parameter transform;

(ii) explicit state-space shape contracts;

(iii) finite invalid-region returns and finite gradients where HMC moves;

(iv) analytic/custom gradients for exact or controlled approximate likelihoods;

(v) spectral-gap and factor diagnostics for any spectral backend;

(vi) full-pipeline JIT evidence, not only outer-wrapper tracing;

(vii) reproducible diagnostics and reset memos for every long experiment.

## 57.3 Hypothesis

The working hypothesis is that NAWM-scale HMC will be more robust with source-backed analytic or custom filtering gradients than with raw tape gradients through spectral decompositions. This is not yet a theorem or a completed benchmark. It is the prevention-first design target motivated by the SVD debugging history and by the derivative risks documented in the Phase 2B literature gate.

# Chapter 58

# Production Checklist

This checklist is the operating definition of BayesFilter production readiness. It is intentionally stricter than a successful smoke test.

## 58.1 Target and Source

Before HMC:

(i) the scalar log target is defined in writing;

(ii) every formula has provenance in source notes, local code/tests, or reviewed literature;

(iii) parameter transforms and Jacobian terms are included;

(iv) invalid structural regions return documented finite values;

(v) approximate filters are labeled as target-defining approximations or corrected surrogates.

## 58.2 Derivative and Filter

The filter backend must provide:

(i) shape, dtype, mask, and static-compilation contracts;

(ii) finite value and finite gradient checks;

(iii) derivative parity against finite differences or autodiff on controlled cases;

(iv) backend parity where multiple linear algebra forms exist;

(v) factor, solve, condition-number, and spectral-gap diagnostics;

(vi) an explicit custom-gradient policy when raw autodiff is not the final production path.

## 58.3 Sampler and JIT

The sampler must report:

(i) whether the complete value-and-gradient path is compiled;

(ii) eager/compiled parity on small targets;

(iii) mass-matrix source and conditioning;

(iv) acceptance rate, divergences, E-BFMI or equivalent energy diagnostics, R-hat, ESS, MCSE/SD, and runtime;

(v) saved JSON and chain artifacts for medium and strict runs;

(vi) sidecar capture artifacts only for bounded debugging, not for every production transition.

## 58.4 Claim Discipline

Use the narrowest honest label:

**value-valid** the likelihood value is finite and reference-checked;

**gradient-valid** the derivative matches the target value and passes parity checks;

**sampler-usable** finite HMC diagnostics justify the next experiment;

**converged** strict multi-chain diagnostics pass and artifacts are saved;

**production-ready** all preceding gates pass under the intended model, data, hardware, and compilation policy.

Anything less should stay in the reset memo as a gap, not disappear into prose.

## 58.5 Fixed-Center Geometry Checklist

Before a fixed-center precision enters HMC:

(i) use `diagnostic_center`, not `map`, unless a separate MAP gate passed;

(ii) preserve raw dense precision, eigenvalues, inertia, and projection burden before SPD regularization;

(iii) require at least two fit replicates and caller-reviewed pairwise caps;

(iv) keep training, selection holdout, and audit clouds disjoint;

(v) verify covariance/precision orientation and coordinate scaling;

(vi) reject unidentified two-factor fits and record anchor/rank diagnostics;

(vii) follow the fallback order and never manufacture identity automatically;

(viii) construct a `PrecomputedMassArtifact` only after every gate;

(ix) run a separate exact Metropolis-corrected HMC mechanics canary;

(x) exclude adaptation and tuning draws from retained inference.

For CPU/XLA cloud evaluation, record the explicit route, core-count source, worker count or override, $B = 1$, XLA flag, child PIDs, deterministic ordering, and semantic progress events. Never convert a GPU failure into automatic CPU fallback. GPU execution must verify incremental memory growth before framework initialization.

Passing this checklist only nominates an exact HMC canary. It does not establish posterior convergence or production readiness. Target-only or child-function XLA success does not establish full-chain XLA.

# Appendices

# Appendix A

# Notation and Shape Conventions

This appendix fixes the notation used throughout BayesFilter. Source projects use overlapping but not identical symbols; this monograph uses the following conventions unless a chapter states otherwise.

| Symbol | Meaning |
|---|---|
| $t = 1, \dots, T$ | observation time index |
| $x_t \in \mathbb{R}^{n_x}$ | latent state |
| $y_t \in \mathbb{R}^{n_y}$ | observation vector before masking |
| $\theta \in \mathbb{R}^{d_\theta}$ | model parameter vector |
| $u \in \mathbb{R}^{d_u}$ | unconstrained sampler coordinate |
| $c, F, Q$ | linear Gaussian transition offset, matrix, covariance |
| $a, H, R$ | linear Gaussian observation offset, matrix, covariance |
| $m_{t\|s}, P_{t\|s}$ | conditional state mean and covariance |
| $v_t, S_t$ | innovation and innovation covariance |
| $K_t$ | Kalman gain |
| $\ell(\theta)$ | filtering log likelihood |
| $g(\theta)$ | likelihood score $\nabla_\theta \ell(\theta)$ |
| $G(\theta)$ | likelihood Hessian $\nabla^2_{\theta\theta} \ell(\theta)$ |

Matrix transposes are written with $^\top$. Positive definiteness and positive semidefiniteness are written $A \succ 0$ and $A \succeq 0$. The operator $\operatorname{tr}(\cdot)$ denotes trace, and $\operatorname{diag}(\cdot)$ forms or extracts a diagonal depending on context.

## A.1 Derivative Shapes

For a parameter-dependent vector $b(\theta) \in \mathbb{R}^n$, first derivatives have shape $d_\theta \times n$ and second derivatives have shape $d_\theta \times d_\theta \times n$. For a matrix $A(\theta) \in \mathbb{R}^{m \times n}$, first derivatives have shape $d_\theta \times m \times n$ and second derivatives have shape $d_\theta \times d_\theta \times m \times n$.

This convention matches the MacroFinance derivative-provider layout used as the current implementation reference. Chapters that discuss TensorFlow or JAX execution may store tensors differently for performance, but must state the mathematical shape convention before changing memory layout.

## A.2 Label Namespace

New labels use a BayesFilter namespace:

`ch:bf-*`, `sec:bf-*`, `def:bf-*`, `asm:bf-*`, `eq:bf-*`, `tab:bf-*`.

Source-project labels should not be copied verbatim. When a source label is important for provenance, it should be recorded in the reset memo or source map.

# Appendix B

# Matrix Calculus Identities

The derivative chapters repeatedly use a small set of matrix identities. This appendix records them as proof obligations to be checked against the source derivations.

For an invertible matrix $S(\theta)$,

$$\partial_i S^{-1} = -S^{-1}(\partial_i S)S^{-1}.$$

For a symmetric positive definite matrix $S(\theta)$,

$$\partial_i \log\det S = \operatorname{tr}(S^{-1}\partial_i S).$$

For a symmetric matrix $S(\theta)$, with $q = v^\top S^{-1} v$ and $Sw = v$,

$$\partial_i q = 2(\partial_i v)^\top w - w^\top(\partial_i S)w.$$

These identities are the local algebra behind the solve-form score in Chapter 9. Second-order formulas should be expanded only with explicit source labels or proof notes because the noncommutative matrix products are easy to reorder incorrectly.

# Appendix C

# Factor Derivative Proofs

This appendix remains a bounded placeholder for full Cholesky, QR, and signed update/-downdate derivative proofs. The current monograph now distinguishes clearly between factor identities that are derived in-book, identities that are checked against source notes and tests, and primitives that are still cited or deferred rather than re-proved here. In particular, the ordered signed rank-one update/downdate recurrence used by the negative-weight SR-UKF branch is *not* yet derived here; Chapter 17 therefore treats that derivative route as conditional unless an external primitive is supplied and reconstruction is checked directly.

**Scope note.** The placeholder status applies only to the signed update/downdate primitive. The ordinary QR and Cholesky branch identities in Chapter 12 remain valid smooth-branch tools for the positive-factor lanes.

Before this appendix is promoted beyond outline form, it should include:

- the source MacroFinance labels for Cholesky derivative formulas;
- the QR perturbation equations used by the square-root Kalman backend;
- reconstruction identities connecting factor derivatives to covariance derivatives;
- code references to QR and Cholesky differentiated Kalman tests;
- MathDevMCP or manual proof-obligation results for each nontrivial step.

# Appendix D

# MathDevMCP Workflows

MathDevMCP is used as an audit assistant, not as an oracle. Its results should be recorded with their status and limitations.

Useful commands for this monograph include:

- `search-latex` to find source labels and neighborhoods;
- `latex-label-lookup` (with `extract-latex-context` kept as a compatibility alias) to record exact source provenance;
- `audit-derivation-label` or `audit-derivation-v2-label` for bounded symbolic checks;
- `audit-kalman-recursion` for AST-level implementation structure.

The current solve-form Kalman audit found the expected solve-oriented operation graph but reported missing strict log-determinant and trace classification, as well as missing explicit shape and covariance guards. The correct response is to document those gaps and use the MacroFinance parity tests as additional evidence, not to relabel the audit as a proof.

# Appendix E

# ResearchAssistant Workflows

ResearchAssistant is the literature-ingestion sidecar for this monograph. It should be used when a chapter needs primary-source support rather than local implementation provenance. It is not a replacement for reading the source and checking whether the cited paper supports the exact BayesFilter claim.

## E.1 Workspace Policy

Use a persistent, ignored workspace under the BayesFilter tree:

`/home/chakwong/BayesFilter/.research/ra-bayesfilter-monograph`.

Downloaded papers, source packages, extracted text, and inbox files stay out of Git. Durable decisions belong in 'docs/plans' result notes, the reset memo, and the source map.

## E.2 Ingestion Workflow

The standard workflow is:

(i) search or fetch the primary source;

(ii) verify metadata against title, authors, venue, DOI or arXiv id;

(iii) reject mismatches explicitly;

(iv) extract the citation neighborhood or source labels relevant to the BayesFilter claim;

(v) write a short result note stating supported claims and limits;

(vi) only then add or use a bibliography entry.

The Phase 2B result note is the model: it accepted exact arXiv source packages for HMC, NUTS, NeuTra, Matrix Backprop, Kitagawa score/Hessian material, and the differentiable particle-filter frontier, while rejecting several metadata mismatches.

## E.3 Claim Gate

A literature-heavy paragraph is draftable only when one of the following is true:

- an accepted ResearchAssistant note supports the claim and records its limits;
- a local source document plus code/test audit supports the claim;
- the paragraph is explicitly labeled as a project decision or an open hypothesis.

Raw '.bib' entries, downloaded PDFs, and search results are not claim support.

## E.4 Remaining Source-Support Blockers

The remaining literature-support blockers are narrower than they were in the earlier draft. Detailed sigma-point and PMCMC support are now cited in the body of the monograph, but several load-bearing gaps remain: exact-source support for the remaining HAC theorem claim in the SSL–LSTM case study, a durable local copy and exact bibliographic resolution for the Li–Coates particle-flow source, and any further source-faithfulness audit needed before unresolved survey claims are strengthened. Until those gaps are closed, the monograph should keep explicit nonclaims where a cited source has not been inspected at theorem or algorithm level. Live bibliography records in the replacement candidate should therefore avoid provisional notes and should either be verified, replaced by inspected sources, or removed from claim-bearing passages.

# Appendix F

# Source Map

The machine-readable source map is

`docs/source_map.yml`.

It records which source project, chapter, reset memo, code file, or test file supports each BayesFilter chapter. The source map is part of the monograph's quality-control system, not merely an index.

Each imported item should move through explicit states:

- `candidate`: relevant but not yet drafted;
- `drafted`: consolidated into BayesFilter prose;
- `audited`: checked against source math, code, tests, or reviewed literature;
- `superseded`: replaced by another source;
- `excluded`: intentionally outside BayesFilter scope.

The source map also records duplicate source groups. For example, Kalman likelihood material appears in the DSGE monograph, CIP monograph, and the MacroFinance analytic derivative note. BayesFilter treats the MacroFinance note and associated tests as the derivative spine while using the monographs as expository sources.

Generated artifacts, paper caches, and local PDFs are not provenance by themselves. A claim becomes monograph-ready only when the relevant source or test has been summarized in a committed note, mapped in the source map, or explicitly cited in the chapter.

The dated fixed-center-curvature source-map entry links the score-curvature, factor-covariance, cloud-size, and consensus equations to the BayesFilter implementation/tests, the reviewed plan, and the MacroFinance DZ5 result/reset evidence. The map distinguishes implemented common algorithms from the still-pending application-specific exact HMC canary and retained estimation.

# Appendix G

# Experiment Templates

This appendix gives reusable experiment templates for future BayesFilter implementation work. The templates are intentionally short. Each experiment should write a reset-memo entry with commands, artifacts, interpretation, and the next justified phase.

## G.1 Filter Reference Template

Use this template before HMC:

(i) define the model, parameter transform, data seed, and fixed structural parameters;

(ii) compute a reference likelihood from an independent exact, dense, or trusted implementation;

(iii) compare the BayesFilter backend value to the reference;

(iv) compare gradients by finite differences or small-model autodiff;

(v) record finite-value, factor, solve, mask, and failure-code diagnostics;

(vi) save the command and tolerance rationale.

Passing this template earns value-valid or gradient-valid status, depending on which checks are included. It does not earn sampler-usability status.

## G.2 HMC Smoke Template

Use this template to test sampler plumbing:

(i) use a small model and static shapes;

(ii) run one or two short chains with deliberately small draw counts;

(iii) require finite chains, finite log targets, and no unhandled exceptions;

(iv) record acceptance rate, divergence count, step size, runtime, and whether the complete value-gradient path was compiled;

(v) state explicitly that the run is not convergence evidence.

Smoke runs justify the next experiment only when the failure taxonomy is clean.

## G.3 Medium Recovery Template

Use this template for controlled posterior recovery:

(i) run at least four chains when feasible;

(ii) save JSON diagnostics and NPZ chain artifacts;

(iii) report R-hat, bulk ESS, tail ESS, MCSE/SD, divergences, acceptance, and runtime;

(iv) compare posterior intervals to synthetic truth or a trusted reference;

(v) classify the result as sampler-usable or not sampler-usable.

Medium recovery is allowed to have looser thresholds than strict convergence, but the looseness must be stated in the result note.

## G.4 Strict Convergence Template

Use this template only after the preceding gates pass:

(i) run at least four chains with planned warmup and draw counts;

(ii) require finite log targets and finite gradients throughout accepted trajectories;

(iii) require zero divergences unless a bounded exception is justified;

(iv) require the planned R-hat, ESS, and MCSE/SD thresholds;

(v) audit saved artifacts before writing a convergence claim.

If any required diagnostic fails, the reset memo should name the blocker and the next diagnostic hypothesis instead of weakening the claim label.

# Bibliography


B. K. Abid and R. M. Gower. Greedy stochastic algorithms for entropy-regularized optimal transport problems. In *Advances in Neural Information Processing Systems*, 2018. arXiv:1803.01347.

William Acevedo, Jana de Wiljes, and Sebastian Reich. Second-order ensemble transform particle filters. *SIAM/ASA Journal on Uncertainty Quantification*, 2017.

Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. The conjugate unscented transform: An approach to evaluate multi-dimensional expectation integrals. In *2012 American Control Conference*, pages 5556–5561, 2012a. Article 6314970.

Nagavenkat Adurthi, Puneet Singla, and Tarunraj Singh. Conjugate unscented transform and its application to filtering and stochastic integral calculation. In *AIAA Guidance, Navigation, and Control Conference 2012*, 2012b. AIAA Guidance, Navigation, and Control Conference, Minneapolis, Minnesota.

Hadi Mohasel Afshar and Justin Domke. Reflection, refraction, and Hamiltonian Monte Carlo. In *Advances in Neural Information Processing Systems*, volume 28, 2015. URL https://papers.nips.cc/paper_files/paper/2015/hash/8303a79b1e19a194f1875981be5bdb6f-Abstract.html.

Mokhtar Z. Alaya, Maxime Bérar, Gilles Gasso, and Alain Rakotomamonjy. Screening sinkhorn algorithm for regularized optimal transport. In *Advances in Neural Information Processing Systems*, 2019.

Jason Altschuler, Jonathan Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via sinkhorn iteration. In *Advances in Neural Information Processing Systems*, 2017.

Jason Altschuler, Francis Bach, Alessandro Rudi, and Jonathan Weed. Massively scalable sinkhorn distances via the nystrom method. In *Advances in Neural Information Processing Systems*, 2019. arXiv:1812.05189.

Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pages 146–155, 2017.

Sungbae An and Frank Schorfheide. Bayesian analysis of dsge models. *Econometric Reviews*, 26(2–4):113–172, 2007.

Daniel Andrade. Stable training of normalizing flows for high-dimensional variational inference. arXiv:2402.16408v1, 2024. URL https://arxiv.org/abs/2402.16408v1.

Martin M. Andreasen, Jesús Fernández-Villaverde, and Juan F. Rubio-Ramírez. The pruned state-space system for non-linear dsge models: Theory and empirical applications. *Review of Economic Studies*, 85(1):1–49, 2018.

Christophe Andrieu and Gareth O. Roberts. The pseudo-marginal approach for efficient monte carlo computations. *The Annals of Statistics*, 37(2):697–725, 2009.

Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. *Journal of the Royal Statistical Society: Series B*, 72(3):269–342, 2010.

Michael Arbel, Alex Matthews, and Arnaud Doucet. Annealed flow transport monte carlo. In *Proceedings of ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 318–330, 2021. URL https://proceedings.mlr.press/v139/arbel21a.html.

Oleg Arenz, Philipp Dahlinger, Zihan Ye, Michael Volpp, and Gerhard Neumann. A unified perspective on natural gradient variational inference with Gaussian mixture models. *Journal of Machine Learning Research*, 24, 2023. URL https://arxiv.org/abs/2209.11533.

M. Sanjeev Arulampalam, Simon Maskell, Neil Gordon, and Tim Clapp. A tutorial on particle filters for online nonlinear/non-gaussian bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, 2002. doi: 10.1109/78.978374.

Thomas Bengtsson, Peter Bickel, and Bo Li. Curse-of-dimensionality revisited: Collapse of the particle filter in very large scale systems. *Probability and Statistics: Essays in Honor of David A. Freedman*, pages 316–334, 2008.

Michael Betancourt. A conceptual introduction to hamiltonian monte carlo. *arXiv preprint arXiv:1701.02434*, 2017.

Denis Blessing, Xiaogang Jia, Johannes Esslinger, Francisco Vargas, and Gerhard Neumann. Beyond ELBOs: A large-scale evaluation of variational methods for sampling. In *Proceedings of ICML*, 2024. URL https://arxiv.org/abs/2406.07423.

Olivier Cappé, Randal Douc, Arnaud Guillin, Jean-Michel Marin, and Christian P. Robert. Adaptive importance sampling in general mixture classes. *Statistics and Computing*, 18:447–459, 2008. URL https://arxiv.org/abs/0710.4242.

Benjamin Charlier, Jean Feydy, Joan-Alexis Glaunès, François-David Collin, and Ghislain Durif. Kernel operations on the GPU, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021.

Sourav Chatterjee and Persi Diaconis. The sample size required in importance sampling. *Annals of Applied Probability*, 28(2):1099–1135, 2018. URL https://arxiv.org/abs/1511.01437.

Junhua Chen, Lorenz Richter, Julius Berner, Denis Blessing, Gerhard Neumann, and Anima Anandkumar. Sequential controlled langevin diffusions. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2412.07081.

Peng Chen and Omar Ghattas. Projected stein variational gradient descent. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/file/14faf969228fc18fcd4fcf59437b0c97-Paper.pdf.

Peter Chen, Yue Xie, and Qingpeng Zhang. SICNN: Sparsity-induced input convex neural network for optimal transport. *NeurIPS 2024 Workshop on Optimal Transport and Machine Learning*, 2024.

Nicolas Chopin and Omiros Papaspiliopoulos. *An Introduction to Sequential Monte Carlo*. Springer Series in Statistics. Springer, 2020. doi: 10.1007/978-3-030-47845-2.

Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport. In *Proceedings of the 38th International Conference on Machine Learning*, pages 2100–2111, 2021.

Tiangang Cui and Sergey Dolgov. Deep composition of tensor-trains using squared inverse rosenblatt transports. *Foundations of Computational Mathematics*, 22(6):1863–1922, 2022.

Tiangang Cui, Sergey Dolgov, and Olivier Zahm. Self-reinforced polynomial approximation methods for concentrated probability densities. *arXiv preprint arXiv:2303.02554*, 2023.

Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013.

Fred Daum and Jim Huang. Nonlinear filters with particle flow. In *Signal Processing, Sensor Fusion, and Target Recognition XVII*, 2008.

M. H. A. Davis. On a multiplicative functional transformation arising in nonlinear filtering theory. *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete*, 54:125–139, 1980.

Pierre Del Moral, Arnaud Doucet, and Sumeetpal S. Singh. Uniform stability of a particle approximation of the optimal filter derivative. *SIAM Journal on Control and Optimization*, 53(3):1278–1304, 2015. doi: 10.1137/140993703.

Bernard Delyon and François Portier. Safe and adaptive importance sampling: A mixture approach. *Annals of Statistics*, 49(2):885–917, 2021. URL https://arxiv.org/abs/1903.08507.

Ishan Deshpande, Yuting Hu, Ruoyu Sun, Alexandros Pyrros, Nadeem Siddiqui, Sanmi Koyejo, Zheng Zhao, David Forsyth, and Alexander G. Schwing. Max-sliced wasserstein distance and its use for GANs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.

Somayajulu L. N. Dhulipala, Yifeng Che, and Michael D. Shields. Bayesian inference with latent hamiltonian neural networks. *arXiv preprint arXiv:2208.06120*, 2022. Version 2, 24 October 2022; official implementation archived at https://github.com/IdahoLabResearch/BIhNNs.

Arnaud Doucet, Nando de Freitas, and Neil Gordon, editors. *Sequential Monte Carlo Methods in Practice*. Springer, 2001.

J. Durbin and S. J. Koopman. *Time Series Analysis by State Space Methods*. Oxford University Press, 2012.

Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows. arXiv:1906.04032v2, 2019. URL https://arxiv.org/abs/1906.04032v2.

Pavel Dvurechensky, Alexander Gasnikov, and Alexey Kroshnin. Computational optimal transport: Complexity by accelerated gradient descent is better than by sinkhorn's algorithm. In *Proceedings of the 35th International Conference on Machine Learning*, 2018.

Deanna C. Easley and Tyrus Berry. A higher order unscented transform. *SIAM/ASA Journal on Uncertainty Quantification*, 9(4):1603–1633, 2021. doi: 10.1137/20M135546X.

Donald Ebeigbe, Tyrus Berry, Andrew J. Whalen, Michael M. Norton, Dan Simon, Timothy D. Sauer, and Steven J. Schiff. Generalized unscented transformation for forecasting non-gaussian processes. *Physical Review E*, 111:054135, 2025. doi: 10.1103/PhysRevE.111.054135.

Víctor Elvira, Émilie Chouzenoux, Ómer Deniz Akyildiz, and Luca Martino. Gradient-based adaptive importance samplers. *Journal of the Franklin Institute*, 2023. URL https://arxiv.org/abs/2210.10785.

Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun-ichi Amari, Alain Trouvé, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, 2019.

James M. Flegal and Lei Gong. Relative fixed-width stopping rules for Markov Chain Monte Carlo simulations, 2013. Locally inspected preprint, sections 2–3.1 and Appendix A.

James M. Flegal and Galin L. Jones. Batch means and spectral variance estimators in Markov Chain Monte Carlo. *The Annals of Statistics*, 38(2):1034–1070, 2010. doi: 10.1214/09-AOS735.

Alexander Forrow, Jan-Christian Hütter, Matan Nitzan, Philippe Rigollet, Geoffrey Schiebinger, and Jonathan Weed. Statistical optimal transport via factored couplings. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, 2019.

Axel Gandy and James A. Scott. Unit testing for MCMC and other Monte Carlo methods. arXiv:2001.06465v2, 2021. URL https://arxiv.org/abs/2001.06465. Inspected Algorithms 1–3, Proposition 2.3, Theorem 3.1, Appendices A–B and the mcunit implementation.

Aude Genevay, Marco Cuturi, Gabriel Peyré, and Francis Bach. Stochastic optimization for large-scale optimal transport. In *Advances in Neural Information Processing Systems*, 2016.

Aude Genevay, Lenaic Chizat, Francis Bach, Marco Cuturi, and Gabriel Peyré. Sample complexity of sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, 2019. arXiv:1810.02733.

Jonathan Geuter, Gregor Kornhardt, Ingimar Tomasson, and Vaios Laschos. Universal neural optimal transport. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pages 17009–17057, 2025.

Inga Girshfeld and Xiaohui Chen. Neural local wasserstein regression. *arXiv preprint arXiv:2511.10824v1*, 2025.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. doi: 10.1198/016214506000001437.

Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. Novel approach to nonlinear/non-gaussian bayesian state estimation. *IEE Proceedings F: Radar and Signal Processing*, 140(2):107–113, 1993.

Maria I. Gorinova, Dave Moore, and Matthew D. Hoffman. Automatic reparameterisation of probabilistic programs. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3648–3657. PMLR, 2020. URL https://proceedings.mlr.press/v119/gorinova20a.html.

Andrew Gracyk and Xiaohui Chen. GeONet: A neural operator for learning the wasserstein geodesic. *Proceedings of the 40th Conference on Uncertainty in Artificial Intelligence*, 2024.

Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13: 723–773, 2012.

Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 15379–15389, 2019.

Uwe D. Hanebeck. Progressive bayesian particle flows based on optimal transport map sequences. *arXiv preprint arXiv:2303.02412*, 2023.

Lars Peter Hansen. Large sample properties of generalized method of moments estimators. *Econometrica*, 50(4):1029–1054, 1982.

Edward P. Herbst and Frank Schorfheide. *Bayesian Estimation of DSGE Models*. Princeton University Press, 2015.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2 edition, 2002.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15: 1593–1623, 2014.

Matthew D. Hoffman, Pavel Sountsov, Joshua V. Dillon, Ian Langmore, Dustin Tran, and Srinivas Vasudevan. Neutra-lizing bad geometry in Hamiltonian Monte Carlo using neural transport. *arXiv preprint arXiv:1903.03704*, 2019. Presented at the Symposium on Advances in Approximate Bayesian Inference.

Lennart Hoogerheide, Anne Opschoor, and Herman K. van Dijk. A class of adaptive importance sampling weighted EM algorithms for efficient and robust posterior and predictive simulation. *Journal of Econometrics*, 171(2):101–120, 2012. Technical material also inspected in the 2011 working paper, A Class of Adaptive EM-based Importance Sampling Algorithms for Efficient and Robust Posterior and Predictive Simulation.

Chih-Chi Hu and Peter Jan van Leeuwen. A particle flow filter for high-dimensional system applications. *Quarterly Journal of the Royal Meteorological Society*, 147(739): 2358–2377, 2021.

Chin-Wei Huang, David Krueger, Alexandre Lacoste, and Aaron Courville. Neural autoregressive flows. arXiv:1804.00779v1, 2018. URL https://arxiv.org/abs/1804.00779v1.

Catalin Ionescu, Orestis Vantzos, and Cristian Sminchisescu. Training deep networks with structured layers by matrix backpropagation. *arXiv preprint arXiv:1509.07838*, 2015. doi: 10.48550/arXiv.1509.07838.

Mark E. Johnson and Victor W. Lowe. Bounds on the sample skewness and kurtosis. *Technometrics*, 21(3):377–378, 1979. doi: 10.1080/00401706.1979.10489785.

Rico Jonschkowski, Deepak Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors. In *Robotics: Science and Systems*, 2018.

Simon J. Julier and Jeffrey K. Uhlmann. A general method for approximating nonlinear transformations of probability distributions. Technical report, Robotics Research Group, Department of Engineering Science, University of Oxford, 1996. 1 November 1996.

Simon J. Julier and Jeffrey K. Uhlmann. A new extension of the kalman filter to nonlinear systems. *Proceedings of AeroSense*, 1997.

Peter Karkus, David Hsu, and Wee Sun Lee. Particle filter networks with application to visual localization. In *Conference on Robot Learning*, 2018.

Jinill Kim, Sunghyun Kim, Ernst Schaumburg, and Christopher A. Sims. Calculating and using second-order accurate solutions of discrete time dynamic equilibrium models. *Journal of Economic Dynamics and Control*, 32(11):3397–3414, 2008.

Diederik P. Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible $1 \times 1$ convolutions. arXiv:1807.03039v2, 2018. URL https://arxiv.org/abs/1807.03039v2.

Genshiro Kitagawa. Monte carlo filter and smoother for non-gaussian nonlinear state space models. *Journal of Computational and Graphical Statistics*, 5(1):1–25, 1996. doi: 10.1080/10618600.1996.10474692.

Mike Klaas, Nando de Freitas, and Arnaud Doucet. Toward practical $n^2$ monte carlo: The marginal particle filter. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence*, pages 308–315, 2005. URL https://www.cs.ubc.ca/~arnaud/klass_defreitas_doucet_marginalparticlefilterUAI2005.pdf.

Soheil Kolouri, Khashayar Nadjahi, Umut Simsekli, Roland Badeau, and Gustavo K. Rohde. Generalized sliced wasserstein distances. In *Advances in Neural Information Processing Systems*, 2019.

Anna Korba and François Portier. Adaptive importance sampling meets mirror descent: A bias-variance tradeoff. In *Proceedings of AISTATS*, volume 151 of *Proceedings of Machine Learning Research*, 2022. URL https://proceedings.mlr.press/v151/korba22a.html.

Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

Sijing Li, Zhongjian Wang, Stephen S.-T. Yau, and Zhiwen Zhang. Solving high-dimensional nonlinear filtering problems using a tensor train decomposition method. *arXiv preprint arXiv:1908.04010*, 2019.

Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116, 2017.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

Tianyi Lin, Nhat Ho, and Michael I. Jordan. On efficient optimal transport: An analysis of greedy and accelerated mirror descent algorithms. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

Hailiang Liu and Yan-Han Chen. Neural solver for wasserstein geodesics and optimal transport dynamics. *arXiv preprint arXiv:2602.22003*, 2026.

Qiang Liu and Dilin Wang. Stein variational gradient descent: A general purpose bayesian inference algorithm. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://papers.neurips.cc/paper_files/paper/2016/file/b3ba8f1bee1238a2f37603d90b58898d-Paper.pdf.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, 2021. doi: 10.1038/s42256-021-00302-5.

Ashok Vardhan Makkuva, Amirhossein Taghvaei, Jason D. Lee, and Sewoong Oh. Optimal transport mapping via input convex neural networks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 6672–6681, 2020.

Donald W. Marquardt. An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):431–441, 1963. doi: 10.1137/0111030.

Alexander G. D. G. Matthews, Michael Arbel, Danilo J. Rezende, and Arnaud Doucet. Continual repeated annealed flow transport monte carlo. In *Proceedings of ICML*, volume 162 of *Proceedings of Machine Learning Research*, 2022. URL https://proceedings.mlr.press/v162/matthews22a.html.

Yuhua Meng, Zhongjian Wang, Stephen S.-T. Yau, and Zhiwen Zhang. Regularity estimate and sparse approximation of pathwise robust duncan-mortensen-zakai equation. *arXiv preprint arXiv:2509.19093*, 2025.

Alessandro Micheli, Yueqi Cao, Anthea Monod, and Samir Bhatt. Riemannian neural optimal transport. *arXiv preprint arXiv:2602.03566v1*, 2026.

Laurence I. Midgley, Vincent Stimper, Gregor N. C. Simm, Bernhard Schölkopf, and José Miguel Hernández-Lobato. Flow annealed importance sampling bootstrap. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2208.01893.

Martin Modrák, Angie H. Moon, Shinyoung Kim, Paul Bürkner, Niko Huurre, Kateřina Faltejsková, Andrew Gelman, and Aki Vehtari. Simulation-based calibration checking for bayesian computation: The choice of test quantities shapes sensitivity. arXiv:2211.02383v3, 2023. URL https://arxiv.org/abs/2211.02383. Inspected data-dependent test quantities, ignored-data example and theory.

Radford M. Neal. Annealed importance sampling. *Statistics and Computing*, 11:125–139, 2001. URL https://arxiv.org/abs/physics/9803008.

Radford M. Neal. Mcmc using hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.

Whitney K. Newey and Kenneth D. West. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica*, 55(3):703–708, 1987. doi: 10.2307/1913610.

Khai Nguyen. An introduction to sliced optimal transport. *arXiv preprint arXiv:2508.12519*, 2025.

Khai Nguyen, Duy Nguyen, Tung Pham, Nhat Ho, and Hung Bui. Improving mini-batch optimal transport via partial transportation. *arXiv preprint arXiv:2108.09645*, 2021.

Khai Nguyen, Nhat Ho, Tung Pham, and Hung Bui. On transportation of mini-batches: A hierarchical approach. In *Proceedings of the 39th International Conference on Machine Learning*, 2022.

M. R. Osborne. Nonlinear least squares—the levenberg algorithm revisited. *The ANZIAM Journal*, 19(3):343–357, 1976. doi: 10.1017/S033427000000120X.

Art B. Owen. *Monte Carlo Theory, Methods and Examples*. Author's online manuscript, 2013. URL https://artowen.su.domains/mc/. Chapter 9: Importance sampling.

Ari Pakman and Liam Paninski. Exact Hamiltonian Monte Carlo for truncated multivariate Gaussians. *Journal of Computational and Graphical Statistics*, 23(2):518–542, 2014. doi: 10.1080/10618600.2013.788448. URL https://arxiv.org/abs/1208.4118.

Matthew D. Parno and Youssef M. Marzouk. Transport map accelerated Markov chain Monte Carlo. *SIAM/ASA Journal on Uncertainty Quantification*, 6(2):645–682, 2018. doi: 10.1137/17M1134640.

François-Pierre Paty and Marco Cuturi. Subspace robust wasserstein distances. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.

Gabriel Peyré and Marco Cuturi. Computational optimal transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019.

Emilia Pompe, Chris Holmes, and Krzysztof Łatuszyński. A framework for adaptive MCMC targeting multimodal distributions. *Annals of Statistics*, 48(5):2930–2952, 2020. URL https://arxiv.org/abs/1812.02609.

Adrian E. Raftery and Le Bao. Estimating and projecting trends in HIV/AIDS generalized epidemics using incremental mixture importance sampling. *Biometrics*, 66(4): 1162–1173, 2010. doi: 10.1111/j.1541-0420.2010.01399.x.

Sebastian Reich. A nonparametric ensemble transform method for bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.

Geoffrey Roeder, Yuhuai Wu, and David Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. arXiv:1703.09194v3, 2017. URL https://arxiv.org/abs/1703.09194v3.

Meyer Scetbon and Marco Cuturi. Linear time sinkhorn divergences using positive features. In *Advances in Neural Information Processing Systems*, 2020.

Meyer Scetbon and Marco Cuturi. Low-rank optimal transport: Approximation, statistics and debiasing. *arXiv preprint arXiv:2205.12365*, 2022.

Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Meyer Scetbon, Michal Klein, Gergely Palla, and Marco Cuturi. Unbalanced low-rank optimal transport solvers. *arXiv preprint arXiv:2305.19727*, 2023.

Bernhard Schmitzer. A sparse multi-scale algorithm for dense optimal transport. *Journal of Mathematical Imaging and Vision*, 56:238–259, 2016.

Bernhard Schmitzer. Stabilized sparse scaling algorithms for entropy regularized transport problems. *SIAM Journal on Scientific Computing*, 41(3):A1443–A1481, 2019.

Chris Snyder, Thomas Bengtsson, Peter Bickel, and Jeffrey Anderson. Obstacles to high-dimensional particle filtering. *Monthly Weather Review*, 136(12):4629–4640, 2008.

Alessio Spantini, Ricardo Baptista, and Youssef Marzouk. Coupling techniques for nonlinear ensemble filtering. *SIAM/ASA Journal on Uncertainty Quantification*, 10(3): 921–953, 2022. doi: 10.1137/20M1312204.

Bharath K. Sriperumbudur, Arthur Gretton, Kenji Fukumizu, Bernhard Schölkopf, and Gert R. G. Lanckriet. Hilbert space embeddings and metrics on probability measures. *Journal of Machine Learning Research*, 11:1517–1561, 2010.

Sean Talts, Michael Betancourt, Daniel Simpson, Aki Vehtari, and Andrew Gelman. Validating bayesian inference algorithms with simulation-based calibration. Author manuscript, 2020. URL https://www.stat.columbia.edu/~gelman/research/unpublished/sbc.pdf. Author-hosted manuscript; inspected rank theorem and dependence discussion.

Théo Uscidda and Marco Cuturi. The monge gap: A regularizer to learn all transport maps. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 35131–35160, 2023.

Lorenz Vaitl, Kim A. Nicoli, Shinichi Nakajima, and Pan Kessel. Gradients should stay on path: Better estimators of the reverse- and forward KL divergence for normalizing flows. arXiv:2207.08219v1, 2022. URL https://arxiv.org/abs/2207.08219v1.

Lorenz Vaitl, Ludwig Winkler, Lorenz Richter, and Pan Kessel. Fast and unified path gradient estimators for normalizing flows. ICLR 2024; archived arXiv:2403.15881v1, 2024. URL https://arxiv.org/abs/2403.15881v1.

Rudolph van der Merwe. *Sigma-Point Kalman Filters for Probabilistic Inference in Dynamic State-Space Models*. PhD thesis, OGI School of Science and Engineering at Oregon Health and Science University, 2004.

Ramon van Handel. Stochastic calculus, filtering, and stochastic control, 2007. ACM 217 lecture notes.

Matthew Vandergrift, Martha White, Yury Polyanskiy, Philippe Rigollet, and Lazar Atanackovic. Measure-to-measure regression with transformers. *arXiv preprint arXiv:2605.28075v1*, 2026.

Dootika Vats and James M. Flegal. Lugsail lag windows for estimating time-average covariance matrices, 2021. Version 3, sections 4–5 and equation (7).

Aki Vehtari, Andrew Gelman, Daniel Simpson, Bob Carpenter, and Paul-Christian Bürkner. Rank-normalization, folding, and localization: An improved $\widehat{R}$ for assessing convergence of mcmc. *Bayesian Analysis*, 16(2):667–718, 2021.

Nina Vesseron and Marco Cuturi. On a neural implementation of brenier's polar factorization. *Proceedings of the 41st International Conference on Machine Learning*, 235: 52713–52750, 2024.

Cédric Villani. *Topics in Optimal Transportation*, volume 58 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 2003.

Mike West. Approximating posterior distributions by mixtures. *Journal of the Royal Statistical Society: Series B*, 55(2):409–422, 1993.

Dongze Wu and Yao Xie. Annealing flow generative models towards sampling high-dimensional and multi-modal distributions. In *Proceedings of ICML*, volume 267 of *Proceedings of Machine Learning Research*, 2025. URL https://proceedings.mlr.press/v267/wu25ak.html.

H. Xiao. Fast log-domain sinkhorn optimal transport with warp-level GPU reductions. *arXiv preprint arXiv:2605.00837*, 2026.

Chen Xu, Xiuyuan Cheng, and Yao Xie. Computing high-dimensional optimal transport by flow neural networks. *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, 258:5413–5421, 2025.

Z. Xu and L. Chen. Accelerating sinkhorn for entropy-regularized optimal transport. *arXiv preprint arXiv:2605.30267*, 2026.

Shing-Tung Yau and Stephen S.-T. Yau. Real time solution of nonlinear filtering problem without memory i. *Mathematical Research Letters*, 7:671–693, 2000.

Shing-Tung Yau and Stephen S.-T. Yau. Real time solution of the nonlinear filtering problem without memory ii. *SIAM Journal on Control and Optimization*, 47(1):163–195, 2008. doi: 10.1137/050648353.

F. X.-F. Ye, X. Li, A. Yu, M.-C. Chang, L. Chu, and D. Wertheimer. FlashSinkhorn: Io-aware entropic optimal transport on GPU. *arXiv preprint arXiv:2602.03067*, 2026.

Yiran Zhao and Tiangang Cui. Tensor-train methods for sequential state and parameter learning in state-space models. *Journal of Machine Learning Research*, 25:1–51, 2024.

Meixin Zhu, Kevin Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.